Higher-Order Logic Programming for `(aux kanren micro)`

A reading of Miller & Nadathur's "Programming with Higher-Order Logic" and of λKanren, with a proposal for `(aux kanren lambda)`

Prepared for Massimo Nocentini, with Claude (Anthropic)

October 2026

`(aux kanren micro)` is a small, fast miniKanren whose terms are first-order Scheme data and whose goals are first-order Horn clauses. λProlog, the language of Miller and Nadathur's *Programming with Higher-Order Logic*, runs a much larger logic: goals may say "for every new name x" and "assuming D", terms may contain λ-binders, and unification works modulo α, β and η, so that programs over syntax with binders need no code for renaming, fresh names or capture-avoiding substitution. This document asks what it would take to give `(aux kanren micro)` the same power. Part I summarizes the book chapter by chapter, with its core examples; Part II surveys the miniKanren literature on these features, centered on Ma and Friedman's higher-order unification algorithm for λKanren; Part III describes the engine as it is today and proposes a concrete answer: a generic extension protocol for the engine and a client module, `(aux kanren lambda)`, providing λ-terms, the goals ∀ and ⊃, and higher-order pattern unification with universe levels and delayed non-patterns, prototyped, tested and benchmarked.

Contents

1 Introduction **13**
- 1.1 What this document is for . . . 13
- 1.2 The whole story in brief . . . 14
  - 1.2.1 The program, twice . . . 14
  - 1.2.2 Running it forwards . . . 15
  - 1.2.3 Running it backwards . . . 16
  - 1.2.4 What the engine had to learn . . . 17
- 1.3 Map of the document . . . 19
- 1.4 Reading paths . . . 21
- 1.5 Conventions . . . 22
- 1.6 Summary . . . 24

I The book: Programming with Higher-Order Logic **25**

2 Logic programming as proof search **26**
- 2.1 Computation as deduction . . . 26
- 2.2 Typed first-order terms . . . 27
  - 2.2.1 Sorts, type constructors and kinds . . . 27
  - 2.2.2 Terms, polymorphism and typing . . . 28
  - 2.2.3 Representing data . . . 29
- 2.3 First-order unification . . . 31
- 2.4 First-order formulas . . . 35
- 2.5 The search semantics of goals . . . 36
- 2.6 Horn clauses and backchaining . . . 37
- 2.7 λProlog concrete syntax and interaction . . . 39
- 2.8 Core examples . . . 41
  - 2.8.1 Search: finite-state machines . . . 41
  - 2.8.2 Recursive data: insertion into a search tree . . . 42
  - 2.8.3 Abstract syntax: a sequent prover . . . 43
- 2.9 Pragmatics of the search . . . 45
- 2.10 Logical notions: cut and presentations of fohc . . . 46
- 2.11 The role of types . . . 47
- 2.12 From fohc to `(aux kanren micro)` . . . 49
  - 2.12.1 Depth-first search versus interleaving . . . 51
- 2.13 Summary . . . 53

3 Hereditary Harrop formulas: implication and universal goals **54**
3.1 Syntax: goals, clauses and polarity . . . 54
3.1.1 Positive and negative occurrences . . . 55
3.1.2 Clausal order . . . 56
3.2 Operational semantics: AUGMENT and GENERIC . . . 56
3.3 Implicational goals and hypothetical reasoning . . . 58
3.3.1 Entailment between Horn clauses . . . 58
3.3.2 Hypothetical reasoning on a database . . . 59
3.3.3 An interactive "what if" database . . . 61
3.4 Universal goals and scoped constants . . . 62
3.4.1 An intensional universal quantifier . . . 63
3.4.2 Substitution must avoid capture . . . 64
3.4.3 Quantifier order matters . . . 64
3.5 Quantification links goals and clauses . . . 66
3.6 The logic behind fohh . . . 68
3.6.1 Not classical logic . . . 68
3.6.2 Minimal and intuitionistic negation . . . 69
3.6.3 Why "hereditary Harrop" . . . 71
3.7 Notable subsets of fohh . . . 71
3.8 What implication and universal goals ask of a miniKanren . . . 72
3.8.1 The program context belongs to the search state . . . 73
3.8.2 Scoped constants need a scope check . . . 74
3.8.3 A throwaway interpreter of the rules . . . 75
3.9 Summary . . . 78

4 Typed λ-terms and unification problems **80**
4.1 Syntax and types of λ-terms . . . 80
4.1.1 Formulas are terms of type o . . . 82
4.2 λ-conversion: the equality of the logic . . . 82
4.3 Normal forms and the power of β . . . 84
4.3.1 Computing with β: Church numerals . . . 85
4.4 Unification problems as quantified equalities . . . 88
4.5 Simplifying prefixes: raising . . . 91
4.6 Unifiers, solutions and empty types . . . 93
4.7 Example problems and their solutions . . . 93
4.7.1 Quantifier order changes the answers . . . 94
4.7.2 Words over a two-letter alphabet . . . 95
4.7.3 Weak functionality . . . 96
4.8 Vocabulary for a unification procedure . . . 96
4.9 Undecidability . . . 98
4.9.1 The Post correspondence problem . . . 98
4.9.2 Hilbert's tenth problem . . . 100
4.10 Summary . . . 101

5 Higher-order programming **102**
5.1 Which formulas are goals and which are clauses . . . 103
5.1.1 Rigid and flexible atoms . . . 103
5.1.2 Why clause heads must be rigid . . . 103
5.1.3 No implications inside terms . . . 104
5.1.4 Herbrand universes, hohc and hohh . . . 104
5.1.5 hohh$^+$: hiding predicate names . . . 106
5.2 Higher-order relational programming . . . 107
5.2.1 Mapping, filtering and quantifying over lists . . . 107
5.2.2 λ-terms of predicate type as arguments . . . 108
5.2.3 Closures, composition and folds . . . 110
5.2.4 Binary clauses: continuation passing . . . 112
5.2.5 Memoisation with hidden, growing predicates . . . 113
5.3 Flexible goals . . . 114
5.4 Reasoning about programs by substitution . . . 116
5.5 Defining the logical constants . . . 117
5.6 Conditional and negation-as-failure . . . 118
5.7 λ-terms as functions . . . 119
5.7.1 `mapfun` and `reducefun` . . . 119
5.7.2 Functional difference lists . . . 121
5.8 Higher-order unification is not a panacea . . . 123
5.9 Comparison with functional programming . . . 124
5.10 Modules: programming in the large with logic . . . 125
5.10.1 Modules, signatures and accumulation . . . 125
5.10.2 The logical reading: $\exists$ over clauses, $\supset$ for queries . . . 127
5.10.3 Abstract datatypes . . . 129
5.10.4 Higher-order programming and predicate visibility . . . 130
5.11 What a Scheme-hosted miniKanren gets for free . . . 132
5.12 Summary . . . 132

6 λ-tree syntax: computing with binders **135**
6.1 The problem: syntax with binders . . . 135
6.2 Encoding objects with binding structure . . . 136
6.2.1 First-order formulas . . . 136
6.2.2 Untyped λ-terms . . . 137
6.2.3 What the encoding gives for free . . . 138
6.3 Object-level substitution is meta-level β . . . 138
6.3.1 Instantiating quantifiers . . . 138
6.3.2 An interpreter for an object logic . . . 139
6.3.3 Evaluators for the untyped λ-calculus . . . 140
6.4 Mobility of binders . . . 142
6.4.1 Why an abstraction cannot be taken apart . . . 142
6.4.2 The proof-search rules behind mobility . . . 142
6.4.3 A first example: recognising terms . . . 142

6.5 Computing with untyped λ-terms . . . 145
6.5.1 β-normal forms . . . 145
6.5.2 Reduction to normal form . . . 146
6.5.3 Reduction based on paths . . . 147
6.5.4 Type inference . . . 150
6.5.5 Translating to and from de Bruijn syntax . . . 152
6.6 Computations over first-order formulas . . . 155
6.6.1 Vacuous quantifiers and quantifier-free formulas . . . 155
6.6.2 Negation normal form . . . 155
6.6.3 Prenex normal form . . . 156
6.6.4 Syntactic classes and a richer interpreter . . . 158
6.7 Specifying substitution without β . . . 159
6.7.1 Copy clauses . . . 159
6.7.2 Copy clauses for first-order formulas, and where they come from . . . 160
6.7.3 Substitution runs backwards . . . 162
6.8 The λ-tree approach to abstract syntax . . . 163
6.9 The L_λ subset . . . 164
6.10 Summary . . . 167

7 Unification of λ-terms and higher-order patterns 169
7.1 What this chapter is about . . . 169
7.2 Unification problems and their normal form . . . 170
7.2.1 The normal form assumed by the algorithms . . . 170
7.3 Why general higher-order unification is hard . . . 171
7.4 Huet's procedure . . . 172
7.4.1 Simplification of rigid-rigid equations . . . 172
7.4.2 Imitation and projection . . . 173
7.4.3 A complete run, and the matching tree . . . 174
7.4.4 Flexible-flexible equations and pre-unification . . . 175
7.4.5 Non-termination . . . 175
7.5 Higher-order patterns . . . 176
7.5.1 Why the procedure becomes deterministic . . . 178
7.5.2 Flexible-flexible pattern equations have mgus . . . 178
7.5.3 Termination needs the occurs-check . . . 179
7.6 The pattern unification algorithm . . . 179
7.7 Worked traces . . . 182
7.8 Nipkow's de Bruijn algorithm . . . 184
7.8.1 An executable transliteration . . . 185
7.9 Mixed prefixes, levels and delayed equations . . . 189
7.9.1 Dynamic L_λ: delay what is not a pattern . . . 191
7.10 Summary . . . 193

8 Applications: provers, functional programs, processes **195**
8.1 Proof systems as logic programs . . . 195
8.1.1 Intuitionistic propositional logic: from a loop to a decision procedure . 195
8.1.2 Natural deduction with proof terms . . . 200
8.1.3 A classical prover with bounded contraction . . . 203
8.1.4 Tactics and tacticals . . . 205
8.2 Computations over functional programs . . . 210
8.2.1 Typing: hypotheses about bound variables . . . 210
8.2.2 Big-step evaluation . . . 212
8.2.3 Evaluation contexts as meta-level functions . . . 214
8.2.4 Partial evaluation: unfolding and mixed evaluation . . . 214
8.2.5 CPS conversion and administrative redexes . . . 215
8.3 A process calculus: the π-calculus . . . 217
8.3.1 Syntax and one-step transitions . . . 217
8.3.2 Traces, may and must . . . 220
8.3.3 Replication, and the λ-calculus inside π . . . 223
8.4 Summary . . . 224

II Higher-order miniKanren **226**

9 Higher-order miniKanren: the literature **227**
9.1 The map of the territory . . . 227
9.2 Ma and Friedman 2021: static closures . . . 228
9.2.1 What kind of paper this is . . . 228
9.2.2 Grafting versus substitution . . . 229
9.2.3 Binder-merging and "true = false" . . . 232
9.2.4 Static closures . . . 233
9.2.5 A reconstruction of the rules (not in the paper) . . . 235
9.2.6 The language: Horn clauses, `assume`, `all` . . . 237
9.3 A critical assessment of static closures . . . 238
9.3.1 Three counter-examples . . . 239
9.3.2 Further observations . . . 240
9.4 λKanren 2020: Miller's algorithm in a shallow embedding . . . 242
9.5 Nominal unification and αKanren . . . 245
9.5.1 αKanren . . . 246
9.6 Eigenvariables, negation and ∀ inside miniKanren . . . 247
9.6.1 The scope-extrusion problem . . . 247
9.6.2 `eigen` in miniKanren with symbolic constraints . . . 248
9.6.3 Moiseenko 2019: constructive negation . . . 249
9.6.4 Jin, Rosenblatt, Might and Zhang 2021: ∀ and ⟹ by relative complement 250
9.7 How λProlog systems do it . . . 251
9.7.1 Miller 1991: L_λ by transitions over a mixed prefix . . . 251
9.7.2 Nipkow 1993: functional pattern unification on de Bruijn terms . . . 252

9.7.3 Dowek, Hardin, Kirchner, Pfenning 1996: explicit substitutions 253
9.7.4 Universe indices: Nadathur, Jayaraman and Kwon 1995 253
9.7.5 Teyjus: suspensions and on-the-fly raising 254
9.7.6 ELPI: de Bruijn levels and a reduction-free fragment 254
9.7.7 Makam 256
9.8 Comparison of the design options 256
9.9 Summary 258

III A proposal for (aux kanren micro) 259

10 Anatomy of (aux kanren micro) 260
10.1 Terms and variables 260
10.2 The state 262
10.3 The substitution and the walk 263
10.4 Unification 265
10.4.1 Binding and the occurs check 265
10.4.2 The unification algorithm 265
10.5 The constraint stores and their re-verification 268
10.5.1 D: disequalities 268
10.5.2 A: disjoint type tags 269
10.5.3 T: absento 269
10.5.4 The post-unification chain 270
10.6 Goals and the interleaving search 271
10.6.1 Streams and goals 271
10.6.2 `define-relation` and the inverse-η delay 272
10.6.3 Soft cut, bounded runs, escape to Scheme 273
10.7 Reification and the runners 274
10.7.1 From a state to a λ-expression 274
10.7.2 Running 276
10.8 Aggregation 276
10.9 Extension points 277
10.9.1 Open versus closed 277
10.9.2 Three tricks that need no edit, and their limits 278
10.9.3 Trick 3: a ∀ prototype with a post-filter scope check 280
10.10 Summary 282

11 Bringing λProlog to (aux kanren micro): the design 284
11.1 Requirements 285
11.1.1 Setting and notation 285
11.1.2 The nine items 286
11.2 The design space 293
11.3 Three prototypes 294
11.3.1 d1: a layer with no edit to micro 294

11.3.2 d2: an extension protocol in micro . . . 295
11.3.3 d3: a native engine . . . 295
11.3.4 The verdict . . . 296
11.4 The extension protocol . . . 297
11.4.1 The seventh field . . . 298
11.4.2 Extensions and the registry . . . 298
11.4.3 The hooks . . . 299
11.4.4 Two primitives: forget and post-unify . . . 305
11.4.5 One generic change to absent° . . . 305
11.4.6 The two inherited invariants . . . 306
11.4.7 From closed points to hooks . . . 307
11.4.8 Cost . . . 309
11.5 Review history . . . 309
11.6 Summary . . . 310

12 The module `(aux kanren lambda)` 312
12.1 Terms . . . 313
12.1.1 Four records . . . 313
12.1.2 The constructors `ϰ` and `@` . . . 313
12.1.3 Three kinds of application . . . 315
12.1.4 Scheme lists versus λ-tree application . . . 315
12.2 The extension's state . . . 316
12.2.1 The record in `X` . . . 316
12.2.2 Copied, never mutated . . . 317
12.3 Unification . . . 318
12.3.1 Where the rules run . . . 318
12.3.2 Notation . . . 319
12.3.3 The rules . . . 319
12.3.4 Variable elimination . . . 321
12.3.5 The eager scope check . . . 323
12.3.6 Fast paths . . . 324
12.3.7 Wake-up and termination . . . 324
12.3.8 Worked traces . . . 326
12.4 `∀°` and `⊃°` . . . 329
12.4.1 GENERIC . . . 329
12.4.2 Discharge at the exit of a scope . . . 331
12.4.3 AUGMENT . . . 332
12.4.4 BACKCHAIN and `define-dynamic-relation` . . . 333
12.4.5 Clause syntax . . . 334
12.4.6 Worked example: type inference . . . 335
12.5 micro's stores on λ-terms . . . 337
12.5.1 Disequality: the `diseq` hook and the store N . . . 337
12.5.2 Absence: the `parts` hook . . . 340
12.5.3 Types: the `tag` hook . . . 340

12.6 Reification . . . 341
12.7 API reference . . . 343
12.8 Summary . . . 344

13 The book's examples in `(aux kanren lambda)` 346
13.1 How to read the examples . . . 346
13.1.1 The dictionary . . . 347
13.1.2 Three ways to print an answer . . . 347
13.1.3 The rules, in one table . . . 348
13.2 λ-tree syntax: α, β and shadowing . . . 349
13.3 Universal goals and the scope of eigenvariables . . . 350
13.4 Hypothetical goals and the program context . . . 352
13.4.1 Clause order under augmentation . . . 352
13.4.2 The course database (book Fig. 3.1) . . . 353
13.4.3 Jars and bugs . . . 355
13.4.4 Reverse with assumed clauses (book Fig. 3.4) . . . 356
13.5 Type inference: ∀ and ⊃ together . . . 357
13.6 Object-level substitution . . . 359
13.6.1 Call-by-name evaluation . . . 359
13.6.2 `subst R N (R N)`: dynamic L_λ . . . 359
13.6.3 Copy and substitution without β (book Fig. 7.10) . . . 360
13.6.4 Existential generalisation (book Fig. 7.11) . . . 361
13.7 Recursion over binders: de Bruijn terms and prenex forms . . . 362
13.7.1 Translating to and from de Bruijn syntax . . . 362
13.7.2 Prenex normal forms (book Fig. 7.8) . . . 364
13.8 Higher-order programming . . . 366
13.8.1 `mappred` . . . 366
13.8.2 Palindromes with functional difference lists (book Fig. 5.11) . . . 366
13.9 Fischer's CPS transformation (book Fig. 10.6) . . . 368
13.10 Higher-order pattern unification . . . 370
13.10.1 Flex-rigid and flex-flex . . . 370
13.10.2 Example 8.H: pruning . . . 370
13.10.3 Example 8.K: delayed raising . . . 371
13.10.4 Failures: occurs check, scope (8.G, 8.I, 8.J) . . . 372
13.10.5 Non-patterns: suspended, reported, woken . . . 372
13.10.6 Occurrences that must not be pruned . . . 374
13.11 Ma and Friedman 2021, answered by λProlog's semantics . . . 375
13.11.1 Where the answers agree . . . 375
13.11.2 Where the answers differ . . . 376
13.12 The examples at a glance . . . 378
13.13 Summary . . . 379

14 Limitations, costs and a roadmap **381**
14.1 How the proposal is tested . . . 381
14.1.1 Three layers of tests . . . 381
14.1.2 How a case is written . . . 382
14.1.3 micro's own suites, run twice . . . 383
14.1.4 The run . . . 384
14.2 Limitations and deviations from λProlog . . . 385
14.2.1 Only dynamic relations take hypotheses . . . 385
14.2.2 Untyped terms . . . 386
14.2.3 Non-patterns are suspended, never enumerated . . . 386
14.2.4 Answer order is micro's interleaving . . . 387
14.2.5 Disequality, absence and types under ∀ . . . 387
14.2.6 Host escapes . . . 388
14.2.7 Binder names are hints; projected constraints . . . 389
14.2.8 Costs . . . 390
14.2.9 The answer variable outside every ∀° . . . 390
14.3 Performance . . . 390
14.3.1 The protocol on first-order search . . . 390
14.3.2 With D or A non-empty . . . 391
14.3.3 After a ∀° . . . 391
14.3.4 λ workloads . . . 392
14.4 A staged merge into the aux repository . . . 393
14.4.1 Stage 1: the protocol . . . 393
14.4.2 Stage 2: the module and its suites . . . 394
14.4.3 Stage 3: the show module . . . 395
14.5 Later work . . . 395
14.5.1 (a) One walk for the occurs and the scope check . . . 395
14.5.2 (b) Var–var orientation by level . . . 396
14.5.3 (c) Indexing E and N by variable . . . 396
14.5.4 (d) Huet pre-unification as an optional search . . . 396
14.5.5 (e) Predicate quantification . . . 397
14.5.6 (f) Simple types . . . 397
14.6 Open questions for the author . . . 397
14.7 Summary . . . 399

Appendices **400**

A Reference card: rules and algorithms **401**
A.1 λProlog concrete syntax and its logical reading . . . 401
A.2 Goal and clause grammars . . . 402
A.3 Uniform proofs . . . 403
A.4 Typed λ-terms . . . 405
A.5 λ-conversion and normal forms . . . 405

A.6 Unification problems, raising, Skolemization . . . 406
A.7 First-order unification . . . 407
A.8 Huet's procedure . . . 408
A.9 Higher-order pattern unification . . . 409
A.10 Universe levels . . . 411
A.11 Summary . . . 412

B Glossary **413**
B.1 How to read this glossary . . . 413
B.2 Terms from A to Z . . . 414
B.3 A vocabulary bridge . . . 431
B.4 Summary . . . 433

Bibliography **434**

1 Introduction

(aux kanren micro) is a small, fast miniKanren. Its terms are first-order Scheme data, its unifier is first-order unification with an occurs check, and its goals are conjunctions, disjunctions and existentials. In logical terms it runs first-order Horn clauses (Section 2.12). λProlog, the language of Miller and Nadathur's *Programming with Higher-Order Logic* [36], runs a much larger logic. Goals may say "for every new name x" and "assuming D", terms may contain λ-binders, and unification works modulo α, β and η. With these three features, programs over syntax with binders (type checkers, evaluators, provers, program transformations) contain no code for renaming, fresh names or capture-avoiding substitution, because the logic does that work.

This document asks what it would take to give (aux kanren micro) the same power, and it ends with a concrete answer: a module, `(aux kanren lambda)`, prototyped, tested and benchmarked. This chapter says what the document is for (Section 1.1), runs one example through the whole story (Section 1.2), maps the chapters (Section 1.3), proposes three ways to read them (Section 1.4), and fixes the conventions (Section 1.5).

1.1 What this document is for

The goal. The goal is to bring three ideas of λProlog into (aux kanren micro):

1. **hereditary Harrop goals**, that is the universal goal $\forall x\, G$ (`pi x\ G`, proved with a new scoped constant) and the hypothetical goal $D \supset G$ (`D => G`, proved with the program temporarily extended by the clause D);
2. **λ-tree syntax**: object-level binders represented by meta-level λ-abstractions, so that α-equivalence is built in and object-level substitution is meta-level β-reduction;
3. **higher-order pattern unification** (L_λ): unification of λ-terms restricted to the fragment where it is decidable and has most general unifiers, with the other equations delayed.

The three are not independent. `pi` is what lets a program walk under a λ-binder, and pattern unification is what lets it build a λ-term back. The book's programs use all three together.

The sources. There are two. The first is the book itself, which Part I summarizes chapter by chapter, with its core examples run on (aux kanren micro) wherever that is possible today. The second is the miniKanren literature on these features, which Part II surveys. Its center is Ma and Friedman's *A New Higher-order Unification Algorithm for λKanren* [29], read together with its predecessor λKanren [30], nominal unification and αKanren [58, 5], the eigenvariable and negation work [21, 41, 26], and the techniques of real λProlog systems: universe levels [43], Miller's and Nipkow's pattern unification [34, 48], delayed equations [8], Teyjus [44] and ELPI [9].

The outcome. Part III is a proposal in two pieces.

- A **generic extension protocol** in (aux kanren micro) (`micro.diff`, +235/−41 lines, comments included; 214 of the added lines are not blank). The state gains one slot that (aux kanren micro) never reads, and a registry of hooks opens the points that are closed today: the unifier's structural clause, the occurs check, the step after each `=°`, the disequality, type and absento stores, and reification. Nothing in the edit mentions λ, levels or clauses.
- A **client module**, `(aux kanren lambda)`, 1255 lines, which registers one extension. It provides λ-terms (`ƛ`, `@`), the goals `∀°` and `⊃°` with `define-dynamic-relation`, and L_λ unification with universe levels and delayed non-patterns. Through the hooks it also makes (aux kanren micro)'s own `≠°`, `absent°`, `symbol°` and `number°` λ-aware; it exports no second version of them.

The design was not chosen on paper. Three prototypes were built: one that leaves (aux kanren micro) untouched, one with a hook protocol, one with λ-terms native in the core. They were compared by three reviews, on engineering, on fidelity to the book and on pedagogy. The hook design, ranked first by two of the three, became the base, and about twenty grafts proposed from the other two were each adopted or explicitly declined. Three adversarial review rounds then attacked the result and sent thirteen, eight and twelve reports. All were real. Every defect but one was fixed and pinned by a regression case (a cost reported by the third round among them); the exception, the cost of the scope check on first-order search once a `∀°` has run, reported by the second round, is measured and documented as a limitation. The final suite has 83 cases with 320 hand-written assertions, plus 6 cases that pin the known limits, and (aux kanren micro)'s own 125 cases pass unchanged, with and without the module loaded: 339 cases, no failures. The protocol costs about 1% on first-order search when no extension is loaded. All of this lives in `doc/hol-kanren/prototype/`, and after merging it would be `src/aux.kanren.lambda.scm` and `src/test/microkanren-lambda*.scm`.

1.2 The whole story in brief

This section follows one program from the book through the whole document: type inference for the untyped λ-calculus [MN p. 190–191]. It is two clauses of λProlog, and it uses every feature at once.

1.2.1 The program, twice

Object-level λ-terms are built with two constructors, `app` for application and `abs` for abstraction. `abs` takes a *meta-level* abstraction: the identity is `abs x\ x`. Types are built with `arr`. The relation `typeof M T` says that term `M` has type `T`:

```
typeof (app M N) A :- typeof M (arr B A), typeof N B.
typeof (abs M) (arr A B) :- pi x\ typeof x A => typeof (M x) B.
```

The second clause reads: "to type `abs M`, take a new constant `x`, assume that `x` has type `A`, and type the body `(M x)` with `B`". There is no clause for variables and no context argument. The hypothesis `typeof x A` *is* the context, and it disappears when the proof of the body is over. In the proposed module the same program reads:

```
(define-dynamic-relation (typeof° m t)
  (cond°
    ((fresh° (M N B) (=° m `(app ,M ,N)) (typeof° M `(arr ,B ,t)) (typeof° N B)))
    ((fresh° (R A B) (=° m `(abs ,R)) (=° t `(arr ,A ,B))
       (∀° (x) (⊃° ((typeof° x A)) (typeof° (@ R x) B)))))))
```

The dictionary is short. `pi x\ G` is `(∀° (x) G)`, `D => G` is `(⊃° (D) G)`, the application `(M x)` of a logic variable is `(@ R x)`, and `x\ t` is `(ƛ (x) t)`. The relation is made by `define-dynamic-relation` instead of `define-relation` because it receives hypotheses. The first-order structure (`abs`, `app`, `arr`) stays ordinary Scheme data, as everywhere in (aux kanren micro).

1.2.2 Running it forwards

The book types the identity and fails on self-application [MN p. 191]:

```
?- typeof (abs x\x) Ty.          Ty = arr T1 T1
?- typeof (abs x\ app x x) Ty.   no
```

The module gives the same answers. `°->list/datum` lists the answers as plain data, and `α` is a variable that the answer leaves unbound:

```
(°->list/datum (fresh° (q) (typeof° `(abs ,(ƛ (x) x)) q)))
(°->list/datum (fresh° (q) (typeof° `(abs ,(ƛ (x) `(app ,x ,x))) q)))
```

```
((arr α α))
()
```

Two rules of the book's proof search are at work, GENERIC for ∀ and AUGMENT for ⊃ (Section 3.2). In the sequent $\Sigma; \mathscr{P} \longrightarrow G$, Σ is the signature, the constants in scope, and $\mathscr{P}$ is the program:

$$\frac{\Sigma \cup \{c\}; \mathscr{P} \longrightarrow G[c/x]}{\Sigma; \mathscr{P} \longrightarrow \forall x\, G}\ \textsc{generic} \qquad (c \notin \Sigma) \qquad\qquad \frac{\Sigma; \mathscr{P} \cup \{D\} \longrightarrow G}{\Sigma; \mathscr{P} \longrightarrow D \supset G}\ \textsc{augment}$$

Read bottom-up, GENERIC proves a universal goal by inventing a constant, and AUGMENT proves an implication by assuming its antecedent. Both changes are local: they hold for the proof of the premise only.

Tutorial: What happens to the state while `typeof°` types λx.x

Write *d* for the current *depth*, the number of enclosing `∀°`, and Γ for the assumed clauses. Both live in the search state, in the slot that the protocol adds; the substitution is (aux kanren micro)'s, as before. Each row of the table below is one step towards the first answer, and two of them carry the ideas. At step 5, R is already bound to $\lambda x.\, x$, so `(@ R c)` reduces by β to c itself: object-level substitution is meta-level β. At step 6 the dynamic relation tries its hypotheses first, most recent first, and then its two static clauses, which both fail on c, since an eigenvariable is neither an `app` nor an `abs`. The hypothesis `typeof° c A` mentions the variable A of the enclosing clause, and A is *shared*,

not renamed. That is how typing the body tells the outer goal what A is (Section 3.5).

	goal	d	effect
1	`typeof° (abs (ƛ (x) x)) q`	0	the `app` clause fails; the `abs` clause applies
2	`(abs R) = (abs (ƛ (x) x))`	0	first-order: $R := \lambda x.\, x$, `q := (arr A B)`
3	`∀° (x)`	1	a new eigenvariable c, of level 1
4	`⊃° ((typeof° c A))`	1	$\Gamma = \{$`typeof° c A`$\}$
5	`typeof° (@ R c) B`	1	`(@ R c)` head-normalizes by β to c
6	backchain on the hypothesis	1	`typeof° c A = typeof° c B`: A, B unified
7	answer leaves `⊃°` and `∀°`	0	$\Gamma = \emptyset$; `q = (arr A A)`, printed `(arr α α)`

Self-application fails at the same step. For `(abs (ƛ (x) (app x x)))`, the body needs `typeof° c (arr B′ B)` and `typeof° c B′`, so the hypothesis forces $A =$ `(arr B′ B)` and $A = B'$: first-order unification fails on its occurs check, as in the book.

The book explains this program in Section 6.5.4; Section 12.4.6 traces it again at the level of the module's state, and Section 13.5 runs it on the book's whole session, the combinator S included.

1.2.3 Running it backwards

Ask instead for terms of type $i \to i$. λProlog's depth-first search loops on this query, because it tries the `app` clause first and descends forever. The interleaving search of (aux kanren micro) does not:

```
(°->list/datum (take° 3 (fresh° (q) (typeof° q '(arr i i)))))
```

```
((abs (ƛ (x) x))
 (abs (ƛ (x) (app (abs (ƛ (x1) x1)) x)))
 (abs (ƛ (x) (app (abs (ƛ (x1) x)) x))))
```

The second answer is $\lambda x.\, (\lambda x_1.x_1)\, x$. The printer renames the inner binder to `x1`, apart from the outer one. The first answer is the instructive one. Now R is unknown when step 5 is reached, so backchaining on the hypothesis must solve the equation $R\, c \doteq c$, where R was created outside the `∀°` and c inside it. That is a *higher-order* unification problem, and it is one of the easy ones.

Definition 1.1 (Levels and patterns)**.** Every logic variable X has a *level* $\ell(X)$, the depth at which it was created. Every eigenvariable c has the level $\ell(c)$ of the `∀°` that made it, one more than the depth outside.

1. *Scope check.* A binding $X := t$, made by any `=°`, is accepted only if every eigenvariable c in t has $\ell(c) \le \ell(X)$. Each unbound variable Y of t with $\ell(Y) > \ell(X)$ is then *lowered*: replaced by a new variable of level $\ell(X)$ or, when Y occurs applied, *pruned* of the arguments that X may not see.
2. *Pattern.* An application $F\, a_1 \ldots a_n$ of an unbound variable F is a *pattern* when its arguments, after head normalization, are distinct eigenvariables, each with $\ell(a_i) > \ell(F)$.

The full rule, including the cases that must wait for a later binding, is in Section 12.3.

The first condition is the *scope check*. It is the operational content of the order of quantifiers in $\exists R\, \forall c$: R was chosen before c existed, so its value cannot mention c (Sections 3.4.3 and 4.4). Levels replace the book's explicit quantifier prefix, as Teyjus does [43]. The second condition defines Miller's fragment L_λ, in which unification is decidable and has most general unifiers [34] (Section 7.5).

Example 1.2 (Scope and patterns)**.** Here $\ell(R) = 0$ and $\ell(c) = 1$. The equation $R \doteq c$ violates the scope check and fails. The equation $R\, c \doteq c$ is a pattern: abstracting c out of the right-hand side gives the most general unifier $R := \lambda x.\, x$. In the module, with `q` in the role of R:

```
(°->list #f (fresh° (q) (∀° (x) (=° q x))))
(°->list/datum (fresh° (q) (∀° (x) (=° (@ q x) x))))
```

```
()
((ƛ (x) x))
```

The first is the book's `sigma x\ pi y\ x = y`, which is not provable [MN p. 85]: no single term equals every new constant.

The other two answers of the backwards run come from the `app` clause under the binder: the body `(@ R c)` must unify with `(app M N)`, where M and N were created inside the `∀°`, at level 1. Variable elimination (Section 7.6) builds $R := \lambda y.$ `(app (M′ y) (N′ y))` and *lowers* M and N to new variables M', N' of level 0, applied to c: $M := M'\, c$, $N := N'\, c$. Scheme code built none of these λ-terms; unification did.

When an equation is not a pattern. Some equations have no most general unifier. The book's $\forall x.\, F\, x\, x \doteq x$ [MN p. 208] has two incomparable solutions, $\lambda a.\, \lambda b.\, a$ and $\lambda a.\, \lambda b.\, b$, and $F\, x\, x$ is not a pattern, since its arguments repeat. The module does what Teyjus does, "dynamic L_λ" (Section 7.9.1): it suspends the equation, retries it after every later binding, which may have turned it into a pattern, and reports it in the answer if it is still there at the end. Under `°->pretty`, which prints constraints in the style of αKanren, the residual equation is closed over x when it leaves the scope of the `∀°`. A later binding decides it:

```
(°->pretty (fresh° (q) (∀° (x) (=° (@ q x x) x))))
(°->list/datum (fresh° (q) (∀° (x) (=° (@ q x x) x)) (=° q (ƛ (a b) b))))
```

```
((α : (= (ƛ (x) (@ α x x)) (ƛ (x) x))))
((ƛ (a b) b))
```

1.2.4 What the engine had to learn

None of the above runs on today's (aux kanren micro). Table 1.1 lists what the example needed, where the book explains it, and where this document builds it.

The state of (aux kanren micro) has a fixed set of fields, and its unifier, walks and reifier are closed (Section 10.9). The proposal opens them once, generically, and puts everything λ-specific in a client module. In the state's new slot the module keeps the depth, the level of each

Table 1.1: The running example, concept by concept.

concept	in the example	theory / in the module
λ-tree syntax, α-equivalence	`(abs (ƛ (x) x))`, binders printed apart	Chapter 6 / Chapter 12
β-reduction, application	`(@ R x)` is `x` once `R := (ƛ (x) x)`	Chapter 4 / Chapter 12
GENERIC, eigenvariables	`(∀° (x) …)`	Chapter 3 / Chapters 11 and 12
AUGMENT, dynamic relations	`(⊃° ((typeof° x A)) …)`	Chapter 3 / Chapter 12
scope check and levels	`R = x` fails	Chapters 3 and 7 / Chapter 12
pattern unification	`R x = x` gives `R := λx.x`	Chapter 7 / Chapter 12
dynamic L_λ, residuals	`F x x = x` waits	Chapter 7 / Chapter 12
interleaving search	typing backwards terminates	Chapter 2 / Chapter 10

variable, the assumed clauses Γ, the suspended constraints (equations, and types or absentos on flexible terms), and the disequalities that involve λ-terms or eigenvariables. Through the hooks it receives every unification problem that contains one of its terms, it checks the scope of the bindings that `=°` makes, and it takes part in printing. Because the hooks sit inside (aux kanren micro)'s own `=°`, every relation built on `=°`, such as (aux kanren micro)'s `cons°` or the user's own, respects the scope of `∀°` with no change. A prototype that wrapped (aux kanren micro) from the outside instead let eigenvariables escape through exactly such relations (Chapter 11).

Kanren angle: Why not just use Scheme closures?

A Scheme procedure can stand for a binder as long as it is only *called*: `mappred` and continuation passing work on today's (aux kanren micro) (Section 5.11). The backwards run above needs more: unification must look *inside* the abstraction, compare it modulo α, and build one. A closure can do none of these, so `ƛ` builds a data structure, a locally nameless term whose binders are de Bruijn indices [3, 6], which prints with the user's names.

Pitfall: Hypotheses need a dynamic relation

In λProlog every predicate can receive clauses through `=>`. In the module only a relation made by `define-dynamic-relation` can, because a relation made by `define-relation` is a closed Scheme procedure. Assuming a clause for it is an error, not a silent no-op:

```
(define-relation (static° x) (=° x 1))
(define-dynamic-relation (dynamic° x) (=° x 1))
(°->list/datum (fresh° (q) (⊃° ((static° 2)) (static° q))))   ; an error
(°->list/datum (fresh° (q) (⊃° ((dynamic° 2)) (dynamic° q))))
```

```
Error: (⊃°) not a dynamic relation, see define-dynamic-relation: #<procedure (static°
↪  x)>
(2 1)
```

The first line is what `csi` prints for the error, which stops the script; the last query was run on its own. In the second answer the hypothesis comes first, then the static clause, as in the book [MN p. 78].

1.3 Map of the document

The document has three parts and two appendices. Part I follows the book, Part II the literature, and Part III builds the proposal.

Part I: the book

Chapter 2, Logic programming as proof search. The book's Introduction and Chapters 1–2: typed first-order terms, first-order unification, the search semantics of goals, and backchaining on Horn clauses (*fohc*). It establishes that pure miniKanren *is fohc* without types and with an interleaving search, and it shows where the interleaving finds answers that depth-first search misses.

Chapter 3, Hereditary Harrop formulas. Chapter 3 of the book: AUGMENT and GENERIC, hypothetical reasoning on a course database, scoped constants, why ∃∀ and ∀∃ differ, and how assumed clauses share variables with their context. The logic is intuitionistic, indeed minimal, not classical. Its last section lists what ∀ and ⊃ ask of a miniKanren: the program context must live in the search state, and scoped constants need a scope check.

Chapter 4, Typed λ-terms and unification problems. Chapter 4: simply typed λ-terms, conversion by α, β and η as the equality of the logic, normal forms, unification problems as quantified formulas, raising, and why higher-order unification is undecidable (Post's correspondence problem, Hilbert's tenth problem).

Chapter 5, Higher-order programming. Chapters 5–6: rigid and flexible atoms, the languages *hohc* and *hohh*, predicate variables (`mappred`, folds, continuation passing), function variables solved by unification (functional difference lists), and modules as logic. It sorts the book's programs into those that run on today's (aux kanren micro) (they only *call* a predicate) and those that need λ-terms (they look *inside* one).

Chapter 6, λ-tree syntax. Chapter 7, the chapter that motivates the proposal: object-level binders as meta-level abstractions, substitution as β, the *mobility* of binders (term-level λ, then `pi`, then an eigenvariable), and the book's programs over λ-terms and formulas: evaluators, type inference, de Bruijn translation, normal forms, prenex forms, copy clauses, and substitution run backwards. It ends with the L_λ subset.

Chapter 7, Unification of λ-terms and higher-order patterns. Chapter 8: Huet's procedure and its matching trees, the pattern fragment, the pattern unification algorithm as one walk of variable elimination (inversion, pruning, occurs and scope checks), traces of the book's examples, Nipkow's de Bruijn algorithm with a running Scheme transliteration, and the pragmatics of real systems: universe levels in place of raising, and delayed non-patterns.

Chapter 8, Applications. Chapters 9–11: provers for intuitionistic and classical logic, natural deduction with proof terms, tactics, typing and evaluation of functional programs, partial evaluation, CPS conversion, and the π-calculus. For each application it records which of ∀, ⊃ and λ-term unification it needs, and which parts run on (aux kanren micro) today.

Part II: the literature

Chapter 9, Higher-order miniKanren. Ma and Friedman 2021 in full: grafting versus substitution, binder merging and its "true = false" hazard, static closures, and a critical assessment with three counter-examples. Then λKanren 2020, nominal unification and αKanren, eigenvariables and negation inside miniKanren, and the implementation techniques of Miller, Nipkow, Dowek et al., Nadathur et al., Teyjus, ELPI and Makam. It ends with a comparison of the design options (Section 9.8) and deliberately does not choose.

Part III: the proposal

Chapter 10, Anatomy of (aux kanren micro). The engine as it is today, at commit `f53bdcb`: terms and variables, the six-field state, the walk, unification, the stores D (disequalities), A (type tags) and T (absento), the interleaving search, reification and aggregation. It closes with what is open to extension and what is closed, and with three tricks that need no edit (including working AUGMENT and GENERIC prototypes) together with their limits.

Chapter 11: requirements, design space, protocol. The nine acceptance items A1–A9 collected from Parts I and II (the scope of ∀ at every binding, AUGMENT with the book's clause order, type inference, λ-tree syntax, L_λ with delay, λProlog's answers where Ma and Friedman differ, the book's programs, (aux kanren micro)'s stores on λ-terms, and (aux kanren micro)'s own suites unchanged). Then the design space, the four meanings of `=°` on λ-terms left open by Section 9.8, of which pattern unification with levels is retained; the three prototypes (a layer on top of (aux kanren micro), hooks in (aux kanren micro), a native engine) and the three reviews that ranked them; and the extension protocol that came out of it: the state's seventh field, the registry, the thirteen hooks with their contracts and call sites, the invariants every client must keep, and its cost. It ends with the history of the adversarial review rounds.

Chapter 12: the module. `(aux kanren lambda)` in detail: locally nameless terms built by `ϰ` and `@`; the extension's state (depth, levels, Γ, suspended constraints, λ-disequalities), copied and never mutated; the unification rules with variable elimination, the eager scope check, the fast paths and the wake-up loop, with worked traces; `∀°` and `⊃°`, including the discharge of

constraints at the exit of a scope and backchaining in `define-dynamic-relation`; disequality, absento and types on λ-terms; reification; and the API reference.

Chapter 13: the book's examples in the module. The core programs of Part I, each beside its λProlog original and with its actual output: α, β and shadowing; the scope of eigenvariables; the course database, jars and bugs, and reverse with assumed clauses; type inference; call-by-name evaluation, copy and substitution, existential generalization; de Bruijn translation and prenex forms; `mappred` and palindromes with functional difference lists; Fischer's CPS transformation; and the unification examples of the book's Chapter 8 one by one. It also runs Ma and Friedman's queries and explains each difference, and closes with a table of all the examples.

Chapter 14: tests, limitations, performance, merge plan. How the proposal is tested (three layers of tests, and (aux kanren micro)'s suites run twice); nine documented limitations and deviations from λProlog, each pinned by a test (among them: only dynamic relations take hypotheses, terms are untyped, non-patterns are suspended and never enumerated, answers come in (aux kanren micro)'s interleaving order); the performance measurements; a merge in three stages (first the protocol, then the module and its suites, then the show extension); six later work items; and nine questions only the maintainer of aux can answer.

Appendices

Appendix A, Reference card. The formal rules on a few pages: λProlog syntax and its logical reading, the goal and clause grammars, uniform proofs, typing, λ-conversion, raising, first-order unification, Huet's procedure, pattern unification and universe levels, each with its page in the book.

Appendix B, Glossary. The vocabulary of the three communities (proof theory, unification, miniKanren) in one alphabetical list, with pointers to the chapters and to the book, and a bridge between their names for the same idea.

1.4 Reading paths

The chapters are written to be read in order, but they need not be. Table 1.2 summarizes the three paths below; page counts are approximate.

The fast path. The minimum to understand the proposal, about 50 pages before Chapters 11 and 12:

1. Section 1.2 of this chapter (5 pages);
2. the two rules, the scope condition, and how quantification links goals and clauses: Sections 3.2, 3.4.3 and 3.5 (6 pages);
3. λ-tree syntax: Sections 6.2 to 6.4 and 6.5.4 (11 pages);
4. what rigid and flexible terms and the normal form of a unification problem are: Sections 7.1 and 7.2 (the introduction says enough about imitation and projection to skip

Section 7.4); then patterns, their unification, the worked traces and what real systems do: Sections 7.5 to 7.7 and 7.9 (14 pages in all);
5. the options: Sections 9.8 and 9.9 (3 pages; the summary defines each term the comparison uses);
6. the engine's state, its constraint stores and its extension points: Sections 10.2, 10.5 and 10.9 (9 pages; skip Section 10.5 if you know micro's D/A/T chain);
7. then Chapters 11 and 12 (about 60 pages), the limitations in Section 14.2, and the staged merge in Section 14.4 (3 pages), which says how to build and load the edited micro.

The λProlog path. Part I alone, Chapter 2 to Chapter 8, about 200 pages, is a self-contained reading of the book. The *Kanren angle* boxes can be skipped. Appendix A serves as a reference card along the way.

The implementer path. For whoever will merge or extend the code: Chapter 7 (the algorithm), Chapter 10 (the engine), Chapter 11 (the protocol), Chapter 12 (the module), and Chapter 14 (tests, costs, merge plan), about 130 pages, with Appendices A.9 and A.10 at hand. Chapter 13 then serves as a set of worked tests.

Table 1.2: Three reading paths. A filled bullet means read in full, a hollow one read the sections named in the text. Pages are those of this document.

chapter	pages	fast	λProlog	implementer
Chapter 1, Introduction	12	◦	•	•
Chapter 2, proof search	28		•	
Chapter 3, hereditary Harrop	26	◦	•	
Chapter 4, λ-terms	22		•	
Chapter 5, higher-order programming	33		•	
Chapter 6, λ-tree syntax	34	◦	•	
Chapter 7, pattern unification	26	◦	•	•
Chapter 8, applications	31		•	
Chapter 9, the literature	32	◦		
Chapter 10, the engine	24	◦		•
Chapter 11, design and protocol	28	•		•
Chapter 12, the module	34	•		•
Chapter 13, the examples	35			•
Chapter 14, status and roadmap	19	◦		•
Appendix A, reference card	12		◦	◦

1.5 Conventions

Boxes. Three kinds of box interrupt the text, and each can be skipped on a first reading. A **Tutorial** box walks through a computation step by step: a trace of unification, of a ∀ or ⊃ step, of the changes of a state (see the box in Section 1.2). A **Kanren angle** box says how a concept

maps to (aux kanren micro), what runs there today, and what the proposal changes. A **Pitfall** box marks a trap: one the book stresses, or one that a Scheme programmer is likely to fall into.

Code. Three environments, told apart by the color of the rule on their left:

```
typeof (abs M) (arr A B) :- pi x\ typeof x A => typeof (M x) B.   % λProlog
```

```
(°->list/datum (fresh° (q) (typeof° `(abs ,(ƛ (x) x)) q)))       ; aux Scheme
```

```
((arr α α))
```

The last one, `answers`, is always the *actual output* of a run of the Scheme code above it, pasted verbatim, never written by hand. Inline, `pi x\ G` is λProlog and `(∀° (x) g)` is Scheme. Answers come in three forms. `°->list/datum` gives plain data, comparable with `equal?`. `°->list #f` gives (aux kanren micro)'s own form, a closed λ-expression whose parameters are the answer's unbound variables, such as `(λ () (ƛ (x) x))`. `°->pretty`, from `(aux kanren micro show)`, gives the αKanren-style view with constraints, such as `(α : (= …))`. Reified variables print as `α`, `β`, `γ`; a working variable that a constraint keeps prints as `_3`. Names follow aux: goals end in `°`, their applicative forms in `ª`, term constructors carry no suffix (`ƛ`, `@`), and internals carry the prefix `μkanren-`.

Pages of the book. [MN p. n] refers to the *printed* page *n* of [36], the number at the top of the page, which is the PDF page minus 16.

Pitfall: Two page numberings

The comments in the prototype's files (`examples.scm`, the test suite, the module's header) cite the book as `p.N` with the *PDF* page. So `p.207` in `examples.scm`, next to the `typeof` queries, is [MN p. 191] here.

Notation. A sequent $\Sigma; \mathscr{P} \longrightarrow G$ asks for a proof of goal G from program $\mathscr{P}$ over signature Σ. Implication is written $\supset$, as in the book, and $D \supset G$ is `D => G`. The logics are *fohc* and *fohh* (first-order Horn and hereditary Harrop) and *hohc* and *hohh* (their higher-order versions). L_λ is the pattern fragment, $=_{\beta\eta}$ is λ-convertibility, $t[s/x]$ substitutes s for x in t, and $\doteq$ is an equation to be solved. $\ell(X)$ and $\ell(c)$ are the levels of Definition 1.1. Logic variables are upper case (F, X, R), eigenvariables lower case (c, x, y). Rules of proof search are set in small capitals (GENERIC, AUGMENT, BACKCHAIN).

Where the outputs come from. Snippets in Parts I and II, and in Chapter 10, run on today's (aux kanren micro), at commit `f53bdcb` of the `aux-lambdakanren` branch, unless a box says otherwise; some define a small throwaway interpreter first (Section 3.8.3). The outputs of the proposal, in this chapter and from Chapter 11 on, come from the prototype as described in Part III: (aux kanren micro) with the protocol applied (`micro.diff`), the show module with `show.diff`, and `(aux kanren lambda)`, all in `doc/hol-kanren/prototype/`. Every snippet was

run by `csi` from CHICKEN 6.0.1pre1 (rev `55615603`), as installed; the benchmarks were compiled with `csc -O2 -d1`. The document was typeset with XeLaTeX, through Tectonic 0.17.0, in Libertinus and DejaVu Sans Mono.

1.6 Summary

- The goal is to bring λProlog's universal and hypothetical goals, λ-tree syntax and higher-order pattern unification into (aux kanren micro), following Miller and Nadathur's book (Part I) and the miniKanren literature, Ma and Friedman 2021 first (Part II).
- The outcome (Part III) is a generic extension protocol in (aux kanren micro) plus the client module `(aux kanren lambda)`. Three prototypes were compared by three reviews, and three adversarial rounds hardened the result: 89 λ cases and (aux kanren micro)'s 125 cases, run with and without the module, pass, and the protocol costs about 1% on first-order search.
- The running example is `typeof`. `∀°` gives the body an eigenvariable one level deeper, and `⊃°` assumes its type. Backchaining on that hypothesis unifies the types, and the assumption disappears with the answer.
- Run backwards, the same program needs $R\,c \doteq c$: a pattern, solved by $R := \lambda x.\, x$, while $R \doteq c$ fails the scope check. Non-patterns wait and are reported as residual equations.
- Three reading paths: a fast path of about 50 pages before the design and the module, the λProlog path (Part I), and the implementer path.
- Answers are actual outputs; [MN p. n] is a printed page; the prototype's comments use PDF pages (printed +16).

Part I

The book: Programming with Higher-Order Logic

2 Logic programming as proof search

Chapter 1 gave the overall plan of this document: read Miller and Nadathur's *Programming with Higher-Order Logic* [36] with a miniKanren programmer's eyes, and find out what (aux kanren micro) would need in order to host its ideas. This chapter lays the first-order foundation. It covers the book's Introduction and its Chapters 1 and 2, plus the parts of the Teyjus appendix needed to run the examples. Nothing here goes beyond Prolog in expressive power. What is new, compared with the usual Prolog textbook, is the *point of view*. A logic program is a set of formulas, a query is a formula, and running the query means *searching for a proof* in a sequent calculus. Each logical connective has a fixed search meaning, independent of the program. Atomic goals are solved by *backchaining* on program clauses. A language counts as a logic programming language when this goal-directed search is *complete*.

For a miniKanren programmer the punchline comes early. Pure miniKanren, with `=°`, `fresh°`, `cond°` and recursive relations, *is* first-order Horn-clause logic programming, except for its search strategy and its lack of types. This chapter makes that statement precise, so that later chapters can say exactly what Horn clauses *lack*: local constants and local clauses (Chapter 3), and λ-terms as data (Chapters 4 and 6).

2.1 Computation as deduction

The Introduction distinguishes two ways of using logic in computer science [MN p. 1–3]. In *computation-as-model*, a computation is a mathematical structure (states, transitions), and logic talks *about* it from outside, as Hoare triples or temporal modalities do. In *computation-as-deduction*, the logical objects themselves (formulas, terms, proofs) *are* the elements of the computation. This second view splits in two again:

- **proof normalization**: the state is a proof (a λ-term), and computing means reducing it to normal form. This is the basis of functional programming;
- **proof search**: the state is a *sequent*, that is, a goal together with its assumptions, and computing means searching for a derivation of it. The way sequents change during the search *is* the dynamics of the computation. This is the basis of logic programming.

Proof search in logic programming is not the proof search of mathematicians. Mathematicians use *lemmas*, and the proof rule for lemmas is the *cut rule*: to prove B, state a lemma C, then prove C and prove B from C. Gentzen's cut-elimination theorem [16] says that cut can always be eliminated, for classical and for intuitionistic logic. Cut-free proofs are huge and of little use to a mathematician, but they are exactly what a machine can look for. So in logic programming "the cut rule is excluded from the execution of logic programs, and computation is

based on the search for cut-free proofs"; cut and cut elimination remain useful to *reason about* programs [MN p. 3]. We come back to this in Section 2.10.

Programs, goals, sequents. A formula is either part of a *logic program*, which gives meaning to the non-logical constants it mentions, or a *goal*, which we try to derive from the program [MN p. 3]. The state of the search is a collection of sequents

$$\Sigma; \mathscr{P} \longrightarrow G$$

where Σ is a *signature* (the typed non-logical constants available), $\mathscr{P}$ is the program and G is the goal. The Introduction states the principle that drives the whole book [MN p. 3–4]:

- if G is *not atomic*, the search commits to decomposing its top-level connective. The *search semantics* of a connective is fixed and does not depend on the program;
- if G is *atomic*, the program is consulted. The search *backchains*: it looks in $\mathscr{P}$ for an implication whose conclusion matches the atom, and then tries to prove the premise.

"Logic programming can be seen abstractly as a logical framework in which a strategy that alternates between goal reduction and backchaining is complete" [MN p. 4].

How can a sequent change? The Introduction then asks a qualitative question [MN p. 4]: if proving $\Sigma; \mathscr{P} \longrightarrow A$ leads to proving $\Sigma'; \mathscr{P}' \longrightarrow A'$, what can differ between the two? For *Horn clauses*, $\Sigma' = \Sigma$ and $\mathscr{P}' = \mathscr{P}$: signature and program are global and fixed, and only the atom changes. For *hereditary Harrop formulas*, $\Sigma' \supseteq \Sigma$ and $\mathscr{P}' \supseteq \mathscr{P}$: the search can add constants and clauses that are visible only in part of the computation, which gives modular programming and data abstraction. Since more of the dynamics then comes from the logic and not from the particular predicates, "the metatheory of the logic can be of more value in proving properties of those programs" [MN p. 4]. This chapter treats the first case; the second is Chapter 3.

2.2 Typed first-order terms

Before writing relations we need data. Book Chapter 1 fixes the language of *terms*. It has no predicates and no programs yet. It uses actual `λProlog` syntax and is the λ-free special case of what Chapter 4 generalises.

2.2.1 Sorts, type constructors and kinds

λProlog has built-in *sorts* (atomic types) [MN p. 10–11]: `int`, `real`, `string`, `in_stream`, `out_stream`, and `o`, the type of formulas, which has a special status (Section 2.4). It also has the built-in *type constructor* `list`, so `(list int)` is a type. Type constructors are classified by *kinds*:

$$\langle \text{kind} \rangle \ ::= \text{type} \mid \text{type} \to \langle \text{kind} \rangle.$$

A kind is therefore just an arity: `int : type`, `list : type -> type`. New type constructors are declared with `kind` [MN p. 12]:

```
kind  pair  type -> type -> type.
```

after which `(pair string int)` is a type. Type expressions are type variables (uppercase tokens), arrows $\tau_1 \to \tau_2$ (written `->`, right associative), and type constructors applied to exactly as many arguments as their arity [MN p. 12–13].

Definition 2.1 (Target type, order [MN p. 13])**.** Every type can be written uniquely as $\tau = \tau_1 \to \cdots \to \tau_n \to \tau_0$ with τ_0 not an arrow. τ_0 is the *target type* and $\tau_1, \ldots, \tau_n$ are the *argument types*; τ is *functional* when $n \geq 1$. The *order* of a type is

$$\mathrm{ord}(\tau) = 0 \text{ if } \tau \text{ is not functional}, \qquad \mathrm{ord}(\tau_1 \to \tau_2) = \max(\mathrm{ord}(\tau_1) + 1,\ \mathrm{ord}(\tau_2)).$$

The order counts how deeply the arrow is nested *to the left*. For example `int` and `list (int -> int)` have order 0 (the order does not look inside constructor arguments), `int -> string -> o` has order 1, and `int -> (int -> o) -> o` has order 2 [MN p. 14]. In the first-order language every declared constant has a type of order at most 1 that does not mention `o`, every variable has a type of order 0, and type variables may only be instantiated with order-0 types [MN p. 13–16]. The last condition is needed because order is not stable under type substitution: `A -> list A -> o` has order 1, but instantiating `A` with `int -> int` gives a type of order 2 [MN p. 13–14].

2.2.2 Terms, polymorphism and typing

All symbols are *constants*; a constant of functional type is a *value constructor*. Lists use the polymorphic constants `nil : list A` and `:: : A -> list A -> list A`, where `::` is infix and right associative, so `2 :: 1 :: nil` has type `list int` [MN p. 15]. Constants are declared with `type` [MN p. 16]:

```
type  pr  A -> B -> pair A B.
```

A crucial asymmetry governs polymorphism [MN p. 17]:

- each *occurrence of a constant* may take its own instance of the declared type. In `(2 :: 1 :: nil) :: (1 :: nil) :: nil` the first two `nil`s have type `list int` and the last one has type `list (list int)`;
- a *variable* has one type at all its occurrences. So `((X :: nil) :: X :: nil)` cannot be typed, because X would need both a type τ and the type `list` τ.

The rules are those of the type assignment calculus of the book's Figure 1.2 [MN p. 18], with judgments $\Sigma; \Gamma \Vdash_f t : \tau$, where Σ types the constants and Γ the variables:

$$\frac{c : \sigma \in \Sigma \qquad \tau \lhd_f \sigma}{\Sigma; \Gamma \Vdash_f c : \tau} \qquad \frac{x : \tau \in \Gamma}{\Sigma; \Gamma \Vdash_f x : \tau} \qquad \frac{\Sigma; \Gamma \Vdash_f g : \tau_1 \to \tau_2 \qquad \Sigma; \Gamma \Vdash_f t : \tau_1}{\Sigma; \Gamma \Vdash_f (g\ t) : \tau_2}$$

Here $\tau \lhd_f \sigma$ ("τ is an instance of σ") holds when τ is obtained from σ by substituting order-0 types other than o for type variables. A *first-order term* is a t with $\Sigma; \Gamma \Vdash_f t : \tau$ for some τ of order 0 different from o [MN p. 18]. The first rule is where constants are polymorphic: the

instance is chosen per occurrence. The second rule is where variables are monomorphic: Γ gives one type per variable.

The book states a principal typing property [MN p. 18–19]: if a term has some typing, it has a most general one, unique up to renaming of type variables. For example `(X :: nil) :: nil` can be typed with X : int or X : list int, and most generally with X : A, the whole term having type `list (list A)`. Application is left associative. In canonical form a term is a variable, a constant, or $(f\ t_1\ \ldots\ t_n)$ whose *head* f is always a constant [MN p. 19]: first-order variables cannot be applied.

2.2.3 Representing data

The recipe for representing a class of objects is [MN p. 19]: pick a type for it; declare one value constructor per way of building an object, all with that common target type; let recursive components have the target type as argument type. Binary trees [MN p. 20]:

```
kind  btree  type -> type.
type  empty  btree A.
type  node   A -> btree A -> btree A -> btree A.
```

`(node 3 (node 1 empty empty) (node 4 empty empty))` has type `btree int`, while a tree that mixes `3` and `"cat"` is ill typed. This declaration is *parametric*: every type variable of an argument type also occurs in the target type. λProlog does not require this; the declaration `type node A -> btree -> btree -> btree.` gives heterogeneous trees [MN p. 21]. Unlike ML datatypes, the set of constructors of a type is *open* (declarations can be added later), and terms can contain variables, so a term describes a whole class of objects [MN p. 21–22].

The same recipe encodes the abstract syntax of a language. First-order formulas [MN p. 22–23]:

```
kind  term, form  type.
type  a, b  term.
type  f     term -> term -> term.
type  var   string -> term.
% f(a, f(x, b))   is represented as   (f a (f (var "x") b))

type  p  term -> term -> form.
type  q  term -> form.

type  ff,        % encoding the false proposition
      tt         form.     % encoding the true proposition
type  &&,        % encoding conjunction
      !!,        % encoding disjunction
      ==>        form -> form -> form. % encoding implication
infixl  &&   5.
infixl  !!   4.
infixr  ==>  3.
```

With the precedences above, `(p a b) && (q a) !! (q (f a b)) ==> (p b (f b a))` stands for $((p(a,b) \wedge q(a)) \vee q(f(a,b))) \supset p(b, f(b,a))$.

The declarations `infixl`, `infixr` (and `infix`, `prefix`, `postfix`, ...) are *operator declarations* ⟨fixity⟩ $c_1, \ldots, c_n$ ⟨precedence⟩: they change only the concrete syntax, a higher number binding more tightly. For instance, after `infixl pr 5.` the expression `(3 pr 4 pr "three")` stands for `((pr ((pr 3) 4)) "three")` [MN p. 16].

Imperative programs are encoded in the same way [MN p. 24–25]. The book deliberately uses only two sorts, `expr` and `stmt`, and one *type-neutral* sort for all expressions, whether integer or boolean valued; object-level typing is left to a later analysis written as a logic program:

```
kind stmt, expr  type.
type id     string -> expr.
type c      int -> expr.
type t, f   expr.
type &&, !!,
     plus, minus,
     mult, <    expr -> expr -> expr.
type :=     expr -> expr -> stmt.        % for assignment
type cond   expr -> stmt -> stmt -> stmt. % for conditionals
type while  expr -> stmt -> stmt.        % for while loops
type seq    stmt -> stmt -> stmt.        % for composition
infixl &&, mult   5.
infixl !!, plus, minus  4.
infix  <    3.
infix  :=   2.
```

The C-like program `v = 1; i = n; while (0 < i) { v = v * i; i = i - 1; }` is then the term

```
(seq ((id "v") := (c 1))
  (seq ((id "i") := (id "n"))
    (while ((c 0) < (id "i"))
      (seq ((id "v") := (id "v") mult (id "i"))
           ((id "i") := (id "i") minus (c 1))))))
```

Because `:=` has the lowest precedence, `(id "v") := (id "v") mult (id "i")` parses as `(:= (id "v") (mult (id "v") (id "i")))`. The term *is* the program's abstract syntax tree: "first-order terms are a generalization of the labeled trees that are used in depicting abstract syntax" [MN p. 25]. Types do not express every well-formedness condition (the left side of `:=` must encode an identifier); such conditions are checked by logic programs over the representation.

Pitfall: First-order syntax does not capture binding

Adding `type all term -> form -> form.` lets us write $\forall x\,(p(a, x) \wedge q(x))$ as `(all (var "x") ((p a (var "x")) && (q (var "x"))))`. The term has the right tree shape, but nothing in it says that `(var "x")` is *bound* by `all`: renaming, capture-avoiding substitution and scoping must be programmed by hand in every computation over the syntax. "This is a fundamental limitation of first-order approaches to the treatment of syntax" [MN p. 23]. Representing object variables by meta-level logic variables instead makes object scoping uncontrollable [MN p. 22]. The book removes the limitation with λ-tree

syntax (Chapter 6).

2.3 First-order unification

Unification is the engine under backchaining. The book presents it as a nondeterministic rewriting of a *multiset of equations* [MN p. 26–29], the Martelli–Montanari style. A miniKanren programmer knows the algorithm; the point of this section is the precise vocabulary (mgu, solved form, occurs check) and the role of types.

Remark 2.2. The book is deliberately "not a formal development of logic" [MN p. 6–7]: it states its results in running prose, without numbered definitions or theorems. The definition, theorem and proposition environments in this chapter are our packaging of those statements, each with the page where the book makes it.

Before the definitions, note the three ways a program uses unification [MN p. 26–27]. Take $X : \mathsf{int}$, $L : \mathsf{list\ int}$.

- *Decomposition*: $\{(X :: L) = (1 :: 2 :: \mathsf{nil})\}$ has the unifier $\{\langle X, 1\rangle, \langle L, 2 :: \mathsf{nil}\rangle\}$: the left side is a *pattern* that extracts head and tail.
- *Recognition*: the same pattern unifies only with non-empty lists, and with T repeated the pattern (node *El T T*) recognises the non-empty trees whose two subtrees are identical.
- *Construction*: in

  $$\{(X :: L_1) = (1 :: \mathsf{nil}),\ L_2 = (2 :: \mathsf{nil}),\ (X :: L_2) = L_3\}$$

  the first equation takes an input apart, the last builds a new term from its pieces, and the "output" variable L_3 receives $1 :: 2 :: \mathsf{nil}$.

The construction case is how a clause head such as `append (X :: L1) L2 (X :: L3)` computes.

Definition 2.3 (Unification problem, unifier, mgu [MN p. 26–28])**.** A *unification problem* is a finite multiset of equations $t = s$ between first-order terms, the two sides of each equation having the same type. A *unifier* is a type-preserving substitution that makes the two sides of every equation identical. A substitution θ is *more general* than φ if φ is obtained from θ by further substitution. A *most general unifier* (mgu) is a unifier more general than every other.

Substitutions are *constraints* on the values of variables. $\{\langle X, Y\rangle\}$ says only that X and Y are equal; $\{\langle X, 1\rangle, \langle Y, 1\rangle\}$ satisfies that constraint but says more [MN p. 27]. A good unifier is as general as possible, because later steps of the computation will add more constraints, and an over-specific choice could make them fail.

Theorem 2.4 (mgu [MN p. 28])**.** *Typed first-order unification problems have most general unifiers whenever they have unifiers, and these are unique up to substitutions that rename variables.*

For instance the problem $\{(X :: L) = (Y :: Z :: \mathsf{nil})\}$ has the two mgu's

$$\{\langle X, Y\rangle, \langle L, Z :: \mathsf{nil}\rangle\} \quad \text{and} \quad \{\langle Y, X\rangle, \langle L, Z :: \mathsf{nil}\rangle\},$$

which differ by the renaming $X \leftrightarrow Y$, and infinitely many less general ground unifiers [MN p. 27–28].

The transformation system. The book gives three transformations, *term reduction*, *reorientation* and *variable elimination* [MN p. 28]. We split them into six rules (this split is ours, the content is the book's). $E \uplus \{\dots\}$ is multiset union, $\bot$ is a problem with no unifier, $\mathscr{V}$ the set of variables, and f^σ means an occurrence of f at type σ.

$$\frac{E \uplus \{f^\sigma\, t_1 \dots t_n = f^\sigma\, s_1 \dots s_n\}}{E \uplus \{t_1 = s_1, \dots, t_n = s_n\}}\ \textsc{Decompose} \qquad \frac{E \uplus \{f^\sigma\, \vec{t} = g^{\sigma'}\, \vec{s}\}}{\bot}\ \textsc{Clash}\ (f \neq g \text{ or } \sigma \neq \sigma')$$

$$\frac{E \uplus \{t = x\}}{E \uplus \{x = t\}}\ \textsc{Orient}\ (t \notin \mathscr{V}) \qquad \frac{E \uplus \{x = x\}}{E}\ \textsc{Delete} \qquad \frac{E \uplus \{x = t\}}{\bot}\ \textsc{Check}\ (x \in \mathrm{fv}(t),\ t \neq x)$$

$$\frac{E \uplus \{x = t\}}{E[t/x] \uplus \{x = t\}}\ \textsc{Eliminate}\ (x \notin \mathrm{fv}(t),\ x \in \mathrm{fv}(E))$$

Decompose and Clash are term reduction. They are justified because a *rigid* term $(f\ t_1 \dots t_n)$ keeps its head under any substitution: different heads, *or the same name at two different types*, can never be made equal [MN p. 28–29]. Orient is bookkeeping. Delete, Check and Eliminate are variable elimination, and they rest on the finiteness of terms: $x = t$ with x strictly inside t has no finite solution. This failure is the *occurs check*. The side condition of Clash is stated here for types without type variables; with polymorphic constants, "$\sigma \neq \sigma'$" becomes "σ and σ' have no common instance", and Decompose first unifies σ with σ' as generally as possible [MN p. 30–31].

Definition 2.5 (Solved form [MN p. 29])**.** E is in *solved form* if every equation has a variable on the left, and each such variable occurs nowhere else in E. A solved form is read as a substitution, and it is its own mgu.

Theorem 2.6 (Termination and correctness [MN p. 29])**.** *If variable elimination is never applied twice to the same unchanged equation, every sequence of transformations terminates in $\bot$ or in a solved form, and every transformation preserves the set of unifiers. The transformations thus define a nondeterministic algorithm for computing mgu's.*

Tutorial: Three unification traces [MN p. 29–30]

Success. The repeated variable T makes the pattern recognise trees with two identical subtrees. Write n_2 for (node 2 empty empty):

$$
\begin{aligned}
& \{(\text{node } El\ T\ T) = (\text{node } 1\ n_2\ n_2)\} && \\
\longrightarrow\ & \{El = 1,\ T = n_2,\ T = n_2\} && \text{term reduction} \\
\longrightarrow\ & \{El = 1,\ T = n_2,\ n_2 = n_2\} && \text{variable elimination} \\
\longrightarrow\ & \{El = 1,\ T = n_2,\ 2 = 2,\ \text{empty} = \text{empty},\ \text{empty} = \text{empty}\} && \text{term reduction} \\
\longrightarrow\ & \{El = 1,\ T = n_2\} && \text{term reduction} \times 3
\end{aligned}
$$

The last step removes equations between constants: Decompose with $n = 0$ arguments. The result is a solved form, hence the mgu.
Constant clash. If the right subtree is (node 3 empty empty) instead, the same steps reach the equation $2 = 3$, and term reduction yields $\bot$.
Occurs check. $\{T = (\text{bt } 1\ T\ T)\}$ fails at once, because T occurs in the right-hand side.

Types in unification. Types enter only in term reduction, when two heads have the same name: their types must agree [MN p. 30–31]. With polymorphic constants the type of an occurrence is not determined by its name, so types cannot simply be checked once in advance. The book's example: with $c : A \to A \to i$ and $a : i$, the problem $\{(c\ 1\ Y) = (c\ X\ a)\}$ is well typed (both sides have type i), but in $(c\ 1\ Y)$ the constant c occurs at type $\text{int} \to \text{int} \to i$ and in $(c\ X\ a)$ at $i \to i \to i$. Ignoring types yields the *ill-typed* unifier $\{\langle X, 1\rangle, \langle Y, a\rangle\}$ [MN p. 31]. The book stops there; completing the example, term reduction with types sees two occurrences of c at the types $\text{int} \to \text{int} \to i$ and $i \to i \to i$, which have no common instance, and fails: the typed problem has no unifier. Types are themselves first-order terms, so the same algorithm unifies them. The book concludes [MN p. 31] that types can be erased safely when the original problem's typing guarantees that the intermediate problems stay well typed, in particular when every type variable in a constructor's argument types also appears in its target type (the parametric condition of Section 2.2.3).

Logical reading and the occurs-check policy. Constants behave like universally quantified variables and logic variables like existentially quantified ones, so a unification problem is a formula $\forall x_1 \ldots \forall x_n\, \exists y_1 \ldots \exists y_m\, [t_1 = s_1 \wedge \cdots \wedge t_p = s_p]$ [MN p. 31–32]. Later chapters mix the quantifier prefix (Chapters 3 and 7). Many Prologs omit the occurs check for speed; the book does not: "we will require failure in any case where the occurs-check test is satisfied", because "finiteness of terms, enforced through this test, will be important to encoding quantifier dependencies" [MN p. 32]. Without it, deduction can be unsound [MN p. 33].

Kanren angle: Unification in (aux kanren micro)

(aux kanren micro)'s unifier already is the unifier of this section, minus types. Terms are Scheme data, so pairs, vectors and records decompose structurally (Decompose), and different atoms or different shapes fail (Clash). The substitution is *triangular*: binding $x \mapsto t$ never rewrites the other equations, and walking a term on demand plays the role of Eliminate. The occurs check is on. Here are the book's problems, run on today's (aux kanren micro) (file `unify.scm`; each `q` collects the variables of interest, and each result is printed after its label):

```
; U1  {(X :: L) = (1 :: 2 :: nil)}
(µkanren-run (q -1 #t) (fresh° (x l) (=° q (list x l)) (=° (cons x l) '(1 2))))
; U2  {(node El T T) = (node 1 (node 2 empty empty) (node 2 empty empty))}
(µkanren-run (q -1 #t)
  (fresh° (el t) (=° q (list el t))
    (=° `(node ,el ,t ,t) '(node 1 (node 2 empty empty) (node 2 empty
    ↪  empty)))))
; U3  the same with a 3 in the right subtree: constant clash
(µkanren-run (q -1 #t)
  (fresh° (el t) (=° q (list el t))
    (=° `(node ,el ,t ,t) '(node 1 (node 2 empty empty) (node 3 empty
    ↪  empty)))))
; U4  {(X :: L1) = (1 :: nil), L2 = (2 :: nil), (X :: L2) = L3}: construction
(µkanren-run (q -1 #t)
  (fresh° (x l1 l2) (=° (cons x l1) '(1)) (=° l2 '(2)) (=° (cons x l2) q)))
; U5  {(X :: L) = (Y :: Z :: nil)}: an mgu, shown up to renaming
(µkanren-run (q -1 #f)
  (fresh° (x l y z) (=° q (list x l y z)) (=° (cons x l) (list y z))))
; U6  {T = (bt 1 T T)}: occurs check
(µkanren-run (q -1 #t) (=° q `(bt 1 ,q ,q)))
; U7  {(c 1 Y) = (c X a)}: untyped unification returns the ill-typed unifier
(µkanren-run (q -1 #t) (fresh° (x y) (=° q (list x y)) (=° `(c 1 ,y) `(c ,x
↪  a))))
```

```
U1 ((1 (2)))
U2 ((1 (node 2 empty empty)))
U3 ()
U4 ((1 2))
U5 ((λ (α β) (cons α (cons (cons β (quote ())) (cons α (cons β (quote ()))))))))
U6 ()
U7 ((1 a))
```

In U4, `q` plays the output variable L_3. U5 is run with `#f` as the third argument of `µkanren-run`, which prints the raw reified answer, a `λ` over the unbound variables, instead of evaluating it as `#t` does; it reads $X = \alpha$, $L = (\beta)$, $Y = \alpha$, $Z = \beta$. Reification picks one representative of the mgu "up to renaming" (Theorem 2.4). U7 is the warning of the book come true: an untyped engine happily returns the ill-typed unifier. A typed layer would require typed constant occurrences, or a static check plus erasure for parametric

signatures. The proposal stays untyped (Section 14.2.2); simple types are a later work item (Section 14.5.6).

2.4 First-order formulas

Book Chapter 2 relaxes the signature: constants may now have target type `o`. They are the *predicate symbols*, and an expression of type `o` headed by one of them is an *atomic formula* [MN p. 34–35]:

```
type memb    A -> list A -> o.
type append  list A -> list A -> list A -> o.
```

The logical constants and their λProlog spellings are [MN p. 35]:

math	λProlog	type	fixity	conventional use
$\top$	`true`	`o`		
$\supset$	`=>`	`o -> o -> o`	infix, right assoc.	implication in goals
$\wedge$	`&`	`o -> o -> o`	infix, right assoc.	joins program clauses
$\wedge$	`,`	`o -> o -> o`	infix, left assoc.	joins goals
$\vee$	`;`	`o -> o -> o`	infix, left assoc.	
$\supset$ (reversed)	`:-`	`o -> o -> o`	infix, non assoc.	"is implied by"
$\forall_\tau x\, F$	`pi x\ F`		binder	
$\exists_\tau x\, F$	`sigma x\ F`		binder	

The two spellings of $\wedge$ and of $\supset$ are a stylistic convention [MN p. 47–48]; breaking it does not affect well-formedness. All these operators bind less tightly than application [MN p. 35], and `&` binds more tightly than `:-` [MN p. 48], so `A & B :- G` reads as $G \supset (A \wedge B)$. Only the logical constants may take arguments of type `o` in the first-order setting [MN p. 35]. Quantifiers carry an optional type, `(pi (x:T)\ F)`, inferred most generally when omitted; the scope of `x\` extends as far to the right as possible, and only the binding pattern matters, so `pi x\ (p x) => sigma y\ (q x y, pi x\ (q y x))` and `pi z\ (p z) => sigma y\ (q z y, pi v\ (q y v))` are the same formula [MN p. 36–37]. The typing rules of the quantifiers (Figure 2.1, [MN p. 38]) extend those of terms:

$$\frac{\Sigma; \Gamma, x : \tau \Vdash_f B : \mathrm{o}}{\Sigma; \Gamma \Vdash_f \forall_\tau x\, B : \mathrm{o}} \qquad \frac{\Sigma; \Gamma, x : \tau \Vdash_f B : \mathrm{o}}{\Sigma; \Gamma \Vdash_f \exists_\tau x\, B : \mathrm{o}}$$

$$\frac{\Sigma; \Gamma \Vdash_f B : \tau}{\Sigma; \Gamma \Vdash_f C : \tau}\ (B, C \text{ differ only in bound names})$$

with the proviso, for the first two, that x is not typed by Σ or Γ; the third rule renames bound variables so that the proviso can be met. A *first-order Σ-formula* is an F with $\Sigma; \emptyset \Vdash_f F : \mathrm{o}$; such formulas are closed [MN p. 38].

2.5 The search semantics of goals

A logic programming language is fixed by four ingredients [MN p. 38]: signatures, program clauses, goals, and a calculus for building proofs. A sequent $\Sigma; \mathscr{P} \longrightarrow G$ has a set $\mathscr{P}$ of Σ-formulas as its program, and it states the judgment "G is provable from $\mathscr{P}$ and Σ".

Definition 2.7 (Search semantics of the logical symbols [MN p. 39])**.** Read as steps of an interpreter:

AND reduce $\Sigma; \mathscr{P} \longrightarrow B_1 \wedge B_2$ to both $\Sigma; \mathscr{P} \longrightarrow B_1$ and $\Sigma; \mathscr{P} \longrightarrow B_2$;

OR reduce $\Sigma; \mathscr{P} \longrightarrow B_1 \vee B_2$ to either $\Sigma; \mathscr{P} \longrightarrow B_1$ or $\Sigma; \mathscr{P} \longrightarrow B_2$;

INSTAN reduce $\Sigma; \mathscr{P} \longrightarrow \exists_\tau x\, B$ to $\Sigma; \mathscr{P} \longrightarrow B[t/x]$ for some Σ-term t of type τ;

AUGMENT reduce $\Sigma; \mathscr{P} \longrightarrow B_1 \supset B_2$ to $\Sigma; \mathscr{P}, B_1 \longrightarrow B_2$;

GENERIC reduce $\Sigma; \mathscr{P} \longrightarrow \forall_\tau x\, B$ to $c : \tau, \Sigma; \mathscr{P} \longrightarrow B[c/x]$, where c is a new constant, not in Σ;

TRUE $\Sigma; \mathscr{P} \longrightarrow \top$ is provable at once.

None of these looks at $\mathscr{P}$. Read downwards, they are the *right-introduction rules* of a sequent calculus, the book's Figure 2.2 [MN p. 40]:

$$\frac{}{\Sigma; \mathscr{P} \longrightarrow \top}\ \top R \qquad \frac{\Sigma; \mathscr{P} \longrightarrow B_1 \qquad \Sigma; \mathscr{P} \longrightarrow B_2}{\Sigma; \mathscr{P} \longrightarrow B_1 \wedge B_2}\ \wedge R \qquad \frac{\Sigma; \mathscr{P} \longrightarrow B_1}{\Sigma; \mathscr{P} \longrightarrow B_1 \vee B_2}\ \vee R$$

$$\frac{\Sigma; \mathscr{P} \longrightarrow B_2}{\Sigma; \mathscr{P} \longrightarrow B_1 \vee B_2}\ \vee R$$

$$\frac{\Sigma; \mathscr{P}, B_1 \longrightarrow B_2}{\Sigma; \mathscr{P} \longrightarrow B_1 \supset B_2}\ \supset R \qquad \frac{\Sigma; \mathscr{P} \longrightarrow B[t/x] \qquad \Sigma; \emptyset \Vdash_f t : \tau}{\Sigma; \mathscr{P} \longrightarrow \exists_\tau x\, B}\ \exists R \qquad \frac{c : \tau, \Sigma; \mathscr{P} \longrightarrow B[c/x]}{\Sigma; \mathscr{P} \longrightarrow \forall_\tau x\, B}\ \forall R$$

(c not declared in Σ in $\forall R$). Each rule is sound in "any interesting logic". The question is *completeness* [MN p. 41]: if a sequent with a non-atomic goal is provable, must the premise(s) of the corresponding rule be provable too? The answer depends on which formulas are allowed and on the logic.

Example 2.8 (Why goal-directedness can fail [MN p. 41–42])**.** Let $\Sigma = \{p : \mathrm{o}, q : \mathrm{o}, r : i \to \mathrm{o}, a : i, b : i\}$.

1. $\Sigma; p \vee q \longrightarrow q \vee p$ is provable, but OR leaves $\Sigma; p \vee q \longrightarrow q$ or $\Sigma; p \vee q \longrightarrow p$, and neither is provable. A disjunction in the *program* breaks goal-directed search.
2. $\Sigma; \emptyset \longrightarrow p \vee (p \supset q)$ is a classical tautology (excluded middle), but OR leaves p, unprovable, or $p \supset q$, which AUGMENT reduces to the unprovable $\Sigma; p \longrightarrow q$.

3. $\Sigma; (r\,a \wedge r\,b) \supset q \longrightarrow \exists x\,(r\,x \supset q)$ is classically provable by cases on $r\,a$, but every choice of witness t leaves an unprovable sequent.

The book concludes [MN p. 42] that programs must not contain disjunctions (they must be *definite*, "because they do not contain indefinite information"), and that the logic should be *intuitionistic*, where excluded middle does not hold. Counterexamples 2 and 3 need $\supset$ in the *goal*, which Horn clauses forbid; the switch of logic pays off only from Chapter 3 on.

Atomic goals need rules that look at the program; the general framework leaves them open on purpose [MN p. 42]. When the search succeeds, the system reports an *answer substitution*: the free variables of a query are read as existentially quantified at its head, and the answer maps them to terms that make the computation succeed [MN p. 42–43].

2.6 Horn clauses and backchaining

Definition 2.9 (fohc [MN p. 43])**.** With A ranging over atomic formulas, the goals G and program clauses D of *first-order Horn clauses* (*fohc*) are

$$G ::= \top \mid A \mid G \wedge G \mid G \vee G \mid \exists_\tau x\,G \qquad D ::= A \mid G \supset D \mid D \wedge D \mid \forall_\tau x\,D.$$

Only $\supset$ and $\forall$ are missing at the top of goals, and only $\vee$ and $\exists$ at the top of clauses. The family contains the familiar Prolog shape $\forall x_1 \ldots \forall x_m\,(A_1 \wedge \cdots \wedge A_n \supset A_0)$, but is more liberal: the head may sit under further implications, conjunctions and universals [MN p. 43–44].

Backchaining is first explained informally [MN p. 44]: if $\mathscr{P}$ contains A, succeed; if it contains $G' \supset A$, reduce the goal A to G'. Formally it uses a *focused* sequent $\Sigma; \mathscr{P} \xrightarrow{D} A$, "try to prove the atom A by backchaining on D". The rules of the book's Figure 2.3 [MN p. 45] are

$$\frac{\Sigma; \mathscr{P} \xrightarrow{D} A}{\Sigma; \mathscr{P} \longrightarrow A}\ \textsc{decide}\ (D \in \mathscr{P}) \qquad \frac{}{\Sigma; \mathscr{P} \xrightarrow{A} A}\ \textsc{initial} \qquad \frac{\Sigma; \mathscr{P} \xrightarrow{D} A \qquad \Sigma; \mathscr{P} \longrightarrow G}{\Sigma; \mathscr{P} \xrightarrow{G \supset D} A}\ \supset L$$

$$\frac{\Sigma; \mathscr{P} \xrightarrow{D_1} A}{\Sigma; \mathscr{P} \xrightarrow{D_1 \wedge D_2} A}\ \wedge L \qquad \frac{\Sigma; \mathscr{P} \xrightarrow{D_2} A}{\Sigma; \mathscr{P} \xrightarrow{D_1 \wedge D_2} A}\ \wedge L \qquad \frac{\Sigma; \mathscr{P} \xrightarrow{D[t/x]} A \qquad \Sigma; \varnothing \Vdash_f t : \tau}{\Sigma; \mathscr{P} \xrightarrow{\forall_\tau x\,D} A}\ \forall L$$

DECIDE picks a clause; INITIAL closes the branch when the focused atom is *identical* to the goal; $\supset L$ proves the body G and continues on D; $\wedge L$ picks a conjunct; $\forall L$ instantiates. The formula over the arrow is on the left of the sequent, in focus, so these are *left*-introduction rules complementing Figure 2.2. For a Prolog-shaped clause $D = \forall \bar{x}\,(A_1 \wedge \cdots \wedge A_n \supset A_0)$ the rules compose into one step [MN p. 45]:

$$\frac{\Sigma; \mathscr{P} \longrightarrow A_1\theta \qquad \cdots \qquad \Sigma; \mathscr{P} \longrightarrow A_n\theta}{\Sigma; \mathscr{P} \xrightarrow{D} A}$$

where θ maps each x_i to a Σ-term of its type and $A = A_0\theta$: the instance $A_1\theta \wedge \cdots \wedge A_n\theta \supset A_0\theta$ follows from $\mathscr{P}$, and modus ponens gives $A_0\theta$. This grouping of small rules into large ones is a first instance of Andreoli's *focusing* [1], which the book cites as the general setting [MN p. 73].

Definition 2.10 (O-proof [MN p. 45])**.** An *O-proof* is a proof built with the rules of the book's Figures 1.2, 2.2 and 2.3.

Theorem 2.11 (Completeness of goal-directed search for fohc [MN p. 45])**.** *If $\mathscr{P}$ is an fohc program and G an fohc goal, then $\Sigma; \mathscr{P} \longrightarrow G$ has an O-proof iff it is provable in classical logic iff it is provable in intuitionistic logic.*

Proposition 2.12 (Flatness [MN p. 46])**.** *Every sequent in an O-proof of $\Sigma; \mathscr{P} \longrightarrow G$ has the form $\Sigma; \mathscr{P} \longrightarrow G'$ or $\Sigma; \mathscr{P} \xrightarrow{D} A$, with the* same *$\Sigma$ and $\mathscr{P}$.*

So in *fohc* the program and signature are flat and global, and AUGMENT and GENERIC never fire. These two rules are exactly what Chapter 3 adds. The book's term for a logic in which this kind of goal-directed ("uniform") search is complete is *abstract logic programming language* [37, 36] [MN p. 73]; *fohc* is the base case.

Tutorial: An O-proof for a query with two answers

Take the program $\mathscr{P} = \{D_1, D_2\}$ of append (Section 2.7), with $D_1 = \forall l\,(\mathsf{app}\ \mathsf{nil}\ l\ l)$ and $D_2 = \forall x \forall l_1 \forall l_2 \forall l_3\,(\mathsf{app}\ l_1\ l_2\ l_3 \supset \mathsf{app}\ (x :: l_1)\ l_2\ (x :: l_3))$, and the goal $\exists X \exists Y\ \mathsf{app}\ X\ Y\ (1 :: \mathsf{nil})$. Instead of guessing terms in $\exists R$ and $\forall L$ we use logic variables and let INITIAL unify (this is the practical version of the rules, Section 2.9).

1. $\exists R$ twice: logic variables X, Y; goal $\mathsf{app}\ X\ Y\ (1 :: \mathsf{nil})$, atomic.
2. DECIDE D_1; $\forall L$ with a new L; INITIAL unifies $\mathsf{app}\ \mathsf{nil}\ L\ L$ with the goal: $X = \mathsf{nil}$, $L = Y$, $Y = 1 :: \mathsf{nil}$. *First answer*: $X = \mathsf{nil}$, $Y = 1 :: \mathsf{nil}$.
3. Back to step 2, DECIDE D_2 instead. $\forall L$ four times with new X', L_1, L_2, L_3, then $\supset L$ splits into a focused premise and a goal premise. INITIAL on the focused premise unifies $\mathsf{app}\ (X' :: L_1)\ L_2\ (X' :: L_3)$ with the goal: $X = X' :: L_1$, $Y = L_2$, $X' = 1$, $L_3 = \mathsf{nil}$. The goal premise is now $\mathsf{app}\ L_1\ Y\ \mathsf{nil}$.
4. For $\mathsf{app}\ L_1\ Y\ \mathsf{nil}$: DECIDE D_1 succeeds with $L_1 = \mathsf{nil}$, $Y = \mathsf{nil}$. *Second answer*: $X = 1 :: \mathsf{nil}$, $Y = \mathsf{nil}$. DECIDE D_2 fails: its head needs the third argument to be $x :: l_3$, which clashes with nil.

The O-proof for the second answer, with the unifier already applied. Write A_0 for $\mathsf{app}\ (1 :: \mathsf{nil})\ \mathsf{nil}\ (1 :: \mathsf{nil})$ and A_1 for $\mathsf{app}\ \mathsf{nil}\ \mathsf{nil}\ \mathsf{nil}$:

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\Sigma; \mathscr{P} \xrightarrow{A_0} A_0}\,\text{INITIAL} \qquad \dfrac{}{\Sigma; \mathscr{P} \longrightarrow A_1}\,\text{DECIDE+}\forall L\text{+INITIAL}}{\Sigma; \mathscr{P} \xrightarrow{A_1 \supset A_0} A_0}\,\supset L}{\Sigma; \mathscr{P} \xrightarrow{D_2} A_0}\,\forall L \times 4}{\Sigma; \mathscr{P} \longrightarrow A_0}\,\text{DECIDE}}{\Sigma; \mathscr{P} \longrightarrow \exists X \exists Y\ \mathsf{app}\ X\ Y\ (1 :: \mathsf{nil})}\,\exists R \times 2$$

Every sequent in it has the same Σ and $\mathscr{P}$, as Proposition 2.12 promises.

2.7 λProlog concrete syntax and interaction

In program clauses, an unbound token beginning with an uppercase letter is a variable universally quantified over the whole clause; _ is an anonymous variable, each occurrence distinct [MN p. 46–47]. So [MN p. 47]

```
type append  list A -> list A -> list A -> o.
append nil L L.
append (X :: L1) L2 (X :: L3) :- append L1 L2 L3.
```

abbreviates `pi L\ append nil L L.` and `pi X\ pi L1\ pi L2\ pi L3\ append (X :: L1) L2 (X :: L3) :- append L1 L2 L3.`; the types of the variables are inferred (`X : A`, the lists `list A`). Anonymous variables save names for values that are never used again [MN p. 47]:

```
type sublist  list A -> list A -> o.
sublist L K :- append _ T K, append L _ T.
```

is the same clause as `sublist L K :- append U T K, append L V T.`: L is a sublist of K if K has a suffix T of which L is a prefix.

Equivalent groupings of clauses. Because the D-grammar allows $\wedge$ and $\supset$ to nest, one predicate can be written in several equivalent ways [MN p. 48–49]. With `kind bool type.`, constructors `neg`, `and`, `or`, `imp` and `type ident bool -> bool -> o.`, the four clauses of Form 1 can be compressed into a single clause, Form 3:

```
ident (neg B) (neg D) :- ident B D.
ident (and B C) (and D E) :- ident B D, ident C E.
ident (or  B C) (or  D E) :- ident B D, ident C E.
ident (imp B C) (imp D E) :- ident B D, ident C E.
```

```
ident (neg B) (neg D) &
 (ident (and B C) (and D E) &
  ident (or  B C) (or  D E) &
  ident (imp B C) (imp D E) :- ident C E) :- ident B D.
```

How does Form 3 parse and run? Since `&` binds more tightly than `:-`, it is the single D-formula

$$G_1 \supset \big(D_{\text{neg}} \wedge (G_2 \supset (D_{\text{and}} \wedge D_{\text{or}} \wedge D_{\text{imp}}))\big)$$

(universally closed), where G_1 stands for the goal `ident B D`, G_2 for `ident C E`, and $D_{\text{neg}}, \ldots$ for the four heads. It is legal because the D-grammar allows $G \supset D$ and $D \wedge D$ to nest. To solve an atom `ident (and B0 C0) X`, DECIDE picks this clause, $\forall L$ instantiates its variables, $\supset L$ sets the body `ident B D` aside as a goal premise and keeps backchaining on the conjunction, $\wedge L$ chooses the right conjunct, $\supset L$ sets `ident C E` aside, $\wedge L$ chooses D_{and}, and INITIAL unifies it with the goal. The two goals set aside are exactly the body of the second clause of Form 1. The book claims that the forms are equivalent in derivable goals and in observable behaviour, and that a smart implementation preprocesses Form 3 into Form 1 [MN p. 49]. (Figure 2.3 does not say in which order the two premises of $\supset L$ are explored; for the behaviour to match Form 1, whose body is `ident B D, ident C E`, the outer premise must be solved first. This remark is ours.) (As printed, `ident` has no base case, so it proves no closed goal; the example is about clause structure only.)

The read–prove–print loop. Programs live in modules (Section 5.10). Teyjus compiles a module with `tjcc`, links it with `tjlink` and runs it with `tjsim`, which shows the prompt `[lists] ?-` [MN p. 281–282]. In *queries* the convention for variables is dual: an uppercase token is *existentially* quantified, with outermost scope [MN p. 50]. The prove phase looks for an O-proof, with three possible outcomes: a proof, no proof, or non-termination. Non-termination cannot be ruled out by any interpreter, because provability in *fohc* is undecidable [MN p. 51]. The print phase shows the bindings of the named query variables only [MN p. 51–53]:

```
[lists] ?- append (1::nil) (2::nil) (3::nil).
no
[lists] ?- append (1::nil) (2::nil) (1::2::nil).
solved
[lists] ?- append (1::nil) (2::nil) X.
X = (1::2::nil)
[lists] ?- append X Y (1::nil).
X = nil
Y = 1::nil;
X = 1::nil
Y = nil;
no
```

Typing `;` asks for another proof. The queries `sigma X\ append (1::nil) (2::nil) X` and `append (1::nil) (2::nil) _` attempt the same sequent as the third query, but print just `solved`: `sigma`-bound and anonymous variables are not shown [MN p. 51–52].

Kanren angle: The same session in (aux kanren micro)

A *fohc* predicate becomes a `define-relation` whose body is a `cond°` with one line per clause: `fresh°` for the clause variables, `=°` for head unification, then the body goals. File `append.scm`:

```
;; append nil L L.
;; append (X :: L1) L2 (X :: L3) :- append L1 L2 L3.
(define-relation (append° l k m)
  (cond° ((=° l '()) (=° k m))
         ((fresh° (x l1 l3)
            (=° l (cons x l1)) (=° m (cons x l3))
            (append° l1 k l3)))))

(µkanren-run (q -1 #t) (append° '(1) '(2) '(3)))
(µkanren-run (q -1 #t) (append° '(1) '(2) '(1 2)))
(µkanren-run (q -1 #t) (append° '(1) '(2) q))
(µkanren-run (q -1 #t) (fresh° (x y) (=° q (list x y)) (append° x y '(1))))
```

```
append (1) (2) (3)    => ()
append (1) (2) (1 2)  => (α)
append (1) (2) X      => ((1 2))
append X Y (1)        => ((() (1)) ((1) ()))
```

`()` is `no`. The answer `(α)` is `solved`: one proof, with the query variable left unconstrained. The run argument `-1` asks for all answers, what typing `;` repeatedly does in Teyjus. The variables of an inner `fresh°` are hidden like `sigma`-bound ones.

2.8 Core examples

The book illustrates three programming styles [MN p. 53–59]: search problems, relations over recursive data, and relations over abstract syntax.

2.8.1 Search: finite-state machines

The book's Figure 2.4 [MN p. 54] describes acceptance by a finite-state machine whose arcs are labelled by *words*:

```
kind state, letter        type.
type q1, q2, q3, q4, q5   state.
type a, b                 letter.
type start, final         state -> o.
type path, trans          state -> list letter -> state -> o.
type accept               list letter -> o.

path S nil S.
path S Letters T :- trans S Arc M, append Arc Rest Letters,
                    path M Rest T.
accept W :- start S, path S W F, final F.
```

The goal `append Arc Rest Letters` consumes the arc's word as a prefix of the input. Figure 2.5 is a nondeterministic machine for $(a + b)^*(ab + ba)$, the words ending in *ab* or *ba*:

```
start q1 & final q2 & final q3.
trans q1 (a::nil) q1      & trans q1 (b::nil) q1.
trans q1 (a::b::nil) q2   & trans q1 (b::a::nil) q3.
```

To enumerate accepted words, the book first generates lists of increasing length [MN p. 55]:

```
type lists  list A -> o.
lists nil.
lists (_::L) :- lists L.
```

and reports [MN p. 55–56]

```
[fsm1] ?- accept (b::b::a::b::nil).
solved
[fsm1] ?- accept (b::a::X::Y::nil).
X = a
Y = b ;
X = b
Y = a ;
```

```
no
[fsm1] ?- lists L.
L = nil ;
L = T1 :: nil ;
L = T1 :: T2 :: nil ;
L = T1 :: T2 :: T3 :: nil
[fsm1] ?- lists L, accept L.
L = a :: b :: nil ;
L = b :: a :: nil ;
L = a :: a :: b :: nil ;
L = a :: b :: a :: nil ;
L = b :: a :: b :: nil
```

Of the last two queries the book notes that "both these queries have an unbounded number of successful proofs"; a depth-first engine "will systematically produce lists of increasing length" [MN p. 55–56]. Note the answer `L = T1 :: nil`: an answer substitution may contain variables, and then every instance of it is an answer. Why is the generator needed? Under depth-first search, `accept W` with `W` unbound tries the first `trans` clause `trans q1 (a::nil) q1` first; `append` then binds `W = a :: Rest`, and `path q1 Rest F` is in the same situation again, forever. No answer is ever printed. `lists L` breaks the loop by giving `accept` lists of fixed length, one length at a time.

Pitfall: An erratum in Figure 2.6

The book also gives a deterministic five-state machine (Figure 2.6, [MN p. 54]), printed as

```
start q1 & final q4 & final q5.
trans q1 (a::nil) q2   &   trans q1 (b::nil) q3.
trans q2 (a::nil) q1   &   trans q2 (b::nil) q4.
trans q3 (a::nil) q5   &   trans q3 (b::nil) q1.
trans q4 (a::nil) q5   &   trans q4 (b::nil) q3.
trans q5 (a::nil) q2   &   trans q5 (b::nil) q4.
```

and says that exactly the same behaviour would be observed with it [MN p. 56]. As printed it does not accept the stated language: the clauses `trans q2 (a::nil) q1` and `trans q3 (b::nil) q1` send the machine back to the start, so it rejects `aab`, `bba`, and so on. Replacing them by `trans q2 (a::nil) q2` and `trans q3 (b::nil) q3` fixes it. A brute-force comparison with the regular expression on all words of length up to 8 confirms both the defect and the fix.

2.8.2 Recursive data: insertion into a search tree

Relations over recursive data follow the structure of the data [MN p. 57]:

```
type insert  int -> btree int -> btree int -> o.
insert X empty (node X empty empty).
insert X (node A L R) (node A NL R) :- X < A,  insert X L NL.
insert X (node A L R) (node A L NR) :- X >= A, insert X R NR.
```

```
[btree] ?- insert 4 (node 3 (node 2 empty empty) empty) T.
T = node 3 (node 2 empty empty) (node 4 empty empty)
```

The comparisons `<` and `>=` are built-ins "correctly used only when both arguments are instantiated to numbers" [MN p. 49], so `insert` is well defined only when its first two arguments are closed terms [MN p. 57]. This is a *mode* restriction: the relation is not usable in every direction.

2.8.3 Abstract syntax: a sequent prover

The third style uses *fohc* to animate an inference system. The book encodes a classical propositional sequent calculus with multisets $\Gamma \longrightarrow \Delta$ on both sides (Figure 2.7, [MN p. 58]), over the formulas of Section 2.2.3:

$$\frac{}{\Gamma, \bot \longrightarrow \Delta}\ \bot L \qquad\qquad \frac{}{\Gamma, A \longrightarrow \Delta, A}\ \textsc{initial}$$

$$\frac{\Gamma, A \longrightarrow \Delta \qquad \Gamma, B \longrightarrow \Delta}{\Gamma, A \vee B \longrightarrow \Delta}\ \vee L \qquad\qquad \frac{\Gamma \longrightarrow \Delta, A, B}{\Gamma \longrightarrow \Delta, A \vee B}\ \vee R$$

$$\frac{\Gamma, A, B \longrightarrow \Delta}{\Gamma, A \wedge B \longrightarrow \Delta}\ \wedge L \qquad\qquad \frac{\Gamma \longrightarrow \Delta, A \qquad \Gamma \longrightarrow \Delta, B}{\Gamma \longrightarrow \Delta, A \wedge B}\ \wedge R$$

$$\frac{\Gamma \longrightarrow \Delta, A \qquad \Gamma, B \longrightarrow \Delta}{\Gamma, A \supset B \longrightarrow \Delta}\ \supset L \qquad\qquad \frac{\Gamma, A \longrightarrow \Delta, B}{\Gamma \longrightarrow \Delta, A \supset B}\ \supset R$$

Several formulas on the right make the calculus classical. Here Γ, Δ, A, B are *schema* variables of the object calculus, not logic variables of λProlog. Multisets are lists, and a rule matches "Γ, P" by selecting P and keeping the rest:

```
type memb_and_rest  A -> list A -> list A -> o.
memb_and_rest X (X :: L) L.
memb_and_rest X (Y :: L) (Y :: L1) :- memb_and_rest X L L1.
```

The translation recipe is mechanical [MN p. 59]: *a rule's conclusion becomes the head of a clause, its premises become the body, and the schema variables become clause variables.* The eight clauses, one per rule in the order $\bot L$, INITIAL, $\wedge L$, $\vee L$, $\supset L$, $\wedge R$, $\vee R$, $\supset R$, as printed:

```
type prv  list form -> list form -> o.

prv Gamma Delta :- memb_and_rest ff Gamma _.
prv Gamma Delta :- memb_and_rest A Gamma _,
                   memb_and_rest A Delta _.
prv Gamma Delta :-
      memb_and_rest (A && B) Gamma Gamma',
      prv (A :: B :: Gamma') Delta.
prv Gamma Delta :-
      memb_and_rest (A !! B) Gamma Gamma',
```

```
      prv (A :: Gamma') Delta, prv (B :: Gamma') Delta.
prv Gamma Delta :-
      memb_and_rest (A ==> B) Gamma Gamma',
      prv Gamma' (A :: Delta), prv (B :: Gamma') Delta.
prv Gamma Delta :-
      memb_and_rest (A && B) Delta Delta',
      prv Gamma (A :: Delta'), prv Gamma (B :: Delta').
prv Gamma Delta :-
      memb_and_rest (A !! B) Delta Delta',
      prv Gamma (A :: B :: Delta').
prv Gamma Delta :-
      memb_and_rest (A ==> B) Delta Delta',
      prv (A :: Delta) (B :: Gamma').
```

Read the $\vee L$ clause against its rule: the conclusion $\Gamma, A \vee B \longrightarrow \Delta$ becomes the head `prv Gamma Delta` together with the selection `memb_and_rest (A !! B) Gamma Gamma'` (so the clause's `Gamma'` is the rule's Γ), and the two premises become the two body goals. The INITIAL clause needs no selection of the rest: it only asks that some A be on both sides.

```
[logic] ?- prv nil (((p a b) !! ((p a b) ==> (q a a))) :: nil).
solved
```

Note the *level shift*. This is $p \vee (p \supset q)$, the formula that Example 2.8 showed goal-directed search cannot prove (it is not even an *fohc* goal, since it contains $\supset$). Here it is object-level *data*, proved by an object-level classical calculus that is *programmed* in *fohc*: the search proves the meta-level atom `prv nil (...)`, by $\vee R$ and then $\supset R$ and INITIAL on `p a b`.

Pitfall: An erratum in the last prv clause

The clause for $\supset R$ is printed [MN p. 59] with the body `prv (A :: Delta) (B :: Gamma')`. It should be `prv (A :: Gamma) (B :: Delta')`. As printed, `Gamma'` is a singleton variable, the right-hand side becomes an open list, and the initial clause can unify a hypothesis into that open tail. The printed program then "proves" $p\,a\,b \supset q\,a\,a$, which is not valid. A (aux kanren micro) port of the printed clause confirms it: with the printed clause, the query below for `(p a b) ==> (q a a)` returns `(α)` instead of `()` (file `prv-printed.scm`).

Kanren angle: The prover in (aux kanren micro)

The port keeps the clause-per-line shape (file `prv.scm`, last clause corrected). Formulas are lists such as `(!! A B)`:

```
(define-relation (memb-and-rest° x l r)
  (cond° ((=° l (cons x r)))
         ((fresh° (y l1 r1)
            (=° l (cons y l1)) (=° r (cons y r1)) (memb-and-rest° x l1 r1)))))

(define-relation (prv° Γ Δ)
  (cond°
    ((fresh° (_) (memb-and-rest° 'ff Γ _)))                              ; ⊥L
```

```
    ((fresh° (a _1 _2) (memb-and-rest° a Γ _1) (memb-and-rest° a Δ _2)))     ;
     ↪ initial
    ((fresh° (a b Γ*) (memb-and-rest° `(&& ,a ,b) Γ Γ*)                    ; ∧L
       (prv° `(,a ,b . ,Γ*) Δ)))
    ((fresh° (a b Γ*) (memb-and-rest° `(!! ,a ,b) Γ Γ*)                    ; ∨L
       (prv° (cons a Γ*) Δ) (prv° (cons b Γ*) Δ)))
    ((fresh° (a b Γ*) (memb-and-rest° `(==> ,a ,b) Γ Γ*)                   ; ⊃L
       (prv° Γ* (cons a Δ)) (prv° (cons b Γ*) Δ)))
    ((fresh° (a b Δ*) (memb-and-rest° `(&& ,a ,b) Δ Δ*)                    ; ∧R
       (prv° Γ (cons a Δ*)) (prv° Γ (cons b Δ*))))
    ((fresh° (a b Δ*) (memb-and-rest° `(!! ,a ,b) Δ Δ*)                    ; ∨R
       (prv° Γ `(,a ,b . ,Δ*))))
    ((fresh° (a b Δ*) (memb-and-rest° `(==> ,a ,b) Δ Δ*)                   ; ⊃R
       (prv° (cons a Γ) (cons b Δ*))))))

(define pab '(p a b))
(define qaa '(q a a))
(µkanren-run (q 1 #t) (prv° '() `((!! ,pab (==> ,pab ,qaa)))))
(µkanren-run (q 1 #t) (prv° '() `((==> ,pab ,qaa))))
```

```
prv nil (p a b !! (p a b ==> q a a))   => (α)
prv nil (p a b ==> q a a)              => ()
```

2.9 Pragmatics of the search

The rules of Sections 2.5 and 2.6 are nondeterministic. λProlog resolves the choices with a "simple and rigid search strategy" so that a programmer can predict behaviour and resource usage [MN p. 60]:

1. for $\vee R$ and $\wedge L$, the left subformula is tried before the right one;
2. for $\wedge R$, the left premise is derived first;
3. for DECIDE, $\mathscr{P}$ is a *list*, not a set: clauses are tried in textual order, and order and multiplicity matter;
4. the terms of $\exists R$ and $\forall L$ are not enumerated (most types are infinite) but replaced by *logic variables*, instantiated by unification [MN p. 61].

The price is *incompleteness*: "there may be sequents that have derivations but for which no derivations will be found because of the strategy. In fact, the chosen strategy even can cause indefinite looping when a more flexible search strategy might be able to find a proof" [MN p. 60]. Since the strategy is known, programs can usually be restructured, as `lists L` did for `accept`.

With a logic variable X that is new in the search, the quantifier rules and INITIAL become [MN p. 61]

$$\frac{\Sigma; \emptyset \Vdash_f X : \tau \qquad \Sigma; \mathscr{P} \longrightarrow B[X/x]}{\Sigma; \mathscr{P} \longrightarrow \exists_\tau x\, B} \qquad \frac{\Sigma; \mathscr{P} \xrightarrow{D[X/x]} A \qquad \Sigma; \emptyset \Vdash_f X : \tau}{\Sigma; \mathscr{P} \xrightarrow{\forall_\tau x\, D} A}$$

$$\frac{}{\Sigma; \mathscr{P} \xrightarrow{A'} A}\ \textsc{initial}\ (A, A' \text{ unifiable})$$

where the unifier must be "percolated all over the derivation that has been constructed thus far", and typed unification guarantees that the delayed typing premises hold. Finally the program can be precompiled [MN p. 61–62]: applying $\supset L$ and $\wedge L$ in advance turns it into a list of clauses $\forall \bar{x}\,(G_1 \wedge \cdots \wedge G_n \supset A)$, which are then partitioned by predicate name, compiled, and indexed on their arguments, as in the Warren Abstract Machine.

Cut, negation as failure, arithmetic. Book Chapter 2 has no pruning operator; it only stresses that the cut *rule* "has no connection to Prolog's pruning operator !" [MN p. 63]. Teyjus provides `!`, `fail` and `not` as metalogical predicates, with the warning that "negation-by-failure has a problematic semantics in a situation where implicational goals are used in an essential way" [MN p. 287]. The book later defines [MN p. 137]

```
type not  o -> o.
not P :- P, !, fail.
not P.
```

and shows how far this is from logic: `?- X = 2, not (1 = X).` succeeds, whereas `?- not (1 = X), X = 2.` fails, so "even a basic property such as commutativity of conjunctions becomes invalid" [MN p. 137–138] (Chapter 5). Arithmetic is *intensional*: `2 + 3` is a term, so the goal `(2 + 3 = 5)` fails [MN p. 66], and only the non-logical predicate `is` evaluates [MN p. 67,286]. The comparisons used by `insert` are of the same non-logical kind.

2.10 Logical notions: cut and presentations of fohc

Theorem 2.11 gives *fohc* the duality the book wants: solving a goal means both "it follows logically" and "a certain derivation exists" [MN p. 62]. The cut rule [MN p. 63] is

$$\frac{\Sigma; \mathscr{P} \longrightarrow B \qquad \Sigma; \mathscr{P}, B \longrightarrow G}{\Sigma; \mathscr{P} \longrightarrow G}\ \textsc{cut}$$

It uses a lemma B. Choosing lemmas takes ingenuity, so cut is not used to compute, but it is used to *reason about* computation. Cut elimination says that adding cut to classical or intuitionistic logic does not change the derivable sequents, and the completeness of O-proofs means that an adapted cut is admissible for them. Consequence: replacing a subformula of a program

by a logically equivalent one gives a program that proves the same goals, "in the sense of O-provability, although maybe not under a depth-first search strategy" [MN p. 63].

This licenses three equivalent presentations of *fohc* clauses [MN p. 63–64]: the liberal one of Definition 2.9; the common Prolog one, $F ::= A \mid F \wedge F$, $D ::= A \mid F \supset A \mid \forall_\tau x\, D$; and a compact one, $D ::= A \mid A \supset D \mid \forall_\tau x\, D$. A clause of one presentation is equivalent to a set of clauses of another through equivalences such as

$$\forall x\,(B_1 \wedge B_2) \equiv (\forall x\, B_1) \wedge (\forall x\, B_2), \qquad B_1 \supset (B_2 \supset B_3) \equiv (B_1 \wedge B_2) \supset B_3,$$

$$(B_1 \vee B_2) \supset B_3 \equiv (B_1 \supset B_3) \wedge (B_2 \supset B_3), \qquad B_1 \supset \forall x\, B_2 \equiv \forall x\,(B_1 \supset B_2)\ (x \notin \mathrm{fv}(B_1)),$$

together with distributivity of $\wedge$ and $\vee$ and the others listed on [MN p. 64].

Example 2.13 (Size and behaviour [MN p. 64–66])**.** In the common presentation, $((p \vee r) \wedge (q \vee t)) \supset s$ must become four clauses

$$(p \wedge q) \supset s, \quad (r \wedge q) \supset s, \quad (p \wedge t) \supset s, \quad (r \wedge t) \supset s,$$

which duplicates p, q, r, t, exponentially in general. It also changes behaviour: with the original clause p is derived at most once per use, with the four clauses it is derived again for each clause that mentions it. Introducing new predicates pr, qt for $p \vee r$ and $q \vee t$ instead,

$$p \supset pr, \quad r \supset pr, \quad q \supset qt, \quad t \supset qt, \quad (pr \wedge qt) \supset s,$$

grows the program only linearly. This is not a logical equivalence (there are new constants), but it is *conservative*: for every goal G over the original signature Σ_1, $\Sigma_1; \mathscr{P}, D_1 \longrightarrow G$ is provable iff $\Sigma_2; \mathscr{P}, D_2 \longrightarrow G$ is.

2.11 The role of types

Types classify expressions. In the first-order setting the only distinction types *force* is between propositions (`o`) and everything else [MN p. 66]. Equality is syntactic: "two closed expressions are equal only if they are exactly the same", so `2 + 3`, `3 + 2` and `5` are different elements of `int`, and `o` is a set of formulas, not of truth values [MN p. 66–67].

Polymorphism, transparency, determinacy. The declaration `type nil list A.` is read as $\forall A\,(\mathit{type}\ \mathit{nil}\ (\mathit{list}\ A))$: `nil` has many types [MN p. 67].

Definition 2.14 (Transparent, determinate [MN p. 67–68])**.** For an order-1 type $\tau_1 \to \cdots \to \tau_n \to \tau_0$, a type variable occurring in τ_0 is *transparent*; the type is *determinate* if all its type variables are transparent.

Knowing the target type of a determinate constructor fixes the types of its arguments: all elements of a `list A` have the same type. The constructor `type cons A -> lst -> lst.` is not determinate and builds heterogeneous lists [MN p. 68].

Checking and inference. λProlog is strongly typed: types of constants are declared, types of variables are inferred most generally, and checking is static [MN p. 68–69]. The asymmetry of Section 2.2 returns: `append (1::nil) (2::nil) X, append ("abc"::nil) ("efg"::nil) Y.` is well typed, with the two `append` occurrences at different instances, but replacing `Y` by `X` makes it ill typed, since a variable has one type [MN p. 68].

Types at run time. With polymorphism and non-determinate constructors, types cannot always be compiled away [MN p. 69]. The book's Figure 2.8 builds heterogeneous lists with the non-determinate `cons` and dispatches on the run-time type of their elements [MN p. 70]:

```
kind lst       type.
type null      lst.
type cons      A -> lst -> lst.
type separate  lst -> list int -> list real -> o.
separate (cons (X:int)  L) (X::K) M :- separate L K M.
separate (cons (X:real) L) K (X::M) :- separate L K M.
separate null nil nil.
```

The annotation `(X:int)` must be matched, during the search, against the type of the head element of the incoming list: this is run-time type processing. Figure 2.9 achieves the same with *injections*, so that the dispatch is on *terms* [MN p. 70]:

```
kind numb      type.
type inj_int   int  -> numb.
type inj_real  real -> numb.
type separate  list numb -> list int -> list real -> o.
separate ((inj_int  X)::L) (X::K) M :- separate L K M.
separate ((inj_real X)::L) K (X::M) :- separate L K M.
separate nil nil nil.
```

The book recommends determinate constructors and injections [MN p. 71–72]: static typing then catches more errors, and less type information has to be carried during proof search. Giving constants the type `A` would make nonsense well typed. The book poses the queries

```
separate (cons 1.0 (cons 2 (cons 3.0 null))) L K
separate ((inj_real 1.0)::(inj_int 2)::(inj_real 3.0)::nil) L K
```

to the two figures and says that both bind "K to (2 :: nil) and L to (1.0 :: 3.0 :: nil)" [MN p. 70]. That swaps the answers: the second argument collects the integers, so `L = (2 :: nil)` and `K = (1.0 :: 3.0 :: nil)`. The (aux kanren micro) port below confirms it.

Kanren angle: Types and (aux kanren micro)

(aux kanren micro) has one universe of Scheme data. The book observes that in the first-order setting the only distinction types *force* is between `o` and the rest, so all other sorts could be collapsed into one [MN p. 66]; (aux kanren micro) is that collapsed case. What the collapse gives up is static error detection, typed unification (U7 above) and the run-time dispatch of Figure 2.8. Injections (Figure 2.9) are ordinary miniKanren style,

tagged lists such as `(inj_int 2)`, and port clause by clause (file `separate.scm`):

```
(define-relation (separate° ns is rs)
  (cond° ((fresh° (x l k) (=° ns `((inj_int ,x) . ,l)) (=° is (cons x k))
   ↪  (separate° l k rs)))
         ((fresh° (x l m) (=° ns `((inj_real ,x) . ,l)) (=° rs (cons x m))
          ↪  (separate° l is m)))
         ((=° ns '()) (=° is '()) (=° rs '()))))

(µkanren-run (q -1 #t)
  (fresh° (l k) (=° q (list l k))
    (separate° '((inj_real 1.0) (inj_int 2) (inj_real 3.0)) l k)))
```

```
separate (...) L K => (((2) (1.0 3.0)))
```

The answer is the pair (L, K): the integers first, as the types demand. Where a type layer could live (a static checker written as a relation over declared signatures, since type unification is just `=°` on type terms) is left open: the proposal stays untyped (Section 14.2.2); simple types are a later work item (Section 14.5.6).

2.12 From fohc to `(aux kanren micro)`

We can now state precisely how *fohc* maps onto (aux kanren micro) (described in Chapter 10):

fohc (book)	(aux kanren micro)	remark
$\top$, `true` ($\top R$)	`✓°`	
$G_1 \wedge G_2$, `,` ($\wedge R$)	`(and° g1 g2)`	bind interleaves answers
$G_1 \vee G_2$, `;` ($\vee R$)	`(or° g1 g2)`, `cond°`	interleaving, not left first
$\exists x\, G$, `sigma` ($\exists R$, logic variable)	`(fresh° (x) g)`	exactly the rule of [MN p. 61]
atomic goal, DECIDE	call `(p° t ...)`	clause choice compiled into the relation
clauses of p, $\forall \bar{x}\, (G \supset p\, \bar{t})$	`define-relation` with one `cond°` line per clause	precompiled and indexed by name, [MN p. 61–62]
$\forall L$ with logic variable	`fresh°` opening each line	renaming apart is automatic
INITIAL with unification	`=°` against the clause head	the threaded substitution percolates θ
occurs check	built in	as the book requires, [MN p. 32]
answer substitution	reified answer, `α`, `β`, ...	up to renaming
typed unification	none	see U7
`<`, `>=`, `is`	`project°` + Scheme	non-logical, moded
`!` (pruning)	no hard cut; soft cut `if°`, `take°`	control, not logic
`not`	`(if° g ✗° ✓°)`	same unsoundness
$\supset R$ (AUGMENT), $\forall R$ (GENERIC)	absent	not needed in *fohc*; Chapter 3

So pure miniKanren *is fohc*, with two caveats. First, the shape of a relation (one disjunction of existentially quantified equations per predicate) is the precompiled form of [MN p. 61–62], and the built-in `=°` is definable in *fohc* by the single clause $\forall x\,(\text{eq}\ x\ x)$. Second, a Scheme-hosted miniKanren is strictly *larger* than *fohc*: goals are first-class Scheme values, and `project°` leaves the logic altogether. The *fohc* fragment is a discipline, not something the host enforces.

Kanren angle: The FSM, side by side with its λProlog original

File `fsm.scm` ports Figures 2.4 and 2.5 clause by clause (`append°` as above):

```
;; Figure 2.4
(define-relation (path° s letters t)
  (cond° ((=° letters '()) (=° s t))
         ((fresh° (arc m rest)
            (trans° s arc m) (append° arc rest letters) (path° m rest t)))))
(define-relation (accept° w)
  (fresh° (s f) (start° s) (path° s w f) (final° f)))

;; Figure 2.5
(define-relation (start° s) (=° s 'q1))
(define-relation (final° s) (cond° ((=° s 'q2)) ((=° s 'q3))))
(define-relation (trans° s arc t)
  (cond° ((=° (list s arc t) '(q1 (a) q1)))
         ((=° (list s arc t) '(q1 (b) q1)))
         ((=° (list s arc t) '(q1 (a b) q2)))
         ((=° (list s arc t) '(q1 (b a) q3)))))

(define-relation (lists° l)
  (cond° ((=° l '())) ((fresh° (h t) (=° l (cons h t)) (lists° t)))))

(μkanren-run (q -1 #t) (accept° '(b b a b)))
(μkanren-run (q -1 #t) (fresh° (x y) (=° q (list x y)) (accept° `(b a ,x ,y))))
(μkanren-run (q 4 #t) (lists° q))
(μkanren-run (q 5 #t) (lists° q) (accept° q))
(μkanren-run (q 5 #t) (accept° q))
```

```
accept (b b a b)         => (α)
accept (b a X Y)         => ((a b) (b a))
lists L          (4)     => (() (α) (α β) (α β γ))
lists L, accept L (5)    => ((a b) (b a) (a a b) (a b a) (b a b))
accept L          (5)    => ((a b) (b a) (a a b) (a b a) (b a b))
```

The first four lines agree with the book (`lists L` gives `nil`, `T1 :: nil`, …). The last line is the interesting one: `(accept° q)` *without* the generator produces answers, where depth-first search loops on the *a*-arc. The order happens to coincide with the book's here; in general one should not expect (aux kanren micro) to reproduce λProlog's answer order.

2.12.1 Depth-first search versus interleaving

λProlog commits to the depth-first, left-to-right, clause-order strategy of Section 2.9. Teyjus compiles it to an abstract machine [MN p. 277]. (aux kanren micro) keeps the same goals but changes the strategy, as μKanren does [22]: `or°` interleaves its two answer streams, switching on each suspension, and a relation call suspends. A clause that recurses forever therefore gets only a share of the search, and the remaining clauses still run. That is why `(accept° q)` terminates answer by answer.

What does this buy, logically? Theorem 2.11 is a statement about the proof system: O-proofs exist whenever proofs exist. A search strategy is complete if it finds a proof whenever one exists, and, for answers, if every answer substitution of an O-proof is eventually produced (up to instantiation). Depth-first search is not [MN p. 60]. For miniKanren's interleaving search, Rozplokhas, Vyatkin and Boulytchev [52] proved soundness and completeness with respect to a denotational semantics: every answer is eventually produced. (aux kanren micro)'s particular interleaving has not been checked against the hypotheses of that proof, so read it as a strong expectation for (aux kanren micro), not a theorem about it. Interleaving is fair for *disjunction* only. Termination is not promised: a query with no answers, or the rest of a query after its last answer, can still diverge, which is unavoidable since *fohc* provability is undecidable [MN p. 51]. Conjunction remains order-sensitive. For example

```
(μkanren-run (q 1 #t) (lists° q) (=° q 'not-a-list))
```

never returns: `lists°` offers infinitely many lists, and each is rejected by the second conjunct. We ran it with a 10-second timeout, and it produced no output. With the conjuncts swapped it fails at once.

Pitfall: Answer order is not part of the logic

The book stresses that programs are lists, so the order and multiplicity of clauses are observable under depth-first search [MN p. 60], and logically equivalent transformations can change the number of derivations (Example 2.13). Under interleaving the same holds, with a different order. A port of a book example to (aux kanren micro) should compare answer *sets*, unless the order has been checked.

Kanren angle: Negation as failure and moded built-ins in (aux kanren micro)

(aux kanren micro) has a soft cut, `(if° g? gt gf)`: if `g?` has answers, all of them continue with `gt`, otherwise `gf` runs. With it, `not` is one line, and the book's commutativity failure [MN p. 137–138] reproduces exactly (file `naf.scm`). For contrast, the last two lines use the disequality constraint `≠°`, which is not part of *fohc* (Chapter 10) but is commutative:

```
(define (not° g) (if° g ✗° ✓°))

(μkanren-run (q -1 #t) (=° q 2) (not° (=° 1 q)))
(μkanren-run (q -1 #t) (not° (=° 1 q)) (=° q 2))
(μkanren-run (q -1 #t) (=° q 2) (≠° 1 q))
(μkanren-run (q -1 #t) (≠° 1 q) (=° q 2))
```

```
X = 2, not (1 = X)   => (2)
not (1 = X), X = 2   => ()
X = 2, X =/= 1       => (2)
X =/= 1, X = 2       => (2)
```

Moded built-ins behave as in λProlog. The port of `insert` guards with `project°`, which hands the current values to Scheme (file `insert.scm`):

```
(define-relation (insert° x t t*)
  (cond° ((=° t 'empty) (=° t* `(node ,x empty empty)))
         ((fresh° (a l r nl)
            (=° t `(node ,a ,l ,r)) (=° t* `(node ,a ,nl ,r))
            (project° ((x° x) (a° a)) (if (< x° a°) ✓° ✗°))
            (insert° x l nl)))
         ((fresh° (a l r nr)
            (=° t `(node ,a ,l ,r)) (=° t* `(node ,a ,l ,nr))
            (project° ((x° x) (a° a)) (if (>= x° a°) ✓° ✗°))
            (insert° x r nr)))))

(µkanren-run (q -1 #t) (insert° 4 '(node 3 (node 2 empty empty) empty) q))
(µkanren-run (q -1 #t) (fresh° (x) (insert° x '(node 3 empty empty) q)))   ;
↪  mode broken
```

```
insert 4 (node 3 (node 2 empty empty) empty) T => ((node 3 (node 2 empty empty) (node 4
↪  empty empty)))
insert X (node 3 empty empty) T =>
Error: (<) bad argument type - not a number: _1
```

The second query breaks the mode: Scheme's `<` receives the unbound logic variable `_1` and the run aborts with the error shown (printed on standard error). Likewise intensional arithmetic (file `arith.scm`):

```
(µkanren-run (q -1 #t) (=° '(+ 2 3) 5))
(µkanren-run (q -1 #t) (=° q '(+ 2 3)))
(µkanren-run (q -1 #t)
  (fresh° (e) (=° e '(+ 2 3)) (project° ((e* e)) (=° q (apply + (cdr e*))))))
```

```
2 + 3 = 5        => ()
X = 2 + 3        => ((+ 2 3))
X is 2 + 3       => (5)
```

Kanren angle: What fohc does not need, and later chapters will

By Proposition 2.12, nothing in this chapter needs the program or the signature to change during the search, so (aux kanren micro) can keep relations as closed Scheme procedures and needs no signature object. Chapter 3 adds AUGMENT (clauses that are visible only along one branch of the search) and GENERIC (fresh constants, together with

a scoping check in unification that the occurs-check discussion of [MN p. 32] anticipates). Chapters 4 and 7 add λ-terms as data. The corresponding requirements on (aux kanren micro) are collected in Chapter 11.

2.13 Summary

- Logic programming is *computation as proof search*: the state is a sequent $\Sigma; \mathscr{P} \longrightarrow G$, computing means looking for a *cut-free* proof, and cut is used to reason about programs, not to run them [MN p. 3,63].
- Data are typed first-order terms. Kinds are arities. Each occurrence of a polymorphic constant takes its own instance of the type, but a variable has one type. Principal types exist. First-order encodings of syntax do not capture binding [MN p. 10–25].
- First-order unification is a terminating transformation system on multisets of equations. It computes mgu's, unique up to renaming. The occurs check is mandatory in the book. Types matter only when two heads have the same name, and they can be erased for parametric signatures [MN p. 26–32].
- Connectives in goals have a fixed search semantics, the right rules (AND, OR, INSTAN, AUGMENT, GENERIC, TRUE); atoms are solved by *backchaining*, the focused left rules DECIDE, INITIAL, $\supset L$, $\wedge L$, $\forall L$ [MN p. 39–45].
- In *fohc* ($G ::= \top \mid A \mid G \wedge G \mid G \vee G \mid \exists x\, G$; $D ::= A \mid G \supset D \mid D \wedge D \mid \forall x\, D$) O-proofs are complete for classical and intuitionistic logic, and the program and signature never change during the search [MN p. 43–46].
- λProlog fixes depth-first, left-to-right, clause-order search with logic variables and unification. It is predictable but incomplete. `!`, `not`, `is` and comparisons are non-logical [MN p. 60–62,137,286].
- Logically equivalent presentations of clauses prove the same goals but may differ in size and in the number of derivations; introducing predicates is conservative [MN p. 63–66].
- Types classify expressions, not values; equality is syntactic. Determinate constructors and injections are preferred to run-time type dispatch [MN p. 66–72].
- Pure miniKanren is *fohc*, without types and with an interleaving search. The interleaving search finds answers that depth-first search misses (`(accept° q)`), but conjunction order still decides termination. (aux kanren micro) reproduces the book's examples and its warnings (ill-typed unifier, non-commutative `not`, moded built-ins).

3 Hereditary Harrop formulas: implication and universal goals

In Chapter 2 we met the logic of first-order Horn clauses, *fohc*, the logic behind Prolog. Its search semantics was given by a handful of reduction rules, one per logical symbol, and two of them never fired: AUGMENT, for goals of the form $D \supset G$, and GENERIC, for goals of the form $\forall x\, G$. The reason is purely syntactic: in *fohc* neither goals nor clause bodies may contain implications or universal quantifiers, so the signature Σ and the program $\mathscr{P}$ stay fixed for the whole computation. Miller and Nadathur [36] observe that such a language "does not make very strong use of the structure afforded by logic" [MN p. 75].

This chapter lifts the restriction. The logic of *first-order hereditary Harrop formulas*, *fohh*, admits $D \supset G$ and $\forall x\, G$ in goals and hence, at any depth, in clause bodies. The two new rules have a simple operational reading:

- to prove $D \supset G$, prove G with D *temporarily added to the program*;
- to prove $\forall x\, G$, prove $G[c/x]$ with a *new constant* c temporarily added to the signature.

So "programs and signatures can grow dynamically in the course of searching for a proof" [MN p. 75], and they grow and shrink with a stack discipline. Three things follow, and they organise the chapter: hypothetical reasoning becomes a programming technique (Section 3.3); the interplay of $\forall$ with logic variables imposes a *scope condition* on unification (Sections 3.4 and 3.5); and the declarative reading of the language is no longer classical logic but intuitionistic, indeed minimal, logic (Section 3.6). The full power of $\forall$, binding *names inside data*, "must await the introduction of higher-order features" [MN p. 75]; it returns in Chapter 6. Here $\forall$ shows its scoping face. The last section (Section 3.8) collects what all this demands of a miniKanren, and of (aux kanren micro) in particular.

3.1 Syntax: goals, clauses and polarity

Definition 3.1 (*fohh* goals and clauses [MN p. 75])**.** Let A range over first-order atomic formulas. Goals G and program clauses D are given by the mutually recursive rules

$$\begin{aligned} G &::= \top \mid A \mid G \wedge G \mid G \vee G \mid \exists_\tau x\, G \mid D \supset G \mid \forall_\tau x\, G \\ D &::= A \mid G \supset D \mid D \wedge D \mid \forall_\tau x\, D \end{aligned}$$

A D-formula is called a *first-order hereditary Harrop formula*; *fohh* names both this class of formulas and the logic programming framework built on it [MN p. 76].

Compare with *fohc* (Chapter 2), where $G ::= \top \mid A \mid G \wedge G \mid G \vee G \mid \exists x\, G$ and D has the same grammar. Two remarks make the definition easy to remember.

- *The D grammar did not change.* The restrictions on clauses "are, in fact, identical to the ones present in the *fohc* setting" [MN p. 76]: no top-level $\vee$ or $\exists$, and the premise of a top-level implication is a goal. The class of clauses is larger only because goals, which appear as clause bodies, are now richer.

- *The two grammars swap roles across* $\supset$. The premise of an implication *goal* is a clause (it will be assumed); the premise of an implication *clause* is a goal (it will be proved). Goals still are not freely generated: $(p \vee q) \supset r$ is not a goal, because $p \vee q$ is not a clause.

Concrete syntax. In λProlog, $D \supset G$ in a goal is written `D => G`; a clause $G \supset A$ is written `A :- G`; `,` and `&` both denote $\wedge$ (the chapter uses `&` inside assumed conjunctions of clauses), `;` is $\vee$, `pi x\ B` is $\forall x\, B$ and `sigma x\ B` is $\exists x\, B$. Free capitalised variables of a clause are implicitly universally quantified at the front of the clause; free variables of a query are implicitly existential. The scope of `x\` extends as far to the right as possible.

Pitfall: the precedence of =>

The connectives are listed in the order `=>`, `&`, `,`, `;`, `:-`; their precedences follow that order, so `=>` binds *tightest*; `=>` and `&` are right associative [MN p. 35]. Hence `D => G1, G2` parses as `(D => G1), G2`, and the hypothesis does *not* scope over `G2`. This is why the parentheses in the queries of this chapter, such as `fact (finished X Y) => (fact (graduates X), fact (cs_major X))` and `(p 2 & p 3) => p X`, are essential, while `p 2 => p 3 => p X` means $p\,2 \supset (p\,3 \supset p\,X)$.

3.1.1 Positive and negative occurrences

With implications inside formulas it becomes meaningful to ask on which side of an arrow a subformula sits.

Definition 3.2 (polarity [MN p. 76])**.** B is a positive subformula occurrence of B. If C is a positive (negative) occurrence in B, then it is a positive (negative) occurrence in $B \wedge B'$, $B' \wedge B$, $B \vee B'$, $B' \vee B$, $B' \supset B$, $\forall x\, B$ and $\exists x\, B$, and it is a negative (positive) occurrence in $B \supset B'$. Equivalently: C is positive in B if it lies to the left of an even number of implications, negative if odd.

Proposition 3.3 ([MN p. 76])**.** *Positive subformulas of goals are goals and negative subformulas of goals are clauses; dually, positive subformulas of clauses are clauses and negative ones are goals. A fohc goal has no negative subformulas. A hereditary Harrop formula is exactly a formula in which no positive subformula occurrence is a disjunction or an existential quantification.*

The last characterisation explains the name. The negative occurrences are the places where a formula is *used as a hypothesis*. Requiring that no positive part of a clause be a disjunction or an existential says that whatever we assume, now or later in the search, never forces us to reason by cases or to invent an anonymous witness. The adjective *hereditary* refers to this "at every positive position" requirement (Section 3.6).

Example 3.4. In the clause $\forall J\,((\forall x\,(\mathsf{bug}\,x \supset \mathsf{in}\,x\,J \supset \mathsf{dead}\,x)) \supset \mathsf{sterile}\,J)$ (Section 3.4) the body $\forall x(\dots)$ is negative, hence a goal, and the subformulas $\mathsf{bug}\,x$ and $\mathsf{in}\,x\,J$ are positive again (each lies to the left of exactly two implications): they are clauses, and indeed they will be *assumed*.

3.1.2 Clausal order

Definition 3.5 (clausal order [MN p. 76–77])**.**

$$\begin{aligned} \mathit{clausal}(A) &= 0 \qquad (A \text{ atomic or } \top) \\ \mathit{clausal}(B_1 \wedge B_2) = \mathit{clausal}(B_1 \vee B_2) &= \max(\mathit{clausal}(B_1), \mathit{clausal}(B_2)) \\ \mathit{clausal}(B_1 \supset B_2) &= \max(\mathit{clausal}(B_1) + 1,\ \mathit{clausal}(B_2)) \\ \mathit{clausal}(\forall x\,B) = \mathit{clausal}(\exists x\,B) &= \mathit{clausal}(B) \end{aligned}$$

In *fohc* goals have clausal order 0 and clauses order 0 or 1; in *fohh* both are unbounded [MN p. 77]. Reading $\supset$ as the type arrow $\to$, the definition is the same as that of the order of a type (Chapter 1); the book stresses that this is "not accidental" and will exploit it in its Section 7.6 [MN p. 77]. Worked values: `q :- (q => q)`, that is $(q \supset q) \supset q$, has order 2; the sterile clause above has order 2; the clause of Fig. 3.5 below has order 3.

3.2 Operational semantics: AUGMENT and GENERIC

Nothing new has to be invented. The framework of Chapter 2 already contains the rules for implications and universal quantifiers at the top of goals: "These are to be treated by the AUGMENT and GENERIC reduction rules" [MN p. 77]. Atomic goals are still solved by backchaining, which is possible because D-formulas have the same shape as in *fohc*, modulo the richer goals. For reference, Fig. 3.1 collects the rules: the right-introduction rules for goals and the backchaining rules for a clause D "in focus", written $\Sigma; \mathscr{P} \xrightarrow{D} A$.

Definition 3.6 (O-proof [MN p. 77])**.** An *O-proof* in *fohh* is a derivation built with the typing rules for first-order terms and the rules of Fig. 3.1 (book Figures 1.2, 2.2 and 2.3).

In the logic-variable versions of the rules (Chapter 2, [MN p. 61]) the typing premise of $\exists$R and $\forall$L is delayed and initial unifies. Section 3.4 shows that once GENERIC is present the delayed choice must still respect the signature in which the variable was created.

Tutorial: programs grow and shrink like a stack

Take the goal $(D_0 \supset ((D_1 \supset G_1) \wedge (D_2 \supset G_2))) \wedge G_3$ and the program $\mathscr{P}$ [MN p. 77]. AND splits it in two. The left conjunct is an implication: AUGMENT moves D_0 into the program, and AND splits again; each conjunct is an implication again. In the end we must prove

$$G_1 \text{ from } \{D_1, D_0\} \cup \mathscr{P}, \qquad G_2 \text{ from } \{D_2, D_0\} \cup \mathscr{P}, \qquad G_3 \text{ from } \mathscr{P}.$$

D_1 is never visible to G_2, and D_0 is never visible to G_3. Read as an interpreter state, the program is pushed when we enter the antecedent's scope and popped when the subproof

$$\frac{}{\Sigma;\mathscr{P}\longrightarrow\top}\ \top\mathrm{R} \qquad \frac{\Sigma;\mathscr{P}\longrightarrow B_1 \qquad \Sigma;\mathscr{P}\longrightarrow B_2}{\Sigma;\mathscr{P}\longrightarrow B_1\wedge B_2}\ \wedge\mathrm{R} \qquad \frac{\Sigma;\mathscr{P}\longrightarrow B_i}{\Sigma;\mathscr{P}\longrightarrow B_1\vee B_2}\ \vee\mathrm{R}$$

$$\frac{\Sigma;\mathscr{P}\longrightarrow B[t/x] \qquad \Sigma;\varnothing\vdash_f t:\tau}{\Sigma;\mathscr{P}\longrightarrow \exists_\tau x\,B}\ \exists\mathrm{R}$$

$$\frac{\Sigma;\mathscr{P},B_1\longrightarrow B_2}{\Sigma;\mathscr{P}\longrightarrow B_1\supset B_2}\ \supset\mathrm{R}\ (\textsc{augment}) \qquad \frac{c:\tau,\Sigma;\mathscr{P}\longrightarrow B[c/x]}{\Sigma;\mathscr{P}\longrightarrow \forall_\tau x\,B}\ \forall\mathrm{R}\ (\textsc{generic})\quad c\notin\Sigma$$

$$\frac{\Sigma;\mathscr{P}\xrightarrow{D}A}{\Sigma;\mathscr{P}\longrightarrow A}\ \textsc{decide}\quad D\in\mathscr{P} \qquad \frac{}{\Sigma;\mathscr{P}\xrightarrow{A}A}\ \textsc{initial} \qquad \frac{\Sigma;\mathscr{P}\xrightarrow{D}A \qquad \Sigma;\mathscr{P}\longrightarrow G}{\Sigma;\mathscr{P}\xrightarrow{G\supset D}A}\ \supset\mathrm{L}$$

$$\frac{\Sigma;\mathscr{P}\xrightarrow{D_i}A}{\Sigma;\mathscr{P}\xrightarrow{D_1\wedge D_2}A}\ \wedge\mathrm{L} \qquad \frac{\Sigma;\mathscr{P}\xrightarrow{D[t/x]}A \qquad \Sigma;\varnothing\vdash_f t:\tau}{\Sigma;\mathscr{P}\xrightarrow{\forall_\tau x\,D}A}\ \forall\mathrm{L}$$

Figure 3.1: The rules of O-proofs: right rules (book Figure 2.2) and backchaining rules (book Figure 2.3). In *fohc* the two rules of the second row are never used. With logic variables, ∃R and ∀L introduce a new variable X instead of guessing t, and initial unifies.

is done: "implicational goals allow logic programs to grow and shrink following a stack-based discipline" [MN p. 78]. Notice that in the proof *tree* nothing is popped: each branch simply carries its own program. Popping is what a sequential interpreter does when it walks the tree.

Programs are lists. Under a depth-first strategy the order of clauses matters, so programs are "best thought of not as sets but rather as lists of clauses" [MN p. 78]. λProlog adds assumed clauses *at the front*: "the most recently added clauses are the first to be used in backchaining" [MN p. 78]. Inside an assumed conjunction `D1 & D2` the textual order is kept. With a program containing only `p 1`:

```
?- p 2 => p 3 => p X.
X = 3;
X = 2;
X = 1
?- (p 2 & p 3) => p X.
X = 2;
X = 3;
X = 1
```

Kanren angle: answer order is a depth-first artefact

Running the same two queries through the small interpreter of Section 3.8.3, which uses the interleaving `or°` of (aux kanren micro) for the choice among clauses, gives the same *sets* of answers in a different order:

```
p 2 => p 3 => p X    : (3 2 1)
(p 2 & p 3) => p X   : (2 1 3)
```

In the second query the ∧L choice inside `p 2 & p 3` is interleaved with the decide choice over the rest of the program. Tests of this interpreter should not compare ordered answer lists with λProlog's; the order of the program still matters, since front clauses are tried first. The proposed module does reproduce the book's order here, `(3 2 1)` and `(2 3 1)`, because a dynamic relation is one flat disjunction of its hypotheses, most recent first (Definition 11.5, Section 13.4.1). Compare ordered lists only where every clause answers without further search.

3.3 Implicational goals and hypothetical reasoning

3.3.1 Entailment between Horn clauses

Let q, r, s, t, u be propositions (type o) and consider the program [MN p. 78]

```
s :- r, q.
t :- q, u.
q :- r.
```

It has no atomic consequences at all, yet clauses follow from it: `t :- r, u.` and `s :- r.` [MN p. 79]. In *fohh* we can ask this directly. The query `?- s :- r.` "is better written as" `?- r => s.` [MN p. 79]: AUGMENT adds `r` and ordinary Horn reasoning proves `s`.

Tutorial: the O-proof of `r => s`

Write $\mathscr{P}_0$ for the three clauses and $\mathscr{P}' = \mathscr{P}_0, r$; the signature is omitted. The search, read goal-first (each line is reduced to the next ones):

1. $\mathscr{P}_0 \longrightarrow r \supset s$: by ⊃R (AUGMENT) it suffices to show $\mathscr{P}' \longrightarrow s$.
2. $\mathscr{P}' \longrightarrow s$: decide on the clause $r \wedge q \supset s$; by ⊃L we need the head to match, $\mathscr{P}' \xrightarrow{s} s$ (initial), and the body, $\mathscr{P}' \longrightarrow r \wedge q$.
3. $\mathscr{P}' \longrightarrow r \wedge q$: by ∧R, both $\mathscr{P}' \longrightarrow r$ and $\mathscr{P}' \longrightarrow q$.
4. $\mathscr{P}' \longrightarrow r$: decide on the assumed clause r, then initial.
5. $\mathscr{P}' \longrightarrow q$: decide on $r \supset q$; initial for the head, and the body $\mathscr{P}' \longrightarrow r$ as in step 4.

The only new step is the first one; the rest is the Horn backchaining of Chapter 2, run in a larger program. In the same way `(r => u) => (r => t)` is provable [MN p. 79]: it

assumes the clauses `r => u` and `r`, and then `t` follows through `q` and `u`. (This is a different entailment from `t :- r, u`, which as a query is `r => u => t`.)

The last example of the section is a warning [MN p. 79]:

```
?- (q :- (q => q)) => (q :- (q => q)).
```

It has the form $B \supset B$, so it has an O-proof from the empty program, but a depth-first interpreter "will not terminate on the given query". To see why, unfold it. AUGMENT assumes $(q \supset q) \supset q$; the goal is again $(q \supset q) \supset q$, so AUGMENT assumes $q \supset q$, i.e. the clause `q :- q`, *at the front*. Now the program is `[q :- q, q :- (q => q)]` and the goal is `q`. Depth-first search picks `q :- q` and loops for ever. The O-proof picks the other clause: its body `q => q` assumes `q` and proves `q` at once.

Pitfall: provable, yet the interpreter loops

Depth-first backchaining is incomplete, and AUGMENT makes this easier to hit, because assumed clauses go to the front. The book makes the same point with conjunction: `loop, fail` loops while `fail, loop` fails, with `loop :- loop.` [MN p. 88]. In the interleaving search of miniKanren the clause choice is fair, and the interpreter of Section 3.8.3 does find the proof of $B \supset B$, in fact infinitely many of them (`run 3` returns `(α α α)`); conjunction, however, remains as order-sensitive as in λProlog.

3.3.2 Hypothetical reasoning on a database

Implications in goals express *hypothetical* questions: "what if this were true?". Figure 3.2 is the book's course database: facts about which students finished which courses, rules for being a CS major and for graduating, and a constraint saying it is inconsistent to have finished both 210 and 250.

Note the $D \wedge D$ clauses on the two fact lines, and that `false` is an ordinary proposition: it has a clause, so it can be proved, but it has no logical meaning (Section 3.6.2). The queries and answers of the book [MN p. 80–81]:

```
?- fact (finished dana X) => false.
X = 210;
no
?- fact (finished X Y) => (fact (graduates X), fact (cs_major X)).
X = dana
Y = 301;
no
?- fact (finished kim X) => fact (finished kim Y) =>
                (fact (graduates kim), fact (cs_major kim)).
X = 250
Y = 301
?- fact (finished kim 250) => fact (finished kim 301) => false.
solved
```

```
kind entry               type.
type fact                entry -> o.
type false               o.
kind person              type.
type kim, dana           person.
type finished            person -> int -> entry.
type cs_major, graduates person -> entry.

fact (finished kim  102) & fact (finished dana 101).
fact (finished kim  210) & fact (finished dana 250).

fact (cs_major X)  :-
     (fact (finished X 101); fact (finished X 102)),
     fact (finished X 250), fact (finished X 301).
fact (graduates X) :-
     (fact (finished X 101); fact (finished X 102)),
     (fact (finished X 210); fact (finished X 250)),
     fact (finished X 301).

false :- fact (finished X 210), fact (finished X 250).
```

Figure 3.2: An encoding of a small course-related database (book Figure 3.1, printed p. 80).

The first asks whether some course would make the database inconsistent if Dana finished it (210, and only 210). The second asks: "If person X took just one more course Y, will he or she graduate with a CS degree?" (only Dana, with 301); this "indirectly implies that there is no single course" Kim can take [MN p. 80], hence the third query, with two hypotheses: Kim needs 250 and 301. The fourth shows that this plan is inconsistent, so "there is no consistent way for Kim to graduate as a computer science major" [MN p. 81].

Tutorial: a logic variable inside an assumed clause

Follow the first query. Its free variable X is existential, so ∃R makes it a logic variable *before* AUGMENT runs. The assumed clause is therefore `fact (finished dana X)` with X *not* universally quantified: it is one specific, still unknown fact. Now the goal `false` backchains on the constraint clause, whose own variable (call it Z) is renamed by ∀L:

1. `fact (finished Z 210)`: decide on the assumed clause unifies Z = dana, $X = 210$;
2. `fact (finished dana 250)`: true in the database.

So proving `false` *binds the variable inside the hypothesis*, and the binding is reported to the query. In the second query the single hypothesis `fact (finished X Y)` likewise has one binding for X and Y, shared by every use of the clause. Section 3.5 turns this into a programming technique.

```
type db                           o.
kind command                      type.
type do                           command -> o.
type enter, query, whatif, check  entry -> command.
type quit, consis                 command.

db :- print "Command?", read Command, do Command.

do quit.
do (enter Fact) :- fact Fact => db.
do (query Q) :- (fact Q, !, print "yes\n"; print "no\n"), db.
do (whatif Conjecture) :- (fact Conjecture => db),
                          print "Resuming\n", db.
do consis :- false, print "no\n", !; print "yes\n".
do (check Entry) :- (fact Entry, print "yes\n", !;
                     fact Entry => false, print "no\n", !;
                     print "no, but it could be true\n"),  db.
```

Figure 3.3: A program that allows hypothetical queries against an encoded database (book Figure 3.2, printed p. 81).

Kanren angle: the four queries on (aux kanren micro)

The interpreter of Section 3.8.3 reproduces the four answers: `(210)` for Dana's inconsistency, `((dana 301))` for the one-course question, a success for the inconsistency of Kim's plan, and, asked for *all* answers, `(250 301)` and the symmetric `(301 250)`, each twice, for the two-course question: the book shows only the first answer, and the others correspond to different proofs: the symmetric pair because either hypothesis can supply either course, the duplicates because the "210 or 250" disjunct in the body of `graduates` can be proved both from the database (210) and from the hypothesis that supplies 250. As for what this demands of the engine: the program must be part of what a goal sees, it must be extended for the duration of a subgoal only, and an assumed clause must be able to *mention logic variables of the surrounding computation without renaming them* (Section 3.8.1).

3.3.3 An interactive "what if" database

Figure 3.3 turns hypothetical reasoning into a small interactive program; it uses the non-logical `print`, `read` and cut `!` [MN p. 81].

The book's session [MN p. 81–82] is reproduced in Fig. 3.4.

The interesting point is where the "database state" lives: *in the stack of nested* AUGMENTS. `enter F` runs the rest of the session inside `fact F => db`, so F persists; `whatif F` runs a *sub-session* inside `fact F => db`, and when `quit` ends that sub-session the clause continues with `print "Resuming\n", db` in the outer program, where F is gone. That is why Dana stops graduating

```
?- db.
Command? query (graduates dana).
no
Command? whatif (finished dana 301).
Command? query (graduates dana).
yes
Command? query (cs_major dana).
yes
Command? quit.
Resuming.
```

```
Command? query (finished kim 101).
no
Command? query (graduates kim).
no
Command? whatif (finished kim 301).
Command? query (graduates kim).
yes
Command? query (cs_major kim).
no
Command? whatif (finished kim 250).
Command? query (cs_major kim).
yes
Command? consis.
no
Command?
```

Figure 3.4: The book's session with the program of Fig. 3.3 (printed pp. 81–82); read the left column first.

after `Resuming`. The Kim hypotheses nest: the innermost session sees both 301 and 250, so `cs_major kim` holds and `consis` reports an inconsistency (210 and 250 together prove `false`). No `assert`/`retract` is involved: undoing a hypothesis is just leaving a scope.

Remark 3.7. The transcript is slightly idealised: `do consis` is the only command besides `quit` whose clause does not end with `db`, so after `consis` the current sub-session ends and the enclosing `whatif` should print `Resuming` before the next prompt.

3.4 Universal goals and scoped constants

GENERIC transforms the goal $\forall_\tau x\, G(x)$ in the context $\langle \Sigma, \mathscr{P} \rangle$ into $G[c/x]$ in the context $\langle \Sigma \cup \{c : \tau\}, \mathscr{P} \rangle$ for a token c not in Σ; which token is chosen does not matter [MN p. 82].

Definition 3.8 (eigenvariable, scoped constant [MN p. 82])**.** The new token c introduced by GENERIC is an *eigenvariable*. Since it is never instantiated during computation, the book also calls it a *scoped constant*.

The parallel with implication is exact: implication augments the *program* for part of the computation, $\forall$ augments the *signature* [MN p. 82].

Tutorial: sterile jars [MN p. 83]

A jar is sterile if every bug in it is dead; a bug in a heated jar is dead; jar `j` has been heated:

```
kind jar, bug        type.
type j               jar.
type sterile, heated jar -> o.
type dead, bug       bug -> o.
type in              bug -> jar -> o.
```

```
sterile J :- pi x\ bug x => in x J => dead x.
dead B    :- heated J, in B J, bug B.
heated j.
```

The signature has *no constants of type* bug: the program assumes nothing about the existence of bugs. Now solve `sterile j`.

1. Backchain on the first clause: the goal becomes `pi x\ bug x => in x j => dead x`.
2. GENERIC: pick a fresh `g`; the goal becomes `bug g => in g j => dead g`, and `g : bug` joins the signature.
3. AUGMENT twice: `bug g` and `in g j` join the program; the goal is `dead g`.
4. Backchain on the second clause: `heated J` succeeds with $J = \mathrm{j}$, then `in g j` and `bug g` succeed by the two assumptions.

"After this goal succeeds, the constant `g` is removed from the signature, and the two clauses `bug g` and `in g j` are similarly removed from the program" [MN p. 83]. The proof talks about an arbitrary bug, never about a particular one.

3.4.1 An intensional universal quantifier

The jar proof illustrates that *fohh*'s ∀ is *intensional*: "Proofs of universal goals do not make any assumptions about the structure of the domain of quantification, and in fact, the same generic proof must work for every element of the domain" [MN p. 83]. The alternative, *extensional*, reading would prove every instance, possibly in different ways, typically by induction; but "the fohh logic does not encompass rules for induction" [MN p. 83–84]. With

```
kind nat   type.
type zero  nat.
type succ  nat -> nat.
type plus  nat -> nat -> nat -> o.

plus zero L L.
plus (succ N) M (succ P) :- plus N M P.
```

the query `?- pi N\ plus N zero N.` is *not* provable, although every closed instance is: the generic goal `plus c zero c` matches no clause head, since `c` is neither `zero` nor a `succ`. The base and step cases of the obvious induction are provable,

```
?- plus zero zero zero,
   pi N\ plus N zero N =>
         plus (succ N) zero (succ N).
```

but no rule of *fohh* combines them into `pi N\ plus N zero N` [MN p. 84].

Pitfall: ∀ is not "for every value"

Do not expect `pi` to enumerate a domain, and do not confuse it with the existential query `plus N zero N`, which enumerates `zero`, `succ zero`, … for ever. An empty type is harmless for ∀: the jar proof works although no bug exists. The bibliographic notes call the two readings *generic* and *universal* judgments; they coincide under the open-world assumption of *fohh*, and under a closed world Miller and Tiu's ∇ quantifier captures the generic one [MN p. 94], [38].
The dual remark concerns ∃. The typing premise $\Sigma; \emptyset \vdash_f t : \tau$ of ∃R is, with logic variables, *delayed*: "all the remaining, delayed typing judgments must be provable for instances of the logic variables" [MN p. 61]. A query that ends with an unbound logic variable of an empty type (say a `sigma` over `bug` in the jar signature, with a body that does not constrain it) has no O-proof, because no closed term of type `bug` exists, although an interpreter reports success. Inside a GENERIC scope over `bug` the type is inhabited by the scoped constant, which is why the jar proof is fine. An untyped miniKanren silently assumes every unbound answer variable has an instance.

3.4.2 Substitution must avoid capture

In *fohc* a clause is $\forall x_1 \ldots \forall x_m (A_1 \wedge \cdots \wedge A_n \supset A_0)$ with all quantifiers outermost, so instantiating them is plain replacement [MN p. 84]. In *fohh* quantifiers occur inside clauses, and plain replacement can capture.

Definition 3.9 (free for [MN p. 85])**.** Let x be a free variable of B and t a term of the same type. t is *free for* x in B if no free occurrence of x in B is in the scope of a quantifier that binds a variable free in t.

If t is free for x in B, replacement realises $B[t/x]$ soundly; otherwise one first renames bound variables of B, and "we shall assume that substitution is always performed in this logically sound fashion" [MN p. 85].

Example 3.10 ([MN p. 85])**.** Substituting `(f y)` for `X` in `p X :- pi y\ q X y` by naive replacement gives `p (f y) :- pi y\ q (f y) y`, where the constant `y` has been captured: there are now two bound occurrences where there was one. The correct result is `p (f y) :- pi z\ q (f y) z`.

3.4.3 Quantifier order matters

"Possibly the simplest example illustrating the relevance of quantifier scopes in computation" is [MN p. 85]

```
?- sigma x\ pi y\ x = y.
```

It is not provable: `pi y\ t = y` has no proof for any term t in which y does not occur. It is *true* in a model with a single element, but not every model is like that [MN p. 85–86]. Proof search agrees: GENERIC creates a new constant c and must prove $c = t$, which fails. Choosing y itself for x does not help, because capture-avoiding substitution turns the goal into `pi z\ y =`

`z` [MN p. 86]. Swapping the quantifiers changes everything: `pi y\ sigma x\ x = y` *is* provable, with witness the scoped constant itself.

Tutorial: the scope condition, derived from the rules

Where does the asymmetry come from in the rules of Fig. 3.1? From the side condition of ∃R: the witness t must satisfy $\Sigma; \emptyset \vdash_f t : \tau$, where Σ is the signature *at the moment the existential is reduced.*

- In $\exists x\, \forall y\, (x = y)$, ∃R is applied first, with Σ not containing the constant c that GENERIC will create later. No Σ-term equals c.
- In $\forall y\, \exists x\, (x = y)$, GENERIC runs first, the signature becomes c, Σ, and $t = c$ is a legal witness.

With logic variables the witness is chosen *later*, by unification. The side condition does not go away; it becomes a constraint on unification: *a logic variable created when the signature was* Σ *may only be bound to terms built from* Σ (and from other variables bound under the same restriction). This is the scope condition behind the remark that unification "must be modified …so as to ensure that quantifier scopes are respected" when *fohh* computations alternate essential existential and universal quantifiers [MN p. 94].

The condition has a subtle consequence when a variable is bound to a term that contains *other*, younger variables. Nadathur, Jayaraman and Kwon's implementation of ∀ goals tags every constant and logic variable with a *universe index*: GENERIC increments the index and tags the new constant with it, a logic variable is tagged with the index current at its creation, and X (tag i) may be bound to t only if (1) the occurs check passes, (2) t contains no constant with tag $> i$, and (3) before binding, the tags of the variables of t that are $> i$ are lowered to i [43]. Rule (3) is the subtle one. Their example: with the program clause $\forall z\, (q(z) \supset p(d(z)))$, consider the goal $\exists x\, \forall y\, (q(y) \supset p(x))$.

1. ∃: a logic variable X with tag 1. ∀: a constant c with tag 2. AUGMENT: assume $q(c)$; the goal is $p(X)$.
2. Backchain on the clause: ∀L creates Z with the current tag 2; unify $p(X) = p(d(Z))$, binding $X := d(Z)$, which is allowed: d belongs to the original signature (tag 1).
3. The body $q(Z)$ must now be proved from the assumption $q(c)$, i.e. $Z := c$. If Z still had tag 2 this would succeed, and X would be $d(c)$, a term mentioning a constant that did not exist when X was created. Rule (3) lowered Z to tag 1 when X was bound, so $Z := c$ fails, as it should.

The goal is not provable, since x must be fixed before y, and only the lowering detects it.

Kanren angle: what ∀ demands of the unifier

A miniKanren that wants *fohh*'s ∀ needs two things that (aux kanren micro) does not have [30, 21, 43]:

1. *A new kind of term*, the scoped (eigen) constant: unique, never bound, equal only to itself. It must be distinct from every existing term, including records with equal slots, since (aux kanren micro) unifies records slot-wise.

2. *A scope check*: no variable that existed before the constant was created may ever reach it through the substitution, including through younger variables (rule (3) above). Known designs: an explicit quantifier prefix consulted by unification, as in λKanren 2020 [30]; numeric universe levels with lowering, as in Teyjus and ELPI [43, 9]; an absento-like "eigen" constraint between the constants and the list of variables in scope, as in miniKanren with symbolic constraints [21]; or a check of the answers when the scope is left, as the interpreter of Section 3.8.3 does.

Which of these fits (aux kanren micro), and how variable creation numbers relate to levels, is a design question taken up in Chapter 11. Note also that αKanren's names do *not* play this role: a nom "only unifies with itself or with an unassociated variable" [4], so nothing prevents an older variable from being bound to it. Section 3.8.2 runs three tempting shortcuts and shows each one proving $\exists x \, \forall y \, (x = y)$.

3.5 Quantification links goals and clauses

In Prolog the program is fixed, so logic variables only ever occur in goals. In λProlog an implication goal can assume a clause that mentions a logic variable created by an enclosing `∃` or by the head of the clause being used; logic variables then "may appear eventually in program clauses" [MN p. 86]. We met this in the Dana query. Here it becomes a technique.

Figure 3.5 defines `reverse` through an accumulator predicate `rev` whose clauses exist only while a `reverse` goal is being solved.

With the first version, `?- reverse (1::2::nil) P.` reduces to `?- rev (1::2::nil) P nil.` in the program augmented with

```
rev nil L L.
rev (X::L) K M :- rev L K (X::M).
```

and eventually binds `P` to `(2::1::nil)` [MN p. 86–87]. The explicit `pi`s "cannot be dropped: Doing so will change the scope of the quantification and hence also the meaning" [MN p. 86]: without `pi L\` the clause `rev nil L L` would mention the *head's* `L`, the list being reversed, instead of being a general fact.

The second version starts from a different observation [MN p. 87]. From the program

```
rv nil (c::b::a::nil).
rv (X::N) M :- rv N (X::M).
```

one can prove `rv (a::b::c::nil) nil`; replacing `(c::b::a::nil)` by any list `L`, the goal `rv K nil` is provable if and only if `L` and `K` are reverses of each other. This cannot be written in *fohc*, "because to do so we require the ability to 'tie' the binding for a variable in the specification

```
type reverse    list A -> list A -> o.
type rev        list A -> list A -> list A -> o.

reverse L K :-
   ((pi L\ rev nil L L) &
    (pi X\ pi L\ pi K\ pi M\ rev (X::L) K M :- rev L K (X::M)))
       => rev L K nil.
```

```
type reverse, rev    list A -> list A -> o.

reverse L K :-
   (rev nil K &
    (pi X\ pi L\ pi K\ rev (X::L) K :- rev L (X::K)))
       => rev L nil.
```

Figure 3.5: Two implementations of reverse with locally assumed clauses: with a three-place auxiliary predicate (top, book Figure 3.3) and with a two-place one (bottom, book Figure 3.4), printed p. 87.

of the reverse predicate to one that appears in a goal at a particular point in the computation" [MN p. 87]. In *fohh* it is the second program of Fig. 3.5.

Tutorial: running the two-place reverse

Solve `?- reverse (1::2::nil) P.` with the second program.

1. Backchain on the `reverse` clause: ∀L renames its `L`, `K` to fresh logic variables; unification binds the first to `1::2::nil` and identifies the second with `P`.
2. AUGMENT adds, at the front,

   ```
   rev nil K.
   rev (X::L) K :- rev L (X::K).
   ```

   Here the `K` of the first clause is the logic variable shared with `P`: "not like the usual implicitly universally quantified variables" [MN p. 88]. The `K` of the second clause is bound by its own `pi K\` and is renamed at every use.
3. The goal `rev (1::2::nil) nil` becomes `rev (2::nil) (1::nil)`, then `rev nil (2::1::nil)`.
4. This last goal matches `rev nil K` by binding the shared `K`, "and hence `P`", to `2::1::nil` [MN p. 88].

The base case of the recursion was written *before* the recursion ran, with a hole for the answer; the computation fills the hole from inside the scope.

Pitfall: implicit quantification lies inside assumed clauses

The display convention "actually becomes somewhat misleading when such variables can appear in clauses" [MN p. 88]: in `rev nil K` the capital `K` looks universally quantified but is a single shared unknown. Two practical consequences. First, one may not copy an assumed clause wholesale at each use (the `copy_term` reflex of Prolog meta-interpreters): only the variables bound by the clause's own quantifiers are renamed, the others stay shared [43]. Second, `rev` is an ordinary global predicate name, so the local clauses mix with any other assumption about `rev`. Under a stray assumption `rev nil Z`, the interpreter of Section 3.8.3 returns two answers to the reverse query, the second one produced by the stray clause:

```
under rev () Z       : (((2 1) α) (α (2 1)))
```

Making `rev` truly local requires quantifying over the predicate itself, a higher-order feature (Chapter 5).

Kanren angle: shared variables come for free with closures

In a miniKanren embedded in Scheme, an assumed clause $\forall \bar{x}\,(G \supset p(\bar{t}))$ can be represented by a host closure that, when called with the arguments of a goal, runs `fresh°` for $\bar{x}$, unifies the arguments with $\bar{t}$ and then runs G. Every use renames $\bar{x}$, as ∀L requires, while every logic variable that is merely *free* in the clause, like `K` above or Dana's `X`, is captured by the closure and therefore shared, exactly the semantics of this section, with no copying machinery. What closures do not provide is the scope information of Section 3.4.3 for the renamed $\bar{x}$: in the universe-index scheme each such variable is tagged with the index current at the time of use [43], and that index must come from the search state. See Section 12.4.

3.6 The logic behind fohh

Logic is useful for programs because logically equivalent formulas, $B \equiv C$, ought to be interchangeable. With side effects or incomplete search they are not: `print "yes", print "no"` differs from its swap, and `loop, fail` from `fail, loop`. "Nevertheless ...if we rule out programs with side effects and consider complete proof-search strategies, then logical equivalences can be used meaningfully in reasoning about logic programs" [MN p. 88]. The question is then: *which* logic?

3.6.1 Not classical logic

For *fohc* the rules are complete for both classical and intuitionistic logic (Chapter 2). Not so for *fohh*.

Theorem 3.11 ([MN p. 89])**.** *The operational semantics for sequents in fohh is sound and complete for intuitionistic logic, but classical logic is not sound for this operational semantics.*

Take $p \vee (p \supset q)$ from the empty program. By $\vee$R it is provable iff p is provable from $\emptyset$ (no) or q is provable from $\{p\}$ (no). So there is no O-proof, in agreement with intuitionistic logic. Classically, however [MN p. 89],

$$B_1 \vee (B_2 \supset B_3) \;\equiv\; B_1 \vee \neg B_2 \vee B_3 \;\equiv\; (B_2 \supset B_1) \vee B_3 \;\equiv\; B_2 \supset (B_1 \vee B_3),$$

so $p \vee (p \supset q) \equiv p \supset (p \vee q)$, which is clearly provable. Classical logic lets the scope of an implication be "extruded" over a disjunction (the term comes from the π-calculus, [MN p. 95]), and so "the interpretation we desire for implication as a scoping mechanism is not valid in classical logic" [MN p. 89]. Three more classically valid goals without O-proofs [MN p. 89]:

$$(r\,a \wedge r\,b \supset q) \supset \exists x\,(r\,x \supset q) \qquad ((p \supset q) \supset p) \supset p \qquad \exists x\,\forall y\,(p\,x \supset p\,y)$$

The middle one is Peirce's formula. "It is clear from all these examples that classical logic cannot provide the declarative semantics of λProlog" [MN p. 89].

Tutorial: why the three formulas fail

First. AUGMENT assumes the clause `q :- r a, r b`; ∃R makes X; AUGMENT assumes $r\,X$; the goal q backchains and needs $r\,a$ and $r\,b$. The only r-fact is $r\,X$: the first subgoal binds $X := a$ and then $r\,b$ fails. Classically one could argue "by cases on x"; a single O-proof must commit to one witness.
Peirce. AUGMENT assumes $(p \supset q) \supset p$, i.e. `p :- (p => q)`, goal p. Backchaining gives the goal $p \supset q$: assume p, prove q. Nothing proves q.
Drinker. (The third formula is a form of the classical "drinker paradox"; the name is not the book's.) ∃R makes X, GENERIC makes c, AUGMENT assumes $p\,X$, goal $p\,c$. The only way is to unify $p\,X = p\,c$, i.e. $X := c$, which the scope condition forbids. This is exactly the failure of $\exists x\,\forall y\,(x = y)$, and indeed an engine without the scope check "proves" it (Section 3.8.3).

3.6.2 Minimal and intuitionistic negation

Define $\neg B$ as $B \supset \bot$. *Minimal logic* treats $\bot$ as essentially a non-logical constant; *intuitionistic logic* adds *ex falso quodlibet* [MN p. 89–90]:

$$\frac{\Sigma;\mathscr{P} \longrightarrow \bot}{\Sigma;\mathscr{P} \longrightarrow B}\;\bot\text{R}$$

A program is *inconsistent* if it entails $\bot$; intuitionistically, it then entails everything [MN p. 90]. In λProlog one picks a proposition, say `false`, for $\bot$; since it is non-logical, ex falso is unavailable, and the negation obtained is *minimal logic negation* [MN p. 90]. That is what the database constraint of Fig. 3.2 uses. The interpreter can be extended to intuitionistic negation: "before failing on a given query, the interpreter would try to determine if `false` is provable. If it is, then it would have to succeed on the query in question. Such a check for inconsistency would have to be coordinated with each (sub)query that leads to the program being augmented" [MN p. 90].

Example 3.12 (where minimal and intuitionistic negation part ways)**.** Consider the goal `p => (p => false) => q`, i.e. $p \supset \neg p \supset q$. AUGMENT twice puts `p` and the clause `false :- p` in the program; the goal `q` matches no clause head, so there is no O-proof, and the formula is not provable in minimal logic either. If `false` is read as $\bot$, it is intuitionistically provable: `false` follows by backchaining on `false :- p`, and ex falso gives `q`. The extended interpreter described above finds this proof: before failing on `q` it tries `false`, in the augmented program, and succeeds. The check must be made in the augmented program, which is why it has to be "coordinated" with AUGMENT: the empty program is consistent, the program $\{p,\ \neg p\}$ is not. (The interpreter of Section 3.8.3, which has no ex falso, returns `()` for this goal.)

Minimal negation is weak but not useless. These are provable [MN p. 90]:

```
(p => q) => ((q => false) => (p => false)).
p => ((p => false) => false).
```

Their converses are provable neither minimally nor intuitionistically, but are classically. The instance of excluded middle `?- p; (p => false).` is not provable; its double negation `?- ((p; (p => false)) => false) => false.` is.

Tutorial: a proof that uses a hypothesis twice [MN p. 91]

In the display below, the formulas assumed so far are written to the left of `?-`:

```
(p; (p => false)) => false ?- false.
(p; (p => false)) => false ?- p; (p => false).
(p; (p => false)) => false ?- p => false.
p, (p; (p => false)) => false ?- false.
p, (p; (p => false)) => false ?- p; (p => false).
p, (p; (p => false)) => false ?- p.
```

In line 1 AUGMENT has assumed the clause `false :- p; (p => false)`. From line 1 to 2 we backchain on it; from 2 to 3 we choose the right disjunct; from 3 to 4 AUGMENT assumes `p`; from 4 to 5 we backchain on the *same* clause again, now in a larger program; from 5 to 6 we choose the left disjunct, and "This last query succeeds immediately because the goal `p` is also in the program at that point." The pattern, a hypothesis used twice, the second time in a bigger world, is typical of intuitionistic proofs.

Theorem 3.13 ([MN p. 91])**.** *If $\mathscr{P}$ is a fohh program and G a fohh goal over Σ, then $\Sigma; \mathscr{P} \longrightarrow G$ has an O-proof if and only if it is provable in minimal logic.*

Since *fohh* formulas contain no $\bot$ (only the non-logical `false`), minimal and intuitionistic provability coincide on them, "for this reason, we shall say that the declarative semantics of λProlog is also given by intuitionistic logic" [MN p. 91].

Kanren angle: `=> false` is not negation as failure

Minimal negation needs nothing from the engine: `false` is an ordinary predicate, and $\neg D$ as a goal is "assume D, prove `false`". It must not be confused with the soft-cut

negation `(if° g ✗° ✓°)` available in (aux kanren micro), which is negation as failure and unsound on non-ground goals, nor with the disequality constraint `≠°`. Under the open-world reading of *fohh* the book notes that $A \supset \bot$ for atomic A "can never be true because there is always some future world in which A is true" [MN p. 94]. Classical principles (excluded middle, Peirce, scope extrusion) must *not* be built into a *fohh* engine: they are unsound for its operational semantics.

3.6.3 Why "hereditary Harrop"

Harrop showed that if all assumptions (the formulas left of the sequent arrow) are of the form $H ::= \top \mid A \mid H \wedge H \mid B \supset H \mid \forall_\tau x\, H$, with B arbitrary, then intuitionistic logic has the *disjunction property* (a proof of $F_1 \vee F_2$ yields a proof of F_1 or of F_2) and the *existence property* (a proof of $\exists x\, F$ yields a witness t with a proof of $F[t/x]$) [20], [MN p. 93]. Intuitively, these are what goal-directed search relies on: when the goal is a disjunction or an existential, committing to one disjunct or one witness loses nothing. Harrop's condition gives them only at the root of a proof; to have them at every point of goal-directed search, disjunctions and existentials are prohibited "hereditarily at all positive locations" [MN p. 93], whence hereditary Harrop formulas [37]. Semantically, intuitionistic logic is modelled by Kripke models over possible worlds; identifying worlds with signature–program pairs, a *fohh* computation moves to larger worlds (the open-world assumption) [MN p. 94].

3.7 Notable subsets of fohh

Can the grammar be simplified without losing expressiveness? The book gives two restrictions [MN p. 91–92]:

$$G ::= \top \mid A \mid G \wedge G \mid G \vee G \mid \exists x\, G \mid D \supset G \mid \forall x\, G \qquad D ::= A \mid G \supset D \mid D \wedge D \mid \forall x\, D \tag{3.1}$$

$$D ::= A \mid G \supset A \mid D \wedge D \mid \forall x\, D \tag{3.2}$$

$$D ::= A \mid D \supset D \mid D \wedge D \mid \forall x\, D \tag{3.3}$$

Atomic heads, (3.2). Clauses of (3.1) can be preprocessed, "without changing their essential operational interpretation", into clauses whose top-level implications have atomic conclusions [MN p. 91]; declaratively, every (3.1) clause is intuitionistically (and minimally) equivalent to a *set* of (3.2) clauses [MN p. 92]. The equivalences doing the work are those of *fohc* normalisation (Chapter 2, [MN p. 64]):

$$\forall x(B_1 \wedge B_2) \equiv \forall x B_1 \wedge \forall x B_2, \qquad B_1 \supset (B_2 \supset B_3) \equiv (B_1 \wedge B_2) \supset B_3,$$
$$B_1 \supset (B_2 \wedge B_3) \equiv (B_1 \supset B_2) \wedge (B_1 \supset B_3),$$

and $B_1 \supset \forall x\, B_2 \equiv \forall x(B_1 \supset B_2)$ when x is not free in B_1.

Example 3.14 (normalising a clause to atomic heads)**.** The (3.1) clause $\forall x\,(g\,x \supset (p\,x \wedge (h\,x \supset q\,x)))$ has a conjunction and an implication in the conclusion of its top-level implication. Distributing $\supset$ over $\wedge$, then $\forall$ over $\wedge$, and finally currying the inner implication gives the two (3.2) clauses

$$\forall x\,(g\,x \supset p\,x) \qquad \text{and} \qquad \forall x\,((g\,x \wedge h\,x) \supset q\,x),$$

that is, `p X :- g X.` and `q X :- g X, h X.` Note that the body $g\,x$ now occurs twice. The book warns, for *fohc*, that this kind of preprocessing can make programs exponentially larger (from $G \supset (D_1 \wedge D_2)$ to $(G \supset D_1) \wedge (G \supset D_2)$ the goal G is duplicated) and need not preserve operational behaviour such as the number of derivations [MN p. 64–65]. Which of $g\,x$ and $h\,x$ is tried first in the second clause is an operational choice that the logic does not fix.

No $\vee$, no $\exists$, (3.3). Dropping disjunction and existentials from goals gives $G ::= A \mid G \wedge G \mid D \supset G \mid \forall_\tau x\,G$ with D as in (3.2); "now the G- and D-formulas are identical" (up to the normalisation above), so one grammar (3.3) serves for both: atoms closed under *unrestricted* $\supset$, $\wedge$ and $\forall$ [MN p. 92].

Pitfall: equivalent as programs, not as formulas

The passage from (3.1)/(3.2) to (3.3) is *not* a logical equivalence [MN p. 92]. The clause $(p \supset (q \vee r)) \supset s$ is legal in (3.2) but is not intuitionistically equivalent to any set of (3.3) formulas; it implies $((p \supset q) \supset s) \wedge ((p \supset r) \supset s)$, but not conversely (classically the two are equivalent). Yet "as program clauses, however, these two formulas prove exactly the same atomic formulas" [MN p. 92]. The book does not spell out why; one way to see it is operational. To use the clause on the goal s we must prove the goal $p \supset (q \vee r)$, i.e. assume p and then, in the augmented program, commit to q or to r: exactly what the two split clauses test. The intuitionistic difference would only show if $p \supset (q \vee r)$ could be *assumed*, and in *fohh* it never is, because it is not a clause.

Another way to get rid of $\vee$ and $\exists$ is to define them as non-logical predicates with higher-order Horn clauses (Chapter 5).

3.8 What implication and universal goals ask of a miniKanren

We can now say precisely what is missing from (aux kanren micro) (Chapter 10). A (aux kanren micro) goal maps a state (a substitution indexed by variable creation number, plus constraint stores) to a lazy stream of states; relations are Scheme procedures, so *the program is the fixed Scheme environment*: exactly the *fohc* situation this chapter started from.

fohh	(aux kanren micro) today
$\top$, $G \wedge G$, $G \vee G$	`✓°`, `and°`, `or°`/`cond°` (fair interleaving)
$\exists x\, G$ ($\exists$R with a logic variable)	`fresh°`
$\forall x\, D$ in a clause ($\forall$L)	`fresh°` in a relation body
decide and $\wedge$L over the clauses of a predicate initial, by unification	the `cond°` of a `define-relation`
	`=°` against the head
$D \supset G$ (AUGMENT)	missing: the program is not part of the state
$\forall x\, G$ (GENERIC)	missing: no scoped constants, no scope check

3.8.1 The program context belongs to the search state

The sequent $\Sigma; \mathscr{P} \longrightarrow G$ shows the program as an *inherited, read-only* context: no rule returns a modified program to its caller. In a miniKanren this translates into four requirements.

1. *A goal must be able to read the current program* when it reaches an atomic goal, so that assumed clauses are tried together with (and, in λProlog, before) the static ones.
2. *The extension is local to a subgoal.* After G in $D \supset G$ produces an answer, the conjunction that follows must see the old program again. If the program lives in the state that `and°` threads from one conjunct to the next, each answer state of G must have its program restored.
3. *The extension is local to a branch.* Two disjuncts with different hypotheses must not see each other's clauses, even though the interleaving search alternates between them.
4. *Free logic variables of an assumed clause are shared*, not renamed (Section 3.5).

Requirements 2 and 3 are what make a host-language *dynamic binding* (`parameterize`, `fluid-let`, or a global variable set and restored) tempting, and wrong. A (aux kanren micro) goal does almost nothing when applied to a state: `fresh°` and `define-relation` wrap the real work in a promise (`δ`), which is forced later by whoever consumes the stream, after the dynamic extent of the binding has ended. The following file puts a "current program" in a parameter and tries it three ways.

```
(import scheme (chicken base) (only (scheme base) make-parameter parameterize)
        (aux base) (aux stream) (aux kanren micro))

;; The "current program" in a host-language parameter: a list of clauses (λ (x) goal).
(define program (make-parameter '()))

(define (any-clause° cs x)                    ; try every clause, most recent first
  (if (null? cs) ✗° (or° ((car cs) x) (any-clause° (cdr cs) x))))

(define-relation (hyp° x)                     ; a predicate whose clauses live in `program`
  (λ (s) ((any-clause° (program) x) s)))
```

```
(define (assume° clause g)                    ; D ⊃ G by dynamic binding: WRONG
  (λ (s) (parameterize ((program (cons clause (program)))) (g s))))

(define (fact a) (λ (y) (=° y a)))            ; the clause  hyp a.

(define (show x) (display x) (newline))
(show (μkanren-run (q -1 #t) (assume° (fact 'a) (hyp° q))))                  ; 1
(show (μkanren-run (q -1 #t) (parameterize ((program (list (fact 'a)))) (hyp° q)))) ; 2
(show (parameterize ((program (list (fact 'a)))) (μkanren-run (q -1 #t) (hyp° q)))) ; 3
```

```
()
()
(a)
```

Run 1 binds the parameter around the *application* of the goal to the state; run 2 around the *construction* of the goal. Both lose the hypothesis: `(hyp° q)` returns a promise, and the program is read only when the promise is forced, outside `parameterize`. Only run 3, which forces the *entire* search inside the binding, works, and it is useless for scoping, since the "scope" is then the whole query. The parameter-based `assume` in the code accompanying λKanren 2020 loses hypotheses in the same way for relation calls nested in delayed bodies; the paper also describes a state-threaded design without the problem [30]. Hence: *hypotheses must live in the search state* (or be passed as an explicit argument, as in Section 3.8.3), never in host dynamic state; see Chapter 11.

3.8.2 Scoped constants need a scope check

Three shortcuts suggest themselves for $\forall y\, G$ in a miniKanren. Each one "proves" $\exists x\, \forall y\, (x = y)$, which has no proof (Section 3.4.3):

```
(import scheme (chicken base) (aux base) (aux stream) (aux kanren micro))
(define (show x) (display x) (newline))

;; Three wrong encodings of the goal  ∃x ∀y (x = y),  which has no proof.

; (a) ∀ read as ∃: y is just another logic variable
(show (μkanren-run (x -1 #t) (fresh° (y) (=° x y))))

; (b) ∀ read as "a brand new constant", with no scope check
(show (μkanren-run (x -1 #t) (let ((c (gensym 'c))) (=° x c))))

; (c) ∀ read as "a fresh variable that must still be unbound at the end"
(define (naive-∀° f)
  (fresh° (y) (f y) (project° ((v y)) (if (μkanren-var? v) ✓° ✗°))))
(show (μkanren-run (x -1 #t) (naive-∀° (λ (y) (=° x y)))))
(show (μkanren-run (x -1 #t) (naive-∀° (λ (y) (=° x (list 'f y))))))
```

```
(α)
(c117)
(α)
((f α))
```

(a) confuses the quantifiers. (b) gets a rigid constant but lets it escape into the answer: this is the "scope extrusion" that the scope condition forbids. (c) is the subtle one: (aux kanren micro) binds the older variable `x` to the younger `y`, so `y` stays unbound and the check passes; and with `(=° x (list 'f y))` it is `x` that gets bound in *any* miniKanren, so checking `y` can never detect the escape. A rigid constant *plus* a check on the variables older than it is required, as stated in the kanren box of Section 3.4.3.

Pitfall: scoped constants and the constraint stores

(aux kanren micro) also has disequality, type and absento constraints, and the book's language has none of them. Moiseenko remarks that "there is no sound implementation of eigen variables with the support of disequality constraints" [41], and the `eigen` of miniKanren with symbolic constraints is documented as unsupported with constraints other than == [21]. Take $\forall x\,(x \neq \alpha)$, with α outer. Under an extensional reading of $\forall$ [26] it should fail, since x could be α's value; under GENERIC's reading of x as a new name, as in λProlog's `pi x\ not (x = A)`, it holds. A naive eigen constant gives the second answer by accident, without a scope check. The proposal implements both readings consistently, the generic one by default (Definition 14.1, Section 12.5).

3.8.3 A throwaway interpreter of the rules

To check the examples of this chapter on today's (aux kanren micro), and to see the rules of Fig. 3.1 at work, here is a small interpreter of O-proofs. It is *not* a proposal for the new module (Chapters 11 and 12 do that): formulas are data, it re-examines every clause at every atomic goal, and the program is an explicit argument `P`. But it is faithful to the rules, and it is complete with respect to the clause choice thanks to the interleaving `or°`.

```
(import scheme (chicken base) (aux base) (aux stream) (aux kanren micro))

;; A throwaway interpreter of O-proofs over (aux kanren micro): formulas are data,
;; binders are Scheme closures.
;;   G ::= #t | A | (and G ...) | (or G ...) | (some ,(λ (x) G)) | (eq t u)
;;       | (imp D G) | (all ,(λ (x) G))
;;   D ::= A | (if G D) | (both D D) | (all ,(λ (x) D))
(define (tag? t F) (and (pair? F) (eq? (car F) t)))

(define (all-of° gs) (if (null? gs) ✓° (and° (car gs) (μ s ((all-of° (cdr gs)) s)))))
(define (one-of° gs) (if (null? gs) ✗° (or° (car gs) (μ s ((one-of° (cdr gs)) s)))))

(define (solve° P G)                                  ; Σ; P → G
  (cond
    ((eq? G #t)       ✓°)                                          ; ⊤R
    ((tag? 'and G)    (all-of° (map (λ (g) (solve° P g)) (cdr G))))  ; ∧R
    ((tag? 'or G)     (one-of° (map (λ (g) (solve° P g)) (cdr G))))  ; ∨R
    ((tag? 'some G)   (fresh° (x) (solve° P ((cadr G) x))))          ; ∃R, logic variable
    ((tag? 'eq G)     (=° (cadr G) (caddr G)))
    ((tag? 'imp G)    (solve° (cons (cadr G) P) (caddr G)))        ; AUGMENT, at the front
    ((tag? 'all G)    (generic° P (cadr G) #t))                    ; GENERIC
    ((tag? 'all/unchecked G) (generic° P (cadr G) #f))
```

```
    (else            (decide° P P G))))                          ; atomic goal

(define (decide° P Ds A)                              ; decide: some D ∈ P, front first
  (if (null? Ds) ✗° (or° (bc° P (car Ds) A) (μ s ((decide° P (cdr Ds) A) s)))))

(define (bc° P D A)                                   ; Σ; P –D→ A
  (cond
    ((tag? 'if D)   (and° (bc° P (caddr D) A) (μ s ((solve° P (cadr D)) s))))  ; ⊃L
    ((tag? 'both D) (or° (bc° P (cadr D) A) (bc° P (caddr D) A)))            ; ∧L
    ((tag? 'all D)  (fresh° (x) (bc° P ((cadr D) x) A)))                      ; ∀L
    (else           (=° D A))))                                               ; initial

;; GENERIC: a new constant c; an answer of G[c/x] survives only if no variable
;; created before c (index < n) reaches c through the substitution.
(define (mentions? c w)                               ; does the walked term w contain c?
  (or (eq? w c) (and (pair? w) (or (mentions? c (car w)) (mentions? c (cdr w))))))

(define (generic° P body check?)
  (λ (s)
    (let* ((n (μkanren-state-vars-count s))
           (c (gensym 'c))
           (ok? (λ (s*) (let loop ((i 0))
                          (or (= i n)
                              (and (not (mentions? c (μkanren-state-find/value
                               ↪  (make-μkanren-var i) s*)))
                                   (loop (add1 i)))))))
           (answers ((solve° P (body c)) s)))
      (if check? (filter§ ok? answers) answers))))

(define-syntax ask                                    ; (ask n (q) P G): first n answers, -1 = all
  (syntax-rules () ((_ n (q) P G) (μkanren-run (q n #t) (solve° P G)))))
(define (show label x) (display label) (display x) (newline))
```

How the code mirrors the rules: AUGMENT is `(solve° (cons D P) G)`, so the new clause goes to the front, the scope ends with the Scheme call, and each branch carries its own `P` (requirements 2 and 3 of Section 3.8.1 hold by construction). Binders are Scheme closures, so ∀L and ∃R call `fresh°` when the clause or goal is used: a clause's own variables are renamed at each use, while a logic variable merely mentioned in it is shared (requirement 4). GENERIC creates a `gensym` constant ((aux kanren micro) compares symbols with `eq?`) and enforces the scope condition lazily: an answer of $G[c/x]$ is kept only if no variable with creation index below `n`, i.e. older than `c`, reaches `c`; the deep walk also catches escapes through younger variables. It prunes late, which is acceptable for a test bench, not for an engine. `all/unchecked` omits the check.

The examples. The file below begins with `(include "hh.scm")`, the listing above, and runs a selection of the chapter's queries, in the book's order.

```
(include "hh.scm")
;; Section 3.2 of the book: clause order, Horn entailment, B ⊃ B
(show "p 2 => p 3 => p X    : " (ask -1 (x) '((p 1)) `(imp (p 2) (imp (p 3) (p ,x)))))
(show "(p 2 & p 3) => p X   : " (ask -1 (x) '((p 1)) `(imp (both (p 2) (p 3)) (p ,x))))
(define P0 '((if (and r q) s) (if (and q u) t) (if r q)))
```

```
(show "r => s ; t           : " (list (ask -1 (q) P0 '(imp r s)) (ask -1 (q) P0 't)))
(show "B => B, 3 proofs     : " (ask 3 (q) '() '(imp (if (imp q q) q) (imp (if q q) q))))
;; Figure 3.1, the course database
(define (fin x c) `(fact (finished ,x ,c)))
(define DB
  `((both ,(fin 'kim 102) ,(fin 'dana 101))
    (both ,(fin 'kim 210) ,(fin 'dana 250))
    (all ,(λ (X) `(if (and (or ,(fin X 101) ,(fin X 102)) ,(fin X 250) ,(fin X 301))
                      (fact (cs_major ,X)))))
    (all ,(λ (X) `(if (and (or ,(fin X 101) ,(fin X 102)) (or ,(fin X 210) ,(fin X 250))
                           ,(fin X 301))
                      (fact (graduates ,X)))))
    (all ,(λ (X) `(if (and ,(fin X 210) ,(fin X 250)) false)))))
(show "dana X => false      : " (ask -1 (x) DB `(imp ,(fin 'dana x) false)))
(show "X Y => grad, major   : "
      (μkanren-run (q -1 #t)
        (fresh° (x y) (=° q (list x y))
          (solve° DB `(imp ,(fin x y) (and (fact (graduates ,x)) (fact (cs_major ,x))))))))
(show "kim X => kim Y => .. : "
      (μkanren-run (q -1 #t)
        (fresh° (x y) (=° q (list x y))
          (solve° DB `(imp ,(fin 'kim x) (imp ,(fin 'kim y)
                          (and (fact (graduates kim)) (fact (cs_major kim)))))))))
(show "kim 250, 301 => false: " (ask -1 (q) DB `(imp ,(fin 'kim 250) (imp ,(fin 'kim 301)
↪  false))))
;; Section 3.3: jars, quantifier order, levels
(define JAR
  `((all ,(λ (J) `(if (all ,(λ (x) `(imp (bug ,x) (imp (in ,x ,J) (dead ,x)))) (sterile ,J))))
    (all ,(λ (B) `(all ,(λ (J) `(if (and (heated ,J) (in ,B ,J) (bug ,B)) (dead ,B))))))
    (heated j)))
(show "sterile Q            : " (ask -1 (q) JAR `(sterile ,q)))
(show "pi y\\ X = f y        : " (ask -1 (x) '() `(all ,(λ (y) `(eq ,x (f ,y))))))
(show "  ... unchecked      : " (ask -1 (x) '() `(all/unchecked ,(λ (y) `(eq ,x (f ,y))))))
(show "pi y\\ sigma x\\ x = y : " (ask -1 (q) '() `(all ,(λ (y) `(some ,(λ (x) `(eq ,x ,y)))))))
(define PQ `((all ,(λ (z) `(if (q ,z) (p (d ,z)))))))
(show "pi y\\ (q y => p X)   : " (ask -1 (x) PQ `(all ,(λ (y) `(imp (q ,y) (p ,x))))))
(show "  ... unchecked      : " (ask -1 (x) PQ `(all/unchecked ,(λ (y) `(imp (q ,y) (p ,x))))))
;; Figure 3.4: reverse; the base clause captures the outer K
(define REV
  `((all ,(λ (L) `(all ,(λ (K)
      `(if (imp (both (rev () ,K)
                      (all ,(λ (X) `(all ,(λ (L) `(all ,(λ (K)
                          `(if (rev ,L (,X . ,K)) (rev (,X . ,L) ,K)))))))))
                (rev ,L ()))
           (reverse ,L ,K)))))))
(show "reverse (1 2) P      : " (ask -1 (p) REV `(reverse (1 2) ,p)))
(show "reverse P (1 2 3)    : " (ask 1 (p) REV `(reverse ,p (1 2 3))))
(show "under rev () Z       : "
      (μkanren-run (q -1 #t)
        (fresh° (p z) (=° q (list p z)) (solve° REV `(imp (rev () ,z) (reverse (1 2) ,p))))))
;; Section 3.4: Peirce, the drinker, double negation of excluded middle
(show "Peirce               : " (ask -1 (q) '() '(imp (if (imp p q) p) p)))
(show "drinker              : " (ask -1 (q) '() `(some ,(λ (x) `(all ,(λ (y) `(imp (p ,x) (p
↪  ,y)))))))))
```

```
(show "  ... unchecked      : "
      (ask -1 (q) '() `(some ,(λ (x) `(all/unchecked ,(λ (y) `(imp (p ,x) (p ,y))))))))
(show "p ; (p => false)     : " (ask -1 (q) '() '(or p (imp p false))))
(show "not not (p ; not p)  : " (ask 1 (q) '() '(imp (if (or p (imp p false)) false) false)))
(show "p => not p => q      : " (ask -1 (q) '() '(imp p (imp (if p false) q))))
```

```
p 2 => p 3 => p X    : (3 2 1)
(p 2 & p 3) => p X   : (2 1 3)
r => s ; t           : ((α) ())
B => B, 3 proofs     : (α α α)
dana X => false      : (210)
X Y => grad, major   : ((dana 301))
kim X => kim Y => .. : ((301 250) (250 301) (250 301) (301 250))
kim 250, 301 => false: (α)
sterile Q            : (j)
pi y\ X = f y        : ()
  ... unchecked      : ((f c488))
pi y\ sigma x\ x = y : (α)
pi y\ (q y => p X)   : ()
  ... unchecked      : ((d c563))
reverse (1 2) P      : ((2 1))
reverse P (1 2 3)    : ((3 2 1))
under rev () Z       : (((2 1) α) (α (2 1)))
Peirce               : ()
drinker              : ()
  ... unchecked      : (α)
p ; (p => false)     : ()
not not (p ; not p)  : (α)
p => not p => q      : ()
```

A provable closed goal answers `(α)` (a proof, nothing to report); an unprovable one answers `()`. Every line agrees with the book; the Kim query lists all answers, while the book shows only the first. The goal `t` on its own fails, since the Horn program has no atomic consequences, and the last line is the minimal-logic failure of $p \supset \neg p \supset q$ discussed in Section 3.6.2. In `sterile Q` the outer variable is bound to the constant `j`, which is fine: `j` belongs to the signature in which `Q` was created. In the reverse example, `(rev () ,K)` is built with the *outer* `K`, a logic variable, which the closure captures: the base case shares the answer with the query. The three `unchecked` lines are the instructive ones. Without the scope condition a scoped constant leaks into an answer, once directly and once through the clause variable Z of the level-lowering example, and the drinker formula $\exists x\, \forall y\, (p\, x \supset p\, y)$, valid classically but not intuitionistically, becomes "provable". Dropping the scope check does not only produce odd answers: it changes the logic.

3.9 Summary

- *fohh* extends *fohc* with $D \supset G$ and $\forall x\, G$ in goals; the clause grammar is unchanged, and the two grammars swap roles across $\supset$. Hereditary Harrop formulas are those with no positive $\vee$ or $\exists$ (Definition 3.1 and Proposition 3.3).

- AUGMENT proves $D \supset G$ with D added at the front of the program; GENERIC proves $\forall x\, G$ with a new scoped constant in the signature. Both follow a stack discipline (Fig. 3.1).
- Implication gives hypothetical reasoning: entailment between clauses, "what if" queries, and a database whose state is the stack of open hypotheses (Section 3.3).
- $\forall$ is intensional: one generic proof for all elements, no induction, empty types harmless (Section 3.4).
- With logic variables, $\exists\forall$ and $\forall\exists$ differ through a *scope condition*: a variable may only be bound to terms over the signature of its creation, and binding must propagate this restriction to the variables of the bound term (universe levels and lowering, Section 3.4.3).
- Assumed clauses may contain logic variables of the surrounding computation; these are shared, not renamed, which lets a clause "tie" its binding to a goal (reverse, Section 3.5).
- O-proofs are sound and complete for intuitionistic logic and coincide with minimal provability on *fohh*; classical logic is unsound for them (scope extrusion, Peirce, drinker). `false` gives minimal negation, not negation as failure (Theorems 3.11 and 3.13).
- Clauses can be normalised to atomic heads (3.2); removing $\vee$ and $\exists$ (3.3) preserves atomic consequences but not logical equivalence (Section 3.7).
- For a miniKanren: the program must live in the search state (per branch, restored after each answer of the scoped subgoal), never in host dynamic binding, because goals run lazily; $\forall$ needs rigid scoped constants and a scope check in, or around, unification (Section 3.8, and Chapter 11 for the design).

4 Typed λ-terms and unification problems

Chapters 2 and 3 did logic programming over *first-order* terms. Chapter 3 added universal quantifiers and implications to goals (*fohh*), and with them eigenvariables: constants made up during the search that no logic variable created earlier may capture. Chapter 4 of the book [36] keeps the logic and changes the *terms*. First-order terms give way to *simply typed λ-terms*. This chapter studies those terms on their own and leaves the higher-order logic built on them to Chapter 5 [MN p. 96].

The change has three consequences, and each of them touches a different part of a miniKanren:

1. *What equality means.* Two terms are equal when they are λ-convertible (α, β, η), not only when they are identical. This affects the term representation and the unifier.
2. *What a logic variable may be bound to.* A unification problem becomes a *quantified formula*. Its prefix mixes ∀ and ∃, and the order of the quantifiers decides which symbols a variable may use. This affects the state, which has to track scope.
3. *How hard unification is.* There may be several, infinitely many, or no most general unifiers, and the problem is undecidable. This affects the search: unification has to become a stream of answers.

The chapter follows the book's order. Sections 4.1 to 4.3 cover the calculus. Sections 4.4 to 4.7 recast unification as quantified equalities. Section 4.8 fixes the vocabulary that Chapter 7 uses for the unification procedure, and Section 4.9 proves undecidability.

4.1 Syntax and types of λ-terms

Typed λ-terms are built from typed constants and variables with two operations. The first is *application*, written as juxtaposition and associating to the left, exactly as for first-order terms. The second is *abstraction*, which is new: from an expression t in which a variable $x : \alpha$ may occur free, it forms the function $\lambda(x:\alpha)\,t$ [MN p. 97]. In λProlog this is written `(x:A)\ T`. The type annotation can be dropped (`x\ T`), and the type is then inferred.

Parsing. The body of an abstraction extends as far to the right as possible. So application binds tighter than abstraction, and abstraction associates to the right [MN p. 97]:

mathematical	λProlog concrete syntax
$\lambda f\, \lambda x\, (f\, (f\, (f\, x)))$	`f\x\ f (f (f x))`
$\lambda x\, (f\, (g\, \lambda y\, (h\, x\, y))\, x)$	`x\ f (g y\ h x y) x`
$\lambda x\, \lambda y\, x$	`x\y\ x`

A backslash cannot occur inside an identifier, so no spaces are needed around it.

Definition 4.1 (Types and terms)**.** Types are generated from primitive types (sorts such as i, and the type o of propositions), type variables A, type constructors such as list A, and the arrow $\tau_1 \to \tau_2$, which associates to the right. Terms are

$$t \ ::= \ c \mid x \mid (t\ t) \mid \lambda(x:\tau)\,t.$$

(The book introduces the grammar in prose [MN p. 97–99]. This compact form is ours.)

Typing. The judgement $\Sigma;\Gamma \Vdash t : \tau$ (in the book, with a double turnstile) says that t has type τ when Σ gives the types of the constants and Γ those of the variables. It is defined by the rules of Figure 4.1 [MN p. 98]:

$$\frac{c : \sigma \in \Sigma \qquad \tau \lhd \sigma}{\Sigma;\Gamma \Vdash c : \tau} \qquad \frac{x : \tau \in \Gamma}{\Sigma;\Gamma \Vdash x : \tau} \qquad \frac{\Sigma;\Gamma \Vdash g : \tau_1 \to \tau_2 \qquad \Sigma;\Gamma \Vdash t : \tau_1}{\Sigma;\Gamma \Vdash (g\ t) : \tau_2}$$

$$\frac{\Sigma;\Gamma, x : \tau \Vdash t : \sigma}{\Sigma;\Gamma \Vdash \lambda(x:\tau)\,t : \tau \to \sigma}\ (\dagger) \qquad \frac{\Sigma;\Gamma \Vdash B : \tau}{\Sigma;\Gamma \Vdash C : \tau}\ (\ddagger)$$

Proviso (†): x is not declared as a type or kind in Σ or Γ. Proviso (‡): B and C differ only in the names of bound variables. $\tau \lhd \sigma$ means that τ is a substitution instance of σ, so a polymorphic constant is instantiated afresh at every use [MN p. 97]. Compared with the first-order rules of Chapter 2, the restrictions on the types in Σ and Γ are gone: a constant may have any type, including a functional type that ends in o. There is one new rule, for abstraction. And there are *no special rules for the quantifiers*, because the quantifiers have become ordinary constants (see below) [MN p. 97–98].

Definition 4.2 (Well-formed, closed, simply typed [MN p. 98])**.** A term t is a *well-formed λ-term over* Σ if $\Sigma;\Gamma \Vdash t : \tau$ is derivable for some Γ and τ. It is *closed* if this holds with Γ empty, and *open* otherwise. It is *simply typed* if neither its type nor the types of its subterms contain type variables.

When every bound variable carries its annotation, the type of a well-formed term is unique up to renaming of type variables. When annotations are omitted, type inference gives each bound variable a fresh type variable and refines it "in as minimal a way as possible". The type found this way is again unique up to renaming, the principal type [MN p. 98–99]. The unification theory of Chapter 7 is stated for simply typed terms.

Example 4.3 (A typing derivation)**.** The Church numeral `f\x\ f (f x)` has type $(\mathsf{i} \to \mathsf{i}) \to \mathsf{i} \to \mathsf{i}$. Write $\Gamma = f : \mathsf{i} \to \mathsf{i},\ x : \mathsf{i}$:

$$\dfrac{\dfrac{\dfrac{\dfrac{f : \mathsf{i} \to \mathsf{i} \in \Gamma}{\Sigma;\Gamma \Vdash f : \mathsf{i} \to \mathsf{i}} \qquad \dfrac{\Sigma;\Gamma \Vdash f : \mathsf{i} \to \mathsf{i} \qquad \Sigma;\Gamma \Vdash x : \mathsf{i}}{\Sigma;\Gamma \Vdash f\,x : \mathsf{i}}\ \textsc{app}}{\Sigma;\Gamma \Vdash f\,(f\,x) : \mathsf{i}}\ \textsc{app}}{\Sigma;\, f : \mathsf{i} \to \mathsf{i} \Vdash \lambda x\, f\,(f\,x) : \mathsf{i} \to \mathsf{i}}\ (\dagger)}{\Sigma;\emptyset \Vdash \lambda f\, \lambda x\, f\,(f\,x) : (\mathsf{i} \to \mathsf{i}) \to \mathsf{i} \to \mathsf{i}}\ (\dagger)$$

Read the derivation bottom up. Each abstraction rule (†) moves one binder into the context. The application rule APP then checks f against a functional type twice, and the leaves are instances of the variable rule (the two leaves above $f\ x$ are left implicit). Because the derivation ends with an empty context, the term is closed. No constant was used, so this derivation works for every Σ, including one with no constant of type i, a case that matters in Sections 4.3.1 and 4.6.

4.1.1 Formulas are terms of type o

A *formula* is a well-formed term of type o. The connectives are constants such as $\wedge : \mathsf{o} \to \mathsf{o} \to \mathsf{o}$. The quantifiers become constants too [MN p. 99]:

$$\forall : (A \to \mathsf{o}) \to \mathsf{o}, \qquad \exists : (A \to \mathsf{o}) \to \mathsf{o} \qquad \text{(concretely } \texttt{pi} \text{ and } \texttt{sigma}\text{)},$$

and $\forall x\ B$ abbreviates $\forall(\lambda x\ B)$, written $\forall_\tau x\ B$ when the type of x matters. A quantified formula is typed with the ordinary rules. From $\Sigma; \Gamma, x : \sigma \Vdash B : \mathsf{o}$ the abstraction rule gives $\lambda x\ B : \sigma \to \mathsf{o}$, the instance $\forall : (\sigma \to \mathsf{o}) \to \mathsf{o}$ of the constant gives the rest, and so $\forall(\lambda x\ B) : \mathsf{o}$. For example, applying `pi` at type `(list int -> o) -> o` to an abstraction of type `list int -> o` [MN p. 99]:

```
y\ append (1::2::nil) y X.
pi y\ append (1::2::nil) y X.
```

Since signatures are no longer restricted, logical symbols may now appear *inside the arguments of non-logical ones* [MN p. 99–100]:

```
type foreach  (A -> o) -> list A -> o.
foreach (x\ x > 5, x < 9) (3::10::6::8::nil).
```

Here the conjunction `,` sits inside an argument of `foreach`. Higher-order programming (Chapter 5) rests on this possibility. One more fact matters a lot later. *The logic has a single binder, λ-abstraction*: $\forall$, $\exists$ and every object-level binder (Chapter 6) are encoded with it [MN p. 100].

Kanren angle: terms are data, goals are procedures

In (aux kanren micro) a goal is a Scheme procedure from a state to a stream of states. A procedure is opaque to the unifier: it cannot be walked, compared or reified. So a formula on which one wants to *unify*, for example to solve for a predicate variable, has to be represented as *data*, such as a record for $\wedge$ or for `pi` applied to an abstraction, and then interpreted by a meta-level relation. This is the design of Chapters 5 and 9. For `foreach` with a ground Scheme predicate, ordinary higher-order Scheme is enough. Note also that the term language now needs a representation of λ itself, which Scheme closures cannot provide: Sections 4.3 and 4.4 use de Bruijn terms [3] (lists in the first, records in the second), and Chapter 11 discusses the options.

4.2 λ-conversion: the equality of the logic

An abstraction is meant to denote a function, and an application of an abstraction is meant to denote the result of the function. The conversion rules make this intended meaning part of the equality. They rely on substitution, which must not capture variables.

Definition 4.4 (Free for; substitution [MN p. 100])**.** t is *free for* x in s if no free occurrence of x in s lies in the scope of an abstraction that binds a free variable of t. If t is free for x in s and both have the same type, $s[t/x]$ is s with every free occurrence of x replaced by t.

For example, $(f\,x)$ is free for u in $\lambda w\,(g\,u\,w)$, but $(f\,w)$ is not: substituting it would put its free w under the binder λw. Ignoring the proviso is logically unsound, for the same reason as with quantified formulas in Chapter 3.

Definition 4.5 (The rules of λ-conversion [MN p. 100–101])**.** The following rewrites may be applied to any subterm:

rule	redex ⇒ contractum	proviso
α-rewriting	$\lambda x\,s \;\Rightarrow\; \lambda y\,s[y/x]$	y free for x in s, y not free in s
β-contraction	$(\lambda x\,s)\,t \;\Rightarrow\; s[t/x]$	t free for x in s (lifted below)
η-contraction	$\lambda x\,(s\,x) \;\Rightarrow\; s$	x not free in s

The converses are β-expansion and η-expansion. *α-conversion* is the reflexive, symmetric, transitive closure of α-rewriting. *β-reduction* is the reflexive, transitive closure of α-conversion together with β-contraction, and *β-conversion* is its symmetric, transitive closure. η-reduction and η-conversion are defined in the same way. *λ-conversion* is the transitive closure of α-, β- and η-conversion.

λ-conversion is an equivalence and a congruence, and *it is the equality of higher-order logic* [MN p. 101]. Two consequences stand out. First, terms related by λ-conversion cannot be told apart in the logic. Second, *no specification can determine the name of a bound variable*, because that name can always be changed by α-conversion without changing the term. Chapter 6 builds λ-tree syntax on this second fact.

Example 4.6 (Four equal terms [MN p. 101])**.**

```
X\Y\ f (g X) Y
x\ f (g x)
x\y\ f ((u\v\v) (2 + 3) (g x)) y
```

The first two terms are α-convertible. The third is the first after an η-contraction:

$$\lambda y\,(f\,(g\,x)\,y) \;\Rightarrow\; f\,(g\,x).$$

A β-contraction of the fourth gives

```
x\y\ f ((v\v) (g x)) y
```

and a second one gives back the first. Note that `(2 + 3)` disappears entirely, because u does not occur in the body `v\v`.

Pitfall: capital letters are not always logic variables

In `X\Y\ f (g X) Y` the tokens `X` and `Y` are *bound* names, not logic variables. In λProlog only *free* capitalised tokens are logic variables. A Scheme reader of λProlog code has to look at where a name is bound before deciding what it is.

Liberalised substitution. From now on $s[t/x]$ is defined even when t is not free for x in s. First α-rename s to some s' in which t is free for x, then substitute. The result depends on the renaming only up to α, so modulo equality it is well defined. β-contraction then becomes $(\lambda x\, s)\, t \Rightarrow s[t/x]$ with no proviso [MN p. 101]. This is ordinary capture-avoiding substitution.

Tutorial: capture-avoiding β, by hand

Contract $(\lambda u\, \lambda w\, (g\, u\, w))\, (f\, w)$. The argument has a free w. A naive textual replacement would give $\lambda w\, (g\, (f\, w)\, w)$, where the outer w has been *captured* and now refers to the bound variable. This is a different function. Liberalised substitution first renames the binder, $\lambda u\, \lambda w'\, (g\, u\, w')$, and then substitutes:

$$(\lambda u\, \lambda w\, (g\, u\, w))\, (f\, w) \;\Rightarrow\; \lambda w'\, (g\, (f\, w)\, w').$$

With de Bruijn indices (Section 4.3) the renaming becomes index arithmetic: the argument is shifted by one when it goes under a binder.

4.3 Normal forms and the power of β

A *β-redex* is a term $(\lambda x\, s)\, t$. An *η-redex* is a term $\lambda x\, (t\, x)$ with x not free in t. A term with no β-redexes is in *β-normal form*. If it has no η-redexes either, it is in *λ-normal form* (also called βη-normal form) [MN p. 101–102].

Theorem 4.7 (Normal forms [MN p. 102])**.** *Every term of the typed language has a λ-normal form, unique up to α-conversion. It is written* λnorm(*s*)*. It can be computed by contracting β-redexes until none is left and then contracting η-redexes.*

Corollary 4.8 (Decidable equality [MN p. 102])**.** *Two terms of the same type are λ-convertible if and only if their λ-normal forms are α-equivalent.*

Remark 4.9 (Background the book does not spell out). The book states Theorem 4.7 as "well known". In the literature it rests on two classical properties of the simply typed λ-calculus. *Strong normalisation* says that every reduction sequence terminates, which gives existence. The *Church–Rosser property* (confluence) says that diverging reductions can be joined again, which gives uniqueness. The algorithm also relies on a standard fact the book uses without proof: η-contracting a β-normal term creates no new β-redex, so doing η after β is enough.

Pitfall: equality is intensional

Logical equivalences do not affect term equality. With $p, q : \mathsf{i} \to \mathsf{o}$, the term $\lambda x\,(p\,x) \wedge (q\,x)$ is *not* equal to $\lambda x\,(q\,x) \wedge (p\,x)$ [MN p. 102]. Types are inhabited by *expressions*, not by sets. A syntactic unifier gets this right automatically. A unifier that knew about commutativity of $\wedge$ would be wrong.

4.3.1 Computing with β: Church numerals

The rule $(\lambda x\,s)\,t \Rightarrow s[t/x]$ can copy t many times or throw it away. Even when s and t are both normal, the result need not be, because x may sit in head position [MN p. 102]. So normalisation is a model of computation. Let the sort i have *no constants*. Then the closed λ-normal forms of type $(\mathsf{i} \to \mathsf{i}) \to \mathsf{i} \to \mathsf{i}$ are exactly the *Church numerals* $\ulcorner n \urcorner = \lambda f\,\lambda x\,(f^n\,x)$ [MN p. 102]. Arithmetic is then definable [MN p. 103]:

$$succ = \lambda n\,\lambda f\,\lambda x\,f\,(n\,f\,x), \quad add = \lambda n\,\lambda m\,\lambda f\,\lambda x\,n\,f\,(m\,f\,x), \quad mul = \lambda n\,\lambda m\,\lambda f\,\lambda x\,n\,(m\,f)\,x.$$

λProlog prints λ-normal forms, so it can run these functions (footnote: the ambient signature has the sort `i`) [MN p. 103]:

```
?- N = ((n\m\f\x\ n (m f) x) ((f:i -> i)\x\ f (f x)) (f\x\ f (f x))).
N = f\x\ f (f (f (f x)))
?-
```

The book adds that the functions definable this way are "limited to the polynomial ones" [MN p. 103]. (The phrasing is informal. The precise class, a result outside the book due to Schwichtenberg and Statman, is the *extended* polynomials, which also allow a test for zero.)

Blow-up. Define the *size* of a term as its number of applications. Normalising can increase the size enormously. The book's series [MN p. 104] is

```
(g\e\ e)           (e\f\ e (e f)) (f\x\ f (f x)).
(g\e\ g e)         (e\f\ e (e f)) (f\x\ f (f x)).
(g\e\ g (g e))     (e\f\ e (e f)) (f\x\ f (f x)).
(g\e\ g (g (g e))) (e\f\ e (e f)) (f\x\ f (f x)).
```

with `x : i`, `f : i -> i`, `e : (i -> i) -> i -> i` and `g : ((i -> i) -> i -> i) -> (i -> i) -> i -> i`. The `g\e\` parts are Church numerals one type level up. `g\e\ g (g e)` has a *fourth-order* type. The $(n{+}1)$-th term has size $n + 6$, and its normal forms are the numerals 2, 4, 16 and 256 [MN p. 104].

Remark 4.10 (An erratum in the size formula). The book says the $(n{+}1)$-th normal form has size $2^{2^{\cdot^{\cdot^{2}}}}$ with $n + 1$ twos [MN p. 104]. The term `e\f\ e (e f)` squares a numeral, and the outer numeral applies it n times to $\ulcorner 2 \urcorner$. So the normal form is $\ulcorner 2^{2^n} \urcorner$, of size 2^{2^n}: 2, 4, 16, 256, 65536. This matches the book's own "256" for the fourth term. A right-nested tower with four twos is 65536, though, so the printed formula is right only if the tower is read left-nested, $((2^2)^2)^{\cdots}$.

The run below confirms the numbers. The point of the passage is unaffected: sizes can explode (Statman [56] showed that λ-conversion is not elementary recursive, as the book's notes cite [MN p. 116]).

Kanren angle: de Bruijn terms and a β-normaliser in plain Scheme

With de Bruijn indices [3] a bound variable is the number of binders between its occurrence and its binder. α-equivalent terms are then *the same data structure*, and capture-avoidance turns into the index arithmetic of `shift`. The following illustration is plain Scheme, not a proposed API (the design is in Chapter 11). It converts named terms to de Bruijn form and computes β-normal forms (file `norm-core.scm`):

```
;; A tiny β-normaliser: de Bruijn inside, names outside.
;; Input: (λ (x y ...) body), applications (f a b ...), symbols.
(define (->db t env)                    ; named term -> de Bruijn
  (cond
    ((symbol? t) (let ((i (index-of t env))) (if i i t)))
    ((eq? (car t) 'λ)
     (let loop ((xs (cadr t)) (env env))
       (if (null? xs)
           (->db (caddr t) env)
           (list 'lam (loop (cdr xs) (cons (car xs) env))))))
    (else
     (let loop ((f (->db (car t) env)) (as (cdr t)))
       (if (null? as)
           f
           (loop (list 'app f (->db (car as) env)) (cdr as)))))))
(define (index-of x l)
  (let loop ((l l) (i 0))
    (cond ((null? l) #f)
          ((eq? (car l) x) i)
          (else (loop (cdr l) (+ i 1))))))
(define (shift d c t)                   ; add d to the indices >= c
  (cond ((integer? t) (if (>= t c) (+ t d) t))
        ((symbol? t) t)
        ((eq? (car t) 'lam) (list 'lam (shift d (+ c 1) (cadr t))))
        (else (list 'app (shift d c (cadr t)) (shift d c (caddr t))))))
(define (subst t j s)                   ; t[s/j], capture-free
  (cond ((integer? t) (if (= t j) s t))
        ((symbol? t) t)
        ((eq? (car t) 'lam)
         (list 'lam (subst (cadr t) (+ j 1) (shift 1 0 s))))
        (else (list 'app (subst (cadr t) j s) (subst (caddr t) j s)))))
(define (beta body arg) (shift -1 0 (subst body 0 (shift 1 0 arg))))
(define (nf t)                          ; β-normal form
  (cond ((or (integer? t) (symbol? t)) t)
        ((eq? (car t) 'lam) (list 'lam (nf (cadr t))))
        (else (let ((f (nf (cadr t))))
                (if (and (pair? f) (eq? (car f) 'lam))
```

```
                   (nf (beta (cadr f) (caddr t)))
                   (list 'app f (nf (caddr t))))))))
(define (size t)                        ; number of applications
  (cond ((not (pair? t)) 0)
        ((eq? (car t) 'lam) (size (cadr t)))
        (else (+ 1 (size (cadr t)) (size (caddr t))))))
(define (church t)                      ; λf λx f^n x -> n, else #f
  (and (pair? t) (eq? (car t) 'lam)
       (pair? (cadr t)) (eq? (car (cadr t)) 'lam)
       (let loop ((b (cadr (cadr t))) (n 0))
         (cond ((eqv? b 0) n)
               ((and (pair? b) (eq? (car b) 'app) (eqv? (cadr b) 1))
                (loop (caddr b) (+ n 1)))
               (else #f)))))
(define (run t) (church (nf (->db t '()))))
```

Running Church arithmetic and the blow-up series (file `church.scm`):

```
(include "norm-core.scm")
(define two '(λ (f x) (f (f x))))
(define three '(λ (f x) (f (f (f x)))))
(define (show . xs) (display xs) (newline))
(show 'succ-3  (run `((λ (n f x) (f (n f x))) ,three)))
(show 'add-2-3 (run `((λ (n m f x) (n f (m f x))) ,two ,three)))
(show 'mul-2-2 (run `((λ (n m f x) (n (m f) x)) ,two ,two)))
(define (series n)        ; (g\e\ g^n e) (e\f\ e (e f)) (f\x\ f (f x))
  `((λ (g e) ,(let loop ((k n)) (if (= k 0) 'e `(g ,(loop (- k 1))))))
    (λ (e f) (e (e f)))
    ,two))
(do ((n 0 (+ n 1))) ((> n 4))
  (let* ((t (->db (series n) '())) (v (nf t)))
    (show 'n n 'size (size t) 'nf-size (size v) 'numeral (church v))))
```

```
(succ-3 4)
(add-2-3 5)
(mul-2-2 4)
(n 0 size 6 nf-size 2 numeral 2)
(n 1 size 7 nf-size 4 numeral 4)
(n 2 size 8 nf-size 16 numeral 16)
(n 3 size 9 nf-size 256 numeral 256)
(n 4 size 10 nf-size 65536 numeral 65536)
```

The input sizes are $n + 6$, as the book says, and the normal-form sizes are 2^{2^n}, as Remark 4.10 says. For the unifier, the useful lesson is that a de Bruijn λ-term built from Scheme records or lists is *already* handled by (aux kanren micro)'s structural `=°` up to α. What `=°` cannot do is β: binding a logic variable that sits in head position creates a redex, so some walk-and-normalise step has to be added (Definition 12.3).

Why logic programs do not explode. The book gives three reasons [MN p. 104–105]. (1) Integers and data structures are built-in or made from non-logical constructors, not Church-encoded. (2) Bound variables usually have primitive (order-0) types. If every bound variable in a term has a primitive type and s, t are λ-normal, then $s[t/x]$ is λ-normal: it may contain many copies of t, but there are no new redexes. (3) In the sublanguage L_λ (Chapter 6), β is restricted to *β_0-contraction* $(\lambda x\, s)\, x \Rightarrow s$, which only renames, and normalisation then *shrinks* terms.

Remark 4.11 (Order of a type). The book uses "n-th order" without giving a formula. The usual definition, consistent with all its uses, is $\mathrm{ord}(\tau) = 0$ for primitive τ and $\mathrm{ord}(\tau_1 \to \tau_2) = \max(\mathrm{ord}(\tau_1) + 1, \mathrm{ord}(\tau_2))$. Checks: the type $(\mathsf{i} \to \mathsf{i}) \to \mathsf{i} \to \mathsf{i}$ is called second-order [MN p. 102], and the type of `g\e\ g (g e)` fourth-order [MN p. 104]. A unification problem whose variables have order at most 2 is called *third-order* [MN p. 116].

4.4 Unification problems as quantified equalities

Section 1.5 of the book (Chapter 1) defined a unification problem as a multiset $\{t_1 = s_1, \ldots, t_p = s_p\}$ and related it to the formula $\forall y_1 \ldots \forall y_m \exists x_1 \ldots \exists x_n\, [t_1 = s_1 \wedge \cdots \wedge t_p = s_p]$, with the y's the constants and the x's the logic variables [MN p. 105]. Following Miller [34, 35], the book now makes the *formula* the official presentation and allows any quantifier prefix.

Definition 4.12 (Generalised unification problem [MN p. 105])**.** A unification problem is a formula

$$Q_1 x_1 \ldots Q_n x_n\, [\, t_1 = s_1 \wedge \cdots \wedge t_m = s_m \,] \qquad (n, m \geq 0,\ Q_i \in \{\forall, \exists\}),$$

with t_j and s_j λ-terms of the same type. The bound variables x_i may have types of any order.

Why mix the quantifiers? The book gives two reasons [MN p. 106], each with a query that fails.

First, fohh proof search creates mixed prefixes. A `pi` in a goal introduces an eigenvariable *after* the logic variables already present:

```
?- pi y\ X = y.
no
?-
```

The problem is $\exists x\, \forall y\, [x = y]$. It fails because no instance of `X` may contain the eigenvariable that stands for y.

Second, binders in terms behave like inner universal quantifiers.

```
?- (y\ X) = (y\ y).
no
?-
```

The problem is $\exists x\, [(\lambda y\, x) = (\lambda y\, y)]$. The equation $(\lambda y\, t) = (\lambda y\, s)$ is logically equivalent to $\forall y\, [t = s]$, so this is again $\exists x\, \forall y\, [x = y]$, and it fails for the same reason: *no instance of `X` may contain a variable bound by an abstraction inside whose body `X` occurs* [MN p. 106].

Tutorial: reading a quantifier prefix

Every problem in this chapter can be read with one rule.

> *An existential variable may be instantiated only with a term built from symbols quantified to its* ***left*** (together with fresh logic variables when one is content with a unifier rather than a solution, Section 4.6).

Here is where each quantifier of a λProlog problem comes from.

- **Constants of the signature** are outermost universals. They are to the left of everything, so every logic variable may use them.
- **Free variables of the query** (`X`) are existentials.
- **A `pi` in a goal** adds a universal *at that point*, to the right of the logic variables that already exist. Variables created later, for example by backchaining inside the scope of the `pi`, sit to its right and may use it (Chapter 3).
- **A λ inside an equation** is an innermost universal around that equation.

Four readings follow, with $g : \mathsf{i} \to \mathsf{i} \to \mathsf{i}$:

problem	what the ∃ may use	verdict
$\forall y\, \exists x\, [x = y]$	y	solved by $x := y$
$\exists x\, \forall y\, [x = y]$	nothing that mentions y	no solution
$\forall a\, \exists F\, [(F\, a) = (g\, a\, a)]$	g and a	four solutions
$\exists F\, \forall a\, [(F\, a) = (g\, a\, a)]$	g only	one solution

The last two rows are worked out in Section 4.7. The same mechanism explains the second failing query. In `(y\ X) = (y\ y)` the logic variable `X` was created *outside* the binder, so it is to the left of the inner $\forall y$. It may be instantiated with anything except a term that mentions that y.

From ∀∃ to ∀∃∀. In *fohc* the natural class is ∀∃ problems: constants first, then logic variables. With λ-terms the natural class is ∀∃∀ [MN p. 106]. The outer ∀'s are the constants. The inner ∀'s come from the binders of the equations: an equation at type $\tau \to \sigma$ is replaced by $\forall (w : \tau)$ around an equation at type σ, and those quantifiers are moved out over the whole conjunction.

Example 4.13 (Turning λ-equations into a ∀∃∀ prefix [MN p. 106–107])**.** Let $f : \mathsf{i} \to \mathsf{i}$ and $a : \mathsf{i}$, and consider

$$\exists G\, \exists H\, [(\lambda w\, f\, a) = (\lambda w\, f\, (G\, w)) \;\wedge\; (\lambda w\, f\, w) = (\lambda w\, H\, (H\, w))],$$

where $G, H : \mathsf{i} \to \mathsf{i}$. The transformation has three steps.

1. Make the constants explicit: $\forall f\, \forall a\, \exists G\, \exists H\, [\ldots]$.
2. Replace each λ-equation by a quantified one:

$$\forall f\, \forall a\, \exists G\, \exists H\, [\forall w\, [(f\, a) = (f\, (G\, w))] \wedge \forall w\, [(f\, w) = (H\, (H\, w))]].$$

3. Prenex (one possibility):

$$\forall f\, \forall a\, \exists G\, \exists H\, \forall w\, \forall w'\, [(f\, a) = (f\, (G\, w)) \wedge (f\, w') = (H\, (H\, w'))].$$

Moving quantifiers to the prefix can introduce dependencies that are not really part of the problem: $\forall w$ and $\forall w'$ now both scope over the whole conjunction, although each was attached to one equation. The book calls these "logically harmless" and assumes prenex form from now on [MN p. 107].

The example only illustrates the transformation. By a short check of ours (the book does not discuss it), it has no solution. The first equation forces $G = \lambda w\, a$. But a closed H over f, a is $\lambda z\, f^k z$ or $\lambda z\, f^k a$, so $H\, (H\, w')$ is $f^{2k} w'$ or $f^k a$, never $f\, w'$.

Kanren angle: miniKanren has only ∃, and binders leak

`fresh°` is an existential, and there is no universal. The second failing query shows what this costs once binders are present. Encode `(y\ X) = (y\ y)` with de Bruijn records (illustrative records, file `scope.scm`):

```
(import (aux kanren micro) (aux base) (aux stream))
;; de Bruijn records: α-equivalent terms are equal data
(define-record lam body)
(define-record dbi n)
(define (show x) (display x) (newline))
;; λx.λy.x and λu.λv.u are the same record tree
(show (°->list #f (fresh° (q) (=° (make-lam (make-lam (make-dbi 1)))
                                  (make-lam (make-lam (make-dbi 1))))
                                (=° q 'equal))))
;; ?- (y\ X) = (y\ y).   λProlog answers no; plain =° answers yes
(show (°->list #f (fresh° (q) (=° (make-lam q) (make-lam (make-dbi 0))))))
;; ∃x[x = x]: the answer is a unifier, not a closed witness
(show (°->list #f (fresh° (q) (=° q q))))
```

```
((λ () (quote equal)))
((λ () (make-record-instance (quote dbi) 0)))
((λ (α) α))
```

The first line is what we want: α for free. The second is a *wrong yes*. `q` is bound to a dangling index, which is exactly the forbidden capture of the inner $\forall y$. (The third line is discussed in Section 4.6.) For the prefix discipline, a miniKanren therefore needs (i) eigenvariables, that is, fresh constants that are never bound, and (ii) a *scope check* on every binding: a variable may not receive a symbol quantified to its right. In (aux kanren micro) the monotone `vars-count` index already records the order in which things are created, which is almost the prefix order. The open question is how a variable inherits the restriction of the variable it is bound to. λProlog implementations answer it with *universe levels* [43, 9], and Chapter 11 spells out the proposal.

4.5 Simplifying prefixes: raising

A prefix can alternate as often as it likes. Raising is the tool that brings every prefix to the $\forall\exists\forall$ shape, or even to $\exists\forall$. It moves an existential to the left of a universal and pays for it by making the existential's type higher-order.

Proposition 4.14 (Raising [MN p. 107]). *Let $x : \tau$, $y : \sigma$ and $h : \sigma \to \tau$. Then*

$$\forall y\, \exists x\, G \quad \textit{is provable iff} \quad \exists h\, \forall y\, G[(h\, y)/x].$$

Proof (the book's argument). ($\Rightarrow$) A proof of the left formula replaces y by a new constant c and x by a term t, which may contain c. Prove the right formula with $h := \lambda c\, t$ and $y := c$. Note that c is not free in $\lambda c\, t$, as the order $\exists h\, \forall y$ requires. ($\Leftarrow$) A proof of the right formula replaces h by some s and y by a new c, and c cannot be free in s. Prove the left formula with $y := c$ and $x := (s\, c)$. □

Applied repeatedly, raising moves every existential to the left of the universals before it. As a rule, for a block of universals:

$$\forall y_1 : \sigma_1 \ldots \forall y_k : \sigma_k\ \exists x : \tau\ G \ \rightsquigarrow\ \exists h : \sigma_1 \to \ldots \to \sigma_k \to \tau\ \forall y_1 \ldots \forall y_k\ G[(h\, y_1 \ldots y_k)/x].$$

The "cost" is that the type of the existential is *raised* by the types of the universals it crosses [MN p. 108].

Tutorial: raising, step by step

Step 0: a problem whose answer depends on order. Let $g : \mathsf{i} \to \mathsf{i} \to \mathsf{i}$ and $a : \mathsf{i}$ be constants, and compare

$$(\mathrm{P}_1)\ \forall w\, \exists z\, [z = g\, a\, w] \qquad \text{and} \qquad (\mathrm{P}_2)\ \exists z\, \forall w\, [z = g\, a\, w].$$

By the reading rule, in P_1 z may use w and $z := g\, a\, w$ solves it. In P_2 z may not use w, and no term without w equals $g\, a\, w$, so P_2 has no solution.

Step 1: raise P_1. Move $\exists z$ across $\forall w$. The new variable $h : \mathsf{i} \to \mathsf{i}$ replaces z by $(h\, w)$:

$$\exists h\, \forall w\, [h\, w = g\, a\, w].$$

Now h comes first and may not mention w. But it is *applied* to w, so it gets w back as an argument. The solution is $h := \lambda c\, g\, a\, c$, and the check is $(\lambda c\, g\, a\, c)\, w \to_\beta g\, a\, w$. Translating back with the proof of Proposition 4.14 gives $z := h\, w = g\, a\, w$, the solution of Step 0. P_2 cannot be raised, because its existential is already on the left. The difference between the two problems is now visible in the syntax: h is applied to w in one, and z stands alone in the other.

Step 2: raise over constants as well ($\exists\forall$ form). Constants are outer universals, so one can also raise across them. Take $\forall a\, \exists F\, [(F\, a) = (g\, a\, a)]$ with $F : \mathsf{i} \to \mathsf{i}$. Raising F across $\forall a$ gives $F' : \mathsf{i} \to \mathsf{i} \to \mathsf{i}$, with F replaced by $(F'\, a)$:

$$\exists F'\, \forall a\, [(F'\, a\, a) = (g\, a\, a)].$$

F' may not use a. It must be $\lambda y\, \lambda x\, (g\, s_1\, s_2)$ with each $s_i \in \{x, y\}$, so there are *four* solutions. Mapped back by $F := F'\, a$, they are the four solutions of the original problem, which Section 4.7 lists. For instance, $F' = \lambda y\, \lambda x\, g\, y\, x$ gives $F = \lambda x\, g\, a\, x$.

What to remember. Raising is an exact logical equivalence. It replaces "may use the universals to my left" by "is a closed function applied to those universals". A logic variable under binders, `X` inside `y\ ...`, can always be read as a raised variable applied to the bound variables, `(H y)`. When the arguments are *distinct* bound variables, as in $h\, w$, the application is a *higher-order pattern* (Chapters 6 and 7), and pattern problems have most general unifiers. In Step 2 the argument a is repeated, $F'\, a\, a$, which is not a pattern, and indeed there is no most general solution.

Skolemization, the dual. Skolemizing an *assumption* $\forall x\, \exists y\, D$ with $x : \sigma$ and $y : \tau$ gives $\forall x\, D[(f\, x)/y]$, where $f : \sigma \to \tau$ is a new *Skolem constant*. For a *goal* the roles swap: $\exists x\, \forall y\, G$ becomes $\exists x\, G[(f\, x)/y]$, that is, $\forall f\, \exists x\, G[(f\, x)/y]$ [MN p. 108]. The two techniques are dual in two senses. Skolemization moves an $\exists$ *to a smaller scope* and introduces a new *constant* of raised type. Raising moves a $\forall$ *to a smaller scope* and introduces a new *existential variable* of raised type. Logically they differ: "relating unifiers for Skolemized unification problems to original, un-Skolemized unification problems is problematic, especially when unification involves variables of higher-order type" [MN p. 108]. Raising, in contrast, is an equivalence (Proposition 4.14), which is why the book uses it to normalise prefixes.

Pitfall: raising creates higher-order variables

Raising a first-order problem gives a higher-order one: in Step 1, $z : \mathsf{i}$ became $h : \mathsf{i} \to \mathsf{i}$. A first-order miniKanren with ∀-goals cannot implement scope by raising unless it also has λ-terms and β. Prenexing (Example 4.13) has a related cost: it may add spurious dependencies. These are logically harmless, but a scope-tracking implementation becomes less permissive than it needs to be.

Kanren angle: raising versus levels

Raising is how the book *explains* logic variables under `pi` and λ. Implementations usually do not raise up front. Nadathur, Jayaraman and Kwon tag each variable and constant with a *universe index* [43]. A binding is allowed only if the term's constants have tags no larger than the variable's, and the tags of the term's variables are lowered to match. ELPI uses the same *levels* [9]. Nadathur and Linnell [44] show how to erase the prefix and raise *on the fly*, only over the universals that actually occur. For (aux kanren micro), the choice is between raising, which needs only terms (λ, application, β) but makes every existential higher-order, and levels, which need a new per-variable piece of state and keep variables first-order. The two must agree on the examples of this chapter. Chapter 11 makes the choice.

4.6 Unifiers, solutions and empty types

Once a problem is a formula, solving it should mean proving it. The connection is subtler than it looks. The higher-order version of ∃R [MN p. 108–109] is

$$\frac{\Sigma;\mathscr{P} \longrightarrow B[t/x] \qquad \Sigma;\varnothing \Vdash t : \tau}{\Sigma;\mathscr{P} \longrightarrow \exists_\tau x\, B}\ \exists\mathrm{R}$$

The witness t must be *closed*, because its context is empty. Proving $\exists x\,[x = x]$ with $x : \mathsf{i}$ therefore needs a closed term of type i, and whether one exists depends on Σ. In the same way, $\exists x\,\exists y\,\exists z\,[x = (f\ y) \wedge y = (f\ z)]$ with $f : \mathsf{i} \to \mathsf{i}$ has the most general unifier $\{\langle X, f\ (f\ Z)\rangle, \langle Y, f\ Z\rangle\}$ in the sense of the book's Section 1.5 (Chapter 1). The formula, however, is provable only if some closed term of type i exists [MN p. 109].

Definition 4.15 (Legitimate substitution, unifier, solution [MN p. 109])**.** Consider the $\forall\exists\forall$ problem $\forall u_1 \ldots \forall u_p\, \exists x_1 \ldots \exists x_n\, \forall w_1 \ldots \forall w_q\, [t_1 = s_1 \wedge \cdots \wedge t_m = s_m]$ and let θ be a substitution for $x_1, \ldots, x_n$.

- θ is *legitimate* if no w_j occurs free in any $\theta(x_i)$.
- A legitimate θ is a *unifier* if $\theta(t_j)$ and $\theta(s_j)$ are λ-convertible for every j.
- A unifier is a *solution* if the only free variables in the terms $\theta(x_i)$ are among $u_1, \ldots, u_p$.

A problem can have a unifier and no solution. *It is a solution that makes the quantified formula provable.*

In the book's Section 1.5 (Chapter 1) "solution" and "unifier" meant the same thing. They have to be separated now because, unlike first-order classical logic, higher-order intuitionistic logic does not assume that every type is *inhabited*. The Church numerals show why this matters. They are the closed terms of type $(\mathsf{i} \to \mathsf{i}) \to \mathsf{i} \to \mathsf{i}$ only if i has *no* closed term t, since otherwise $\lambda f\, \lambda x\, t$ would be a further, non-numeral inhabitant [MN p. 109–110].

Example 4.16 (Unifier but no solution)**.** Let Σ contain only $u, v : \mathsf{i} \to \mathsf{i}$, so i is empty. The problem $\exists x\,[x = x]$ has the unifier $\{x \mapsto X\}$ with X a fresh logic variable. It has no solution, because there is no closed term of type i. Add a constant $a : \mathsf{i}$ and $\{x \mapsto a\}$ becomes a solution.

Kanren angle: miniKanren answers are unifiers

The third answer of `scope.scm` above, `((λ (α) α))` for $\exists x\,[x = x]$, is a *unifier* in the sense of Definition 4.15. The reified `α` is a fresh logic variable, not a closed witness. This is also λProlog's practice: answers may contain variables, and leftover flex–flex constraints (Section 4.8). An untyped miniKanren cannot see empty types at all. Obtaining *solutions* would need an explicit generator of closed inhabitants of a type over a signature, itself a proof-search relation. Section 4.9 shows that the gap between unifiers and solutions is exactly where undecidability lives.

4.7 Example problems and their solutions

These examples [MN p. 110–111] are the core intuition pump of the chapter. Each one shows a way in which higher-order unification differs from the first-order kind.

4.7.1 Quantifier order changes the answers

Let $g : \mathsf{i} \to \mathsf{i} \to \mathsf{i}$ be a constructor of i.

- $\forall a\, \exists F\, [(F\, a) = (g\, a\, a)]$ has **four** solutions: $F \in \{\lambda x\, g\, a\, a,\ \lambda x\, g\, x\, a,\ \lambda x\, g\, a\, x,\ \lambda x\, g\, x\, x\}$.
- $\exists F\, \forall a\, [(F\, a) = (g\, a\, a)]$ has the **unique** solution $F = \lambda a\, g\, a\, a$.

Walk-through. $F : \mathsf{i} \to \mathsf{i}$ is closed, so $F = \lambda x\, b$ with b built from g, x and whatever F may use. The β-normal form of $F\, a$ is $b[a/x]$, and it must be $g\, a\, a$. So b is $g\, s_1\, s_2$ where each s_i either *is* a or *becomes* a under the substitution, that is, $s_i \in \{a, x\}$. In the first problem a is to the left of F, so both choices are allowed for each position: $2 \times 2 = 4$. In the second, F may not mention a, only $s_i = x$ remains, and there is one solution. (Section 4.5 reached the same count of four by raising.)

Kanren angle: a first-order simulation of both prefixes

In this example F is applied to a single argument and its body is first-order, so the problem can be simulated in today's (aux kanren micro). Represent F's body as a first-order term in which `x` stands for the bound variable, and relate a body to its instance $b[a/x]$ with a relation `inst°`. The universal a is the symbol `a`. The ∃F∀a prefix, "F may not mention a", is then the constraint `(absent° 'a body)`. This is an illustration, not general higher-order unification (file `order.scm`):

```
(import (aux kanren micro) (aux base) (aux stream))
;; F = λx.body, body a first-order term over g, a and x = (dbi 0)
(define-record dbi n)
(set-record-printer! dbi (λ (r port) (display "x" port)))
(define x (make-dbi 0))
(define (show x) (display x) (newline))
;; (inst° body arg out): out = body[arg/x], the normal form of (F arg)
(define-relation (inst° body arg out)
  (cond°
    ((=° body x) (=° out arg))
    ((symbol° body) (=° out body))
    ((null° body) (null° out))
    ((fresh° (h t h* t*)
       (=° body (cons h t)) (=° out (cons h* t*))
       (inst° h arg h*) (inst° t arg t*)))))
;; ∀a ∃F [(F a) = (g a a)]: F's body may mention the constant a
(show (°->list/ground
        (fresh° (body) (inst° body 'a '(g a a)))))
;; ∃F ∀a [(F a) = (g a a)]: a comes after F, F's body must avoid it
(show (°->list/ground
        (fresh° (body) (absent° 'a body) (inst° body 'a '(g a a)))))
```

```
((g x x) (g a x) (g x a) (g a a))
((g x x))
```

These are the book's four answers and its one answer. Two points carry over to the real design. First, the scope check is a *constraint* on what a variable may contain, like

`absent°`, and it is attached at the moment the variable is created. Second, a relation that runs substitution *backwards* (`inst°` with `out` known) is what imitation and projection do in Huet's procedure (Chapter 7): each `a` in the output is either imitated or projected from the argument.

4.7.2 Words over a two-letter alphabet

Let the only constructors with target i be $u, v : \mathsf{i} \to \mathsf{i}$. Then i has no closed terms, and the closed terms of type $\mathsf{i} \to \mathsf{i}$ are *words*: $\lambda w\, w$ is the empty word ε, $\lambda w\, u\, w$ is "u", and $\lambda w\, u\, (v\, (u\, w))$ is "uvu". Function composition is concatenation [MN p. 110]. Three problems, all with $F, G : \mathsf{i} \to \mathsf{i}$:

1. $\exists F\, \exists G\, [(\lambda w\, F\, (G\, w)) = (\lambda w\, u\, (v\, (u\, w)))]$ has **four** solutions, the four ways of splitting uvu: $(F, G) \in \{(\text{uvu}, \varepsilon), (\text{uv}, \text{u}), (\text{u}, \text{vu}), (\varepsilon, \text{uvu})\}$. Replacing the right-hand side by any other closed term asks for all ways of splitting that word.

2. $\exists F\, [(\lambda w\, u\, (F\, (u\, w))) = (\lambda w\, u\, (v\, (v\, (u\, w))))]$ has the **only** solution $F = \lambda w\, v\, (v\, w)$. The left-hand side is a *pattern* that checks whether a word starts and ends with u.

3. $\exists F\, [(\lambda w\, u\, (F\, w)) = (\lambda w\, F\, (u\, w))]$ is the word equation $\text{u} \cdot F = F \cdot \text{u}$. Its solutions are **infinitely many**: $F \in \{\lambda w\, w,\ \lambda w\, u\, w,\ \lambda w\, u\, (u\, w), \ldots\}$, that is, u^k for every $k \geq 0$. They are closed and pairwise distinct, so none is an instance of another: *there is no most general solution.*

Kanren angle: words are lists, composition is append

Because i is empty, a closed term of type $\mathsf{i} \to \mathsf{i}$ *is* a word, and in this one case higher-order unification becomes ordinary relational list programming (file `words.scm`):

```
(import (aux kanren micro) (aux base) (aux stream))
;; closed terms of type i -> i over u, v : i -> i are words,
;; and composition is concatenation
(define-relation (append° r s rs)
  (cond°
    ((null° r) (=° s rs))
    ((fresh° (a d c) (cons° a d r) (cons° a c rs) (append° d s c)))))
(define (show x) (display x) (newline))
;; ∃F ∃G [(λw F (G w)) = (λw u (v (u w)))]
(show (µkanren-run (q -1 #t)
        (fresh° (F G) (append° F G '(u v u)) (=° q (list F G)))))
;; ∃F [(λw u (F (u w))) = (λw u (v (v (u w))))]
(show (µkanren-run (q -1 #t)
        (fresh° (uF) (append° '(u) q uF) (append° uF '(u) '(u v v u)))))
;; ∃F [(λw u (F w)) = (λw (F (u w)))]: five of infinitely many
(show (µkanren-run (q 5 #t)
        (fresh° (W) (append° '(u) q W) (append° q '(u) W))))
```

```
((() (u v u)) ((u) (v u)) ((u v) (u)) ((u v u) ()))
```

```
((v v))
(() (u) (u u) (u u u) (u u u u))
```

The third query shows the right attitude toward problems without an mgu. A miniKanren does not need a single answer: it returns a *stream* of them, and `take°` (here `µkanren-run` with $n = 5$) cuts the stream. This is the shape a general higher-order unifier in (aux kanren micro) must have (Section 4.9).

4.7.3 Weak functionality

Let i have at least the constants $a, b, c, d : \mathsf{i}$. Then $\exists F\,[(F\,a) = b \wedge (F\,c) = d]$ has **no solution**. The first equation is solved only by $F = \lambda x\,b$: F cannot produce b by projecting its argument a, so it must be the constant function. That F fails the second equation [MN p. 110–111]. A function that maps $a \mapsto b$ and $c \mapsto d$ exists as a set-theoretic object, but it is *not expressible* by these λ-terms, which have no case analysis on constants. The book calls this "a relatively weak notion of functionality".

Pitfall: what a first-order intuition gets wrong

- *Quantifier order is semantic*: $\forall a\,\exists F$ and $\exists F\,\forall a$ give four answers and one answer.
- *There may be several incomparable solutions* (four splittings of uvu), *or infinitely many with no mgu* (u·F = F·u). A unifier returning one substitution, as `µkanren-state-unify` does, cannot express this.
- *λ-terms are not arbitrary functions*: no case analysis, so $F\,a = b \wedge F\,c = d$ is unsolvable.

4.8 Vocabulary for a unification procedure

Chapter 7 presents Huet's procedure [25]. The book prepares it with some vocabulary and a few structural observations [MN p. 111–113].

Definition 4.17 (Binder, head, arguments, body [MN p. 111])**.** Every β-normal term has the shape

$$\lambda x_1 \ldots \lambda x_n\,(h\;t_1 \cdots t_p) \qquad (n, p \geq 0),$$

where h is a constant or a variable and every t_i is β-normal. $x_1, \ldots, x_n$ is the *binder*, h the *head*, $t_1, \ldots, t_p$ the *arguments*, and $(h\,t_1 \cdots t_p)$ the *body*.

Definition 4.18 (βη-long normal form [MN p. 111])**.** A term is in *βη-long normal form* if it has the shape above, every t_i is βη-long, and the body has a non-functional type. For simply typed terms, that means a primitive type. Every β-normal form can be η-expanded into an equal βη-long form.

Example 4.19 (η-expanding to long form [MN p. 111])**.** Let $f : (\mathsf{i} \to \mathsf{i}) \to \mathsf{i}$. The term f is β-normal but not long, since its body has functional type. One η-expansion gives $\lambda w\,f\,w$ with

$w : \mathsf{i} \to \mathsf{i}$. This is still not long, because the argument w has a non-primitive type. Expanding w gives $\lambda w\, f\, (\lambda u\, w\, u)$ with $u : \mathsf{i}$, which is βη-long. Note that deciding when to stop requires knowing the *types*.

Definition 4.20 (Rigid and flexible [MN p. 111–113])**.** In a problem over β-normal terms, an occurrence $\lambda \bar{x}\, (h\, t_1 \cdots t_p)$ is *rigid* if h is ∀-bound in the prefix (constants included) or belongs to the binder $\bar{x}$. It is *flexible* if h is ∃-bound in the prefix. Equations are rigid–rigid, rigid–flexible, flexible–rigid or flexible–flexible, and a problem is *flexible–flexible* if all of its equations are.

The names are meant to suggest what substitution can do. Substitution never changes the head of a rigid term: every instance has a β-normal form with the same head. It can change the head of a flexible term [MN p. 112].

Observation 1: rigid heads must agree. By α and η, two rigid terms of the same type can be given the same binder: $\lambda \bar{x}\, (h\, t_1 \cdots t_p)$ and $\lambda \bar{x}\, (k\, s_1 \cdots s_q)$. Since neither head can change, a unifier can exist only if h and k are the same symbol [MN p. 112]. With F, G logic variables and c, d constants, none of the following has a unifier:

$$\{(\lambda x\, (d\, (c\, (F\, x)))) = (\lambda x\, (c\, (d\, (G\, x))))\} \qquad \{(\lambda x\, (x\, (F\, x))) = (\lambda x\, (c\, (G\, x)))\}$$
$$\{(\lambda x\, \lambda y\, (x\, (F\, x\, y))) = (\lambda x\, \lambda y\, (y\, (G\, x\, y)))\}$$

The first one, written precisely, is $\forall c\, \forall d\, \exists F\, \exists G\, [\lambda x\, (d\, (c\, (F\, x))) = \lambda x\, (c\, (d\, (G\, x)))]$. Note that in the second and third problems one of the heads is a *bound* variable, which counts as rigid.

Observation 2: substitution cannot create captured occurrences. If a binder variable has no free occurrence in the body of a β-normal form, it has none in any instance either, because substitution avoids capture. So

$$\{(\lambda x\, \lambda y\, x\, (F\, x\, y)) = (\lambda x\, \lambda y\, G\, y\, y)\}$$

has no unifier [MN p. 112–113]. Every instance of the left side has head x. No instance of the right side can contain the bound x: G may be instantiated with a term containing a *free* x, but that is a different x.

Observation 3: flexible terms are limited only in their use of the binder. A flexible term can become almost any term of its type, *except* in how it uses the binder variables. For instance, $\lambda x\, \lambda y\, F\, (f\, x\, (G\, x))$ can become any $\lambda x\, \lambda y\, t$ with x, y not free in t: take $F := \lambda z\, t$, which throws its argument away. Some instances also contain x, such as $\lambda x\, \lambda y\, (f\, x\, x)$, obtained with $F := \lambda z\, z$ and $G := \lambda z\, z$. But no instance contains y [MN p. 113], because y is passed to neither F nor G, and by Observation 2 substitution cannot introduce it.

Observation 4: a flex–flex pair. Unify $\lambda x\, \lambda y\, G\, x$ with $\lambda x\, \lambda y\, H\, y$. Instances of the first ignore y, and instances of the second ignore x, so a common instance is $\lambda x\, \lambda y\, t$ with neither variable free in t. The "most general" unifier, derived in Chapter 7, is $G := \lambda w\, T$ and $H := \lambda w\, T$, with T a new variable [MN p. 113]. (Pattern unification calls this step *pruning*, Chapter 7.)

Tutorial: classifying heads in practice

Take the prefix $\forall c\, \exists F\, \exists G$ and three occurrences:

term	head	status
$\lambda x\,(c\,(F\,x))$	c, $\forall$-bound	rigid: every instance starts with c
$\lambda x\,(x\,(F\,x))$	x, in the binder	rigid: every instance starts with the bound x
$\lambda x\,(F\,(c\,x))$	F, $\exists$-bound	flexible: $F := \lambda z\, z$ makes the head c

A rigid–rigid equation either fails immediately or decomposes into equations between arguments (Chapter 7). A flex–rigid equation is where the search branches (imitate the rigid head, or project an argument). A flex–flex equation always has a unifier, as in Observation 4, so Huet's procedure *postpones* it (Chapter 7). It need not have a *solution*, as Section 4.9 shows.

Kanren angle: what the vocabulary means for the unifier

(aux kanren micro)'s `=°` already performs "rigid–rigid": two records or pairs with different constructors fail at once, and the same constructor decomposes slot-wise. A de Bruijn representation whose application node exposes head and arguments, as in Definition 4.17, makes Observation 1 a constructor clash. Bound-variable heads (`dbi` records) are rigid automatically. Everything about *flexible* heads is new: the unifier must recognise a logic variable in head position, normalise after binding it, and either solve the equation deterministically (patterns, Chapter 6) or branch and delay it. Delayed flex–flex equations need a store in the state, next to the disequality store, and Chapter 11 proposes one.

4.9 Undecidability

Derivability from *fohc* programs is already undecidable, and so is unifiability of typed λ-terms modulo λ-conversion [MN p. 113]. The book proves the second fact by two reductions. It also stresses that the specifications of later chapters produce "much tamer" problems.

4.9.1 The Post correspondence problem

Definition 4.21 (PCP [MN p. 114])**.** Given $n \geq 1$ and two lists $s_1, \ldots, s_n$ and $t_1, \ldots, t_n$ of words over $\{u, v\}$, decide whether there is a *non-empty* sequence $i_1, \ldots, i_k$ with $1 \leq i_j \leq n$ such that $s_{i_1} \cdots s_{i_k} = t_{i_1} \cdots t_{i_k}$. This problem is undecidable.

Encode a word $r = r_1 \cdots r_m$ as $\hat{r} = \lambda w\,(r_1\,(\cdots\,(r_m\,w)\cdots))$, as in Section 4.7.2. Concatenation is then $\lambda w\,(\hat{r}\,(\hat{q}\,w))$. Take a sort i *with no constructors*, and introduce $u, v : \mathsf{i} \to \mathsf{i}$ by universal quantifiers [MN p. 114]:

$$\exists F\, \exists G\, \forall u\, \forall v\, [\,(F\,\hat{s}_1 \cdots \hat{s}_n) = (F\,\hat{t}_1 \cdots \hat{t}_n) \;\wedge\; (F\,u \cdots u) = \lambda w\,(u\,(G\,u\,w))\,],$$

with $G : (\mathsf{i} \to \mathsf{i}) \to \mathsf{i} \to \mathsf{i}$. For the lists uv, u and u, vu [MN p. 114]:

$$\exists F\, \exists G\, \forall u\, \forall v\, [\, (F\,(\lambda w\, u\,(v\, w))\, u) = (F\, u\,(\lambda w\, v\,(u\, w))) \;\wedge\; (F\, u\, u) = \lambda w\,(u\,(G\, u\, w))\,],$$

where F has the second-order type $(\mathsf{i} \to \mathsf{i}) \to (\mathsf{i} \to \mathsf{i}) \to \mathsf{i} \to \mathsf{i}$.

Tutorial: why the encoding is correct

Read the prefix. F and G come *before* $\forall u\, \forall v$, so they may not mention u or v. They can only rearrange their arguments. i has no constructors, so a closed F must be $\lambda x_1 \ldots \lambda x_n\, \lambda w\, d$ with $d : \mathsf{i}$ built only from $x_1, \ldots, x_n, w$. The only possible shape is $d = x_{i_1}(\cdots (x_{i_k}\, w) \cdots)$ with $k \geq 0$ [MN p. 114].

Read off the index sequence. $F\, \hat{s}_1 \cdots \hat{s}_n$ normalises to the word $s_{i_1} \cdots s_{i_k}$, and the right side to $t_{i_1} \cdots t_{i_k}$. So the first equation says exactly that $i_1, \ldots, i_k$ solves the PCP, except in the degenerate case $k = 0$, where both sides are $\lambda w\, w$.

Exclude $k = 0$. The second equation says that $F\, u \cdots u$ starts with u, which forces $k \geq 1$. In the other direction, a PCP solution gives a unification solution directly [MN p. 114–115]. The argument extends to *unifiers* and to types i with constructors [MN p. 115].

Check the instance. For uv, u / u, vu, the sequence 1, 2 works: $\mathsf{uv} \cdot \mathsf{u} = \mathsf{u} \cdot \mathsf{vu} = \mathsf{uvu}$. This suggests $F = \lambda x_1\, \lambda x_2\, \lambda w\, x_1\, (x_2\, w)$ and $G = \lambda y\, \lambda w\, y\, w$. G may not mention u and has to obtain it from its argument. The book does not print this solution. The normaliser of Section 4.3, with u, v as free symbols, confirms both equations, and also the Diophantine gadget of the next subsection (file `checks.scm`):

```
(include "norm-core.scm")
(define (same? s t) (equal? (nf (->db s '())) (nf (->db t '()))))
(define (show . xs) (display xs) (newline))
;; PCP instance uv,u / u,vu; u and v are free symbols
(define F '(λ (x1 x2 w) (x1 (x2 w))))
(define G '(λ (y w) (y w)))
(show 'pcp (same? `(,F (λ (w) (u (v w))) u) `(,F u (λ (w) (v (u w)))))
           (same? `(,F u u) `(λ (w) (u (,G u w)))))
;; gadget for x = 1: λf λx N f (N f x) = λf λx N f (f x)
(define (numeral n)
  `(λ (f x) ,(let loop ((k n)) (if (= k 0) 'x `(f ,(loop (- k 1)))))))
(define (gadget N)
  (same? `(λ (f x) (,N f (,N f x))) `(λ (f x) (,N f (f x)))))
(show 'numerals-0..3 (map (lambda (n) (gadget (numeral n))) '(0 1 2 3)))
(show 'λf.λx.U (gadget '(λ (f x) U)))
```

```
(pcp #t #t)
(numerals-0..3 (#f #t #f #f))
(λf.λx.U #t)
```

Historically, Huet [24] gave the PCP reduction, which shows that *third-order* unification is undecidable [MN p. 116].

4.9.2 Hilbert's tenth problem

Hilbert's tenth problem asks for an algorithm that decides whether a system of Diophantine equations (variables, numerals, $+$, $\times$ over $\mathbb{N}$) has a solution. No such algorithm exists [MN p. 115]. It is enough to treat equations of the forms $x = 1$, $x + y = z$ and $x \times y = z$. Take a sort i without constructors, so that the closed terms of $(\mathsf{i} \to \mathsf{i}) \to \mathsf{i} \to \mathsf{i}$ are exactly the Church numerals $\ulcorner n \urcorner$, and give every existential this type. Then [MN p. 115]:

1. The flex–flex problem $\exists N\,[(\lambda f\,\lambda x\,N\,f\,(N\,f\,x)) = (\lambda f\,\lambda x\,(N\,f\,(f\,x)))]$ has the unique solution $N = \ulcorner 1 \urcorner$. With $N = \ulcorner n \urcorner$ the sides are $f^{2n}x$ and $f^{n+1}x$, so $2n = n + 1$.

2. The flex–flex problem $\exists M\,\exists N\,\exists P\,[(\lambda f\,\lambda x\,N\,f\,(M\,f\,x)) = (\lambda f\,\lambda x\,P\,f\,x)]$ has the solution $\{(N, r), (M, s), (P, t)\}$ iff $r = \ulcorner n \urcorner$, $s = \ulcorner m \urcorner$ and $t = \ulcorner n + m \urcorner$ for some $n, m \geq 0$.

3. The flex–flex problem $\exists M\,\exists N\,\exists P\,[(\lambda f\,\lambda x\,N\,(M\,f)\,x) = (\lambda f\,\lambda x\,P\,f\,x)]$ has the solution $\{(N, r), (M, s), (P, t)\}$ iff $r = \ulcorner n \urcorner$, $s = \ulcorner m \urcorner$ and $t = \ulcorner n \times m \urcorner$.

Remark 4.22 (An erratum in the addition gadget). The book prints the left side of item 2 as $\lambda f\,\lambda x\,(N\,f)\,(M\,f)\,x$ [MN p. 115]. This parses as $((N\,f)\,(M\,f))\,x$, which is ill-typed, since $N\,f : \mathsf{i} \to \mathsf{i}$ is applied to $M\,f : \mathsf{i} \to \mathsf{i}$. The intended term is the addition combinator of Section 4.3.1, $\lambda f\,\lambda x\,N\,f\,(M\,f\,x)$, and that is the term shown above.

Theorem 4.23 ([MN p. 116])**.** *A system of such equations becomes a flex–flex problem with one existential variable per* $\mathbb{N}$*-variable and one equation per equation. The problem has a* solution *iff the system has a solution in* $\mathbb{N}$*. The variables have order at most 2, so the problem is third-order, and the construction extends to arbitrary Diophantine equations. Hence* ***it is undecidable whether a third-order flex–flex problem has a solution.***

Unifiers versus solutions, once more. The encoding *always* has a unifier: pick a new variable $U : \mathsf{i}$ and set every existential to $\lambda f\,\lambda x\,U$. Both sides of every gadget then normalise to $\lambda f\,\lambda x\,U$. This term is not closed, so it is not a solution [MN p. 116]. The run above confirms both facts for gadget 1. Among $\ulcorner 0 \urcorner, \ldots, \ulcorner 3 \urcorner$ only $\ulcorner 1 \urcorner$ works, and $\lambda f\,\lambda x\,U$ works too. So the undecidable question concerns *solutions*. Asking only for unifiers, a flex–flex problem is trivially unifiable, and this is why Huet's procedure can leave flex–flex pairs for last. In the bibliographic notes [MN p. 116–117] the book credits this reduction to Miller [35]. It differs from Goldfarb's [17], which already shows that *second-order* unification, for unifiers, is undecidable.

Kanren angle: undecidability and the search

Since unification is undecidable, no complete procedure terminates on every input. A complete higher-order unifier is a *semi-decision procedure*: a search, some of whose branches never finish. In a miniKanren this is a familiar situation. Such a unifier would be a *goal* that produces a stream of unifiers. Huet's branching steps would become disjunctions, and (aux kanren micro)'s *interleaving* `orᵍ` would explore them fairly, which a depth-first exploration does not. Flex–flex pairs, which always have unifiers, would be left in a constraint store and reified with the answer, which then is a unifier and not necessarily a solution. The proposal keeps only the deterministic part: pattern problems

are solved by one `=°`, and non-patterns, flex–flex pairs included, wait in a store and are reported (Section 12.3.7). Huet's branching is left to an optional search over that store, which needs types (Section 14.5.4).

Remark 4.24 (Polymorphism). All the formal results of the book's chapter concern *simply typed* terms. Type variables are allowed for convenience, for instance one declaration for `pi`, but *unification does not instantiate type variables*: they behave like sorts, so the simply typed results still apply [MN p. 117].

4.10 Summary

- Terms are simply typed λ-terms (Figure 4.1). Formulas are terms of type o, the quantifiers are constants $\forall, \exists : (A \to o) \to o$ applied to abstractions, and λ is the *only* binder.
- The equality of the logic is λ-conversion (α, β, η). Bound names are unobservable, substitution avoids capture, and equality ignores logical equivalences.
- Every term has a λ-normal form, unique up to α, so equality is decidable. β is powerful (Church numerals) and normal forms can be huge (2^{2^n} in the book's series, see the erratum). Practical programs avoid this through built-in data, order-0 binders and L_λ.
- A unification problem is a quantified formula $Q_1 x_1 \dots Q_n x_n\, [\bigwedge t_j = s_j]$. *An existential may use only what is quantified to its left.* `pi` and λ both introduce universals, which is why `pi y\ X = y` and `(y\ X) = (y\ y)` fail.
- *Raising* ($\forall y\, \exists x\, G \rightsquigarrow \exists h\, \forall y\, G[(h\, y)/x]$) is an exact equivalence that brings any prefix to ∀∃∀ or ∃∀, at the cost of higher-order types. Skolemization is its dual but does not preserve unifiers well.
- A *unifier* may leave free variables, while a *solution* uses only the outer universals. Only solutions give proofs, and empty types make the difference real. miniKanren answers are unifiers.
- Higher-order unification has several, infinitely many, or no solutions, and no mgu in general. Quantifier order changes the count (4 versus 1), and λ-terms cannot do case analysis.
- Vocabulary for Chapter 7: binder, head, arguments, body; βη-long form; rigid and flexible terms; rigid heads must agree; vacuous binders stay vacuous; flex–flex pairs always have unifiers.
- Unifiability is undecidable (PCP, third order), and so is the existence of solutions of third-order flex–flex problems (Hilbert's tenth problem). Hence a miniKanren higher-order unifier must be a goal producing a fair stream of unifiers, with flex–flex pairs postponed.

5 Higher-order programming

Chapter 4 replaced first-order terms by simply typed λ-terms and allowed quantification at every type, including predicate types (types whose target is o); it also showed that unification of such terms is undecidable and may have infinitely many incomparable solutions. This chapter, which follows Chapter 5 of the book [36] and the core of its Chapter 6, asks the practical question: *what logic programming languages do we get, and what do they buy us?*

Two kinds of higher-order variable will appear, and it is worth keeping them apart from the start.

- **Predicate variables** (target type o) are instantiated by relations and then *invoked as goals*. They give higher-order programming in the style a Scheme programmer knows: `mappred`, folds, closures of relations, continuation passing.
- **Non-predicate function variables** (target type not o) are instantiated by λ-terms that are only ever *evaluated by $\beta\eta$-conversion*. Their values can also be *found by solving unification problems*; the book calls this "perhaps the single most novel aspect of higher-order logic programming" [MN p. 138].

We first fix which formulas may be goals and which may be clauses (Section 5.1), then program with predicate variables (Sections 5.2 to 5.6), then with function variables (Sections 5.7 and 5.8), compare with functional programming (Section 5.9), and finally show how the same two logical devices that hide a predicate, $\forall$ in goals and $\supset$ in goals, also give a modules language (Section 5.10).

A question runs through the whole chapter, and the kanren boxes answer it case by case: *in a miniKanren hosted in Scheme, where a relation is a Scheme procedure, which of these features are free, and which really need λ-terms in the term language?* Section 5.11 collects the answers.

Tutorial: How to run the Scheme listings

Every Scheme listing in this chapter was run on today's (aux kanren micro) and its output is shown verbatim below it. Each file begins with `(include "prelude.scm")`, which loads this prelude (the include line is left out of the listings):

```
(import scheme (chicken base) (chicken pretty-print)
        (aux base) (aux stream) (aux kanren micro))
(define-syntax-rule (show label expr)
  (begin (display label) (display " ") (write expr) (newline)))
```

`(µkanren-run (q n grounded) g ...)` returns up to `n` answers for `q` (all answers when `n` is negative). With `grounded` set to `#t` it returns them as plain data; with `#f` it returns them as reified Scheme λ-expressions.

5.1 Which formulas are goals and which are clauses

At first order (Chapters 2 and 3) goals and clauses of *fohh* were

$$G ::= \top \mid A \mid G \wedge G \mid G \vee G \mid \forall x\, G \mid \exists x\, G \mid D \supset G$$
$$D ::= A \mid G \supset D \mid \forall x\, D \mid D \wedge D$$

and *fohc* is the fragment without $\forall$ and $\supset$ in goals [MN p. 118]. The higher-order languages keep this top-level shape. What changes is *what an atom A may be*: once quantification over predicates is allowed, an atom may contain λ-terms and predicate variables, and its head may itself be a variable.

5.1.1 Rigid and flexible atoms

Definition 5.1 (Rigid and flexible atoms [MN p. 119])**.** The λ-normal form of a higher-order atomic formula has the shape $(h\, t_1 \ldots t_n)$ where h is either a variable or a non-logical constant. The atom is *rigid* if h is a constant and *flexible* if h is a variable.

So `age bob 23` and `mappred P L K` are rigid, while `P X Y` is flexible. The atom `(x\ age x 23) bob` is rigid too, because its λ-normal form is `age bob 23`. Flexible atoms in *goals* are what make higher-order programming possible: in the body of `mappred P (X::L) (Y::K) :- P X Y, mappred P L K.` the goal `P X Y` is flexible. Flexible atoms as *clause heads* are a different matter.

5.1.2 Why clause heads must be rigid

The book gives two reasons for rejecting flexible heads [MN p. 119–120].

Consistency. A program is *inconsistent* if every formula is provable from it. Every *fohc* and *fohh* program is consistent. This matters in practice: otherwise a complete interpreter would have to check the program for consistency before it could answer "no". With flexible heads, inconsistent programs are easy to write:

```
P 5 :- q.
q.
```

To prove any goal B, backchain on the first clause with $P := \lambda x.\, B$; what is left is `q`, which is provable. The single clause $\forall p\, p$ is an even shorter example [MN p. 120].

Modularity. A clause $\forall x_1 \ldots \forall x_m (A_1 \wedge \cdots \wedge A_n \supset A_0)$ is read as part of the definition of the relation named by the head of A_0. If that head is a variable, the clause adds meaning to *every* predicate. That defeats modular reading of code, and it also defeats indexing clauses by predicate name.

Two natural uses are lost, and the book argues that losing them is acceptable. *Leibniz equality* (two terms are equal if they satisfy the same properties) would turn $x + 0 = x$ into

```
P (X + 0) :- P X.
P X :- P (X + 0).
```

This is logically meaningful but operationally a disaster. A goal containing n occurrences of, say, `5 + 0` admits 2^n abstractions for `P`, and even a goal with no occurrence can always backchain on the first clause, so the search loops [MN p. 120]. *Computed clauses*, as in `?- convert d P, P => g.`, put a variable where a clause is expected. There is no simple way to guarantee statically that the value bound to `P` will be a legal clause [MN p. 121].

5.1.3 No implications inside terms

Logical symbols inside the arguments of atoms are useful (the symmetric closure `x\y\ adj x y ; adj y x` is a term containing $\vee$), but an *implication* inside a term breaks goal-directed search. The book's counterexample [MN p. 121] uses constants $r : \mathrm{o} \to \mathrm{o}$ and $s, t : \mathrm{o}$, and a variable $Q : \mathrm{o} \to \mathrm{o} \to \mathrm{o}$:

$$\exists Q \left[\forall p \, \forall q \left[r\,(p \supset q) \supset r\,(Q\,p\,q) \right] \wedge Q\,(t \vee s)\,(s \vee t) \right].$$

The first conjunct forces $Q := \lambda x.\, \lambda y.\, x \supset y$: after the $\forall$ and $\supset$ introductions the goal $r\,(Q\,p\,q)$ must match the hypothesis $r\,(p \supset q)$. The second conjunct then becomes $(t \vee s) \supset (s \vee t)$, which is provable in any reasonable logic, but *not* by the goal-directed rules of Chapter 2: the instance of Q moved a disjunction from a goal position into a hypothesis, where a left rule for $\vee$ would be needed.

Definition 5.2 (Polarity [MN p. 121–122])**.** When implications are excluded from terms, an occurrence of a logical symbol is *positive* if and only if it is either within an atomic formula or is in the scope of only logical constants and to the left of an even number of implications. All other occurrences are *negative*.

The point of the definition is an invariant: during cut-free proof search, positive occurrences stay positive and negative ones stay negative. Goal-directed search relies on it. In the counterexample both disjunctions start out positive, and instantiating Q makes one of them negative. (The ban is a design choice, not the only one possible: the bibliographic notes recall Felty's result that $\supset$ may occur in terms if $\vee$ and $\exists$ are banned from formulas and terms instead [MN p. 148].)

5.1.4 Herbrand universes, hohc and hohh

The quantifier rules $\exists R$ and $\forall L$ substitute terms. To keep goals goals and clauses clauses, the terms that may be substituted have to be restricted.

Definition 5.3 (Herbrand universes [MN p. 122])**.** Let Σ contain at least the logical constants. $\mathscr{H}_1^\Sigma$ is the set of all (possibly open) λ-normal terms over Σ that contain neither $\forall$ nor $\supset$. $\mathscr{H}_2^\Sigma$ is the set of all (possibly open) λ-normal terms over Σ that do not contain $\supset$.

Terms of $\mathscr{H}_1^\Sigma$ may contain $\top, \wedge, \vee, \exists$, and the types of their subterms are unrestricted. $\mathscr{H}_2^\Sigma$ also allows $\forall$.

Definition 5.4 (*hohc*: higher-order Horn clauses [MN p. 122])**.** Let A range over atoms in $\mathscr{H}_1^\Sigma$ and A_r over *rigid* atoms in $\mathscr{H}_1^\Sigma$. Goals and clauses are the formulas whose λ-normal forms are

$$G ::= \top \mid A \mid G \wedge G \mid G \vee G \mid \exists_\tau x\, G$$
$$D ::= A_r \mid G \supset D \mid D \wedge D \mid \forall_\tau x\, D$$

with no restriction on τ. The G-formulas are exactly the formulas of $\mathscr{H}_1^\Sigma$ of type o.

The operational semantics is the goal-directed calculus of Chapter 2, with three changes [MN p. 123]: formulas are taken in λ-normal form, INITIAL works up to α-conversion, and the quantifier rules instantiate with λ-terms from the Herbrand universe ($\Vdash$ is the typing judgement for the instantiation term):

$$\frac{}{\Sigma;\mathscr{P} \xrightarrow{A'} A}\ \textsc{initial}\ (A' \equiv_\alpha A) \qquad \frac{\Sigma;\mathscr{P} \longrightarrow B[t/x] \qquad \Sigma;\emptyset \Vdash t : \tau \qquad t \in \mathscr{H}_1^\Sigma}{\Sigma;\mathscr{P} \longrightarrow \exists_\tau x\, B}\ \exists\mathrm{R}$$

$$\frac{\Sigma;\mathscr{P} \xrightarrow{D[t/x]} A \qquad \Sigma;\emptyset \Vdash t : \tau \qquad t \in \mathscr{H}_1^\Sigma}{\Sigma;\mathscr{P} \xrightarrow{\forall_\tau x\, D} A}\ \forall\mathrm{L}$$

Why is the restriction needed? Without it, the goal $\exists p\, p$ could be instantiated with $\forall x\,(q\,x)$ for some $q \in \Sigma$, which is not an *hohc* goal. Substituting a term of $\mathscr{H}_1^\Sigma$ into a goal or a clause keeps it in its category. Intuitively, we never substitute something that could not appear as the argument of an atom or in the structure of a goal [MN p. 123]. In an implementation, t is not guessed: it is a logic variable that higher-order unification later fixes in INITIAL.

Theorem 5.5 (Completeness of goal-directed search for *hohc* [MN p. 123–124], [MN p. 148])**.** *Derivability of goals from program clauses in this calculus coincides exactly with provability in classical higher-order logic. By the bibliographic notes, this holds for both the classical and the intuitionistic versions of Church's Simple Theory of Types, without the axioms of extensionality, infinity and choice [45].*

Since $\forall R$ and $\supset R$ never fire in *hohc*, the signature and the program stay fixed during a computation [MN p. 124].

Definition 5.6 (*hohh*: higher-order hereditary Harrop formulas [MN p. 124])**.** Let A range over atoms in $\mathscr{H}_2^\Sigma$ and A_r over rigid atoms in $\mathscr{H}_2^\Sigma$:

$$G ::= \top \mid A \mid G \wedge G \mid G \vee G \mid \exists_\tau x\, G \mid D \supset G \mid \forall_\tau x\, G$$
$$D ::= A_r \mid G \supset D \mid D \wedge D \mid \forall_\tau x\, D$$

The instances for $\exists R$ and $\forall L$ are terms of $\mathscr{H}_2^\Sigma$, where Σ is the signature *current at that point of the derivation.*

Goals are richer than $\mathscr{H}_2^\Sigma$ (they may contain $\supset$; no term of $\mathscr{H}_2^\Sigma$ does). Derivability now coincides with provability in higher-order *intuitionistic* logic [37] [MN p. 124]. Unlike *hohc*, $\forall R$ (GENERIC) and $\supset R$ (AUGMENT) are active, so the signature and the program change during computation, as in Chapter 3.

5.1.5 hohh$^+$: hiding predicate names

Chapter 3 could limit the *scope* of an auxiliary predicate's clauses with $\supset$, but not the *visibility of its name*. A context that already had clauses for `rev` could corrupt a `reverse` that assumed its own `rev`. One small liberalisation fixes this.

Definition 5.7 (*hohh*$^+$ [MN p. 125])**.** *hohh*$^+$ is *hohh* in which the head of an atomic program clause may also be a *variable*, provided that variable is bound by an *essentially universal* quantifier that has the clause in its scope.

In a goal, a bound variable is essentially universal if it is bound by a positive $\forall$, a negative $\exists$, or a term-level λ. It is essentially existential otherwise. The definition is dualised for clauses ([MN p. 205]; see Chapter 6). Essentially universal variables only ever become eigenvariables. When search reaches the goal $\forall p\,(\ldots D \ldots \supset G)$, $\forall R$ has already replaced p by a *new constant*, so by the time D is added to the program it is a legal *hohh* clause with a rigid head. Hence *hohh*$^+$ does not change computational behaviour; it only lets a program introduce *closed, hidden* predicate definitions. The book's two versions of `reverse` [MN p. 126]:

```
type reverse   list A -> list A -> o.
reverse L K :-
  pi rev\
    (pi L\ rev nil L L) &
    (pi X\ pi L\ pi K\ pi M\ rev (X::L) K M :- rev L K (X::M))
    => rev L K nil.
```

```
type reverse   list A -> list A -> o.
reverse L K :- pi rv\
  ( rv nil K &
    (pi X\ pi N\ pi M\ rv (X::N) M :- rv N (X::M)))
  => rv L nil.
```

Tutorial: What the interpreter does with `reverse (1::2::3::nil) K`

Using the first version [MN p. 125]:

1. backchain on `reverse`; the body is a `pi` goal, so $\forall R$ generates a new predicate symbol, call it `c`, and adds it to the signature;
2. the body is now an implication, so $\supset R$ adds the two clauses

   ```
   pi L\ c nil L L.
   pi X\ pi L\ pi K\ pi M\ c (X::L) K M :- c L K (X::M).
   ```

3. it proves `c (1::2::3::nil) K nil` by ordinary backchaining and obtains `K = (3::2::1::nil)`;
4. on leaving the goal, it removes `c` and its clauses from the context.

No clause outside `reverse` can mention `c`: the name did not exist when those clauses were written.

Pitfall: Shadowing and shared variables in assumed clauses

In the first version the inner `pi L\` and `pi K\` *shadow* the outer `L` and `K`. In the second version the assumed fact `rv nil K` mentions the *outer* logic variable `K`, so the hypothetical clause is not closed: `K` is shared with the caller and is *not* renamed at each use of the clause.

Kanren angle: Hidden predicates are lexical scope

In a Scheme-hosted miniKanren a relation is a procedure, and Scheme already has a way to make a name invisible: a local definition. The first `reverse`, with `rev` hidden, is one `let`:

```
;; Figure 5.1 with the hidden predicate rev as a lexically local relation
(define reverse°
  (let ()
    (define-relation (rev l k m)
      (cond° ((null° l) (=° k m))
             ((fresh° (x l*) (cons° x l* l) (rev l* k (cons x m))))))
    (λ (l k) (rev l k '()))))
(show 1 (µkanren-run (q -1 #t) (reverse° '(1 2 3) q)))
(show 2 (µkanren-run (q 1 #t) (reverse° q '(1 2 3))))
```

```
1 ((3 2 1))
2 ((3 2 1))
```

No client of `reverse°` can add clauses to `rev`. This is the encapsulation that *hohh*$^+$ provides, and it costs nothing. (The second query asks for one answer: in that mode a request for all answers does not terminate.) What lexical scope does *not* give is a clause set that *grows during the search*, such as the memo table of Example 5.9, or a hidden name that exists as a first-class *constant*, one that can be stored in terms and must not escape its scope. Both need new machinery in the state: a dynamic clause store for ⊃, and eigenvariables with a scope check for ∀. See Chapter 11.

5.2 Higher-order relational programming

5.2.1 Mapping, filtering and quantifying over lists

The basic higher-order relations of the book [MN p. 127]:

```
type foreach, forsome   (A -> o) -> list A -> o.
type mappred            (A -> B -> o) -> list A -> list B -> o.
type sublist            (A -> o) -> list A -> list A -> o.

foreach P nil.
foreach P (X::L) :- P X, foreach P L.
```

```
forsome P (X::L) :- P X; forsome P L.

mappred P nil nil.
mappred P (X::L) (Y::K) :- P X Y, mappred P L K.

sublist P (X::L) (X::K) :- P X, sublist P L K.
sublist P (X::L) K :- sublist P L K.
sublist P nil nil.
```

Read relationally [MN p. 126]: `mappred P L K` says the lists have equal length and are related pointwise by `P`; `forsome P L` says some member satisfies `P`; `foreach P L` says all members do; `sublist P L K` says `K` is an order-preserving sublist of `L` whose members all satisfy `P`. Because of the clause order, the *maximal* sublists come first. A small database:

```
kind name    type.
type bob, sue, ned   name.
type age     name -> int -> o.
type male, female    name -> o.

age bob 23 & age sue 24 & age ned 23.
male bob & female sue & male ned.
```

Example 5.8 (Queries over the database) [MN p. 126–127]**.**

```
?- mappred age (ned::bob::sue::nil) L.
L = (23::23::24::nil)
?- mappred age L (23::24::nil).
L = (bob::sue::nil);
L = (ned::sue::nil)
?- sublist male (ned::bob::sue::nil) L.
L = ned::bob::nil;
L = ned::nil;
L = bob::nil;
L = nil;
no
?- forsome female (ned::bob::sue::nil).
solved
?- foreach female (ned::bob::sue::nil).
no
```

The second query runs `mappred` "backwards": `age` is a relation, not a function, so the list of names is computed from the list of ages.

5.2.2 λ-terms of predicate type as arguments

A predicate argument need not be a constant. `x\ age jane x` has type `int -> o` (it is η-equivalent to `age jane`), and `n\ age n 24` has type `name -> o` [MN p. 129]. The book reports:

```
?- (n\ age n 24) W.          % β-equal to ?- age W 24.
W = sue
?- mappred (x\y\ age x y) (ned::bob::sue::nil) L.
L = (23::23::24::nil)
```

```
?- mappred (x\y\ age y x) (23::24::nil) K.
K = (bob::sue::nil);
K = (ned::sue::nil)
?- foreach (x\ sigma y\ age x y) (ned::bob::sue::nil).
solved
?- foreach (x\ age x A) (ned::bob::sue::nil).
no
?- foreach (x\ age x A) (ned::bob::nil).
A = 23
```

Pitfall: A logic variable inside a λ is shared, not abstracted

In `x\ age x A`, the variable `A` lies *outside* the abstraction: the same `A` is used by every instance of the body. The last two queries therefore ask for a *common* age. Compare `x\ sigma y\ age x y`, where each element gets its own `y`.

Kanren angle: Predicate variables with known values are free

In a Scheme-hosted miniKanren, a predicate argument whose value is known when it is called is just a procedure that returns a goal. `x\y\ age y x` becomes `(λ (x y) (age° y x))`, and `x\ sigma y\ age x y` becomes `(λ (x) (fresh° (y) (age° x y)))`. A logic variable captured by the Scheme `λ` is shared by every call, exactly as in the pitfall above:

```
;; Figure 5.4: the age database, one relation per predicate constant
(define-relation (age° n a)
  (cond° ((=° n 'bob) (=° a 23))
         ((=° n 'sue) (=° a 24))
         ((=° n 'ned) (=° a 23))))
(define-relation (male° n) (or° (=° n 'bob) (=° n 'ned)))
;; Figure 5.3: the argument P is a Scheme procedure returning a goal
(define-relation (mappred° P L K)
  (cond° ((null° L) (null° K))
         ((fresh° (X L* Y K*) (cons° X L* L) (cons° Y K* K)
            (P X Y) (mappred° P L* K*)))))
(define-relation (foreach° P L)
  (cond° ((null° L))
         ((fresh° (X L*) (cons° X L* L) (P X) (foreach° P L*)))))
(define-relation (sublist° P L K)
  (cond° ((fresh° (X L* K*) (cons° X L* L) (cons° X K* K)
            (P X) (sublist° P L* K*)))
         ((fresh° (X L*) (cons° X L* L) (sublist° P L* K)))
         ((null° L) (null° K))))

(show 1 (µkanren-run (q -1 #t) (mappred° age° '(ned bob sue) q)))
(show 2 (µkanren-run (q -1 #t) (mappred° age° q '(23 24))))
(show 3 (µkanren-run (q -1 #t) (sublist° male° '(ned bob sue) q)))
;; λ-terms of predicate type are Scheme lambdas
(show 4 (µkanren-run (q -1 #t) (mappred° (λ (x y) (age° y x)) '(23 24) q)))
```

```
(show 5 (μkanren-run (q -1 #t)
          (foreach° (λ (x) (fresh° (y) (age° x y))) '(ned bob sue))
          (=° q 'solved)))
(show 6 (μkanren-run (q -1 #t) (foreach° (λ (x) (age° x q)) '(ned bob sue))))
(show 7 (μkanren-run (q -1 #t) (foreach° (λ (x) (age° x q)) '(ned bob))))
```

```
1 ((23 23 24))
2 ((bob sue) (ned sue))
3 (() (bob) (ned) (ned bob))
4 ((bob sue) (ned sue))
5 (solved)
6 ()
7 (23)
```

Every answer *set* matches the book. The *order* of the `sublist` answers (line 3) does not, because `cond°` interleaves its branches while λProlog searches them depth-first in clause order. Compare answer sets unless the search strategy is part of the specification.

5.2.3 Closures, composition and folds

Relations that take relations and return relations [MN p. 127–128]:

```
type ref, sym, trans   (A -> A -> o) -> A -> A -> o.
ref   R X Y :- X = Y; R X Y.
sym   R X Y :- R X Y; R Y X.
trans R X Y :- R X Y.
trans R X Z :- R X Y, trans R Y Z.

kind node  type.
type a, b, c, d, e   node.
type adj   node -> node -> o.
adj a b & adj b c & adj b d & adj d c & adj c e.
```

Both `?- trans adj a d.` and `?- sym adj b a.` succeed. Thanks to currying, the equivalence closure is just the partial application `(trans (sym (ref adj)))`. The non-recursive closures can also be written as λ-terms, for example `(x\y\ x = y ; adj x y)`. The book adds a warning [MN p. 128]. The transitive closure can be written *declaratively* as a single term,

```
x\y\ pi p\
   (pi U\ pi V\ adj U V => p U V) =>
   (pi U\ pi V\ pi W\ adj U V => p V W => p U W) =>
   p x y
```

but it contains $\supset$. It is therefore *not* in $\mathscr{H}_2^{\Sigma}$ and cannot be passed as an argument (Section 5.1.3).

Union, composition and a left fold [MN p. 130]:

```
type union    (A -> B -> o) -> (A -> B -> o) -> A -> B -> o.
type compose  (A -> B -> o) -> (B -> C -> o) -> A -> C -> o.
```

```
type foldl    (A -> B -> B -> o) -> list A -> B -> B -> o.

union   R S X Y :- R X Y; S X Y.
compose R S X Y :- R X Z, S Z Y.
foldl P nil X X.
foldl P (Z::L) X Y :- P Z X W, foldl P L W Y.
```

If `P : A -> B -> B -> o` is a family of binary relations on `B` indexed by `A`, then `(foldl P)` composes them one after another along a list. Reversing a list then means pushing every element onto a stack and popping them all off [MN p. 130–131]:

```
kind stack   type -> type.
type emp     stack A.
type stk     A -> stack A -> stack A.
type empty   stack A -> o.
type enter, remove   A -> stack A -> stack A -> o.
type reverse list A -> list A -> o.

empty emp.
enter X S (stk X S).
remove X (stk X S) S.
reverse L K :- compose (foldl enter L) (foldl remove K) emp emp.
```

Kanren angle: Partial application is curried `define`

λProlog is curried, while `define-relation` has a fixed arity. A relation-returning relation is written with CHICKEN's curried `define`, `(define ((sym° R) x y) ...)`; a recursive one defines a local relation and returns it:

```
(define-relation (adj° x y)
  (cond° ((=° x 'a) (=° y 'b)) ((=° x 'b) (=° y 'c)) ((=° x 'b) (=° y 'd))
         ((=° x 'd) (=° y 'c)) ((=° x 'c) (=° y 'e))))
;; curried closure builders: (sym° R) is itself a binary relation
(define ((ref° R) x y) (or° (=° x y) (R x y)))
(define ((sym° R) x y) (or° (R x y) (R y x)))
(define (trans° R)
  (define-relation (T x z) (cond° ((R x z)) ((fresh° (y) (R x y) (T y z)))))
  T)
(show 1 (µkanren-run (q -1 #t) ((trans° adj°) 'a 'd) (=° q 'solved)))
(show 2 (µkanren-run (q -1 #t) ((sym° adj°) 'b 'a) (=° q 'solved)))
(show 3 (µkanren-run (q -1 #t) ((trans° adj°) 'a q)))
(show 4 (µkanren-run (q 1 #t)
          ((trans° (sym° (ref° adj°))) 'e 'a) (=° q 'solved)))
```

```
1 (solved)
2 (solved)
3 (b c d e c e)
4 (solved)
```

Line 3 shows that `trans` is a relation, not a set: `c` and `e` come back once per path. Line 4 uses the equivalence closure built by partial application. Composition and the stack-based reverse:

```
;; Figures 5.5 and 5.6: compose, foldl, and reverse through a stack
(define ((compose° R S) x y) (fresh° (z) (R x z) (S z y)))
(define-relation (foldl° P L X Y)
  (cond° ((null° L) (=° X Y))
         ((fresh° (Z L* W) (cons° Z L* L) (P Z X W) (foldl° P L* W Y)))))
(define-relation (enter° X S S*) (=° S* `(stk ,X ,S)))
(define-relation (remove° X S S*) (=° S `(stk ,X ,S*)))
(define-relation (reverse° L K)
  ((compose° (λ (s s*) (foldl° enter° L s s*))
             (λ (s s*) (foldl° remove° K s s*)))
   'emp 'emp))
(show 1 (μkanren-run (q -1 #t) (reverse° '(1 2 3) q)))
(show 2 (μkanren-run (q -1 #t) (foldl° enter° '(1 2 3) 'emp q)))
```

```
1 ((3 2 1))
2 ((stk 3 (stk 2 (stk 1 emp))))
```

The partial application `(foldl enter L)` becomes an explicit `λ`. None of this needs anything beyond Scheme closures.

5.2.4 Binary clauses: continuation passing

Predicate variables of type o are *goal-valued*, so a program can pass its own continuation [MN p. 131]. For each predicate $p : \tau_1 \to \cdots \to \tau_j \to \mathrm{o}$ introduce $p' : \tau_1 \to \cdots \to \tau_j \to \mathrm{o} \to \mathrm{o}$, and write A' for $(p'\, t_1 \ldots t_j)$ when $A = (p\, t_1 \ldots t_j)$. Then rewrite every clause:

$$\forall \bar{x}\, [A_1 \wedge \cdots \wedge A_n \supset A_0]\ (n > 0) \ \rightsquigarrow\ \forall \bar{x}\, \forall K\, [\, (A_1' \,(\ldots (A_n' \, K) \ldots)) \supset (A_0' \, K)\,]$$
$$\forall \bar{x}\, A_0 \ \rightsquigarrow\ \forall \bar{x}\, \forall K\, [\, K \supset (A_0' \, K)\,]$$

with $K : \mathrm{o}$. Conjunction becomes explicit sequencing, and every clause body has at most one, possibly complex, atom. For example

```
adj a b.
adj b c.
path X Y :- adj X Y.
path X Y :- adj X Z, path Z Y.
```

becomes

```
adj' a b K :- K.
adj' b c K :- K.
path' X Y K :- adj' X Y K.
path' X Y K :- adj' X Z (path' Z Y K).
```

and `?- path X Y.` is provable if and only if `?- path' X Y true.` is provable [MN p. 132].

Kanren angle: Goals are already first-class

In µKanren a goal is a value, so the binary-clause form is an idiom, not a feature. The continuation is a goal argument:

```
;; Section 5.3: binary clauses, the continuation K is a goal
(define (adj/k° x y k)
  (cond° ((=° x 'a) (=° y 'b) k) ((=° x 'b) (=° y 'c) k)))
(define-relation (path/k° x y k)
  (cond° ((adj/k° x y k))
         ((fresh° (z) (adj/k° x z (path/k° z y k))))))
(show 1 (µkanren-run (q -1 #t)
          (fresh° (x y) (path/k° x y ✓°) (=° q (list x y)))))
(show 2 (µkanren-run (q -1 #t)
          (fresh° (x y) (path/k° x y (=° q (list x y))))))
```

```
1 ((a b) (b c) (a c))
2 ((a b) (b c) (a c))
```

In the second query the continuation does the work of the query itself. Conjunction in µKanren (`and`[a], which feeds every state of the first goal to the second) is this same continuation composition.

5.2.5 Memoisation with hidden, growing predicates

Example 5.9 (Memoised Fibonacci in *hohh*$^{\pm}$ [MN p. 132]). **Example 5.9** (Memoised Fibonacci in *hohh*$^{\pm}$ [MN p. 132]).

```
fib_memo N G :-
   pi memo\ pi loop\
     memo 0 0 => memo 1 1 =>
     (pi F1\ pi F2\ pi F3\ pi C\ pi C'\
        (loop C F1 F2 :- (C >  N, (G memo)) ;
                         (C =< N, C' is C + 1, F3 is F1 + F2,
                          memo C F3 => loop C' F2 F3))) =>
     loop 2 0 1.
```

`memo` and `loop` are hidden predicates, bound by `pi` and used as heads of assumed clauses. The memo table *grows* through `memo C F3 => ...`. When `C > N`, the continuation `G`, a predicate that takes a predicate, is called with the hidden name `memo`, whose extension is then $\{(0,0),(1,1),\ldots,(N, fib\,N)\}$. For the query

```
?- fib_memo 20 (fib\ sigma M\ fib N M, M is N * N).
```

the book states, without listing them, that "there are exactly three values for N that make this atom provable" [MN p. 132]. A short computation gives $N \in \{0, 1, 12\}$: the continuation asks for $fib\,N = N \cdot N$, and $fib\,0 = 0$, $fib\,1 = 1$, $fib\,12 = 144$.

This example mixes three things: higher-order programming, dynamic clause addition, and a hidden name handed to a continuation. A Scheme `letrec` cannot express the growing table, which is why the kanren box of Section 5.1.5 lists dynamic clauses as a genuine requirement.

5.3 Flexible goals

What happens when a goal's head is a variable that is *still unbound* when the goal is reached? Take `P bob 23` against the age database. `P := age` works, but so do `x\y\ age x 23, age ned y`, `x\y\ age sue 24`, `x\y\ memb 4 (3::4::5::nil)`, and in fact `x\y\ g` for *any* provable closed goal `g` [MN p. 132–133]. Such a goal is *underconstrained*. Three treatments are possible [MN p. 134]:

1. **Suspend** the goal in the hope that other goals will instantiate its head, and resume it when it becomes rigid. If only flexible goals remain, solve each of them by instantiating its head with $\lambda x_1. \ldots \lambda x_n. \top$, the universally true relation of the right type, which is "the (extensionally) largest one that works".
2. **Eagerly** bind the head to the universally true relation.
3. Issue a **run-time error**.

The first option is complete with respect to provability in intuitionistic higher-order logic. The other two are incomplete. Yet experience with λProlog implementations says the third is the most useful while developing code: a flexible goal met by depth-first search is usually a bug, for example a predicate constant mistyped with a capital letter [MN p. 134]. Option 1 is the same idea that unification-based interpreters apply to flexible-flexible unification problems: the completeness results "provide the foundations for a unification-based interpreter … that explicitly delays the consideration of flexible-flexible unification problems as well as flexible goals" [MN p. 148].

A predicate variable becomes meaningful when the program first *constrains* it to a structured space of candidates [MN p. 133]:

```
kind i  type.
type jane, mary, john   i.
type mother, father, wife, husband   i -> i -> o.
type primrel, rel   (i -> i -> o) -> o.

primrel father & primrel mother & primrel wife & primrel husband.
rel R :- primrel R.
rel (x\y\ sigma z\ R x z, S z y) :- primrel R, primrel S.
mother jane mary & wife john jane.
```

`?- R john mary.` on its own is underconstrained. `?- rel R, R john mary.` is meaningful: `rel` enumerates candidate relations, after which `R john mary` is rigid. The only answer is `R = x\y\ sigma Z\ wife x Z, mother Z y`. Note that the head of the second `rel` clause contains a λ-term with `∃` and ∧. This is legal because the term lies in $\mathscr{H}_1^{\Sigma}$. Whether a flexible goal is underconstrained thus "depends on when the goal is invoked" [MN p. 133]: goal order matters operationally.

Tutorial: Depth-first search for `rel R, R john mary`

Rename the clause variables of the second `rel` clause to `R1` and `S1`, so that they are not confused with the query's `R`.

1. `rel R` backchains on the first clause and `primrel R` binds `R := father`. The second goal is now the *rigid* atom `father john mary`, which has no clauses and fails. Backtracking tries `mother`, `wife` and `husband` in turn; all fail the same way.

2. `rel R` backchains on the second clause: `R := x\y\ sigma z\ R1 x z, S1 z y`, with the subgoals `primrel R1` and `primrel S1`. After they bind `R1` and `S1`, the goal `R john mary` is β-reduced to `sigma z\ R1 john z, S1 z mary`, which is again built from rigid atoms.

3. With `R1 := father` or `R1 := mother` the atom `R1 john z` fails for every choice of `S1`. With `R1 := wife` it gives `z := jane`; then `S1 := father` fails on `father jane mary` and `S1 := mother` succeeds on `mother jane mary`.

The answer is `R` with `R1` and `S1` instantiated, which is the book's answer. The predicate variable is never invoked while unbound: `rel` always gives it a value first.

Kanren angle: Binding a logic variable to a relation: possible, but opaque

Any Scheme value can be bound to a logic variable, procedures included, so `rel` can be transcribed. The composed relation is built by `project°` once `r` and `s` are known, and invoking the value of `R` means walking it and applying it:

```
(import (chicken condition))
;; Figure 5.8: relations as values of logic variables
(define-relation (mother° x y) (=° x 'jane) (=° y 'mary))
(define-relation (wife° x y) (=° x 'john) (=° y 'jane))
(define-relation (father° x y) ✗°)
(define-relation (husband° x y) ✗°)
(define-relation (primrel° R)
  (cond° ((=° R father°)) ((=° R mother°)) ((=° R wife°)) ((=° R husband°))))
(define-relation (rel° R)
  (cond° ((primrel° R))
         ((fresh° (r s) (primrel° r) (primrel° s)
            (project° ((r* r) (s* s))
              (=° R (λ (x y) (fresh° (z) (r* x z) (s* z y)))))))))
;; invoking the value of R: walk it, then apply it
(define-relation (rel-john-mary° R)
  (rel° R) (project° ((R* R)) (R* 'john 'mary)))
(show 1 (µkanren-run (q -1 #f) (rel-john-mary° q)))
(show 2 (µkanren-run (q -1 #f) (fresh° (R) (rel-john-mary° R) (=° q 'found))))
;; a flexible goal: R is still unbound when it is invoked
(show 3 (handle-exceptions exn
            (list 'error ((condition-property-accessor 'exn 'message) exn))
          (µkanren-run (q -1 #t) (project° ((R* q)) (R* 'bob 23)))))
```

```
1 ((λ () #<procedure (? x y)>))
2 ((λ () (quote found)))
3 (error "call of non-procedure")
```

The search finds the single answer (line 2), but the answer itself is `#<procedure (? x y)>`, not `x\y\ sigma Z\ wife x Z, mother Z y`. Grounded reification (`#t`) even fails on it, because it `eval`s the reified expression, and `eval` rejects a procedure as an "illegal non-atomic object". Line 3 is a flexible goal: `project°` hands over an unbound variable record, and applying it is a Scheme error. That is the book's option 3, but only by accident. Options 1 and 2 need a goal that can be *inspected* before it is run. Option 1 also needs a store of suspended goals that unification wakes up, much as the disequality store is rechecked after each `=°`. Printing `R` as a λ-term needs relations represented as terms. The proposal keeps relations as closures (Section 14.2.1); predicate quantification is sketched as Section 14.5.5.

5.4 Reasoning about programs by substitution

Logic gives a means of *reasoning* about programs, and hidden predicates make it sharper [MN p. 134–136]. Claim: the second `reverse` of Section 5.1.5 is symmetric, that is, if `reverse L K` is provable then so is `reverse K L`. The proof outline [MN p. 135–136]:

1. The only way to prove `reverse L K` is by backchaining, so its body is provable:

   ```
   pi rv\ ( rv nil K &
            (pi X\ pi N\ pi M\ rv (X::N) M :- rv N (X::M)))
          => rv L nil
   ```

2. A provable universal has provable instances. Instantiate `rv` with `x\y\ (not (rv y x))`, where `not` is *logical* negation, not negation as failure:

   ```
   not(rv K nil) &
   (pi X\ pi N\ pi M\ not(rv M (X::N)) :- not(rv (X::M) N))
   => not(rv nil L).
   ```

3. Take the contrapositive of each implication, which is classically valid:

   ```
   rv nil L & (pi X\ pi N\ pi M\ rv (X::M) N :- rv M (X::N))
   => rv K nil.
   ```

4. This is a first-order Horn entailment, and on Horn clauses classical and intuitionistic provability coincide.

5. Generalise over `rv` and rename `M` and `N`. The result is the body of `reverse` with `L` and `K` swapped. □

Intuitively, the trace of `rv` on `(a::b::c::nil)` is a table with rows `(a::b::c::nil) nil`, `(b::c::nil) (a::nil)`, `(c::nil) (b::a::nil)`, `nil (c::b::a::nil)`. Swapping the columns and reading the rows in reverse order gives a valid trace for the reverse of `(c::b::a::nil)` [MN p. 135].

Three subtleties are worth noting. The `rv` inside the instantiating term `x\y\ (not (rv y x))` is a *new* predicate variable that happens to reuse the name; step 5 generalises over it. The instantiating term contains $\neg$, that is $\supset \bot$, so it is *not* in $\mathscr{H}_2^{\Sigma}$. That is fine, because the argument uses the metatheory (soundness, then instantiation, contraposition and completeness),

not goal-directed search. And the argument needs `rv` to be hidden: no other clause can mention it, so any λ-term may be "plugged in" at its place.

Kanren angle: Not available to closures

A relation that is a Scheme closure offers nothing to substitute into: the argument above treats the program as a *formula*. A port can at most turn the theorem into a property test (for random lists, `reverse L K` implies `reverse K L`). Reasoning of this kind belongs to a deep embedding, where clauses are data.

5.5 Defining the logical constants

Predicate variables are powerful enough to *define* logical constants inside the logic [MN p. 136–137]. In *hohc*, clauses can specify the right-introduction rules:

```
type tt, ff   o.
type or       o -> o -> o.
type exists   (A -> o) -> o.
tt.                   % true
or P Q :- P.          % disjunction
or P Q :- Q.
exists B :- B T.      % existential quantifier
```

`ff` has no clauses, so proving it fails. Only the right rules are captured. The left rules (for example the rule of cases for a disjunctive hypothesis) are not. In *hohh* and *hohh*$^+$, $\forall$ and $\supset$ give the standard impredicative encodings [MN p. 136]:

$$\bot \equiv \forall p\, p \qquad \top \equiv \forall p\, (p \supset p)$$
$$B \vee C \equiv \forall p\, ((B \supset p) \supset (C \supset p) \supset p) \qquad \exists x\, (G\, x) \equiv \forall p\, ((\forall x\, (G\, x \supset p)) \supset p)$$

(the last three are *hohh*$^+$ goals, since p heads assumed clauses and is bound by a positive $\forall$).

Tutorial: Running the encodings

Goal-directed search reads them as follows. For $\forall p\, p$, p becomes a fresh constant with no clauses, so the goal fails. For $\forall p\, ((B \supset p) \supset (C \supset p) \supset p)$, a fresh c is created, the clauses c `:-` B and c `:-` C are assumed, and c is proved by backchaining on either clause, that is, by proving B or C. For $\forall p\, ((\forall x\, (G\, x \supset p)) \supset p)$, a fresh c is created, the clause $\forall x\, (c$ `:-` $G\, x)$ is assumed, and c is proved by backchaining: x becomes a logic variable X and the goal $G\, X$ remains.

Kanren angle: Which definitions are free

The *hohc* definitions are one-liners over existing goals: `tt` is `✓°`, `ff` is `✗°`, `or` is `or°`, and `exists B :- B T` is `(fresh° (t) (B t))`. The *hohh* encodings are not free. They need a goal-level $\forall$ that creates a fresh constant and a goal-level $\supset$ that assumes clauses for it.

Those are the two state extensions of the kanren box in Section 5.1.5.

5.6 Conditional and negation-as-failure

With the cut `!`, the book defines a conditional and negation-as-failure [MN p. 137]:

```
type if   o -> o -> o -> o.
if P Q R :- P, !, Q.
if P Q R :- R.

type not  o -> o.
not P :- P, !, fail.
not P.
% alternatively
not P :- if P fail true.
```

`if P Q R` tries `P`. If it succeeds, the cut commits to its first solution and `Q` is proved; only if `P` fails is `R` proved. This is not logic: `?- X = 2, not (1 = X).` succeeds, while `?- not (1 = X), X = 2.` fails. "Thus even a basic property such as commutativity of conjunctions becomes invalid in this setting", and the book uses these predicates "extremely sparingly" [MN p. 137–138].

Kanren angle: The cut as take° 1

(aux kanren micro) has the soft cut `if°`, which keeps *every* answer of its test (like miniKanren's `conda`), and `take°`. The cut-based `if` commits to the first answer, so the faithful transcription of `not P` is `(if° (take° 1 P) ✗° ✓°)`. The `take° 1` is not cosmetic: without it, a test with infinitely many answers would make the conditional fail on each of them forever. The non-commutativity reproduces exactly, and the disequality constraint `≠°` is the declarative alternative that commutes:

```
;; Section 5.7: not P :- if P fail true, the cut read as "first answer only"
(show 1 (μkanren-run (q -1 #t) (=° q 2) (if° (take° 1 (=° 1 q)) ✗° ✓°)))
(show 2 (μkanren-run (q -1 #t) (if° (take° 1 (=° 1 q)) ✗° ✓°) (=° q 2)))
;; the declarative alternative: a disequality constraint commutes
(show 3 (μkanren-run (q -1 #t) (=° q 2) (≠° 1 q)))
(show 4 (μkanren-run (q -1 #t) (≠° 1 q) (=° q 2)))
```

```
1 (2)
2 ()
3 (2)
4 (2)
```

5.7 λ-terms as functions

So far the higher-order variables have been predicate variables. Variables of non-predicate functional type are instantiated by λ-terms that are only *evaluated*, by $\beta\eta$-conversion. As Chapter 4 showed, these functions are weak. "This weakness, however can be an asset for a different kind of computation: Rather than instantiating function variables with λ-terms, we can think of finding values for them by solving unification problems" [MN p. 138].

5.7.1 `mapfun` and `reducefun`

```
type mapfun     (A -> B) -> list A -> list B -> o.
type reducefun  (A -> B -> B) -> list A -> B -> B -> o.

mapfun F nil nil.
mapfun F (X::L) ((F X)::K) :- mapfun F L K.

reducefun F nil Z Z.
reducefun F (H::T) Z (F H R) :- reducefun F T Z R.
```

Assume a type `i`, constants `a, b, c, d : i` and `g : i -> i -> i`.

Forwards. `?- mapfun (x\ g a x) (a::b::nil) L.` has exactly one answer, `L = ((g a a)::(g a b)::nil)`, obtained by β-normalising `((x\ g a x) a)` and `((x\ g a x) b)` [MN p. 138]. `mappred` runs arbitrary goals, so it is strictly more powerful. In fact `mapfun F L K :- mappred (x\y\ y = F x) L K.`

Backwards: function synthesis. [MN p. 139]

```
?- mapfun F (a::b::nil) ((g a a)::(g a b)::nil).
F = x\ g a x                        % the only answer
```

The first element alone poses `F a = g a a`, which has four unifiers:

```
(x\ g x x)   (x\ g a x)   (x\ g x a)   (x\ g a a)
```

Only `x\ g a x` also solves `F b = g a b`, so an interpreter that tries another unifier first has to backtrack. Weakness can also show up as failure: `?- mapfun F (a::b::nil) (c::d::nil).` answers `no`. Simply typed λ-terms cannot do case analysis on constants, so no term maps `a` to `c` and `b` to `d` (Chapter 4) [MN p. 139–140].

`reducefun`. Here `+` is only a term constructor, with no arithmetic [MN p. 140]:

```
?- reducefun (x\y\ x + y) (3::4::8::nil) 6 R.
R = 3 + (4 + (8 + 6));
?- reducefun F (4::8::nil) 6 (1 + (4 + (1 + (8 + 6)))).
F = x\y\ 1 + (4 + (1 + (8 + 6)));
F = x\y\ 1 + (x + (1 + (8 + 6)));
F = x\y\ 1 + (x + y);
```

```
no
?- pi z\ reducefun F (4::8::nil) z (1 + (4 + (1 + (8 + z)))).
F = x\y\ 1 + (x + y);
no
```

Tutorial: Why three answers, and then one

Unfolding the two clauses turns the second query into the single equation $F\,4\,(F\,8\,6) = 1{+}(4{+}(1{+}(8{+}6)))$. Its solutions are the bodies obtained by abstracting some occurrences of the constants of the target consistently. The constant function abstracts nothing. The second answer abstracts the `4`. The third abstracts both the `4` and the inner `1 + (8 + 6)`, which has the shape `x + y`. A candidate such as `x\y\ 1 + (4 + y)` fails, because $F\,8\,6$ would give $1 + (4 + 6) \neq 1 + (8 + 6)$. In the third query, 6 is replaced by a new constant z introduced by `pi` *inside* the scope of F's implicit $\exists$. Instantiations of F may not contain z, and that eigenvariable check removes the two answers that would have to copy the base case into F. Only the real fold function is left.

Kanren angle: Forwards is free, synthesising `F` is not

With `F` a Scheme procedure that *builds a term*, `mapfun` and `reducefun` run forwards unchanged. Since `F` is applied to a logic variable while the goal is being built, they also run backwards in the *list* argument:

```
;; Figure 5.10, forward mode only: F is a Scheme procedure that builds a term
(define-relation (mapfun° F L K)
  (cond° ((null° L) (null° K))
         ((fresh° (X L* K*) (cons° X L* L) (cons° (F X) K* K)
            (mapfun° F L* K*)))))
(define-relation (reducefun° F L Z R)
  (cond° ((null° L) (=° Z R))
         ((fresh° (H T R*) (cons° H T L) (=° R (F H R*))
            (reducefun° F T Z R*)))))
(show 1 (µkanren-run (q -1 #t) (mapfun° (λ (x) `(g a ,x)) '(a b) q)))
(show 2 (µkanren-run (q -1 #t)
          (reducefun° (λ (x y) `(+ ,x ,y)) '(3 4 8) 6 q)))
;; F is applied to a logic variable while the goal is built:
;; the term (g a X) contains X, so the list argument can be unknown
(show 3 (µkanren-run (q -1 #t)
          (mapfun° (λ (x) `(g a ,x)) q '((g a a) (g a b)))))
```

```
1 (((g a a) (g a b)))
2 ((+ 3 (+ 4 (+ 8 6))))
3 ((a b))
```

What cannot be transcribed is the case where `F` itself is unknown. A Scheme procedure is opaque, so there is nothing to unify `F a` against. Function synthesis needs λ-terms as data, with β-normalisation, and a unifier that handles applications of a variable. The problems above, `F a = g a a` and `F 4 R = ...`, are *not* higher-order patterns in the sense

of [34], because `F` is applied to constants. The first is a second-order *matching* problem (its right-hand side is closed); the second is close to one (the flexible side also contains the unknown `R`). Both need Huet-style branching [25] (see Chapter 7). Section 14.5.4 discusses where that belongs in (aux kanren micro).

5.7.2 Functional difference lists

A first-order difference list `(dl L K)`, with `K` a suffix of `L`, denotes `L` without `K`. For instance, both of the terms `(dl (1::2::3::4::5::nil) (4::5::nil))` and `(dl (1::2::3::L) L)` denote the list `(1::2::3::nil)`, and with the second form concatenation needs no recursion. A *functional* difference list (due to Brisset and Ridoux [2]) represents the list as a function from its tail to the whole list [MN p. 140–142]. The book's Figure 5.11 collects the labels of a binary tree into a list in two ways, with ordinary and with functional difference lists. Here are the declarations and the functional version:

```
kind dlist    type -> type.
type dl       list A -> list A -> dlist A.

type concat   dlist A -> dlist A -> dlist A -> o.
concat (dl L1 L2) (dl L2 L3) (dl L1 L3).

kind btree    type -> type.
type empty    btree A.
type bt       A -> btree A -> btree A -> btree A.

kind fdlist   type -> type.
type fdl      (list A -> list A) -> fdlist A.

type collect'  btree A -> list A -> o.
type aux'      btree A -> fdlist A -> o.
collect' Bt (A nil) :- aux' Bt (fdl A).
aux' empty (fdl x\ x).
aux' (bt N L R) (fdl x\ A (N::(B x))) :- aux' L (fdl A),
                                         aux' R (fdl B).
```

`(fdl x\ (1::2::3::x))` denotes `(1::2::3::nil)`, with a "pointer" to its tail. Each operation is a single unification [MN p. 142]:

- head and tail: unify with `x\ (H::(T x))`; the head is `H` and the tail is `(fdl T)`;
- last and front: unify with `x\ F (Y::x)`; the last element is `Y` and the front is `(fdl F)`;
- concatenation of `(fdl A)` and `(fdl B)`: `(fdl x\ A (B x))`, that is, function composition;
- conversion to a list: apply to `nil`, as `(A nil)` in `collect'`.

Palindromes then need no explicit reversal [MN p. 142]:

```
type palindrome   fdlist A -> o.
palindrome (fdl x\x).
palindrome (fdl x\ Y::x).
palindrome (fdl x\ Y::(F (Y::x))) :- palindrome (fdl F).
```

```
?- palindrome (fdl x\ a::b::c::b::a::x).
solved
?- palindrome (fdl x\ a::b::a::(F x)).
F = x\ x;
F = x\ b::a::x;
F = x\ a::b::a::x;
F = x\ Y::a::b::a::x;
F = x\ Z::Z::a::b::a::x;
F = x\ Y::Z::Y::a::b::a::x;
F = x\ Y::Z::Z::Y::a::b::a::x;
F = x\ Y::Z::U::Z::Y::a::b::a::x;
F = x\ Y::Z::U::U::Z::Y::a::b::a::x
```

The second query has infinitely many answers. It finds the `F` whose list, appended after `a::b::a`, makes a palindrome.

Pitfall: Types do not enforce the invariants

`(dl (1::2::3::nil) (4::5::nil))` is well typed, but it does not denote a list in the intended sense. Likewise a function that ignores its argument, as in `(fdl x\ (1::2::3::nil))`, has no tail "pointer" [MN p. 143]. (The book prints the second term as `(fdl x\ (1::2::3))`, which is literally ill typed.)

Kanren angle: A closure is an fdl you can only run

In its intended mode (tree given, list wanted), `collect'` needs no higher-order unification. Every clause head is unified with a fresh variable, so the only requirement is λ-terms that can be built and applied. A Scheme closure provides that, using `project°` to compose once the parts are known:

```
;; Figure 5.11, collect' in its intended mode, an fdl as a Scheme closure
(define-relation (aux/fdl° bt F)
  (cond° ((=° bt 'empty) (=° F (λ (x) x)))
         ((fresh° (n l r a b) (=° bt `(bt ,n ,l ,r))
            (aux/fdl° l a) (aux/fdl° r b)
            (project° ((a* a) (b* b))
              (=° F (λ (x) (a* (cons n (b* x))))))))))
(define-relation (collect/fdl° bt L)
  (fresh° (a) (aux/fdl° bt a) (project° ((a* a)) (=° L (a* '())))))
(define tree '(bt 2 (bt 1 empty empty) (bt 3 empty empty)))
(show 1 (μkanren-run (q -1 #t) (collect/fdl° tree q)))
```

```
1 ((1 2 3))
```

`project°` makes this non-relational: it only works when the subtrees are known. The operations that make functional difference lists interesting (head and tail by unification with `x\ (H::(T x))`, last and front with `x\ F (Y::x)`, `palindrome`, and `collect'` run backwards) all unify a pattern *against* a λ-term. They need λ-terms in the term language, and `x\ F (Y::x)` already applies a variable to a non-variable argument (Chapter 7).

5.8 Higher-order unification is not a panacea

It is tempting to make higher-order unification do all the work. The book's examples show why that is often a mistake [MN p. 143–145]. Take `kind i type. type a,b i. type f i -> i -> i.`. The aim is to abstract all occurrences of `a` in `(f a (f a b))`, that is, to obtain `(x\ f x (f x b))`. The naive program:

```
type extract_a  i -> (i -> i) -> o.
extract_a (F a) F.

?- extract_a (f a (f a b)) F.
F = x\ f x (f x b);
F = x\ f x (f a b);
F = x\ f a (f x b);
F = x\ f a (f a b);
no
```

With n occurrences there are 2^n "extractions". A cut, `extract_a (F a) F :- !.`, relies on the order in which the unifiers are enumerated, so it is not declarative. Two better options:

- **Reorder the quantifiers** so that the eigenvariable check does the selection:

  ```
  pi a\ sigma F\ (F a) = (f a (f a b)).   % four proofs
  sigma F\ pi a\ (F a) = (f a (f a b)).   % exactly one: F = x\ (f x (f x b))
  ```

  In the second line `a` is a constant that is new for `F`, so `F` cannot mention it and every occurrence must be abstracted. (This is also a higher-order pattern, `F` applied to a distinct bound variable, which is what makes it well behaved; see Chapter 6.)

- **Recurse on the syntax**, using the signature of `i`:

  ```
  extract_a a x\x.
  extract_a b x\b.
  extract_a (f T S) (x\ f (U x) (V x)) :- extract_a T U, extract_a S V.
  ```

A second example is rewriting by congruence [MN p. 145]:

```
type rewrite  int -> int -> o.
rewrite (0 + X) X.
rewrite (1 * X) X.
rewrite (X - X) 0.
rewrite (C X) (C Y) :- rewrite X Y.
```

The last clause uses `C` as a "context with a hole". `rewrite ((1 * 5) - 5) (5 - 5)` is derivable, but it has *infinitely many* derivations: the last clause can be used any number of times with `C = x\x` before `C = x\ x - 5` is finally chosen. The remedy is again explicit recursion over the structure of terms.

Kanren angle: Interleaving as a home for non-unitary unification

Every example in this section is a case where unification returns a *stream* of solutions. In a miniKanren, alternative unifiers are naturally branches of a disjunction, and interleaving enumerates infinite families fairly. That fits Huet's procedure [25] well. The lesson of the book still applies: design relations so that unification has at most one answer (patterns, structural recursion, the right quantifier order), and treat branching unification as a last resort. Chapter 7 and Section 14.5.4 take this up.

5.9 Comparison with functional programming

Most higher-order idioms of Scheme or ML carry over to logic programming easily (`mappred`, `sublist`, folds). The book argues that the logic programming notion is stronger in two ways [MN p. 146–147]:

1. predicate variables are only one kind of functional variable. Non-predicate function variables give new kinds of computation (Section 5.7), developed in Chapter 6 for syntax with binders;

2. predicate expressions can be *compared*, intensionally:

   ```
   type eq_pred (A -> o) -> (A -> o) -> o.
   eq_pred R R.
   ```

   succeeds if and only if its arguments are unifiable predicate *expressions*. For example, `?- eq_pred (x\ p x, q x) (x\ q x, p x).` fails whatever the clauses for `p` and `q`. The comparison is structural, modulo λ-conversion, not equality of sets. Functional languages forbid comparing functions because their semantics is extensional, and extensional equality is undecidable.

A design remark follows [MN p. 147]. If the only goal were functional-style higher-order programming, a more restricted syntax would do, for example one that forbids quantification over higher-order non-predicate types. What general higher-order quantification adds is λ-terms that represent syntax with binding, together with functional quantification to manipulate them. That was "much of the impetus for the development of λProlog". Nor is λProlog a merger of functional and logic programming: typed λ-terms are too weak as functions for that [MN p. 148].

Kanren angle: Unification on closures is pointer identity

(aux kanren micro) unifies procedures by `eq?`, which is finer than λProlog's λ-convertibility:

```
;; Section 5.10: unification on procedures is pointer identity (eq?)
(define-relation (age° n a)
  (cond° ((=° n 'bob) (=° a 23)) ((=° n 'sue) (=° a 24))))
(define (older-than n)
  (λ (x) (fresh° (a) (age° x a) (project° ((a* a)) (if (> a* n) ✓° ✗°)))))
(show 1 (µkanren-run (q -1 #t) (=° age° age°) (=° q 'same)))
(show 2 (µkanren-run (q -1 #t)
          (=° (older-than 20) (older-than 20)) (=° q 'same)))
(show 3 (µkanren-run (q -1 #t)
          (let ((p (older-than 20))) (=° p p)) (=° q 'same)))
```

```
1 (same)
2 ()
3 (same)
```

Two evaluations of the same `λ` with a captured value give distinct closures (line 2), whereas `eq_pred (x\ p x) (x\ p x)` succeeds in λProlog. Intensional comparison, and every feature that inspects a predicate (printing it, matching on it, `eq_pred`), needs predicates represented as terms.

5.10 Modules: programming in the large with logic

Languages usually start with programming-in-the-small, and later "a second modules language is often imposed on the previously existing core language". Its constructs may have "little or no connection" with the semantics of the core [MN p. 150]. Chapter 6 of the book argues the opposite for logic programming: the abstractions needed for modularity are *already present* in $hohh^+$ [MN p. 150–151]:

- composing code is *conjunction* of program clauses;
- visibility of names is *existential quantification* over program clauses (in effect, ∀ in goals);
- using a module to answer a query is an *implicational goal* $E \supset G$.

The modules language is designed to support abstraction, information hiding, parametrization, interfaces and separate compilation. It must not complicate the meaning of the core language, and it must let higher-order programming coexist with modules (in Prolog, getting `call/1` to interact correctly with modules). Finally, it should support a notion of module equivalence (representation independence) [MN p. 150–151].

5.10.1 Modules, signatures and accumulation

A module is a named sequence of kind, type and clause declarations, and a signature with the same name is its interface [MN p. 152–153]. The book's module `smlists` declares and defines `id`, `memb`, `member` (a deterministic membership test that uses the cut), an auxiliary `revapp`, `reverse` (via `revapp`), `append` and `memb_and_rest` (select an element and return the rest). Its signature is

```
sig smlists.
type id              list A -> list A -> o.
type memb, member    A -> list A -> o.
type reverse         list A -> list A -> o.
type append          list A -> list A -> list A -> o.
type memb_and_rest   A -> list A -> list A -> o.
end
```

The signature restricts the external visibility of the module's names to exactly those it mentions, so `revapp` is hidden. Hiding applies to predicates and also to constants and data constructors, and "such hidden symbols remain hidden even through the results of computations" [MN p. 153]. A query `[smlists] ?- revapp ...` is *ill formed*: a static error, not a run-time failure [MN p. 154]. `accumulate m1, ..., mn.` conceptually inserts the code of the listed modules, after *renaming* their private constants, so that the accumulated copy is entirely local to the accumulating module [MN p. 154]. `accum_sig` inserts signatures, which is how names are re-exported [MN p. 155–156].

A module *matches* a signature when its implicit signature (its own declarations, plus the *explicit* signatures of the modules it accumulates) is *mergeable* with it: kinds are identical, types are identical up to renaming of type variables, and operator declarations agree [MN p. 156–158]. The book's running example [MN p. 158]:

```
module m1.
kind   item   type.
type   p,q    item -> o.
type   a      item.
type   r      list item -> o.
p X :- q X.
r (a :: nil).
end

sig m1.
kind   item   type.
type   p,q    item -> o.
type   a      item.
end
```

```
module m2.
kind   item   type.
type   q, r   item -> o.
type   a      item.
q X :- r X.
r a.
end

sig m2.
kind   item   type.
type   q, r   item -> o.
type   a      item.
end
```

```
module m3.
accumulate m1, m2.
type   s, t   item -> o.
type   b      item.
s X :- p X.
t b.
end

sig m3.
kind   item   type.
type   s, t   item -> o.
type   a      item.
end
```

`r` is hidden in `m1` at type `list item -> o`, while `m2` exports an `r : item -> o`. The two never meet. If `sig m1` also exported `r`, the signatures of `m1` and `m2` would not be mergeable and `m3` would be ill formed [MN p. 159–160]. Hiding is what lets two modules use the same name at incompatible types. The name `q`, on the other hand, is exported by both `m1` and `m2` at the same type, so in `m3` it is *one* constant. It is the only link between `m1`'s clause `p X :- q X` and `m2`'s clause `q X :- r X`.

5.10.2 The logical reading: ∃ over clauses, ⊃ for queries

Two equivalences, valid in most logics, motivate the construction [MN p. 160]:

$$\exists y\, \forall x\, (D(x) \supset G(y)) \equiv \exists y\, ((\exists x\, D(x)) \supset G(y)) \qquad \exists x\, (D_1(x) \wedge D_2) \equiv (\exists x\, D_1(x)) \wedge D_2$$

(when x is not free in $G(y)$, respectively in D_2). The first says that a universal in a goal makes x *local to the clause* $D(x)$. The second lets the scope of such an ∃ be narrowed to part of a large conjunction.

Definition 5.10 (E-formulas [MN p. 161])**.** The goals of $hohh^+$ are extended to

$$\begin{aligned} G &::= \top \mid A \mid G \wedge G \mid G \vee G \mid \exists x\, G \mid E \supset G \mid \forall x\, G \\ D &::= A_r \mid G \supset D \mid D \wedge D \mid \forall x\, D \\ E &::= D \mid E \wedge E \mid \exists x\, E \end{aligned}$$

A rigid head may now also be a variable bound by an ∃ of an enclosing E-formula. E-formulas are not user syntax. They appear only in the translation of modules [MN p. 162].

A program contains only D-formulas, so $\supset R$ is replaced by a small phase that decomposes the antecedent. It uses a sequent $\Sigma; \mathscr{P} \multimap\langle \Delta \rangle\longrightarrow G$, where Δ is a multiset of E-formulas [MN p. 161]:

$$\frac{\Sigma; \mathscr{P} \multimap\langle E \rangle\longrightarrow G}{\Sigma; \mathscr{P} \longrightarrow E \supset G}\ \supset\mathrm{R}' \qquad \frac{\Sigma; \mathscr{P} \longrightarrow G}{\Sigma; \mathscr{P} \multimap\langle\rangle\longrightarrow G}\ \textsc{finish} \qquad \frac{\Sigma; \mathscr{P}, D \multimap\langle \Delta \rangle\longrightarrow G}{\Sigma; \mathscr{P} \multimap\langle D, \Delta \rangle\longrightarrow G}\ \textsc{reclassify}$$

$$\frac{\Sigma; \mathscr{P} \multimap\langle E_1, E_2, \Delta \rangle\longrightarrow G}{\Sigma; \mathscr{P} \multimap\langle E_1 \wedge E_2, \Delta \rangle\longrightarrow G}\ \wedge\mathrm{L}' \qquad \frac{\Sigma, y; \mathscr{P} \multimap\langle E[y/x], \Delta \rangle\longrightarrow G}{\Sigma; \mathscr{P} \multimap\langle \exists x\, E, \Delta \rangle\longrightarrow G}\ \exists\mathrm{L}'\ (y \notin \Sigma)$$

All these rules are *invertible*, so this phase never backtracks. "During this new phase, the local constants are introduced dynamically into computation, and the clauses using these new constants are integrated with the ambient logic program" [MN p. 161]. Note that $\exists L'$ is the same mechanism as $\forall R$ for goals: a fresh constant is added to Σ.

Definition 5.11 (A module as a formula [MN p. 162])**.** Let $h_1, \dots, h_k$ be the hidden constants of a module M, that is, those in its implicit signature but not in its explicit one. If M accumulates $M_1, \dots, M_n$ and has its own clauses $C_1, \dots, C_m$, then

$$[\![M]\!] = \exists h_1 \cdots \exists h_k \, \big([\![M_1]\!] \wedge \cdots \wedge [\![M_n]\!] \wedge C_1 \wedge \cdots \wedge C_m\big).$$

A query `[M] ?- g X.` is the goal $\exists x \, ([\![M]\!] \supset g\ x)$ [MN p. 163].

For the running example [MN p. 162–163]:

$$[\![m1]\!] = \exists r \, \big((\forall x (q\ x \supset p\ x)) \wedge (r\ (a :: nil))\big) \qquad [\![m2]\!] = \forall x (r\ x \supset q\ x) \wedge (r\ a)$$

$$[\![m3]\!] = \exists p \exists q \exists r \exists b \Big(\exists r \big((\forall x (q\ x \supset p\ x)) \wedge (r\ (a :: nil))\big) \wedge (\forall x (r\ x \supset q\ x) \wedge (r\ a)) \wedge (\forall x (p\ x \supset s\ x) \wedge (t\ b)) \Big)$$

The inner $\exists r$ (`m1`'s private `r`) shadows the outer one (`m2`'s exported `r`, hidden again by `m3`), as α-conversion dictates. `a`, `s` and `t` stay free, because they are `m3`'s exports.

Tutorial: Using a hidden constant versus exposing it

The book reports [MN p. 163–164]:

```
[m3] ?- sigma x\ t x.      % succeeds (x instantiated to the hidden constant b)
[m3] ?- t X.                % FAILS
```

`[m3] ?- p a.` is ill formed (`p` is not in `sig m3`), and `[m3] ?- s a.` succeeds. The difference between the first two lies in where the quantifiers sit.

- `t X` is $\exists x \, ([\![m3]\!] \supset t\ x)$. The logic variable X is created *first*, at a signature that does not contain `b`. Then $\supset R'$ and $\exists L'$ introduce a fresh b'. Backchaining on `t b'` needs $X := b'$, which is not a term over X's signature, so the goal fails.
- `sigma x\ t x` is $[\![m3]\!] \supset \exists x \, (t\ x)$. $\exists L'$ adds b' to the signature *before* the existential is processed, so b' is an admissible witness and the goal succeeds.

In the book's words, this is "the difference between using a constant in the course of searching for a solution ... and attempting to expose the identity of this constant externally" [MN p. 164].

Tutorial: The rules of the E-phase at work on `[m3] ?- s a.`

The query has no logic variables, so it is the goal $[\![m3]\!] \supset s\ a$. Read the derivation bottom-up, starting from the empty program:

1. $\supset R'$ moves $[\![m3]\!]$ into the E-multiset: $\Sigma; \emptyset \dashv\langle [\![m3]\!] \rangle \longrightarrow s\,a$.
2. $\exists L'$ four times introduces fresh constants p', q', r', b' for `m3`'s hidden names and adds them to Σ.
3. $\wedge L'$ splits the body into its conjuncts: `m1`'s formula $\exists r\,(\ldots)$, the clauses $\forall x(r'\ x \supset q'\ x)$ and $r'\ a$ from `m2`, and the clauses $\forall x(p'\ x \supset s\,x)$ and $t\,b'$ of `m3`.
4. $\exists L'$ once more introduces a fresh r'' for `m1`'s private `r`, and $\wedge L'$ yields $\forall x(q'\ x \supset p'\ x)$ and $r''\ (a :: nil)$.
5. RECLASSIFY moves each of these D-formulas into the program, and FINISH returns to an ordinary sequent $\Sigma, p', q', r', b', r''; \mathscr{P}_{m3} \longrightarrow s\,a$.
6. Backchaining now proves $s\,a$ from $p'\ a$, $p'\ a$ from $q'\ a$, $q'\ a$ from $r'\ a$, and $r'\ a$ is a fact.

Steps 1–5 make no choices, which is what invertibility means in practice: loading a module never backtracks. The fact $r''\ (a :: nil)$ is never used, and it could not be: no clause outside `m1`'s own formula can mention r''.

Elaboration. Rather than replay Definition 5.11 for every query, an implementation inlines the accumulated modules and *renames* their private names. The elaborated `m3` contains the clauses `p X :- q X.`, `r' (a :: nil).`, `q X :- r X.`, `r a.`, `s X :- p X.` and `t b.`, where `r'` is a new name for `m1`'s local `r` [MN p. 165]. The book's implementation section adds that, at run time, the machinery that already handles alternating $\exists/\forall$ in goals also handles local constants, provided they are annotated properly when the module is used to prove queries [MN p. 172].

5.10.3 Abstract datatypes

Exporting a type constructor while hiding its data constructors gives an abstract datatype [MN p. 166]:

```
sig stack.
kind store          type -> type.
type init           store A -> o.
type add, remove    A -> store A -> store A -> o.
end
```

```
module stack.
kind store          type -> type.
type emp            store A.
type stk            A -> store A -> store A.
type init           store A -> o.
type add, remove    A -> store A -> store A -> o.
init emp.
add    X S (stk X S).
```

```
remove X (stk X S) S.
end
```

`[stack] ?- init A.` *fails*, because it tries to exhibit the representation of the empty stack. `[stack] ?- sigma A\ sigma B\ sigma C\ init A, add 1 A B, remove X B C.` uses the representation anonymously and succeeds with `X = 1` [MN p. 166]. A client such as `graph_search` can hold store values in its variables, but only `init`, `add` and `remove` can examine them [MN p. 167].

Extension and parametrization. An exported predicate can be *extended* by later modules. A propositional prover `proplogic` exports the type `form` and the predicate `prove`, and a module `quantlogic` that accumulates it adds both constructors and clauses [MN p. 168–169]:

```
sig quantlogic.
accum_sig proplogic.
kind term        type.
type all, some   (term -> form) -> form.
end

module quantlogic.
accumulate proplogic, smlists.
kind term        type.
type all, some   (term -> form) -> form.

prove L F :- memb_and_rest (some P) L L',
             pi c \ prove ((P c)::L) F.
prove L (all P) :- pi c\ prove L (P c).
prove L (some P) :- prove L (P T).
prove L F :- memb_and_rest (all P) L L',
             append L' [all P] L'', prove ((P T)::L'') F.
end
```

This works because `form` and `prove` are in `proplogic`'s *explicit* signature, so accumulation neither renames nor hides them. The recursive calls inside `proplogic`'s clauses reach the new quantifier clauses too. Both provers accumulate `smlists`, which gives two private copies. The book calls this "at least wasteful" [MN p. 169] and offers an alternative: accumulate the *signature* `smlists` into both modules, and supply one implementation when everything is accumulated at the top [MN p. 170]. Parametrization is then done by shared names that are closed at the top, not by passing arguments explicitly.

5.10.4 Higher-order programming and predicate visibility

The test case for the second desideratum [MN p. 171]:

```
sig comblibrary.
type  call  o -> o.
end

module comblibrary.
```

```
type  call  o -> o.
type  p  list int -> o.
p (1 :: nil).
call Q :- Q.
end

sig test.
type  test  list int -> o.
end

module test.
accumulate comblibrary.
type  test  list int -> o.
type  p  list int -> o.
p (2 :: nil).
test X :- call (p X).
end
```

For `[test] ?- test X.`, which `p` does `call (p X)` invoke: the one visible where `call` is defined (`X = 1 :: nil`), or the one visible where the name is written (`X = 2 :: nil`)? "The second interpretation is the commonly used resolution", and it "is a natural consequence of the semantics that we have presented" [MN p. 171]. In the elaborated module, `comblibrary`'s `p` is renamed `p'`, so the only answer is `2 :: nil` [MN p. 171–172]. `p X` is a *term* of type `o` whose head is fixed when the program is parsed. It is not a name looked up at run time.

Kanren angle: Modules: what Scheme scoping gives, and what it does not

Scheme's lexical scope already does module elaboration with renaming: a local definition is private, and each evaluation of a scope creates fresh closures. Goals are closures, so Figure 6.10 resolves to the book's "second interpretation" automatically. A Scheme-level abstract type is equally easy:

```
;; Figure 6.10: a "module" is a lexical scope returning its exported relations
(define call°                           ; module comblibrary, exports call
  (let ()
    (define-relation (p l) (=° l '(1)))  ; hidden
    (define (call g) g)
    call))
(define test°                           ; module test, exports test
  (let ()
    (define-relation (p l) (=° l '(2)))
    (define-relation (test x) (call° (p x)))
    test))
(show 1 (µkanren-run (q -1 #t) (test° q)))
;; Figure 6.6: the stack "abstract datatype", constructors made by gensym
(define-values (init° add° remove°)
  (let ((emp (gensym 'emp)) (stk (gensym 'stk)))
    (define-relation (init s) (=° s emp))
    (define-relation (add x s s*) (=° s* (list stk x s)))
```

```
    (define-relation (remove x s s*) (=° s (list stk x s*)))
    (values init add remove)))
(show 2 (μkanren-run (q -1 #t)
          (fresh° (a b c) (init° a) (add° 1 a b) (remove° q b c))))
(show 3 (μkanren-run (q -1 #f) (init° q)))
```

```
1 ((2))
2 (1)
3 ((λ () (quote emp223)))
```

Line 1 is the book's answer, and line 2 uses the stack anonymously, as the `sigma` query does. Line 3 shows the honest difference. Scheme hides the *name* `emp`, but the answer to `init A` still *exhibits* the representation, where λProlog fails. Matching the book needs hidden constructors that are eigen-constants, introduced when the module is used, and a scope check in unification that stops a query variable created earlier from being bound to them. That is the same machinery as ∀ in goals (Section 5.1.5). Two more things are not free. One is *extensible* predicates (`quantlogic` adding clauses to `proplogic`'s `prove`), because a `define-relation` is closed. The other is parametrization by shared names closed at the top. Signature matching is mostly about types and kinds, which an untyped miniKanren can only approximate (names and arities). The proposal is in Chapters 11 and 12. One rule follows directly from Figure 6.10: never resolve a relation from a symbol at run time, because that reintroduces exactly the Prolog `call/1` ambiguity.

5.11 What a Scheme-hosted miniKanren gets for free

Table 5.1 collects the kanren boxes of this chapter.

The dividing line is simple. Whatever only *calls* a predicate or a function is free, because a Scheme procedure can be called. Whatever must *look inside* one (unify it, print it, compare it, synthesise it) needs λ-terms in the term language. A third group, ∀ and ⊃ in goals, needs neither λ-terms nor inspection but does need new components in the search state: eigenvariables with a scope check, and a dynamic clause store. Chapter 11 builds on this classification.

5.12 Summary

- Atoms are *rigid* (constant head) or *flexible* (variable head). Clause heads must be rigid, which keeps programs consistent and modular; flexible atoms are allowed in goals.
- Implications may not occur inside terms. Instantiation terms are drawn from Herbrand universes: $\mathscr{H}_1$ (no ∀, ⊃) for *hohc* and $\mathscr{H}_2$ (no ⊃) for *hohh*. Goal-directed search is then complete: classical and intuitionistic for *hohc*, intuitionistic for *hohh*.
- *hohh*$^+$ lets a variable bound by an essentially universal quantifier head assumed clauses, which gives hidden, closed predicate definitions.

Table 5.1: Book features versus a miniKanren whose relations are Scheme procedures.

Feature	With closures	What is needed beyond closures
`mappred`, `foreach`, `forsome`, `sublist`, folds, `compose`	free	nothing (curried `define` for partial application)
λ-terms of predicate type as arguments	free	nothing (Scheme `λ`)
closures `ref`/`sym`/`trans`, binary clauses (CPS)	free	nothing
hidden predicates with fixed clauses (`reverse`)	free	lexical scope
`if`, `not`	free	`if°` with `take° 1`
`mapfun`/`reducefun` forwards, `collect'` forwards	free	closures that build terms; `project°`
module hiding, predicate visibility (Fig. 6.10)	free	lexical scope
`rel R, R john mary`	opaque answers	relations as terms, to print or compare them
flexible goals	accidental error	inspectable goals; a store of suspended goals
growing hidden predicates (`fib_memo`), `∀`/$\supset$ encodings of $\vee$, `∃`	no	goal-level `∀` (eigenvariables, scope check) and $\supset$ (dynamic clauses)
abstract datatypes that stay hidden in answers	partial	eigen-constants and a scope check
extensible predicates across modules	no	open clause sets, linking
`eq_pred`, reasoning by substitution	no	predicates and clauses as λ-terms
`mapfun`/`reducefun` backwards, `extract_a`, `palindrome`, fdl destructuring	no	λ-terms, $\beta\eta$-normalisation, higher-order unification

- Predicate variables give `mappred`, `sublist`, closures, `compose`, `foldl` and the CPS transformation. Logic variables inside a λ are shared.
- An unbound flexible goal is underconstrained. Suspending it is complete; in practice it is reported as an error. Constrain predicate variables first (`rel R, R john mary`).
- Hidden predicates support reasoning by substitution (the symmetry of `reverse`). Logical constants are definable. `if` and `not` via cut are non-logical.
- Function variables are evaluated by $\beta\eta$ and can be synthesised by unification (`mapfun`, `reducefun`, functional difference lists). Higher-order unification is not unitary, so prefer structural recursion and the right quantifier order.
- Unlike functions in functional programs, predicate expressions can be compared intensionally, with `eq_pred`.
- Modules are logic: conjunction composes code, `∃` over clauses hides names, and a query against a module is $\exists x\,([\![M]\!] \supset G)$. Hidden constants may be used but not exposed, and predicate names resolve statically.

- In a Scheme-hosted miniKanren, calling is free and looking inside is not; ∀ and ⊃ in goals need new state (Table 5.1, Chapter 11).

6 λ-tree syntax: computing with binders

The previous chapters gave us two kinds of machinery. Chapter 4 and Chapter 5 gave us simply typed λ-terms as *data*, with an equality that already contains α-, β- and η-conversion. Chapter 3 gave us the two goal connectives of hereditary Harrop formulas: the universal goal `pi x\ G`, proved with a fresh *eigenvariable*, and the implicational goal `D => G`, proved with the program temporarily extended by `D`. This chapter, which follows Chapter 7 of the book [36] ([MN p. 175]–[MN p. 210]), shows what the two kinds of machinery can do together: they let a logic program compute over syntax that contains *binders*.

This is the chapter that motivates the whole proposal of this document. Every program in it manipulates formulas with quantifiers or λ-terms. None of them contains a single line about variable names, renaming, or capture-avoiding substitution. All of that is "relegated to the meta-level". A miniKanren that wants to run these programs must therefore provide the meta-level: λ-terms as data, eigenvariables with a scope discipline, and a scoped program extension. We will read the book's programs closely, walk through how `pi` and `=>` make a binder "mobile", and, in the *Kanren angle* boxes, record what each concept asks of (aux kanren micro).

6.1 The problem: syntax with binders

Many symbolic objects contain binders [MN p. 175]–[MN p. 176]: the x in $\int f(x)\,dx$, in $\frac{d}{dx}$ and in $\sum_{x=1}^{100}$; quantifiers; λ-abstractions; local declarations and formal parameters in programming languages; type quantifiers of polymorphic types; name restriction in the π-calculus. Programs that handle such objects (differentiators, compilers, interpreters, theorem provers, type checkers) all need the same three notions [MN p. 176]:

1. the distinction between *free* and *bound* occurrences of a variable;
2. the irrelevance of bound names (α-conversion): the formulas $\forall x.\, p(x)$ and $\forall y.\, p(y)$ are the same;
3. *capture-avoiding substitution*, needed both for instantiating quantifiers and for passing parameters.

The book observes that "a computational treatment of such binding-related notions is complicated, and programming language support for realizing them can be valuable" [MN p. 176]. Anyone who has written a substitution function over a named representation, with its renaming case, knows why.

The book's answer has three parts. We state them now and spend the chapter on them.

1. **Encoding.** Each object-level binder becomes a constant applied to a λ-abstraction of the meta-level. So `all (x\ B)` encodes the formula $\forall x.\, B$, and `abs (x\ t)` encodes the term $\lambda x.\, t$.

2. **Substitution is β.** Instantiating the bound variable of an abstraction `B` with `T` is just writing the application `(B T)`; the meta-level β-conversion does the rest.
3. **Mobility.** To recurse *under* a binder, the program turns the term-level λ into a formula-level `pi`, which proof search turns into a proof-level eigenvariable; facts about the new eigenvariable are assumed with `=>`.

6.2 Encoding objects with binding structure

6.2.1 First-order formulas

The object logic is first-order logic over two sorts, `term` and `form`. The propositional connectives are ordinary constants (re-listed on [MN p. 177]):

```
type   ff,          % encoding the false proposition
       tt   form.   % encoding the true proposition
type   &&,          % encoding conjunction
       !!,          % encoding disjunction
       ==>  form -> form -> form.   % encoding implication
type   neg  form -> form.           % encoding negation
infixl &&  5.
infixl !!  4.
infixr ==> 3.
```

A sample non-logical signature [MN p. 177]:

```
type p  term -> form.
type q  term -> term -> form.
type f  term -> term.
type a  term.
```

Terms of type `term -> form` are "abstractions of a term over a formula" [MN p. 178], for example

```
x\ (p x) ==> (p (f x))
Z\ (p Z) ==> (p (f Z))
Z\ (p Z) && (p (f Z))
```

The first two are *the same* λ-term, since they differ only in the bound name. Note that the capital `Z` after `Z\` is a bound variable, not a logic variable. The quantifiers are then *second-order constants*: their argument has a functional type [MN p. 178].

```
type all, some  (term -> form) -> form.
```

```
all x\ (p x) ==> (p (f x))          % ∀x (p(x) ⊃ p(f(x)))
some Z\ (p Z) && (p (f Z))          % ∃Z (p(Z) ∧ p(f(Z)))
```

The book states the principle behind this as follows: "the mechanism that we have described divides the encoding of the object logic quantifiers into two parts: a second order constant that identifies the quantifier and an abstraction that captures its binding effect" [MN p. 178]. For instance $\forall x\, \forall y\, (q(x, y) \supset \exists z\, (p(z) \wedge q(z, y)))$ is encoded as

```
all x\ all y\ (q x y) ==> some z\ p z && q z y.
```

where `\` extends as far to the right as possible and `&&` (priority 5) binds tighter than `==>` (priority 3).

Definition 6.1 (The encoding of formulas)**.** For first-order terms and formulas over the signature above, define $\ulcorner\cdot\urcorner$ by

$$\ulcorner x\urcorner = x, \qquad \ulcorner f(t_1,\dots,t_n)\urcorner = f\,\ulcorner t_1\urcorner\cdots\ulcorner t_n\urcorner,$$
$$\ulcorner B \wedge C\urcorner = \ulcorner B\urcorner\ \&\&\ \ulcorner C\urcorner \quad (\text{similarly for } \vee, \supset, \neg, \top, \bot), \qquad \ulcorner \forall x.\, B\urcorner = \mathsf{all}\,(\lambda x.\ulcorner B\urcorner),$$
$$\ulcorner \exists x.\, B\urcorner = \mathsf{some}\,(\lambda x.\ulcorner B\urcorner).$$

The book does not state $\ulcorner\cdot\urcorner$ as a definition or prove anything about it; this formal version is ours, and so is the name. The chapter relies on two properties of it without stating them. First, it is a bijection between closed object formulas over the signature, taken up to renaming of bound variables, and closed terms of type `form` over the same signature, taken up to $\alpha\beta\eta$-conversion. A canonical representative of each class is the β-normal, η-long term, which is what $\ulcorner\cdot\urcorner$ produces. Second, it commutes with substitution: $\ulcorner B[t/x]\urcorner = \ulcorner B\urcorner[\ulcorner t\urcorner/x]$. The second property is what makes "substitution is β" correct.

6.2.2 Untyped λ-terms

The second running object language is the untyped λ-calculus [MN p. 178]–[MN p. 179]:

```
kind tm   type.
type app  tm -> tm -> tm.
type abs  (tm -> tm) -> tm.
```

untyped term	encoding
$\lambda x\, x$	`(abs x\x)`
$\lambda x\,(x\, x)$	`(abs x\ (app x x))`
$\lambda x\,\lambda y\, x$	`(abs x\ abs y\ x)`
$\lambda x\,\lambda y\, y$	`(abs x\ abs y\ y)`
$\lambda x\,\lambda y\,(y\, x)$	`(abs x\ abs y\ app y x)`
$\lambda x\,\lambda y\,\lambda z\,((x\, z)\,(y\, z))$	`(abs x\abs y\abs z\ (app (app x z) (app y z)))`
$(\lambda x\,(x\, x))\,(\lambda x\,(x\, x))$	`(app (abs x\ app x x) (abs x\ app x x))`

Each object application becomes one `app`; each object λ becomes one `abs` followed by one meta-level λ. Two remarks help in reading the rest of the chapter.

- No closed term of type `tm` can be built from `app` alone: every closed term starts with an `abs`, whose argument (of type `tm -> tm`) is something like `x\x` or `x\ app x x`.
- "the type tm can be seen as admitting new constructors as one descends under binders" [MN p. 179]. Below `abs x\ abs y\` both `x` and `y` behave as extra constants of type `tm`. Mobility (Section 6.4) makes this literally true: under a binder the program works with a genuinely new constant.

6.2.3 What the encoding gives for free

The abstractions over syntax live at arrow types (`term -> form`, `tm -> tm`), and the equality of those types includes the three conversions of Chapter 4.

α. The object language identifies formulas up to renaming of bound variables, and so does the meta-language. "Since *α*-conversion is included within the logical notion of equality, it is impossible to access the names of bound variables" [MN p. 180]. The book presents this as a strength: the programmer is spared what does not matter, and the implementer may drop names entirely (for instance with de Bruijn indices [3]). The cost is that printing and parsing must invent names.

β. This is the subject of Section 6.3.

η. Its effect is "rather mild" [MN p. 180]: for $f : \mathsf{i} \to \mathsf{i}$, $f =_\eta \lambda x.\, f\, x$. Hence $\forall x\,(p\,x)$ may be written `(all x\ p x)` *or* `(all p)`: "not only does the hohh language abstract away from names of bindings, but it also sometimes allows one to abstract away from the presence of actual binders" [MN p. 180].

Pitfall: An *η*-short quantifier is still a quantifier

A head pattern `(all B)` must also match the term `(all p)`, and `(B T)` must then reduce to `(p T)`. An implementation either unifies modulo *η* or keeps terms *η*-long. A representation that stores the binder "shape" syntactically and expects every quantifier to carry an explicit *λ* will reject `(all p)`.

6.3 Object-level substitution is meta-level *β*

6.3.1 Instantiating quantifiers

The formula $\forall x\,(p\,x \supset \exists y\,(q\,x\,y))$ is `all x\ p x ==> some y\ q x y`. To instantiate it with $f(a)$ we simply apply the abstraction to `(f a)` [MN p. 180]–[MN p. 181]:

```
(x\ p x ==> some y\ q x y) (f a)
```

which *β*-converts to `p (f a) ==> some y\ q (f a) y`. (The book prints `p (h a)` here, a typo; the page image confirms it, and it must be `p (f a)`.) Note what was *not* needed: no renaming of the inner `y`, no check that `(f a)` has no free `y`. Capture avoidance is the meta-level's job.

The predicate `list_instan` relates a list of $n \geq 0$ terms and a formula with at least n outer universal quantifiers to the result of instantiating the first n [MN p. 181]:

```
type list_instan  list term -> form -> form -> o.
list_instan nil B B.
list_instan (T::Ts) (all B) C :- list_instan Ts (B T) C.
```

The head `(all B)` is *η*-equal to `(all x\ B x)`: matching it binds `B` to the abstraction itself, and "the substitution for a bound variable is realized simply by writing down (B T)" [MN p. 181].

6.3.2 An interpreter for an object logic

A more substantial example is an interpreter for first-order Horn clauses, written in higher-order Horn clauses (Figure 7.1, [MN p. 181]). The object-level program is an explicit argument `D`, a single formula.

```
type interp     form -> form -> o.
type backchain  form -> form -> form -> o.
type atom       form -> o.

interp D tt.
interp D (G1 && G2)  :- interp D G1, interp D G2.
interp D (G1 !! G2)  :- interp D G1; interp D G2.
interp D (some   G)  :- interp D (G X).
interp D A           :- atom A, backchain D D A.
backchain D A A.
backchain D (D1 && D2) A   :- backchain D D1 A; backchain D D2 A.
backchain D (all D1) A     :- backchain D (D1 X) A.
backchain D (G ==> D1) A   :- backchain D D1 A, interp D G.
```

The two higher-order clauses are the interesting ones. An object goal `(some G)` is solved by solving `(G X)` for a *fresh logic variable* `X`: the meta-level β plugs the logic variable in for the object-bound variable, and later unification will find a witness. An object clause `(all D1)` is used by instantiating it with a fresh logic variable as well. The sample object program of Figure 7.2 ([MN p. 182]) is

```
type a, b, c    term.
type adj, path  term -> term -> form.
type prog       form -> o.
atom (adj _ _) & atom (path _ _).
prog ((adj a b) && (adj b c) &&
      (all x\ all y\ (adj x y) ==> (path x y)) &&
      (all x\ all y\ all z\ (adj x y) && (path y z) ==> (path x z))).
```

Two notational points recur in the rest of the chapter. An `&` between two clause heads, as in `atom (adj _ _) & atom (path _ _).`, is clause-level conjunction. The heads share one body (here an empty one), so the line stands for two clauses written together. The underscore `_` is an anonymous logic variable. The predicate `atom` is not part of the interpreter. It is defined for each object signature by listing its predicate constants, and here it says that `adj` and `path` atoms are atomic. The query `?- prog P, interp P (path a X).` has two answers, `X = b` and then `X = c` [MN p. 182].

Tutorial: How `interp` finds `X = b`

The goal `interp P (path a X)` matches the last `interp` clause, because `atom (path a X)` holds. We must prove `backchain P P (path a X)`. The program `P` is a left-nested `&&` of four parts, so the second `backchain` clause descends into it. The two `adj` facts do not match `path a X`. The third part is `all x\ all y\ (adj x y) ==> (path x y)`: the `(all D1)` clause applies it to a fresh `X1`, giving `(D1 X1)` $=_\beta$ `all y\ (adj X1 y) ==> (path X1 y)`; once more

with `Y1`, giving `(adj X1 Y1) ==> (path X1 Y1)`. The `==>` clause then requires `backchain P (path X1 Y1) (path a X)` (the fact clause, binding `X1 = a`, `Y1 = X`) and then `interp P (adj a X)`, which backchains to the fact `adj a b` with `X = b`. On backtracking, the fourth part of the program yields `X = c` through `adj a b` and `path b c`. At no point did the interpreter rename a bound variable: each β-step `(D1 X1)` replaced an object-level bound variable by a logic variable.

6.3.3 Evaluators for the untyped λ-calculus

Call-by-name and call-by-value evaluation are four clauses (Figure 7.3, [MN p. 182]):

```
type cbn, cbv  tm -> tm -> o.
cbn (abs R) (abs R).
cbn (app M N) V :- cbn M (abs R), cbn (R N) V.
cbv (abs R) (abs R).
cbv (app M N) V :- cbv M (abs R), cbv N U, cbv (R U) V.
```

`(R N)` is the object-level β-contraction, done by the meta-level β. (The text on [MN p. 182] says "the term U such that eval N U is provable"; `eval` is a loose name for `cbv`.) The behaviour is the expected one [MN p. 183]. $\Omega = (\lambda x\,(x\,x))(\lambda x\,(x\,x))$ has no value under either evaluator: the search for a derivation is infinite. On $(\lambda x\,\lambda w\,w)\,\Omega$, call-by-name terminates and call-by-value loops:

```
?- cbn (app (abs x\ abs w\w) (app (abs x\ app x x) (abs x\ app x x))) V.
   % V = (abs w\w)
?- cbv (app (abs x\ abs w\w) (app (abs x\ app x x) (abs x\ app x x))) V.
   % infinite search
```

The book draws the moral of this section in one paragraph [MN p. 183]. The specifications are succinct and declarative because "all the details regarding object-level bound variables and object-level substitution has been relegated to the meta-level". They are still formal and precise: the details "have not gone away; they simply have become the burden of an implementation of a metalogic such as λProlog", and the programmer "is relieved of dealing with the details behind such 'concrete nonsense' as binder names, capture avoiding substitutions, etc."

Kanren angle: What works today: abstractions as Scheme closures

All the programs of this section use an abstraction only as an *input*. The abstraction is already known when `(R N)` is formed. A miniKanren can imitate this with *functional* higher-order abstract syntax: represent `abs R` by a list holding a Scheme procedure, and perform `(R N)` with `project°`. This runs on today's (aux kanren micro), unchanged:

```
(define (abs f) `(abs ,f))
(define (app m n) `(app ,m ,n))
(define ω (abs (λ (x) (app x x))))
(define Ω (app ω ω))
(define K* (abs (λ (x) (abs (λ (w) w)))))          ; λx λw w
```

```
; call-by-name evaluation, Figure 7.3: (R N) is Scheme application, available
↪  only through project°
(define-relation (cbn° m v)
  (cond°
    ((fresh° (r) (=° m `(abs ,r)) (=° v m)))
    ((fresh° (m1 n r) (=° m `(app ,m1 ,n)) (cbn° m1 `(abs ,r))
       (project° ((r* r)) (cbn° (r* n) v))))))

; printing a HOAS term needs invented names: apply each closure to a generated
↪  symbol
(define (hoas->sexp t)
  (let loop ((t t) (k 1))
    (cond
      ((and (pair? t) (eq? (car t) 'abs))
       (let ((w (string->symbol (string-append "w" (number->string k)))))
         `(λ (,w) ,(loop ((cadr t) w) (+ k 1)))))
      ((and (pair? t) (eq? (car t) 'app)) `(,(loop (cadr t) k) ,(loop (caddr t)
       ↪  k)))
      (else t))))

(print (μkanren-run (q -1 #t) (fresh° (v) (cbn° (app K* Ω) v) (project° ((v*
↪  v)) (=° q (hoas->sexp v*))))))
; α-equivalent closures are different Scheme objects: unification fails
(print (μkanren-run (q -1 #t) (=° (abs (λ (x) x)) (abs (λ (y) y)))))
```

```
((λ (w1) w1))
()
```

The first answer is the book's `V = (abs w\w)`. The second answer is the problem. Two α-equivalent abstractions are distinct procedures, and (aux kanren micro) compares procedures with `eq?`, so they do not unify. A closure cannot be printed without being applied to invented names (that is what `hoas->sexp` does). A closure cannot be inspected except by applying it, and no unification can *produce* one. Recognisers such as `term`, `bnorm` and `typeof` below could still be run on closures. They only apply an input abstraction to a fresh constant, although they also need `pi` and `=>`. But every program that *builds* an abstraction as output (`red1`, path synthesis, `trans` backwards, `nnf`, `prenex`, `copy`, `subst` backwards), and every program that asks whether a body depends on its bound variable (`vacuous`, the η-redex), is out of reach. Section 6.8 comes back to this "functional" reading of higher-order abstract syntax.

6.4 Mobility of binders

6.4.1 Why an abstraction cannot be taken apart

Equal expressions should have equal parts: equal non-empty lists have equal heads and equal tails. Applied to λ-terms, this principle says that "we should not be able to decompose an abstraction into the variable that it binds and its body" [MN p. 183]. The terms `(all x\ some y\ p x ==> p y)` and `(all z\ some y\ p z ==> p y)` are equal, yet the names `x` and `z` differ, and so do the would-be bodies `(some y\ p x ==> p y)` and `(some y\ p z ==> p y)`. "Such decompositions, in fact, have no logical status within the hohh language: Because equality includes α-conversion, the name for a bound variable is a fiction, and an implementation may not represent it at all" [MN p. 183].

So how does a program recurse into the body of a binder? The book's answer [MN p. 184]: "during proof search, a term-level binding (i.e., a λ-abstraction) can be converted into a formula-level binding (i.e., a quantifier), which then can be converted into a proof-level binding (i.e., an eigenvariable)."

$$\underbrace{\lambda y.\, t}_{\text{term level}} \;\rightsquigarrow\; \underbrace{\forall x.\, (\cdots \supset P\,((\lambda y.\, t)\, x))}_{\text{formula level}} \;=_\beta\; \forall x.\, (\cdots \supset P\,(t[x/y])) \;\rightsquigarrow\; \underbrace{P\,(t[c/y]) \;\text{ with } c \text{ fresh}}_{\text{proof level}}$$

6.4.2 The proof-search rules behind mobility

Recall from Chapter 3 the rules for the two connectives that make mobility work, together with the backchaining rule. A goal is proved relative to a signature Σ (constants and eigenvariables in scope) and a program $\mathscr{P}$.

$$\frac{\Sigma \cup \{c\};\, \mathscr{P} \longrightarrow G[c/x] \qquad c \notin \Sigma}{\Sigma;\, \mathscr{P} \longrightarrow \forall x.\, G}\ \textsc{generic} \qquad\qquad \frac{\Sigma;\, \mathscr{P} \cup \{D\} \longrightarrow G}{\Sigma;\, \mathscr{P} \longrightarrow D \supset G}\ \textsc{augment}$$

$$\frac{\Sigma;\, \mathscr{P} \longrightarrow G[t/x] \qquad t \text{ a } \Sigma\text{-term}}{\Sigma;\, \mathscr{P} \longrightarrow \exists x.\, G}\ \textsc{instance}$$

For an atomic goal A, BACKCHAIN picks a clause $\forall \bar{x}\, (G \supset A')$ of $\mathscr{P}$ and an instance with $A'\theta =_{\beta\eta} A$, then proves $G\theta$. In an implementation the term t of INSTANCE and the substitution θ are found by unification with logic variables. Then the side condition $c \notin \Sigma$ becomes the **scoping constraint**: a logic variable created *before* the eigenvariable c must never be instantiated with a term containing c, because at the time it was created its witness had to be a Σ-term. This is why `sigma X\ pi y\ X = y` fails while `pi y\ sigma X\ X = y` succeeds. Chapter 7 never mentions the constraint, but several of its programs (`vacuous`, the η-redex clause, `prenex`) are correct only because of it.

6.4.3 A first example: recognising terms

Because of typing, the predicate `term : tm -> o` could be defined by the single clause `term T.`. A more instructive definition reflects the two constructors of `tm` into two clauses [MN p. 184]. Written so that their logical content is visible:

```
pi M\ term M => pi N\ term N => term (app M N).
pi R\ (pi x\ term x => term (R x)) => term (abs R).
```

or, equivalently, in the usual clause syntax:

```
term (app M N) :- term M, term N.
term (abs R)   :- pi x\ term x => term (R x).
```

Tutorial: Following the binder of `(abs y\ app y y)`

The book traces the query `?- (term (abs y\ app y y)).` [MN p. 184]–[MN p. 185].

```
?- (term (abs y\ app y y)).
?- pi x\ term x => term ((y\ app y y) x).     % backchain on abs clause
?- pi x\ term x => term (app x x).            % β-conversion: term-level y ->
 ↪  formula-level x
?- term (app c c).                           % eigenvariable c, program +
 ↪  (term c)
```

1. *Backchaining* on the `abs` clause unifies `R` with the abstraction `y\ app y y`. Nothing has been decomposed: `R` holds the whole abstraction.
2. The body of the clause applies `R` to the `pi`-bound `x`. The redex `((y\ app y y) x)` is a β_0-redex (Section 6.9): a λ applied to a bound variable. Contracting it moves the term-level binder `y` to the formula-level binder `x`.
3. GENERIC replaces `x` by a new eigenvariable `c`. AUGMENT adds the clause `term c` to the program. The formula-level binder has become a proof-level binder.
4. `term (app c c)` backchains on the `app` clause, and both subgoals `term c` are closed by the assumed clause.

Here `c` is exactly one of the "new constructors" of `tm` that appear under a binder. The program has no clause for variables, and it needs none: each variable gets its clause at the moment its binder is crossed, and loses it when the proof of the body is complete.

The book distils the example into a programming idiom. We record it as a definition because every program of the chapter is an instance of it.

Definition 6.2 (The binder-mobility idiom, [MN p. 185])**.** "In order to continue a recursive analysis within the scope of a binder, first, apply that binder to a universal quantified goal variable, for which the metalogic will substitute an eigenvariable, and second, using an implicational goal, assume new clauses that extend the definition of various predicates so that they can deal with the presence of this new eigenvariable."

In code, the idiom is always the shape `p (c R) ... :- pi x\ (hyp x => p (R x) ...).` The abstraction `R` is never opened. It is applied to a fresh eigenvariable, and the assumed `hyp x` tells the predicates of the program what to do when they meet it.

Pitfall: A fresh constant is not an eigenvariable

Proving `pi x\ G` with a fresh constant is only half of GENERIC. The other half is the scoping constraint. Here is what happens without it, in today's (aux kanren micro), if a universal goal is "implemented" by handing a `gensym` to the body:

```
; a NAIVE universal goal: a fresh Scheme constant, with no eigenvariable scope
↪  check
(define (naive-pi f) (f (gensym 'c)))

; λProlog: ?- sigma X\ pi y\ X = y.   must fail; the naive encoding succeeds
(print (μkanren-run (q -1 #t) (naive-pi (λ (y) (=° q y)))))
; λProlog: ?- pi y\ sigma X\ X = y.   must succeed (X is created inside the
↪  scope of y)
(print (μkanren-run (q -1 #t) (naive-pi (λ (y) (fresh° (x) (=° x y) (=° q
↪  'ok))))))
```

```
(c102)
(ok)
```

The first query should fail. The query variable `q` was created before the constant, yet it now holds that constant, which has escaped its scope. The same mistake in a unifier, letting a clause variable be bound to a term that mentions a variable bound inside its scope, makes `vacuous` (Section 6.6) accept formulas whose quantifier is not vacuous. The constraint is not a matter of efficiency: it is part of the meaning of ∀.

Kanren angle: What mobility requires of a miniKanren

Mobility needs three things that (aux kanren micro) lacks (see Chapter 10 for the engine).

1. *Eigenvariables*: constants created during the search, distinct from every other term, with a scope. The engine must enforce the scoping constraint: a logic variable older than an eigenvariable can never be bound to a term containing it, *even indirectly* through younger variables (binding $X := f(Z)$ and later $Z := c$ leaks c into X). λProlog implementations use universe levels [43]. The miniKanren with symbolic constraints uses an absento-like "eigen" constraint over the variables in scope [21].
2. *Scoped program extension*: `D => G` adds `D` only while `G` is being proved. Other branches of the search and the answers of the enclosing goal must not see `D`. Because miniKanren's search interleaves branches, the extension must live in the search *state*, not in a host-language dynamic variable. A relation must be able to consult the assumed clauses as well as its static ones.
3. *Application of an abstraction to an eigenvariable*, `(R x)`, on both sides of the equation: as input (when `R` is known) and as output (when `R` is a logic variable to be solved, as in most programs below).

How the proposal provides each of these is the subject of Chapter 11.

6.5 Computing with untyped λ-terms

This section applies the idiom to the untyped λ-calculus. The programs grow from pure recognisers to programs that build abstractions as output, to programs that assume rules with bodies.

6.5.1 β-normal forms

Definition 6.3 (β-normal, β-body-normal, [MN p. 185])**.** An untyped λ-term is in *β-normal form* if it contains no β-redex. Equivalently and positively, it has the shape $\lambda x_1 \ldots \lambda x_n\,(v\,t_1 \ldots t_m)$ with $n, m \geq 0$, where v is a variable and every t_i is β-normal. The inner part $v\,t_1 \ldots t_m$ is called *β-body-normal*.

The definition becomes three inference rules and three clauses [MN p. 186]:

$$\frac{\forall x.\ \mathrm{bbnorm}(x) \supset \mathrm{bnorm}(M\,x)}{\mathrm{bnorm}(\mathrm{abs}\,M)} \qquad \frac{\mathrm{bbnorm}(H)}{\mathrm{bnorm}(H)} \qquad \frac{\mathrm{bbnorm}(M) \qquad \mathrm{bnorm}(N)}{\mathrm{bbnorm}(\mathrm{app}\,M\,N)}$$

```
type bnorm, bbnorm  tm -> o.

bnorm (abs M)    :- pi x\ bbnorm x => bnorm (M x).
bnorm H          :- bbnorm H.
bbnorm (app M N) :- bbnorm M, bnorm N.
```

There is no static clause saying that a variable is β-body-normal, and there could not be one, since bound variables are not terms of the program. A variable becomes body-normal only through the assumption `bbnorm x` made when its binder is crossed. A redex `app (abs R) N` is rejected because `bbnorm (abs R)` has no clause.

Tutorial: A normal term and a redex

This walk-through is ours; the book gives no query for `bnorm`. Take first `?- bnorm (abs x\ app x x).`

1. The `abs` clause matches with `M = x\ app x x`. The body is `pi x\ bbnorm x => bnorm (M x)`: an eigenvariable `c` is created, `bbnorm c` is assumed, and the goal is `bnorm (app c c)` (after β_0).
2. `bnorm (app c c)` is not an `abs`, so the second clause reduces it to `bbnorm (app c c)`, and the third clause to `bbnorm c` and `bnorm c`.
3. `bbnorm c` is the assumption. `bnorm c` goes through the second clause to `bbnorm c` again, also the assumption. The query succeeds.

Now take `?- bnorm (app (abs x\x) (abs x\x)).` The `abs` clause does not match. The second clause asks for `bbnorm (app (abs x\x) (abs x\x))`, and the third clause for `bbnorm (abs x\x)`. No static clause has a head `bbnorm (abs _)`, and the only assumptions ever made are `bbnorm c` on eigenvariables, so the query fails. The structure of the clauses alone, "a variable is body-normal only when its binder said so", rules the redex out.

A pleasant consequence is a substitution lemma that costs nothing.

Theorem 6.4 (Substitution lemma "for free", [MN p. 186]). *Let* $\Gamma = \{\text{bbnorm}\, c_1, \ldots, \text{bbnorm}\, c_n\}$, $n \geq 0$, *with the* c_i *eigenvariables. If* `(bnorm (abs R))` *and* `(bbnorm S)` *are both derivable from the program above together with* Γ, *then* `(bnorm (R S))` *is derivable too. In object-level terms: if* $\lambda x\, t$ *is* β*-normal and* s *is* β*-body-normal, then* $t[s/x]$ *is* β*-normal.*

Proof (book's argument, with the inversion step made explicit). A derivation of `bnorm (abs R)` must end with the `abs` clause, so `pi x\ bbnorm x => bnorm (R x)` is derivable. Derivability is intuitionistic, so the universal quantifier can be instantiated with `S`, giving `bbnorm S => bnorm (R S)`, and modus ponens with `bbnorm S` gives `bnorm (R S)`. □

The book says only "since derivability is based on intuitionistic logic". Two facts sit behind that phrase: cut admissibility, and the fact that an eigenvariable may be instantiated by any term. The inversion step ("must end with the `abs` clause") also uses the shape of Γ. Γ contains only atoms `bbnorm c` on eigenvariables, so no assumption can match `bnorm (abs R)`. In a named representation this lemma is a page of index bookkeeping. Here the logic proves it.

6.5.2 Reduction to normal form

Figure 7.4 ([MN p. 187]) specifies one-step reduction and reduction to normal form:

```
type redex, red1, reduce  tm -> tm -> o.
redex (app (abs R) N) (R N).
red1 M N :- redex M N.
red1 (app M N) (app P N) & red1 (app N M) (app N P) :- red1 M P.
red1 (abs M) (abs N) :- pi x\ red1 (M x) (N x).
reduce M M :- bnorm M.
reduce M N :- red1 M P, reduce P N.

type repeat   (A -> A -> o) -> A -> A -> o.
type reduce'  tm -> tm -> o.
repeat Pred M N :- Pred M P, !, repeat Pred P N.
repeat Pred M M.
reduce' M N :- repeat red1 M N.
```

- `redex` relates a β-redex to its contractum, again by meta-level β: `(R N)`.
- `red1` contracts exactly one redex somewhere in the term. The `abs` clause is new. Its *output* is the abstraction `N`, and the goal `red1 (M x) (N x)` asks unification to solve `(N x) = t` for an unknown `N`: the answer is `N := x\ t`, with the eigenvariable abstracted out of `t`. This is a *higher-order pattern* problem (Section 6.9 and Chapter 7). There is no `=>` here: `red1` should simply fail on a bare variable, so no clause about `x` is needed.
- `reduce` uses `bnorm` to decide when to stop. `reduce'` uses the polymorphic higher-order `repeat`, which iterates its argument until it fails. The cut `!` "prune[s] away alternative solutions to the parent goal" [MN p. 186]. (The text there says `repeat'`; the figure's predicate is `repeat`.)

Tutorial: One step under a binder: `red1 (abs x\ app (abs y\y) x) N`

This is our own example of an output abstraction, traced against the clauses of Figure 7.4 without the η-clause below. The term is $\lambda x\,((\lambda y\,y)\,x)$, with a redex under the binder.

1. `red1 M N :- redex M N` fails: the term is an `abs`, not an `app`. The two `app` clauses do not match either.
2. The `abs` clause matches with `M = x\ app (abs y\y) x` and `N = abs N1` for a new logic variable `N1 : tm -> tm`. It creates an eigenvariable `c`. The goal is `red1 (app (abs y\y) c) (N1 c)`.
3. The `redex` clause applies with `R = y\y` and contractum `(R c)`, which is `c`. Now unification must solve `(N1 c) = c`. This is a pattern equation: `N1` is applied to the eigenvariable `c` only. Its unique solution abstracts `c` out of the right-hand side: `N1 := x\ x`.
4. The answer is `N = abs x\x`. On backtracking, the `app` clauses try to reduce `(abs y\y)` or `c` in one step. The first descends under `y\` to the goal `red1 d _` on a new eigenvariable `d`, the second gives `red1 c _`. Both fail, because no clause, static or assumed, applies to a bare eigenvariable. There is no other answer.

The program contains no code for "rebuild the binder around the reduced body". Unification does it in step 3, and the scoping constraint guarantees that `N1`, created before `c`, receives `c` only through its argument.

The η-redex needs one clause [MN p. 187]:

```
redex (abs x\ app M x) M.
```

It is correct because of the scoping constraint. `M` is a clause variable, quantified *outside* the binder `x`, so no unifier can let `M` depend on `x`. This is exactly the side condition of η: x not free in M. For instance, `abs x\ app x x` is not an η-redex: matching it against `abs x\ app M x` would need `M = x`, and the scoping constraint forbids that binding. With this clause added, the term of the tutorial above is also an η-redex. Matching `abs x\ app (abs y\y) x` against `abs x\ app M x` gives `M = abs y\y`, which does not mention `x`, so `red1` now finds `abs y\y` first, through `redex`, and then the α-equal `abs x\x` again through the `abs` clause.

The book adds a semantic remark [MN p. 186]–[MN p. 187]. A model of the untyped λ-calculus is a domain D with maps $f : D \to [D \to D]$ and $g : [D \to D] \to D$; `tm` plays D, `app` plays f, `abs` plays g. The β-clause states $(f \circ g)\,R = R$ and the η-clause states $(g \circ f)\,M = M$: "these are the familiar properties stating that f and g are, in fact, retracts".

6.5.3 Reduction based on paths

A *path* walks from the root of a term to an occurrence of a variable. It passes *through* an abstraction and chooses left or right at an application [MN p. 187]. Paths contain binders too:

```
kind path type.
type bnd          (path -> path) -> path.
type left, right  path -> path.
type path         tm -> path -> o.
```

```
path (app M _) (left P) &
path (app _ M) (right P) :- path M P.
path (abs R) (bnd S) :- pi x\ pi p\ path x p => path (R x) (S p).
```

The `abs` clause introduces *two* eigenvariables at once: a term variable `x` and a path variable `p`. The assumption `path x p` links them: the path to the occurrence of `x` *is* `p`. The paths of `(abs x\ app x (abs y\ app y x))` are [MN p. 188]

```
bnd u\ left u
bnd u\ right (bnd v\ left v)
bnd u\ right (bnd v\ right u)
```

The relation also runs backwards: from a set of paths it synthesises the term, using `foreach` from Chapter 5 (`foreach P nil. foreach P (X::L) :- P X, foreach P L.`) [MN p. 188]:

```
?- foreach (path N)
     ((bnd u\ left u) ::
      (bnd u\ right (bnd v\ left v))::
      (bnd u\ right (bnd v\ right u))::nil).
N = abs W1\ app W1 (abs W2\ app W2 W1);
no
```

Tutorial: Synthesising a term from its paths

Take the first path. `path N (bnd u\ left u)` can only use the `abs` clause, so `N = abs R` and `S = u\ left u`. Under eigenvariables `x`, `p` with `path x p` assumed, the goal is `path (R x) (left p)`. The `left` clause requires `(R x) = app M _`. Unification solves this pattern problem with `R := x\ app (M' x) (K x)`, two new logic variables applied to `x`. Then `path (M' x) p` is solved by the assumption `path x p` itself, giving `M' := x\ x`. The second and third paths fill in the right branch in the same way. Nothing in the program says "build an abstraction". The abstraction in the answer is produced by unification, which abstracts the eigenvariable `x` out of a term that contains it. This is the operation that Scheme closures could not perform.

Paths can also cross a *marked* β-redex. A path through `(app (abs R) N)` goes through the body `R`; when it reaches the bound variable of `R`, it continues into `N`. Marked redexes are written `(beta N R)` (Figure 7.5, [MN p. 189]):

```
type beta     tm -> (tm -> tm) -> tm.
type addbeta  tm -> tm -> o.
type bpath    tm -> path -> o.

bpath (app M _) (left P)  &
bpath (app _ M) (right P) :- bpath M P.
bpath (abs R)   (bnd S)   :- pi x\ pi p\ bpath x p =>
                                         bpath (R x) (S p).
bpath (beta N R) P :-
   pi x\ (pi Q\ bpath x Q :- bpath N Q) => bpath (R x) P.
```

```
addbeta (app (abs R) N) (beta M S) :- addbeta (abs R) (abs S),
                                      addbeta N M.
addbeta (app (app M N) P) (app O Q) :- addbeta (app M N) O,
                                       addbeta P Q.
addbeta (abs R) (abs S) :-
  pi x\ (pi M\ pi N\ addbeta (app x M) (app x N) :- addbeta M N) =>
        (addbeta x x) => addbeta (R x) (S x).
```

Two features are new.

- The `beta` clause assumes a *rule*, not a fact: `pi Q\ bpath x Q :- bpath N Q`, "the paths of `x` are the paths of `N`". The assumed clause has a body and its own universally quantified variable `Q`, renamed at each use. It also mentions `N`, a variable of the enclosing clause, which it shares rather than renames.
- `addbeta` marks every redex. It dispatches on the head of an application: an `abs` head is marked, an `app` head recurses, and a bound-variable head `x` is handled by the rule assumed when `x`'s binder was crossed. A bare variable is copied by the assumed fact `addbeta x x`.

Normalisation then has two stages: collect the `bpath`s, then synthesise with `path` [MN p. 189]:

```
?- sigma B\ addbeta (app (abs x\x) (abs x\x)) B, bpath B Path.
Path = bnd W1\ W1;
no
?- foreach (P\ path T P) (bnd (W1\ W1) :: nil).
T = abs W1\ W1;
no
```

For $K\,(S\,K)$ with $K = \lambda x \lambda y\, x$ and $S = \lambda x \lambda y \lambda z\, ((x\,z)\,(y\,z))$ [MN p. 189]–[MN p. 190], the query

```
?- sigma K\ sigma S\ sigma B\
     K = (abs x\ abs y\ x),
     S = (abs x\ abs y\ abs z\ app (app x z) (app y z)),
     addbeta (app K (app S K)) B, bpath B Path.
```

yields four paths, and synthesis gives

```
?- foreach (path T)
     ((bnd W1\ bnd W2\ bnd W3\ left (left (bnd W4\ bnd W5\ W4))) ::
      (bnd W1\ bnd W2\ bnd W3\ left (right W3)) ::
      (bnd W1\ bnd W2\ bnd W3\ right (left W2)) ::
      (bnd W1\ bnd W2\ bnd W3\ right (right W3)) ::nil).
T = abs W1\ abs W2\ abs W3\
      app (app (abs W4\ abs W5\ W4) W3) (app W2 W3);
no
```

The result $\lambda w_1 \lambda w_2 \lambda w_3\, (((\lambda w_4 \lambda w_5\, w_4)\, w_3)\, (w_2\, w_3))$ is *not* β-normal. The redex $(K\, w_3)$ was *created* by the reduction, so it was not marked. Iterating "mark, collect, synthesise" reaches the normal form if it terminates, and in the untyped calculus it need not terminate [MN p. 190].

Pitfall: Collecting all answers is outside the logic

The two-stage method collects all solutions of `bpath B Path` into a list. The book stresses that this "is not formally possible in the simple proof-theoretic framework that we have described here" [MN p. 190]. Some Prolog systems provide `bagof`, and persistent storage can simulate it. A miniKanren has the same situation. A goal that consumes a whole sub-search, such as the aggregation goals of (aux kanren micro) (Chapter 10), is available, but it is extra-logical. It also terminates only when the sub-search is finite.

6.5.4 Type inference

The book's type inference for the untyped calculus ([MN p. 190]–[MN p. 191]) is the best-known instance of the idiom:

```
kind ty type.
type arr     ty -> ty -> ty.
type typeof  tm -> ty -> o.
typeof (app M N) A :- typeof M (arr B A), typeof N B.
typeof (abs M) (arr A B) :- pi x\ typeof x A => typeof (M x) B.
```

Read as inference rules, the typing context Γ is the set of assumed atoms `typeof c A`:

$$\frac{\Gamma \vdash M : B \to A \qquad \Gamma \vdash N : B}{\Gamma \vdash \mathrm{app}\, M\, N : A} \qquad\qquad \frac{\Gamma,\ c : A \vdash M\, c : B \qquad c \text{ fresh}}{\Gamma \vdash \mathrm{abs}\, M : A \to B}$$

The context is not a data structure of the program. It is the set of clauses that the `=>` goals have added, and the variable rule of the type system is BACKCHAIN on one of them. The interaction is [MN p. 191]:

```
?- typeof (abs x\ abs y\ abs z\ app (app x z) (app y z)) Ty.
Ty = arr (arr T1 (arr T2 T3)) (arr (arr T1 T2) (arr T1 T3))
?- typeof (abs x\x) Ty.
Ty = arr T1 T1
?- typeof (abs x\ app x x) Ty.
no
```

Tutorial: Why `typeof (abs x\x) Ty` answers `arr T1 T1`

1. Backchaining on the `abs` clause binds `Ty = arr A B` for fresh logic variables `A`, `B`, and `M = x\x`.
2. GENERIC creates an eigenvariable `c`. AUGMENT adds the clause `typeof c A`. This assumed clause contains the *logic variable* `A`, which is shared with the rest of the derivation. Assumptions are not closed facts.
3. The goal is now `typeof ((x\x) c) B`, that is, `typeof c B`. Neither static clause applies, since `c` is neither an `app` nor an `abs`. The assumed clause applies and unifies `B` with `A`.

4. So `Ty = arr A A`, printed `arr T1 T1`.

For `abs x\ app x x` the same steps give the assumption `typeof c A`, and `typeof (app c c) B` needs `typeof c (arr B0 B)` and `typeof c B0`. With the assumption these are `A = arr B0 B` and `A = B0`, hence `B0 = arr B0 B`. The occurs check rejects this, and the answer is `no`. Without an occurs check the answer would be a cyclic type.

Two more details matter. The type `ty` has only the constructor `arr`, so it has *no closed inhabitants*. Inferred types are therefore necessarily open, which links back to the distinction between unifiers and solutions of Chapter 4. With an added constant `type i ty.` one can ask closed questions [MN p. 191]:

```
?- typeof (abs x\x) (arr i i).
yes
?- typeof (abs x\x) (arr i Ty).
Ty = i
```

Theorem 6.5 (Subject reduction for call-by-name, [MN p. 191]–[MN p. 192])**.** *If `(cbn P V)` and `(typeof P A)` are derivable, then `(typeof V A)` is derivable.*

Proof (book's argument). By induction on the derivation of `cbn P V`. If it uses the first `cbn` clause, $P = V$. Otherwise `P = (app M N)`, and there are shorter derivations of `cbn M (abs R)` and `cbn (R N) V`. From `typeof (app M N) A` there is a `B` with `typeof M (arr B A)` and `typeof N B`. By the induction hypothesis, `typeof (abs R) (arr B A)`. Hence `pi x\ typeof x B => typeof (R x) A` is derivable. Instantiating `x` with `N` and using `typeof N B` gives `typeof (R N) A`. The induction hypothesis on `cbn (R N) V` gives `typeof V A`. □

The book adds that the same holds for `cbv` (left to the reader), and comments: "The hard technical part in such a proof is establishing a substitution lemma ... Here, that substitution lemma is obtained essentially for free by referring to well-known properties of intuitionistic logic" [MN p. 192].

Kanren angle: The usual miniKanren rendering: an explicit context

The standard miniKanren way to write a type inferencer avoids both `pi` and `=>`. It uses de Bruijn indices, so that bound names do not exist, and it passes the context as an explicit list. This runs on today's (aux kanren micro) (indices are Peano numerals and start from `z` for the innermost binder; the book's de Bruijn indices in Section 6.5.5 start from 1):

```
(define-relation (nth° Γ k a)
  (fresh° (b Γ* k*)
    (=° Γ `(,b . ,Γ*))
    (cond° ((=° k 'z) (=° a b))
           ((=° k `(s ,k*)) (nth° Γ* k* a)))))

(define-relation (typeof° Γ m a)
  (cond°
    ((fresh° (k) (=° m `(var ,k)) (nth° Γ k a)))
```

```
    ((fresh° (m1 n b) (=° m `(app ,m1 ,n)) (typeof° Γ m1 `(arr ,b ,a))
    ↪  (typeof° Γ n b)))
    ((fresh° (body a1 b) (=° m `(lam ,body)) (=° a `(arr ,a1 ,b)) (typeof°
    ↪  `(,a1 . ,Γ) body b)))))

(define S '(lam (lam (lam (app (app (var (s (s z))) (var z)) (app (var (s z))
↪  (var z)))))))
(define I '(lam (var z)))
(define ω '(lam (app (var z) (var z))))

(print (μkanren-run (q -1 #t) (typeof° '() S q)))
(print (μkanren-run (q -1 #t) (typeof° '() I q)))
(print (μkanren-run (q -1 #t) (typeof° '() ω q)))
```

```
((arr (arr α (arr β γ)) (arr (arr α β) (arr α γ))))
((arr α α))
()
```

These are the book's three answers. The occurs check of (aux kanren micro) rejects $\lambda x\,(x\,x)$ exactly as in the tutorial above. The difference between the two programs is where the context lives. In `typeof°` it is data, threaded through every call and looked up with a hand-written `nth°`. In `typeof` it is part of the program, and the logic guarantees the substitution lemma behind Theorem 6.5. With an explicit context, every relation that may meet a bound variable must accept and pass on the context. With `=>`, a predicate written without any knowledge of binders, such as `atom` in Section 6.6, keeps working under them. Supporting the second style is the point of the proposal (Chapter 11).

6.5.5 Translating to and from de Bruijn syntax

In de Bruijn's notation [3] a variable is a positive integer n that refers to the n-th enclosing abstraction, counting from 1 [MN p. 192]:

term	de Bruijn	term	de Bruijn
$\lambda x\,x$	$\lambda 1$	$\lambda x \lambda y\,(y\,x)$	$\lambda\lambda(1\,2)$
$\lambda x\,(x\,x)$	$\lambda(1\,1)$	$\lambda x \lambda y \lambda z\,((x\,z)(y\,z))$	$\lambda\lambda\lambda((3\,1)(2\,1))$
$\lambda x \lambda y\,x$	$\lambda\lambda 2$	$\lambda x\,(x\,(\lambda y\,x))$	$\lambda(1\,\lambda 2)$
$\lambda x \lambda y\,y$	$\lambda\lambda 1$	$\lambda x\,(x\,(\lambda y\,(x\,(\lambda w\,(w\,x)))))$	$\lambda(1\,(\lambda(2\,(\lambda(1\,3)))))$

The same variable receives different numbers at different depths (the x of the last row is 1, 2 and 3). The encoding [MN p. 193], in which `deb` is both a kind and a constructor, and the translation are:

```
kind deb  type.
type ab   deb -> deb.
type ap   deb -> deb -> deb.
type deb  int -> deb.
```

```
type trans  int -> tm -> deb -> o.
type depth  int -> tm -> o.
trans D (abs M) (ab P) :- pi c\ depth D c =>
                            (E is D + 1, trans E (M c) P).
trans D (app M N) (ap P Q) :- trans D M P, trans D N Q.
trans D X (deb E) :- depth N X, E is (D - N).
```

In `trans D t d`, `D` is one more than the number of enclosing binders. Each eigenvariable is tagged, by the assumption `depth D c`, with the level at which it was bound. At an occurrence, the index is the current level minus the binding level. The relation works in both directions [MN p. 193]:

```
?- trans 1 (abs x\ app x (abs y\ app x (abs w\ app w x))) D.
D = ab (ap (deb 1) (ab (ap (deb 2) (ab (ap (deb 1) (deb 3))))))
?- trans 1 P
     (ab (ap (deb 1) (ab (ap (deb 2) (ab (ap (deb 1) (deb 3)))))))).
P = abs W1\ app W1 (abs W2\ app W1 (abs W3\ app W3 W1))
```

Tutorial: Tracing `?- trans 1 (abs x\ abs y\ abs z\ y) P.`

The book traces this query [MN p. 193]–[MN p. 194].

1. `P := (ab P1)`. A new eigenvariable `c` is introduced and `depth 1 c` is assumed. Goal: `trans 2 (abs y\ abs z\ y) P1`. (The meta-level β has replaced the bound `x` by `c`; since `x` does not occur in the body, nothing visible changes.)
2. `P1 := (ab P2)`, eigenvariable `d`, assume `depth 2 d`. Goal: `trans 3 (abs z\ d) P2`.
3. `P2 := (ab P3)`, eigenvariable `e`, assume `depth 3 e`. Goal: `trans 4 d P3`.
4. Only the third clause applies: `P3 := (deb E)` and `depth N d, E is (4 - N)`. The assumption gives `N = 2`, so `E = 2`.
5. Answer: `P = (ab (ab (ab (deb 2))))`.

The book's text writes the assumptions as "depth c 1", "depth d 2", "depth e 3", with the arguments swapped with respect to `type depth int -> tm -> o`. The program assumes `depth 1 c`, and so on. In the reverse direction the same clauses run with `P` unknown. The third clause then finds the eigenvariable `X` whose `depth` gives the right index, and the `(M c)` of the first clause is solved as a pattern problem, as in the path synthesis of Section 6.5.3.

A variant ([MN p. 194]) with no `depth` predicate assumes new `trans` clauses for each eigenvariable. It has *no* static clause for variables:

```
trans D (app M N) (ap P Q) :- trans D M P, trans D N Q.
trans D (abs M) (ab P) :-
   pi u\
     (pi N\ pi H\ trans N u (deb H) :- H is (N - D))
       => (D' is D + 1, trans D' (M u) P).
```

For the query traced above, the assumed clauses are

```
pi N\ pi H\ trans N c (deb H) :- H is (N - 1).
pi N\ pi H\ trans N d (deb H) :- H is (N - 2).
pi N\ pi H\ trans N e (deb H) :- H is (N - 3).
```

and `?- trans 4 d P3.` is solved by the second. Each assumed clause has captured the value of the outer `D` at the time it was assumed. It behaves as a closure over the context.

Kanren angle: The same translation with an explicit environment

In (aux kanren micro) the translation is written with an explicit environment of names. It is the miniKanren analogue of the `depth` assumptions:

```
; named terms: a symbol, (app m n), (λ x b); de Bruijn terms as in typeof.scm
 ↪  (z = innermost binder)
(define-relation (index° env x k)
  (fresh° (y env*)
    (=° env `(,y . ,env*))
    (cond° ((=° x y) (=° k 'z))
           ((fresh° (k*) (≠° x y) (=° k `(s ,k*)) (index° env* x k*))))))

(define-relation (trans° env t d)
  (cond°
    ((symbol° t) (fresh° (k) (=° d `(var ,k)) (index° env t k)))
    ((fresh° (m n p r) (=° t `(app ,m ,n)) (=° d `(app ,p ,r)) (trans° env m p)
     ↪  (trans° env n r)))
    ((fresh° (x b p) (=° t `(λ ,x ,b)) (symbol° x) (=° d `(lam ,p)) (trans°
     ↪  `(,x . ,env) b p)))))

; λx (x (λy (x (λw (w x))))), the p.193 example, forwards
(print (μkanren-run (q -1 #t) (trans° '() '(λ x (app x (λ y (app x (λ w (app w
 ↪  x)))))) q)))
; λx λy x backwards: the binder names come back as constrained logic variables
(print (μkanren-run (q -1 #f) (trans° '() q '(lam (lam (var (s z)))))))
```

```
((lam (app (var z) (lam (app (var (s z)) (lam (app (var z) (var (s (s z))))))))))
((λ (α β) (begin (deny (equal? β α))) (assert (every (μ v (symbol? v)) (list α β)))
 ↪  (cons (quote λ) (cons α (cons (cons (quote λ) (cons β (cons α (quote ())))) (quote
 ↪  ()))))))
```

The forward answer is the book's `D`, shifted to 0-based indices. The backward answer shows what names cost. λProlog answers `abs W1\ abs W2\ W1`: there are no names, and the printer invents `W1`, `W2`. The named miniKanren answer is $(\lambda\,\alpha\,(\lambda\,\beta\,\alpha))$ for *any* two symbols $\alpha \neq \beta$. The disequality is needed to rule out the shadowed reading $\lambda x \lambda x\, x$, which would be $\lambda\lambda 1$. The names are logic variables with side conditions, and the programmer had to write the `≠°` that excludes capture. In λ-tree syntax there is no such condition to write.

6.6 Computations over first-order formulas

6.6.1 Vacuous quantifiers and quantifier-free formulas

Figure 7.6 ([MN p. 195]):

```
type vacuous    form -> form -> o.
type quantfree  form -> o.
vacuous (all  x\ P) P.
vacuous (some x\ P) P.
quantfree tt & quantfree ff.
quantfree F :- atom F.
quantfree (neg F) :- quantfree F.
quantfree (F && G) & quantfree (F !! G) & quantfree (F ==> G) :-
   quantfree F, quantfree G.
```

`vacuous` "succeeds exactly when the top-level abstracted variable does not appear in the body of the formula and, in this case, binds the second argument to the body" [MN p. 195]. There is no occurrence test in the code. Matching `(all x\ P)` against `(all x\ p a)` succeeds with `P = p a`. Matching it against `(all x\ p x)` would need `P = p x`, but `P` is quantified outside `x\`, so `x` cannot appear in its value and unification fails. The scoping constraint performs the occurrence test. `quantfree` relies on `atom` (as in Figure 7.2), which is defined for each object signature.

6.6.2 Negation normal form

A formula is in *negation normal form* (NNF) if it contains no implication and negation is applied only to atoms. It is in *prenex normal form* if no quantifier is in the scope of a connective. In classical first-order logic every formula is equivalent to one in NNF, to one in prenex form, and to one in both [MN p. 195]. The NNF transformation orients the equivalences

$$\neg\neg B \equiv B, \qquad B_1 \supset B_2 \equiv \neg B_1 \vee B_2, \qquad \neg\forall x\, B\, x \equiv \exists x\, \neg B\, x,$$
$$\neg(B_1 \wedge B_2) \equiv \neg B_1 \vee \neg B_2, \quad \neg(B_1 \vee B_2) \equiv \neg B_1 \wedge \neg B_2, \quad \neg\exists x\, B\, x \equiv \forall x\, \neg B\, x$$

from left to right (Figure 7.7, [MN p. 196]):

```
type nnf  form -> form -> o.
nnf A A & nnf (neg A) (neg A) :- atom A.
nnf (neg (neg B)) D :- nnf B D.
nnf (neg (B && C)) (D !! E) &
nnf (neg (B !! C)) (D && E) :- nnf (neg B) D, nnf (neg C) E.
nnf (B ==> C) (D !! E) :- nnf (neg B) D, nnf C E.
nnf (B !! C) (D !! E) &
nnf (B && C) (D && E) :- nnf B D, nnf C E.
nnf (neg (all B)) (some D) &
nnf (neg (some B)) (all D) :- pi x\ nnf (neg (B x)) (D x).
nnf (all B) (all D) &
nnf (some B) (some D) :- pi x\ nnf (B x) (D x).
```

The quantifier clauses are the idiom without `=>`. Under `pi x\` the eigenvariable `x` is an object *term*, and the only predicate that meets the atoms containing it is `atom`, which does not look inside terms. The output `(D x)` is again a pattern problem. Its solution abstracts `x` out of the transformed body, rebuilding the quantifier around it. For example (our trace, assuming `atom` holds of `p` atoms), `?- nnf (neg (all x\ p x)) D.` skips the two `atom`-guarded clauses, since neither the whole formula nor `all x\ p x` is atomic. It reaches the `neg (all B)` clause with `B = x\ p x` and `D = some D1`. Under an eigenvariable `c` the goal is `nnf (neg (p c)) (D1 c)`. The clause `nnf (neg A) (neg A) :- atom A` gives `(D1 c) = neg (p c)`, so `D1 := x\ neg (p x)` and `D = some x\ neg (p x)`, the formula $\exists x\, \neg p(x)$.

The book claims that no two heads overlap, so `nnf` is a partial function, and that "a simple argument by induction shows also that the function represented is total" [MN p. 196]. Both claims need qualification.

Pitfall: Non-overlap modulo a guard, and a missing case

Syntactically, the head `nnf A A` overlaps every other head, since `A` is a variable. The clauses are disjoint only *modulo the guard* `atom A`, which fails on compound formulas. A miniKanren port must keep that guard decisive. The `atom` relation must *fail*, not enumerate or suspend, on compound or unbound input, or `nnf` produces spurious answers. Moreover, `tt`, `ff` and `neg (B ==> C)` have no clause. The last matches none of `neg (neg _)`, `neg (_ && _)`, `neg (_ !! _)`, `neg (all _)`, `neg (some _)`. Totality holds only for the formulas without these cases. The fix when porting is one more clause, `nnf (neg (B ==> C)) (D && E) :- nnf B D, nnf (neg C) E.`

6.6.3 Prenex normal form

For formulas in NNF, the prenex transformation uses the equivalences [MN p. 196]

$$(\forall x\, B_1 x) \wedge (\forall x\, B_2 x) \equiv \forall x\, (B_1 x \wedge B_2 x), \qquad (\exists x\, B_1 x) \vee (\exists x\, B_2 x) \equiv \exists x\, (B_1 x \vee B_2 x),$$

and, for $Q \in \{\forall, \exists\}$ and $\circ \in \{\wedge, \vee\}$, $B_1 \circ (Q x\, B_2 x) \equiv Q x\, (B_1 \circ B_2 x)$ and $(Q x\, B_2 x) \circ B_1 \equiv Q x\, (B_2 x \circ B_1)$. The usual side condition, x not free in B_1, is not written anywhere in the program, because it is automatic: in the clauses below `B` and `C` are clause variables quantified outside the `pi x\`. (The restriction to NNF input "is not necessary and is used only to shorten the specification", [MN p. 196].) Figure 7.8 ([MN p. 197]):

```
type prenex, merge  form -> form -> o.
prenex A A & prenex (neg A) (neg A) :- atom A.
prenex (B && C) D :- prenex B U, prenex C V,
                     merge (U && V) D.
prenex (B !! C) D :- prenex B U, prenex C V,
                     merge (U !! V) D.
prenex (all B) (all D) &
prenex (some B) (some D) :- pi x\ prenex (B x) (D x).
merge ((all B) && (all C)) (all D) :-
   pi x\ merge ((B x) && (C x)) (D x).
```

```
merge ((all B) && C) (all D) &
merge ((some B) && C) (some D) :-
   pi x\ merge ((B x) && C) (D x).
merge (B && (all C)) (all D) &
merge (B && (some C)) (some D) :-
   pi x\ merge (B && (C x)) (D x).
merge ((some B) !! (some C)) (some D) :-
   pi x\ merge ((B x) !! (C x)) (D x).
merge ((some B) !! C) (some D) &
merge ((all B) !! C) (all D) :-
   pi x\ merge ((B x) !! C) (D x).
merge (B !! (some C)) (some D) &
merge (B !! (all C)) (all D) :-
   pi x\ merge (B !! (C x)) (D x).
merge B B :- quantfree B.
```

Look at the clause `merge ((all B) && C) (all D) :- pi x\ merge ((B x) && C) (D x).` The formula `C` moves under the new quantifier without any renaming, because `C` is a closed value with respect to `x`. Capture is impossible by construction.

Some formulas have a unique prenex form [MN p. 196]. For example

```
(all x\ (p x) && (all y\ q x y) && (p (f x))) !! (p a).
```

has only

```
all x\ all y\ ((p x) && (q x y) && (p (f x))) !! (p a).
```

In general, though, `prenex` is a relation, not a function [MN p. 196]–[MN p. 197]:

```
?- prenex ((all x\ q x x) && (all z\ all y\ q z y)) P.
```

has five answers [MN p. 197]:

```
all z\ all y\ (q z z) && (q z y)
all x\ all z\ all y\ (q x x) && (q z y)
all z\ all x\ (q x x) && (q z x)
all z\ all x\ all y\ (q x x) && (q z y)
all z\ all y\ all x\ (q x x) && (q z y)
```

Tutorial: Where the five prenex forms come from

Both conjuncts are already prenex, so the goal reduces to `merge ((all x\ q x x) && (all z\ all y\ q z y)) P`. The following explanation is our own hand trace against the clause order of Figure 7.8. It reproduces the book's order under depth-first search.

1. The $\forall \wedge \forall$ clause shares one eigenvariable between `x` and `z`, leaving `merge ((q c c) && (all y\ q c y))`, which pulls `y` out: answer 1.
2. The `((all B) && C)` clause pulls `x` alone, then `z` and `y` come out of the right side one at a time: answer 2.
3. The `(B && (all C))` clause pulls `z` first, leaving `merge ((all x\ q x x) && (all y\ q`

`c y))`. That subproblem again has three ways out: the ∀ ∧ ∀ clause sharing `x` with `y` (answer 3), pulling `x` then `y` (answer 4), or pulling `y` then `x` (answer 5).

Every answer is a correct prenex form. No other clause applies at any step: the ∨ clauses need `!!`, and `merge B B :- quantfree B` fails while a quantifier remains. A miniKanren port with the same clauses would have to produce exactly these five answers, but its interleaving search may list them in a different order.

6.6.4 Syntactic classes and a richer interpreter

Figure 7.9 ([MN p. 198]) recognises the goal and clause formulas of *fohc* and *fohh* (Chapters 2 and 3):

```
type fohcG, fohcD, fohhG, fohhD  form -> o.
fohcG tt.
fohcG A :- atom A.
fohcG (some B) :- pi x\ fohcG (B x).
fohcG (B && C) & fohcG (B !! C) :- fohcG B, fohcG C.
fohcD A :- atom A.
fohcD (G ==> D) :- fohcG G, fohcD D.
fohcD (D1 && D2) :- fohcD D1, fohcD D2.
fohcD (all D) :- pi x\ fohcD (D x).
fohhG tt.
fohhG A :- atom A.
fohhG (D ==> G) :- fohhD D, fohhG G.
fohhG (B && C) & fohhG (B !! C) :- fohhG B, fohhG C.
fohhG (some B) & fohhG (all B) :- pi x\ fohhG (B x).
fohhD A :- atom A.
fohhD (D1 && D2) :- fohhD D1, fohhD D2.
fohhD (G ==> D) :- fohhG G, fohhD D.
fohhD (all D) :- pi x\ fohhD (D x).
```

Here the book remarks on the `pi x\` without `=>`, a deviation from the idiom [MN p. 198]. The eigenvariables are object terms and `atom` does not inspect terms, so no assumptions are needed. (The same silent deviation occurs in `red1`, `nnf` and `prenex`.) The book also notes a free substitution theorem: "If $\forall x\, D$ is a first-order Horn clause and t is a term, then $D[t/x]$ is a first-order Horn clause" [MN p. 198]. It is the analogue of Theorem 6.4 and is proved the same way.

Two clauses extend the interpreter of Figure 7.1 from *fohc* to *fohh* [MN p. 198]:

```
interp D (D1 ==> G) :- interp (D1 && D) G.
interp D (all G) :- pi x\ interp D (G x).
```

Object ∧, ∨, ∃ in goals are handled by meta ∧, ∨, ∃, and the object ∀ in goals by the meta `pi`. Object implication in goals extends the object program. Since the object program is an explicit argument, the extension is just `(D1 && D)`, and no meta-level `=>` is used.

6.7 Specifying substitution without β

Substitution into a term with an abstraction, by β, is one clause [MN p. 199]:

```
type subst (tm -> tm) -> tm -> tm -> o.
subst R N (R N).
```

As Chapter 4 showed, however, "full β-conversion is a complex operation on terms" [MN p. 199]. The book therefore shows how to specify substitution with only the weak β_0 of Section 6.9. The tools are a *copy* predicate and a hypothetical assumption.

6.7.1 Copy clauses

Figure 7.10 ([MN p. 200]):

```
type copy   tm -> tm -> o.
type subst  (tm -> tm) -> tm -> tm -> o.
copy (app M N) (app P Q) :- copy M P, copy N Q.
copy (abs M) (abs N) :- pi x\ copy x x => copy (M x) (N x).
subst M T S :- pi x\ copy x T => copy (M x) S.
```

`copy` specifies equality: for closed terms, `copy t s` is derivable if and only if $t = s$. A trace [MN p. 199]:

```
?- copy (abs x\ abs y\ app y x) M.
   % M := (abs M1); eigenvariable c; assume (copy c c)
?- copy (abs y\ app y c) (M1 c).
   % (M1 c) := (abs M2); eigenvariable d; assume (copy d d)
?- copy (app d c) (M2 d).
   % three backchainings: app clause, then (copy d d), then (copy c c)
   % finally (M2 d) unified with (app d c)
M2 = u\app u c
M1 = v\abs u\app u v
M  = abs v\abs u\app u v.
```

The equations `(M1 c) = (abs M2)` and `(M2 d) = (app d c)` are pattern problems, solved by abstracting the eigenvariable out. So `copy` *rebuilds* its input binder by binder, and the abstractions of the output are produced by unification.

Tutorial: Reading `subst` as a hypothetical copy

`subst M T S` copies the body of `M` except that the outermost bound variable copies to `T` [MN p. 200]. The clause does this in three steps:

1. the binder of `M` becomes a formula-level `pi x\`;
2. the assumption `copy x T` says "wherever you meet `x`, produce `T`";
3. `copy (M x) S` copies the body.

Inside the body, the assumptions `copy c c` for inner binders preserve inner variables. The assumption `copy x T` replaces the substituted one. Since `T` is closed with respect to every eigenvariable created inside, no capture can happen.

Proposition 6.6 (Correctness of `subst`, [MN p. 200])**.** *If* `(subst M T S)` *is derivable, then* `S` *is equal to* `(M T)`*.*

Proof. `pi x\ copy x T => copy (M x) S` is derivable. Since `copy` is equality in the program without assumptions, `copy T T` is derivable. Instantiate `x` with `T`: the assumption becomes `copy T T` and can be discharged by modus ponens. So `copy (M T) S` is derivable from the program without assumptions, hence `(M T)` = `S`. □

6.7.2 Copy clauses for first-order formulas, and where they come from

For the first-order formulas over the signature

```
type a  term.
type f  term -> term.
type g  term -> term -> term.
type p  term -> form.
type q  term -> term -> form.
```

the copy clauses and substitution are Figure 7.11 ([MN p. 201]):

```
kind term, form  type.
type copyterm  term -> term -> o.
type copyform  form -> form -> o.
type subst     (term -> form) -> term -> form -> o.

copyterm a a.
copyterm (f X) (f U) :- copyterm X U.
copyterm (g X Y) (g U V) :- copyterm X U, copyterm Y V.

copyform tt tt.
copyform ff ff.
copyform (neg B) (neg D) :- copyform B D.
copyform (B && C) (D && E) &
copyform (B !! C) (D !! E) &
copyform (B ==> C) (D ==> E) :- copyform B D, copyform C E.
copyform (all B) (all D) &
copyform (some B) (some D) :- pi y\ copyterm y y =>
                                    copyform (B y) (D y).
copyform (p X) (p U) :- copyterm X U.
copyform (q X Y) (q U V) :- copyterm X U, copyterm Y V.

subst M T N :- pi x\ copyterm x T => copyform (M x) N.
```

These clauses are not ad hoc. They are generated from the signature by a translation that "lowers" the equality at a higher type to relations at the base types [MN p. 201]–[MN p. 202].

Definition 6.7 (Type lowering, [MN p. 201]–[MN p. 202])**.** For terms t, s of type τ built from the base types `term` and `form`, define the formulas $[\![t, s : \tau]\!]^+$ (for goals) and $[\![t, s : \tau]\!]^-$ (for

clauses) by

$$\begin{aligned}
[\![t, s : \mathsf{term}]\!]^{\pm} &= \mathsf{copyterm}\ t\ s, \qquad [\![t, s : \mathsf{form}]\!]^{\pm} = \mathsf{copyform}\ t\ s,\\
[\![t, s : \tau \to \sigma]\!]^{+} &= \forall x \left([\![x, x : \tau]\!]^{-} \supset [\![t\ x,\ s\ x : \sigma]\!]^{+}\right),\\
[\![t, s : \tau \to \sigma]\!]^{-} &= \forall x\, \forall y \left([\![x, y : \tau]\!]^{+} \supset [\![t\ x,\ s\ y : \sigma]\!]^{-}\right).
\end{aligned}$$

The positive case is the idiom itself: to compare two functions, apply both to a fresh x and assume that x is related to itself. The negative case, used for clauses, compares arguments pairwise. Polarity flips at each arrow, just as an arrow-typed argument of a constructor turns a clause into a hypothetical goal.

Example 6.8 (Lowering the quantifier `all`, [MN p. 201]–[MN p. 202])**.** For $\mathsf{all} : (\mathsf{term} \to \mathsf{form}) \to \mathsf{form}$:

$$\begin{aligned}
&[\![\mathsf{all}, \mathsf{all} : (\mathsf{term} \to \mathsf{form}) \to \mathsf{form}]\!]^{-}\\
&\quad = \forall x \forall y \left([\![x, y : \mathsf{term} \to \mathsf{form}]\!]^{+} \supset [\![\mathsf{all}\, x, \mathsf{all}\, y : \mathsf{form}]\!]^{-}\right)\\
&\quad = \forall x \forall y \left((\forall z\, [\![z, z : \mathsf{term}]\!]^{-} \supset [\![x\, z,\ y\, z : \mathsf{form}]\!]^{+}) \supset [\![\mathsf{all}\, x, \mathsf{all}\, y : \mathsf{form}]\!]^{-}\right)\\
&\quad = \forall x \forall y \big(\forall z\, (\mathsf{copyterm}\ z\ z \supset \mathsf{copyform}\ (x\, z)\ (y\, z))\\
&\qquad\quad \supset \mathsf{copyform}\ (\mathsf{all}\, x)\ (\mathsf{all}\, y)\big).
\end{aligned}$$

This is the `all` clause of Figure 7.11. "In fact, the clauses in that figure are exactly the formulas $[\![c, c : \tau]\!]^{-}$ for each of the constants c : τ of the object logic" [MN p. 202].

Proposition 6.9 ([MN p. 202])**.** *`(copyterm T S)` is derivable if and only if `T` and `S` are equal terms, and `(copyform B C)` is derivable if and only if `B` and `C` are equal formulas. Hence, if `(subst M T S)` is provable, then `S` is equal to `(M T)`.*

Two remarks from [MN p. 202]. First, these programs are *signature-dependent*: the object signature is reflected into clauses, whereas the one-line β version is not. Second, lowering is general. Each primitive type maps to its own predicate, the relation need not be binary, and the `term` predicate of Section 6.4.3 is the arity-one lowering for the signature `abs`, `app`.

Substitution can also be defined by direct recursion on the abstraction, with no copy predicate (Figure 7.12, [MN p. 203]):

```
type subst      (term -> form) -> term -> form -> o.
type substterm  (term -> term) -> term -> term -> o.
subst (x\ tt) T tt.
subst (x\ ff) T ff.
subst (x\ neg (B x)) T (neg D) :- subst B T D.
subst (x\ (B x) && (C x))  T (D && E)  &
subst (x\ (B x) !! (C x))  T (D !! E)  &
subst (x\ (B x) ==> (C x)) T (D ==> E) :- subst B T D,
                                          subst C T E.
subst (x\ all  (B x)) T (all  D) &
subst (x\ some (B x)) T (some D) :-
   pi y\ substterm (x\y) T y => subst (x\ B x y) T (D y).
subst (x\ p (X x)) T (p U) :- substterm X T U.
```

```
subst (x\ q (X x) (Y x)) T (q U V) :- substterm X T U,
                                      substterm Y T V.
substterm (x\ x) T T.
substterm (x\ a) T a.
substterm (x\ f (F x)) T (f S) :- substterm F T S.
substterm (x\ g (F x) (G x)) T (g S R) :-
   substterm F T S,
   substterm G T R.
```

The book says only that these clauses are signature dependent, and in its notes that they are loosely based on Andrews' equational presentation of λ-conversion [MN p. 210]. Three points are worth seeing. The patterns `(x\ a)` and `(x\y)` are *vacuous* abstractions: they match only bodies that do not depend on `x`. Under a nested quantifier the body becomes a two-place abstraction `(x\ B x y)`. The assumption `substterm (x\y) T y` says that the inner bound `y`, seen as a constant function of `x`, substitutes to itself.

6.7.3 Substitution runs backwards

Every version of `subst` is a relation ([MN p. 202]). Given `T` and the result, it computes the abstraction:

```
?- subst M (f a) ((p (f a)) && (q a (f a))).
M = x\ (p x) && (q a x)
M = x\ (p x) && (q a (f a))
M = x\ (p (f a)) && (q a x)
M = x\ (p (f a)) && (q a (f a))
```

Each of the two occurrences of `(f a)` may or may not be abstracted, so there are 2×2 answers. They are the four *existential generalisations* of $p(f(a)) \wedge q(a, f(a))$ on the term $f(a)$ [MN p. 203]: $\exists x\ p(x) \wedge q(a, x)$, $\exists x\ p(x) \wedge q(a, f(a))$, $\exists x\ p(f(a)) \wedge q(a, x)$, $\exists x\ p(f(a)) \wedge q(a, f(a))$. The "abstracted" answer comes first. This is consistent with the convention of Chapter 3 that clauses added by an implication are tried before the static ones, so `copyterm x (f a)` is tried before `copyterm (f X) (f U)`.

Pitfall: Copy is equality logically, not operationally

"the operational behavior of the copy predicate can be quite different from that of the equality notion it encodes" [MN p. 207]. `?- M = N.` binds `M` to `N` at once. `?- copy M N.` builds terms. With

```
kind i     type.
type a     i.
type f     i -> i.
type copyi i -> i -> o.
copyi a a.
copyi (f X) (f Y) :- copyi X Y.
```

`?- copyi M N.` enumerates `a`, `(f a)`, `(f (f a))`, ..., and "if the order of the clauses for copyi were reversed, then this query would cause the depth-first λProlog interpreter to loop without providing any solution" [MN p. 207].

Kanren angle: Clause order and interleaving

The same predicate in (aux kanren micro), in both clause orders:

```
(define-relation (copyi° m n)
  (cond° ((=° m 'a) (=° n 'a))
         ((fresh° (x y) (=° m `(f ,x)) (=° n `(f ,y)) (copyi° x y)))))
(define-relation (copyi-rev° m n)
  (cond° ((fresh° (x y) (=° m `(f ,x)) (=° n `(f ,y)) (copyi-rev° x y)))
         ((=° m 'a) (=° n 'a))))

(print (μkanren-run (q 4 #t) (fresh° (m n) (=° q `(,m ,n)) (copyi° m n))))
(print (μkanren-run (q 4 #t) (fresh° (m n) (=° q `(,m ,n)) (copyi-rev° m n))))
```

```
((a a) ((f a) (f a)) ((f (f a)) (f (f a))) ((f (f (f a))) (f (f (f a)))))
((a a) ((f a) (f a)) ((f (f a)) (f (f a))) ((f (f (f a))) (f (f (f a)))))
```

The reversed order loops in λProlog but produces the same answers here. `define-relation` delays the recursive call and `cond°` interleaves its branches, so the base case is reached. Interleaving changes the *order* of answers (the five prenex forms come out in the order 1 3 2 4 5, Section 13.7.2; the four generalisations happen to keep the book's order, Section 13.6) and turns some depth-first loops into productive streams. It does not remove genuine divergence. `cbv` on $(\lambda x \lambda w\, w)\,\Omega$ has no answer under any strategy, and a conjunction such as "`copyi°` and then fail" still runs forever, because it has infinitely many candidates and no finite failure.

6.8 The λ-tree approach to abstract syntax

Concrete syntax (strings, whitespace, parentheses, infix operators, comments) is for humans. Parsing produces *abstract syntax*, a tree [MN p. 203]. For binders there are several choices [MN p. 204]: keep explicit names; or map object binding to the meta-language's binding, which is called *higher-order abstract syntax* (HOAS) [49]. The term covers "a range of encoding techniques", and the book distinguishes two of them sharply.

- In functional programming, and in constructive type theories that compute by evaluation, meta-level abstraction builds *rich function spaces*. "Because of the richness of these spaces, it is not possible to compare function-valued expressions for equality in any simple way". If quantified formulas are encoded with such functions, "we cannot also compare or structurally analyze formulas under such an encoding" [MN p. 204].
- Most logic programming languages have no meta-level binders in terms at all.
- The λ-terms of hohh use "a λ-calculus with a weak notion of functionality that makes it possible to analyze the structures of objects under the encoding" [MN p. 204].

The book contrasts "the two extremes in the treatment of binding—one using functions that are best thought of extensionally and the other using λ-expressions that are easily treated intensionally".

Definition 6.10 (λ-tree syntax, [MN p. 204])**.** "We shall use the term λ-tree syntax to denote the approach to higher-order abstract syntax that uses typed λ-terms modulo equality based on α-conversion, some sufficiently weak form of β-conversion, and possibly η-conversion."

In this chapter β is "weakened" by simple typing, which makes it terminating. The next section weakens it much further, to β_0.

The book's bibliographic notes ([MN p. 208]–[MN p. 210]) trace the idea back a long way. Encoding a binder as a constant applied to a λ-abstraction goes back at least to Church's simple theory of types (1940), and λProlog's `pi` and `sigma` are named after Church's Π and Σ. Computing with such terms starts with Huet and Lang (1978) and Miller and Nadathur (1987). The phrase "higher-order abstract syntax" comes from Pfenning and Elliott [49], and the name "λ-tree syntax" from Miller and Palamidessi (1999). The copy clauses of Section 6.7 go back to Miller (1991) at second order and to Felty for arbitrary orders. The L_λ fragment of the next section is Miller's [34].

For a miniKanren programmer, the options can be compared directly. The first column is the "functional" reading of HOAS that we ran in Section 6.3. The second is nominal logic programming as in αKanren [5, 4], based on nominal unification [58]. The third is λ-tree syntax.

	host closures	names + freshness (αKanren)	λ-tree syntax
α-equivalence	no (`eq?` only)	yes, via swapping	yes, built in
substitution	host application	a user relation with freshness side conditions	meta-level β (or β_0 + copy)
analysing a body	only by applying to a name	by unification under `tie`	by mobility (`pi`, `=>`)
building an abstraction by unification	impossible	yes	yes (pattern unification)
"x not free in M"	not expressible	freshness constraint $a \# M$	scoping of variables

The same notes ([MN p. 209]–[MN p. 210]) name nominal logic as the alternative approach: it is first-order, with name-swapping and freshness, and binding is derived from these primitives. Porting the book's programs to αKanren would mean rewriting them, not transliterating them: there is no β, mobility is replaced by fresh names, and hypothetical contexts must be threaded explicitly.

6.9 The L_λ subset

Predicate quantification is useful for manipulating λ-tree syntax, but "it is certainly not central to it" [MN p. 205]. Can β be weakened further? The book recalls the chapter's two principles: bound names are a fiction (α), and binders are mobile, from term level to formula level to proof level. It then identifies the variables that carry mobility.

Definition 6.11 (Essentially universal and essentially existential occurrences, [MN p. 205])**.** In a goal formula G, a bound variable occurrence is *essentially universal* if it is bound by a positive

∀, a negative ∃, or a term-level λ. It is *essentially existential* otherwise, that is, if it is bound by a negative ∀ or a positive ∃. In a program clause the classification is dual: an occurrence is essentially universal if it is bound by a negative ∀, a positive ∃, or a term-level λ, and essentially existential if it is bound by a positive ∀ or a negative ∃.

During proof search, essentially existential variables become logic variables, instantiated by unification with general terms. Essentially universal ones are instantiated only with eigenvariables. "It is the essentially universal bound variable occurrences that provide mobility during proof search" [MN p. 205]. In the clause `typeof (abs M) (arr A B) :- pi x\ typeof x A => typeof (M x) B.`, the clause variables `M`, `A`, `B` are essentially existential, while the `pi`-bound `x` sits in the negative body of the clause and is essentially universal.

Definition 6.12 (L_λ, [MN p. 205]–[MN p. 206])**.** L_λ restricts hohh by requiring (1) that all quantification be over non-predicate types, and (2) that "Every subterm in B with an essentially existential variable occurrence as its head, that is, a subterm of the form $(x\, t_1 \ldots t_n)$ $(n \geq 0)$, where x is essentially existentially quantified in B, must be such that $t_1, \ldots, t_n$ are distinct essentially universal variable occurrences bound in the scope of x."

Such an application $x\, y_1 \ldots y_n$ is a *higher-order pattern*. If x is ever instantiated with a term t, the only redexes created have the form $(t\, y_1 \ldots y_n)$ with the y_i not free in t. Up to α and η, $t = \lambda y_1 \ldots \lambda y_n\, t'$, and the reduction just strips the binders [MN p. 206]. Only this sub-case of β is needed.

Definition 6.13 (β_0, [MN p. 206])**.**

$$(\lambda x.\, s)\, x = s \qquad (\beta_0)$$

In the presence of α-conversion this is equivalent to $(\lambda y.\, s)\, x = s[x/y]$, provided x is not free in $\lambda y.\, s$.

Example 6.14 (Goal versus clause, [MN p. 206])**.** With $p : \mathsf{i} \to \mathsf{o}$ and $f : \mathsf{i} \to \mathsf{i}$, the formula $\forall_{\mathsf{i}\to\mathsf{i}} x\, \forall_{\mathsf{i}} y\, (p\,(x\, y) \supset p\,(f\, y))$ is an L_λ *goal*: both quantifiers are positive, so x and y are essentially universal, and nothing existential is applied. As a *program clause* it is not in L_λ: there both x and y are essentially existential, and $(x\, y)$ applies an existential variable to an existential one. Membership in L_λ depends on polarity.

The book states which programs of the chapter belong to L_λ [MN p. 206]: "all the program clauses in Figures 7.7 through 7.12 belong to L_λ", and exactly one clause of Figure 7.4 does not, namely `redex (app (abs R) N) (R N).`, where `R` is applied to the existential `N`. By the same definition, more programs of the chapter are outside L_λ, although the book singles out only `redex`. The table summarises the situation; the classification of the rows other than `redex` and Figures 7.7–7.12 is ours.

in L_λ (applications are `(M x)` with `x` `pi`-bound)	outside L_λ
`term`, `bnorm`, `red1`, `reduce`, `path`, `bpath`, `addbeta`, `typeof`, `trans`, `vacuous`, `nnf`, `prenex`, `fohcG` …, `copy`, every `subst` of Figures 7.10–7.12	`redex ((R N))`; `list_instan ((B T))`; interp/`backchain` `((G X), (D1 X))`; cbn/cbv `((R N), (R U))`; the one-line `subst R N (R N)`; `repeat` and `foreach` (they quantify over predicates)

In every non-L_λ clause of the chapter, the abstraction is applied to an argument of *base* type (`term` or `tm`). Instantiating `R : tm -> tm` with `N : tm` therefore never creates a new redex. It is plain "instantiate the bound variable", provided `R` is known when the application is formed.

A non-L_λ program can often be rewritten into L_λ [MN p. 206]. With the copy clauses of Figure 7.10, the offending clause becomes

```
redex (app (abs R) N) S :- subst R N S.
```

"If we do this, we obtain a specification that is entirely within L_λ." The translation has two costs [MN p. 207]. The signature must be reflected into copy clauses with $[\![\cdot]\!]^{\pm}$. And copy behaves operationally unlike equality (the `copyi` pitfall above).

Why care? Compare the two rules [MN p. 207]:

$$(\lambda x.\, M)\, N = M[N/x] \quad (\beta) \qquad\qquad (\lambda x.\, M)\, x = M \quad (\beta_0).$$

A β_0 step removes one abstraction–application pair, and the result is strictly smaller and has no new redexes. A β step may shrink the term (if x does not occur in M), may blow it up (if x occurs many times and N is large), and may create new redexes (if x occurs applied in M). The difference carries over to unification, the subject of Chapter 7 [MN p. 208]:

- unification modulo $\beta\eta$ is undecidable and may have several most general unifiers;
- L_λ unification is "much more like first-order unification": it is decidable, and when a unifier exists there is a single most general one [34, 48];
- "an overwhelming majority of clauses in 'typical' λProlog programs are within the L_λ fragment".

Pitfall: β_0 is not the same as L_λ

All L_λ unification problems are problems modulo α, β_0, η, but not conversely [MN p. 208]. Finding F with $\forall x\,(F\,x\,x = x)$ needs only β_0 to justify the solutions $F := \lambda u \lambda v\, u$ and $F := \lambda u \lambda v\, v$. It is not an L_λ problem, because F is applied to the same universal variable twice, and it has two incomparable most general unifiers. A unifier for a miniKanren must decide what to do with such non-pattern equations: reject them, or postpone them until they become patterns.

Finally, raising (Chapter 4), the transformation of $\forall y\, \exists x\, G$ into $\exists h\, \forall y\, G[(h\, y)/x]$ that implementations use to simplify quantifier alternation, "preserves the class of L_λ formulas" [MN p. 208]: $(h\, y)$ is a pattern.

Kanren angle: A requirements checklist for (aux kanren micro)

Gathering the boxes of this chapter, a miniKanren that runs the book's programs needs the following. Each item is justified by a program above. The design that provides them is in Chapters 11 and 12.

requirement	needed by
abstractions as data, equal up to α (and η)	every program; `(all p)`; answers like `abs W1\ W1`
application of a known abstraction to a term	`list_instan`, `interp`, `cbn`, `cbv`, `redex`
eigenvariables with the scoping constraint	every `pi`; correctness of `vacuous`, η-`redex`, `prenex`
scoped assumed facts, possibly with logic variables	`term`, `bnorm`, `typeof`, `trans`, `copy`
scoped assumed rules with bodies and own `∀` variables	`bpath`, `addbeta`, the `trans` variant
pattern unification for output abstractions `(D x)`	`red1`, `path` synthesis, `nnf`, `prenex`, `copy`, `subst` backwards
occurs check, also through abstractions	`typeof (abs x\ app x x)`
printing binders with invented names	every answer containing an abstraction

Some of this exists in the miniKanren literature. The miniKanren with symbolic constraints has an "eigen" operator with a scope check [21]. λKanren adds `tie`, application, `all` and assumed relations to μKanren [30], and its successor discusses the α-equivalence surprises of combining higher-order unification with miniKanren [29] (see Chapter 9). Today's (aux kanren micro) has none of these features. It does have first-order unification with occurs check and records, which can represent de Bruijn terms, and an interleaving search, which already changes some of the book's loops into productive streams.

6.10 Summary

- An object-level binder is a constant applied to a meta-level abstraction: `all (x\ B)`, `abs (x\ t)`. Equality of the meta-language includes α (bound names are a fiction), β and η (so `(all p)` is a quantifier too).
- Object-level substitution is meta-level β: `(B T)`. Interpreters (`interp`) and evaluators (`cbn`, `cbv`) need no renaming code at all.
- An abstraction cannot be decomposed into a name and a body. Recursion under a binder uses *mobility*: term-level λ, then formula-level `pi`, then proof-level eigenvariable, with `=>` assuming the clauses that describe the new eigenvariable (Definition 6.2).
- The eigenvariable's scoping constraint is part of the meaning of `pi`. It silently implements every "x not free in M" side condition (`vacuous`, η, prenex). A fresh constant without it is unsound.

- Assumptions are scoped, may contain logic variables (`typeof x A`), and may be rules with bodies and their own quantifiers (`bpath`, `addbeta`, `trans`). Some predicates have no static clause for variables at all.
- Metatheory comes cheap: the substitution lemma for `bnorm`, subject reduction for `typeof`, and the correctness of `subst` all follow from instantiating a `pi` and discharging a `=>`.
- Programs that *build* abstractions (`red1`, `nnf`, `prenex`, `copy`, path synthesis, `trans` backwards) rely on unification solving `(D x) = t` by abstracting the eigenvariable out: pattern unification.
- Copy clauses, generated from the signature by type lowering $[\![\cdot]\!]^{\pm}$, specify substitution using only β_0. They are logically equality but operationally term generators.
- The relations are not functions: `prenex` has five answers on the book's example, `subst` run backwards enumerates the four existential generalisations.
- λ-tree syntax is HOAS with λ-terms modulo α, a weak β and possibly η, which can be analysed intensionally, unlike host-language closures. L_λ restricts existential variables to patterns, so that only β_0 is needed and unification is decidable and unitary (Chapter 7).
- On today's (aux kanren micro), abstractions as closures work only as inputs, and explicit contexts with de Bruijn indices replace `pi`/`=>` at the cost of threading the context and writing capture conditions by hand. The requirements checklist above is the starting point of the proposal (Chapter 11).

7 Unification of λ-terms and higher-order patterns

7.1 What this chapter is about

The previous chapters built programs whose execution keeps asking one question: *find values for the logic variables that make two λ-terms equal modulo α, β and η.* Chapter 4 showed how such questions are written as quantified formulas; Chapters 5 and 6 produced them by the thousand, from higher-order predicates such as `mappred` and from the `pi x\ ...` goals that walk under binders in λ-tree syntax. Chapter 6 also isolated the fragment L_λ, in which a logic variable is applied only to distinct bound variables. This chapter, which follows Chapter 8 of the book [36], explains how an implementation *solves* these questions, and why L_λ is the fragment to aim for.

The story has two halves.

1. **General higher-order unification** is hard: there may be no most general unifier, complete sets of unifiers may be infinite, and unifiability is undecidable (Section 7.3). Still, Huet's procedure [25] organises the search (Section 7.4): it simplifies rigid-rigid equations, *guesses* imitation or projection substitutions for flexible-rigid ones, and stops when only flexible-flexible equations are left. Its search space is a *matching tree*, which can be infinite.
2. **Higher-order pattern unification** (L_λ-unification, Sections 7.5 and 7.6) is the same procedure restricted to patterns. There it becomes *deterministic*, has *most general unifiers* and, with an occurs-check, *terminates* [34]. Its final form, *variable elimination*, is "computationally similar to first-order unification" [MN p. 224]: one walk over the rigid side that inverts it, prunes, and checks occurrences and scope.

We then do the algorithm by hand on the book's examples (Section 7.7), rebuild Nipkow's de Bruijn implementation [48] and run a Scheme transliteration of it (Section 7.8), and finish with what real systems do: universe levels instead of raising, and delayed non-pattern equations (Section 7.9). If you only need the implementer's digest, read Definition 7.21 and Theorem 7.22 and Sections 7.8 and 7.9.

Conventions. As in the book's Chapter 8, uppercase F, G, H, U, V, W, X, Y are *existentially* quantified variables (logic variables), and lowercase $a, b, c, d, f, g, u, w, x, y, z$ are *universally* quantified ones (signature constants or eigenvariables). $\bar{x}$ abbreviates $x_1 \dots x_n$, and $(F\,\bar{x})$ is $(F\,x_1 \dots x_n)$.

7.2 Unification problems and their normal form

Definition 7.1 (Unification problem [MN p. 211])**.** A *unification problem* is a formula

$$Q_1 x_1 \ldots Q_n x_n \, [\, t_1 = s_1 \wedge \cdots \wedge t_m = s_m \,]$$

where $n, m \geq 0$, each Q_i is $\forall$ or $\exists$, and t_j, s_j are λ-terms of the same type. The quantifier sequence is the *prefix*, the conjunction is the *matrix*. A *unifier* is a substitution for the existentially quantified variables that makes every t_j λ-convertible to s_j.

Reading the problem as a formula is what makes the scoping right. In $\exists X \, \forall y \, [X = y]$ no unifier exists, because X is chosen before y and cannot mention it; in $\forall y \, \exists X \, [X = y]$ the unifier is $X := y$. A miniKanren with only first-order terms never sees the difference because it has no universal quantifier in goals; a λProlog with `pi` sees it all the time.

Chapter 4 distinguished a unifier from a *solution*, which in addition uses only closed terms and therefore "embodies an actual proof of the quantified formula"; following common practice the book's Chapter 8 limits itself to unifiers [MN p. 211]. (The difference matters with empty types; see the last pitfall of Section 7.9.1.)

7.2.1 The normal form assumed by the algorithms

Before solving, the book puts every problem in a fixed shape [MN p. 212]. The four steps are worth knowing one by one, because an implementation performs each of them lazily.

1. **A $\forall\exists\forall$ prefix.** Using *raising* [MN p. 107–108], the formulas $\forall y \, \exists x \, G$ and $\exists h \, \forall y \, G[(h\, y)/x]$ are equiprovable when $h : \sigma \to \tau$ for $y : \sigma$, $x : \tau$. Moving every $\exists$ to the left of every $\forall$ in this way yields a prefix $\forall \bar{a} \; \exists \bar{X} \; \forall \bar{y}$. The outer $\forall \bar{a}$ represent "the ambient signature comprising the declared constants in a λProlog program, and we will interpret them as such" [MN p. 212]. The price: "the type of the variable that is existentially quantified needs to be 'raised' by the type of the universally quantified variable" [MN p. 108].

2. **Substitution as β-redex.** For $\theta = \{\langle x_1, t_1 \rangle, \ldots, \langle x_n, t_n \rangle\}$, the book defines [MN p. 212]

   $$\theta(t) := ((\lambda x_1 \ldots \lambda x_n \, t)\, t_1 \ldots t_n).$$

   Thus $\theta(t)$ and $\theta(t')$ are equal whenever t and t' are λ-convertible, and we may assume all terms are in β-normal form.

3. **Equal binders.** By η-conversion the two sides of every equation get binders of the same length.

4. **Binders become universals.** Using $(\lambda x \, t = \lambda x \, s) \equiv \forall x \, (t = s)$ and moving such $\forall x$ out of the conjunction, the binders at the heads of the equations turn into universals that *extend the inner block*. After this step "the equations in the matrix are between terms that have empty binders. We shall assume that all unification problems are of this kind" [MN p. 212].

Example 7.2 (Normalising a flexible-flexible problem)**.** Chapter 4 met the problem [MN p. 113]

$$\exists G\, \exists H\, [\lambda x \lambda y\, (G\, x) = \lambda x \lambda y\, (H\, y)].$$

Step 4 turns it into $\exists G\, \exists H\, \forall x\, \forall y\, [(G\, x) = (H\, y)]$. Now x and y are universals quantified *inside* the scope of G and H. Any common instance must be a term in which neither x nor y occurs, and the book announces that the "most general" unifier substitutes $\lambda w\, T$ for both G and H, with T new [MN p. 113]. We derive it in Example 7.27.

Remark 7.3. Step 4 is exactly what an implementation does when it meets $\lambda x\, s = \lambda x\, t$: open both binders with one fresh eigenvariable and continue. Because the new $\forall$ goes at the *end* of the prefix, no existing logic variable may depend on it, which is what a "newest level" eigenvariable gives you (Section 7.9).

Definition 7.4 (Rigid and flexible [MN p. 214], [MN p. 113])**.** In a normalised problem, a term on either side of an equation is *rigid* if its head is bound by a universal quantifier of the prefix, and *flexible* if its head is bound by an existential one. Equations are accordingly rigid-rigid, rigid-flexible, flexible-rigid or flexible-flexible; a problem is flexible-flexible if all its equations are.

The key fact about a rigid term is that "its head is unaffected by any substitutions that might be made for existentially quantified variables" [MN p. 214]. A flexible head, on the other hand, can be replaced by any term, including one that throws away or reorders the head's arguments. This asymmetry drives every rule below.

Kanren angle: first-order unification is the degenerate case

In (aux kanren micro) every Scheme atom, pair, vector or record constructor plays the role of an outer universal $\forall \bar{a}$ (a rigid head), and every logic variable is an existential. There is no inner $\forall$ block and no logic variable is ever applied to arguments, so every flexible term is a bare X, which is a pattern with $n = 0$. The unifier `µkanren-state-unify` is precisely the rigid-rigid decomposition plus the $n = 0$ case of variable elimination (bind X after an occurs-check). Everything in this chapter is an extension of that code path, not a replacement. What a higher-order extension needs is: inner universals (eigenvariables) as a new kind of rigid head, binders, applications of logic variables, and a scope discipline; see Chapter 11.

7.3 Why general higher-order unification is hard

Recall from first-order unification (Chapter 2) that a *most general unifier* (mgu) is a unifier from which every other unifier is obtained "by making further substitutions" [MN p. 212]. It satisfies all the constraints while leaving every other way of satisfying them open, which is what makes Prolog's and miniKanren's "one substitution per answer" work. At higher order this breaks down in three ways.

Example 7.5 (No mgu [MN p. 213])**.** Let $a : i$, $g : i \to i \to i$, $F : i \to i$ and consider $\forall a\, \forall g\, \exists F\, [(F\, a) = (g\, a\, a)]$. It has exactly four unifiers,

$$F := \lambda x\, (g\, x\, x), \qquad F := \lambda x\, (g\, a\, x), \qquad F := \lambda x\, (g\, x\, a), \qquad F := \lambda x\, (g\, a\, a),$$

because each occurrence of a on the right can come either from the argument of F or from the constant itself. All four are closed, so none is an instance of another: *there is no mgu.*

Definition 7.6 (Complete set of unifiers [MN p. 213])**.** A set $\mathcal{U}$ of substitutions is a *complete set of unifiers* (CSU) for a problem if every member of $\mathcal{U}$ is a unifier, and every unifier is obtained from some member of $\mathcal{U}$ by a further substitution. One would like $\mathcal{U}$ to be finite.

Example 7.7 (No finite CSU [MN p. 213])**.** With $w : i$ and $F, u : i \to i$, the problem

$$\forall u\, \exists F\, \forall w\, [(u\, (F\, w)) = (F\, (u\, w))]$$

is solved by $F := \lambda y\, y$, $\lambda y\, (u\, y)$, $\lambda y\, (u\, (u\, y))$, ..., that is, by every power u^k. Each of these is closed, so every CSU contains all of them.

One might settle for a CSU without redundancy, in which no two members have a common instance; but "a general guarantee of this kind of nonredundancy cannot be given in the higher-order setting" [MN p. 213]. Worse, *unifiability* itself is undecidable: Chapter 4 encoded the Post correspondence problem as a unification problem [MN p. 114], so "higher-order unification also must be undecidable" [MN p. 214]. The bibliographic notes add that undecidability already holds at third order (Huet; Lucchesi) and even at second order (Goldfarb, 1981), while *matching*, where only one side contains existential variables, is decidable (second order: Huet and Lang 1978; third order: Dowek 1992; general case: Stirling 2009) [MN p. 226–227].

Pitfall: do not promise "the" answer

A higher-order equation in general has no single best answer, may have infinitely many incomparable ones, and may run forever without being able to report "no". Any design that represents an answer as *one* substitution, as (aux kanren micro) does, can be exact only on a fragment that has mgus. That fragment is L_λ.

7.4 Huet's procedure

Despite all of the above there is a systematic, complete search [MN p. 214]. We present it as a rewrite system on pairs $\langle \mathcal{Q}; E \rangle$ of a prefix and a multiset of equations, in the style of Snyder and Gallier that the book mentions as an alternative presentation [MN p. 227]. The side conditions and substitution terms are the book's; the rule names are ours.

7.4.1 Simplification of rigid-rigid equations

$$\frac{\langle \mathcal{Q};\ \{(c\, s_1 \ldots s_n) = (c\, t_1 \ldots t_n)\} \uplus E \rangle}{\langle \mathcal{Q};\ \{s_1 = t_1, \ldots, s_n = t_n\} \cup E \rangle}\ \textsc{Dec} \qquad \frac{\langle \mathcal{Q};\ \{(c\, \bar{s}) = (d\, \bar{t})\} \uplus E \rangle}{\bot}\ \textsc{Clash}\ \ (c \neq d)$$

Here c, d are universally quantified in $\mathcal{Q}$; for $n = 0$, Dec just deletes the equation. Since the head of a rigid term can never change, an equation between rigid terms with different heads is hopeless, and one with equal heads is equivalent to the equations between the arguments. Dec removes two symbols from the problem [MN p. 214]. "The process of repeatedly treating all rigid-rigid pairs in this fashion will be called *simplification*" [MN p. 214]. Rigid-flexible equations are turned around and treated as flexible-rigid ones.

7.4.2 Imitation and projection

Take a flexible-rigid equation

$$(F\, t_1 \ldots t_n) = (c\, s_1 \ldots s_m), \qquad F : \tau_1 \to \ldots \to \tau_n \to \beta, \quad c : \sigma_1 \to \ldots \to \sigma_m \to \beta,$$

with β primitive. "This equation can be proved only by making the head of the left term identical to c" [MN p. 215], and there are only two ways to do that: the term substituted for F has head c itself, or it uses one of F's arguments as head.

Definition 7.8 (Imitation and projection [MN p. 215–216])**.**

- The *imitation* substitution, allowed only if "the quantification over F occurs within the scope of the quantifier that binds c", is

  $$\theta_{\text{imit}} = \{\langle F,\ \lambda x_1 \ldots \lambda x_n\, (c\, (H_1\, x_1 \ldots x_n) \cdots (H_m\, x_1 \ldots x_n))\rangle\}$$

  with fresh $H_i : \tau_1 \to \ldots \to \tau_n \to \sigma_i$.

- The i-th *projection* substitution, allowed only if the target type of τ_i is β, say $\tau_i = \varphi_1 \to \ldots \to \varphi_k \to \beta$, is

  $$\theta_{\text{proj}_i} = \{\langle F,\ \lambda x_1 \ldots \lambda x_n\, (x_i\, (H_1\, x_1 \ldots x_n) \cdots (H_k\, x_1 \ldots x_n))\rangle\}$$

  with fresh $H_j : \tau_1 \to \ldots \to \tau_n \to \varphi_j$.

In both cases $\exists F$ is replaced in the prefix by the new $\exists H_j$ at the same place, θ is applied to the matrix, and the result is β-normalised.

The design idea is minimal commitment. Imitation fixes only the head c and leaves "all choices of what the arguments should be to the determination of substitutions for the H_j variables in subsequent steps" [MN p. 215]; each H_j receives all of F's arguments, so nothing is lost. Projection commits F "only to projecting onto its ith argument" [MN p. 216]. For one flexible-rigid equation there may or may not be an imitation and there are between 0 and n projections, so "up to $n + 1$ substitutions may have to be considered" [MN p. 216].

Pitfall: imitation is scope-sensitive, projection is type-sensitive

Imitating c is illegal when c is quantified *inside* the scope of F: F cannot mention it. This side condition is the logical root of every "eigenvariable escape" check. Projection needs the target type and the arity of τ_i, and imitation needs the arity m of c and its argument

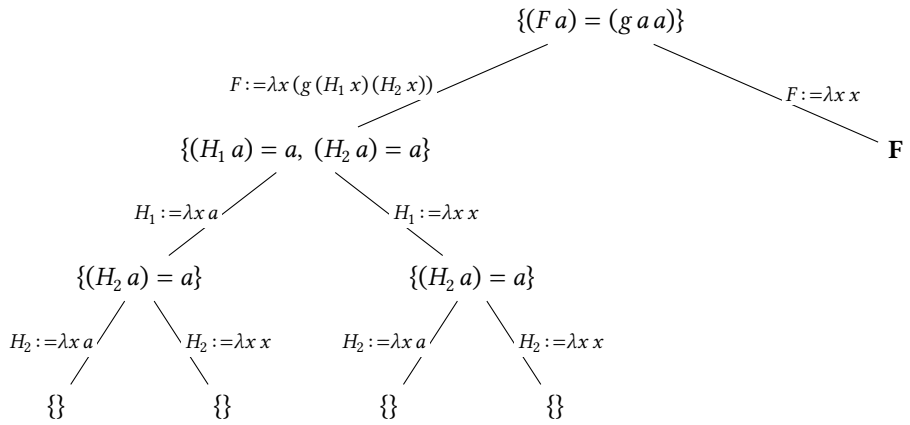


Figure 7.1: The matching tree for $\forall a\, \forall g\, \exists F\, ((F\, a) = (g\, a\, a))$, after Figure 8.1 of the book [MN p. 219]. Arcs are labelled by substitutions; **F** marks a failure node; the four $\{\}$ leaves are success nodes.

types σ_i: *Huet's procedure cannot run without types.* Pattern unification can (Section 7.6).

7.4.3 A complete run, and the matching tree

Tutorial: Huet's procedure on `(F a) = (g a a)`

We solve Example 7.5, following the book [MN p. 216–217].

1. The only equation is flexible-rigid with head g, and $\forall g$ is to the left of $\exists F$, so imitation is allowed: $F := \lambda x\, (g\, (H_1\, x)\, (H_2\, x))$ with $H_1, H_2 : i \to i$. The single projection is $F := \lambda x\, x$ (the argument type i is the target type).
2. Projection gives $\forall a\, \forall g\, [a = (g\, a\, a)]$: a rigid-rigid clash, a *dead end*.
3. Imitation gives $\forall a\, \forall g\, \exists H_1\, \exists H_2\, [(g\, (H_1\, a)\, (H_2\, a)) = (g\, a\, a)]$, which Dec simplifies to $[(H_1\, a) = a \wedge (H_2\, a) = a]$.
4. For $(H_1\, a) = a$ both the imitation $\lambda x\, a$ and the projection $\lambda x\, x$ work; either leaves $[\top \wedge (H_2\, a) = a]$.
5. The same two choices for H_2 empty the matrix.

Composing the substitutions along the four successful paths gives the four unifiers $\lambda x\, (g\, a\, a)$, $\lambda x\, (g\, a\, x)$, $\lambda x\, (g\, x\, a)$, $\lambda x\, (g\, x\, x)$ [MN p. 217], in the order of a left-to-right depth-first walk of Fig. 7.1.

Definition 7.9 (Matching tree [MN p. 219])**.** The *matching tree* of a problem has as nodes

the simplified matrices, viewed as multisets of equations (the prefix is implicit), and as arcs the imitation and projection substitutions for one selected flexible-rigid equation. Leaves are *failure nodes*, "for which it is immediately apparent that there are no solutions", or *success nodes*, whose matrix contains at most flexible-flexible equations. The *pre-unifier* of a success node is the composition of the arc labels on the path to it.

Theorem 7.10 (Completeness of matching trees [MN p. 219–220])**.** *If the problem at the root has a unifier, then the tree has a success node at a finite depth, and that unifier is obtained by composing the pre-unifier of the success node with a unifier of the problem at that node. This holds however the flexible-rigid equation to expand is chosen at each node.*

So the choice of *which equation* to work on is don't-care nondeterminism, while the choice of *which substitution* is don't-know nondeterminism. The bibliographic notes add that Huet's procedure, "which is limited to determining unifiability, is nonredundant" [MN p. 227].

7.4.4 Flexible-flexible equations and pre-unification

Why stop at flexible-flexible equations? Because they always have a unifier, and their unifiers are uncontrollable.

Example 7.11 (A flexible-flexible residue [MN p. 217])**.** With $a : i$ and $X, Y, g : i \to i$, take $\forall a\, \forall g\, \exists X\, \exists Y\, [(X\, a) = (g\, (Y\, a))]$. Imitation $X := \lambda x\, (g\, (H\, x))$ and simplification give $\forall a\, \forall g\, \exists H\, \exists Y\, [(H\, a) = (Y\, a)]$, which no rule can reduce further. (The other alternative for X, the projection $X := \lambda x\, x$, gives $a = (g\, (Y\, a))$, a rigid-rigid clash.) The residue has the unifier $H, Y := \lambda x\, a$ (making $X = \lambda x\, (g\, a)$), and many more: any term built from a, g and other existentials, abstracted over some of the occurrences of a, will do, "too large and uncontrolled a set to try to generate explicitly" [MN p. 218].

Proposition 7.12 (Canonical substitution [MN p. 217–218])**.** *Associate with each primitive type τ a fixed new existential variable H^τ, and with each type $\tau_1 \to \ldots \to \tau_n \to \sigma$ (σ primitive) the* canonical constant term $\lambda x_1 \ldots \lambda x_n\, H^\sigma$*. The* canonical substitution *maps every existential variable of the prefix to the canonical constant term of its type. For a problem with only flexible-flexible equations, the canonical substitution is a unifier.*

The reason is simple: under it both sides of every equation become H^β. For Example 7.11, with W the variable chosen for i, the canonical substitution is $\{\langle H, \lambda x\, W\rangle, \langle Y, \lambda x\, W\rangle\}$ [MN p. 217]. Hence: "The reduction of a unification problem to a flexible-flexible form thus constitutes success from the perspective of determining unifiability. The process of carrying out such a reduction is known as *pre-unification*, and the substitution that leads to the corresponding success is called a *pre-unifier*" [MN p. 218].

7.4.5 Non-termination

Example 7.13 (An infinite branch with infinitely many answers [MN p. 218])**.** Let $x : i$ and $g, F : i \to i$: $\forall g\, \exists F\, \forall x\, [(F\, (g\, x)) = (g\, (F\, x))]$. The single projection $F := \lambda x\, x$ solves it. The imitation $F := \lambda x\, (g\, (H\, x))$ followed by simplification gives $\forall g\, \exists H\, \forall x\, [(H\, (g\, x)) = (g\, (H\, x))]$:

the same problem with H in place of F. Repeating forever, the paths of the tree yield $F := \lambda x\, x,\ \lambda x\, (g\, x),\ \lambda x\, (g\, (g\, x)), \ldots$

Example 7.14 (An infinite branch and no answer [MN p. 220])**.** Add a second equation, with constants $a, b : i$: $\forall g\, \exists F\, \forall x\, [((F\, (g\, x)) = (g\, (F\, x))) \wedge (F\, a) = (F\, b)]$. "The second equation in the matrix here causes each of the unifiers that we have seen for the first equation to be rejected" [MN p. 220]: $F = g^k$ gives $g^k\, a = g^k\, b$, a clash. The tree has no success node but has an infinite path. As unifiability is undecidable, such problems must exist [MN p. 220].

Kanren angle: the matching tree is a miniKanren search tree

Imitation versus projection is don't-know choice among at most $n + 1$ alternatives, i.e. a disjunction; the matching tree is the search tree of those disjunctions. A depth-first interpreter, like λProlog's, can be trapped in the infinite imitation branch of Example 7.13 before it ever tries the projection. miniKanren's interleaving `or°` instead reaches every success node at finite depth, provided each expansion step is guarded by a suspension (as `define-relation` and `fresh°` provide). The fairness is skewed: `or°` nests the binary interleaving to the right, so with k alternatives the first one gets about half of the steps; and conjunction gives no fairness guarantee across an infinite first conjunct. Two hard limits remain. First, Example 7.14 makes even `(run 1 ...)` diverge, and no search strategy can fix that. Second, imitation and projection need types at run time, which (aux kanren micro) does not have. A Huet layer is therefore at best an opt-in, typed search over equations that the deterministic core has set aside; see Section 14.5.4 (and Section 11.2 for why Huet's search is not the meaning of `=°`).

7.5 Higher-order patterns

Chapter 6 introduced L_λ as a programming discipline. The book recalls it as follows.

Definition 7.15 (The L_λ condition [MN p. 220], [34])**.** A program clause or goal formula B is in L_λ if all quantification in B is over non-predicate types and "every subterm of B of the form $(x\, t_1 \ldots t_n)$ in which x is an essentially existentially quantified variable is such that $t_1, \ldots, t_n$ are distinct variables that are universally quantified within the scope of the quantifier binding x" [MN p. 220]. Transposed to a unification problem: every flexible subterm $(F\, t_1 \ldots t_n)$ has as arguments *distinct* universal variables quantified *to the right of* F in the prefix. Such a term is a *(higher-order) pattern.*

The case $n = 0$ is allowed, so a bare logic variable is a pattern, and first-order unification is a special case. Miller's formal definition [34, Fig. 3, p. 10–11] is a judgement $Q \vdash_0 t : \tau$ relative to a prefix Q, whose key rule says that $x\, y_1 \cdots y_n$ is well formed when "x is existentially quantified in Q to the left of where the distinct variables $y_1 \ldots y_n$ are universally quantified". Table 7.1 gives a feel for the definition.

Table 7.1: Patterns and non-patterns among the chapter's examples.

problem	pattern?	why
$\exists F\,\forall x\,\forall y\,[(F\,x\,y) = \dots]$	yes	distinct universals, right of F
$\forall a\,\exists F\,[(F\,a) = \dots]$ (Example 7.5)	no	a is quantified *outside* F
$\exists F\,\forall x\,[(F\,x\,x) = x]$	no	arguments not distinct
$\forall g\,\exists F\,\forall x\,[(F\,(g\,x)) = \dots]$ (Example 7.13)	no	argument is not a variable
$\exists F\,\exists N\,[(F\,N) = \dots]$ (the `subst` clause)	no	argument is existential
$\forall f\,\exists X\,[X = (f\,X)]$	yes	$n = 0$

Example 7.16 (Discharging a constant [34, p. 4–5])**.** Miller motivates the fragment with `append`. With the program

$$\forall x\,\forall l\,\forall k\,\forall m\,(\mathit{append}\,l\,k\,m \supset \mathit{append}\,(\mathit{cons}\,x\,l)\,k\,(\mathit{cons}\,x\,m)), \qquad \forall k\,(\mathit{append}\,\mathit{nil}\,k\,k),$$

try to find X such that $\forall y\,(\mathit{append}\,(\mathit{cons}\,a\,(\mathit{cons}\,b\,\mathit{nil}))\,y\,X)$ is provable. The $\forall y$ introduces a new constant k, and two backchaining steps succeed only if X is instantiated with $(\mathit{cons}\,a\,(\mathit{cons}\,b\,k))$, which is impossible because X was chosen before k existed. "Such a failure here is quite sensible since the value of X should be independent of the choice of the constant used to instantiate $\forall y$". Now ask instead for $\forall y\,(\mathit{append}\,(\mathit{cons}\,a\,(\mathit{cons}\,b\,\mathit{nil}))\,y\,(H\,y))$. The same steps produce the pattern equation

$$(H\,k) = (\mathit{cons}\,a\,(\mathit{cons}\,b\,k)).$$

Two λ-terms for H make it true, $\lambda w\,(\mathit{cons}\,a\,(\mathit{cons}\,b\,k))$ and $\lambda w\,(\mathit{cons}\,a\,(\mathit{cons}\,b\,w))$, but "since H cannot contain k free, only the second of these possible substitutions will succeed". Miller calls this *discharging* the constant k from the term: H is the computed answer with the choice of eigenvariable abstracted away. In terms of Huet's procedure: imitation of *cons*, *a*, *cons*, *b* is forced, and at the leaf k only the projection is legal, because imitating k would violate the scope condition. The pattern algorithm below does this in one walk.

Definition 7.17 (β_0 [MN p. 206])**.** β_0-conversion is the special case $(\lambda x\,s)\,x = s$ of β; with α it is equivalent to $(\lambda y\,s)\,x = s[x/y]$, provided x is not free in $\lambda y\,s$.

Under the L_λ condition, instantiating a logic variable only ever creates β_0-redexes: "simply replacing the bound variables within the body of an abstraction with a new set of names" [MN p. 220]. A β_0-step makes the term smaller, so we may expect its inverse, L_λ-unification, to be much better behaved than inverting full β [MN p. 220].

Pitfall: β_0 alone is not enough

L_λ problems are unification problems modulo α, β_0, η, but not conversely. The problem $\forall x\,(F\,x\,x = x)$ needs only β_0, yet has the two incomparable unifiers $\lambda u \lambda v\,u$ and $\lambda u \lambda v\,v$ [MN p. 208]. The *distinctness* of the arguments is what buys most general unifiers, not merely the shape of the redexes.

7.5.1 Why the procedure becomes deterministic

Raising preserves the L_λ condition [MN p. 208, 220–221], so we may still assume a $\forall \bar{a}\, \exists \bar{X}\, \forall \bar{y}$ prefix, and rigid-rigid simplification "applies unchanged" [MN p. 221]. The interesting case is flexible-rigid.

Lemma 7.18 (At most one useful substitution [MN p. 221])**.** *Let $(F\, c_1 \dots c_n) = (c\, t_1 \dots t_m)$ with F existential and $c_1, \dots, c_n$ distinct universals quantified in the scope of F. Then at most one imitation or projection substitution avoids an immediate clash.*

Argument. If F is in the scope of c (c is to the left of F, e.g. a signature constant), then c differs from every c_i, which are to the right of F; every projection produces a head $c_i \neq c$ and clashes, so only imitation can succeed. If c is in the scope of F, imitation is not allowed, and the i-th projection succeeds only if $c_i = c$; since the c_i are distinct, at most one i qualifies, and if $c \notin \bar{c}$ the equation fails. "The preceding observations can be built into the substitution selection process so as to make it entirely deterministic" [MN p. 221]. □

Moreover the class is preserved: the substitutions considered have the form

$$\lambda x_1 \dots \lambda x_n\, (@\, (H_1\, x_1 \dots x_n) \cdots (H_k\, x_1 \dots x_n))$$

with $@$ either c or some x_i and the H_j quantified where F was; substituted into $(F\, c_1 \dots c_n)$ and β_0-reduced, it yields again patterns $(H_j\, c_1 \dots c_n)$ [MN p. 221].

7.5.2 Flexible-flexible pattern equations have mgus

Here is the second change. In L_λ a flexible-flexible equation has the form

$$(F\, c_1 \dots c_n) = (G\, d_1 \dots d_m),$$

with the c_i and d_j universals in the scope of F and G, and in a $\forall\exists\forall$ prefix F and G are "effectively bound 'at the same place'" [MN p. 222].

Proposition 7.19 (Flexible-flexible mgus [MN p. 222])**.** *Let H be a new existential quantified at the place of F.*

- ***Different heads*** *($F \neq G$). Let $e_1, \dots, e_\ell$ list the variables common to $c_1, \dots, c_n$ and $d_1, \dots, d_m$. Then*

  $$\{\langle F,\ \lambda c_1 \dots \lambda c_n\, (H\, e_1 \dots e_\ell)\rangle,\ \langle G,\ \lambda d_1 \dots \lambda d_m\, (H\, e_1 \dots e_\ell)\rangle\}$$

  is an mgu.

- ***Same head*** *($F = G$, so $n = m$). Let $e_1, \dots, e_\ell$ list the c_i such that $c_i = d_i$. Then*

  $$\{\langle F,\ \lambda c_1 \dots \lambda c_n\, (H\, e_1 \dots e_\ell)\rangle\}$$

  is an mgu.

In the first case a common instance t cannot contain a c_i that is not also among the d_j, and conversely; every such t is an instance of $(H\, \bar{e})$. In the second case the *same* substitution acts on both sides, so a variable survives only if it sits in the *same argument position* on both sides [MN p. 222]. (The binders reuse the names c_i, so in $\lambda c_1 \dots \lambda c_n\, (H\, e_1 \dots e_\ell)$ the e's are bound.)

Pitfall: same head: positions, not sets

For $\forall x\, \forall y\, [(F\, x\, y) = (F\, y\, x)]$ the arguments are the same *set*, but no position agrees, so $\ell = 0$ and the mgu is $F := \lambda x \lambda y\, H$. Intersecting sets, as in the different-heads case, would wrongly keep both arguments.

Theorem 7.20 (Mgus for higher-order patterns [MN p. 222], [34])**.** *"By combining the completeness property of matching trees with the observations that there is at most one productive path in the tree and that most general unifiers can be provided for flexible-flexible pairs in the L_λ setting, it follows that the higher-order pattern unification problem possesses the property of having most general unifiers"* [MN p. 222]. *For $\forall\exists\forall$ problems the mgu is unique up to renaming of variables [34, Prop. 9.2].*

7.5.3 Termination needs the occurs-check

The procedure so far may still loop, even on first-order problems. For $\forall f\, \exists X\, [X = (f\, X)]$ the only useful substitution is the imitation $X := (f\, H)$, which produces $\forall f\, \exists H\, [H = (f\, H)]$, "identical to the one with which we started" [MN p. 222]. In L_λ such behaviour arises only when the flexible head occurs in the rigid side; so "we can ensure termination for the unification procedure by adding an 'occurs-check' to the substitution-generation step" [MN p. 223]. Note that in L_λ every occurrence of F in the rigid side is fatal: since pattern arguments are variables, there is no flexible occurrence that a later substitution could erase.

7.6 The pattern unification algorithm

The book's final step is to fuse the imitation and projection traversal with the occurs-check traversal into "a larger, more efficient substitution generation operation" [MN p. 223], called *variable elimination* "because of its similarity to an operation of the same name discussed for first-order unification" [MN p. 223]. We now state the whole algorithm precisely. The rules restate the book's §8.3 and Miller's transitions [34, §6, p. 14–16]. Two things the book leaves implicit are made explicit here: binders *inside* terms, and the order of pruning.

Setting. The prefix is $\forall \bar{a}\ \exists \bar{X}\ \forall \Gamma$, where $\bar{a}$ are the signature constants and Γ is the growing list of inner universals, or *eigenvariables*. A state is a pair $\langle E; \theta \rangle$ of a multiset of equations and the substitution computed so far. Equations are symmetric. θ is applied eagerly, and terms are kept β_0-normal.

$$\frac{\langle \{\lambda x\, s = \lambda x\, t\} \uplus E; \theta\rangle}{\langle \{s[z/x] = t[z/x]\} \cup E; \theta\rangle}\ \textsc{Abs} \qquad \frac{\langle \{\lambda x\, s = t\} \uplus E; \theta\rangle}{\langle \{s[z/x] = (t\, z)\} \cup E; \theta\rangle}\ \textsc{Eta}$$

$$\frac{\langle \{(h\, s_1 \dots s_n) = (h\, t_1 \dots t_n)\} \uplus E; \theta\rangle}{\langle \{s_1 = t_1, \dots, s_n = t_n\} \cup E; \theta\rangle}\ \textsc{Dec} \qquad \frac{\langle \{(h\, \bar{s}) = (k\, \bar{t})\} \uplus E; \theta\rangle}{\bot}\ \textsc{Clash}$$

$$\frac{\langle \{(F\, \bar{c}) = t\} \uplus E; \theta\rangle}{\langle \rho'(\rho(E));\ \rho' \circ \rho \circ \theta\rangle}\ \textsc{Elim} \qquad \frac{\langle \{(F\, \bar{c}) = (G\, \bar{d})\} \uplus E; \theta\rangle}{\langle \sigma(E);\ \sigma \circ \theta\rangle}\ \textsc{FlexFlex}$$

Side conditions:

- Abs, Eta: z is a fresh eigenvariable appended to Γ; in Eta, t is not an abstraction. This is step 4 of the normal form, applied on demand.
- Dec, Clash: h, k are rigid heads (signature constants or eigenvariables) and $h \neq k$ in Clash. Untyped, an arity mismatch for the same head is also a clash [34, §9.3].
- Elim: $(F\, \bar{c})$ is a pattern, t is rigid, and the elimination $\mathscr{E}_{F,\bar{c}}(t)$ of Definition 7.21 succeeds with result t' and pruning substitution ρ; then $\rho' = \{\langle F, \lambda \bar{c}.\, t'\rangle\}$. If $\mathscr{E}$ fails, the state goes to $\bot$.
- FlexFlex: both sides are patterns and σ is the mgu of Proposition 7.19; for the same head with all positions agreeing, the equation is simply deleted.
- A flexible term that is *not* a pattern blocks the rules; we come back to it in Section 7.9.1.

Definition 7.21 (Variable elimination)**.** Let $(F\, c_1 \dots c_n)$ be a pattern. The elimination $\mathscr{E}_L(t)$ of a β_0-normal term t, relative to a set L of variables bound by λs *inside* t (initially $L = \emptyset$), either fails or returns a term, and may record pruning bindings in ρ. It is defined by cases on t, after applying the pruning bindings recorded so far:

$\mathscr{E}_L(a)$	$= a$	a a signature constant (outer $\forall$)
$\mathscr{E}_L(y)$	$= y$	$y \in \bar{c} \cup L$
$\mathscr{E}_L(y)$	= fail (scope)	$y \in \Gamma \setminus (\bar{c} \cup L)$
$\mathscr{E}_L(\lambda z\, s)$	$= \lambda z\, \mathscr{E}_{L \cup \{z\}}(s)$	
$\mathscr{E}_L(h\, s_1 \dots s_k)$	$= \mathscr{E}_L(h)\, \mathscr{E}_L(s_1) \dots \mathscr{E}_L(s_k)$	h rigid
$\mathscr{E}_L(F\, \bar{s})$	= fail (occurs)	
$\mathscr{E}_L(G\, d_1 \dots d_m)$	$= G'\, e_1 \dots e_\ell$	$G \neq F$ a pattern; see below

In the last case $e_1 \dots e_\ell$ is the subsequence of $d_1 \dots d_m$ of the variables in $\bar{c} \cup L$. If $\ell = m$, nothing is pruned and $G' = G$. Otherwise the *pruning substitution* $\langle G,\ \lambda d_1 \dots \lambda d_m\, (G'\, e_1 \dots e_\ell)\rangle$, with G' new and quantified at the place of G, is added to ρ.

This is the book's description [MN p. 223], case by case. "This pruning substitution replaces G with the term $\lambda d_1 \dots \lambda d_m\, (G'\, e_1 \dots e_\ell)$, where G' is a new existentially quantified variable, and

$e_1, \ldots, e_\ell$ is some listing of the variables that are common to $c_1, \ldots, c_n$ and $d_1, \ldots, d_m$. Now let the result of applying these auxiliary substitutions to the term t be t'. Then the main part of variable elimination checks whether there are occurrences in t' either of F or of variables different from the ones in $c_1, \ldots, c_n$ that are quantified universally within the scope of the quantifier binding F. If there are such occurrences, then it marks the problem as one for which no unifiers exist. Otherwise, it generates the substitution $\lambda c_1 \ldots \lambda c_n\, t'$ for F and replaces the equation under consideration by $\top$."

Read the definition as an *inversion*. F applied to $\bar{c}$ must produce t, so $\lambda \bar{c}.\, t'$ must "undo" the application: every c_i in t becomes the i-th bound variable of F's binder. A universal that F cannot see and that F does not receive as an argument has no preimage. In a rigid position it makes the problem unsolvable; inside the arguments of another flexible term G it can be removed by *pruning*, that is, by committing G to ignore that argument. Imitation (rigid heads copied) and projection (a c_i copied as a bound variable) are both inside this single walk.

Pitfall: four things the prose leaves implicit

1. An out-of-scope universal is fatal only in a *rigid* position. Inside the argument list of a flexible subterm it is pruned. The book's Example 7.25 below says exactly this: "in a position that is not the argument of a flexible term" [MN p. 224].
2. Variables bound by λs *inside* t are always allowed, both in rigid positions and as kept arguments of pruned terms; this is the set L. The book never needs L because its normal form has no inner binders at the top, but terms like $h\,(\lambda z\,(G\,z\,w))$ have them below the top.
3. Pruning *binds other variables* (G) as a side effect, although the equation is about F. If G occurs twice, the second occurrence must be examined *after* the first pruning, which may prune again (Example 7.29).
4. Pruning never meets F itself: that is the occurs failure.

Theorem 7.22 (Pattern unification is unitary and terminating)**.** *On a* $\forall\exists\forall$ *problem whose flexible subterms are all patterns, the rules terminate. If they reach* $\bot$ *the problem has no unifier; otherwise they end in* $\langle \varnothing; \theta \rangle$ *and* θ *is an mgu (*[MN p. 222–224]*; [34, Thm. 7.1, Prop. 7.3]).*

Miller proves termination with a lexicographic measure: the number of existentials, the total size of the equations, the number of equations, a weight of the universals, and the number of possibly prunable occurrences [34, Thm. 7.1, p. 18]. Intuitively, Elim and the different-heads case of FlexFlex each remove one existential for good (a pruned G is traded for one G'; F and G are traded for one H); the same-head case trades F for one H, so the number of existentials stays put, but the equation disappears (both sides become $H\,\bar{e}$); and the other rules shrink the problem. The occurs-check is what prevents the $X = (f\,X)$ loop. Qian later showed the problem can be solved in linear time [50].

Two remarks complete the picture. First, "types have no bearing on the shapes of unifiers": the algorithm "makes no use at all of type information in determining unifying substitutions", though types are still needed to guarantee correctness, for example that same-head flexible terms have equal arity [MN p. 224]. Miller's untyped variant replaces the type guarantee by

two failures, an arity mismatch in Dec and same-head flexible terms of different arity, and uses η-expansion "only on terms which, if typed, must have functional type" [34, §9.3]. That is the variant an untyped Scheme host needs. Second, the result is a genuine algorithm, as for first-order unification: "In both cases, term simplification and variable elimination constitute the critical parts of the computation, although variable elimination has a more involved structure in the higher-order pattern unification case" [MN p. 224].

7.7 Worked traces

Every result below was also checked mechanically with the Scheme transliteration of Section 7.8. The book does not number its unification problems. We label them 8.A–8.M in their order of appearance in the book's Chapter 8, as the proposal's test suite does, except 8.M, the problem $\forall x\,(F\,x\,x = x)$ of [MN p. 208] in the book's Chapter 7. The main example below is 8.H, the occurs and scope failures are 8.I and 8.J, and the eager-versus-delayed raising of Example 7.31 is 8.K.

Example 7.23 (The book's main example [MN p. 223–224])**.** With $x, y, w : i$, $f, g : i \to i$ and $U, V : i \to i \to i$, solve

$$\forall f\,\forall g\,\exists U\,\exists V\,\forall w\,\forall x\,\forall y\,[(f\,(U\,x\,y)) = (f\,(g\,(V\,y\,w)))].$$

Tutorial: the book's main example, step by step

1. Both sides are rigid with head f: Dec leaves $(U\,x\,y) = (g\,(V\,y\,w))$.
2. The left side is the pattern U applied to $\bar{c} = (x, y)$, and the right side is rigid: Elim. Walk $t = g\,(V\,y\,w)$ with $L = \emptyset$:
   - g is a signature constant, so it is kept (this is the imitation part);
   - $(V\,y\,w)$ is a flexible pattern with $\bar{d} = (y, w)$. Of these, $y \in \bar{c}$ is kept and $w \notin \bar{c}$ must go: prune $V := \lambda y\,\lambda w\,(V'\,y)$, so the subterm becomes $(V'\,y)$.

   Now $t' = g\,(V'\,y)$. U does not occur in it, and its only inner universal y is in $\bar{c}$, so bind $U := \lambda x\,\lambda y\,(g\,(V'\,y))$.
3. The matrix is empty. The mgu is $\{\langle V, \lambda y \lambda w\,(V'\,y)\rangle,\ \langle U, \lambda x \lambda y\,(g\,(V'\,y))\rangle\}$ [MN p. 224].

Check: $U\,x\,y \to_{\beta_0} g\,(V'\,y)$ and $V\,y\,w \to_{\beta_0} V'\,y$, so both sides become $f\,(g\,(V'\,y))$.

Example 7.24 (Occurs failure [MN p. 224])**.** $\forall f\,\forall g\,\exists U\,\exists V\,\forall w\,\forall x\,\forall y\,[(U\,x\,y) = (g\,(U\,y\,w))]$: the walk meets $(U\,y\,w)$, whose head is the variable being eliminated, so the problem fails, "because of the occurrence of U, the head of the flexible term, in the rigid term". Note that pruning cannot help: whatever U does with its arguments, the right side is one g larger than the left.

Example 7.25 (Scope failure [MN p. 224]). $\forall f\, \forall g\, \exists U\, \exists V\, \forall w\, \forall x\, \forall y\, [(U\, x\, y) = (g\, w)]$ "has no unifiers because the universally quantified variable w, which is not one of the arguments in $(U\, x\, y)$, appears on the right-hand side of the equation and in a position that is not the argument of a flexible term". Logically: U is chosen before w and does not receive it, so it cannot produce it.

Example 7.26 (Discharging, by elimination). For Example 7.16, $(H\, k) = (\mathit{cons}\, a\, (\mathit{cons}\, b\, k))$ with $\bar{c} = (k)$: the walk keeps the signature constants *cons*, *a*, *b* (imitation), meets $k \in \bar{c}$ and maps it to H's bound variable (projection), finds no flexible subterm to prune and no occurrence of H, and returns $H := \lambda w\, (\mathit{cons}\, a\, (\mathit{cons}\, b\, w))$, Miller's unique answer.

Kanren angle: the scope failure that surprised λKanren

Ma and Friedman [29] open with the miniKanren query `(run 1 X (== (λ (a) X) (λ (b) b)))` and observe that it "fails. Under any existing higher-order unification algorithm, X and a cannot be unified due to the scope restrictions", while a novice expects `X = a`. Read through Abs, the equation is $\exists X\, \forall z\, [X = z]$: Example 7.25 with $n = 0$, so the failure is the *correct* answer under the quantifier reading (no single X equals every z). Their proposal changes the semantics with de Bruijn levels so that the query answers `(d0)`; Chapter 9 examines that choice. Whichever semantics a new module picks, it must say so explicitly; see Chapter 11.

Example 7.27 (Flexible-flexible, different heads). For Example 7.2, $\exists G\, \exists H\, \forall x\, \forall y\, [(G\, x) = (H\, y)]$: here $\bar{c} = (x)$ and $\bar{d} = (y)$ have nothing in common, $\ell = 0$, and Proposition 7.19 gives

$$\{\langle G, \lambda x\, T\rangle,\ \langle H, \lambda y\, T\rangle\},$$

the book's "$\lambda w\, T$ for both" [MN p. 113]. Had the equation been $(G\, x\, y) = (H\, y)$, then $\bar{e} = (y)$ and the mgu would be $G := \lambda x \lambda y\, (T\, y)$, $H := \lambda y\, (T\, y)$.

Example 7.28 (Pruning under an inner binder). $\forall h\, \exists F\, \exists G\, \forall w\, \forall x\, [(F\, x) = (h\, (\lambda z\, (G\, z\, w)))]$. Elim with $\bar{c} = (x)$ walks h (kept), then the abstraction, adding z to L, then $(G\, z\, w)$: $z \in L$ is kept and w is pruned, $G := \lambda z \lambda w\, (G'\, z)$. Result: $F := \lambda x\, (h\, (\lambda z\, (G'\, z)))$. The local z is legal everywhere below its binder, which is item 2 of the pitfall after Definition 7.21.

Example 7.29 (Two occurrences, two prunings). $\forall g\, \exists F\, \exists G\, \forall w\, \forall x\, [(F\, x) = (g\, (G\, x\, w)\, (G\, w\, x))]$, with $\bar{c} = (x)$. The first occurrence prunes the second argument: $G := \lambda d_1 \lambda d_2\, (G'\, d_1)$. The second occurrence now reads $(G'\, w)$, and w must be pruned again: $G' := \lambda d\, G''$. So $G := \lambda d_1 \lambda d_2\, G''$ and $F := \lambda x\, (g\, G''\, G'')$. This is correct: $G\, x\, w$ must not mention w, so G ignores its second argument, and $G\, w\, x$ must not mention w, so it ignores its first.

Example 7.30 (η on demand). $\forall g\, \exists F\, [\lambda x\, (F\, x) = g]$: Eta gives $(F\, z) = (g\, z)$ with z fresh, and Elim binds $F := \lambda z\, (g\, z)$, which is η-equal to g. Without Eta, an abstraction facing a constant would be a spurious clash.

Table 7.2: A test battery. The Huet column describes the pre-unification procedure exactly as the book presents it in its §8.2: it stops at flexible-flexible equations, and it has no occurs-check, since that is added only for L_λ [MN p. 223]. The pattern column assumes the delaying discipline of Section 7.9.1.

problem (matrix)	L_λ?	pattern unification	Huet pre-unification
$(F\,a) = (g\,a\,a)$	no	not applicable (delay)	4 answers
$(G\,x) = (H\,y)$	yes	$G, H := \lambda w\, T$	already flex-flex; residue kept
$X = (f\,X)$	yes	fail (occurs)	diverges (no occurs-check)
$(f\,(U\,x\,y)) = (f\,(g\,(V\,y\,w)))$	yes	mgu of Example 7.23	$U := \lambda x \lambda y\, g\,(H\,x\,y)$, residue $(H\,x\,y) = (V\,y\,w)$
$(U\,x\,y) = (g\,(U\,y\,w))$	yes	fail (occurs)	diverges
$(U\,x\,y) = (g\,w)$	yes	fail (scope)	fail
$(F\,(g\,x)) = (g\,(F\,x))$	no	delay	infinitely many answers
$(F\,x\,x) = x$	no	delay	2 answers
$(F\,x\,y) = (F\,y\,x)$	yes	$F := \lambda x \lambda y\, H$	already flex-flex; residue kept

7.8 Nipkow's de Bruijn algorithm

Nipkow [48] turned the transformation rules into "a directly executable functional program" and then adapted it "to lambda-terms in de Bruijn's notation", for typed and untyped terms alike. It is short and complete, so we reconstruct it from the author's ML code. It assumes a ∀∃∀ prefix, raised in a preprocessing phase [MN p. 227].

Encoding the prefix in the terms. Existentials are global names `V F`, signature constants are `C c`, and an abstraction `A t` has a body in which bound variables are de Bruijn indices `B i` (Chapter 6). The inner ∀ block is not represented at all. To unify $\lambda w \lambda x \lambda y.\, s$ with $\lambda w \lambda x \lambda y.\, t$, the algorithm simply strips the binders, which is rule Abs, and the *loose* indices of s and t then play the inner universals: under three binders, y is `B 0`, x is `B 1`, w is `B 2`. Because every existential is quantified outside every inner universal, the values bound to the `V`s are closed terms, and "fresh eigenvariable" costs nothing.

The algorithm in pseudo-code; S is a triangular substitution, i.e. an association list that is walked rather than applied.

```
devar(S, s):                          ; β0 by renaming
  if s = (V F) b1..bk and S binds F to λ^k.body
  then devar(S, body with each bound index renamed to the matching b)  else s
unif(S, s, t):  s := devar(S, s);  t := devar(S, t);  case
  A s', A t'  → unif(S, s', t')                          ; Abs
  A s', t     → unif(S, s', (↑t) (B 0))                  ; Eta (and symmetric)
  F ȳ,  G z̄   → flexflex(F, ȳ, G, z̄, S)
  F ȳ,  t     → flexrigid(F, ȳ, t, S)                    ; (and symmetric)
  a s̄,  b t̄   → if a = b and |s̄| = |t̄| then fold unif over the pairs else FAIL
flexrigid(F, ȳ, t, S):
  if F occurs in t through S then FAIL                   ; occurs-check
  u := t with each loose index i replaced by the position
```

```
      of (B i) in ȳ counted from the right, or by •     ; inversion
  return proj(S + [F ↦ λ^|ȳ|. u], u)
proj(S, s):  case devar(S, s) of                        ; scope check and pruning
  A s'               → proj(S, s')
  (C c) s̄, (B i) s̄   → fold proj over s̄
  • s̄                → FAIL                             ; out of scope, rigid position
  (V G) b̄            → S + [G ↦ λ^|b̄|. H (positions of b̄ that are not •)],  H new
flexflex(F, ȳ, F, z̄, S):                                ; same head
  if ȳ = z̄ then S else S + [F ↦ λ^|ȳ|. H (positions i with y_i = z_i)]
flexflex(F, ī, G, j̄, S):                                ; different heads
  if ī ⊆ j̄ then S + [G ↦ λj̄. F ī]  elif j̄ ⊆ ī then S + [F ↦ λī. G j̄]
  else S + [F ↦ λī. H (ī ∩ j̄),  G ↦ λj̄. H (ī ∩ j̄)]
```

Compare it with the rules of Section 7.6.

- **Inversion first, pruning second.** `flexrigid` binds F to $\lambda^{|\bar{y}|}.u$ *before* pruning, so u may still contain markers • inside the arguments of other flexible terms. That is harmless: `proj` then binds each such G to a term that drops those positions, and since S is triangular, every later `devar` through F also goes through G and never sees a marker. (Nipkow's ML encodes the marker as the index `B(~10000)`, which shifting keeps negative.)
- **One walk.** Imitation is the copying of `C` heads, projection the renaming of loose indices, the scope check the • in rigid position, and pruning the `V` case: this is the book's "variable elimination". Inner binders need no set L: below `A`, indices that refer to local binders are never shifted into markers.
- **`devar` is β_0.** Arguments of a bound existential are indices, so "substituting" only renames indices; no shifting of arbitrary terms is needed.
- **An optimisation in `flexflex`.** If one argument list is contained in the other, one variable is bound directly to the other instead of introducing H; up to renaming, this is the mgu of Proposition 7.19.
- **No pattern test.** The code assumes its input is a pattern problem; a non-pattern argument would simply not be handled correctly. A dynamic-L_λ implementation adds a test there and delays (Section 7.9.1).

Tutorial: Example 7.23 in de Bruijn form

Under the three binders w, x, y the equation is $f\,(U\,1\,0) = f\,(g\,(V\,0\,2))$. Dec on f leaves $U\,1\,0 = g\,(V\,0\,2)$. Here $\bar{y} = (1, 0)$: index 1 (that is x) sits at position 1 from the right, index 0 (y) at position 0, and index 2 (w) is absent. So $u = g\,(V\,0\,\bullet)$ and $U := \lambda\lambda.\, g\,(V\,0\,\bullet)$. `proj` meets $V\,0\,\bullet$ and prunes: $V := \lambda\lambda.\, H\,1$, i.e. $\lambda y \lambda w\,(H\,y)$. Walking U's binding through S gives $\lambda x \lambda y\,(g\,(H\,y))$, the book's answer.

7.8.1 An executable transliteration

The following plain Scheme is a line-by-line transliteration of the de Bruijn ML code (file `lics93-db.ML`). It is *not* a proposal for the new module's interface (that is Chapter 11); it is

here so that you can run, trace and modify the algorithm. Terms are tagged lists, failure escapes through a continuation, and `(OUT)` is the marker •.

```
(define (tag t) (car t))
(define (V n) (list 'V n))  (define (C n) (list 'C n))  (define (B i) (list 'B i))
(define (A t) (list 'A t))  (define ($ s t) (list '$ s t))  (define OUT '(OUT))
(define escape #f)
(define (fail) (escape 'fail))

(define (strip t)                       ; head and arguments of an application spine
  (let loop ((t t) (ts '()))
    (if (eq? (tag t) '$) (loop (cadr t) (cons (caddr t) ts)) (cons t ts))))
(define (apps h ts) (fold (λ (t h) ($ h t)) h ts))  ; foldl op $
(define (abs n t) (if (zero? n) t (abs (- n 1) (A t))))
(define (hnf n h ts) (abs n (apps h ts)))           ; λ^n. h ts

(define (mapbnd f t)                    ; f on the loose indices of t; #f gives OUT
  (let mpb ((d 0) (t t))
    (case (tag t)
      ((B) (let ((i (cadr t)))
             (if (< i d) t (let ((j (f (- i d)))) (if j (B (+ j d)) OUT)))))
      ((A) (A (mpb (+ d 1) (cadr t))))
      (($) ($ (mpb d (cadr t)) (mpb d (caddr t))))
      (else t))))
(define (incr t) (mapbnd add1 t))

(define (red s bs js)                   ; β0: (λ^k s) applied to indices renames them
  (if (and (eq? (tag s) 'A) (pair? bs) (memq (tag (car bs)) '(B OUT)))
      (red (cadr s) (cdr bs) (cons (and (eq? (tag (car bs)) 'B) (cadr (car bs))) js))
      (apps (mapbnd (λ (i) (list-ref js i)) s) bs)))

(define (devar S s)                     ; walk the head through the triangular S
  (let ((ht (strip s)))
    (cond ((and (eq? (tag (car ht)) 'V) (assoc (cadr (car ht)) S))
           => (λ (p) (devar S (red (cdr p) (cdr ht) '()))))
          (else s))))

(define (occ F S t)
  (let occ ((t t))
    (case (tag t)
      ((V) (or (equal? F (cadr t))
               (cond ((assoc (cadr t) S) => (λ (p) (occ (cdr p)))) (else #f))))
      (($) (or (occ (cadr t)) (occ (caddr t))))
      ((A) (occ (cadr t)))
      (else #f))))

(define counter 0)
(define (newV)
  (set! counter (add1 counter)) (V (string->symbol (format "H~a" counter))))

(define (pos p xs)                      ; indices (from the right) of the xs with p
  (let loop ((xs xs))
    (cond ((null? xs) '())
          ((p (car xs)) (cons (B (length (cdr xs))) (loop (cdr xs))))
```

```
          (else (loop (cdr xs))))))
(define (idx bs b)                      ; the index of b in the binder list bs, or #f
  (let loop ((bs bs))
    (cond ((null? bs) #f)
          ((equal? (car bs) b) (length (cdr bs)))
          (else (loop (cdr bs))))))

(define (proj S s)                      ; scope check + pruning, one walk over s
  (let* ((ht (strip (devar S s))) (h (car ht)) (ss (cdr ht)))
    (case (tag h)
      ((A) (proj S (cadr h)))
      ((C B) (fold (λ (t S) (proj S t)) S ss))
      ((OUT) (fail))                    ; out-of-scope universal, rigid position
      ((V) (let ((keep (pos (λ (b) (eq? (tag b) 'B)) ss)))
             (cons (cons (cadr h) (hnf (length ss) (newV) keep)) S))))))

(define (flexflex1 F ym zn S)           ; same head: keep the positions that agree
  (if (equal? ym zn) S
      (let ((keep (pos (λ (p) (equal? (car p) (cdr p))) (map cons ym zn))))
        (cons (cons F (hnf (length ym) (newV) keep)) S))))
(define (lam im G jn) (hnf (length im) G (map (λ (j) (B (idx im j))) jn)))
(define (subset? xs ys) (every (λ (x) (member x ys)) xs))
(define (flexflex2 F im G jn S)         ; different heads
  (cond ((subset? im jn) (cons (cons G (lam jn (V F) im)) S))
        ((subset? jn im) (cons (cons F (lam im (V G) jn)) S))
        (else (let ((kl (filter (λ (i) (member i jn)) im)) (H (newV)))
                (cons (cons F (lam im H kl)) (cons (cons G (lam jn H kl)) S))))))
(define (flexflex F ym G zn S)
  (if (equal? F G) (flexflex1 F ym zn S) (flexflex2 F ym G zn S)))

(define (flexrigid F im t S)            ; occurs check, inversion, then pruning
  (if (occ F S t) (fail)
      (let ((u (mapbnd (λ (i) (idx im (B i))) t)))
        (proj (cons (cons F (abs (length im) u)) S) u))))

(define (unif S s t)
  (let ((s (devar S s)) (t (devar S t)))
    (cond ((and (eq? (tag s) 'A) (eq? (tag t) 'A)) (unif S (cadr s) (cadr t)))
          ((eq? (tag s) 'A) (unif S (cadr s) ($ (incr t) (B 0))))     ; η on demand
          ((eq? (tag t) 'A) (unif S ($ (incr s) (B 0)) (cadr t)))
          (else (cases S s t)))))
(define (cases S s t)
  (let ((hs (strip s)) (ht (strip t)))
    (cond ((and (eq? (tag (car hs)) 'V) (eq? (tag (car ht)) 'V))
           (flexflex (cadr (car hs)) (cdr hs) (cadr (car ht)) (cdr ht) S))
          ((eq? (tag (car hs)) 'V) (flexrigid (cadr (car hs)) (cdr hs) t S))
          ((eq? (tag (car ht)) 'V) (flexrigid (cadr (car ht)) (cdr ht) s S))
          ((and (equal? (car hs) (car ht)) (= (length (cdr hs)) (length (cdr ht))))
           (fold (λ (a b S) (unif S a b)) S (cdr hs) (cdr ht)))
          (else (fail)))))
```

The test driver, a printer that applies S and turns indices back into names (omitted here), and the problems are in `writers/ch07-hounif/nipkow.scm`. For instance, Example 7.23 is posed as

```
(define f (C 'f)) (define g (C 'g)) (define h (C 'h))
(define (lam3 t) (A (A (A t))))     ; ∀w ∀x ∀y as three binders: y = 0, x = 1, w = 2
;; Example 8.H: ∀f∀g∃U∃V∀w∀x∀y [(f (U x y)) = (f (g (V y w)))]
(run "8.H  f (U x y) = f (g (V y w))" '(U V)
     (lam3 ($ f ($ ($ (V 'U) (B 1)) (B 0))))
     (lam3 ($ f ($ g ($ ($ (V 'V) (B 0)) (B 2))))))
```

and the whole battery prints, with `H1`, `H2` the fresh variables and `V1` standing for the book's V':

```
8.H  f (U x y) = f (g (V y w))
  U := λx1. λx2. g (H1 x2)
  V := λx1. λx2. H1 x1
8.I  U x y = g (U y w)
  no unifier
8.J  U x y = g w
  no unifier
4.5  λx λy (G x) = λx λy (H y)
  G := λx1. H1
  H := λx1. H1
ff   λx λy (F x y) = λx λy (F y x)
  F := λx1. λx2. H1
8.K  U x = f (V1 x y z)
  U := λx1. f (H1 x1)
  V1 := λx1. λx2. λx3. H1 x1
eta  λx (F x) = g
  F := λx1. g x1
dup  λx (F x) = λx (g x x)
  F := λx1. g x1 x1
loc  F x = h (λz (G z w))
  F := λx1. h (λx2. H1 x2)
  G := λx1. λx2. H1 x1
two  F x = g (G x w) (G w x)
  F := λx1. g H2 H2
  G := λx1. λx2. H2
```

Every line agrees with the hand derivations: Examples 7.23 to 7.25 and 7.27 to 7.29, the same-head pitfall, and the eagerly raised form of Example 7.31 below.

Kanren angle: what first-order unification does with the same terms

(aux kanren micro) can already store de Bruijn terms as records, but its unifier treats an application of a logic variable as a pair of slots. This sometimes gives the right answer by luck, sometimes an ill-scoped one, and sometimes a wrong failure. With `(define-record lam body)`, `(define-record app rator rand)` and `(define-record dbi n)`:

```
(show "X = (f X)"
      (°->list #f (fresh° (q) (=° q (list 'f q)))))
(show "λx (F x) = λx (g x)"
      (°->list #f (fresh° (f) (=° (make-lam (make-app f (make-dbi 0)))
                                  (make-lam (make-app 'g (make-dbi 0)))))))
(show "λx (F x) = λx (g x x)"
      (°->list #f (fresh° (f) (=° (make-lam (make-app f (make-dbi 0)))
```

```
                                (make-lam (make-app (make-app 'g (make-dbi 0))
                                  ↪  (make-dbi 0)))))))
(show "λx (F x) = λx (g x 7)"
      (°->list #f (fresh° (f) (=° (make-lam (make-app f (make-dbi 0)))
                                (make-lam (make-app (make-app 'g (make-dbi 0))
                                  ↪  7))))))
```

```
;; X = (f X)
()
;; λx (F x) = λx (g x)
((λ () 'g))
;; λx (F x) = λx (g x x)
((λ () (make-record-instance 'app 'g (make-record-instance 'dbi 0))))
;; λx (F x) = λx (g x 7)
()
```

The occurs-check case is already right. The second answer is right up to η. The third binds F to "g applied to index 0" *outside* any binder: a dangling index, where the mgu is $F := \lambda y\,(g\,y\,y)$ (`dup` above). The fourth fails, where the mgu is $F := \lambda y\,(g\,y\,7)$. A higher-order extension must therefore recognise a flexible application and run Elim on it rather than decomposing it structurally; see Chapter 11.

7.9 Mixed prefixes, levels and delayed equations

Everything so far assumed a ∀∃∀ prefix. Real proof search does not deliver one. Each `pi x\ G` adds a universal, and each backchaining step adds existentials *after* it, giving prefixes such as ∀∃∀∃∀ … [MN p. 224]. Raising them eagerly "has to be performed dynamically", may raise existentials "over long lists of universally quantified variables" (typical of the `pi`-based recursion of Chapter 6), and "often can be redundant and may have to be undone by a pruning substitution at a later point" [MN p. 224].

Example 7.31 (Eager versus delayed raising [MN p. 224–225])**.** Consider

$$\forall f\,\exists U\,\forall x\,\forall y\,\forall z\,\exists V\,[(U\,x) = (f\,V)].$$

- *Eagerly*, raise V over x, y, z:

  $$\forall f\,\exists U\,\exists V'\,\forall x\,\forall y\,\forall z\,[(U\,x) = (f\,(V'\,x\,y\,z))].$$

  Variable elimination immediately prunes $V' := \lambda x\lambda y\lambda z\,(W\,x)$, giving $U := \lambda x\,(f\,(W\,x))$, so two of the three raised arguments were wasted work.

- *Delayed*: record that U may be raised over f and V over x, y, z, f. When solving, "noting the presence of x as an argument of the flexible term and the possibility of raising V over x, we would replace V with the term $(V'\,x)$, where V' is a new variable quantified at the same place in the prefix as U and would produce the unifying substitution $\lambda x\,(f\,(V'\,x))$ for U" [MN p. 225].

The lists of universals that each existential may depend on "are all initial segments of the sequence of variables that are universally quantified in the prefix, … and this information can be maintained and used efficiently by attaching numerical indices to the quantified variables" [MN p. 225]. This idea has two literature sources.

Universe indices for `pi` goals [43]. In a WAM-style interpreter for first-order hereditary Harrop formulas, "each time a universal quantifier is encountered, a new constant must be introduced, giving rise to the next universe in the hierarchy": the universe index is incremented and the new constant is tagged with it. A logic variable is tagged with the current index, meaning "that a term may be substituted for the variable only if all the constants appearing in the term have a smaller or identical tag" (p. 16). Binding $X^i := t$ then requires (1) the usual occurs-check; (2) that t contains no constant with tag greater than i; and (3) that, before binding, the tags of the variables of t greater than i be lowered to i, "to prevent a later instantiation of these variables from violating the restrictions on instantiations for X" (p. 16).

Example 7.32 (Why tags must be lowered [43, p. 16])**.** Program $\forall z\,(q(z) \supset p(d(z)))$, goal $\exists x\,\forall y\,(q(y) \supset p(x))$. After the quantifiers the goal is $q(c^2) \supset p(X^1)$. Backchaining on the clause with a new Z^2 unifies $p(d(Z^2))$ with $p(X^1)$; binding $X := d(Z)$ lowers Z to Z^1. The remaining goal $q(Z^1)$ must be matched against the hypothesis $q(c^2)$ and *fails*, because c has tag $2 > 1$. Without step (3), Z would be bound to c and X would end up as $d(c)$, mentioning a constant introduced after X.

Prefix erasure for patterns [44]. Nadathur and Linnell tag every variable y with $l(y)$, "the number of times an existential quantifier is immediately followed by a universal one in the prefix up to and including the quantifier binding that occurrence". Then "a universal variable u is quantified within the scope of the quantifier for an existential variable x just in case $l(x) < l(u)$" (p. 8), so the pattern test becomes local. Raising is restricted to what is needed: for $f(\bar{y}) = g(\bar{z})$ with $l(f) \le l(g)$, g needs to be raised only over $\bar{y}{\Uparrow}g = \{u \in \bar{y} \mid l(u) \le l(g)\}$, the universals among f's arguments that g may depend on. A combined rule (9′) binds f and g to a common new h with $l(h) = l(f)$, applied first to those universals and then to the common arguments (p. 9). Their Theorem 2 states that prefix erasure preserves termination and mgus (p. 10), and their Figure 2 gives the single-traversal `mksubst/bnd` that computes the binding, the pruning and the raising together (p. 11). The algorithm is used in Bedwyr, Abella and Teyjus 2 [MN p. 228].

Variable elimination with levels. Combining the two, Definition 7.21 generalises to tagged variables. This is our reconstruction, checked by hand on Examples 7.23 and 7.31; Nadathur and Linnell's Figure 2 is the authoritative version. Write $l(\cdot)$ for levels, with signature constants at the lowest level. To eliminate F in $(F\,\bar{c}) = t$, where pattern means that the c_i are distinct universals with $l(c_i) > l(F)$:

- a universal u in rigid position is kept if $u \in \bar{c} \cup L$ or $l(u) \le l(F)$ (since F may mention it directly), and fails otherwise;
- a flexible pattern $(G\,\bar{d})$ keeps the $d_j \in \bar{c} \cup L$ and the d_j with $l(d_j) \le l(F)$, prunes the rest, and, if $l(G) > l(F)$, is *also raised* over the c_i with $l(c_i) \le l(G)$ that it does not already

receive; the new G' gets level $l(F)$. In Example 7.31, $l(U) = 0$ and $l(x) = l(V) = 1$, so V (no arguments) is raised over x: $V := (V'\ x)$ with $l(V') = 0$, and $U := \lambda x\,(f\ (V'\ x))$, as in the book;

- when no c_i qualifies, raising degenerates to *lowering*: $G := \lambda \bar{d}.\,(G'\ \bar{e})$ with $l(G') = l(F)$ and nothing added, which is step (3) of the universe-index discipline above.

De Bruijn levels in ELPI [9]. ELPI makes the two notions of level coincide. "When a clause's head is unified with the goal in a context of length n, the universally quantified variables of the clause are instantiated to unification variables X^n where the level n records that X has only visibility of the initial prefix of length n of the context. If later a fresh constant is added by the `pi` rule, the constant occupies position $n+1$ ... and it will not be allowed to occur in instances of the variable X^n" (p. 4). Lowering a level is done by pruning: "unifying B_0 with $(arr\ C_1\ D_1)$ prunes C and D to level 0" (p. 7), i.e. the higher-level variable is bound to a fresh lower-level one.

Kanren angle: levels and (aux kanren micro)

(aux kanren micro) already allocates variables from a monotone counter, `vars-count`, so creation order is a linearisation of the dynamically built prefix. If eigenvariables were allocated from the same counter, the creation index would give every variable its initial level for free. Three facts of the current engine matter for any level discipline.

1. *Level is not index.* Pruning and raising create a *new* variable "quantified at the same place" as an *old* one [MN p. 225], so a variable's level must be stored separately from its index.
2. *Orientation of var-var bindings.* Today the older variable is bound to the newer one. With scopes, binding a low-level X to a high-level Y lets a later binding of Y smuggle a young eigenvariable into X. Either the orientation must follow levels, or the binding must lower Y, as in step (3) above.
3. *Unification allocates.* Pruning, raising and the flexible-flexible H create variables, so the unifier must return an updated `vars-count`, and the bindings it reports must include the prunings of *other* variables (item 3 of the pitfall after Definition 7.21), because constraint re-verification keys on them.

How these requirements are met is the subject of Chapter 11.

7.9.1 Dynamic L_λ: delay what is not a pattern

A static L_λ restriction "can rule out some useful programming idioms" [MN p. 225], the standard one being substitution by meta-level β-reduction:

```
type   subst   (tm -> tm) -> tm -> tm -> o.
subst R N (R N).
```

Here `R` and `N` are essentially existential, so `(R N)` is not a pattern. "A common practical resolution to this dilemma is to not impose the L_λ restriction on program clauses statically but

rather to expect the condition to be satisfied by the unification problems that are encountered dynamically" [MN p. 225–226]: when `subst` is called, `R` and `N` are expected to be instantiated, and `(R N)` is then just a β-redex to reduce. More generally: "we would treat any unification problem as if it is a higher-order pattern-unification one. In the course of such a treatment, we might discover that a particular equation does not respect the L_λ condition. In this case, we might defer further consideration of this equation until such time that substitutions resulting from solving other parts of the problem have altered its status" [MN p. 226].

The literature agrees on this design.

- Dowek, Hardin, Kirchner and Pfenning [8], §5.4 (p. 13): equations outside the pattern fragment "should be postponed as constraints in the hope that they will be instantiated further", which "avoids all branching during constraint simplification". Their motivation is empirical [33]: the static restriction is too severe, but "almost all dynamically arising equations fall within the pattern fragment" (p. 2). Their algorithm phrases Elim as the inversion of a pattern substitution: $X[s] = b$ is solved by $X = b[s^{-1}]$ when every index of b occurs in s (Prop. 8), with pruning computed together with the inverse (p. 11–12).
- Nadathur and Linnell [44], §6: a non-pattern subterm met inside `bnd` can be replaced by a fresh flexible term, with the equation between them *deferred*.
- Systems: Teyjus 1 [46] implemented (pre-)unification for unrestricted terms, while Teyjus 2 "only implements L_λ unification"; Twelf and Minlog also implement only L_λ unification, over an unrestricted term language [MN p. 228]. ELPI's default for a problem outside the fragment is an error, with a deprecated option to delay it "for Teyjus compatibility"; its supported mechanism for suspension is instead a general syntactic constraint on goals, resumed when a variable is assigned.

Going the other way, `subst` can be written inside L_λ by making the object signature explicit with `copy` clauses (Chapter 6), and by extension "it also may be possible to describe general higher-order unification as a logic program in the L_λ language" [MN p. 226] [34].

Pitfall: unifier versus solution; delayed is not solved

A state with suspended equations has succeeded only *conditionally*: its answer is a pre-unifier together with residual constraints, and those must be printed with it. The canonical substitution of Proposition 7.12 would make flexible-flexible residues "true", but it introduces new existentials H^τ: it is a unifier, not a solution, and in a typed setting with an empty type there may be no closed solution at all [MN p. 211]. Untyped Scheme terms avoid that last problem [34, §9.3], but not the first.

Kanren angle: what delaying asks of a miniKanren

Following the book's "defer further consideration" [MN p. 226], an (aux kanren micro)-style engine needs: a place in the state for suspended equations; a wake-up step after each successful unification that retries the suspended equations mentioning a variable just bound (the same "touched variables" mechanism that already re-verifies disequal-

ities); and reification that prints what is still suspended. Soft-cut, projection and disequality must be audited, because each of them can observe a state whose success is only conditional. Disequality needs care even without delays: the mgu of a pattern problem may contain *new* variables (pruning, the flexible-flexible H), and in $\neg\exists H.(X = \lambda x\, H)$ these are existentially quantified inside the negation (equivalently, universally outside it), which a store of plain bindings would get wrong (Definition 12.20). The concrete design is in Chapter 11.

7.10 Summary

- A unification problem is a quantified formula; its prefix decides who may depend on whom. The book normalises to a $\forall\exists\forall$ prefix by raising, to β-normal terms, to equal binders by η, and turns top binders into inner universals [MN p. 212].
- General higher-order unification has no mgus, may need infinite complete sets of unifiers, and is undecidable [MN p. 213–214].
- Huet's procedure: Dec/Clash on rigid-rigid pairs, imitation (scope-sensitive) or projection (type-sensitive) on flexible-rigid ones, success when only flexible-flexible equations remain (pre-unification; the canonical substitution unifies them). Its matching tree is complete but may have infinite branches, with or without answers [MN p. 214–220].
- In L_λ, flexible terms are patterns $F\,\bar{c}$ with distinct universals bound inside F's scope. Only β_0 arises, at most one imitation or projection is useful, flexible-flexible pairs have explicit mgus (common arguments for different heads, agreeing positions for the same head), and with an occurs-check the procedure terminates: pattern unification is decidable and unitary [MN p. 220–224], [34].
- Variable elimination in one walk: keep rigid heads, map each c_i to F's i-th bound variable, keep local binders, prune the arguments of other flexible terms that F cannot see, and fail on F itself (occurs) or on an unseeable universal in rigid position (scope). Then $F := \lambda\bar{c}.\,t'$ [MN p. 223].
- Nipkow's de Bruijn version is short (our Scheme transliteration of its core is under a hundred lines); loose indices are the inner universals, `devar` is β_0 by renaming, and inversion uses a marker that pruning removes [48]. No types are needed [MN p. 224].
- In practice prefixes are mixed and dynamic: replace raising by levels on variables and eigenvariables, lower levels when binding, raise lazily only over the arguments needed [MN p. 225], [43, 44, 9].
- Dynamic L_λ: treat every equation as a pattern problem and *delay* the non-patterns until bindings make them patterns; report what remains as residual constraints [MN p. 226], [8].

- For (aux kanren micro): first-order unification is the $n = 0$ case; the extension needs eigenvariables, binders, flexible applications, levels, allocation inside unification, and a store of delayed equations; see Chapter 11.

8 Applications: provers, functional programs, processes

The previous chapters assembled three tools: λ-tree syntax, which represents object-level binders by meta-level abstractions (Chapter 6); the universal and implicational goals of hereditary Harrop formulas, which give fresh constants and scoped hypotheses (Chapter 3); and the unification of λ-terms, decidable and most-general in the pattern fragment L_λ (Chapter 7). Chapters 9–11 of the book [36] put the three to work on three families of meta-programs: *proof systems* (intuitionistic and classical provers, proof checkers, tactics), *functional programs* (type inference, evaluators, partial evaluation, CPS conversion) and *processes* (the π-calculus, its transitions, traces, and an encoding of the λ-calculus).

All three follow the same recipe, and it is the main idea of this chapter:

1. a judgment of the object system becomes a predicate, and each inference rule becomes a clause (the book calls this a "reflection of provability relations directly into predicates", [MN p. 240]);
2. object-level binders are meta-level λs, so object-level substitution is β-reduction;
3. to go under a binder, the program introduces a fresh constant with `pi x\ ...` and, usually, a hypothetical fact about it with `=>`.

The side conditions that clutter the usual presentations ("y fresh", "y not free in the conclusion", "$w \notin fn(P)$") then disappear: the nesting of quantifiers already enforces them. For each application we ask three questions, since they decide what a miniKanren needs: which binding facilities does it use ($\forall$, $\supset$, pattern unification, more than patterns)? Which part runs on today's (aux kanren micro) with a first-order encoding? What changes when depth-first search is replaced by miniKanren's interleaving search? Four small ports below were actually run on (aux kanren micro). Their outputs are reproduced verbatim.

8.1 Proof systems as logic programs

8.1.1 Intuitionistic propositional logic: from a loop to a decision procedure

Formulas are built from two constants and three connectives [MN p. 229–230]:

```
kind  term, form    type.             % types for terms and formulas
type  ff,                             % encoding the false proposition
      tt            form.             % encoding the true proposition
type  &&,                             % encoding conjunction
      !!,                             % encoding disjunction
      ==>           form -> form -> form.   % encoding implication
infixl &&  5.
```

```
infixl !!  4.
infixr ==> 3.
```

A sequent $\Gamma \longrightarrow G$ has a multiset Γ of hypotheses and one conclusion. The book's starting point is the following calculus [MN p. 230]. (In the printed figure the axiom for $\top$ is mislabelled $\bot L$.)

$$\frac{}{\Gamma, A \longrightarrow A}\ \textsc{init}\ (A \text{ atomic}) \qquad \frac{}{\Gamma, \bot \longrightarrow G}\ \bot L \qquad \frac{}{\Gamma \longrightarrow \top}\ \top R \qquad \frac{\Gamma, A, B \longrightarrow G}{\Gamma, A \wedge B \longrightarrow G}\ \wedge L$$

$$\frac{\Gamma \longrightarrow A \qquad \Gamma \longrightarrow B}{\Gamma \longrightarrow A \wedge B}\ \wedge R \qquad \frac{\Gamma, A \longrightarrow G \qquad \Gamma, B \longrightarrow G}{\Gamma, A \vee B \longrightarrow G}\ \vee L \qquad \frac{\Gamma \longrightarrow A_i}{\Gamma \longrightarrow A_1 \vee A_2}\ \vee R_i$$

$$\frac{\Gamma, A \supset B \longrightarrow A \qquad \Gamma, B \longrightarrow G}{\Gamma, A \supset B \longrightarrow G}\ \supset L \qquad \frac{\Gamma, A \longrightarrow B}{\Gamma \longrightarrow A \supset B}\ \supset R$$

Why the direct translation loops. Each rule, read from conclusion to premises, removes a connective, except $\supset L$, whose left premise still contains the principal formula $A \supset B$. The instance

$$\frac{p \supset q \longrightarrow q \qquad q \longrightarrow q}{p \supset q \longrightarrow q}\ \supset L$$

has a left premise identical to its conclusion. A depth-first interpreter that tries this premise first recurses forever. The book: "Depth-first search usually steers the interpreter into endless loops when trying to work backwards from a given sequent" [MN p. 230]. Soundness of such direct translations is easy. Completeness of their *execution* is not.

The cure: a better calculus, not a better interpreter. Replace $\supset L$ by six rules that look at the shape of the antecedent [10, 23] ([MN p. 231]). (The text says "four rules". The figure shows six.)

$$\frac{\Gamma, A, B \longrightarrow G}{\Gamma, A, A \supset B \longrightarrow G}\ \supset L_1\ (A \text{ atomic}) \qquad \frac{\Gamma, C \supset D \supset B \longrightarrow G}{\Gamma, (C \wedge D) \supset B \longrightarrow G}\ \supset L_2 \qquad \frac{\Gamma, C \supset B, D \supset B \longrightarrow G}{\Gamma, (C \vee D) \supset B \longrightarrow G}\ \supset L_3$$

$$\frac{\Gamma, D \supset B \longrightarrow C \supset D \qquad \Gamma, B \longrightarrow G}{\Gamma, (C \supset D) \supset B \longrightarrow G}\ \supset L_4 \qquad \frac{\Gamma \longrightarrow G}{\Gamma, \bot \supset B \longrightarrow G}\ \supset L_5 \qquad \frac{\Gamma, B \longrightarrow G}{\Gamma, \top \supset B \longrightarrow G}\ \supset L_6$$

Theorem 8.1 (Hudelmaier, Dyckhoff; [MN p. 230–231])**.** *The calculus obtained by replacing $\supset L$ with $\supset L_1$–$\supset L_6$ is sound and complete for this fragment of propositional intuitionistic logic, and every sequence of sequents produced by backward proof search is finite. A weight can be assigned to connectives and sequents so that every conclusion weighs more than each of its premises. Consequently the direct translation of the calculus into λProlog "actually constitutes a decision procedure"* [MN p. 231].

The book does not give the weight function. Provability in this logic is PSPACE-complete (Statman, cited on [MN p. 245]). Here is the program [MN p. 232]:

```
type  atom     form -> o.
type  seq      list form -> form -> o.

seq Gamma A :- atom A, memb_and_rest A Gamma _.
seq Gamma tt.
seq Gamma (A && B)  :- seq Gamma A, seq Gamma B.
seq Gamma (A !! B)  &
seq Gamma (B !! A)  :- seq Gamma A.
seq Gamma (A ==> B) :- seq (A::Gamma) B.
seq Gamma _ :- memb_and_rest ff Gamma _.
seq Gamma G :- memb_and_rest (A && B) Gamma Gamma',
               seq (A::B::Gamma') G.
seq Gamma G :- memb_and_rest (A !! B) Gamma Gamma',
               (seq (A::Gamma') G; seq (B::Gamma') G).
seq Gamma G :- memb_and_rest (A ==> B) Gamma Gamma',
  ( atom A, memb_and_rest A Gamma' _, seq (B::Gamma') G;
    A = (C && D),  seq ((C ==> D ==> B)::Gamma') G;
    A = (C !! D),  seq ((C ==> B)::(D ==> B)::Gamma') G;
    A = (C ==> D), seq ((D ==> B)::Gamma') A,
                   seq (B::Gamma') G;
    A = ff,        seq Gamma' G;
    A = tt,        seq (B::Gamma') G  ).
```

The relation `memb_and_rest X L R` (from [MN p. 58]) holds when `L` contains `X` and `R` is `L` with that occurrence removed. It plays the role of multiset selection. The head `H1 & H2 :- B` is two clauses sharing a body, so one line gives both $\vee R$ rules. The last clause is $\supset L_1$–$\supset L_6$ written as a single six-way disjunction on the shape of `A`. The predicate `atom` is deliberately left open: "This predicate can be defined simply by adding clauses to the program that enumerate all the propositional letters" [MN p. 231]. The book proves the formula $a \supset (a \supset b) \supset (a \supset b \supset c) \supset c$ by introducing the letters locally [MN p. 231] (the printed formula has an unbalanced parenthesis):

```
?- pi a\ pi b\ pi c\ atom a => atom b => atom c =>
              seq nil (a ==> (a ==> b) ==> (a ==> b ==> c) ==> c).
```

The query "achieves the same effect by introducing three new constants and assuming that each of them satisfies the atom predicate" [MN p. 231]. It succeeds. It is a compact showcase of `pi` (fresh constants) together with `=>` (local clauses).

Tutorial: the proof the program finds

Three $\supset R$ steps give the context $[a \supset b \supset c,\ a \supset b,\ a]$ (most recent first, since $\supset R$ conses) and goal c. The atom c is not in the context. `memb_and_rest (A ==> B)` first selects $a \supset (b \supset c)$. Since a is present, $\supset L_1$ replaces it by $b \supset c$. On this implication $\supset L_1$ fails because b is not yet present, so the search backtracks to $a \supset b$, and $\supset L_1$ adds b. Now $\supset L_1$ on $b \supset c$ adds c, and INIT closes the branch. No rule ever puts back the formula it

consumed. This is the decreasing weight of Theorem 8.1 at work.

Pitfall: a typo that makes the prover unsound

The $\vee L$ clause printed in the figure reads `(seq (A::Gamma') G; seq (B::Gamma') G)`. It succeeds if *either* branch succeeds, but $\vee L$ needs *both* premises. With `;` the sequent $a \vee b \longrightarrow a$ is "proved". The body must be the conjunction `seq (A::Gamma') G, seq (B::Gamma') G`. The page image confirms the semicolon is really there. The run below shows both versions.

Kanren angle: the decision procedure on today's (aux kanren micro)

Nothing in this program needs binders, so it ports to (aux kanren micro) directly. Formulas become tagged lists, atoms become Scheme symbols, and `atom` becomes the type constraint `symbol°`. The constraint does not enumerate letters, and it stays pending if its argument is unbound. Because `tt` and `ff` are symbols too, they are excluded with `≠°`. The parameter `both` chooses how the two $\vee L$ premises combine: `and²` gives the correct rule, `or²` the printed typo.

```
(define-relation (atom° a) (symbol° a) (≠° a 'tt) (≠° a 'ff))

(define-relation (memb-and-rest° x l rest)
  (cond° ((fresh° (d) (=° l (cons x d)) (=° rest d)))
         ((fresh° (y d r) (=° l (cons y d)) (=° rest (cons y r)) (memb-and-rest° x d
         ↪  r)))))

;; Figure 9.3, with the vL clause written as a conjunction; `both` combines the two vL
↪  premises
(define ((seq°/with both) Γ G)
  (define seq° (seq°/with both))
  (λ (s)
    (δ ((cond°
          ((atom° G) (fresh° (r) (memb-and-rest° G Γ r)))
          ((=° G 'tt))
          ((fresh° (A B) (=° G `(&& ,A ,B)) (seq° Γ A) (seq° Γ B)))
          ((fresh° (A B) (=° G `(!! ,A ,B)) (or° (seq° Γ A) (seq° Γ B))))
          ((fresh° (A B) (=° G `(==> ,A ,B)) (seq° `(,A . ,Γ) B)))
          ((fresh° (r) (memb-and-rest° 'ff Γ r)))
          ((fresh° (A B r) (memb-and-rest° `(&& ,A ,B) Γ r) (seq° `(,A ,B . ,r) G)))
          ((fresh° (A B r) (memb-and-rest° `(!! ,A ,B) Γ r)
             (both (seq° `(,A . ,r) G) (seq° `(,B . ,r) G))))
          ((fresh° (A B r) (memb-and-rest° `(==> ,A ,B) Γ r)
             (cond°
               ((atom° A) (fresh° (r*) (memb-and-rest° A r r*)) (seq° `(,B . ,r) G))
               ((fresh° (C D) (=° A `(&& ,C ,D)) (seq° `((==> ,C (==> ,D ,B)) . ,r) G)))
               ((fresh° (C D) (=° A `(!! ,C ,D)) (seq° `((==> ,C ,B) (==> ,D ,B) . ,r)
               ↪  G)))
               ((fresh° (C D) (=° A `(==> ,C ,D)) (seq° `((==> ,D ,B) . ,r) A) (seq°
               ↪  `(,B . ,r) G)))
               ((=° A 'ff) (seq° r G))
```

```
              ((=° A 'tt) (seq° `(,B . ,r) G))))))
       s))))

(define seq° (seq°/with and²))          ; the correct vL: both premises
(define seq-typo° (seq°/with or²))      ; the printed vL: either premise

(define (proofs g) (length (μkanren-run (q -1 #t) g (=° q 'yes))))

(define f1 '(==> a (==> (==> a b) (==> (==> a (==> b c)) c))))   ; the formula of p.247
(define em '(!! a (==> a ff)))                                    ; excluded middle
(define peirce '(==> (==> (==> a b) a) a))                        ; Peirce's formula

(pretty-print
  (list (list 'f1      (μkanren-run (q 1 #t) (seq° '() f1) (=° q 'yes)) (proofs (seq°
  ↪  '() f1)))
        (list 'or-left (proofs (seq° '((!! a b)) 'a)) (proofs (seq-typo° '((!! a b))
        ↪  'a)))
        (list 'or-comm (proofs (seq° '() '(==> (!! a b) (!! b a)))))
        (list 'em      (proofs (seq° '() em)))
        (list 'peirce  (proofs (seq° '() peirce)))))
```

```
((f1 (yes) 3) (or-left 0 1) (or-comm 1) (em 0) (peirce 0))
```

The body of `seq°/with` is wrapped in `(λ (s) (δ ...))`, exactly as `define-relation` expands, so that building the recursive goal does not loop. Every `run*` above terminates, as Theorem 8.1 promises for a ground query. For the formula of [MN p. 231] the search finds three derivations. $a \vee b \longrightarrow a$ has none with the correct rule and one with the typo. Excluded middle and Peirce's formula, which are classically valid but not intuitionistically (Chapter 3), get the answer *no*.

Kanren angle: what interleaving changes for the looping calculus

Interleaving does not make the naive calculus terminate, but it does change which queries succeed. Take only the atomic, $\supset R$ and the original $\supset L$ rule:

```
(define-relation (naive° Γ G)
  (cond°
    ((symbol° G) (fresh° (r) (memb-and-rest° G Γ r)))
    ((fresh° (A B) (=° G `(==> ,A ,B)) (naive° `(,A . ,Γ) B)))
    ((fresh° (A B r) (memb-and-rest° `(==> ,A ,B) Γ r) (naive° Γ A) (naive° `(,B . ,r)
    ↪  G)))))

(pretty-print (μkanren-run (q 1 #t) (naive° '((==> p q) (==> q r) q) 'r) (=° q 'yes)))
(flush-output)
(pretty-print (μkanren-run (q 1 #t) (naive° '((==> p q)) 'q) (=° q 'yes)))
```

```
(yes)
```

The first query is a theorem, and `run 1` answers immediately. A depth-first reading of the same clauses, in clause order, would select $p \supset q$ first and then loop on its left premise

... $\longrightarrow p$, which reproduces itself. Interleaving gives the branch through $q \supset r$ its turn. The second query is the book's looping instance $p \supset q \longrightarrow q$, which is not a theorem. Here the stream is infinite and empty: the run was killed after 30 seconds with no output. In short, interleaving makes the search *complete for answers*, while termination still has to come from the calculus.

8.1.2 Natural deduction with proof terms

For first-order intuitionistic logic, add the quantifiers `type all, some (term -> form) -> form.` of Chapter 6. Natural deduction has *hypothetical* premises (in $\supset I$, $\vee E$, $\exists E$ a derivation may start from a discharged assumption) and *eigenvariable* conditions (for $\forall I$ and $\exists E$, "This variable must not be free in the conclusion of the rule or in any assumption that remains undischarged", [MN p. 232]). Four representative rules out of the fourteen [MN p. 233]:

$$\frac{\begin{array}{c}[A]\\ \vdots \\ B\end{array}}{A \supset B}\ \supset I \qquad \frac{A \qquad A \supset B}{B}\ \supset E \qquad \frac{A[y/x]}{\forall x\, A}\ \forall I\ (y \text{ fresh}) \qquad \frac{\forall x\, A}{A[t/x]}\ \forall E$$

The book records whole derivations as λ-terms of a type `proof`, with one constructor per rule [MN p. 233]. An excerpt of the fourteen declarations:

```
type imp_i          (proof -> proof) -> proof.
type imp_e          form -> proof -> proof -> proof.
type all_e          term  -> (term -> form) -> proof -> proof.
type all_i          (term -> proof) -> proof.
```

Three design choices are visible in the types. A rule that *discharges* a hypothesis takes an abstraction over proofs (`imp_i (proof -> proof)`), and the bound proof variable *is* the hypothesis label. A rule with an *eigenvariable* abstracts over terms (`all_i (term -> proof)`). *Elimination* rules store the formulas that appear in a premise but not in the conclusion, such as the antecedent in `imp_e A`, which makes checking syntax-directed. The checking relation `P # A`, "P is a proof of A", follows Felty [12] ([MN p. 234]):

```
type  #     proof -> form -> o.
infix # 2.

true_i # tt.
(and_i P1 P2) # (A && B) :- (P1 # A), (P2 # B).
(or_i1 P) # (A !! B)  :- P # A.
(or_i2 P) # (A !! B)  :- P # B.
(imp_i Q) # (A ==> B) :- pi p\ (p # A) => ((Q p) # B).
(some_i T P) # (some A)  :- P # (A T).
(all_i Q) # (all A)   :- pi y\ (Q y) # (A y).
```

```
(false_e A P) # A     :- P # ff.
(and_e1 B P) # A      :- P # (A && B).
(and_e2 A P) # B      :- P # (A && B).
(or_e A B P Q1 Q2) # C
   :- (P # (A !! B)),
      (pi p1\ (p1 # A) => ((Q1 p1) # C)),
      (pi p2\ (p2 # B) => ((Q2 p2) # C)).
(imp_e A P1 P2) # B   :- (P1 # A), (P2 # (A ==> B)).
(some_e A P1 Q) # B
   :- (P1 # (some A)),
      pi y\ pi p\ (p # (A y)) => ((Q y p) # B).
(all_e T A P) # (A T) :- (P # (all A)).
```

Tutorial: reading the $\supset I$ clause

To check `imp_i Q` against $A \supset B$, the clause picks a fresh proof constant `p` (`pi p\`), assumes the atomic fact `p # A` (`=>`), and checks `Q p` against B. Since `Q` has type `proof -> proof`, `Q p` β-reduces and puts the hypothesis label in place. Discharge is automatic, because the assumption `p # A` disappears when the goal `(Q p) # B` has been proved. In the `all_i` clause, `pi y\` produces a constant that occurs nowhere else, which is exactly the eigenvariable condition. In the book's words, "implicational goals are used to capture the hypothetical judgments ...and universal goals are used to introduce and subsequently abstract over the proofs of the assumed formulas" and "to enforce the newness constraint" [MN p. 234].

The same clauses can be run in four modes. Reading them is the best way to see what "relational" means for a meta-program. The vocabulary is `a, b : form`, `q : term -> form`, `c : term`, `f : term -> term` [MN p. 234].

1. Checking. All three queries are solvable [MN p. 234]:

```
?- (imp_i w\w) # (a ==> a).
?- (imp_i x\ imp_i y\ imp_e a x y) # (a ==> ((a ==> b) ==> b)).
?- (imp_i P\ all_i y\ imp_i Q\
      (imp_e (q (f y))
        (imp_e (q y) Q (all_e y (x\ (q x) ==> (q (f x))) P))
        (all_e (f y) (x\ (q x) ==> (q (f x))) P)))
   # ((all x\ (q x) ==> (q (f x))) ==>
      (all x\ (q x) ==> (q (f (f x))))).
```

The third proof term reads, from the outside in: assume `P` : $\forall x(q\,x \supset q(f\,x))$; for a fresh `y` assume `Q` : $q\,y$; `all_e y _ P` proves $q\,y \supset q(f\,y)$, so the inner `imp_e (q y) Q (...)` proves $q(f\,y)$; `all_e (f y) _ P` proves $q(f\,y) \supset q(f(f\,y))$, and the outer `imp_e (q (f y)) ...` concludes $q(f(f\,y))$. Note that the instantiation bodies `x\ (q x) ==> (q (f x))` are spelled out inside both `all_e` nodes: this is the information an elimination rule stores so that checking stays syntax-directed.

2. Inference. With the formula unknown, the checker computes it [MN p. 235]:

```
?- (imp_i w\ (and_i (and_e2 a w) (and_e1 b w))) # R.
R = a && b ==> b && a.
```

3. Completion. "Sometimes proofs can be incompletely described by including free variables in them" [MN p. 235]. Here the two instantiation bodies of the third checking query are replaced by logic variables `A` and `A'` (of type `term -> form`), and the formula by `B`. (Here `P` and `Q` are λ-bound variables, not logic variables, despite the capitals.)

```
?- (imp_i P\ all_i y\ imp_i Q\
      (imp_e (q (f y)) (imp_e (q y) Q (all_e y A P))
             (all_e (f y) A' P))) # B.

A = w\ p w ==> p (f w)
A' = w\ p w ==> p (f w)
B = all (w\ p w ==> p (f w)) ==> all (w\ p w ==> p (f (f w)))
```

(The printed answer says `p` where the query says `q`. Read `q`.) The two unknowns are found in different ways. `A` is found by a pattern equation: `(all_e y A P) # (q y ==> q (f y))` yields `A y = q y ==> q (f y)` with `y` a `pi`-constant, which is in L_λ and has the unique solution $\lambda w.\, q\, w \supset q\,(f\, w)$. `A'` appears in `A' (f y) = ...`, which is *not* a pattern. It is fixed only because `P # (all A')` must match the stored hypothesis about `P`, forcing `A' = A`. A Teyjus-style implementation delays the non-pattern equation until then (Chapter 7).

4. Proving fails. `?- P # (a ==> ((a ==> b) ==> b)).` loops, although the second checking query above contains its proof [MN p. 235]. After two `imp_i` steps the goal `X # b` matches no hypothesis. The elimination clauses then accept any formula as conclusion: `(false_e A P) # A` asks for `P # ff`, which again only `false_e` matches, and so on forever. Elimination rules used backwards have unconstrained premises.

Pitfall: mode matters

One specification serves as checker, inferencer and completer, and fails as a prover. The logical reading is the same in all four modes. The operational behaviour is not. When a specification is "relational", it is worth asking for each argument whether it is given, partially given, or unknown.

Kanren angle: what natural deduction needs

∀. Fresh proof labels (`pi p\`) and eigenvariables (`pi y\`) are needed, the second *with a scope check*: a logic variable created before `y` must never be bound to a term containing `y`. ⊃. Every hypothesis is an *atomic* fact (`p # A`). An explicit context argument with a lookup clause, the standard miniKanren technique for type environments, is therefore enough. **Patterns.** Checking and inference work with first-order named or locally nameless binders plus a substitution relation. Completion with an unknown `A` in `all_e` needs pattern unification (`A y = t` means "abstract `y` out of `t`"), and `A' (f y)` needs post-

ponement. **Search.** Under interleaving the prover mode becomes complete in principle: the `false_e` branch is only one of several fair branches. The search space of the unconstrained elimination rules remains huge, however. The sources do not measure it. See Chapter 11 for how the proposal provides these features.

8.1.3 A classical prover with bounded contraction

For classical first-order logic, the book uses formulas in *negation normal form* (no $\supset$, negation only on atoms) and a one-sided calculus CL with four *zones* [MN p. 236]:

$$\Sigma \;:\; \mathscr{L};\; \Gamma;\; \Phi$$

Here Σ holds the eigenvariables, $\mathscr{L}$ is a *set* of literals, Γ is a *list* of formulas still to be introduced, and Φ is a *list* of existential formulas. The rules are (with A atomic, t a Σ-term, $y \notin \Sigma$):

$$\frac{\Sigma \;:\; A, \mathscr{L};\; \Gamma;\; \Phi}{\Sigma \;:\; \mathscr{L};\; A, \Gamma;\; \Phi}\;\textsc{lit} \qquad \frac{\Sigma \;:\; \neg A, \mathscr{L};\; \Gamma;\; \Phi}{\Sigma \;:\; \mathscr{L};\; \neg A, \Gamma;\; \Phi}\;\textsc{lit} \qquad \frac{\Sigma \;:\; \mathscr{L};\; \Gamma;\; \Phi, \exists x.B}{\Sigma \;:\; \mathscr{L};\; \exists x.B, \Gamma;\; \Phi}\;\exists$$

$$\frac{}{\Sigma \;:\; \mathscr{L};\; \top, \Gamma;\; \Phi}\;\top R \qquad \frac{\Sigma \;:\; \mathscr{L};\; B, \Gamma;\; \Phi \qquad \Sigma \;:\; \mathscr{L};\; C, \Gamma;\; \Phi}{\Sigma \;:\; \mathscr{L};\; B \wedge C, \Gamma;\; \Phi}\;\wedge R \qquad \frac{\Sigma \;:\; \mathscr{L};\; B, C, \Gamma;\; \Phi}{\Sigma \;:\; \mathscr{L};\; B \vee C, \Gamma;\; \Phi}\;\vee R$$

$$\frac{\Sigma \cup \{y\} \;:\; \mathscr{L};\; B[y/x], \Gamma;\; \Phi}{\Sigma \;:\; \mathscr{L};\; \forall x\, B, \Gamma;\; \Phi}\;\forall R \qquad \frac{}{\Sigma \;:\; A, \neg A, \mathscr{L};\; \cdot;\; \Phi}\;\textsc{init} \qquad \frac{\Sigma \;:\; \mathscr{L};\; B[t/x];\; \Phi, \exists x\, B}{\Sigma \;:\; \mathscr{L};\; \cdot;\; \exists x\, B, \Phi}\;\exists R$$

The calculus is a systematic tableau in the sense of Smullyan [54]. A non-empty Γ determines the rule completely. When Γ is empty, the only choices are INIT versus $\exists R$, and the witness t. The rule $\exists R$ puts $\exists x\, B$ back at the *end* of Φ, so all existentials get their turn. This alone does not give termination: for $\cdot \;:\; \cdot;\; \exists x.q(x);\; \cdot$, "proof search …leads to the attempt to prove sequents of the form $\cdot \;:\; q(t_1), \ldots, q(t_n); \cdot; \exists x.q(x)$, for increasingly larger sets of terms" [MN p. 237]. The remedy is to bound the number of instances of each existential and to retry with a larger bound (iterative deepening).

Pitfall: no rule for $\bot$

"If the first formula in the introduction zone is $\bot$ …that sequent is not provable" [MN p. 236]. Hence $\bot \vee \top$ is not provable as written. In a one-sided classical calculus $\bot$ should simply be dropped from Γ. A port should add `prv (ff::Gamma) Phi :- prv Gamma Phi.` (This is our notes' analysis. The book does not comment.)

Literals are `p A` and `n A` for an atom `A : atm`. The existential zone is a queue of pairs (formula, remaining uses), implemented as a *functional difference list* [MN p. 238]: a list is represented by a function from its tail to the whole list, so the empty queue is `que x\x` and `qpush X (que k\ Phi k) (que k\ Phi ((pr X N)::k)) :- bound N.` appends at the end by β-reduction. `qpop` takes

the front pair and, if uses remain, re-appends it with one use fewer. All these matches are L_λ patterns. Popping the empty queue fails because `x\x` and `k\ (pr X 1)::(Phi k)` clash rigidly, so no emptiness test is needed. The prover itself is [MN p. 239]:

```
type lit         form -> o.
type prv         list form -> que form -> o.

prv (tt       :: Gamma) Phi.
prv ((B && C) :: Gamma) Phi :- prv (B::Gamma) Phi,
                               prv (C::Gamma) Phi.
prv ((B !! C) :: Gamma) Phi :- prv (B::C::Gamma) Phi.
prv ((all B)  :: Gamma) Phi :- pi x\ prv ((B x)::Gamma) Phi.
prv ((some B) :: Gamma) Phi :- qpush (some B) Phi Q,
                               prv Gamma Q.
prv (p A      :: Gamma) Phi :- lit (p A) => prv Gamma Phi.
prv (n A      :: Gamma) Phi :- lit (n A) => prv Gamma Phi.
prv nil Phi :- lit (n A), lit (p A).
prv nil Phi :- qpop (some B) Phi Q, prv ((B T)::nil) Q.

posints 1.
posints N :- posints M, N is M + 1.

thm B :- posints N, bound N => prv (B::nil) (que x\x).
```

Two of the four zones are not arguments at all. Σ is realized by the `pi x\` of the $\forall R$ clause: "it uses λProlog-level universal quantifiers and the generic constants they introduce during computation to implicitly realize the eigenvariables" [MN p. 238]. $\mathscr{L}$ is realized by `lit A =>`, a hypothetical fact. In the $\exists R$ clause the witness `T` is a *fresh logic variable*. It is not guessed: the initial clause `lit (n A), lit (p A)` chooses it later, by unification. The side condition "t is a Σ-term" is enforced by the unifier, which refuses to bind `T` to an eigenvariable introduced after `T` was created. Iterative deepening comes from `posints`: a failure at bound N backtracks into `posints`, which produces $N + 1$. "B represents a formula in negation normal form that has a CL proof if and only if the goal (thm B) is provable" [MN p. 239]. On non-theorems `thm` runs forever, since first-order logic is only semi-decidable.

Pitfall: cut plus logic variables loses completeness

An improvement closes a branch as soon as a complementary literal arrives [MN p. 239]: `prv (p A :: Gamma) Phi :- lit (n A) ; lit (p A) => prv Gamma Phi.` (and the dual clause). Adding a cut, `lit (n A), ! ; ...`, "is acceptable when there are no existential quantifiers", but with them "the interaction between such logic variables and the Prolog-cut (!) can lead to incompleteness" [MN p. 240]. The counterexample is

```
((p (r c)) !! (p (r t)) !! (p (g c)) !! (some x\ (n (r x)) && (n (g x)))).
```

It is a theorem, but "the logic variable that instantiates x gets bound to t first and the cut then eliminates the path in which it would get bound to c" [MN p. 240]. Cut never breaks soundness, since it only removes alternatives. It keeps completeness only after a test that cannot bind logic variables.

Kanren angle: what the classical prover needs, and what interleaving gives

∀ with a scope check is soundness-critical here. Our notes derive two tests (not in the book). The drinker formula $\exists x(Dx \supset \forall y\, Dy)$, in NNF `some x\ (n (d x)) !! (all y\ p (d y))`, must need bound 2: the first witness is created before y and cannot become y. The invalid $(\forall y \exists x\, R(x,y)) \supset \exists x \forall y\, R(x,y)$ must never be proved. Without the scope check a port proves it. **⊃.** Only atomic facts (`lit A`, `bound N`) are assumed, so an explicit literal-list argument and an explicit bound suffice. **Patterns.** The functional difference list needs L_λ. A naive first-order open-tailed list is *wrong*: popping an empty `(dl V V)` unifies `V` with a cons and conjures an item. With a ground spine, an ordinary list and `append°` is the safe port. **Search.** Under interleaving the round-robin queue alone already makes the prover complete, because every node has finitely many children and the infinite $\exists R$ chain is interleaved with the INIT alternatives. The bound then serves only to make failing searches stop. **Cut.** (aux kanren micro)'s `if°` is a *soft* cut: it keeps every answer of the test. Translating `!` as `(if° (take° 1 test) ...)` reproduces the book's incompleteness exactly. Plain `if°` happens to prove the formula above but is still incomplete in general. Only the cut-free disjunction is safe. See Chapter 11.

8.1.4 Tactics and tacticals

Direct reflection "has the benefit of yielding a transparently correct specification. However, it also means that we must use the control regimen of the interpreter" [MN p. 240]. The remedy is the LCF architecture [18], recast as relations [13, 12]. An inference rule becomes *data*: a relation between the goal of proving the conclusion and the goal of proving the premises. Search strategies then become ordinary higher-order predicates over such relations.

Definition 8.2 (goals, tactics, tacticals; [MN p. 241])**.** A *primitive goal* is an object to be proved (a sequent, a typing judgment `P # A`, ...). *Compound goals* are `trueg` (closed), `G1 cc G2` (both) and `allg (x\ G)` (for a generic `x`). A *tactic* is a predicate of type `goal -> goal -> o` relating a primitive goal to the goal of its premises. It fails when it does not apply, and it may leave logic variables, such as quantifier witnesses, in its output. A *tactical* is a higher-order predicate combining tactics.

The goal language and its simplifier are [MN p. 241]:

```
kind  goal     type.
type  trueg    goal.                    % vacuously true goal
type  cc       goal -> goal -> goal.    % conjunctive goal
type  allg     (A -> goal) -> goal.     % universally quantified goal
infixl cc 3.

type  goalreduce, redex, red1   goal -> goal -> o.
type  primgoal                  goal -> o.

redex (trueg cc G) G & redex (G cc trueg) G.
redex (allg x\trueg) trueg.
```

```
red1 G H :- redex G H, !.
red1 (G cc H) (Gx cc H) &
red1 (H cc G) (H cc Gx) :- red1 G Gx.
red1 (allg G) (allg Gx) :- pi x\ red1 (G x) (Gx x).
goalreduce G H :- red1 G Gx, !, goalreduce Gx H.
goalreduce G G.
```

`goalreduce` rewrites with $\top_g \mathit{cc}\, G \to G$, $G \mathit{cc}\, \top_g \to G$ and $\mathit{allg}(\lambda x.\top_g) \to \top_g$ anywhere in a goal, until no rewrite applies. A goal "is proved" when it reduces to `trueg`. The pattern `allg x\trueg` matches only a *vacuous* abstraction, one whose body does not mention `x`. The binder type `A` of `allg` is polymorphic, so the same goal language serves goals over terms and over proofs. The cuts are safe here because the input is ground and the rewriting is confluent.

A first example: reachability. The book's smallest instance has nodes, edges as primitive goals `adj X Y`, and paths as primitive goals `path X Y` [MN p. 242]:

```
kind node   type.
type a, b, c, d, e   node.
type adj, path   node -> node -> goal.

primgoal (adj _ _)  &  primgoal (path _ _).

type adj_tac, path_base_tac, path_rec_tac   goal -> goal -> o.

adj_tac (adj a b) trueg  &  adj_tac (adj a c) trueg.
adj_tac (adj b d) trueg  &  adj_tac (adj c d) trueg.
adj_tac (adj d a) trueg  &  adj_tac (adj d e) trueg.

path_base_tac  (path X Y) (adj X Y).
path_rec_tac   (path X Y) ((adj X Z) cc (path Z Y)).
```

`adj_tac` closes an edge goal. The two path tactics are the two inference rules for `path`. In `path_rec_tac` the intermediate node `Z` appears only in the output: a tactic may leave an unknown in the premises, to be fixed later when `adj_tac` is applied to `adj X Z`. The graph ($a{\to}b$, $a{\to}c$, $b{\to}d$, $c{\to}d$, $d{\to}a$, $d{\to}e$) has the cycle $a{\to}b{\to}d{\to}a$. The book gives no query for this figure. A run on (aux kanren micro) is in the kanren box below.

Sequent calculus as tactics. The rules of an intuitionistic sequent calculus for $\wedge$, $\supset$ and $\forall$ become tactics [MN p. 242]:

```
type sq   list form -> form -> goal.
primgoal (sq _ _).

type initial, and_r, imp_r, all_r, and_l, imp_l, all_l, all_l'   goal -> goal -> o.

initial (sq Gamma A) trueg :- memb_and_rest A Gamma _.
and_r   (sq Gamma (A && B)) ((sq Gamma A) cc (sq Gamma B)).
```

```
imp_r   (sq Gamma (A ==> B)) (sq (A::Gamma) B).
all_r   (sq Gamma (all A)) (allg x\ sq Gamma (A x)).
and_l   (sq Gamma A) (sq (B::C::Gamma') A) :-
          memb_and_rest (B && C) Gamma Gamma'.
imp_l   (sq Gamma A) ((sq Gamma B) cc (sq (C::Gamma') A)) :-
          memb_and_rest (B ==> C) Gamma Gamma'.
all_l   (sq Gamma A) (sq ((B T)::Gamma) A) :-
          memb_and_rest (all B) Gamma Gamma'.
all_l'  (sq Gamma A) (sq ((B T)::Gamma') A) :-
          memb_and_rest (all B) Gamma Gamma'.
```

Each clause relates a conclusion to the goal of its premises. `imp_l` is the plain $\supset L$ of the first calculus: its left premise `sq Gamma B` keeps $B \supset C$. The tactic `all_r` does not introduce the eigenvariable itself. It returns a quantified *goal* `allg x\ ...`, and the eigenvariable becomes a real `pi`-constant only when a tactical enters that goal. The tactic `all_l` keeps the universal hypothesis (contraction: it uses the original `Gamma`), while `all_l'` drops it (it uses `Gamma'`). In both, `T` is an open witness. The tacticals are [MN p. 244]:

```
type maptac   (goal -> goal -> o) -> goal -> goal -> o.

maptac Tac trueg trueg.
maptac Tac (I1 cc I2) (O1 cc O2) :- maptac Tac I1 O1,
                                    maptac Tac I2 O2.
maptac Tac (allg In) (allg Out) :- pi t\ maptac Tac (In t) (Out t).
maptac Tac In Out :- primgoal In, Tac In Out.

type idtac          goal -> goal -> o.
type repeat, try    (goal -> goal -> o) -> goal -> goal -> o.
type then,
     orelse, orelse! (goal -> goal -> o) ->
                     (goal -> goal -> o) -> goal -> goal -> o.

idtac             In In.
then    Tac1 Tac2 In Out :- Tac1 In Mid, maptac Tac2 Mid Out.
orelse  Tac1 Tac2 In Out :- Tac1 In Out  ; Tac2 In Out.
orelse! Tac1 Tac2 In Out :- Tac1 In Out, ! ; Tac2 In Out.
repeat  Tac       In Out :- orelse (then Tac (repeat Tac))
                                   idtac In Out.
try     Tac       In Out :- orelse Tac idtac In Out.
```

Read tactics as binary relations. Then `idtac` is the identity, `orelse` is union, `orelse!` is "the first answer of the first tactic, else the second", `try T` is "`T` if it applies, else no change", and `repeat T` applies `T` as often as possible first, with fewer applications on backtracking. `maptac T` lifts `T` from primitive goals to compound goals. It enters `allg` with `pi t\` and re-abstracts the result through the pattern `Out t`, and it fails if `T` fails on any primitive leaf. `then T1 T2` is the relational composition of `T1` with `maptac T2`. It cannot simply be `T1 In Mid, T2 Mid Out`, because `Mid` may be compound [MN p. 243].

Invertible rules first. A rule is *invertible* when its premises are provable iff its conclusion is. Such rules "can be forced to appear at the end of a proof without any loss of completeness" [MN p. 243]:

```
invertible In Out :-
   repeat (orelse and_r (orelse and_l (orelse imp_r all_r))) In Out.
```

With `p : term -> form` and `f : term -> term`, the book reports [MN p. 243–244]:

```
?- invertible (sq [] ((a && (a ==> b)) ==> (a && b))) Out.
Out = ((sq (a :: (a ==> b) :: nil) a) cc (sq (a :: (a ==> b) :: nil) b))

?- invertible (sq [] ((all x\ (p x) ==> (p (f x))) ==>
                      (all x\ (p x) ==> (p (f (f x))))))
              Out.
Out = (allg (w\ sq (p w :: (all w\ p w ==> p (f w)) :: nil)
                   (p (f (f w)))))
```

What remains in the second answer becomes propositional after two instantiations of the universal hypothesis. To finish it, the book links the decision procedure of Section 8.1.1 in as a tactic and composes a *script* [MN p. 244–245]:

```
ip_decide (sq Gamma A) trueg :- seq Gamma A.

?- then invertible (then all_l (then all_l' ip_decide))
       (sq [] ((all x\ (p x) ==> (p (f x))) ==>
               (all x\ (p x) ==> (p (f (f x))))))
       Out.
Out = allg W1\ trueg
```

Tutorial: how the script closes the goal

`invertible` leaves `allg w\ sq [p w, all (...)] (p (f (f w)))`. `maptac` enters the `allg` with a fresh constant `w`. `all_l` adds `p T1 ==> p (f T1)` and keeps the ∀. `all_l'` adds `p T2 ==> p (f T2)` and drops it. Because `T1` and `T2` were created *inside* the scope of `w`, they may be bound to terms containing `w`. `ip_decide` runs `seq`. There, $\supset L_1$ on the first implication needs the atom `p T2` in the context, and `memb_and_rest` *unifies* `T2 := w`, adding `p (f w)`. The second implication then gets `T1 := f w`, adding `p (f (f w))`, and INIT closes the sequent. The witnesses are found by unification at the leaves, exactly as in the classical prover. (`seq` needs `atom` for first-order atoms, for example `atom (p _).`, which the book does not show.) The output `allg W1\ trueg` reduces to `trueg` with the goal-simplifier `goalreduce` of [MN p. 241].

Kanren angle: tacticals in Scheme, and what interleaving does to them

Tacticals are higher-order relations, and in Scheme they are just procedures from relations to relations. Everything except `allg` is first order, so the reachability example [MN p. 242] runs on (aux kanren micro) today. Goals are tagged lists, and `maptac°` has no `allg` case:

```
(define-relation (primgoal° g)
  (fresh° (x y) (or° (=° g `(adj ,x ,y)) (=° g `(path ,x ,y)))))

(define-relation (adj-tac° in out)
  (=° out 'trueg)
  (cond° ((=° in '(adj a b))) ((=° in '(adj a c))) ((=° in '(adj b d)))
         ((=° in '(adj c d))) ((=° in '(adj d a))) ((=° in '(adj d e)))))
(define-relation (path-base-tac° in out)
  (fresh° (x y) (=° in `(path ,x ,y)) (=° out `(adj ,x ,y))))
(define-relation (path-rec-tac° in out)
  (fresh° (x y z) (=° in `(path ,x ,y)) (=° out `(cc (adj ,x ,z) (path ,z ,y)))))

(define (maptac° tac)
  (define-relation (m in out)
    (cond° ((=° in 'trueg) (=° out 'trueg))
           ((fresh° (i1 i2 o1 o2) (=° in `(cc ,i1 ,i2)) (=° out `(cc ,o1 ,o2)) (m i1 o1)
            ↪  (m i2 o2)))
           ((primgoal° in) (tac in out))))
  m)
(define (idtac° in out) (=° in out))
(define ((then° t1 t2) in out) (fresh° (mid) (t1 in mid) ((maptac° t2) mid out)))
(define ((orelse° t1 t2) in out) (or° (t1 in out) (t2 in out)))
(define ((try° t) in out) ((orelse° t idtac°) in out))
(define ((repeat° t) in out)
  (λ (s) (δ (((orelse° (then° t (repeat° t)) idtac°) in out) s))))

;; stands in for `goalreduce Out trueg`: Out is made of trueg and cc only
(define-relation (closed° g)
  (cond° ((=° g 'trueg))
         ((fresh° (g1 g2) (=° g `(cc ,g1 ,g2)) (closed° g1) (closed° g2)))))

(define step° (orelse° adj-tac° (orelse° path-base-tac° path-rec-tac°)))

(pretty-print (μkanren-run (q -1 #t) ((then° path-rec-tac° (try° adj-tac°)) '(path a e)
↪  q)))
(pretty-print (μkanren-run (q 4 #t) ((repeat° step°) '(path a e) q)))
(pretty-print (μkanren-run (q 3 #t) ((repeat° step°) '(path a e) q) (closed° q)))
```

```
((cc (adj a α) (path α e)) (cc trueg (path b e)) (cc trueg (path c e)))
((path a e) (adj a e) (cc (adj a α) (path α e)) (cc trueg (path b e)))
((cc trueg (cc trueg trueg))
 (cc trueg (cc trueg trueg))
 (cc trueg (cc trueg (cc trueg (cc trueg (cc trueg trueg))))))
```

The open witness `α` of `path-rec-tac°` is chosen later by `adj-tac°`, just as `T` is in `all_l`. The first answer of `repeat°` is *zero* applications, `(path a e)`. The book's depth-first `repeat`

would instead answer first with the maximal sequence, which here is the unprovable leftover `adj a e`. Interleaving enumerates all application lengths fairly. On the cyclic graph it produces the two three-edge proofs ($a{\to}b{\to}d{\to}e$, $a{\to}c{\to}d{\to}e$) and then a six-edge one around the cycle. A depth-first enumeration of all answers would never terminate here, and interleaving does not terminate either, but it keeps producing proofs. A script built from `orelse` and `repeat` is therefore *complete* under interleaving, while committed choice (`orelse!`, the cuts of `goalreduce`) needs `(if° (take° 1 ...) ...)`. **What is missing** is `maptac` over `allg`. It needs a universal goal (a fresh eigenvariable) and the abstraction of that eigenvariable out of the result (`Out t`, a pattern). See Chapter 11.

8.2 Computations over functional programs

Chapter 10 of the book turns to programming-language semantics. The object language miniFP is an untyped call-by-value λ-calculus with a conditional, a fixed point, booleans, integers and lists [MN p. 248]:

$$M ::= x \mid \lambda x.M \mid M\,N \mid \mathsf{cond}\,M\,N\,P \mid \mathsf{fix}\,x.M \mid \mathsf{cns}\,M\,N \mid \mathsf{i}\,n \mid \mathsf{tt} \mid \mathsf{ff} \mid \mathsf{null} \mid \mathsf{cons} \mid \mathsf{car} \mid \ldots$$

```
type abs    (tm -> tm) -> tm.          % function abstraction
type @      tm -> tm -> tm.            % application
infixl @ 4.                            % application is infix
type cond   tm -> tm -> tm -> tm.      % conditional
type fixpt  (tm -> tm) -> tm.          % recursive functions
```

The remaining constructors are first order: `cns` is the list *value* constructor, `i` injects integers, and constants such as `cons`, `car`, `sum`, `equal` have type `tm`. `cons` is a built-in *function*, applied with `@`. The encoding is untyped, so `cons @ (i 4) @ (i 5)` is a legal `tm` but not a valid program. Validity is a separate typing judgment. Four sample programs are named by `prog` [MN p. 249]. For instance:

```
prog "map" (fixpt map\ abs f\ abs l\
  cond (nullp @ l) null (cons @ (f @ (car @ l)) @
                                (map @ f @ (cdr @ l)))).
```

8.2.1 Typing: hypotheses about bound variables

The four structural rules of simple typing are [MN p. 250]:

```
typeof (M @ N) A         :- typeof M (arr B A), typeof N B.
typeof (cond P Q R) A    :- typeof P bool, typeof Q A, typeof R A.
typeof (abs M) (arr A B) :- pi x\ typeof x A => typeof (M x) B.
typeof (fixpt M) A       :- pi x\ typeof x A => typeof (M x) A.
```

One axiom per constant follows, such as `typeof car (arr (lst A) A).` There is no clause for variables. Instead, when the `abs` clause enters the binder, it *assumes* a clause `typeof x A` for the fresh `x`. The book reports [MN p. 249–250] that `typeof (abs w\ w)` has type `arr int int`, `arr bool bool`, and even `pi t\ typeof (abs w\ w) (arr t t)`. For the four programs it reports:

```
?- sigma Exp\ prog Name Exp, typeof Exp Ty.
Ty = arr int int.
Ty = arr A (arr (lst A) bool).
Ty = arr (lst A) (arr (lst A) (lst A)).
Ty = arr (arr A B) (arr (lst A) (lst B)).
```

Pitfall: polymorphic constants, monomorphic variables

The type variables of a constant's axiom are renamed at each use of the clause, so each occurrence of `car` may get its own instance. The assumed clause `typeof x A`, by contrast, contains the *outer* logic variable `A`, which is not renamed, so all occurrences of `x` share one type. That is exactly simple typing. It is also why let-polymorphism is out of reach: "no simple extension to the program provided in Figure 10.3 can realize this algorithm" [MN p. 260].

Kanren angle: the classic miniKanren type inferencer is this program

Since every hypothesis is an atomic fact about a fresh variable, an explicit environment Γ can replace `=>`. Variables are then named by Scheme symbols, and constants are wrapped as `(c name)` to keep the two apart. Sharing a logic variable inside Γ is precisely the non-renaming of `A`. Wrapping each constant's type variables in `fresh°` is the per-use renaming. Excerpt (the remaining constant clauses, and the four programs written with a helper `@*` for iterated application, are in the run file):

```
(define-relation (lookup° Γ x t)
  (fresh° (y u r)
    (=° Γ `((,y . ,u) . ,r))
    (cond° ((=° x y) (=° t u))
           ((≠° x y) (lookup° r x t)))))

(define-relation (typeof° Γ m t)
  (cond°
    ((symbol° m) (lookup° Γ m t))
    ((fresh° (M N B) (=° m `(@ ,M ,N)) (typeof° Γ M `(arr ,B ,t)) (typeof° Γ N B)))
    ((fresh° (P Q R) (=° m `(cond ,P ,Q ,R)) (typeof° Γ P 'bool) (typeof° Γ Q t)
     ↪  (typeof° Γ R t)))
    ((fresh° (x M A B) (=° m `(abs ,x ,M)) (=° t `(arr ,A ,B)) (typeof° `((,x . ,A) .
     ↪  ,Γ) M B)))
    ((fresh° (f M) (=° m `(fixpt ,f ,M)) (typeof° `((,f . ,t) . ,Γ) M t)))
    ((fresh° (n) (=° m `(i ,n)) (=° t 'int)))
    ((=° m '(c tt)) (=° t 'bool))
    ;; ... (c ff), (c null), (c equal), (c cons) ...
    ((fresh° (A) (=° m '(c car)) (=° t `(arr (lst ,A) ,A))))
    ;; ... (c cdr), (c nullp), (c consp), (c and), (c zerop), (c minus), (c sum)
```

```
    ))

(for-each (λ (p) (pretty-print (cons (car p) (μkanren-run (q -1 #t) (typeof° '() (cdr
↪  p) q)))))
  `((id . (abs w w)) (fib . ,fib) (mem . ,mem) (appnd . ,appnd) (map . ,map) (self .
  ↪  (abs w (@ w w)))))
```

```
(id (arr α α))
(fib (arr int int))
(mem (arr α (arr (lst α) bool)))
(appnd (arr (lst α) (arr (lst α) (lst α))))
(map (arr (arr α β) (arr (lst α) (lst β))))
(self)
```

These are the book's types. Self-application has none, because (aux kanren micro)'s occurs check rejects $B = B \to A$. Running the relation *backwards* enumerates inhabitants of a type:

```
(pretty-print (μkanren-run (q 5 #t) (typeof° '() q '(arr int int))))
```

```
((abs α α)
 (fixpt α α)
 (abs α (i β))
 (@ (abs α α) (abs β β))
 (abs α (fixpt β β)))
```

(Grounded answers omit the pending `symbol°` constraints on α, β.) Note the well-typed divergent term `(fixpt α α)`. In the book's clause order, depth-first search would try the application clause first and recurse on an ever larger function type. Interleaving gives every clause its turn. The named-binder encoding works here because typing never substitutes. Evaluation does, and needs more (see below).

8.2.2 Big-step evaluation

The natural semantics [27] of miniFP is a relation $M \Downarrow V$ [MN p. 251–252]. Its structural core reads:

$$\frac{V \text{ a value}}{V \Downarrow V} \qquad \frac{M \Downarrow F \qquad N \Downarrow U \qquad \text{apply}(F, U) \Downarrow V}{M\,N \Downarrow V} \qquad \frac{M[\text{fix}\,x.M/x] \Downarrow V}{\text{fix}\,x.M \Downarrow V}$$

$$\frac{M[U/x] \Downarrow V}{\text{apply}(\lambda x.M,\, U) \Downarrow V}$$

```
val (abs _) & val (i _) & val tt & val ff & val null.
val (cns _ _) & val (spec _ _ _).
```

```
eval V V            :- val V.
eval (M @ N) V      :- eval M F, eval N U, apply F U V.
eval (fixpt R) V    :- eval (R (fixpt R)) V.
eval (cond C L R) V :- eval C B, if (B = tt) (eval L V) (eval R V).
eval F (spec I F nil) :- special I F.

apply (abs R) U V         :- eval (R U) V.
apply (spec 1 F Args) U V :- eval_spec F (U::Args) V.
apply (spec C F Args) U (spec D F (U::Args)) :- C > 1, D is C - 1.
```

Application of an abstraction "corresponds to a substitution that is realized elegantly through β-conversion in the specification language" [MN p. 251]: `eval (R U) V`. A built-in `F` of arity n evaluates to the partial application `spec n F nil`, which collects already evaluated arguments *in reverse order* until the δ-rule `eval_spec` fires, for example `eval_spec minus ((i N)::(i M)::nil) (i V) :- V is M - N.` With these clauses `prog "fib" F, eval (F @ (i 12)) V` gives `V = (i 144)`, and mapping fib over $[9, 4]$ gives `(cns (i 34) (cns (i 3) null))` [MN p. 251–253]. Since `eval_spec equal` compares with λProlog's `=`, which is equality of λ-terms up to $\alpha\beta\eta$, `eval (equal @ (abs x\x) @ (abs y\y)) V` gives `V = tt` [MN p. 253]. This is "stronger" than a functional language, which cannot compare functions. (The book suggests replacing `B = C` by a first-order `eq B C` that only compares integers, booleans and lists. Since `eq` has no clause for `abs` and the test sits inside an `if`, the same query then gives `V = ff`, our notes observe.)

Tutorial: evaluating a small application

Take `eval ((abs x\ cond x (i 1) (i 2)) @ tt) V`. The clause for `@` evaluates the function, which is a value and evaluates to itself, so `F = abs x\ cond x (i 1) (i 2)`; it evaluates the argument `tt` to `U = tt`; then `apply (abs R) U V` calls `eval (R U) V`. Here `R` is the meta-level function `x\ cond x (i 1) (i 2)`, so `R U` β-reduces to `cond tt (i 1) (i 2)`: object-level substitution has been done by the metalanguage. The `cond` clause evaluates `tt` to `B = tt` and takes the then branch, `V = i 1`.

New binding constructs fit the same pattern. The book's bibliographic notes add `let`, with concrete `let x = t in b` represented as `(let t (x\b))`, and one clause [MN p. 259–260]: `eval (let T R) V :- eval T U, eval (R U) V.`

Pitfall: `if` is a commit, not logic

The conditional is defined with a cut [MN p. 137]:

```
type if   o -> o -> o -> o.
if P Q R :- P, !, Q.
if P Q R :- R.
```

It commits to the first proof of `P`. The book says it is used "only for convenience" and can be rewritten as `(U = tt, G ; U = ff, H)` when `U` is a boolean [MN p. 251]. The two readings differ on a non-boolean test. They also differ, more seriously, on an *eigenvariable* test: `k = tt` fails for a generic `k`, so `if` silently takes the else branch. This matters for mixed evaluation below.

8.2.3 Evaluation contexts as meta-level functions

The reduction semantics of Felleisen and Hieb [11] splits a term into an *evaluation context* with a hole and the next redex: $E[R] \longmapsto E[N]$ when R reduces to N, with $E ::= [\,] \mid \mathrm{cond}\, E\, N\, P \mid E\, N \mid V\, E$. In λProlog a context is simply a function `tm -> tm`, and plugging is application [MN p. 254]:

```
context R (x\ x) R :- redex R.
context (cond M N P) (x\ cond (E x) N P) R :-
    non_val M, context M E R.
context (M @ N) (x\ (E x) @ N) R :-
    non_val M, context M E R.
context (V @ M) (x\ V @ (E x)) R :-
    val V, non_val M, context M E R.

evalc V V :- val V.
evalc M V :- context M E R, reduce R N, evalc (E N) V.
```

`redex`, `non_val` and `reduce` are first-order tables (for example `reduce ((abs R) @ N) (R N).`). Each decomposition succeeds exactly once [MN p. 255]:

```
?- context (cond ((abs x\ ff) @ tt) (i 2) (i 3)) E R.
%  succeeds exactly once:  R = ((abs x\ ff) @ tt),  E = (W\ cond W (i 2) (i 3))
```

The output context `x\ cond (E x) N P` is built from the recursively computed `E` by abstraction. Every context answer is therefore a higher-order output.

8.2.4 Partial evaluation: unfolding and mixed evaluation

Both evaluators unfold `fixpt R` into `R (fixpt R)`. The query `prog "map" (fixpt Body), Unfold = (Body (fixpt Body))` exhibits this as a source-to-source transformation [MN p. 255]. (The printed result is mis-parenthesized: `@ f @ (cdr @ l)` must apply the inner `fixpt`, not lie inside its body.) More interesting is *mixed evaluation*, which evaluates under a binder by treating the bound variable as an unknown value [MN p. 256]:

```
type mixeval  tm -> tm -> o.
mixeval (abs R) (abs S) :- pi k\ val k => eval (R k) (S k).

?- prog "appnd" App,
   eval (App @ (cons @ (i 1) @ (cons @ (i 5) @ null))) R,
   mixeval R S.
```

This "yields the single answer substitution, binding the variable S to" `abs w\ cons @ (i 1) @ (cons @ (i 5) @ w)` [MN p. 256]. (By the `eval_spec cons` clause the value form should be `cns (i 1) (cns (i 5) w)`, the same function.) Append has been specialized away. Note `(S k)`: a logic variable applied to an eigenvariable, the pattern case. Its solution abstracts `k` out of the computed value.

Proposition 8.3 ([MN p. 256]). *If* `mixeval (abs R) (abs S)` *holds and* `val T` *is provable, then evaluating* `(R T)` *yields the value* `(S T)`.

The proof idea is the substitution property of $\forall$ and $\supset$: instantiate the generic derivation of `eval (R k) (S k)` at `T` and discharge `val k` with `val T`.

Pitfall: the proposition needs the logical reading

With the cut-based `if`, Proposition 8.3 fails. Our notes' counterexample is `mixeval (abs b\ cond b (i 1) (i 2)) S`. The test `if (k = tt) ...` cannot unify the eigenvariable, so the else branch gives `S = b\ i 2`, but `eval (cond tt (i 1) (i 2))` is `i 1`. With `(B = tt, ... ; B = ff, ...)` the conditional on `k` simply fails, and soundness holds. Failing on an eigenvariable means "unknown", not "different".

8.2.5 CPS conversion and administrative redexes

Fischer's call-by-value CPS transformation [14] [MN p. 257] on the pure fragment is defined by $\mathscr{F}$ on terms and Ψ on values (abstractions and variables). The marked $\bar{\lambda}$ are the *administrative* abstractions:

$$\begin{aligned} \mathscr{F}[V] &= \bar{\lambda}k.\,(k\ \Psi[V]) \\ \mathscr{F}[M\,N] &= \bar{\lambda}k.\,(\mathscr{F}[M]\ \bar{\lambda}m.\,(\mathscr{F}[N]\ \bar{\lambda}n.\,(m\,k\,n))) \\ \Psi[x] &= x \\ \Psi[\lambda x.M] &= \lambda k.\,\lambda x.\,(\mathscr{F}[M]\,k) \end{aligned}$$

```
type ftrans, phi  tm -> tm -> o.

ftrans (abs V) (abs k\ k @ U) :- phi (abs V) U.
ftrans (M @ N) (abs k\ P @ (abs m\ Q @ (abs n\ m @ k @ n))) :-
    ftrans M P, ftrans N Q.

phi (abs M) (abs k\ abs x\ (P x) @ k) :-
    pi x\ pi y\ ftrans x (abs k\ k @ y) => ftrans (M x) (P y).
```

Tutorial: where did the variable case go?

Neither `ftrans` nor `phi` has a clause for variables. When `phi` enters the binder of `abs M`, it creates a source variable `x`, a target variable `y`, and the local clause `ftrans x (abs k\ k @ y)`. That clause *is* the equation $\mathscr{F}[x] = \bar{\lambda}k.(k\,\Psi[x])$ for this one variable. The translated body is `P y`, a pattern, so `P` is the body abstracted over `y`, and `abs x\ (P x) @ k` re-binds it. The same device reappears in the λ-to-π translation of Section 8.3.3: "one base case is assumed for every bound variable encountered" [MN p. 274].

On $(\lambda u.u)(\lambda u.u)$ the book reports [MN p. 257]:

```
?- ftrans ((abs u\u) @ (abs u\u)) F.

F = (abs W1\ (abs (W2\ W2 @ abs W3\ abs W4\ abs (W5\ W5 @ W4) @ W3)) @
     (abs W2\ (abs (W3\ W3 @ abs W4\ abs W5\ abs (W6\ W6 @ W5) @ W4)) @
     (abs W3\ W2 @ W1 @ W3))).
```

To read it, let U be $\Psi[\lambda u.u]$ before any simplification, `abs a\ abs b\ ((abs c\ c @ b) @ a)`. Then `F` is `abs k\ (abs k1\ k1 @ U) @ (abs m\ (abs k2\ k2 @ U) @ (abs n\ m @ k @ n))`, which is the application equation of $\mathscr{F}$ with both sides instantiated by the value equation. The result contains many β-redexes that exist "solely for the purpose of providing a compositional account of the transformation". These are the administrative redexes. The two-phase version [MN p. 258] builds administrative abstractions with a separate constructor `adm : (tm -> tm) -> tm` and then reduces them away:

```
admred ((adm R) @ N) (R N).

red1 M N :- admred M N.
red1 (M @ N) (M' @ N) & red1 (N @ M) (N @ M') :- red1 M M'.
red1 (adm R) (abs S) & red1 (abs R) (abs S) :-
    pi x\ red1 (R x) (S x).

red M N :- red1 M P, !, red P N.
red M M.

?- ftrans ((abs x\x) @ (abs x\x)) T, red T S.
S = (abs W1\ (abs W2\ abs W3\ W2 @ W3) @ W1 @ (abs W2\ abs W3\ W2 @ W3)).
```

The `ftrans`/`phi` clauses are those above with `adm` for the marked $\bar{\lambda}$. The answer is $\lambda k.\,\Psi'\,k\,\Psi'$ with $\Psi' = \lambda k.\lambda x.(k\,x)$. The cut in `red` is safe because administrative reduction is confluent and terminating.

Kanren angle: what functional meta-programs need

∀ and ⊃. Every hypothesis in this chapter is an atomic fact about a fresh variable: `typeof x A`, `val k`, `ftrans x (abs k\ k @ y)`. A universal goal plus a scoped atomic hypothesis covers them all. The scope check is mostly invisible here, since no query lets an eigenvariable escape. **Patterns.** The *outputs* `(S k)` of `mixeval`, `(P y)` of `phi`, `(S x)` of `red1` and the contexts `x\ cond (E x) N P` are pattern equations. Each means "abstract the eigenvariable out of the result". **Beyond patterns.** `(R U)`, `(R (fixpt R))`, `(E N)` are not patterns, but in the intended modes their heads are known, so they are plain β-reductions. Running them backwards would need general higher-order unification. **First-order alternatives.** The book cites Hannan and Miller's transformation of β-based specifications into closure-based ones [19]. The standard relational miniKanren interpreter with environments and closures is that transformation applied to `eval`. It runs backwards, but values become closures, which changes `equal` on functions and rules out `mixeval`. Contexts can be defunctionalized into first-order frames with a structural plug relation. **Conditionals.** `if (B = C) ...` becomes a `cond°` of `=°` and `≠°`, which is *more* relational than the book and works with unbound arguments. Under the module's default generic

reading (Definition 14.1), however, `≠°` does decide $k \neq \mathrm{tt}$ for an eigenvariable k, so a `cond°` of `=°` and `≠°` reproduces the `mixeval` unsoundness. Under `(fresh° (q) ...)`,

```
(∀° (k) (cond° ((=° k 'tt) (=° q 1)) ((≠° k 'tt) (=° q 2))))
```

answers `(2)`. With `(μkanren-λ-reading 'extensional)` the disequality is kept and the conditional on k fails, answering `()`, as soundness requires. Where a literal translation of a commit is wanted, (aux kanren micro)'s soft-cut `if°` needs `take° 1`. See Section 12.5 and Section 14.2.5.

8.3 A process calculus: the π-calculus

The π-calculus [40] has the most delicate binders of the three applications. Names are bound by input and by restriction, and a restricted name can be sent out of its scope (*scope extrusion*). Chapter 11 of the book shows that λ-tree syntax treats this "naturally" [MN p. 261], and it also uses the calculus to expose a limit of the logic.

8.3.1 Syntax and one-step transitions

Processes are $P ::= 0 \mid P \mid P \mid P + P \mid x(y).P \mid \bar{x}y.P \mid [x = y].P \mid \tau.P \mid (y)P \mid !P$, where y is bound in $x(y).P$ and in $(y)P$. Free names are constants, and bound names are λ-variables [MN p. 262–263]:

```
kind name  type.
kind proc  type.
type null        proc.
type plus, par   proc -> proc -> proc.
type in          name -> (name -> proc) -> proc.
type out, match  name -> name -> proc -> proc.
type taup        proc -> proc.
type nu          (name -> proc) -> proc.
type bang        proc -> proc.

type a, b, c     name.
type example     int -> proc -> o.

example 1 (par (in b y\ null) (out b a null)).
example 2 (plus (in b y\ out b a null) (out b a (in b y\ null))).
example 3 (nu x\ par (in x y\ null) (out x a null)).
example 4 (nu x\ out a x null).
example 5 (in a y\ par (in y w\ null) (out b b null)).
example 6 (in a y\ plus (in y w\ out b b null)
                        (out b b (in y w\ null))).
example 7 (nu y\ out a y (par (in y w\ null) (out b b null))).
example 8 (nu y\ out a y (plus (in y w\ out b b null)
                               (out b b (in y w\ null)))).
```

For example, example 1 is $(b(y).0) \mid (\bar{b}a.0)$ and example 3 is $(x)((x(y).0) \mid (\bar{x}a.0))$. Actions are `tau`, the free output `up x y` $(\bar{x}y)$ and the free input `dn x y`. A *bound* action is the partial application `up x` or `dn x`, of type `name -> action`. There are two transition predicates. `one P A P'` has a free action and a process as result. `onep P A P'` has a bound action and an *abstracted* continuation of type `name -> proc`. The book's rules with binding side conditions are [MN p. 264–265]:

$$\frac{P \xrightarrow{\alpha} P'}{(y)P \xrightarrow{\alpha} (y)P'}\ \text{RES}\ (y \notin n(\alpha)) \qquad \frac{}{x(z).P \xrightarrow{x(w)} P\{w/z\}}\ \text{INPUT}\ (w \notin fn((z)P))$$

$$\frac{P \xrightarrow{\bar{x}y} P'}{(y)P \xrightarrow{\bar{x}(w)} P'\{w/y\}}\ \text{OPEN}\ (y \neq x,\ w \notin fn((y)P')) \qquad \frac{P \xrightarrow{\bar{x}(w)} P' \qquad Q \xrightarrow{x(w)} Q'}{P \mid Q \xrightarrow{\tau} (w)(P' \mid Q')}\ \text{CLOSE}$$

Together with the binder-free rules (prefixes, match, sum, par, and the communication rule com), they become [MN p. 264]:

```
kind action  type.
type tau     action.
type up, dn  name -> name -> action.

type one   proc -> action -> proc -> o.
type onep  proc -> (name -> action) -> (name -> proc) -> o.

one  (taup P)      tau      P.
one  (out X Y P)   (up X Y) P.
onep (in X M)      (dn X)   M.
one  (match X X P) A P' :- one  P A P'.
onep (match X X P) A P' :- onep P A P'.
one  (plus P Q) A P'    :- one  P A P'; one  Q A P'.
onep (plus P Q) A P'    :- onep P A P'; onep Q A P'.
one  (par P Q) A (par P' Q) &
one  (par Q P) A (par Q P') :- one P A P'.
onep (par P Q) A (y\ par (P' y) Q) &
onep (par Q P) A (y\ par Q (P' y)) :- onep P A P'.
one  (nu P) A (nu P')           :- pi y\ one  (P y) A (P' y).
onep (nu P) A (x\ nu y\ P' y x) :- pi y\ onep (P y) A (P' y).
onep (nu P) (up X) P'           :-
       pi y\ one (P y) (up X y) (P' y).
one  (par P Q) tau (nu y\ par (P' y) (Q' y)) &
one  (par Q P) tau (nu y\ par (Q' y) (P' y)) :-
       onep P (up X) P', onep Q (dn X) Q'.
one  (par P Q) tau (par S (T Y)) :- one  P (up X Y) S,
                                    onep Q (dn X)   T.
one  (par P Q) tau (par (S Y) T) :- onep P (dn X) S,
                                    one  Q (up X Y) T.
```

Tutorial: where the side conditions went

res: in `one (nu P) A (nu P') :- pi y\ one (P y) A (P' y)`, the variable `A` is quantified over the whole clause, outside the `pi y`. No legal instance can put `y` into `A`: "the side condition associated with the res rule is realized declaratively through a proper nesting of quantification" [MN p. 265]. *input* and *open*: the bound name is "represented by an explicit abstraction in the abstracted processes that result" [MN p. 266]. The input clause simply returns the abstraction `M`, so there is nothing to rename. *open*'s $y \neq x$ holds because `y` is quantified inside the scope of `X`. *close*: the abstraction over the restricted name lies inside the scope of `P'` and `Q'`, so the name cannot occur free in them [MN p. 266]. The input continuation stays abstract until a communication plugs the received name in, at `(T Y)` and `(S Y)`. This is the late semantics.

The book animates the specification with queries [MN p. 266–267]:

```
?- example 1 P, one P A P'.
P = par (in b W\ null) (out b a null)
A = up b a
P' = par (in b W\ null) null;
P = par (in b W\ null) (out b a null)
A = tau
P' = par null null;
no

?- example 1 P, onep P A P'.
P = par (in b W\ null) (out b a null)
A = W\ dn b W
P' = W\ par null (out b a null);
no
```

For example 3 the query for `one` has a single answer, `tau` to `nu W\ par null null` [MN p. 267]: every visible action would mention the restricted name. Our notes also derive, by the same reading of the clauses, that example 4, $(x)(\bar{a}x.0)$, has *no* free transition (sending x would leak it) but has the bound output `A = W\ up a W`, `P' = W\ null` by *open*.

Tutorial: three transitions, clause by clause

The τ of example 1. The goal is `one (par (in b y\ null) (out b a null)) tau P'`. The first communication clause needs the left component to output, which an input cannot do. The second, `one (par P Q) tau (par (S Y) T) :- onep P (dn X) S, one Q (up X Y) T`, works: `onep (in b y\ null) (dn X) S` matches the input clause with `X = b` and `S = y\ null`; then `one (out b a null) (up b Y) T` matches the output clause with `Y = a`, `T = null`. The result `par (S Y) T` is `par ((y\ null) a) null`, which β-reduces to `par null null`. The received name was plugged in only now, by the application `(S Y)`: this is the late semantics.

Why example 3 has no visible action. The *res* clause asks `pi y\ one (P y) A (P' y)`, with `P y = par (in y z\ null) (out y a null)`. The output component offers `up y a`, so `A = up`

`y a` would be needed. But `A` was created *before* the `pi y`, and the unifier refuses to bind it to a term that contains `y`. Only the internal communication, whose label `tau` does not mention `y`, survives.

Scope extrusion in example 4. `one (nu x\ out a x null) A P'` fails for the same reason (`A = up a y` is out of scope). The bound-output clause for *open*, `onep (nu P) (up X) P' :- pi y\ one (P y) (up X y) (P' y)`, succeeds instead: the action pattern `up X y` already contains the fresh `y`, so only `X = a` is needed, and `(P' y) = null` is a pattern equation with the vacuous solution `P' = W\ null`. The extruded name is carried by the abstraction in the result, not by a free constant.

8.3.2 Traces, may and must

Traces record sequences of actions. Bound outputs are continued with a *generic* name, while bound inputs are continued with a *logic variable*, "for an input action, we would want to consider different possible instantiations" [MN p. 268]:

```
kind trace  type.
type empty  trace.
type tr     action -> trace -> trace.
type trp    (name -> action) -> (name -> trace) -> trace.

type trace  proc -> trace -> o.
trace P empty.
trace P (tr Act Tr)     :- one P Act Q, trace Q Tr.
trace P (trp (up X) Tr) :- onep P (up X) Q,
                           pi x\ trace (Q x) (Tr x).
trace P (tr (dn X Y) Tr) :- onep P (dn X) Q, trace (Q Y) Tr.
```

For example 1 the query `?- example 1 P, trace P Tr.` produces six traces [MN p. 268]:

```
empty
tr (up b a) empty
tr (up b a) (tr (dn b T) empty)
tr tau empty
tr (dn b T) empty
tr (dn b T) (tr (up b a) empty)
```

(The book prints `trp (dn b T)` in the last two lines. Its own clauses produce `tr (dn b T)`, since only bound *outputs* build `trp`, and its later `comptrace` answers use `tr (dn b T1)`.) Each input leaves a logic variable `T`: the trace says "some name was received". The logic variable can be *narrowed* later by `match`. The query below enumerates exactly four traces [MN p. 268–269]:

```
?- trace (in a Y\ plus (match Y b (out Y Y null))
                       (match Y c (out Y Y null))) Tr.


empty
```

```
tr (dn a T) empty
tr (dn a b) (tr (up b b) empty)
tr (dn a c) (tr (up c c) empty)
```

After the input, the continuation is `plus (match T b (out T T null)) (match T c ...)`. The clause `one (match X X P) A P'` can fire only if its two arguments unify, so it binds `T := b` in one branch and `T := c` in the other. Had the input been continued with a *generic* name (`pi`), both matches would have failed and the last two traces would be lost: "this process can do an unrestricted input and … if that input is chosen to be b or c, then further, distinct output actions can follow" [MN p. 269].

Definition 8.4 (may- and must-judgments; [MN p. 269])**.** A *may-judgment* asserts the existence of a computation path, so it corresponds to the existence of a proof. A *must-judgment* quantifies over all behaviours: a process can never do some action, all traces of P are traces of Q, P is simulated by Q.

"The style of logic specifications that we have described in this book does not, in general, provide declarative treatments of must-judgments. Negation-as-failure can be used, however, to capture or approximate some of these judgments" [MN p. 269]. Negation-as-failure is the cut-based predicate of [MN p. 137]:

```
type not  o -> o.
not P :- P, !, fail.
not P.
```

With it the book defines terminal processes, complete traces, separating traces and trace equivalence [MN p. 270]:

```
type possible, terminal  proc -> o.
type comptrace           proc -> trace -> o.
type separating_trace    proc -> proc -> trace -> o.
type trace_equiv         proc -> proc -> o.

possible P :- one P _ _ ; onep P _ _.
terminal P :- not (possible P).

comptrace P empty :- terminal P.
comptrace P (tr Act Tr)      :- one P Act Q, comptrace Q Tr.
comptrace P (tr (dn X Y) Tr) :- onep P (dn X) P',
                                comptrace (P' Y) Tr.
comptrace P (trp (up X) Tr)  :- onep P (up X) P',
                                pi x\ comptrace (P' x) (Tr x).

separating_trace P Q T :- trace P T,
                          not (trace Q T).
trace_equiv P Q :- not (separating_trace P Q _),
                   not (separating_trace Q P _).
```

A complete trace is one that cannot be extended: it may end with `empty` only at a process that can do nothing. Each of the four predicates hides a "for all" inside a `not`: `terminal P` says that

no transition exists, `trace_equiv` that *no* trace separates. Examples 5 and 6 are separated, in one direction only, by an input of `b` followed by an internal step [MN p. 270–271]:

```
?- example 5 P, example 6 Q, separating_trace P Q T.
T = tr (dn a b) (tr tau empty)
```

Example 5 is $a(y).((y(w).0) \mid (\bar{b}b.0))$. After the input, its two parallel components can communicate, but only on a common channel, so the communication clause unifies the received logic variable with `b`. Example 6, $a(y).((y(w).\bar{b}b.0) + (\bar{b}b.y(w).0))$, offers the same two actions only in sequence, inside a choice, and can never make the internal step. The query in the other direction answers `no`. Examples 7 and 8 replace the input prefix by a bound output. They have no separating trace in either direction [MN p. 271], because the extruded name is generic and can never equal `b`.

An incorrect simulation. Trace equivalence is too coarse: $\bar{x}a.(\bar{y}b.0 + \bar{y}c.0)$ and $(\bar{x}a.\bar{y}b.0 + \bar{x}a.\bar{y}c.0)$ are trace-equivalent, but after $\bar{x}a$ the first can still choose between two outputs and the second cannot [MN p. 271]. The finer notion is simulation: "P is simulated by Q if whenever P can perform an action and evolve into a process P', Q can perform an identical action that transforms it into a process Q' that simulates P'" [MN p. 271], read coinductively. The book attempts it with negation-as-failure and labels its own attempt *incorrect* [MN p. 272]:

```
type foreach2  (A -> B -> o) -> (A -> B -> o) -> o.
type sim       proc -> proc -> o.

foreach2 P Q :- not (sigma X\ sigma Y\ P X Y, not (Q X Y)).

sim P Q :- foreach2 (A\P'\ one P A P')
                    (A\P'\ sigma Q'\ one Q A Q', sim P' Q'),
           foreach2 (A\P'\ onep P A P')
                    (A\P'\ sigma Q'\ onep Q A Q',
                           pi x\ sim (P' x) (Q' x)).
```

The query

```
?- sim (in a x\ par (in x y\ null) (out c b null))
       (in a x\ plus (in x y\ out c b null) (out c b (in x y\ null))).
solved
```

answers `solved`, which is wrong. After receiving `c` on `a`, the first process can do a τ and the second cannot. Diagnosis: "the universal quantifier in goals …has an *intensional* character: The generic goal must be derived by showing that an *identical* derivation exists for any of its instances. However, π-calculus processes may have different evolutions for different instantiations of an input action" [MN p. 272]. What is needed is "a universal quantifier that has an *extensional* interpretation". The bibliographic notes point to logics with definitions [32], where $\forall$ is extensional, together with a separate *generic* quantifier ∇ for names [38] ([MN p. 275]).

Pitfall: one `pi`, two jobs

In λProlog, `pi` serves both as "a fresh name" (binders, where it is right) and as "for all instances" (must-judgments, where it is wrong). Negation-as-failure is "at best an ad hoc device" with uninstantiated variables [MN p. 273], and it needs finite failure, which replication can destroy.

8.3.3 Replication, and the λ-calculus inside π

Replication $!P$, "the parallel composition of any number of copies of P" [MN p. 262], adds four clauses [MN p. 273]:

```
one  (bang P) A (par P1 (bang P)) :- one P A P1.
onep (bang P) X (y\ par (M y) (bang P)) :- onep P X M.
one  (bang P) tau (par (par R (M Y)) (bang P)) :-
     onep P (dn X) M, one P (up X Y) R.
one  (bang P) tau (par (nu y\ par (M y) (N y)) (bang P)) :-
     onep P (up X) M, onep P (dn X) N.
```

A copy acts with a free action, a copy acts with a bound action, two copies communicate a free name, and two copies communicate an extruded one. In each case a fresh `bang P` remains. Milner's call-by-name encoding [39] sends arguments over a channel u [MN p. 273]:

$$[\![x]\!](u) = \bar{x}u.0 \qquad [\![\lambda x M]\!](u) = u(x).u(v).[\![M]\!](v)$$
$$[\![M\,N]\!](u) = (v).([\![M]\!](v) \mid (x).(\bar{v}x.\bar{v}u.!x(w).[\![N]\!](w)))$$

```
type trans  tm -> (name -> proc) -> o.

trans (abs M) (u\ in u x\ in u v\ P x v) :-
    pi x\ pi y\ trans x (u\ out y u null) => trans (M x) (P y).

trans (app M N)
      (u\ nu v\ par (P v)
                    (nu x\ out v x (out v u (bang (in x Q))))) :-
    trans M P, trans N Q.
```

As in `phi`, the base case for variables is assumed as a local clause when the recursion enters a binder. The book checks the encoding by computing complete traces [MN p. 274]:

```
?- trans (abs w\ w) P, comptrace (P b) T.
P = u\ in u x\ in u y\ out x y null
T = tr (dn b T1) (tr (dn b T2) (tr (up T1 T2) empty));
no
```

For `(app (abs w\ w) (abs w\ w))` the complete trace is `tr tau (tr tau (tr tau (tr (dn b T1) (tr (dn b T2) (tr (up T1 T2) empty)))))`: three silent steps "corresponding to reduction of the term $(\lambda x\,x)(\lambda x\,x)$ to $\lambda x\,x$", then the trace of the identity [MN p. 275]. (The printed process term for this query shadows a name and swaps the two `par` components, but the trace is unaffected.)

Remark 8.5 (L_λ and πI; [MN p. 275]). "Almost all the one-step transition rules ...are in the L_λ fragment". The exceptions are the last two clauses of the `one`/`onep` program and the third replication clause, where an abstraction is applied to a received logic variable (`T Y`, `S Y`, `M Y`). Deleting them gives Sangiorgi's πI [53], in which "only new names are bound to inputs", so the metalogic needs only β_0-conversion.

Kanren angle: what the π-calculus needs

$\forall$ with a real scope check. Here the scope check is the whole point, because *res*, *open* and *close* hold only because outer variables cannot capture the fresh name. Without it, example 4 would perform the free output $\bar{a}x$ and example 3 would perform `up W a`. A cheaper discipline, posting an `absent°` constraint on the clause's own variables after each generic subgoal, is *not* enough. Our notes give the test `trace (in a z\ nu x\ match z x (out b b null)) Tr`. In λProlog it yields only `empty` and `tr (dn a T) empty`, because the received `T`, a variable hidden inside the process, may not become the restricted `x`. An absent-on-clause-variables port also yields `tr (dn a T) (tr (up b b) empty)`. The check must live in unification. $\supset$**.** `trans` assumes atomic clauses about fresh variables, as in Section 8.2. **Patterns.** Abstracted continuations, `(P' y)` under `pi y`, and the trace abstraction `(Tr x)` are all pattern equations. **Beyond patterns.** `(T Y)` with `Y` a logic variable is outside L_λ. A faithful engine postpones it until `T` is known. A first-order locally nameless encoding turns it into a harmless structural "open" relation. **Negation.** `not G` is `(if° (take° 1 G) ✗° ✓°)`. Without `take° 1`, (aux kanren micro)'s soft cut would map `✗°` over every answer of `G` and diverge whenever `G` has infinitely many. **Search.** With `bang`, `trace` becomes an infinite but productive stream under interleaving, but negation still needs finite failure. **Must-judgments.** Our notes propose, beyond the book, to make the bound-input case of `sim` extensional by case analysis over the free names plus one fresh name. This is a finite substitute for the definitions-and-∇ reading. See Chapter 11 for which of these the proposal adopts.

8.4 Summary

- **One recipe for three domains.** A judgment becomes a predicate. Object binders become meta λs. Going under a binder means `pi` plus a hypothetical fact. Freshness and non-capture side conditions are then discharged by quantifier nesting.
- **Specification versus search.** Direct translations are sound, but depth-first execution can loop ($\supset L$, `#` as a prover). A better calculus (Dyckhoff's) gives a decision procedure. Programmable control (tactics and tacticals) moves the strategy into the program.
- **Logic variables as delayed choices.** Witnesses of $\exists R$, `all_l` and `path_rec_tac`, and received names in traces, are chosen by unification at the leaves. The unifier must respect eigenvariable scope.
- **Cut and `if` are dangerous with logic variables and eigenvariables.** They can lose completeness (the CL counterexample) or break soundness theorems (`mixeval`).

- **Must-judgments are beyond hohh.** `pi` is intensional. Simulation needs an extensional ∀ (definitions) and a generic ∇.
- **Interleaving search** makes naive calculi, prover modes and tactic scripts complete for answers, and it makes the classical prover complete without iterative deepening. It never provides termination.

The requirements, application by application ("atomic" means every assumed clause is an atomic fact, so an explicit context could replace `=>`):

application	∀ (eigenvariables)	⊃ (hypotheses)	λ-term unification
Dyckhoff `seq`	only in the query	atomic, query only	none: **runs on (aux kanren micro) today**
natural deduction `#`	proof labels, $\forall I$, $\exists E$	atomic (`p # A`)	patterns for completion, `A' (f y)` delayed
classical `prv`	$\forall R$, scope check vital	atomic (`lit`, `bound`)	patterns (difference-list queue)
tactics	`maptac` under `allg`	none	pattern `Out t`; rest **runs today**
`typeof`	under `abs`, `fixpt`	atomic	none with named binders: **runs today**
`eval`, `evalc`	none	none	β with known heads; contexts as outputs
`mixeval`, CPS	under the binder	atomic	patterns `(S k)`, `(P y)`
π transitions, traces	*res*/*open*/*close*, scope check vital	atomic (`trans`)	patterns, plus non-L_λ `(T Y)`

Part II

Higher-order miniKanren

9 Higher-order miniKanren: the literature

The previous chapters described what λProlog is. It has a logic, higher-order hereditary Harrop formulas (Chapter 3). It has a term language, simply typed λ-terms used as λ-tree syntax (Chapters 4 and 6). It has a unifier, Huet's procedure or its decidable core, pattern unification (Chapter 7). And it has a style of programming (Chapter 8). This chapter changes the point of view. Several people have tried to put these ideas into a miniKanren, and several systems implement λProlog efficiently. What did they actually do, and what went wrong?

The user named one paper as the key reference: Ma and Friedman, *A New Higher-order Unification Algorithm for λKanren* [29]. We present it in full in Section 9.2, followed by a critical assessment against pattern unification and nominal unification (Section 9.3). Then we survey the rest of the literature. The earlier λKanren paper [30] comes first (Section 9.4). Then nominal unification and αKanren [58, 5] (Section 9.5). Then eigenvariables, negation and ∀ inside miniKanren [21, 41, 26] (Section 9.6). Last come the techniques used by real λProlog systems, Teyjus, ELPI and Makam (Section 9.7). The chapter ends with a comparison table of the design options (Section 9.8). It does *not* choose between them: the choice is made in Chapter 11.

9.1 The map of the territory

The book gives the logic and an operational semantics. It has the search rules AND, AUGMENT, GENERIC and BACKCHAIN, plus unification of λ-terms. A miniKanren that wants to host this must answer six concrete questions. The rest of the chapter is organized around them.

Q1. Representation of λ-terms. The options are named binders, de Bruijn indices, de Bruijn levels, locally nameless terms, nominal atoms, or explicit substitutions. The goal is to make α-equivalence cheap and to avoid β as far as possible.

Q2. Which unification runs inside $=^{\circ}$. There are five candidates: first-order (today), nominal (αKanren), Miller's pattern unification L_λ (λKanren 2020, Teyjus 2, ELPI), Huet's full pre-unification, or the "static closures" of Ma and Friedman 2021.

Q3. How to implement GENERIC ($\forall x.G$). You need fresh eigen-constants plus a *scope check*. The check can use an explicit quantifier prefix [34, 30], numeric universe indices or levels [43, 9], or an absento-like constraint over the variables in scope [21].

Q4. How to implement AUGMENT ($D \supset G$). A hypothetical clause must be visible to every relation call made during the proof of G, and only there. In a shallow embedding this clashes with the lexical scope of host-language procedures.

Q5. What to do outside the decidable fragment. You can fail, delay the equation as a constraint, or enumerate unifiers.

Q6. How the new features meet the other constraints `≠°`, `absent°` and the type tags. Eigen-constants and disequality do not mix naively.

The historical arc, in the notes' order: Miller 1991 (L_λ, mixed prefixes, raising and pruning) [34]; Nipkow 1993 (functional, de Bruijn) [48]; Nadathur, Jayaraman and Kwon 1995 (universe indices) [43]; Dowek, Hardin, Kirchner and Pfenning 1996 (explicit substitutions) [8]; Urban, Pitts and Gabbay 2004 (nominal unification) [58]; Nadathur and Linnell 2005 (on-the-fly raising) [44]; αKanren 2007 [5]; ELPI 2015 [9]; then the miniKanren papers of 2019–2021 [41, 30, 29, 26].

9.2 Ma and Friedman 2021: static closures

9.2.1 What kind of paper this is

The paper is a ten-page workshop paper, presented at the miniKanren Workshop 2021. Its abstract states the thesis (p. 1):

> "λKanren, adapted from λProlog, extends miniKanren with higher-order logic programming. The existing higher-order unification algorithms, however, are neither sound nor complete w.r.t. an "intuitive" understanding of α-equivalence. This paper shows these gaps and proposes a new method to handle α-equivalence in higher-order unification."

It opens provocatively: "Is there a world where true is equal to false? Indeed, there is, if higher-order unification and miniKanren meet each other" (p. 1).

The background section recalls the one notion from Chapter 7 that the whole paper argues about, in its own words (p. 1): "For an application form Fa, if F is a unification variable, then a must be a list of zero or more distinct bound variables. Such restricted application forms are called *patterns* and we hereafter refer to the problem identified by Miller as *pattern unification*." The related work (p. 7–8) adds Huet's vocabulary: a term is *rigid* "if its head symbol is a bound variable or a constant", and *flex* "if its head symbol is a unification variable". Keep in mind one fact for later: in every query of §2–4, the unknown X sits under a λ but is *not* applied to anything. It is a pattern with zero arguments, so pattern unification lets it depend on none of the bound variables.

Pitfall: a preview, not an algorithm

The conclusion says: "This paper shows a preview of static closures" (p. 9). The paper has **no algorithm listing, no inference rules for unification, no theorems and no proofs**. Its formal content is two grammars, three operations on scopes and three worked examples. Its code is a set of REPL transcripts. Soundness and completeness are explicitly future work: "We anticipate extending the formulation of higher-order unification with static closures and derive a sound and complete algorithm for α-equivalence" (p. 9). The rules in Section 9.2.5 below are our own *reconstruction*, not the paper's, and we flag them as such. (Where this chapter says "the notes", it means the unpublished working notes from which this survey was written.)

The paper has seven sections. §1 gives background. §2 asks "grafting or substitution?" (p. 2–4). §3 shows how one can "turn false to true" (p. 4). §4 introduces static closures (p. 5–6). §5 describes a shallow-embedding implementation of the language (p. 6–7). §6 is related work (p. 7–8) and §7 the conclusion. The paper uses two box styles for examples: "solid boxes for examples that are complete and dash boxes for examples that are incomplete or somewhat questionable" (footnote 2, p. 2). In every `(run 1 X ...)` of the paper, `X` is simply existentially quantified at the top, outside every λ (footnote 1, p. 2).

9.2.2 Grafting versus substitution

First-order miniKanren already has a word for what unification produces. For

```
> (run 1 X
    (== X 'cat))
'(cat)
```

the paper says "internally, the answer is a grafting … written as $\{X \mapsto \texttt{'cat}\}$" (p. 2).

Definition 9.1 (Grafting and substitution [29, p. 2–3])**.** A *grafting* $\{X \mapsto t\}$ is a purely textual replacement of the unification variable X by t. It is *not* capture-avoiding: grafting $X \mapsto a$ into $\lambda a.\, X$ yields $\lambda a.\, a$, so the name a gets captured. A *substitution* is the capture-avoiding instantiation of standard higher-order unification: "Standard higher-order unification relies on substitution, but not grafting, to instantiate unification variables. Substitution is capture-avoiding, whereas grafting is not" (p. 3).

In first-order unification the two notions coincide, because there are no binders. They part ways as soon as terms have binders. The paper follows a "novice" through three queries. The first one, α-equivalence of closed terms, is unproblematic:

```
> (run 1 X
    (== (λ (a) a) (λ (b) b)))
'(_0)
```

(`_0` means that X is unrestricted). Then, "out of curiosity", the novice puts the query variable under a binder and asks "Does this query succeed?":

```
> (run 1 X
    (== (λ (a) X) (λ (b) b)))
'()
```

"Surprisingly enough, with higher-order unification, there is no answer." The novice expects $X = a$. The reason is the *scope restriction.*

Definition 9.2 (Scope restriction [29, p. 3])**.** "Unification variables are ∃-quantified and lambda variables are ∀-quantified. A scope is a list of those quantifiers. Further, a ∀-quantified variable may instantiate (or occurs in the instantiation of) an ∃-quantified variable, only if the ∀ variable precedes the ∃ variable in the scope." As a side condition on a binding in a mixed prefix Q:

$$\frac{X \text{ is } \exists\text{-quantified in } Q \qquad \forall y \in FV_{\forall}(t).\ y \prec_Q X}{X \mapsto t \text{ is allowed}}$$

For `λa.X =? λb.b` the scope is `'((∃ X) (∀ a))`. The variable *a* comes *after* *X*, so it cannot instantiate *X*. This is exactly λProlog's behaviour: it is the eigenvariable condition of GENERIC (Chapter 3), and in unification it is what raising and pruning enforce (Chapter 7). The paper draws the opposite conclusion (p. 3): "It is fair to say that the existing pattern unification algorithms are not complete w.r.t. the natural sense of α-equivalence." It also says (p. 3): "For beginners who are not familiar with higher-order unification, α-equivalence is only decided by the binder positions. But pattern unification imposes an appendage limitation by taking unification variables into account."

A conjunction shows why the restriction has a reason to exist (dashed box, p. 3):

```
> (run 1 X
    (== (λ (a) X) (λ (b) b))
    (== (λ (c) X) (λ (d) d)))
```

"Associating X with a cannot work, neither does c. If we text-replace X with either, then the other one fails." The paper's first proposal is to stop treating λ-variables as names: "We first propose to treat unification variables and lambda variables differently. In particular, lambda variables are effectively constants if they are represented by de Bruijn levels" (p. 3).

Definition 9.3 (de Bruijn levels and indices [29, footnote 3, p. 4])**.** "There are two ways to implement de Bruijn's nameless syntax, indices and levels. Both use numbers to represent variables" [3]. A *level* counts binders from the *outside*: the outermost binder is 0. An *index* counts binders from the *inside*: the nearest binder is 0.

Example 9.4. In $\lambda c.\, \lambda d.\, c$, the variable c has level 0 and index 1, and d has level 1 and index 0. The term is $\lambda.\lambda.\, 0$ with levels and $\lambda.\lambda.\, 1$ with indices. The paper writes the level ℓ as $\mathtt{d}\ell$. So `d0` names the outermost enclosing binder, whatever it is called, and the answer `'(d0)` reads: "these two terms can be made α-equal, if X corresponds to de Bruijn level 0" (p. 3).

With levels, the two problematic queries succeed (solid boxes, p. 3):

```
> (run 1 X
    (== (λ (a) X) (λ (b) b)))
'(d0)
> (run 1 X
    (== (λ (a) X) (λ (b) b))
    (== (λ (c) X) (λ (d) d)))
'(d0)
```

"If we instantiate X with d0 and reify these levels w.r.t. their binders, we have the problem solved with no surprise": the two equations become `(== (λ (a) a) (λ (b) b))` and `(== (λ (c) c) (λ (d) d))`. The paper goes on (p. 3–4): "using grafting with de Bruijn levels captures the essence of α-equivalence: names don't matter. A unification algorithm based on de Bruijn levels would also be more straightforward because it's practically first-order. But, we cannot yet lift the scope restriction on lambda variables and drop in de Bruijn levels, because sometimes names do matter and …". The sentence continues in the title of §3.

Kanren angle: (aux kanren micro) already grafts

Grafting is what a first-order unifier does on nameless terms. Today's (aux kanren micro) unifies records slot by slot, so a `define-record` is a new term constructor for free. If we write λ-terms with de Bruijn indices as records, `=°` grafts. It reproduces the paper's answers, and also the problems discussed below. The demonstration uses indices, not levels; for the examples here only the outermost binder matters, and it is index 0 under one λ.

```
(import scheme (chicken base) (chicken pretty-print)
        (aux base) (aux stream) (aux kanren micro))

(define-syntax-rule (show label expr)
  (begin (display ";; ") (display label) (newline)
         (pretty-print expr)))

;; 1. Named binders as plain lists: syntactic unification ignores alpha.
(show "named: (λ a a) = (λ b b)"
  (°->list #f (fresh° (q) (=° '(λ a a) '(λ b b)))))

;; 2. Nameless binders as records with de Bruijn indices.
(define-record lam body)
(define-record dbi n)
(define (λ. t) (make-lam t))
(define (ix n) (make-dbi n))

(show "de Bruijn: λ.0 = λ.0"
  (°->list #f (fresh° (q) (=° (λ. (ix 0)) (λ. (ix 0))))))
(show "grafting: λa.X = λb.b"
  (°->list #f (fresh° (x) (=° (λ. x) (λ. (ix 0))))))
(show "grafting: λa.X = λb.b  and  λc.X = λd.d"
  (°->list #f (fresh° (x) (=° (λ. x) (λ. (ix 0)))
                          (=° (λ. x) (λ. (ix 0)))))))
(show "true = false: λa.λa.X = λc.λd.c"
  (°->list #f (fresh° (x) (=° (λ. (λ. x)) (λ. (λ. (ix 1))))))))
(show "dangling: λa.X = λb.b, then Q = X at top level"
  (°->list #f (fresh° (q) (fresh° (x) (=° (λ. x) (λ. (ix 0)))
                                      (=° q x)))))
```

```
;; named: (λ a a) = (λ b b)
()
;; de Bruijn: λ.0 = λ.0
((λ (α) α))
;; grafting: λa.X = λb.b
((λ () (make-record-instance 'dbi 0)))
;; grafting: λa.X = λb.b  and  λc.X = λd.d
((λ () (make-record-instance 'dbi 0)))
;; true = false: λa.λa.X = λc.λd.c
((λ () (make-record-instance 'dbi 1)))
```

```
;; dangling: λa.X = λb.b, then Q = X at top level
((λ () (make-record-instance 'dbi 0)))
```

The first query shows that named binders as plain data are useless: α-equivalent terms do not unify. The next three reproduce the paper's `'(_0)`, `'(d0)`, `'(d0)`. The fifth is the "true = false" problem of the next subsection. The answer `(dbi 1)` is perfectly correct for the nameless terms, but it denotes the *outer* binder. In the user's named term $\lambda a.\, \lambda a.\, X$ that binder is shadowed. The last query shows a *dangling* index: Q lives at top level, outside every binder, yet it is bound to "the nearest binder". Section 9.3 returns to it.

9.2.3 Binder-merging and "true = false"

Definition 9.5 (Binder-merging [29, p. 4])**.** "When two terms are unified, one term's binders are replaced by the other's. Then α-equivalence is as simple as structural equivalence." The paper says that it is "used in all pattern unification algorithms, except for those with de Bruijn numbers".

Example 9.6 (binder-merging at work, p. 4)**.**

$$\lambda x.\, \lambda y.\, X\, x\, y \stackrel{?}{=} \lambda a.\, \lambda b.\, a\, b \;\longrightarrow\; \lambda a.\, \lambda b.\, X\, a\, b \stackrel{?}{=} \lambda a.\, \lambda b.\, a\, b$$

This is solved by $\{X \mapsto \lambda a.\, \lambda b.\, a\, b\}$. Instantiating gives $\lambda a.\, \lambda b.\, ((\lambda a.\, \lambda b.\, a\, b)\, a\, b)$, which β-reduces to $\lambda a.\, \lambda b.\, a\, b$.

Now take a binder name that is reused, i.e. *shadowing* (dashed box, p. 4):

```
> (run 1 X
    (== (λ (a) (λ (a) X)) (λ (c) (λ (d) c))))
```

"With either binder-merging or de Bruijn syntax, the unification problem is effectively reduced to" $\lambda c.\, \lambda d.\, X \stackrel{?}{=} \lambda c.\, \lambda d.\, c$. "Then it appears to be solvable by $\{X \mapsto c\}$, or equivalently $\{X \mapsto \texttt{d0}\}$, that associates X with the outer-binder." Naive reification, which writes each level back as the name of its binder, then gives

```
(== (λ (a) (λ (a) a)) (λ (c) (λ (d) c)))
```

"which claims that the λ-encoded false is true!" With Church booleans true $= \lambda t.\, \lambda f.\, t$ and false $= \lambda t.\, \lambda f.\, f$, the left-hand side $\lambda a.\, \lambda a.\, a$ is α-equivalent to false, because the occurrence a refers to the inner binder. The right-hand side is true.

The paper argues that binder-merging goes wrong "even without lifting the scope restrictions":

$$\lambda x.\, \lambda y.\, x\, y \stackrel{?}{=} \lambda a.\, \lambda a.\, a\, a \quad \text{is converted to} \quad \lambda a.\, \lambda a.\, a\, a \stackrel{?}{=} \lambda a.\, \lambda a.\, a\, a,$$

"which then ends up with another false positive". The correct answer is failure: $\lambda a.\, \lambda a.\, a\, a \equiv_\alpha \lambda a.\, \lambda b.\, b\, b$ applies the *inner* variable to itself. The paper's claim: "binder-merging does not

handle the name shadowing problem. This is overlooked by all pattern unification algorithms. Algorithms that use what we call binder-merging state upfront that they don't consider name shadowing". Nipkow–Prehofer and Qian are cited as examples. The existing remedies are "a pre-processor that adds gensyms to lambda variables to make each individual unique or a smart reifier that detects name shadowing" (p. 4). Their downsides are unreadable gensyms and extra complexity. The related work gives a real case (p. 8): Agda uses de Bruijn numbers in its core language, so it "must make all variable names unique in a "gensym" manner", which "bloats type checking with randomly generated symbols and makes the error messages painful to read".

9.2.4 Static closures

"We propose a simple and efficient alternative, static closures. Static closures are integrated into the unification process and they decide α-equivalence while correctly handling name shadowing. We obtain a new algorithm by extending traditional pattern unification with static closures" (p. 5). No efficiency evidence or complexity bound is given.

Definition 9.7 (Terms, scopes, static closures [29, Fig. 1, p. 5])**.**

t	$::=$	X	unification variable
	$\mid$	$t\ t$	function application
	$\mid$	$\lambda a.\, t$	named binder
	$\mid$	a	name
	$\mid$	$\lambda\, t$	nameless binder (derived)
	$\mid$	ℓ	level (derived)
ϕ	$::=$	$\epsilon \mid a, \phi$	scope
κ	$::=$	$\langle \phi;\ t \rangle$	static closure

"The nameless syntax, i.e., nameless-binders and levels are derived during unification" (p. 5): the user writes only X, $t\,t$, $\lambda a.\, t$ and a. "A static closure is a pair of a scope and a unification term. A scope is a list of binders. In essence, a static closure captures the surrounding binders of a term. E.g., the term λa.λb.X reduces to the static closure $\langle a, b;\ X \rangle$."

Two remarks from our notes. First, the grammar has no constants, although the examples use them (`'cat`); read "name" as covering free constants too. Second, the grammar conses on the left (a, ϕ), but the example writes the scope of $\lambda a.\, \lambda b.\, X$ as a, b, outermost first. We adopt *outermost first*: then the level of a binder is its position in the scope.

Definition 9.8 (Operations on scopes [29, p. 5])**.** In the paper's words:

- "ext(ϕ, a) adds the name a to the scope ϕ. If a has been previously added to ϕ, then this older one is shadowed and its corresponding level becomes unusable."
- "db(ϕ, a) is Just ℓ where ℓ is the de Bruijn level of a w.r.t. the scope ϕ, if a is not shadowed in ϕ; otherwise, the result is Nothing."
- "db(ϕ, ℓ) is Just ℓ, if ℓ is available in ϕ; otherwise, the result is Nothing."

So db is overloaded. On a name it converts the name to a level; on a level it checks that the level is available. The notes make this precise. Write $\phi = a_0, \ldots, a_{n-1}$ outermost first:

$$\mathrm{ext}(\phi, a) = \phi, a \qquad \mathrm{avail}(\phi) = \{\ell < |\phi| \ : \ a_\ell \notin \{a_{\ell+1}, \ldots, a_{n-1}\}\}$$

$$\mathrm{db}(\phi, a) = \begin{cases} \mathsf{Just}\ \max\{\ell \ : \ a_\ell = a\} & a \in \phi \\ \mathsf{Nothing} & a \notin \phi \end{cases} \qquad \mathrm{db}(\phi, \ell) = \begin{cases} \mathsf{Just}\ \ell & \ell \in \mathrm{avail}(\phi) \\ \mathsf{Nothing} & \text{otherwise} \end{cases}$$

In words: a level is *available* if its binder's name is not rebound further in. The name-to-level case needs an interpretation. An *occurrence* of a name always refers to its innermost binder, which is never shadowed, so the paper's "if a is not shadowed" can only mean "the innermost a". The other reading would make $\lambda a.\, \lambda a.\, a$ impossible to convert, which is absurd. This remains an interpretation.

Tutorial: the three examples of §4, step by step

Example 1 (p. 5). Both sides are turned into closures:

$$\lambda a.\, \lambda b.\, X \stackrel{?}{=} \lambda c.\, \lambda d.\, c \ \longrightarrow\ \langle a, b;\, X\rangle \stackrel{?}{=} \langle c, d;\, c\rangle.$$

"We first apply db on the right-hand side": $\mathrm{db}((c, d), c) = \mathsf{Just}\ 0$, so the problem is $\langle a, b;\, X\rangle \stackrel{?}{=} \mathsf{Just}\ 0$. "Then we try to solve X by applying db on the scope of the left-hand side and 0": $\mathrm{db}((a, b), 0) = \mathsf{Just}\ 0$, because a is not rebound. Then $\mathsf{Just}\ 0 \stackrel{?}{=} \mathsf{Just}\ 0$. "It works. So, the solution is $\{X \mapsto \mathtt{d0}\}$."

Example 2 (p. 5–6), the shadowing case:

$$\lambda a.\, \lambda a.\, X \stackrel{?}{=} \lambda c.\, \lambda d.\, c \ \longrightarrow\ \langle a, a;\, X\rangle \stackrel{?}{=} \langle c, d;\, c\rangle.$$

As before, the right side gives $\mathsf{Just}\ 0$. "This time, $\mathrm{db}(\langle a, a; 0\rangle)$ is Nothing because the outer binder a is shadowed". Here the paper puts the 0 inside the closure brackets, an abuse of notation for $\mathrm{db}((a, a), 0)$. We get $\mathsf{Nothing} \stackrel{?}{=} \mathsf{Just}\ 0$, "and we know the unification fails". True is not false any more.

Example 3 (p. 6), two unknowns:

$$\lambda a.\, \lambda b.\, X \stackrel{?}{=} \lambda c.\, \lambda d.\, Y \ \longrightarrow\ \langle a, b;\, X\rangle \stackrel{?}{=} \langle c, d;\, Y\rangle.$$

"Here, the grafting itself is not able to describe the solution. Now our solution needs a context: the set of equations between static closures of unification variables. This problem has the solution: the grafting being empty and the context being $\{\langle a, b;\, X\rangle \stackrel{?}{=} \langle c, d;\, Y\rangle\}$." The paper adds that, in practice, "the accumulated equations between closures of one unification problem are usually solved and reduced to grafting, during the unification on other problems".

So a solution is a pair (θ, C): a grafting θ and a *context* C of postponed flex-flex equations between closures. This is analogous to the flex-flex pairs that Huet's procedure leaves unsolved (Chapter 7).

9.2.5 A reconstruction of the rules (not in the paper)

To check that the three examples fit together, we wrote a minimal rule system that reproduces all of them. It was implemented as a standalone CHICKEN program of about 150 lines, which does not use (aux kanren micro). We repeat that this is **our reconstruction, not the authors' algorithm**.

Level form. A surface term t under scope ϕ is converted to internal syntax $[\![t]\!]_\phi$:

$$[\![X]\!]_\phi = \langle \phi;\, X \rangle \qquad [\![a]\!]_\phi = \begin{cases} \ell & \mathrm{db}(\phi, a) = \mathsf{Just}\ \ell \\ a & \text{otherwise (a constant)} \end{cases}$$

$$[\![\lambda a.\, t]\!]_\phi = \lambda\, [\![t]\!]_{\mathrm{ext}(\phi,a)} \qquad [\![t\ u]\!]_\phi = [\![t]\!]_\phi\ [\![u]\!]_\phi$$

Both sides of an equation are converted from ϵ, so they are always compared at equal depth. A binding is $\theta(X) = (\phi_0, t)$, where t is in level form at depth $|\phi_0|$. Levels below $|\phi_0|$ are *free* (scope) levels; the others are bound inside t. To *instantiate* $\theta(X)$ at an occurrence $\langle \phi;\, X \rangle$:

- a free level ℓ must satisfy $\mathrm{db}(\phi, \ell) = \mathsf{Just}\ \ell$, or else the occurrence fails;
- a local level $\ell \geq |\phi_0|$ is shifted to $\ell + |\phi| - |\phi_0|$;
- a nested closure is re-rooted at ϕ.

Rules. The judgement is $\theta; C \vdash t \stackrel{?}{=} u \Rightarrow \theta'; C'$. Terms are walked first, i.e. a bound closure is replaced by its instance. Any pair of head constructors not listed fails.

$$\frac{}{\theta; C \vdash \ell \stackrel{?}{=} \ell \Rightarrow \theta; C}\ \textsc{Lvl} \qquad \frac{c \text{ a constant}}{\theta; C \vdash c \stackrel{?}{=} c \Rightarrow \theta; C}\ \textsc{Const} \qquad \frac{\theta; C \vdash t \stackrel{?}{=} u \Rightarrow \theta'; C'}{\theta; C \vdash \lambda t \stackrel{?}{=} \lambda u \Rightarrow \theta'; C'}\ \textsc{Lam}$$

$$\frac{\theta; C \vdash t_1 \stackrel{?}{=} u_1 \Rightarrow \theta_1; C_1 \qquad \theta_1; C_1 \vdash t_2 \stackrel{?}{=} u_2 \Rightarrow \theta_2; C_2}{\theta; C \vdash t_1\, t_2 \stackrel{?}{=} u_1\, u_2 \Rightarrow \theta_2; C_2}\ \textsc{App}$$

$$\frac{X \notin \operatorname{dom}\theta \qquad FL_{<|\phi|}(t) \subseteq \operatorname{avail}(\phi) \qquad X \notin FV(t) \qquad \theta[X \mapsto (\phi, t)]; \emptyset \vdash C \Rightarrow \theta'; C'}{\theta; C \vdash \langle \phi;\, X\rangle \stackrel{?}{=} t \Rightarrow \theta'; C'}\ \textsc{Graft}$$

$$\frac{}{\theta; C \vdash \langle \phi;\, X\rangle \stackrel{?}{=} \langle \phi;\, X\rangle \Rightarrow \theta; C}\ \textsc{Refl}$$

$$\frac{X, Y \notin \operatorname{dom}\theta \qquad (X, \phi) \neq (Y, \psi)}{\theta; C \vdash \langle \phi;\, X\rangle \stackrel{?}{=} \langle \psi;\, Y\rangle \Rightarrow \theta; C \cup \{\langle \phi;\, X\rangle \stackrel{?}{=} \langle \psi;\, Y\rangle\}}\ \textsc{Postpone}$$

In Graft, t is not an unbound closure. $FV(t)$ is the set of unification variables of t (the occurs check). $FL_{<|\phi|}(t)$ is the set of levels below $|\phi|$ that occur in t: those that point into the scope ϕ rather than to a nameless binder inside t. The availability premise says that a solution may mention only scope binders that are visible, i.e. not shadowed, at the occurrence of X. The premise "$\vdash C$" means that every postponed equation is solved again, the *wake-up*. Example 2 fails in the availability premise of Graft: $FL(0) = \{0\} \not\subseteq \operatorname{avail}(a, a) = \{1\}$.

Tutorial: a wake-up, step by step

Take Example 3 and then bind Y, in two variants. Both start from $\lambda a.\, \lambda b.\, X \stackrel{?}{=} \lambda c.\, \lambda d.\, Y$ (first variant) or $\lambda a.\, \lambda a.\, X \stackrel{?}{=} \lambda c.\, \lambda d.\, Y$ (second variant).

1. Conversion gives $\lambda\lambda\langle a, b;\, X\rangle \stackrel{?}{=} \lambda\lambda\langle c, d;\, Y\rangle$. Lam twice, then Postpone: $\theta = \emptyset$, $C = \{\langle a, b;\, X\rangle \stackrel{?}{=} \langle c, d;\, Y\rangle\}$. This is the line `(X ctx: 1)` below.
2. The second equation $\lambda c.\, \lambda d.\, Y \stackrel{?}{=} \lambda c.\, \lambda d.\, c$ converts to $\lambda\lambda\langle c, d;\, Y\rangle \stackrel{?}{=} \lambda\lambda\, 0$. Graft checks $FL(0) = \{0\} \subseteq \operatorname{avail}(c, d) = \{0, 1\}$ and binds $Y \mapsto ((c, d), 0)$.
3. Wake-up: C is solved again. Walking $\langle c, d;\, Y\rangle$ instantiates Y's solution at (c, d), giving 0. The equation becomes $\langle a, b;\, X\rangle \stackrel{?}{=} 0$, and Graft checks $0 \in \operatorname{avail}(a, b)$. It holds, so $X \mapsto ((a, b), 0)$, printed `d0`.
4. In the second variant the same wake-up yields $\langle a, a;\, X\rangle \stackrel{?}{=} 0$, but $\operatorname{avail}(a, a) = \{1\}$,

so the whole conjunction fails.

The second variant is “true = false” again, smuggled through the context. A wake-up that only re-unified, without the availability check, would accept it.

Running the prototype gives the following results. In `(λ d1)` the level is printed absolute, at the depth where X was solved.

```
X = cat                              => (cat ctx: 0)
λa.a = λb.b                          => (X ctx: 0)
λa.X = λb.b                          => (d0 ctx: 0)
λa.X = λb.b ∧ λc.X = λd.d            => (d0 ctx: 0)
λa.λa.X = λc.λd.c  (true=false)      => FAIL
λx.λy.(x y) = λa.λa.(a a)            => FAIL
Ex1 λa.λb.X = λc.λd.c                => (d0 ctx: 0)
Ex2 λa.λa.X = λc.λd.c                => FAIL
Ex3 λa.λb.X = λc.λd.Y                => (X ctx: 1)
Ex3 then λc.λd.Y = λc.λd.c           => (d0 ctx: 0)
λa.λa.X = λc.λd.Y then Y = d0        => FAIL
λa.X = λb.λc.c ; λa.λb.X = λ_.λ_.λc.c => ((λ d1) ctx: 0)
λa.X = λb.λc.b                       => ((λ d0) ctx: 0)
λa.X = λb.b ∧ X = cat                => FAIL
λa.X = λb.b ∧ λa.λa.X = λc.λd.d      => FAIL
λa.λb.X = λa.λb.a ∧ λb.λa.X = λc.λd.c => (d0 ctx: 0)
```

Three lines are worth reading closely. In “λa.λa.X = λc.λd.Y then Y = d0”, the wake-up must re-run the availability check, or the shadowing bug comes back through the context. In “λa.X = λb.λc.c ; ...”, a binder *inside* a grafted solution must be shifted when the solution is used at a different depth. The last line shows that the semantics is *positional*: the binder names above X are irrelevant as long as no level is shadowed.

9.2.6 The language: Horn clauses, `assume`, `all`

§5 of the paper (p. 6–7) recalls the logic from the 2020 paper [30]. Its Fig. 2 contrasts Horn clauses with hereditary Harrop formulas. In the figure the shared rows are $D ::= G \supset D \mid D \wedge D \mid \forall x\, D$ and $G ::= G \wedge G \mid G \vee G \mid \exists x\, G$, and the HH-only rows are $D \supset G$ and $\forall x\, G$ for goals. The atoms A and $\top$ are missing from the figure. With them it is exactly the book’s grammar (3.1) [MN p. 91]:

$$G ::= \top \mid A \mid G \wedge G \mid G \vee G \mid \exists x\, G \mid D \supset G \mid \forall x\, G \qquad D ::= A \mid G \supset D \mid D \wedge D \mid \forall x\, D.$$

The four code boxes are these (all solid, p. 6–7):

```
> (run* (X Y)
    (conde
      [(== X Y) (== X 'cat)]
      [(== X 'cat) (== Y 'dog)]))
'((cat cat) (cat dog))
```

```
(define (empty? X)
  (== X '()))

>(run 2 X
   (assume (== 'apple 'orange)
     (== 'apple X)))
'(apple orange)

>(run 1 F
   (all (A)
     (== A (F A))))
'((lambda (_0) _0))
```

The paper prints `conde` with a superscript *e*. The first box is $\exists X \exists Y((X = Y \wedge X = \texttt{'cat}) \vee (X = \texttt{'cat} \wedge Y = \texttt{'dog}))$. The `define` is the clause $\forall X\,(X = \texttt{'()}) \supset (\texttt{empty?}\ X)$. `assume` "introduces hypotheses in its clause. As an example, we may use the assume operator to temporarily extend the == relation". Our reading, inferred because the paper gives no operational account: the first answer comes from ordinary unification, and the second from the hypothesis used as a clause whose head matches `(== 'apple X)` with $X = \texttt{orange}$. For `all`, "we may synthesize a function that yields whatever it takes in". Also, "the == operator recognizes β-equivalence. Indeed, it will be powered by the unification algorithm proposed in the last section and recognize αβ-equivalence" (p. 7). Note that `(F A)` is a Miller pattern (Chapter 7): a variable applied to a distinct eigenvariable.

9.3 A critical assessment of static closures

The paper attacks pattern unification. The fair comparison also includes nominal unification (Section 9.5), which the paper mentions only in related work. This section gives our own analysis. In the λProlog column, X (and Y) are quantified outside the λ, as in the paper's queries.

Table 9.1: The paper's examples under three semantics (our analysis, after [29]).

problem	λProlog (patterns)	αKanren (nominal)	static closures
$\lambda a.\, a \stackrel{?}{=} \lambda b.\, b$	✓	✓	✓
$\lambda a.\, X \stackrel{?}{=} \lambda b.\, b$	✗ (scope)	✓ $X = a$	✓ $X = \texttt{d0}$
$\ldots \wedge \lambda c.\, X \stackrel{?}{=} \lambda d.\, d$	✗	✗ ($a \neq c$)	✓ $\texttt{d0}$
$\lambda a.\, \lambda a.\, X \stackrel{?}{=} \lambda c.\, \lambda d.\, c$	✗	✗ (needs $a\#a$)	✗
$\lambda x.\, \lambda y.\, x\ y \stackrel{?}{=} \lambda a.\, \lambda a.\, a\ a$	✗	✗	✗
$\lambda a.\, \lambda b.\, X \stackrel{?}{=} \lambda c.\, \lambda d.\, Y$	✓ $X = Y$ (neither may depend on a, b, c, d)	$X = (a\,c)(b\,d) \cdot Y$ with $a\#Y$, $b\#Y$	postponed

Tutorial: what λProlog offers the novice instead: make the dependency explicit

The paper calls the failure of $\lambda a.\, X \stackrel{?}{=} \lambda b.\, b$ counter-intuitive. Miller's 1991 paper [34, p. 4–5] explains why it is the intended behaviour, and how to get the novice's answer anyway. Its program is the usual `append`, and its goal is $\forall y\,(\texttt{append}\ (\texttt{cons}\ a\ (\texttt{cons}\ b\ \texttt{nil}))\ y\ X)$. GENERIC replaces y by a new constant k, and backchaining twice would need $X = \texttt{cons}\ a\ (\texttt{cons}\ b\ k)$. This is rejected, and Miller says why: "Such a failure here is quite sensible since the value of X should be independent of the choice of the constant used to instantiate $\forall y$." If we do want a value that depends on y, we must say so, by writing $(H\ y)$ instead of X. Then $(H\ k)$ must equal $\texttt{cons}\ a\ (\texttt{cons}\ b\ k)$. Two λ-terms would do, $\lambda w.\,\texttt{cons}\ a\ (\texttt{cons}\ b\ k)$ and $\lambda w.\,\texttt{cons}\ a\ (\texttt{cons}\ b\ w)$, but "since H cannot contain k free, only the second" is legal. Miller calls this "discharging the constant k".

The same move answers the novice. Write $\lambda a.\,(X\ a) \stackrel{?}{=} \lambda b.\, b$: this is a pattern ($X$ applied to one bound variable), and its most general unifier is $X \mapsto \lambda x.\, x$. The conjunction with $\lambda c.\,(X\ c) \stackrel{?}{=} \lambda d.\, d$ is solved by the same X, and a top-level $Q = X$ gets the closed term $\lambda x.\, x$, with nothing dangling. Both λKanren papers show this style of query: `(== (tie (a b) (app a b)) (tie (x y) (app f x y)))` of 2020 and `(all (A) (== A (F A)))` of 2021 (Sections 9.2.6 and 9.4). Static closures make this dependency *implicit*: $\langle \phi;\, X \rangle$ behaves like X applied to the available levels of ϕ (observation 1 below).

9.3.1 Three counter-examples

The notes isolate three examples that pin down what static closures really are. They are worth keeping as regression tests for any implementation, whatever it chooses.

Tutorial: Counter-example 1: the conjunction separates static closures from nominal unification

Problem: $\lambda a.\, X \stackrel{?}{=} \lambda b.\, b \ \wedge\ \lambda c.\, X \stackrel{?}{=} \lambda d.\, d$.
Nominal, using the rule $a.t \approx^{?} b.s \Rightarrow t \approx^{?} (a\,b) \cdot s,\ a\#^{?}s$ of Section 9.5. The first equation gives $X \approx^{?} (a\,b) \cdot b = a$ with $a\#b$, which holds, so $X := a$. The second becomes $c.a \approx^{?} d.d$, i.e. $a \approx^{?} (c\,d) \cdot d = c$, which fails. Nominal unification thus agrees with static closures on the single equation (X is the bound name) and disagrees on the conjunction.
Pattern: both equations fail at once, by the scope restriction.
Static closures: $X \mapsto \texttt{d0}$ satisfies both.
So static closures are *neither* pattern unification *nor* nominal unification. X denotes an open term *relative to the binder depth*. This is grafting on de Bruijn terms *without* the "pre-cooking" step of Dowek et al. (Section 9.7.3).

Tutorial: Counter-example 2: the same variable at two depths dangles

Problem: `(== Q X)` together with `(== (λ (a) X) (λ (b) b))`.
Under static closures, $X \mapsto$ `d0`, so the top-level answer is $Q =$ `d0`. That is a level which is *not available* in the empty scope where Q lives. The answer is meaningful only relative to the occurrence of X under one binder. The reading becomes incoherent as soon as the same variable occurs at different depths. Our (aux kanren micro) run above shows the same thing with indices. The prototype chooses to check availability inside unification, so "λa.X = λb.b then top-level Y = X" fails there. But it skips the check when printing, which is how the paper can print `'(d0)`. Dowek et al. fix this with $X[\uparrow^n]$; Teyjus and ELPI fix it with levels on variables. The 2021 paper does not address it. It also gives no rules for application and β, nor for flex-flex closures beyond postponing them.

Tutorial: Counter-example 3: "true = false" is a bug of binder-merging, not of pattern unification

Problem: $\lambda a.\, \lambda a.\, X \stackrel{?}{=} \lambda c.\, \lambda d.\, c$.
Pattern: X cannot depend on the bound variables at all, so this fails.
Nominal: $a.(a.X) \approx^? c.(d.c)$ gives $a.X \approx^? (a\,c) \cdot (d.c) = d.a$, with $a\#(d.c)$, which holds. Then $a.X \approx^? d.a$ gives $X \approx^? (a\,d) \cdot a = d$, with $a\#a$, which is false. So it fails.
Static closures: fails (Example 2).
All three agree. The "false = true" danger is specific to *binder-merging* implementations that do not rename apart. The earlier λKanren paper renames apart: its ξ step uses gensyms (Section 9.4), so it does not have the bug.

9.3.2 Further observations

The notes add several observations that a porter must know. The paper does not say them.

1. **The meaning of a unification variable changes.** In λProlog, X in $\lambda a.\, X$ cannot mention a. With static closures, X implicitly depends on the *positions* of its enclosing binders. Logically, $\langle \phi; X \rangle$ behaves like Miller's raising $X\, \ell_{i_1} \cdots \ell_{i_k}$ over the available levels of ϕ, with one twist: the "function" is indexed by absolute position, so occurrences under scopes of different lengths share it. The paper's related work quotes Huet's "elementary substitution": "Each unification variable should capture all its visible bound variables" (p. 8). Static closures make that idea implicit, minus the shadowed binders.

2. **α-equivalence is no longer a congruence on open terms.** $\lambda a.\, \lambda a.\, X$ and $\lambda a.\, \lambda b.\, X$ are α-equivalent in the usual sense, because X is a metavariable with no free names. Yet against the same $\lambda c.\, \lambda d.\, c$, Example 2 fails and Example 1 succeeds. With grafting, the names of the binders above a metavariable matter. Nominal logic manages exactly this with suspended permutations $\pi \cdot X$ and freshness $a\#X$. The paper's "names don't matter" holds for the levels, not for the closures.

3. **Levels are stable for scope variables, not for binders inside solutions.** A grafted $\lambda.\ell_1$ solved at depth 1 must become $\lambda.\ell_2$ at depth 2. A locally nameless representation avoids the shift [31, 6]: levels for scope variables, indices for binders inside solutions.

4. **Flex-flex need not be postponed.** With levels, $\langle \phi;\ X\rangle \stackrel{?}{=} \langle \psi;\ Y\rangle$ at equal depth has a most general solution: $X \mapsto Y$, plus the restriction that Y may use only levels in $\text{avail}(\phi) \cap \text{avail}(\psi)$. This is Miller-style pruning, phrased as a freshness-like constraint.

5. **The trick must not reach `all`.** For ∀-goals the eigenvariable must respect the scope restriction, or `(run 1 X (all (A) (== X A)))` would succeed. That is unsound for ∀. The level semantics is meant only for λ-binders inside terms.

6. **β brings undecidability back.** Once `==` "recognizes αβ-equivalence", unification outside the pattern fragment is undecidable (Chapter 7). One must restrict to patterns, delay, or enumerate.

The paper's related work (p. 8) recalls that "nominal unification and pattern unification are equally expressive". Cheney reduces pattern unification to nominal unification [7], and Levy and Villaret give the converse reduction [28]. Pattern unification is linear (Qian, although the paper notes that Qian's algorithm "has not been implemented"); for nominal unification "$O(n^2)$ is the state-of-the-art" (p. 8). The notes summarize the correspondence like this: the static closure $\langle \phi;\ X\rangle$ is close to the nominal suspension $\pi \cdot X$ (both record the context around an unknown); db is close to the action of π on names; and "level unavailable in ϕ" is close to the freshness $\ell \# X$. But static closures are *positional* where nominal terms are *name-based*, as counter-example 1 shows. The design is closer to de Bruijn-level "contextual" metavariables than to nominal logic.

Kanren angle: what static closures would demand of a miniKanren

The notes list what a static-closure layer on (aux kanren micro) would need. This is a requirements list, not a design (see Chapter 11):

- **New term kinds, dispatched early.** λ, level and closure records must be handled *before* the generic record branch. Today `µkanren-structure?` matches any record instance and unifies it slot by slot, names included. The same applies to every walker: the occurs check, `µkanren-state-find/value`, reification, `µkanren-hasvar?`, and so on.

- **Conversion at the boundary.** Converting both sides of `=°` to level form under ϵ leaves a unifier that is "practically first-order" except for closures.

- **A walk that can fail.** Walking $\langle \phi;\ X\rangle$ instantiates X at ϕ, which can fail on an unavailable level. So the walk needs a third outcome besides "value" and "unbound". The first-order hot path should stay untouched.

- **A context store with wake-up.** The postponed equations must be re-checked when their variables get bound. The `µkanren-verify-D/touched` idiom, which re-

runs only the entries that mention a just-bound variable, already does this for disequalities.

- **Disequality.** With postponement, a tentative unification inside `≠°` may succeed by only adding a context equation, producing no bindings. An empty prefix in D means "already equal", so this would be wrong. A disequality entry would then have to carry its postponed part as well.

9.4 λKanren 2020: Miller's algorithm in a shallow embedding

The earlier paper, *Higher-order Logic Programming with λKanren* [30], is the "[17]" of the 2021 paper. Its workshop title is "λKanren: Higher-order Logic Programming with Shallow Embedding". It adds four operators to μKanren [22]: `tie` (a binder), `app` (application), `assume-rel` (an implication goal $D \supset G$) and `all` (a universal goal $\forall x\, G$). In it, `==` "recognizes αβ-conversions". The paper claims "about 40 lines" added to μKanren (p. 7); the repository's `unify.rkt` has 648 lines.

Terms and state. The terms are *Term* ::= *Constant* | *Var* | App(*rator*, *rand*) | Tie(*var*, *body*) | (*Term* . *Term*). Pairs stay a separate first-order constructor. The state is the list `(qs es σ name->idx)`. Here `qs` is an **explicit quantifier prefix** of `(∃ x)` and `(∀ x)` entries, appended at the end by `fresh` and `all`. `es` holds postponed equations and σ is the substitution.

The unifier is a direct transcription of Miller 1991 §6 [34]: ξ, rigid-rigid, raising, pruning, flex-flex and flex-rigid (Section 9.7.1). Its ξ step strips the `Tie`s on both sides and renames both binder lists to the same *fresh* gensyms by capture-avoiding substitution. It then pushes a `(∀ w)` onto the prefix for each name. So shadowing is handled by renaming apart, and the 2021 problem does not arise. Non-pattern applications are **not delayed**; they make the unifier fail.

```
(define-syntax tie
  (syntax-rules ()
    [(_ () t) t]
    [(_ (x0 x ...) body)
     (let ([x0 (Var 'x0)])
       (Tie x0 (tie (x ...) body)))]))

(define-syntax app
  (syntax-rules ()
    [(_ rator rand) (App rator rand)]
    [(_ rator rand0 rand ...)
     (app (App rator rand0) rand ...)]))
```

(The paper prints `[(_ () t) body]`; the repository has `t`.) The paper reports these unification queries (p. 2–3):

```
(run 1 q (== (tie (a b) (app a b)) (tie (x y) (app x y))))      ; ⇒ '(_0)
(run 2 f (== (tie (a b) (app a b)) (tie (x y) (app f x y))))    ; ⇒ '((tie (_0) (tie
↪  (_1) (app _0 _1))))
(run 1 f (== (tie (a) a) (tie (b) (app f b b))))              ; ⇒ '()   not a pattern
(run 1 q (== (tie (a b) (app a b)) (tie (x y) (app x q))))      ; ⇒ '()   scope
↪  restriction
```

The second query is a pattern, so `run 2` still finds one answer, the most general unifier: "There is only one instantiation" (p. 3). Binders and unification variables are both reified as *_n* symbols. The third is not a pattern (f is applied to b twice) and fails, even though it has solutions ($\lambda x.\, \lambda y.\, x$ and $\lambda x.\, \lambda y.\, y$); in the paper's words, "we cannot decide which b takes control". The fourth fails by the scope restriction, which the paper phrases as visibility: "A binding variable x is visible to a unification variable q if the introduction of x lexically precedes that of q" (p. 3). Here q would have to be the bound y, which is introduced after q.

Implication and the shallow-embedding problem. The motivating program is a curriculum:

```
(defrel (taken name class)
  (conde
    [(== 'Josh name) (== 'B521 class)]
    [(== 'Josh name) (== 'B522 class)]))
(defrel (pl-major name)
  (taken name 'B521)
  (taken name 'B523)
  (taken name 'B522))
> (run 1 q
    (assume-rel [(taken name class) (== 'Josh name) (== q class)]
      (pl-major 'Josh)))
'(B523)
```

The hypothetical clause extends `taken`, and `pl-major`, which was defined *before*, must see it. In a shallow embedding, relation definitions live in the host language's lexical scope, so `pl-major` cannot see the new clause. The paper describes a **dynamic-scope layer**: a map `name->idx` threaded through the query, a global `idx->def`, and calls that go through both. (Our notes write `(== q class)`; the extracted text has a quote, `'q`, which could not produce `B523`.) Concretely, before the query `name->idx` is `((pl-major . 2) (taken . 1))`; `assume-rel` pushes a new index, so inside it the map is `((taken . 3) (pl-major . 2) (taken . 1))`, and the newer pair shadows the older one when `pl-major` calls `taken` (p. 5).

The paper also offers `assume`, "a shorter version of the assume-rel operator" that "only takes an atomic hypothesis" (p. 5), and uses it on a relation `eq` that is reflexive, transitive and symmetric. Plain `(run 1 q (eq 'apple 'orange))` does not terminate in μKanren, because the recursive clauses never run out (p. 5). With hypotheses (p. 5–6):

```
(defrel (eq x y)
  (conde
    [(== x y)]
```

```
    [(fresh (z) (eq x z) (eq z y))]
    [(eq y x)]))
> (run 5 q (assume (eq 'orange 'apple) (assume (eq 'orange 'dog) (eq 'orange q))))
'(dog apple orange orange dog)
> (run 1 q (assume (eq 'orange q) (eq 'apple 'orange)))
'(apple)
```

In the first query the answers come from the two hypotheses (`dog`, `apple`) and from reflexivity (`orange`); `run 5` reports repetitions, because the recursive clauses derive the same facts again along different paths. The second query "runs backward": "the hypothesis can be inferred as well" (p. 6). The hypothesis `(eq 'orange q)` mentions the query variable q, and it is *not* renamed when it is used. The goal `(eq 'apple 'orange)` can use the symmetry clause to become `(eq 'orange 'apple)`, which matches the hypothesis with $q =$ `apple`. This is the book's semantics for a clause with a free logic variable (Chapter 3): only a clause's own ∀-bound variables are freshened per use.

Universal goals. "the all operator effectively creates a constant ... we create an all variable as a free binding variable. Thus, the all variable cannot be unified with anything but itself" (p. 6). The scope restriction is enforced by the prefix. The examples (p. 6–7):

```
> (run 1 q (all (x)
             (assume-rel (taken x 'B521)
               (assume-rel (taken x 'B522)
                 (assume-rel (taken x 'B523)
                   (pl-major x))))))
'(_0)
> (run 1 f (all (x) (== x (app f x))))
'((tie (_0) _0))
(defrel (proved x) (== x 'trivial))
> (run 1 g (all (p) (proved p)))            '()
> (run 1 g (proved g))                      '(trivial)
> (run 1 g (all (p q) (assume ((proved p) (proved q)) (proved q) (proved p))))   '(_0)
> (run 1 goal (all (p q) (assume ((proved p)) (conde [(proved p)] [(proved q)]))))
 ↪  '(_0)
```

The query `(all (p) (proved p))` shows GENERIC at work. `proved` holds only of `trivial`, and a generic p is not `trivial`, so $\forall p.$ `proved` p is not provable, whereas the same goal without `all` succeeds with $g =$ `trivial`. The last two queries use λKanren "as a theorem prover" (§5, p. 6–7). The first proves the commutativity of conjunction, $\forall p, q\,(p \wedge q) \supset (q \wedge p)$: GENERIC makes p and q new constants, AUGMENT adds `(proved p)` and `(proved q)` as hypotheses, and the two conjuncts of the body are then proved by backchaining on them. The second proves disjunction introduction, $\forall p, q\ p \supset (p \vee q)$, by the first `conde` branch. The answers `'(_0)` only say that the query variable was left unconstrained: the information is in the success.

Pitfall: hypotheses in a host dynamic parameter get lost

The repository's active code does not use the described state-threaded design. It uses Racket parameters: `defrel` makes each body a `make-parameter`, and `assume` is a `parameterize` that extends it. We reproduced this structure in CHICKEN. The body of `pl-major` is only *constructed* when the goal is applied to a state, which happens after `parameterize` has returned. So the inner call to `taken` reads the *original* parameter, and the output was `(orig Josh B523)`. Relation calls nested inside other relations' delayed bodies therefore lose the hypotheses. The described `name->idx` design does not have this problem.

Kanren angle: hypotheses belong to the state

(aux kanren micro)'s `define-relation` is inverse-η-delayed: it expands to `(λ (s) (δ ((and° g ...) s)))`. This is exactly the situation of the experiment above. A hypothetical clause database for AUGMENT must therefore be a *field of the state*, pushed on entry to G and invisible outside it. It can never be a host parameter. A further remark from the notes: a hypothetical clause $\forall \bar{x}(G \supset A)$ can be a host closure whose body runs `fresh°`. Each use then renames the clause's own variables, while the variables captured from the enclosing scope stay shared. This is exactly λProlog's semantics for clauses with free logic variables. How the module does this is shown in Section 12.4.

9.5 Nominal unification and αKanren

Nominal unification [58] takes a different route to α-equivalence. Bound names stay names, called *atoms*. Metavariables are holes, and substitution of a term for a metavariable is *capturing*, i.e. grafting. Capture is controlled by two devices: *permutations* suspended on variables, and *freshness* constraints. Our presentation of the rules follows Urban's survey [57].

Definition 9.9 (Nominal terms and permutations)**.**

$$t ::= \langle\rangle \mid \langle t_1, t_2\rangle \mid f\, t \mid a \mid a.t \mid \pi \cdot X$$

where $a.t$ abstracts the atom a in t, and $\pi \cdot X$ is a variable with a suspended permutation. A permutation π is a list of swappings $(a\, b)$:

$$[] \cdot a = a, \qquad ((a_1\, a_2) :: \pi) \cdot a = \begin{cases} a_2 & \pi \cdot a = a_1 \\ a_1 & \pi \cdot a = a_2 \\ \pi \cdot a & \text{otherwise.} \end{cases}$$

Composition is list append and the inverse is reversal. The *disagreement set* is $ds(\pi, \pi') = \{a \mid \pi \cdot a \neq \pi' \cdot a\}$. A permutation acts on an abstraction on *both* the binder and the body, and it suspends on variables: $\pi \cdot (\pi' \cdot X) = (\pi @ \pi') \cdot X$. Substitution is capturing: $\sigma(a.t) = a.\sigma(t)$ with no side condition, and $\sigma(\pi \cdot X) = \pi \cdot \sigma(X)$.

Definition 9.10 (Freshness and α-equality [57, Fig. 1])**.** With a freshness context ∇, a set of pairs $a\#X$:

$$\frac{}{\nabla \vdash a\#\langle\rangle} \qquad \frac{\nabla \vdash a\#t_1 \qquad \nabla \vdash a\#t_2}{\nabla \vdash a\#\langle t_1, t_2\rangle} \qquad \frac{\nabla \vdash a\#t}{\nabla \vdash a\#f\,t} \qquad \frac{}{\nabla \vdash a\#a.t} \qquad \frac{\nabla \vdash a\#t \qquad a \neq b}{\nabla \vdash a\#b.t}$$

$$\frac{a \neq b}{\nabla \vdash a\#b} \qquad \frac{(\pi^{-1} \cdot a, X) \in \nabla}{\nabla \vdash a\#\pi \cdot X}$$

$$\frac{\nabla \vdash t_1 \approx t_2}{\nabla \vdash a.t_1 \approx a.t_2} \qquad \frac{a \neq b \qquad \nabla \vdash a\#t_2 \qquad \nabla \vdash t_1 \approx (a\,b) \cdot t_2}{\nabla \vdash a.t_1 \approx b.t_2} \qquad \frac{\forall c \in ds(\pi, \pi').\ (c, X) \in \nabla}{\nabla \vdash \pi \cdot X \approx \pi' \cdot X}$$

plus the congruence rules for unit, pairs, function symbols and atoms.

The intuition: $a\#t$ says that a does not occur *free* in t, also after any later instantiation of the variables of t. To compare $a.t_1$ with $b.t_2$, rename b to a in t_2 by a *swapping*, which is always safe because it is a bijection, and require that a was not already free in t_2.

Definition 9.11 (Nominal unification [57, p. 9])**.**

$$\{\langle t_1, t_2\rangle \approx^? \langle s_1, s_2\rangle, \dots\} \Rightarrow \{t_1 \approx^? s_1,\ t_2 \approx^? s_2, \dots\} \qquad \{a.t \approx^? a.s, \dots\} \Rightarrow \{t \approx^? s, \dots\}$$

$$\{a.t \approx^? b.s, \dots\} \Rightarrow \{t \approx^? (a\,b) \cdot s,\ a\#^? s, \dots\} \qquad \{\pi \cdot X \approx^? \pi' \cdot X, \dots\} \Rightarrow \{a\#^? X \mid a \in ds(\pi, \pi')\} \cup \{\dots\}$$

and $\pi \cdot X \approx^? t$, with X not in t, gives $X := \pi^{-1} \cdot t$. A solution is a pair (∇, σ) with $\nabla \vdash \sigma(t_1) \approx \sigma(t_2)$. For example, $a.X \approx^? b.X$ has the solution $(\{a\#X,\ b\#X\}, [\,])$.

Most general solutions exist, and **no fresh names are ever generated**: every atom of a solution occurs in the problem. Naive rules are exponential; quadratic algorithms exist.

9.5.1 αKanren

αKanren [5, 4] embeds nominal unification in miniKanren. `fresh` introduces *noms*: "a nom behaves more like a constant than a variable, since it only unifies with itself or with an unassociated variable". The constructor `(tie a t)`, typeset ⋈, binds the nom a in t; it delimits scope and does not create noms. `(# a t)`, entered `hash`, asserts that a does not occur free in t. Ordinary logic variables are introduced by `exist`. The thesis examples (ch. 9; ◃▹ is the thesis glyph for tie):

```
(run* (q) (fresh (a b) (≡ (◃▹ a a) (◃▹ b b))))      ⇒ (_.0)
(run* (q) (fresh (a b) (≡ (◃▹ a q) (◃▹ b b))))      ⇒ (a.0)
(run* (q) (fresh (a b) (# a (◃▹ b a))))             ⇒ ()
(run* (q) (fresh (a) (# a (◃▹ a a))))               ⇒ (_.0)
(run* (q) (fresh (a b) (exist (x y) (≡ (◃▹ a (◃▹ a x)) (◃▹ a (◃▹ b y))) (≡ `(,x ,y)
↪  q))))
  ⇒ ((((susp ((a.0 a.1)) _.0) _.0) : ((a.0 . _.0))))
```

The second line is counter-example 1's single equation: q becomes the bound nom. The last line shows a suspension in an answer, with a freshness constraint printed after the colon. The capture-avoiding substitution relation shows the programming style:

```
(define substo
  (λ (e new a out)
    (matche (e out)
      (((var ,a) ,new))
      (((var ,y) (var ,y)) (# a y))
      (((app ,rator ,rand) (app ,rator-res ,rand-res))
       (substo rator new a rator-res)
       (substo rand new a rand-res))
      (((lam ,(◃▹ @c body)) (lam ,(◃▹ c body-res)))
       (# c a)
       (# c new)
       (substo body new a body-res)))))
(run* (q) (fresh (a b) (substo `(lam ,(◃▹ a `(app (var ,a) (var ,b)))) `(var ,b) a
 ↪  q)))
  ⇒ ((lam (tie a.0 (app (var a.0) (var a.1)))))     ; [b/a]λa.ab = λc.cb
```

(`@c` in a `matche` pattern introduces c as a nom.) The unifier works in two phases. First the σ-rules turn equations into a substitution plus pending freshness constraints. Then the ∇-rules reduce freshness constraints until each is $a\#x$ with x an unbound variable. Variables are suspensions with an empty permutation. A triangular-substitution version, due to Joe Near, is reported "as much as ten times faster". The thesis states a limitation (p. 178): "neither currently supports disequality constraints". αKanren also powers αleanTAP [47], where ∀-bound variables of the object logic are noms. There, "Universally quantified formulas are used as templates, rather than instantiated directly".

Kanren angle: what nominal unification asks of a miniKanren

It needs three new term kinds (nom, `tie`, suspension $\pi \cdot X$) and a freshness store ∇ of $a\#X$ pairs, re-reduced after each unification. That re-reduction works like (aux kanren micro)'s post-`=°` verification of D, A and T. Note that `absent°` is *not* freshness: freshness is about *free* occurrences, so `(# a (◃▹ a a))` succeeds, whereas absento forbids any occurrence. Nominal unification has no function variables and no β, so it is a different logic from λProlog's. Noms are distinct constants, not eigenvariables, and ∀-goals still need their own mechanism (see Chapter 11).

9.6 Eigenvariables, negation and ∀ inside miniKanren

9.6.1 The scope-extrusion problem

GENERIC proves $\forall x.G$ by proving $G[c/x]$ for a *new* constant c. If the constant is only a gensym, nothing stops an older logic variable from being bound to it later. That "proves" $\exists X \forall y.\, X = y$, which must fail. Contrast $\forall y \exists X.\, X = y$, which must succeed. The fixes in the literature are

exactly four: (i) an explicit prefix [34, 30]; (ii) universe indices or levels [43, 9]; (iii) an absento-like eigen constraint over the variables in scope [21]; (iv) an end-of-scope check. The notes call (iv) the weakest. It is sound only if nothing created inside the scope survives it in a form that can be bound later, and its failures are detected late.

9.6.2 `eigen` in miniKanren with symbolic constraints

Hemann, Byrd and Friedman added `eigen` to miniKanren-with-symbolic-constraints in 2013 [21]. The header comment of `mk.scm` reads: "E = (e* . x*)*, where e* is a list of eigens and x* is a list of variables. Each e in e* is checked for any of its eigens be in any of its x*. Then it fails." The state carries B, the list of all variables created so far. `eigen` creates fresh eigen objects and installs an eigen-absento constraint $(e^* . B)$: *none of the variables that existed before the eigen may ever walk to a term containing it.*

```
(define-syntax eigen
  (syntax-rules ()
    ((_ (x ...) g0 g ...)
     (lambdag@ (c : B E S)
       (let ((x (eigen-var)) ...)
         ((fresh () (eigen-absento `(,x ...) B) g0 g ...) c))))))

(define eigen-absento
  (lambda (e* x*)
    (lambdag@ (c : B E S D Y N T)
      (cond
        ((eigen-occurs-check e* x* S) (mzero))
        (else (unit `(,B ((,e* . ,x*) . ,E) ,S ,D ,Y ,N ,T)))))))
```

This is the universe-index check implemented with explicit sets instead of numbers. Its cost is $O(|B| \cdot |\text{term}|)$ per check. The README warns: "Beware: this implementation does *not* support use of `eigen` with constraints other than ==."

Kanren angle: the eigen-absento idea, by hand, on today's engine

(aux kanren micro) already has `absent°`, which takes any datum as its tag, including a record. So the Hemann–Byrd–Friedman check can be written by hand. Here there is one outer variable Q, and the eigen is a fresh record that Q may never contain. This illustrates the idea; it is not an API proposal.

```
(import scheme (chicken base) (chicken pretty-print)
        (aux base) (aux stream) (aux kanren micro))

(define-syntax-rule (show label expr)
  (begin (display ";; ") (display label) (newline)
         (pretty-print expr)))

;; A generic constant as a fresh, unique record: no scope check at all.
(define-record eigen id)
(show "naive: exists Q forall x. Q = x"
```

```
  (°->list #f (fresh° (q) (=° q (make-eigen 1)))))

;; The eigen-absento idea, by hand with absent°: the constant
;; may never occur in a variable that existed before it.
(show "with absent°: exists Q forall x. Q = x"
  (°->list #f
    (fresh° (q)
      (let ((c (make-eigen 1)))
        (and° (absent° c q) (=° q c))))))
(show "with absent°: forall x exists Y. Y = x, then Q = ok"
  (°->list #f
    (fresh° (q)
      (let ((c (make-eigen 1)))
        (and° (absent° c q) (fresh° (y) (=° y c)) (=° q 'ok))))))
(show "with absent°: escape through an alias is caught"
  (°->list #f
    (fresh° (q)
      (let ((c (make-eigen 1)))
        (and° (absent° c q)
              (fresh° (y) (=° q (list y)) (=° y c)))))))
```

```
;; naive: exists Q forall x. Q = x
((λ () (make-record-instance 'eigen 1)))
;; with absent°: exists Q forall x. Q = x
()
;; with absent°: forall x exists Y. Y = x, then Q = ok
((λ () 'ok))
;; with absent°: escape through an alias is caught
()
```

Three cautions come from the notes on (aux kanren micro). First, `µkanren-state-find/value` rebuilds records, so eigens must be compared by an identifier (with `equal?`), never by `eq?`; and two eigens must never share an identifier. Second, (aux kanren micro) binds the *older* variable to the *newer* one in a variable-variable equation. So "rank = creation index" is not a scope discipline by itself, and creation time is not scope depth. Third, the check is first-order only: there is no higher-order unification here.

9.6.3 Moiseenko 2019: constructive negation

Moiseenko [41] builds the answers to $\neg g$ by collecting *all* answers of g and negating them. To make this closed under negation, states become formulas (eq. (1), p. 6):

$$\exists \bar{x}.\Big(\bigwedge_i v_i = t_i \ \wedge \ \bigwedge_j \forall \bar{y}_j. \bigvee_k w_{jk} \neq u_{jk} \Big).$$

This needs a solver for *universally quantified disequalities*:

Lemma 9.12 ([41, Lemma 3, p. 9])**.** *$t \neq u$ is satisfiable iff* $\mathrm{mgu}(t, u) \neq \top$.

This is exactly what (aux kanren micro)'s `≠°` implements.

Lemma 9.13 ([41, Def. 11, Lemma 4, p. 10]). *Let* $\theta = \mathrm{mgu}(t, u)$*, oriented so that* $\bar{x} \not\subseteq \mathrm{codom}\,\theta$*. Then* $\forall \bar{x}.\, t \neq u$ *is satisfiable iff* $\theta = \bot$ *or* $\mathrm{dom}(\theta) \not\subseteq \bar{x}$*, and unsatisfiable iff* $\mathrm{dom}(\theta) \subseteq \bar{x}$*.*

The orientation is a normalization: if the mgu maps an outer variable v to some $x \in \bar{x}$, flip that binding to $x \mapsto v$. In words: $\forall \bar{x}.\, t \neq u$ is unsatisfiable exactly when t and u can be made equal by choosing values for the universal variables alone. Example: for $\forall x.\, x \neq \alpha$, the mgu is $\{x \mapsto \alpha\}$, its domain is $\{x\} \subseteq \bar{x}$, so the constraint is unsatisfiable. For $\forall x.\, \alpha \neq (1, x)$ the mgu $\{\alpha \mapsto (1, x)\}$ binds the outer α, so it is satisfiable (by any α that is not a pair with first component 1).

Lemma 9.14 ([41, Def. 12, Lemma 5, p. 10–11]). *For* $\forall \bar{x} \exists \bar{y}.\, t \equiv u$*, let* θ *be idempotent and* $\hat{\theta}$ *its restriction to the non-*$\bar{y}$ *variables. The formula is satisfiable iff* $\mathrm{dom}(\hat{\theta}) \cap \bar{x} = \emptyset$ *and no* $p \in \mathrm{codom}(\hat{\theta})$ *has* $\mathrm{Vars}(p) \cap \bar{x} \neq \emptyset$*.*

The notes observe that Lemma 5 is exactly the eigenvariable condition. Universal variables may not be bound, and they may not appear in the bindings of outer variables. With negation one gets *ifte* $c\ t\ e = (c \wedge t) \vee (\neg c \wedge e)$, $g_1 \Rightarrow g_2 = \neg g_1 \vee (g_1 \wedge g_2)$ and *forall* $g = \neg\, \mathit{fresh}\,(x)\,(\neg\, g\ x)$. The price: *stratification* is required, negating a goal with infinitely many answers diverges, and infinitely many constructors of each arity are assumed. A remark on p. 4: "there is no sound implementation of eigen variables with the support of disequality constraints. We observed that our implementation of universal quantifications through double negation works nicely with disequalities".

Pitfall: eigen-constants and disequality

λProlog's generic ∀ is sound because λProlog has no negation. Add `≠°` and consider $\forall x.\, x \neq \alpha$, with α an outer variable. It must *fail*: take $x := \alpha$. Moiseenko's Lemma 4 agrees, since the mgu $\{x \mapsto \alpha\}$ has domain $\subseteq \{x\}$. A naive eigen-constant c makes $c \neq \alpha$ hold forever, because α can never be bound to c, and the query wrongly succeeds. The same issue arises for type constraints: c stands for an arbitrary term, so `symbol°` of c should not be provable unless assumed. It also arises for `absent°`.

9.6.4 Jin, Rosenblatt, Might and Zhang 2021: ∀ and ⇒ by relative complement

Jin et al. [26] build on the *first-order miniKanren representation* [51], where goals are data that can be inspected. `(forall (v) domain goal)` works in four steps: it checks the domain's satisfiability; it extracts candidates by running `(exists (v) (conj domain body))` and splitting the states into disjunction-free ones; it computes a *relative complement*, recursing on `(forall (v) (conj relcomp domain) body)`; and it applies a domain relative negation. This is an *extensional* ∀ over the universe of literal values: the classical "for every value" reading, not λProlog's generic one. `(implies A B)` combines *semantic* solving, which rewrites to $\neg A \vee B$ when A is negatable, with *syntactic* solving. In the latter, an assumptions list is threaded through, a user-relation call can be solved by an assumed call, and relations are unfolded under a *closed-world* reading. The paper's opening examples (p. 1):

```
> (run 1 (a) (forall (v) () (== v a)))                          '()
> (run 1 () (forall (a) () (exists (v) () (== v a))))           '(())
> (run 1 (a b) (forall (v) () (disj* (== v a) (=/= v b))))      '((_.0 _.0))
> (run 1 (b) (forall (a) () (disj* (not-symbolo a) (=/= a b))))  '((#f))
> (run 1 (f) (implies (evalo f 1) (exists (a) (evalo f a))))    '((_.0))
```

The third query is the instructive one. $\forall v.\,(v = a \vee v \neq b)$ holds when $a = b$. That is a classical case split on v, which λProlog's generic ∀ cannot do; λProlog has no $\neq$ at all. The authors list the limitations themselves (§7): the system is slow, produces duplicates, is incomplete, and soundness is unproven. Their related work notes that λKanren 2020 supports only relation calls and equalities in antecedents, with "no unfolding … consistent with an open-world interpretation".

Pitfall: three different ∀

λProlog's ∀ is GENERIC: intuitionistic, proved by a new constant, with no case analysis (Chapter 3). Jin et al.'s ∀ is extensional: "for every value in the universe", by relative complements. Moiseenko's ∀ is ¬∃¬: classical, stratified, and it needs finitely many answers. A query that succeeds in one may fail or diverge in another. For instance, Jin et al.'s §6.6 shows that `(forall (expr) () (disj (neg (halto expr)) (halto expr)))` does not terminate. Only the first ∀ is the book's.

9.7 How λProlog systems do it

9.7.1 Miller 1991: L_λ by transitions over a mixed prefix

Chapter 7 presents pattern unification as the book does. Here we recall only the shape of Miller's original formulation [34], because both λKanren 2020 and Nipkow transcribe it. The book's statement of the restriction is: "a program clause or goal formula B is in L_λ if all quantification in B is over nonpredicate types and every subterm of B of the form $(x\ t_1 \ldots t_n)$ in which x is an essentially existentially quantified variable is such that $t_1, \ldots, t_n$ are distinct variables that are universally quantified within the scope of the quantifier binding x" [MN p. 220]. Then only β_0 is needed: $(\lambda y.\,s)\,x = s[x/y]$ when x is a variable not free in $\lambda y.\,s$.

Miller's interpreter turns GENERIC into a *meta-level* universal quantifier, and BACKCHAIN turns the clause's ∀s into ∃s placed at the point of use. This is why mixed prefixes ∀∃∀ … arise dynamically. The unification transitions are ξ (descend under binders, renaming apart), rigid-rigid, *raising*, *pruning*, flex-flex and flex-rigid. The algorithm terminates (Theorem 7.1), and when it succeeds it yields a most general unifier (Proposition 7.3). One of Miller's worked examples (p. 14–15):

$$Q\exists u\exists v\forall y\,[f(\lambda x.\,f(u\,x\,y)) = f(\lambda w.\,v\,y)] \xrightarrow{\text{rigid}} \lambda x.\,f(u\,x\,y) = \lambda w.\,v\,y \xrightarrow{\xi} \forall y\forall x\,[f(u\,x\,y) = v\,y]$$

$$\xrightarrow{\text{pruning}} [u \mapsto \lambda x.\,\lambda y.\,u'\,y] \xrightarrow{\text{flex-rigid}} [v \mapsto \lambda y.\,f(u'\,y)].$$

Read the last two steps with the prefix in mind, which is now $Q\exists u\exists v\forall y\forall x$. The equation is flex-rigid: $v\,y$ against $f(u\,x\,y)$. The universal x is quantified after v and is not among v's arguments, so v's solution may not mention it. Every occurrence of x sits under the flexible

head u, so the problem is not hopeless: *pruning* removes u's dependency on its first argument, $u \mapsto \lambda x.\, \lambda y.\, u'\ y$. After that the rigid side is $f(u'\ y)$, and flex-rigid abstracts v's argument: $v \mapsto \lambda y.\, f(u'\ y)$. Had x occurred *outside* any flexible head, pruning would fail and so would the equation.

In the untyped version (§9.3), rigid-rigid with different arities fails, and η-expansion is used only on terms that must have functional type. This matters for an untyped Scheme term language. Miller also shows how to stay inside L_λ by replacing meta-level β with *copy clauses* and hypothetical reasoning, as in the book's Figure 7.10 [MN p. 200]:

```
subst M T S :- pi x\ copy x T => copy (M x) S.
```

The same trick encodes a *non-pattern* problem inside L_λ [34, §10.3, p. 30–31]. With copy clauses `copyterm a a`, `copyterm (f X) (f U) :- copyterm X U.` and so on for the signature, define

```
substterm M T N :- pi c\(copyterm c T => copyterm (M c) N).
```

The unifiers of $F\ a \stackrel{?}{=} g\ a\ a$, which is not a pattern because a is a constant, are exactly the F that make `substterm F a (g a a)` provable. Miller lists the four answers an L_λ interpreter should return: `w\(g w w)`, `w\(g w a)`, `w\(g a w)`, `w\(g a a)`. The choice "is this a the argument or the constant?" is made by backchaining, either on the hypothesis `copyterm c a` or on the clause `copyterm a a`. Huet's branching has become ordinary proof search, which a miniKanren would interleave.

The book's position on programs outside L_λ is pragmatic. A common resolution is "to not impose the L_λ restriction on program clauses statically but rather to expect the condition to be satisfied by the unification problems that are encountered dynamically". One "might defer further consideration of this equation until such time that substitutions resulting from solving other parts of the problem have altered its status" [MN p. 225–226].

9.7.2 Nipkow 1993: functional pattern unification on de Bruijn terms

Nipkow [48] restricts attention to ∀∃∀ problems, where raising is never needed. Existential variables are global (`V F`), outer universals are constants (`C c`), and inner universals are loose de Bruijn indices (`B i`). The substitution is triangular. From the author's ML code, the algorithm is:

1. head-normalize both sides through the substitution, with β_0 done by renaming indices;
2. $\lambda s = \lambda t \Rightarrow s = t$; η on demand, $\lambda s = t \Rightarrow s = (\uparrow t)\ 0$;
3. rigid-rigid: equal heads give pairwise unification, different heads fail;
4. flex-rigid $F\ \bar{y} = t$: occurs check, then *invert* t with respect to $\bar{y}$ and prune the flexible subterms that mention indices outside $\bar{y}$;
5. flex-flex, same head: $F\ \bar{y} = F\ \bar{z} \Rightarrow F := \lambda^{|\bar{y}|}.H$ (positions where $y_i = z_i$);
6. flex-flex, different heads: if $\bar{\imath} \subseteq \bar{\jmath}$ then $G := \lambda \bar{\jmath}.F\ \bar{\imath}$, symmetrically in the other case, and otherwise both are bound to a fresh H applied to $\bar{\imath} \cap \bar{\jmath}$.

9.7.3 Dowek, Hardin, Kirchner, Pfenning 1996: explicit substitutions

DHKP [8] work in the λσ-calculus, where substitution $a[s]$ is syntax. Meta-variables are *grafted*, i.e. replaced first-order. To recover the logical, capture-avoiding reading, terms are **pre-cooked**: $F(\lambda a, n) = \lambda F(a, n{+}1)$, $F(a\,b, n) = F(a, n)\,F(b, n)$, $F(k, n) = 1[\uparrow^{k-1}]$, $F(X, n) = X[\uparrow^n]$. A metavariable under n binders is explicitly *shifted past them.* This is precisely the step that makes grafting agree with substitution, and that static closures omit (counter-example 1). In a λσ-pattern, every $X[s]\,b_1 \ldots b_q$ has $s = a_1 \ldots a_p \cdot \uparrow^n$ with the a's and b's distinct indices. Pattern substitutions are injective and have a right inverse, so $X[s] = b$ is solved by $X := b[s^{-1}]$ when every index of b occurs in s (Prop. 8, inversion). Unification of λσ-patterns is decidable and unitary (Theorem 17), and "our presentation does not need mixed prefixes": raising is just a rule. Outside the fragment, §5.4 says that equations "should be postponed as constraints in the hope that they will be instantiated further", which "avoids all branching during constraint simplification". Empirically, "almost all dynamically arising equations fall within the pattern fragment" (p. 2).

9.7.4 Universe indices: Nadathur, Jayaraman and Kwon 1995

The idea behind every efficient implementation of GENERIC is numeric [43]: "Each time a universal quantifier is encountered, a new constant must be introduced, giving rise to the next universe in the hierarchy. This requirement can be accounted for by increasing the "universe index" by 1 and introducing a new constant tagged with this index. ... When an existential quantifier is encountered, it is instantiated by a logic variable. This variable may be tagged with the current value of the universe index, the intended interpretation of the tag being that a term may be substituted for the variable only if all the constants appearing in the term have a smaller or identical tag" (p. 15–16).

Definition 9.15 (Binding with universe indices [43, p. 16])**.** To bind a variable X with tag i to t: (1) perform the usual occurs check; (2) check that t contains no constant with tag $> i$; (3) *before binding*, change to i the tags of the variables of t whose tag is $> i$.

Step (3) is *lowering.* It is "required in order to prevent a later instantiation of these variables from violating the restrictions on instantiations for X".

Tutorial: why lowering is needed

Program: $\forall z(q(z) \supset p(d(z)))$. Goal: $\exists x \forall y(q(y) \supset p(x))$.

1. $\exists x$ creates X with tag 1. $\forall y$ raises the index to 2 and creates c with tag 2. AUGMENT adds $q(c)$. The goal is now $p(X^1)$.
2. BACKCHAIN on the clause gives a new variable Z tagged with the current index, 2. Unifying $p(X^1) = p(d(Z^2))$ binds $X := d(Z)$, and **lowers** Z to tag 1.
3. The subgoal $q(Z^1)$ meets the hypothesis $q(c^2)$. Binding $Z^1 := c^2$ violates step (2), so it fails.

Without lowering, $Z^2 := c$ would be accepted and X would become $d(c)$. That smuggles the eigen-constant into a variable quantified outside its scope. The failure is correct: the

goal would need $x = d(y)$, but x is quantified before y.

The abstract interpreter works on tuples $\langle G, P, I\rangle$. A $\exists$-goal tags its new variable with I; a $\forall$-goal goes to $\langle [c/x]G, P, I{+}1\rangle$; and backchaining instantiates the clause's variables with tag I. The interpreter is sound and complete with respect to intuitionistic provability, and goal selection order is irrelevant (Prop. 4.1). In the WAM, a *UI register* is incremented on entering a universal goal and decremented on its completion, and choice points save it.

9.7.5 Teyjus: suspensions and on-the-fly raising

Teyjus represents term-bound variables by de Bruijn indices and logic variables by mutable cells [42]. β-reduction is made lazy with the *suspension notation* $[[t, ol, nl, e]]$: "a term t that used to occur inside ol abstractions but now appears within nl of them", with an environment e. Some of the rules:

$$(\beta_s)\ (\lambda t_1)\, t_2 \to [[t_1, 1, 0, (t_2, 0) :: nil]] \qquad (r_9)\ [[t, 0, 0, nil]] \to t$$
$$(r_7)\ [[\lambda t, ol, nl, e]] \to \lambda[[t, ol{+}1, nl{+}1, @nl :: e]]$$

Terms also carry a closed/open annotation bit; empirically, substitution walks were reduced to "a third to an eighth".

Nadathur and Linnell [44] eliminate the explicit prefix. Each variable gets a tag $l(y)$, the number of $\exists$-followed-by-$\forall$ alternations up to its quantifier, and "a universal variable u is quantified within the scope of the quantifier for an existential variable x just in case $l(x) < l(u)$". Raising is restricted to the universals that actually appear as arguments. A single traversal of the rigid side (`mksubst/bnd`) computes the binding, i.e. the inversion, and does pruning and raising on the fly. A non-pattern subterm met during the traversal can be replaced by a fresh $h(\bar{y})$ with the equation $h(\bar{y}) = t$ **deferred**. The book describes the same idea of "attaching numerical indices to the quantified variables" [MN p. 225]. It also notes that Teyjus 2 implements only L_λ unification [MN p. 228].

Example 9.16 (no mgu outside the fragment [44, p. 5])**.**
The problem $\forall u_1 \forall u_2 \exists x\, (x(u_2) = u_1(u_2))$ is not a pattern: u_2 is quantified *outside* the scope of x, so it is a constant for x. It has "two incomparable solutions", $x \mapsto \lambda w.\, u_1(w)$ and $x \mapsto \lambda w.\, u_1(u_2)$ (written $\lambda(1, u_1(1))$ and $\lambda(1, u_1(u_2))$ in the paper's de Bruijn notation), and no most general one. Move the quantifier, $\forall u_1 \exists x \forall u_2\, (x(u_2) = u_1(u_2))$, and it becomes a pattern with the single solution $x \mapsto \lambda w.\, u_1(w)$. A unifier that must return one mgu can only defer the first problem. An enumerating one would return both answers, as Miller's `substterm` does for $F\,a \stackrel{?}{=} g\,a\,a$ (Section 9.7.1).

9.7.6 ELPI: de Bruijn levels and a reduction-free fragment

ELPI [9] starts from an observation about the two roles of β:

```
of (lam F) (arr A B) :- pi x\ of x A => of (F x) B.
cbn (app (lam F) N) M :- cbn (F N) M.
```

In `of`, β only replaces a bound name with a fresh constant. In `cbn`, β is genuine object-level substitution. The design has three parts.

Levels for the prefix. "When a clause's head is unified with the goal in a context of length n, the universally quantified variables of the clause are instantiated to unification variables X^n where the level n records that X has only visibility of the initial prefix of length n of the context. If later a fresh constant is added by the pi rule, the constant occupies position $n + 1$ … and it will not be allowed to occur in instances of the variable X^n" (p. 4). Level lowering is done by pruning: the higher-level variable is bound to a fresh lower-level one.

De Bruijn levels, not indices. The example is $\lambda x.\,(\lambda y.\,\lambda z.\,f\ x\ y\ z)\ x$:

$$\begin{aligned} \text{indices:}\quad & \lambda x.(\lambda y.\lambda z.f\ x_2\ y_1\ z_0)\ x_0 \to_\beta \lambda x.\lambda z.f\ x_1\ x_1\ z_0 && (x \text{ is renumbered}),\\ \text{levels:}\quad & \lambda x.(\lambda y.\lambda z.f\ x_0\ y_1\ z_2)\ x_0 \to_\beta \lambda x.\lambda z.f\ x_0\ x_0\ z_1 && (x \text{ keeps its number}). \end{aligned}$$

Three properties follow. A variable bound in Γ keeps its name in any extension of Γ (DBL1) and in a β-reduct (DBL2), and pushing a binder into the context needs no lifting (DBL3). "variables already pushed in the context are treated exactly as constants, and … the two notions of level — De Bruijn's and the position in the context — coincide". Terms are $t ::= x_i \mid X^j \mid \lambda x_i.t \mid t\,t$, with global constants as negative i; X^j sees the names below j.

Definition 9.17 (Reduction-free fragment L_λ^β [9, Def. 1])**.** Every occurrence of a unification variable X^j is applied to $x_j \ldots x_{j+k-1}$, for $k \geq 0$: exactly the next k context names, in order.

This fragment is contained in L_λ (Property 1). Head reductions of $(X^j\ x_j \ldots x_{j+k-1})\sigma$ take constant time (Property 2), and so does solving $X^j\ x_j \ldots x_{j+k-1} \equiv t$ when no occurs check is needed and the maximum level of t is cached (Property 3). But L_λ^β is **not stable under resolution** (p. 7), so the fast path must fall back to general pattern unification. ELPI reports large speed-ups: typeof in 0.27 s versus 5.64 s for Teyjus, reduce-cbv in 0.15 s versus 11.11 s.

Outside the fragment, ELPI's default is an **error**: "Unification problem outside the pattern fragment. … Pass -delay-problems-outside-pattern-fragment … in order to delay (deprecated, for Teyjus compatibility)". Delay is offered instead through *syntactic constraints*. `declare_constraint (even X) [X]` suspends a goal on variables, and the goal is resumed when any of them is assigned, so `declare_constraint (even X) [X], X = 1.` fails. Suspended goals keep their hypothetical context. The documentation's example (`ELPI.md`, "Syntactic constraints"):

```
goal> pi x\ sigma Y\ even x => declare_constraint (even Y) [Y].
Success:
Constraints:
 {x} : even x ?- even (W x)  /* suspended on W */
```

The residual constraint remembers the eigenvariable `x` and the hypothesis `even x`. Note also the automatic raising: `Y` was created inside the scope of `x`, so it is printed as `W x`. Constraint Handling Rules can operate on suspended goals.

9.7.7 Makam

Makam [55] is a λProlog dialect with polymorphism, used to prototype type systems. Two idioms are relevant for us. `bindmany` is a binder over a *list* of variables, with `openmany`/`applymany` to open many binders at once. And reflective predicates such as `refl.isunif` test whether a term is an uninstantiated unification variable. That test is non-logical, and is the analogue of (aux kanren micro)'s `project°`.

Kanren angle: what the implementations ask of a miniKanren

The real systems agree on three mechanisms: (i) a *level* on every logic variable and eigen-constant, with the binding check and lowering of Definition 9.15; (ii) pattern unification with inversion and pruning (Nipkow, Nadathur–Linnell, ELPI); (iii) a store of *delayed* equations or goals, woken on assignment (DHKP §5.4, Teyjus, ELPI's syntactic constraints). (aux kanren micro) has no notion of level. Its creation index is a time stamp, not a scope depth: a variable created after a ∀-scope has closed has a larger index, but it must not see that scope's constant. On the other hand, the `assocs`-driven re-verification of D is exactly the wake-up hook that a delay store needs. Search is unchanged: `or°`'s fair interleaving can host Huet's branching. That fits Miller's remark that a breadth-first search could compute the converse of his `prenex` relation, which the depth-first interpretation cannot [34]. The design itself is in Chapter 11.

9.8 Comparison of the design options

Table 9.2 sets the four candidate meanings of `=°` on λ-terms side by side. The rows summarize the analysis of Sections 9.2, 9.3, 9.5 and 9.7. The table records trade-offs and does not rank the options: the choice, and the reasons for it, are the subject of Chapter 11.

Some combinations are possible and appear in the literature. Option D can sit on top of A as a separate, explicitly nondeterministic relation; the book itself presents Huet's procedure as a search over a matching tree (Chapter 7), and miniKanren's interleaving could explore it fairly. Option B can live beside A, for users who want names as data. Option C behaves differently from all the others on the three counter-examples of Section 9.3.1; any implementation should state its answers to them explicitly. The smaller alternatives for ∀ alone are the explicit prefix of λKanren 2020 (simplest, but each check walks the prefix and the prefix only grows), the eigen-absento of Hemann–Byrd–Friedman (smallest change, $O(|B|)$ per check, first-order only), and the end-of-scope check (weakest). The constructive ∀/⟹/¬ of Moiseenko and Jin et al. is orthogonal: it has a classical, extensional reading and does not implement the book's GENERIC and AUGMENT.

Table 9.2: Design options for higher-order unification in a miniKanren. No option is chosen here; see Chapter 11.

	A. Pattern unification, de Bruijn + levels (Miller, Nipkow, Nadathur–Linnell, ELPI)	**B. Nominal** (Urban–Pitts–Gabbay, αKanren)	**C. Static closures** (Ma–Friedman 2021)	**D. Huet pre-unification, enumerated by the stream**
Fidelity to the book	exact: L_λ is what Teyjus 2 and ELPI run	a different logic (nominal); no function variables, no β	non-standard grafting semantics: $\lambda a.\, X \stackrel{?}{=} \lambda b.\, b$ succeeds, unlike λProlog	exact for full higher-order unification (pre-unifiers)
Decidable / unitary	yes / yes (mgu)	yes / yes, with a freshness context	claimed; no full algorithm published (a "preview")	semi-decidable; possibly infinitely many pre-unifiers
New term forms	λ node (indices), eigen-constant with level, flexible application $X\,\bar{y}$	nom, `tie`, suspension $\pi \cdot X$	named and nameless binders, levels, closures	as A, plus imitation and projection
New state	a level per variable, current depth, delayed-equation store	freshness store ∇ of $a\#X$	a "context" of closure equations	as A; branching via `or°`
Cost of α	free (syntactic on indices)	swappings on the fly, suspensions	levels	free
β	only β_0; general β via copy/subst relations, or delayed	none (write `substo`)	"will be powered" (not given)	full
∀-goals (GENERIC)	eigen-constant + level check + lowering	a separate mechanism is still needed: noms are distinct constants, not eigenvariables	`all` from λKanren 2020 (prefix)	as A
With `=/=°`, `absent°`, types	eigens need a ∀-aware check (Moiseenko)	αKanren has no disequality at all	not discussed	as A
Effort in aux	medium–high: a Nipkow-size core, plus levels and a delay store	medium: Byrd's ch. 11 is about 300 lines; a triangular version exists	unclear	high, plus control of infinite branching

9.9 Summary

- Ma and Friedman 2021 [29] is a workshop *preview*. It has two grammars, the operations ext and db, three examples, and no algorithm, rules or theorems.
- Its first complaint is that pattern unification rejects $\lambda a.\, X \stackrel{?}{=} \lambda b.\, b$. That is the scope restriction, which λProlog needs for the soundness of ∀. Its fix is to graft de Bruijn levels, giving $X \mapsto \texttt{d0}$. λProlog's own answer is to make the dependency explicit: $\lambda a.\, (X\, a) \stackrel{?}{=} \lambda b.\, b$ is a pattern with mgu $X \mapsto \lambda x.\, x$ (Miller's "discharging a constant").
- Its second complaint is that binder-merging plus naive reification makes $\lambda a.\, \lambda a.\, X \stackrel{?}{=} \lambda c.\, \lambda d.\, c$ succeed, so "true = false". Static closures $\langle \phi;\, t \rangle$ make shadowed levels unavailable, and the equation fails. Flex-flex closure equations are postponed in a *context*.
- The assessment uses three counter-examples. The conjunction separates static closures from nominal unification. The dangling `d0` at top level shows that X has no depth-independent meaning. "True = false" also fails under pattern and nominal unification, so it is a bug of binder-merging only. Static closures are grafting on de Bruijn terms without DHKP's pre-cooking, and α is no longer a congruence on open terms.
- λKanren 2020 transcribes Miller 1991 with an explicit prefix and gensym renaming. Non-patterns fail. Hypotheses must be threaded in the state, because a host parameter loses them under delayed relation bodies.
- Nominal unification (αKanren) handles α with swappings and freshness, using capturing substitution. It has no function variables, no β and no disequality, and noms are not eigenvariables.
- Eigenvariables in miniKanren: a fresh constant without a scope check is unsound. Hemann, Byrd and Friedman implement the check as an absento over the earlier variables. Disequality and eigen-constants conflict (Moiseenko). Jin et al.'s ∀ is extensional, not generic.
- Real λProlog systems use universe indices or levels with lowering (Nadathur et al., Teyjus, ELPI), pattern unification with inversion and pruning (Nipkow, Nadathur–Linnell), and delayed non-pattern equations (DHKP, Teyjus, ELPI's syntactic constraints). Teyjus adds the suspension calculus; ELPI uses de Bruijn levels and the L_λ^β fast path.
- The options are pattern+levels, nominal, static closures, and Huet enumerated by the stream (Table 9.2). The choice is made in Chapter 11.

Part III

A proposal for `(aux kanren micro)`

10 Anatomy of (aux kanren micro)

What and why

The chapters so far studied what a higher-order logic programming language *is*: the uniform proofs of hereditary Harrop formulas (Chapter 3), λ-terms as data (Chapters 4 and 6), higher-order pattern unification (Chapter 7), and, in Chapter 9, how λKanren grafts these features onto a miniKanren [29, 30]. This chapter opens the *proposal* part of the document. Before deciding how (aux kanren micro) should grow, we need an exact picture of the engine as it is today: what a variable is, what a state contains, how unification and the constraint stores interact, how the search interleaves, and how answers are printed. The point is not to explain the code to its author but to put next to each other the facts that matter when the engine is extended: which invariants a new feature must preserve, which parts can be reused from outside, and which parts are closed.

The engine is a μKanren in the sense of Hemann and Friedman [22] (a goal maps a state to a stream of states) with a cKanren-like constraint store holding disequalities, disjoint type tags and *absento* constraints, in the style of *The Reasoned Schemer* [15]. All references `:N` are line numbers of `src/aux.kanren.micro.scm` (713 lines) at commit `f53bdcb`. Every Scheme snippet in this chapter was run on that engine; its output is pasted verbatim below it. The snippets import `(aux base)`, `(aux stream)`, `(aux fds sbral)` and (aux kanren micro).

Table 10.1: A map of `aux.kanren.micro.scm`.

lines	content	section
3–26	module header, explicit export list, imports	
28–59	sentinel, tags, variables	Section 10.1
60–107	the state record, sbral helpers	Sections 10.2 and 10.3
110–204	occurs check and binding, walks, unification	Sections 10.3 and 10.4
206–326	reification and projection	Section 10.7
330–523	D/A/T verification and subsumption	Section 10.5
525–629	goals, combinators, constraint goals	Section 10.6
631–684	aggregation	Section 10.8
686–713	`define-relation`, runners	Sections 10.6 and 10.7

10.1 Terms and variables

There is no term datatype: a term is any Scheme value, and the engine gives structure to some of them.

Definition 10.1 (Terms of (aux kanren micro))**.** A *term* is one of: a logic variable; an atom (symbol, boolean, `'()`, character, number, string, or any other Scheme object); a pair `(t . u)` of terms; a vector of terms; an instance of a CHICKEN record type (`define-record`) whose slots are terms. Vectors and record instances together are the *structures* (`µkanren-structure?`, `:110`).

A logic variable is a record with a single integer slot (`:41`):

```
(define-record µkanren-var index)
(define (µkanren-var-working? α) (and (µkanren-var? α) (<= 0 (µkanren-var-index α))))
```

The sign of the index separates two kinds of variable:

- a *working* variable has index $i \geq 0$; it lives during the search, its index is its slot in the substitution, and it prints as `_i`;
- a *reified* variable has index $-1, -2, \ldots$; it exists only in answers, is created only by reification (Section 10.7), and prints as a Greek letter `α`, `β`, ..., `ω`, then `α1`, ...(`:52`–`59`).

The helper `µkanren-var-index>=0` (`:50`) maps both kinds onto $\mathbb{N}$: $i \mapsto i$ for working variables, $-1 \mapsto 0$, $-2 \mapsto 1$, ... for reified ones.

```
(write (map make-µkanren-var '(0 1 7 -1 -2 -24 -25))) (newline)
```

```
(_0 _1 _7 α β ω α1)
```

Indices are allocated by `fresh`ª from a counter in the state (Section 10.6), so along one branch of the search they increase monotonically: *the index of a variable is its age*, lower meaning older. This fact, invisible to a user, is the most useful one for the λProlog extension.

Pitfall: A variable is a record

A `µkanren-var` is itself a `record-instance?`. Every structural walker of the engine therefore tests `µkanren-var?` *before* the generic record case (e.g. `:117` before `:124`, and the explicit `(not (µkanren-var? w*))` at `:190`). Any new walker, such as a scope check, must do the same, or it will descend into a variable as if it were a two-slot structure.

Kanren angle: Indices as levels

λProlog's GENERIC rule reduces $\Sigma; \mathscr{P} \longrightarrow \forall x\, G$ to $c : \tau, \Sigma; \mathscr{P} \longrightarrow G[c/x]$ for a token c not in Σ [MN p. 39], and the query `sigma x\ pi y\ x = y` must fail [MN p. 85–86]: with logic variables, a variable created *before* c must never be bound to a term containing it. Implementations enforce this with universe indices or levels attached to variables and constants [43]. In (aux kanren micro) the monotone index already is a level boundary: if the eigenvariable is created when the counter is n, then "created before c" is exactly "index $< n$". The prototype of Section 10.9.3 uses precisely this. Index order alone is not enough, though: a variable created after the ∀ has exited must not see c either, and lowering changes a variable's level after its creation. The proposal therefore keeps a level per variable, in an `sbral` indexed like S and padded only when the depth changes or the

unifier creates a variable of another level (Definition 11.1, Definition 12.4, Section 12.2).

10.2 The state

Definition 10.2 (State)**.** A state is a record of six fields (`:69`):

```
(define-record µkanren-state vars-count S D A T tags)
(define µkanren-state-empty (make-µkanren-state 0 empty/sbral empty/sbral
 ↪  empty/sbral empty/sbral '()))
```

field	content	written by / read by
`vars-count`	the next fresh index, i.e. the number of variables introduced on this branch	written and read by `freshª` (`:532–537`)
`S`	the substitution: a skew-binary random-access list (sbral) of terms or of the sentinel `*µkanren-unbound*`	written only by `µkanren-state-update` (`:113–134`); read by every walk
`D`	disequality store: a list of *prefixes*, each a list of `(var . term)` pairs	`≠°` (`:554–567`), re-verification after `=°`
`A`	type store: a list of `(var . tag)`; the tags are pairwise disjoint	`symbol°`, `number°`, user tags; verified after `=°`
`T`	absento store: a list of `(term . tag)`, the term a variable or a compound	`absent°` (`:617–624`); verified after `=°`
`tags`	tags asserted through A	written at `:481`, read by nobody in the engine (a dead field)

Only S is a real sbral; D, A and T are plain lists (the `empty/sbral` in the empty state is just `'()`). States are immutable and copied by functional update with keyword arguments (`:76–83`),

```
(µkanren-state-with s D: (append D* D))          ; a copy of s with a new D
```

except at the two hottest sites, `µkanren-state-update` and `freshª`, which call the positional constructor directly. A positional destructuring macro `µkanren-state-match` of fixed arity six (`:96–103`) is used by six goals (`:130`, `:515`, `:533`, `:540`, `:555`, `:618`).

To see states, we use a small printer (not part of the engine) and run goals on the empty state:

```
(define (dump s)   ; the fields of a state on one line; a tag is shown by its name
  (for-each (λ (k v) (display k) (display " ") (write v) (display "  "))
            '(vars-count S D A T)
            (list (µkanren-state-vars-count s)
```

```
                  (sbral->list (µkanren-state-S s))
                  (µkanren-state-D s)
                  (map (λ (p) (cons (car p) (µkanren-tag-name (cdr p))))
                  ↪  (µkanren-state-A s))
                  (map (λ (p) (cons (car p) (µkanren-tag-name (cdr p))))
                  ↪  (µkanren-state-T s))))
  (newline))
(define (states g) (§->list (g µkanren-state-empty)))   ; every answer state of g

(for-each dump (states (fresh° (q x y) (=° q (list x y)) (≠° x 1) (symbol° y)
↪  (absent° 'k x))))
```

```
vars-count 3  S ((_1 _2))  D (((_1 . 1)))  A ((_2 . µkanren-tag/sym87))  T ((_1 . k))
```

The three variables `q`, `x`, `y` are `_0`, `_1`, `_2`; S holds one binding (`_0` is bound to the list `(_1 _2)`); each store holds one constraint. The numeric suffix of a tag name such as `µkanren-tag/sym87` is a `gensym` counter (`:33–34`): it identifies the tag but its value depends on the program that loaded the module, so the same tag prints as `µkanren-tag/sym98` in another script of this chapter.

Proposition 10.3 (The eq?-invariant). *States are compared by identity: `(define (µkanren-state-equal? s s*) (eq? s s*))` (`:73`). Unification returns* the very same state object *when it adds no binding, and a freshly allocated one otherwise (by inspection of `:160–181`: the non-binding cases call `return`, which closes over the input state, and only `µkanren-state-update` allocates).*

This is not an optimisation but semantics: every entailment test of the engine reads "unification gave back `s`" as "the two terms are already equal". The users are `≠°` (`:559`: the terms are already equal, so the disequality fails), `=°` (`:544`: nothing new, so skip re-verification), the re-verification of one disequality (`:460`), the absento test `µkanren-tag-unify` (`:394`) and subsumption (`:304`).

Pitfall: Preserving the eq?-invariant

Any extension that runs inside unification, or after it, must return the input state itself when it changes nothing. A hook that rebuilds the state with `µkanren-state-with`, which always allocates, on a no-op unification would make `(≠° 1 1)` look satisfiable.

How the record has grown. The state record was widened in place several times: from `(substitution counter)` to `(S D A T)` (commit `61bb868`), then `vars-count` (`1e7994e`), then `tags` (`d118ddd`). Each step edited micro and the positional state constructions of `test/microkanren.scm`. No sibling module (arith, date, show, unionfind) ever builds a state: every store so far was added *inside* micro. This is the precedent the proposal of Chapter 11 has to reckon with.

10.3 The substitution and the walk

S as an sbral. `(aux fds sbral)` implements Okasaki's skew-binary random-access lists: `cons/sbral`, `car/sbral`, `cdr/sbral` in $O(1)$, `sbral-ref` and `update/sbral` in $O(\log n)$, `length/sbral`

in $O(\log n)$. S stores the *most recent variable first*: the slot of the working variable of index i sits at position $\ell - 1 - i$, where ℓ is the length of S (`µkanren-var-index/sbral`, `:86`). Extending S at the front with `cons/sbral` therefore keeps the positions of older variables valid when counted from the end.

Lazy extension. `freshª` does not touch S. Binding the variable of index i first pads S with the sentinel `*µkanren-unbound*` until it has a slot i (`µkanren-var-extend/sbral`, `:89–94`). Hence $\ell = 1 + \max\{\text{bound index}\}$, and a variable whose index is $\geq \ell$ is unbound *without* having a slot.

```
(for-each dump (states (fresh° (a b c d) (=° c 5))))
```

```
vars-count 4  S (5 #<aux.kanren.micro#µkanren-unbound> #<aux.kanren.micro#µkanren-unbound>)
↪  D ()  A ()  T ()
```

Four variables were introduced but S has three slots: `c` (index 2) is bound to 5 at the front, `a` and `b` hold the sentinel, `d` (index 3) lies beyond S.

Definition 10.4 (Walk)**.** Let ℓ be the length of S. The *walk* of a term t (`µkanren-state-find/SL`, `:139–145`) is computed by the loop $F(\beta_0, \beta)$, started with $\beta_0 = \beta = t$:

1. if β is a variable with $idx(\beta) \geq \ell$ (with $idx =$ `µkanren-var-index>=0`), answer β (unbound, no slot);
2. if β is a working variable, continue with $F(\beta, S[\ell - 1 - i])$;
3. if β is the sentinel, answer β_0, the last variable visited;
4. otherwise answer β.

Case 4 catches non-variables but also *reified* variables, which are not working: to the walk a reified variable is a constant. Bindings are triangular (S maps a variable to a term that may contain bound variables) and the walk never compresses paths. The variant with explicit `S` and `l` hoists the $O(\log n)$ `length/sbral` out of deep traversals; `µkanren-state-find` (`:147–148`) computes it once.

Deep walks. Three exported procedures resolve a term completely:

- `µkanren-state-find/value` (`:183–191`) returns a Scheme value, rebuilding pairs with `cons`, vectors with `list->vector` and records with `(apply make-record-instance slots)`; unbound variables stay `µkanren-var` records;
- `µkanren-state-find/values` (`:632`) maps it over a list (the key of an aggregation group);
- `µkanren-state-find/repr` (`:193–204`) returns an *expression* that rebuilds the value: a variable becomes its symbol, a symbol `'sym`, a pair `(cons a d)`, a vector `(vector ...)`, a record `(make-record-instance 'type ...)`.

Pitfall: Deep walks rebuild records

Because `find/value` rebuilds every record, the value it returns is `equal?` but not `eq?` to the record that was unified. Here a record `c` of the type `eigen` of Section 10.9.3 is bound to `q` and read back:

```
(let* ((c (make-eigen 99)) (r (car (§->list ((fresh° (q) (=° q c))
↪  µkanren-state-empty))))
       (v (µkanren-state-find/value (make-µkanren-var 0) r)))
  (pretty-print (list (eq? v c) (equal? v c) (eigen? v))))
```

```
(#f #t #t)
```

Constructed terms must be compared with predicates and `equal?` (or by an id slot), never with `eq?`; Section 10.9.3 shows a scope check that goes wrong exactly this way.

10.4 Unification

10.4.1 Binding and the occurs check

Exactly one procedure creates a binding (`:113–134`, abridged):

```
(define (µkanren-state-update α v s assocs #!key (unsafe #f))
  (define (occur? v*) ...)                  ; does α occur in the walk of v*?
  (and
    (or unsafe (not (occur? v)))
    (µkanren-state-match ((vc S D A T tags) s)
      (let* ((S* (µkanren-update/sbral α v S))
             (s* (make-µkanren-state vc S* D A T tags))
             (assocs* (cons `(,α . ,v) assocs)))
        `(,s* ,assocs*)))))
```

It answers `#f` if α occurs in v (the occurs check walks v, its pairs and its structures through S), otherwise the pair `(s* assocs*)`. The list `assocs` accumulates *the bindings made by the current unification* as `(var . term)` pairs: this is cKanren's "prefix". The `unsafe:` key is never passed, so the occurs check is always on. The procedure is not exported; it is the only place where a binding can be refused before it enters the state, e.g. for violating a scope; a check run right after each `=°`, before the next goal, works as well (the route the proposal takes, Section 11.4.3).

10.4.2 The unification algorithm

Definition 10.5 (Unification, `:160–181`)**.** `(µkanren-state-unify u v s assocs)` walks both sides to u^*, v^* and tries these cases *in order*:

1. u^* `eq?` v^*: success, nothing bound;
2. two strings that are `string=?`: success;

3. two numbers that are `µkanren-number-eqv?` (`:153–158: eqv?`, but two NaN flonums are equal and non-real numbers are compared part by part): success;
4. two working variables of indices i, j: if $i = j$ success; if $i < j$ bind $u^* := v^*$, else $v^* := u^*$, i.e. *the older variable is bound to the newer one*;
5. one working variable: bind it to the other side;
6. two structures: unify the lists of their slots (`µkanren-structure->list`, `:111`), where `record->vector` puts the record's type name first;
7. two pairs: unify the cars, then the cdrs in the resulting state;
8. otherwise fail with `#f`.

The result is `(s* assocs*)` or `#f`. With `assocs` = `'()`, the second component is exactly the list of new bindings.

Read as rules, the algorithm is the standard first-order syntactic one with occurs check; what is specific to (aux kanren micro) is (a) the orientation of variable–variable bindings, (b) structures, (c) the treatment of atoms. The exported procedure can be called on a raw state:

```
(define q (make-µkanren-var 0))
; a state where _0 exists
(define s0 (µkanren-state-with µkanren-state-empty vars-count: 1))

(define r (µkanren-state-unify q '(1 2) s0 '()))
(pretty-print (cadr r))                                    ; the new bindings
(define r2 (µkanren-state-unify q '(1 2) (car r) '()))
; nothing new: the very same state
(pretty-print (list (eq? (car r2) (car r)) (cadr r2)))
(pretty-print (µkanren-state-unify q (list 1 q) s0 '()))   ; occurs check
```

```
((_0 1 2))
(#t ())
#f
```

The first line is the one-element prefix `((_0 . (1 2)))`, printed as Scheme prints it; the second shows Proposition 10.3 at work; the third is the occurs check.

Orientation. Unifying two fresh variables binds the older to the newer:

```
(for-each dump (states (fresh° (a b) (=° a b))))
```

```
vars-count 2  S (_1)  D ()  A ()  T ()
```

S has one slot, the one of `_0`, and it holds `_1`. A level discipline in the style of λProlog would prefer the opposite orientation for a variable–variable pair (bind the younger variable to the older one, so that such a binding never moves a variable out of its scope). Flipping the orientation would not be enough, though: in `sigma X\ pi y\ sigma Z\ (X = f Z, Z = y)` the binding

$X := f\,Z$ makes the young Z reachable from the old X, so a real level discipline must also lower the level of every variable of a term bound to an older variable (this is the shape of the fourth query of Section 10.9.3, with `list` for f). With the orientation of (aux kanren micro) an old variable can moreover be aliased to a young one, and a scope check must look through such chains (Section 10.9.3).

Structures and atoms. Records unify as the list of their slots, so any `define-record` is a new term constructor for free; but a record and a vector with the same slot list unify too. Numbers are compared with `eqv?`, so an exact and an inexact number never unify.

```
(define-record dbi n)
(pretty-print (record->vector (make-dbi 3)))
(pretty-print (°->list #f (fresh° (q) (=° (make-dbi q) (vector 'dbi 7)))))
(pretty-print (°->list #f (fresh° (q) (=° (vector 1 q) '(1 2)))))
(pretty-print (°->list #f (fresh° (q) (=° (/ 0. 0.) (/ 0. 0.)) (=° "ab" (string #\a
↪  #\b)) (=° q 'ok))))
(pretty-print (°->list #f (fresh° (q) (=° 1 1.0) (=° q 'ok))))
```

```
#(dbi 3)
((λ () 7))
()
((λ () 'ok))
()
```

In order: the slot list of a record starts with its type name; a `dbi` record unifies with the vector `#(dbi 7)`; a vector never unifies with a list; NaN equals NaN and equal strings unify although they are distinct objects; `1` and `1.0` do not unify. Anything else (procedures, bytevectors, ...) unifies only with itself under `eq?`.

Pitfall: Procedures are opaque

A Scheme closure is an atom compared with `eq?`. Higher-order abstract syntax with real Scheme procedures (the λProlog `abs x\ ...` rendered as `(λ (x) ...)`) cannot be unified, walked, constrained or reified. λ-terms must be first-order data, e.g. de Bruijn records such as `dbi` above (Chapter 6).

Kanren angle: Where higher-order unification would go

The `cond` of Definition 10.5 is closed: there is no dispatch table and no hook. A new term kind with *syntactic* equality needs nothing (it is a record); a term kind with a different equality, such as L_λ flexible terms $F\,x_1 \ldots x_n$ solved by pattern unification (Chapter 7) or nominal terms with suspended swappings [5, 58], needs either a new clause here (before case 6) or a goal that solves the problem outside and emits ordinary bindings (Section 10.9.2). The choice between them is made in Chapter 11.

10.5 The constraint stores and their re-verification

Unification only binds; the three stores keep *negative* and *type* information, and must be re-checked whenever a binding is made.

10.5.1 D: disequalities

Definition 10.6 (Prefix)**.** A *prefix* is a list $d = ((x_1.t_1)\dots(x_k.t_k))$ of variable–term pairs; it denotes the constraint $\neg(x_1 = t_1 \wedge \dots \wedge x_k = t_k)$. The store D is a list of prefixes, read as their conjunction.

The goal `(≠° u v)` (`:554–567`) runs a *trial* unification of u and v with an empty `assocs` and reads the result:

- `#f`: u and v can never be equal, so the constraint is dropped and the goal succeeds with the state unchanged;
- the very same state (Proposition 10.3): u and v are already equal, so the goal fails;
- a new state with bindings d: the goal succeeds with the *old* state extended by the prefix d in D, unless a type tag of A or an absento of T makes d vacuous (subsumption, below).

A tuple disequality gives one prefix with several pairs. After a later `=°`, a prefix is re-verified by trial unification of its pairs (`μkanren-verify-D+`, `:458–461`): if the trial fails the prefix is satisfied forever and dropped; if it adds nothing the prefix is violated and the `=°` fails; otherwise the prefix is replaced by its normalised version, the new trial bindings.

Tutorial: The life of a tuple disequality

We assert $(x, y) \neq (1, 2)$ and then bind x, watching D.

```
(for-each dump (states (fresh° (q x y) (=° q (list x y)) (≠° (list x y) '(1
↪  2)))))
(for-each dump (states (fresh° (q x y) (=° q (list x y)) (≠° (list x y) '(1 2))
↪  (=° x 1))))
(for-each dump (states (fresh° (q x y) (=° q (list x y)) (≠° (list x y) '(1 2))
↪  (=° x 3))))
(write (states (fresh° (q x y) (=° q (list x y)) (≠° (list x y) '(1 2)) (=° x
↪  1) (=° y 2)))) (newline)
```

```
vars-count 3  S ((_1 _2))  D (((_2 . 2) (_1 . 1)))  A ()  T ()
vars-count 3  S (1 (_1 _2))  D (((_2 . 2)))  A ()  T ()
vars-count 3  S (3 (_1 _2))  D ()  A ()  T ()
()
```

1. The trial unification of `(_1 _2)` with `(1 2)` would bind both variables, so D holds the single prefix `((_2 . 2) (_1 . 1))`: “not both”.
2. After `(=° x 1)`, `_1` is touched; the trial now binds only `_2`, and the prefix is normalised to `((_2 . 2))`: the constraint has become $y \neq 2$.

3. After `(=° x 3)` the trial fails: the prefix can never be violated and disappears.
4. After `(=° x 1)` and `(=° y 2)` the trial binds nothing: the prefix is violated and the goal has no answer state.

Incremental re-verification. After an `=°`, only the prefixes that mention a variable just bound (on either side of a pair) are re-verified (`µkanren-verify-D/touched`, `:472–478`); the `assocs` returned by unification is exactly the list needed to decide which prefixes are touched. An untouched prefix cannot have become violated: its keys are unbound variables, and since none of them was just bound, its trial unification would still bind its first key (the reasoning of the comment at `:466–471`). It may, however, have become satisfied through the occurs check deep in its terms; projection runs one last full `µkanren-verify-D` to drop those (`:466–471`, `:317`).

10.5.2 A: disjoint type tags

A *tag* is a record `(name def pred)` (`:31`): an identity, the quoted source of a predicate (printed in answers), and the predicate itself. Type tags are made with `define-µkanren-tag` and turned into goals by `µkanren-make-tag-A` (`:498–504`); the built-in ones are (`:611–615`)

```
(define-µkanren-tag (µkanren-tag/sym v) (symbol? v))
(define-µkanren-tag (µkanren-tag/num v) (number? v))
(define symbol° (µkanren-make-tag-A µkanren-tag/sym))
(define number° (µkanren-make-tag-A µkanren-tag/num))
```

On a bound term the goal just tests the predicate; on an unbound variable it extends A. Tags are *mutually exclusive*: `µkanren-ext-A` (`:431–436`) answers `#f` when the variable already carries a different tag. Adding a tag also drops the prefixes of D that it makes vacuous (`µkanren-subsume`, `:370–371`: a pair `(α . u)` whose α is tagged by a predicate rejecting u).

```
(for-each dump (states (fresh° (q) (≠° q 'a) (number° q))))
(write (states (fresh° (q) (number° q) (symbol° q)))) (newline)
```

```
vars-count 1  S ()  D ()  A ((_0 . µkanren-tag/num99))  T ()
()
```

A number can never be the symbol `a`, so the disequality is gone; and no term is both a number and a symbol.

10.5.3 T: absento

`(absent° datum term)` (`:617–624`; note the argument order) states that *datum*, which may be any Scheme value, atomic or compound, does not occur in *term*. The store holds `(term . tag)` pairs where the tag is an ad-hoc tag whose name is the datum itself. The term is a variable, or a compound that could still become the datum (it is then "watched whole" and decomposed only when it no longer can, `µkanren-verify-T+`, `:410–419`). When a watched variable is bound, the constraint moves to the parts:

```
(for-each dump (states (fresh° (q x y) (absent° 'k q) (=° q (list x y)))))
(write (states (fresh° (q x y) (absent° 'k q) (=° q (list x y)) (=° y 'k)))) (newline)
```

```
vars-count 3  S ((_1 _2))  D ()  A ()  T ((_2 . k) (_1 . k))
()
```

When a variable carrying an absento gets a type tag, the absento is converted into a disequality (e.g. `symbol° α` plus `absent° 'closure α` becomes the prefix `((α . closure))`), by `µkanren-subsume-A` and `µkanren-update-D/T` (`:340–356`, `:480–485`). The conversion happens for *any* tag, and because the subsumption of D by the new tag runs before it (`:493`), a converted prefix that the tag makes vacuous stays in the state and is dropped only at projection:

```
(for-each dump (states (fresh° (q) (absent° 'closure q) (symbol° q))))
(for-each dump (states (fresh° (q) (absent° 'closure q) (number° q))))
(pretty-print (°->list #f (fresh° (q) (absent° 'closure q) (number° q))))
```

```
vars-count 1  S ()  D (((_0 . closure)))  A ((_0 . µkanren-tag/sym98))  T ()
vars-count 1  S ()  D (((_0 . closure)))  A ((_0 . µkanren-tag/num99))  T ()
((λ (α) (assert (every (µ v (number? v)) (list α))) α))
```

For a symbol the prefix is meaningful ($q \neq$ closure); for a number it is vacuous, and the answer does not show it. A new store that reads D directly must expect such leftovers.

10.5.4 The post-unification chain

The goal `=°` (`:539–552`, re-indented to fit the page) is where everything meets:

```
(define ((=° u v) s)
  (µkanren-state-match ((vc S D A T tags) s)
    (define post°
      (λ1-match/first
        ((,s* ,assocs)
         (cond
           ((µkanren-state-equal? s s*) (✓° s*))          ; nothing bound: done
           ; D, touched prefixes only
           ((µkanren-verify-D/touched D s* assocs)
            => (µ D* (cond                                 ; then A, T, subsumptions
                       ((µkanren-verify-D/post D* A T s*) => ✓°)
                       (else (✗° s*)))))
           (else (✗° s*))))))
    (cond
      ((µkanren-state-unify u v s '()) => post°)
      (else (✗° s)))))
```

After a unification that binds something, the stores are re-verified in a hard-wired order:

D (touched prefixes only) → A (in full) → D subsumed by A → T (in full) → D subsumed by T
→ absentos of typed variables turned into D,

through µkanren-verify-D/post (:450–453), µkanren-verify-A/post (:424–429), µkanren-verify-T/post (:373) and µkanren-subsume-T (:352–356), which rebuilds the state with the new D, A and T. Only D is incremental; A and T are re-verified in full after every binding =°. The whole chain is skipped only when nothing was bound (Proposition 10.3).

Kanren angle: A new store has nowhere to plug in

There is no list of verifiers to which a new store could append itself: the chain is a fixed sequence of unexported procedures. λProlog features that need state of their own all hit this wall: a dynamic program for AUGMENT [MN p. 39], scope information for GENERIC, a store of postponed non-pattern equations that must be woken after each binding (Chapter 7). What such a store requires of the engine is (i) a place in the state that every goal copies forward, (ii) a call after each binding with the new assocs, (iii) a say in reification. Section 10.9.2 shows how far one gets without (ii) and (iii); Chapter 11 proposes how to provide them.

10.6 Goals and the interleaving search

10.6.1 Streams and goals

Definition 10.7 (Streams and goals)**.** A stream ((aux stream)) is '(), a *mature* pair (state . stream), or an *immature* CHICKEN promise made with δ (delay, memoising). A *goal* is a procedure from a state to a stream of states. A *relation* is a procedure from terms to a goal.

The primitives (:529–570; an excerpt, comments ours) are few:

```
; succeed once with the given state
(define ✓° list)
(define ✗° (K '()))                                  ; fail
(define ((freshª f) s)
  (µkanren-state-match ((vc S D A T tags) s)
    (let* ((α   (make-µkanren-var vc))
           (g   (f α))                               ; the body is built only now
           (s*  (make-µkanren-state (add1 vc) S D A T tags)))
      (δ (g s*)))))                                  ; and suspended
(define ((orª f g) s) (append§/interleaved/2 (δ (f s)) (δ (g s))))
(define ((andª f g) s) (append-map§ g (δ (f s))))
```

The suffix ª marks the *applicative* procedure behind a macro; ° marks a goal, a goal-making macro or a relation. Interleaving lives in (aux stream) (aux.stream.scm:80–85):

```
(define append§/interleaved/2
  (λ (s1 s2)
    (cond
      ((null? s1) s2)
      ; SWAP on suspension
      ((promise? s1) (δ (append§/interleaved/2 s2 (force s1))))
      (else (cons§ (car s1) (append§/interleaved/2 (cdr s1) s2))))))
```

When the left stream is suspended, the two streams swap places; this is what makes disjunction complete, and every `δ` is a point where control can pass to another branch.

The macros (:575–593, :626–629). `(fresh° (x y) g ...)` nests one `freshᵃ` per variable, each suspending once; `(fresh° q ← (a b) g ...)` also binds `q` to `(list a b)`. `and°` is *left*-nested, `(and° g1 g2 g3)` = `(andᵃ (andᵃ g1 g2) g3)`; `or°` is *right*-nested, `(or° g1 g2 g3)` = `(orᵃ g1 (orᵃ g2 g3))`. `cond°` is conde: `(or° (and° g ...) ...)`, with no `else`. `null°`, `boolean°`, `cons°` are one-liners over `=°` and `or°`.

Tutorial: Fairness, and its shape

Two infinite generators in a disjunction share the answers:

```
(define-relation (nat° n) (or° (=° n 'z) (fresh° (m) (=° n `(s ,m)) (nat° m))))
(write (°->list/ground (take° 6 (fresh° (q) (or° (fresh° (n) (nat° n) (=° q
↪  `(left ,n)))
                                                 (fresh° (n) (nat° n) (=° q `(right
                                                     ↪  ,n)))))))) (newline)
```

```
((left z) (right z) (left (s z)) (right (s z)) (left (s (s z))) (right (s (s z))))
```

The interleaving is fair between two branches, but `or°` nests to the right, so with three branches the first one competes against the *pair* of the other two:

```
(define-relation (numb° n k) (or° (=° n k) (numb° n (+ k 1))))
(write (°->list/ground (take° 12 (fresh° (q) (fresh° (n)
          (or° (and° (numb° n 0) (=° q (list 'a n)))
               (and° (numb° n 0) (=° q (list 'b n)))
               (and° (numb° n 0) (=° q (list 'c n))))))))) (newline)
```

```
((a 0) (a 1) (b 0) (a 2) (c 0) (a 3) (b 1) (a 4) (c 1) (a 5) (b 2) (a 6))
```

In the long run branch `a` gets half of the answers, `b` and `c` a quarter each (here 7, 3 and 2 of the first 12). Conjunction, on the other hand, is not fair: `(and° generator failing-test)` diverges, so as in every miniKanren generators go before tests and recursive calls last. For instance, the `append°` of Section 10.7 with its recursive call placed *before* `(cons° a c rs)` still finds the four splits of `(1 2 3)`, but then searches forever for a fifth one under an unbounded run, while the order shown there terminates.

10.6.2 `define-relation` and the inverse-η delay

Because `and°` and `or°` expand into *procedure calls* of `andᵃ` and `orᵃ`, their goal arguments are evaluated when the goal tree is built, before any state exists. A recursive relation written with plain `define` whose recursive call is not under a `fresh°` therefore builds an infinite tree:

```
(define (ones° x) (or° (=° x 1) (ones° x)))          ; plain define: no delay
(write (°->list/ground (take° 3 (fresh° (q) (ones° q))))) (newline)
```

Run under `timeout 10`, this prints nothing and is killed (`exit=124`). The macro of `:690` applies an inverse-η expansion and a suspension:

```
(define-syntax-rule (define-relation (name arg ...) g ...) (define (name arg ...) (λ
↪  (s) (δ ((and° g ...) s)))))
```

so the body is built only when the goal meets a state, and then suspended:

```
(define-relation (ones° x) (or° (=° x 1) (ones° x)))
(write (°->list/ground (take° 3 (fresh° (q) (ones° q))))) (newline)
```

```
(1 1 1)
```

Plain `define` remains fine when every recursive call sits under a `fresh°`, whose body is under `λ (α)`; the relational interpreter of `test/microkanren-untagged.scm` is written that way.

Pitfall: Answer order is observable

Each `fresh°` variable and each relation call adds a suspension, `or°` adds two. The test suites pin the raw answers, *including their order*. A new goal that adds or removes a single `δ` (for example a new `=°`-like goal) can permute the answers of existing suites.

10.6.3 Soft cut, bounded runs, escape to Scheme

`(if° g? gt gf)` (`:596–603`) is LogicT's `ifte`, i.e. miniKanren's `conda` for one clause: if `g?` has at least one answer, *all* its answers are continued with `gt`; otherwise `gf` runs on the original state. `(take° n g)` (`:605`) keeps the first n answers of `g` (all of them if $n < 0$); `(take° 1 g)` is `once`. With `gt` $=$ `✗°` and `gf` $=$ `✓°`, `if°` is negation as failure:

```
(write (°->list/ground (fresh° (q) (if° (or° (=° q 1) (=° q 2)) ✓° (=° q 3)))))
(newline)
(write (°->list/ground (fresh° (q) (if° ✗° ✓° (=° q 3))))) (newline)
(write (list (°->list/ground (fresh° (q) (=° q 1) (if° (=° q 2) ✗° ✓°)))
             (°->list/ground (fresh° (q) (=° q 2) (if° (=° q 2) ✗° ✓°)))
             (°->list/ground (fresh° (q) (if° (=° q 2) ✗° ✓°))))) (newline)
(write (°->list/ground (fresh° (q) (take° 2 (nat° q))))) (newline)
```

```
(1 2)
(3)
((1) () ())
(z (s z))
```

The third line exhibits the classic unsoundness: with `q` unbound, "`q` is not 2" fails, because the test succeeds by binding `q`.

λProlog's `if` and `not` are defined with the cut [MN p. 137]:

```
type if   o -> o -> o -> o.
if P Q R :- P, !, Q.
if P Q R :- R.
```

```
type not  o -> o.
not P :- P, !, fail.
not P.
```

and the book's example of their non-logical character, "`?- X = 2, not (1 = X).` succeeds, whereas `?- not (1 = X), X = 2.` fails" [MN p. 137–138], behaves the same way with `if°`. There is one difference: the cut keeps only the *first* proof of `P`, while `if°` keeps all the answers of its test, so λProlog's `if P Q R` is `(if° (take° 1 P) Q R)`:

```
; X = 2, not (1 = X)
(write (list (°->list/ground (fresh° (q) (=° q 2) (if° (=° 1 q) ✗° ✓°)))
             ; the converse
             (°->list/ground (fresh° (q) (if° (=° 1 q) ✗° ✓°) (=° q 2)))))
(newline)
(write (list (°->list/ground (fresh° (q) (if° (or° (=° q 1) (=° q 2)) ✓° ✗°)))
             (°->list/ground
               (fresh° (q) (if° (take° 1 (or° (=° q 1) (=° q 2))) ✓° ✗°)))))
(newline)
```

```
((2) ())
((1 2) (1))
```

`(project° ((x* x) ...) g ...)` (`:626–627`) binds Scheme variables to the deep-walked values of logic terms and runs `g ...`: the non-relational escape hatch. An unbound variable is seen as a `μkanren-var` record.

```
(write (°->list/ground (fresh° (q x) (=° x 20) (project° ((x* x)) (=° q (* 2 x*))))))
↪  (newline)
(pretty-print (°->list #f (fresh° (q x) (project° ((x* x)) (=° q (list (μkanren-var?
↪  x*) (μkanren-var-index x*)))))))
```

```
(40)
((λ () (cons #t (cons 1 '()))))
```

Kanren angle: AUGMENT and GENERIC as goals

A goal sees the whole state and returns a stream of states, so it can do more than `=°` does: it can change the state on entry and post-process every answer state on exit. That is the shape that both $D \supset G$ (add D, solve G, then restore the program on every answer of G) and $\forall x\, G$ (create c, solve G, then check that c did not escape) require [MN p. 39]. The two prototypes of Section 10.9.2 are written exactly this way, with `append-map§` over the answers of G.

10.7 Reification and the runners

10.7.1 From a state to a λ-expression

An answer is not a substitution but a *closed Scheme expression*

`(λ (α β ...) constraint-form ... term)`

which, applied to values for the free variables, re-checks the constraints with `assert` and `deny` and returns the term. It is built by `µkanren-project` (`:310–326`) in six steps:

1. deep-walk the query term;
2. name its free variables in order of first occurrence (left to right, car before cdr) by *binding* each of them to a fresh reified variable $-1, -2, \ldots$ in a scratch state (`µkanren-state-reify`, `:207–222`);
3. re-verify D in full;
4. *purify*: drop every constraint that mentions a variable unreachable from the answer (still working in the scratch state); this is sound for disequalities, since such a variable can always be chosen to satisfy them;
5. drop the prefixes subsumed by others or made vacuous by a tag;
6. render with `µkanren-form` (`:243–249`), sorting everything lexicographically.

store	form in the answer
D, prefix of one pair	`(begin (deny (equal? α <repr>)))`
D, prefix of *n* pairs	`(begin (deny (and (equal? α ...) (equal? β ...))))`
A, grouped by tag	`(assert (every (µ v (symbol? v)) (list α γ)))`
T	`(assert (absent? '<datum> <repr>))`

```
(pretty-print (°->list #f (fresh° (q x y) (=° q (list x y)) (≠° x 'a) (symbol° x)
↪  (absent° 'z y))))
(pretty-print (°->list #f (fresh° (q x y) (=° q (list x y)) (≠° (list x y) '(1 2)))))
(pretty-print (°->list #f (fresh° (q) (fresh° (h) (≠° h 3) (=° q 7)))))
(pretty-print (°->list/ground (fresh° (q x) (=° q (list x 1)) (≠° x 2))))
```

```
((λ (α β)
    (begin (deny (equal? α 'a)))
    (assert (every (µ v (symbol? v)) (list α)))
    (assert (absent? 'z β))
    (cons α (cons β '())))
 (λ (α β)
    (begin (deny (and (equal? α 1) (equal? β 2))))
    (cons α (cons β '())))
 ((λ () 7))
 ((α 1))
```

The third answer shows purification: the disequality on the unreachable `h` is gone. The fourth shows grounding: `°->list/ground` evaluates only `(lambda vars term)`, *dropping the constraints*, and applies it to the variable symbols themselves, so the free variable reads back as the symbol `α`. The show module (`aux kanren micro show`) parses the answer forms back without `eval` (`aux.kanren.micro.show.scm:58–72`), so a new kind of constraint in answers needs a case there too.

10.7.2 Running

The runners are at `:686–713`:

```
(define (°->§ g)
  (let* ((§ (δ (g µkanren-state-empty)))
         (P (µkanren-project (make-µkanren-var 0))))
    (map§ P §)))
(define-syntax-rule (µkanren-run (v n grounded) g ...) (°->list grounded (take° n
↪  (fresh° (v) g ...))))
```

`(°->list grounded g)` forces `°->§` and, if `grounded`, `eval`s each answer in micro's own module environment; `°->list/ground` is `(°->list #t g)`. `µkanren-run` is the TRS-like `run`, with $n < 0$ meaning "all". The query variable is always *the variable of index 0*, i.e. the first one the goal introduces. With the usual `append°`:

```
(define-relation (append° r s rs)
  (cond°
    ((null° r) (=° s rs))
    ((fresh° (a d c) (cons° a d r) (cons° a c rs) (append° d s c)))))
(pretty-print (µkanren-run (q -1 #t) (fresh° (x y) (append° x y '(1 2 3)) (=° q (list
↪  x y)))))
(pretty-print (µkanren-run (q 3 #f) (fresh° (x y z) (append° x y z) (=° q (list x y
↪  z)))))
```

```
((() (1 2 3)) ((1) (2 3)) ((1 2) (3)) ((1 2 3) ()))
((λ (α) (cons '() (cons α (cons α '()))))
 (λ (α β) (cons (cons α '()) (cons β (cons (cons α β) '()))))
 (λ (α β γ)
    (cons (cons α (cons β '())) (cons γ (cons (cons α (cons β γ)) '())))))
```

Pitfall: The query variable is index 0

`°->§` hard-wires the empty initial state and projects variable 0. An extension that reserves indices (for eigenvariables, registers, ...) *before* the first `fresh°` shifts the query variable; a module that needs a richer initial state needs its own runner, which is easy since `make-µkanren-var` and `µkanren-project` are exported.

Kanren angle: What answers will have to show

Eigenvariables and a dynamic program never appear in today's answers. If a `≠°` mentioning an eigenvariable survives its scope, or a postponed higher-order equation is left at the end, reification must either drop it or print it; purification (step 4) does not know about scopes. See Section 10.9.3 for an example and Chapter 11 for the proposal.

10.8 Aggregation

Four SQL-like goals (`:631–684`) consume the *whole* answer stream of a sub-goal with `foldr§` and turn it into Scheme data, so the sub-goal must be finite and the goals are non-relational

(`findall`/`bagof`-like). `groupby°` groups the answers of `g` by the values of the keys and runs the continuation once per group *on the original state*, with the aggregate and the key names bound as Scheme variables; `window°` is `OVER (PARTITION BY ...)` and runs the continuation on each answer state of `g`; `set°` folds per key; `enumerate°` numbers the answers in stream order.

```
(pretty-print (°->list/ground (fresh° (q) (fresh° (k v)
   (groupby° (((vs (λ (l) l)) v)) over (k) from
             (or° (and° (=° k 'a) (=° v 1)) (and° (=° k 'b) (=° v 2)) (and° (=° k 'a)
              ↪  (=° v 3)))
             => (=° q (list k vs)))))))
```

```
((b (2)) (a (1 3)))
```

They are the precedent for goals that run a sub-search to completion, the mechanism a `findall`-style predicate would use.

10.9 Extension points

10.9.1 Open versus closed

Table 10.4 sorts the engine's parts by whether an importing module can use or extend them without editing `aux.kanren.micro.scm`.

Table 10.4: The inventory of (aux kanren micro) for an extension.

open (usable from outside)	**closed** (needs an edit of micro)
Goals are plain procedures *state* → *stream*; all field accessors and `μkanren-state-with` are exported.	The field list of the state (`:69`), its positional constructions (`:71`, `:83`, `:132`, `:536`), the six-ary `μkanren-state-match` and its six users.
Raw unification `μkanren-state-unify` returns the new state and the exact new bindings (but does not re-verify D/A/T).	The unification `cond` (`:160–181`): no hook for a term kind with non-syntactic equality.
Walks `μkanren-state-find`, `/value`, `/values`, `/repr`.	The binding site with occurs check, `μkanren-state-update`, unexported: the only place to refuse a binding before it enters the state (a check right after each `=°` works as well, Section 11.4.3).
Variables: `make-μkanren-var`, `μkanren-var-index`, ...; a fresh unique integer by bumping `vars-count`.	The post-`=°` chain (`:539–552`, `:373–478`), wired for D/A/T, its pieces unexported.
New first-order constructors: any `define-record` unifies, reifies, works with `≠°` and `absent°`.	Reification (`:310–326`, `:243–249`): knows only D/A/T; purification and subsumption are store-specific.

open	closed
New disjoint types: `define-μkanren-tag` with `μkanren-make-tag-A`. Projection `μkanren-project`; the combinators `freshᵃ`, `andᵃ`, `orᵃ`, `if°`, `take°`, `project°`; all of `(aux stream)`; `define-relation`.	`°->§`: fixed empty initial state, fixed query index 0.

Two of the open entries in action: a user-defined disjoint type, and a goal written as a bare lambda that reads the state.

```
(define-μkanren-tag (tag/str v) (string? v))
(define string° (μkanren-make-tag-A tag/str))
(pretty-print (°->list #f (fresh° (q) (string° q))))
(pretty-print (map (λ (g) (°->list #f g))
                   (list (fresh° (q) (string° q) (=° q "hi"))
                         (fresh° (q) (string° q) (=° q 3))
                         (fresh° (q) (string° q) (symbol° q))
                         (fresh° (q) (string° q) (≠° q 3)))))
(pretty-print (°->list/ground (fresh° (q x y) (λ (s) ((=° q
↪  (μkanren-state-vars-count s)) s)))))
```

```
((λ (α) (assert (every (μ v (string? v)) (list α))) α))
(((λ () "hi")) () () ((λ (α) (assert (every (μ v (string? v)) (list α))) α)))
(3)
```

The new tag is printed in answers, is disjoint from `symbol°`, and subsumes `(≠° q 3)`. The last goal sees `vars-count` = 3.

A CHICKEN rule constrains every new module: a macro exported from a module whose expansion refers to an unexported binding fails at the use site. This is why `freshᵃ`, `andᵃ`, `orᵃ`, `✓°`, `✗°`, `=°` and `μkanren-group§` are exported (`:4`–17); macros such as a future `∀°` or `⊃°` must follow the same discipline.

10.9.2 Three tricks that need no edit, and their limits

Trick 1: wrapper goals that emit `=°`. A solver written outside the engine (say, higher-order pattern unification computing a most general unifier $\{F \mapsto t_1, \ldots\}$) should express its result as ordinary `=°` goals, `(and° (=° F t1) ...)`, so that each binding runs the D/A/T chain. The tempting shortcut, calling the exported raw unification and returning its state, silently bypasses the stores:

```
; WRONG: bypasses the D/A/T re-verification
(define ((bind/raw x t) s)
  (let1 (r (μkanren-state-unify x t s '())) (if r (✓° (car r)) (✗° s))))
; right: the result is expressed as an ordinary =°
(define ((bind/emit x t) s) ((=° x t) s))

(define raw (§->list ((fresh° (q) (≠° q 'a) (bind/raw q 'a)) μkanren-state-empty)))
```

```
; q = a and q ≠ a!
(pretty-print (map (λ (s) (list (sbral->list (μkanren-state-S s)) (μkanren-state-D
↪  s))) raw))
(pretty-print (handle-exceptions e (list 'error ((condition-property-accessor 'exn
↪  'message) e))
                (°->list #f (fresh° (q) (≠° q 'a) (bind/raw q 'a)))))
(pretty-print (°->list #f (fresh° (q) (≠° q 'a) (bind/emit q 'a))))
```

```
(((a) (((_0 . a)))))
(error "bad argument type - not a list")
()
```

The raw version produces a state where S says $q = \mathrm{a}$ and D says $q \neq \mathrm{a}$; reification then re-verifies D, gets `#f` for "violated", and crashes. The emitting version fails, as it should. *Limits*: extra suspensions and repeated verification; and plain `=°` applied by a user to terms that need the special solver would still unify them syntactically, so a module must forbid that or shadow `=°`.

Trick 2: S-registers. There is no way to add a field from outside: every goal of micro rebuilds the state positionally, so a wrapper record would be lost at the next `=°`. Dynamic scoping (`parameterize`) fails too, because promises are forced outside its extent, and so does a side table keyed by the state, because a new state is allocated at every binding. But S itself is copied forward by every goal and is persistent and branch-local. So: reserve a fresh index n (bump `vars-count`), bind variable n by raw unification to an opaque record carrying the data, and find the current value as the register of highest index. Raw unification is harmless here because the register variable is fresh and unconstrained, and registers are never reachable from the query variable, so they are never reified. Here the data is a λProlog-style program context, giving a prototype of AUGMENT:

```
; a register: an opaque record bound in S
(define-record ctx clauses)
; reserve index n, bind var n (raw unify) to a ctx record
(define (ctx-push s Γ)
  (let* ((n (μkanren-state-vars-count s))
         (s1 (μkanren-state-with s vars-count: (add1 n))))
    (car (μkanren-state-unify (make-μkanren-var n) (make-ctx Γ) s1 '()))))
; the most recent register, by a shallow walk downwards
(define (ctx-current s)
  (let loop ((i (sub1 (μkanren-state-vars-count s))))
    (cond ((< i 0) '())
          (else (let1 (v (μkanren-state-find (make-μkanren-var i) s))
                  (if (ctx? v) (ctx-clauses v) (loop (sub1 i))))))))
; D ⊃ G, D a list of clause procedures (λ (x) goal)
(define ((⊃ª D g) s)
  (let1 (Γ (ctx-current s))
    (append-map§ (λ (s2) (✓° (ctx-push s2 Γ))) (δ (g (ctx-push s (append D Γ)))))))
; a dynamic predicate: try every clause of the context
(define ((hyp° x) s)
```

```
  (let1 (Γ (ctx-current s)) ((fold-right (λ (c g) (orª (c x) g)) ✗° Γ) s)))

(define (is° v) (λ (x) (=° x v)))            ; the clause  hyp X :- X = v
(for-each (λ (g) (write (°->list/ground g)) (newline))
  (list (fresh° (q) (⊃ª (list (is° 'a)) (hyp° q)))
        (fresh° (q) (⊃ª (list (is° 'a)) ✓°) (hyp° q))
        (fresh° (q) (⊃ª (list (is° 'a)) (⊃ª (list (is° 'b)) (hyp° q))))
        (fresh° (q) (or° (⊃ª (list (is° 'a)) (hyp° q)) (hyp° q)))
        (fresh° (q) (fresh° (y) (⊃ª (list (λ (x) (=° x y))) (fresh° (u) (hyp° u) (=°
        ↪  u 7))) (=° q y)))))
```

```
(a)
()
(b a)
(a)
(7)
```

In order: the assumed clause is used; outside its scope it is gone; nested assumptions stack, newest first; two branches of a disjunction do not see each other's assumptions; and a free logic variable `y` of an assumed clause is *shared* with the context, not renamed, as in λProlog [MN p. 86–88]. *Limits*: the lookup scans the indices, $O($`vars-count`$\cdot\log)$ per call; registers must be allocated after the query variable; it is a library convention, not an engine guarantee.

10.9.3 Trick 3: a ∀ prototype with a post-filter scope check

The eigenvariable is a record whose id is a reserved index, hence unique; distinct eigenvariables never unify (their id slots differ). The scope check is a post-filter on each answer state of the body: every variable of index $< n$, i.e. older than the eigenvariable, is deep-walked and must not contain it.

```
(define-record eigen id)
(define (contains? c t)
  (cond
    ; find/value REBUILDS records: compare ids, not identity
    ((eigen? t) (= (eigen-id t) (eigen-id c)))
    ((pair? t) (or (contains? c (car t)) (contains? c (cdr t))))
    ((vector? t) (any (λ (u) (contains? c u)) (vector->list t)))
    ((and (record-instance? t) (not (µkanren-var? t)))
     (any (λ (u) (contains? c u)) (vector->list (record->vector t))))
    (else #f)))
(define ((∀ª f) s)
  (let* ((n (µkanren-state-vars-count s))
         (c (make-eigen n))                      ; unique: index n is reserved
         (s* (µkanren-state-with s vars-count: (add1 n)))
         (escapes? (λ (s1) (let loop ((i 0))
                             (and (< i n)
                                  (or (contains? c (µkanren-state-find/value
                                   ↪  (make-µkanren-var i) s1))
                                      (loop (add1 i)))))))))
```

```
    (append-map§ (λ (s1) (if (escapes? s1) '() (✓° s1))) (δ ((f c) s*)))))
(define-syntax-rule (∀° (x) g ...) (∀ª (λ (x) (and° g ...))))

(for-each (λ (g) (pretty-print (°->list #f g)))
  (list (fresh° (q) (∀° (x) (=° q x)))                              ; 1. ∃Q ∀x. Q = x
        (fresh° (q) (∀° (x) (fresh° (y) (=° y x))) (=° q 'ok))      ; 2. ∀x ∃Y. Y = x
        (fresh° (q) (∀° (x) (∀° (y) (=° x y))))                     ; 3. ∀x ∀y. x = y
        (fresh° (q) (∀° (x) (fresh° (y) (=° q (list y)) (=° y x)))) ; 4. alias
        (fresh° (q) (∀° (x) (fresh° (z) (=° q (list 'f z)))))       ; 5.
        (fresh° (q) (∀° (x) (≠° q x))))                             ; 6. ≠ survives
```

```
()
((λ () 'ok))
()
()
((λ (α) (cons 'f (cons α '()))))
((λ (α) (begin (deny (equal? α (make-record-instance 'eigen 1)))) α))
```

Tutorial: Reading the six answers

1. $\exists Q\, \forall x.\, Q = x$ is the book's `sigma x\ pi y\ x = y`, "not provable" [MN p. 85]. Here `q` is `_0`; `∀°` reserves index 1 and makes `c` = `(eigen 1)`; the body binds `_0 := c`; the post-filter walks `_0`, finds `c`, and discards the state.
2. $\forall x\, \exists Y.\, Y = x$ succeeds: `y` has index $2 \geq n = 1$, so it is inside the scope.
3. Two eigenvariables `(eigen 1)` and `(eigen 2)` differ in their id slot: unification fails.
4. The escape is indirect: `_0 := (list _2)` and `_2 := c`. Because the check deep-walks the old variable, it sees `(list c)` and rejects the state. This is the chain created by the older-to-newer orientation of Section 10.4.
5. A younger variable left unbound is harmless and is reified as `α`.
6. A disequality mentioning `c` survives in D and is reified with the eigenvariable in it: the check filters bindings, not constraints.

Why is checking at the end of the body enough for soundness? After the scope closes, `c` is lexically unreachable and, by the check, no term reachable from an older variable contains it, so no later goal can bring it back. (This argument is informal; the prototype was tested, not proved.) What is lost is *early pruning*: a bad binding is detected only when the body has produced an answer, at a cost of $O(n \cdot |\text{term}|)$ per answer. Eager checking needs the scope test at the binding site (closed), or a post-unification hook with a level map, or a module-level `=°` that shadows micro's and checks its bindings against the open scopes kept in an S-register; the last option covers only goals written with the new module's `=°`.

Pitfall: The scope check must not use eq?

With the first line of `contains?` written as `((eigen? t) (eq? t c))`, the same query as answer 1 wrongly succeeds, because `find/value` rebuilt the record (Section 10.3):

```
(pretty-print (°->list #f (fresh° (q) (∀° (x) (=° q x)))))
```

```
((λ () (make-record-instance 'eigen 1)))
```

Kanren angle: What the tricks leave open

The three tricks show that the shape of AUGMENT and GENERIC fits (aux kanren micro) today: a goal that changes the state on entry and filters or restores it on every answer. What they cannot provide is what only the engine can do: a *named* place in the state (instead of a register scan), a call after every binding (for eager scope checks and for waking postponed equations), and a say in reification (to drop out-of-scope constraints and print new ones). These are the requirements Chapter 11 turns into a proposal, and Chapter 12 into a module.

10.10 Summary

- Terms are Scheme data; logic variables are records with an integer index, working (≥ 0, printed `_i`) or reified (< 0, printed α, β, ...). Indices grow monotonically along a branch, so an index is an age and doubles as a level boundary.
- A state has six fields `vars-count`, S, D, A, T, `tags` (the last one dead). States are compared by `eq?`: unification returns the same state iff it binds nothing, and every entailment test relies on this.
- S is an sbral, most recent variable first, extended lazily; the walk follows triangular bindings and treats reified variables as constants; deep walks rebuild records, losing their identity.
- Unification is first-order with occurs check; the only binding site is `μkanren-state-update`; the older variable is bound to the newer; vectors and records unify as slot lists (so records and same-shaped vectors unify); numbers by a NaN-reflexive `eqv?`; closures only by `eq?`.
- D holds prefixes (negated conjunctions of bindings), re-verified incrementally on the touched prefixes; A holds disjoint type tags, T absento constraints, both re-verified in full; the chain after `=°` is fixed and has no slot for a new store.
- Goals are *state* → *stream*; `orª` interleaves by swapping on suspensions (`or°` nests right, so three branches get $\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{4}$); conjunction is not fair; `define-relation` delays recursive relations; `if°` is `conda` (it keeps every answer of its test, so λProlog's cut-based `if` is `if°`

on a `(take° 1 ...)` test), `take°` bounds a run, `project°` escapes to Scheme; answer order is pinned by the tests.

- Answers are closed expressions `(λ (vars) constraints ... term)`, purified of unreachable variables; grounding drops the constraints; the query variable is index 0.
- Open to extension: goals as procedures, raw unification, walks, records as constructors, user type tags, projection, the combinators. Closed: the state's fields, the unification `cond`, the binding site, the post-`=°` chain, reification, the runner.
- Without editing micro one can: emit `=°` from external solvers (never return raw-unified states), carry branch-local data in S-registers (a working AUGMENT prototype), and check eigenvariable scope by post-filtering the answers of a `∀` body (a working GENERIC prototype). Their limits — scans, late pruning, constraints outliving scopes — are the requirements of Chapter 11.

11 Bringing λProlog to `(aux kanren micro)`: the design

What and why

Chapter 10 ended with a precise list of what an importing module cannot obtain from (aux kanren micro) today: a named place in the state for its own data, a call after every binding (to check the scope of eigenvariables eagerly and to wake postponed equations), and a say in reification (to print new constraints and to hide what must not escape). The three tricks of Section 10.9.2 show that AUGMENT and GENERIC *fit* the shape of a μKanren goal; they do not make them sound under every goal of the engine, nor cheap.

This chapter turns that observation into a design. It proceeds in four steps.

1. **Requirements** (Section 11.1). The nine acceptance items A1–A9 against which the proposal was built and tested, stated as properties of a run of `°->list`.

2. **The design space** (Section 11.2). The four meanings of unification on λ-terms compared in Section 9.8, and why higher-order pattern unification with universe levels is the one retained.

3. **Three prototypes** (Section 11.3). Three complete implementations of the same requirements, which differ in where the code sits with respect to (aux kanren micro): a layer that does not touch the engine, a small generic protocol in the engine, and a native engine. They were compared by three reviews (engineering, fidelity to the book, pedagogy); the protocol won.

4. **The extension protocol** (Section 11.4). The edit to (aux kanren micro) in full: the seventh state field, the registry, every hook with its contract and call site, the two new primitives, the one generic change to `absent°`, and the two invariants a client must keep.

The chapter closes with what the edit costs a first-order program (Section 11.4.8) and with the history of the review rounds that hardened the proposal (Section 11.5). The λ layer that sits on the protocol, `(aux kanren lambda)`, with its unifier, its `∀°` and `⊃°` and its treatment of micro's stores, is the subject of Chapter 12; the book's examples run on it are in Chapter 13; the plan for merging is in Chapter 14. In the repository the proposal lives under `doc/hol-kanren/prototype/` (the module, `micro.diff`, `show.diff`, the design notes `DESIGN.md`, the tests, the examples, the benchmarks and the scripts that build and run them, Section 14.1.4). After merging, the module and its suites become `src/aux.kanren.lambda.scm` and `src/test/microkanren-lambda*.scm`.

Every Scheme snippet of this chapter with an answer was run against the proposal (the edited micro and `(aux kanren lambda)`) unless it says otherwise, and its output is pasted verbatim. Snippets of the form `(show "label" query)` are quoted from the proposal's `examples.scm`, where `show` prints the label as a comment and then the answers; their outputs are quoted from `examples.out`.

11.1 Requirements

The proposal was written against nine acceptance items, A1 to A9. Each one is a property of the answers that `°->list` returns for a class of goals, illustrated by the book's own examples with the book's own answers. This section states each item in a form precise enough to be tested, then gives its intuition and one worked example. The general statements are *tested*, by 83 hand-written cases with 320 assertions, not proved.

11.1.1 Setting and notation

Goals and terms. The goal language is that of (aux kanren micro) (`=°`, `≠°`, `absent°`, `symbol°`, `number°`, `fresh°`, `and°`, `or°`, `cond°`, relations) extended with three forms: `(∀° (x) G)`, the universal goal `pi x\ G`; `(⊃° (D ...) G)`, the implication `D => G`; and relations declared with `define-dynamic-relation`, the only ones that take hypotheses. The term language is micro's (Scheme data and logic variables) extended with abstractions `(ƛ (x) t)`, applications `(@ t u ...)` and eigenvariables. We write Ans(G) for the list returned by `(°->list #f G)`, and say that G *fails* when Ans(G) is empty.

Reference semantics. The specification is the goal-directed search of the book for hereditary Harrop formulas, that is, the six rules AND, OR, INSTAN, GENERIC, AUGMENT and BACKCHAIN (Fig. A.1; [MN p. 77], [MN p. 82], [MN p. 124]; see Section 3.2 and Appendix A). Uniform proofs are complete for intuitionistic provability of these formulas (Theorem 3.11), so implementing these rules faithfully *is* the semantics. Where λProlog says nothing (disequality, *absento*, types), the reading adopted extends micro's own semantics over λProlog's term model (item A8).

Definition 11.1 (Universe levels)**.** A state carries a *depth* $d \in \mathbb{N}$, the number of `∀°` goals whose body is running. Every logic variable X has a *level* $\ell(X)$, the depth at its creation unless later lowered. An eigenvariable c created by a `∀°` at depth d has $\ell(c) = d + 1$. The names that the unifier itself opens when it goes under a binder have level ∞.

These levels are Nadathur, Jayaraman and Kwon's tags minus one: their query starts at tag 1 (Fig. A.7), ours at depth 0.

The level is the linear trace of Miller's mixed prefix: a variable of level d stands for an existential quantifier that follows the first d universal quantifiers of its branch. The intended reading is Nadathur, Jayaraman and Kwon's [43] (Definition 9.15): a term may be substituted for X only if every constant in it has a level $\leq \ell(X)$.

Example 11.2. In `(fresh° (q) (∀° (x) (fresh° (y) G)))`, `q` has level 0, the eigenvariable `x` level 1, and `y` level 1. So `y` may be bound to a term containing `x`, while `q` may not. This is $\exists q\, \forall x\, \exists y.\, G$ read off the creation depths.

Definition 11.3 (Order independence)**.** A conjunction is *order independent* when permuting its conjuncts changes at most the order of its answers, not the set of answers (taken as data, with their residual constraints).

Order independence is not an item of its own. It runs through all of them, because it is what Nadathur, Jayaraman and Kwon prove for their tagged interpreter: the choice of the goal to reduce next is irrelevant (their Prop. 4.1, Section 9.7). A miniKanren conjunction is fair to no conjunct, so any order-dependence is visible to the user. Most defects found by the reviews of Section 11.5 were violations of it, "in one goal order of three".

11.1.2 The nine items

Definition 11.4 (A1: the scope of ∀)**.**

(a) **GENERIC.** In a state of depth d, `(∀° (x) G)` runs $G[c/x]$ at depth $d+1$, with c a new eigenvariable of level $d+1$, and gives every answer state of G back the depth d.

(b) **Scope invariant.** In every state reached by the search, every binding $X := t$ of the substitution satisfies the three binding conditions of Definition 9.15 (stated also in Section 3.4.3 and Definition A.27): (1) X does not occur in t; (2) every eigenvariable occurring in t has a level $\leq \ell(X)$; (3) every unbound variable Y of t has $\ell(Y) \leq \ell(X)$ (*lowering*; for a flexible subterm $Y\,\bar{a}$, the arguments not visible at $\ell(X)$ are pruned).

(c) **Universality of the invariant.** (b) holds for every binding, whatever goal made it: the goals of the new module, micro's own `=°`, every relation built on it (`cons°`, the user's relations), and the unifier's internal bindings.

Intuitively, (a) is the book's GENERIC rule [MN p. 82] (Definition 3.8) and (b) is the reason why it is sound: no eigenvariable can reach a variable created before its `∀°`, directly or through an alias, so a proof never assumes that a fresh name equals an older term. Restoring the depth on each answer matters: a variable created *after* the `∀°` has exited must not see c, which a creation timestamp alone would not ensure.

The invariant (b) is stated on single bindings, but it controls whole values. Suppose $X := t$ and Y occurs unbound in t; by (3), $\ell(Y) \leq \ell(X)$, so if Y is bound later to some t', condition (2) for $Y := t'$ admits only eigenvariables of level $\leq \ell(Y) \leq \ell(X)$. By induction on the chain of bindings, the fully walked value of X never holds an eigenvariable of level $> \ell(X)$: checking each binding once, when it is made, is enough. Condition (3) is what makes the induction go through; without lowering, an alias created before the eigenvariable could receive it later.

Item (c) is what separates a sound design from a layered one, as Section 11.3.1 shows. The book's two classical queries [MN p. 85] and two escapes through aliases:

```
(show "∃Y∀x. Y = x  and  ∀x∃Y. Y = x"
      (list (°->list #f (fresh° (q) (∀° (x) (=° q x))))
            (°->list #f (fresh° (q) (∀° (x) (fresh° (y) (=° y x))) (=° q 'ok)))))

(show "escape through an alias (q = (y), y = x), and through micro's cons°: both fail"
      (list (°->list #f (fresh° (q) (∀° (x) (fresh° (y) (=° q (list y)) (=° y x)))))
            (°->list #f (fresh° (q) (∀° (x) (cons° x '() q))))))
```

```
;; ∃Y∀x. Y = x  and  ∀x∃Y. Y = x
(() ((λ () 'ok)))

;; escape through an alias (q = (y), y = x), and through micro's cons°: both fail
(() ())
```

In the third query the binding `y := x` is legal (`y` has level 1), but `q := (y)` was made earlier with `y` unbound; condition (3) lowered `y` to level 0 at that moment (by binding it to a fresh variable of level 0), so the later `y := x` violates (2). The fourth query never mentions the new module's unifier: `cons°` is micro's, and its `=°` is checked all the same. The suite adds Nadathur, Jayaraman and Kwon's own example (Section 3.4.3, and the tutorial after Definition 9.15), where lowering reaches a variable created by backchaining on an assumed clause: $\exists x \forall y (q\, y \supset p\, x)$ with the clause $\forall z (q\, z \supset p\, (d\, z))$ fails, as it must.

Definition 11.5 (A2: AUGMENT and BACKCHAIN)**.**

(a) **AUGMENT.** $\Sigma; \mathscr{P} \longrightarrow D \supset G$ reduces to $\Sigma; \mathscr{P} \cup \{D\} \longrightarrow G$: `(⊃° (D ...) G)` runs G with the clauses D added to a context Γ kept in the state, and gives every answer state of G back the old Γ. Hypotheses are visible exactly during G, and the branches of an `or°` do not see each other's hypotheses.

(b) **Clause order.** Γ is a list, extended at the front. A dynamic relation p called on $\bar{t}$ is the disjunction of its clauses in Γ, most recent first, followed by its static body [MN p. 78].

(c) **Variables of a clause.** The logic variables free in D belong to the context: they are shared by every use of the clause and with the goal around it [MN p. 86–88]. The clause's own universally quantified variables, `(∀ (z) ...)`, are renamed at each use, at the current level (Nadathur, Jayaraman and Kwon's "new tagged instance").

(d) **Static relations are closed.** Assuming a clause for a relation not declared with `define-dynamic-relation` is an error, not a silent no-op.

Point (b) is a statement about *order*, which a miniKanren usually leaves to interleaving. The book fixes it with two queries against the program `p 1` [MN p. 78]:

```
?- p 2 => p 3 => p X.
X = 3;
X = 2;
X = 1
?- (p 2 & p 3) => p X.
X = 2;
X = 3;
X = 1
```

```
(define-dynamic-relation (p° x) (=° x 1))
(show "clause order under augmentation (p.94)"
      (list (°->list/datum (fresh° (q) (⊃° ((p° 2)) (⊃° ((p° 3)) (p° q)))))
            (°->list/datum (fresh° (q) (⊃° ((p° 2) (p° 3)) (p° q))))))
```

```
;; clause order under augmentation (p.94)
((3 2 1) (2 3 1))
```

(The label's "p.94" is the PDF page, [MN p. 78]; see Section 1.5.) Each clause answers without suspending, so the flat disjunction returns its disjuncts in order, and the order is the book's. Point (c) is the subtle one: in `pi z\ (q z => p (d z))` the variable `z` is the clause's own, while a free `Y` in an assumed `p Y` is the goal's. Section 3.8.1 explains why Γ must live in the state rather than in a Scheme parameter.

Definition 11.6 (A3: ∀ and ⊃ together)**.** The book's type inference for untyped λ-terms (Section 6.5.4), whose abstraction case is `typeof (abs M) (arr A B) :- pi x\ typeof x A => typeof (M x) B`, computes principal types: for a closed term it answers the book's type up to the names of type variables, and it fails on terms that have no simple type.

This item exercises all of A1, A2 and A4 in one clause: `∀°` makes the bound variable a scoped constant, `⊃°` assumes its type, `(@ R x)` β-reduces the object-level body, and the type variables `A`, `B`, created outside the `∀°`, must never be bound to a type containing `x`. In the module's syntax:

```
(define-dynamic-relation (typeof° m t)
  (cond°
    ((fresh° (M N B) (=° m `(app ,M ,N)) (typeof° M `(arr ,B ,t)) (typeof° N B)))
    ((fresh° (R A B) (=° m `(abs ,R)) (=° t `(arr ,A ,B))
       (∀° (x) (⊃° ((typeof° x A)) (typeof° (@ R x) B)))))))
```

Run on the combinators S and I and on $\lambda x.\, x\, x$, written in the object syntax with `ƛ` for the meta-level binder:

```
(show "typeof S, I and λx.x x"
      (list (°->list/datum (fresh° (q) (typeof° `(abs ,(ƛ (x) `(abs ,(ƛ (y) `(abs ,(ƛ
       ↪  (z)
                                          `(app (app ,x ,z) (app ,y ,z)))))))) q)))
            (°->list/datum (fresh° (q) (typeof° `(abs ,(ƛ (x) x)) q)))
            (°->list/datum (fresh° (q) (typeof° `(abs ,(ƛ (x) `(app ,x ,x))) q)))))
```

```
;; typeof S, I and λx.x x
(((arr (arr α (arr β γ)) (arr (arr α β) (arr α γ)))) ((arr α α)) ())
```

The third query fails by the occurs check: the type of `x` would have to be an arrow type containing itself.

Definition 11.7 (A4: λ-tree syntax and β)**.**

(a) **α.** `(ƛ (x) t)` builds an abstraction whose bound variable is a Scheme variable; terms built alike are `equal?`, and unification is modulo α.

(b) **β.** `(@ (ƛ (x) t) s)` is $t[s/x]$ at construction; an application whose head becomes an abstraction through a later binding is reduced on the fly, in unification and in every deep walk.

(c) **η.** Terms are kept η-short; η-expansion happens on demand when one side of an equation is an abstraction.

(d) **Object-level substitution is meta-level β** (Section 6.3): the book's evaluators, `copy` and `subst` give the book's results.

```
(show "α-equivalence: λa.a = λb.b, and the β-redex (λx.g x x) a"
      (list (°->list #f (fresh° (q) (=° (ƛ (a) a) (ƛ (b) b))))
            (°->list/datum (fresh° (q f) (=° f (ƛ (x) `(g ,x ,x))) (=° q (@ f 'a))))))
```

```
;; α-equivalence: λa.a = λb.b, and the β-redex (λx.g x x) a
(((λ (α) α)) ((g a a)))
```

The first unification succeeds and binds nothing, so `q` stays fresh. In the second, `f` is bound first and the application `(@ f 'a)` is β-reduced when the deep walk of the answer meets it. With call-by-name evaluation, the book's $(\lambda x \lambda w.w)\,\Omega$ [MN p. 183] evaluates to `abs w\w` without ever evaluating Ω, because the substitution of the argument is a meta-level β that drops it (Chapter 13).

Definition 11.8 (A5: L_λ unification and delay)**.** Call *pattern* for a variable F an argument list $\bar{a}$ of $F\,\bar{a}$ whose members head-normalise to distinct eigenvariables of level $> \ell(F)$ (Definition 7.15, with levels in place of the prefix).

(a) **Patterns.** For an equation all of whose flexible subterms are patterns, unification terminates; it fails if and only if the equation has no unifier, and otherwise it binds a most general unifier (Theorem 7.22). On the problems 8.G–8.K it gives the book's answers and mgus [MN p. 222–225]. (The book does not number these problems; the labels 8.A–8.M are those of Section 7.7, in their order of appearance, as the test suite uses them; 8.M is the problem of [MN p. 208].)

(b) **Dynamic** L_λ [MN p. 225–226]. An equation outside L_λ is neither failed nor enumerated: it is suspended, retried after every later binding, solved as soon as it becomes a pattern, and otherwise reported in the answer as a residual `(assert (≐ u v))`, closed over the eigenvariables whose scope it outlived.

The level condition is Miller's "universally quantified inside the scope of F". To see it at work, compare $\exists F\,\forall x.\,F\,x = x$ with $\forall x\,\exists F.\,F\,x = x$. In the first, $\ell(F) = 0 < 1 = \ell(x)$, so `(@ F x)` is a pattern, and the only unifier is $F := \lambda x.\,x$. In the second, F is created under the `∀°`, so $\ell(F) = 1 = \ell(x)$ and `(@ F x)` is not a pattern: F may mention x as a constant, and both $\lambda z.\,z$ and $\lambda z.\,x$ solve the equation, so no unifier is most general. The proposal solves the first; the second it suspends, accepting either solution when a later goal supplies it:

```
(pretty-print
 (list (°->list/datum (fresh° (q) (∀° (x) (=° (@ q x) x))))
       (°->list/datum (fresh° (q) (∀° (x) (fresh° (F) (=° (@ F x) x)
                                                  (=° F (ƛ (z) z)) (=° q 'id)))))
       (°->list/datum (fresh° (q) (∀° (x) (fresh° (F) (=° (@ F x) x)
                                                  (=° F (ƛ (z) x)) (=° q 'const)))))))
(pretty-print
 (°->pretty (fresh° (q) (∀° (x) (fresh° (F) (=° (@ F x) x) (=° q 'ok))))))
```

```
(((ƛ (x) x)) (id) (const))
((ok : (= (ƛ (x) (@ α x x)) (ƛ (x) x))))
```

The last residual is instructive. When the `∀°` exits, F (of level 1) is *raised*, $F := \alpha\, x$ with $\ell(\alpha) = 0$, and the equation is closed over x: $\forall x\, \exists F.\, F\, x = x$ has become $\exists \alpha\, \forall x.\, \alpha\, x\, x = x$, which is problem 8.M below.

Part (a) is Miller's theorem [34], whose algorithm (Definition 7.21) prunes and raises; part (b) is what Teyjus 2 does with the equations it cannot solve, after Dowek, Hardin, Kirchner and Pfenning [8] (Section 7.9.1). The book's main example, problem 8.H [MN p. 223–224] (Example 7.23), $\forall w\, \forall x\, \forall y.\ f\,(U\, x\, y) = f\,(g\,(V\, y\, w))$ with U, V quantified outside, is the λProlog query `pi w\ pi x\ pi y\ f (U x y) = f (g (V y w))`; it has the mgu $U = \lambda x.\, \lambda y.\, g\,(V'\, y)$, $V = \lambda y.\, \lambda w.\, V'\, y$ (Example 7.23):

```
(show "Example 8.H, pruning: (U V)" (°->list/datum (fresh° (q u v) (=° q (list u v))
 ↪  (∀° (w x y) (=° `(f ,(@ u x y)) `(f (g ,(@ v y w))))))))
```

```
;; Example 8.H, pruning: (U V)
(((ƛ (x y) (g (@ α y))) (ƛ (y w) (@ α y))))
```

The fresh `α` is the book's V'. A non-pattern such as problem 8.M, `pi x\ F x x = x`, has two incomparable unifiers, so it waits; it is reported, and a later binding wakes it. Problem 8.A, $F\, a = g\, a\, a$ with a a constant, is not a pattern either (its argument is no eigenvariable):

```
(show "non-patterns: 8.M and 8.A, suspended; 8.M woken by F := λa.λb.b"
      (list (°->pretty (fresh° (q) (∀° (x) (=° (@ q x x) x))))
            (°->pretty (fresh° (q) (=° (@ q 'a) '(g a a))))
            (°->list/datum (fresh° (q) (∀° (x) (=° (@ q x x) x))
                                       (=° q (ƛ (a b) b))))))
```

```
;; non-patterns: 8.M and 8.A, suspended; 8.M woken by F := λa.λb.b
(((α : (= (ƛ (x) (@ α x x)) (ƛ (x) x))))
 ((α : (= (@ α a) (g a a))))
 ((ƛ (a b) b)))
```

(The first two answers, 8.M and 8.A, are printed by `°->pretty` of the show module, the third by `°->list/datum`; the residual of 8.M is closed over `x`, which has left its scope.)

Definition 11.9 (A6: λProlog's semantics where Ma and Friedman differ)**.** On the equations by which Ma and Friedman [29] motivate static closures (Section 9.2), the answer is λProlog's: (1) $\lambda a.\, X = \lambda b.\, b$ and their conjunction containing it fail, since $\exists X \forall a.\, X = a$ is false (Example 7.25); the paper answers $X =$ `d0`; (2) $\lambda a.\, \lambda a.\, X = \lambda c.\, \lambda d.\, c$ fails, with no false "true = false", because shadowing is lexical; (3) their Example 3, $\lambda a.\, \lambda b.\, X = \lambda c.\, \lambda d.\, Y$, gives $X = Y$; (4) an assumption is tried before the program: with a dynamic `eq°`, their `assume` query gives `(orange apple)` where the paper prints `(apple orange)`; (5) their E12, $\forall A.\, A = F\, A$, gives $F = \lambda A.\, A$, as in the paper.

```
(show "λa.X = λb.b fails (the paper: X = d0); λa.λa.X = λc.λd.c fails; ∀A. A = F A
 ↪  gives F = λA.A"
      (list (°->list #f (fresh° (q) (=° (ƛ (a) q) (ƛ (b) b))))
```

```
            (°->list #f (fresh° (q) (=° (ƛ (a) (ƛ (a) q))
                                        (ƛ (c) (ƛ (d) c)))))
            (°->list/datum (fresh° (q) (∀° (a) (=° a (@ q a)))))))
```

```
;; λa.X = λb.b fails (the paper: X = d0); λa.λa.X = λc.λd.c fails; ∀A. A = F A gives F = λA.A
(() () ((ƛ (a) a)))
```

Section 9.3.1 discusses these equations; the point here is only that the proposal answers them the way λProlog's quantifier reading dictates.

Definition 11.10 (A7: the book's programs)**.** The book's programs over λ-tree syntax give the book's answers, in the book's order where the interleaving search allows it: de Bruijn translation in both directions [MN p. 193]; prenex normal forms, the unique one and the five of a non-unique case [MN p. 196–197]; `mappred` (Figs. 5.3–5.4); palindromes over functional difference lists, the book's first nine answers [MN p. 142]; Fischer's CPS transformation, exactly the book's term [MN p. 257].

These programs are the integration tests: each needs several of A1–A5 at once, and each has an answer printed in the book to compare with. Chapter 13 walks through them.

Definition 11.11 (A8: micro's stores on λ-terms)**.**

(a) `≠°` decides modulo αβη and respects scope; its outcome does not depend on whether a binding comes before or after it. Under `∀°` it reads an eigenvariable as GENERIC's new name by default (`pi x\ not (q = x)` succeeds), and as any term of its universe when the parameter `μkanren-λ-reading` is `'extensional` (the reading of Moiseenko and of Jin et al. [41, 26]).

(b) `absent°` looks for its datum in the parts of a λ-term, never in binder names, de Bruijn indices or the internals of an eigenvariable; on a flexible term it waits.

(c) `symbol°`, `number°` and the user's types wait on a flexible term, and decide a rigid λ-term or an eigenvariable by the type's predicate.

(d) No eigenvariable is ever printed free in an answer; residual constraints print as `≐` and `(∀ (x) ...)`; binders print renamed apart; ground answers evaluate back to live λ-terms.

λProlog has no disequality, *absento* or types, so this item is not fidelity but coherence: micro's stores must keep meaning what they mean once terms have binders. For instance:

```
(show "≠ modulo αβη, flex-flex with an existential, a flexible absento, a type on a
↪  flexible term"
      (list (°->pretty (fresh° (q) (≠° q (ƛ (x) x))))
            (°->pretty (fresh° (q f g) (=° q (list f g))
                               (≠° (ƛ (x y) (@ f x)) (ƛ (x y) (@ g y)))))
            (°->pretty (fresh° (q F) (=° q F) (absent° 'c (@ F 'c))))
            (°->list/datum (fresh° (q F) (=° q F) (absent° 'c (@ F 'c))
                                  (=° F (ƛ (z) 'b))))
            (°->list/datum (fresh° (q F) (=° q (@ F 'a)) (symbol° q)
                                  (=° F (ƛ (z) z))))))
```

```
;; ≠ modulo αβη, flex-flex with an existential, a flexible absento, a type on a flexible term
(((α : (≠ α (ϰ (x) x))))
 (((α β) : (∀ (_3) (≠ (α β) ((ϰ (x) _3) (ϰ (y) _3))))))
 ((α : (absento c (@ α c))))
 ((ϰ (z) b))
 (a))
```

Line by line. (1) $q \neq \lambda x.\, x$ is kept as a constraint, as micro keeps $q \neq 5$. (2) The trial unification of $\lambda x.\, \lambda y.\, f\ x$ with $\lambda x.\, \lambda y.\, g\ y$ prunes both f and g to a common constant function: the line says $\neg\exists H.\, f = \lambda x.\, H \wedge g = \lambda y.\, H$, printed in the equivalent form $\forall H.\, (f\ \ g) \neq (\lambda x.\, H\ \ \lambda y.\, H)$. The variable H (`_3`), which the trial created, is quantified *inside* the negation, which micro's prefixes cannot express. (3) Whether `c` is absent from F c depends on F, so the constraint waits on the whole flexible term. (4) Once $F := \lambda z.\, \mathrm{b}$, the argument is dropped and the constraint holds. (5) `(symbol° q)` waits on F a and holds once $F := \lambda z.\, z$ makes it `a`. Chapter 12 explains each of these answers in detail.

Definition 11.12 (A9: conservativity)**.**

(a) micro's six suites (`microkanren`, `-aggregation`, `-untagged`, `-show`, `-arith`, `-date`; 125 cases) pass with unchanged expectations, answer order included, against the edited micro, both with and without `(aux kanren lambda)` loaded.

(b) A first-order program pays at most a few per cent for the edit (measured in Section 11.4.8).

Table 11.1 summarises the nine items and the evidence for each.

Table 11.1: The acceptance items and the hand-written cases that pin them (83 cases, 320 assertions in `microkanren-lambda.scm`; 6 more cases in `microkanren-lambda-limits.scm` pin the known limits).

item	requirement	book	cases / assertions
A1	scope of ∀: GENERIC, NJK's binding conditions on every binding	[MN p. 82, 85]	9 / 22
A2	⊃: AUGMENT, clause order, shared and renamed variables, closed static relations	[MN p. 77–78, 86–88]	8 / 21
A3	∀ and ⊃ together: type inference	[MN p. 191]	3 / 8
A4	λ-tree syntax: α, β, η; object substitution by meta β	[MN p. 177–199]	4 / 9
A5	L_λ unification with mgus; dynamic L_λ (delay)	[MN p. 220–226]	10 / 40
A6	λProlog's answers where Ma–Friedman differ	—	5 / 13
A7	the book's programs, the book's answers	[MN p. 142, 193–197, 257]	5 / 12
A8	≠, absento, types and reification on λ-terms	—	11 / 65
A9	micro's suites unchanged; first-order cost	—	1 / 2, + 125 cases ×2
R, R2, R3	regressions of the three review rounds (Section 11.5)		27 / 128

11.2 The design space

Section 9.8 put four candidate meanings of `=°` on λ-terms side by side, without choosing (Table 9.2). In one paragraph each:

A. Pattern unification with de Bruijn terms and levels, after Miller, Nipkow, and Nadathur and Linnell [34, 48, 44]; what Teyjus 2 and ELPI [9] run. Decidable and unitary on L_λ; non-patterns are delayed.

B. Nominal unification (Urban–Pitts–Gabbay [58]; αKanren [5]). Decidable, unitary with a freshness context. A different logic: names as data, no function variables, no β; noms are distinct constants, not eigenvariables, so GENERIC still needs a separate mechanism; αKanren has no disequality.

C. Static closures (Ma–Friedman [29]). Grafting semantics: $\lambda a.\, X = \lambda b.\, b$ succeeds, unlike λProlog; a workshop preview with no published algorithm.

D. Huet's pre-unification, enumerated by the stream (Huet [25]; Teyjus 1). Exact for full higher-order unification, but semi-decidable, with possibly infinitely many pre-unifiers.

The proposal retains A, for three reasons.

1. **λProlog's semantics.** The requirements are the book's. Option A is what the book's own implementations run, and with delayed non-patterns its behaviour on the book's examples is the book's (A5). Option C gives different answers on A6's equations by design; option B answers a different question.

2. **Decidability and most general unifiers.** Miller's theorem (Theorem 7.22) makes unification on patterns terminating and unitary. This matches the shape of micro's `=°`, which maps a state to *at most one* state (`✓°` or `✗°`). Option D needs branching inside unification, so it cannot be an `=°`; it can be added later as an explicit search over the suspended equations, which are its natural agenda (Chapter 14).

3. **Universe levels make GENERIC cheap and exact.** The tagged interpreter of Nadathur, Jayaraman and Kwon is sound and complete, and needs one integer per variable and one comparison per binding (Definition 9.15). The alternatives for ∀ alone are weaker or costlier: the explicit prefix of λKanren 2020 [30] is walked at each check and only grows (Section 9.4); the eigen-absento of Hemann, Byrd and Friedman [21] costs $O(|B|)$ per check and is first-order only (Section 9.6); the end-of-scope check of Section 10.9.3 prunes late and lets constraints outlive their scope.

Three secondary choices follow from A and are shared by every prototype below. Terms are *locally nameless* [31, 6] (de Bruijn indices inside bodies, eigenvariables for free names), so α costs nothing and terms built alike are `equal?`. Raising is done lazily through levels rather than eagerly as in the book's §4.4.1 [MN p. 107]; the two differ in the shape of the mgu, not in its meaning (Example 7.31). And non-patterns are delayed rather than rejected: rejecting them,

as ELPI does by default, would lose `subst R N (R N)` with `R` unknown, and the palindrome program.

What A leaves open is *where* the code lives with respect to (aux kanren micro), and that question was answered by building it three times.

11.3 Three prototypes

Three complete prototypes were built against A1–A9. They share the representation (locally nameless abstractions, applications, eigenvariables with levels) and the algorithmic core (variable elimination with pruning and lowering, a store of suspended equations). They differ in architecture. Table 11.2 compares them at a glance; the subsections describe each one, its strengths, and the concrete defects the three reviews found.

Table 11.2: The three prototypes.

	d1, layered	**d2, hooks**	**d3, native**
edit to micro	none (byte-identical binary)	+105/−25 lines: a field and seven hooks	micro grows from 713 to 1217 lines; a 931-line diff
λ state	a record bound to the reserved variable 0	a seventh field `X`	`X` holds U, E, P; levels on the variable record
unifier	a second one, `=λ°`, beside micro's	micro's, plus an `unify` hook	micro's, rewritten
runner	its own, `λrun`	micro's `°->list`	micro's `°->list`
suite	29 cases, 137 assertions, hand-written	42 cases, 102 assertions, hand-written	14 cases, 116 assertions, a generated snapshot
rank	last for all three reviews	first for two reviews	first for the pedagogy review

11.3.1 d1: a layer with no edit to micro

Architecture. d1 takes Section 10.9.2 at its word: it imports micro unchanged and adds a pure library. Its whole context (current level, level boundaries and overrides, the clause database, delayed equations, pending constraints) is a record bound to the reserved variable 0, the "S-register" of Trick 2, overwritten by a persistent `update/sbral` in $O(\log |S|)$, so each branch of an `or°` keeps its own context; the query variable is variable 1. Since micro's binding site is closed, d1 brings its own unifier `=λ°`, its own constraints `≠λ°` and `absentλ°`, and its own runner `λrun`. Levels need no slot in the variables: micro allocates indices monotonically, so a variable's level at creation is a step function of its index, recorded as boundaries pushed by `∀°`; lowering is recorded as overrides.

Strengths. Micro's behaviour and speed are untouched by construction. d1's `absent°` was the most precise on flexible terms, it offered the extensional reading of ≠ under ∀, it had an αKanren-style printer, a data runner comparable with `equal?`, and a table of every place where purity leaks, each pinned by a test. All these were grafted into the final proposal.

Defects. The leaks are the architecture's, not the code's. Micro's `=°`, and every relation built on it, binds without d1 seeing the binding, so A1(c) fails:

```
; d1-layered, its own runner
(pretty-print (λrun (q -1) (∀° (x) (cons° x '() q))))
(pretty-print (λrun (q -1) (∀° (x) (=λ° q x))))
```

```
((x₂))
()
```

With d1's own unifier the escape is caught; with micro's `cons°` the eigenvariable `x₂` leaks into the answer. The proposal answers `()` to the first query (A1, above). The reviews found four more defects: the module's goals do not run under `°->list` (they need the register); conjunction is not commutative, because d1 read a list headed by a symbol as a rigid application in spine notation, conflating lists with λ-tree applications, and together with η this made the answers of some conjunctions depend on the order of the conjuncts; d1 duplicates a large part of micro; and `=λ°` costs 4.4 times micro's `=°` on search. Its clause database was a linear list, on which the reviews measured slowdowns of 18 times. d1 was ranked last by all three reviews.

11.3.2 d2: an extension protocol in micro

Architecture. d2 makes the smallest principled edit that opens the closed points of Table 10.4, and builds the whole λ layer as a client module on top of it. The state gets a seventh field `X`, which micro copies and never reads. A record of procedures, registered globally, has hooks for `term?`, `unify`, `post`, `diseq`, `value`, `repr` and `forms`; micro calls them at the matching points. The edit was +105/−25 lines (counted on its `micro.diff`), the module 708 lines.

Strengths. The reviews found no soundness defect in d2. It kept terms η-short as an invariant, so both $\lambda x.\, q\, x = q$ and $F\, x = G\, x \Rightarrow F = G$ hold; it had a "soft" occurs check that suspends $F\, x = G\, (F\, x)$ instead of failing it (it holds with $G := \lambda z.\, z$); its suite was hand-written with book page references; its printer was the only one that renamed binders correctly; and the edit cost at most 2.8% on micro's benchmarks.

Defects. The reviews found seven, listed in Table 11.3. None is a soundness defect; most are imprecisions of the stores on λ-terms and one is a cost.

One more defect turned up while grafting: $F = \lambda x.\, G\, (F\, x)$, reached through η-short construction, failed micro's occurs check before any hook could see it. That is why the final protocol has an `occurs` hook.

11.3.3 d3: a native engine

Architecture. d3 integrates λ-terms into micro itself, in the style of ELPI and Teyjus. The variable record gets a level slot; `µkanren-unify` knows abstractions, eigenvariables and flexible applications; the binding site does the occurs check, the scope check and lowering in one walk; `∀°` and `⊃°` are core goals; the current universe, the delayed equations and the program are packed in one extra state field. Micro grew from 713 to 1217 lines.

Table 11.3: The defects of d2 found by the reviews, and their fixes (design notes, §10.2).

	defect	fix in the proposal
D1	a suspended disequality printed as `(deny (and))`	the denied equation, `(deny (≐ u v))`
D2	`absent°` over-strict on flexible terms	the `parts` hook: a flexible term is watched whole
D3	trivially true residuals kept ($F\,x\,x = F\,x\,x$)	delay drops α-equal sides
D4	Miller's n7 left suspended and printed with `_2`	discharge at `∀` exit; the `terms` hook names residual variables
D5	registering the extension slowed $\neq^{\circ}$-heavy first-order code by 45–50%	migration only when a new binding holds a λ-term
D6	goal suffix on term constructors, levels hidden in property lists, λ-terms not `equal?`	`ƛ`, `@`; plain records thanks to `parts`, `value`, `atom?`; interned hints
D7	`symbol°`/`number°` failed eagerly on flexible terms; `⊃°` on a static relation silently ignored	the `tag` hook; an error for non-dynamic relations

Strengths. The pedagogy review ranked d3 first for three reasons: one vocabulary, readable answers in `ƛ`, `@` and `≐`, and answers usable as code. d3 also discharged residual constraints at `∀` exit by raising and closing them, and it was the fastest: type inference of the Church numeral 400 took 105 ms, against d2's 441 ms.

Defects. The reviews found a crash in `µkanren-verify-D+` on a disequality prefix whose trial unification delays, and an η-incompleteness that made `≠°` unsound: after $\lambda x.\,F\,x \neq \lambda x.\,G\,x$, the binding $G := F$ was accepted.

```
(pretty-print (°->list #f (fresh° (q F G) (=° q (list F G)) (≠° (ƛ (x) (@ F x)) (ƛ
↪  (x) (@ G x))) (=° G F))))
```

Run against d3, then against the proposal:

```
((λ (α) (cons α (cons α '()))))
```

```
()
```

The answer of d3 is a pair (F, F), which violates the disequality it was asked to keep. Besides, its suite was a snapshot generated from the actual answers rather than written from the book, and its diff carried trailing-whitespace churn.

11.3.4 The verdict

The three reviews gave aggregate scores of **23** to d2, **21** to d3 and **15.5** to d1. Two reviews ranked d2 first, the pedagogy review ranked d3 first, and all three ranked d1 last. The choice of d2 as the base rests on four facts.

- **Size and scope of the edit.** The final protocol is +235/−41 lines of micro (713 to 907 lines; 214 of the added lines are not blank), comments included, all generic: nothing in micro mentions λ, levels or clauses. d3 special-cases every walker. The same protocol could carry another store, such as nominal terms or finite domains.

- **Soundness** is equal. d3's eager binding rule and the λ layer's eager `post` check enforce the same binding conditions; every scope attack of the reviews fails correctly on both, and on the final version.
- **Correctness** favours d2: d3 had a crash and an unsound `≠°`, d2 neither.
- **Pedagogy.** d3's three advantages were grafted onto d2 (one vocabulary of `ƛ`, `@`, `≐`; readable residuals; ground answers as live terms through the `environment` hook), so the pedagogical argument for the native design no longer holds.

The reviews also recommended about twenty grafts from d1 and d3. Each was either done or explicitly declined; Table 11.4 lists the main ones.

Table 11.4: Grafts recommended by the reviews (a selection from the design notes, §10.3).

from	graft	status
d3	discharge residual constraints at ∀ exit by raising and closing	done
d3	`≐` exported, residuals as `(assert (≐ …))`, merged binders `(ƛ (x y) …)`	done
d3	a first-order fast path for the scope check	done (`µkanren-plain?`, `top`, the `atom?` hook)
d3	ground answers as live terms	done (`environment` hook)
d3	the 2–4× faster λ machinery	partly: Church 400 from 441 to 254 ms; d3: 105 ms
d3	var–var bindings oriented by level	declined: a hook on micro's second-hottest path
d3	levels on the variable record	declined: `freshᵃ` would need the universe, so micro would become λ-aware
d1	precise `absent°` on flexible terms	done
d1	the extensional reading of ≠ under ∀, as an option	done (`µkanren-λ-reading`)
d1	a data runner, an αKanren-style view	done (`°->list/datum`, `°->pretty`)
d1	a limits table, each row pinned by a case	done
d1	truly nameless λ-records, `equal?` modulo α	partly: interned hints; α-variants with different hints are not `equal?`

The two declined grafts share a reason. Both would put knowledge of levels into micro's core: the var–var clause of the unifier, its hottest path after pair–pair, and the variable constructor `freshᵃ`, which would need the current universe kept in the extension's data. The protocol's discipline is that micro stays λ-agnostic; for the orientation, the lowering done by the scope check yields the same mgu up to renaming.

11.4 The extension protocol

The protocol is the part of the proposal that edits (aux kanren micro). It is generic: it lets a client module own a class of records (its *terms*), keep per-branch data in the state, and be

consulted at each point where micro would otherwise treat those records as plain slot vectors. This section gives it in full. Excerpts are verbatim from `micro.diff`; lines starting with + are added, with - removed, and with a space unchanged context. Comment lines of the diff are left out of some excerpts, as is the removed body of `=°`.

11.4.1 The seventh field

The state gets a field `X`, the extensions' slot. Micro never reads it: it copies it at its two positional constructions, the binding site `µkanren-state-update` and `freshª`, and `µkanren-state-with` accepts `X:`. The positional constructor keeps its six-argument signature, with `X` optional, so the positional states of `test/microkanren.scm` still work.

```
-  (define-record µkanren-state vars-count S D A T tags)
+  (define-record-type µkanren-state
+    (make-µkanren-state/X vars-count S D A T tags X)
+    µkanren-state?
+    (vars-count µkanren-state-vars-count) (S µkanren-state-S) (D µkanren-state-D)
 ↪  (A µkanren-state-A)
+    (T µkanren-state-T) (tags µkanren-state-tags) (X µkanren-state-X))
+
+  (define (make-µkanren-state vars-count S D A T tags #!optional (X '()))
 ↪  (make-µkanren-state/X vars-count S D A T tags X))
```

and in `freshª`:

```
-             (s*  (make-µkanren-state (add1 vc) S D A T tags)))
+            (s*  (make-µkanren-state/X (add1 vc) S D A T tags (µkanren-state-X s))))
```

`X` is `'()` until an extension writes it, which the λ layer reads as "everything empty"; so `°->§` keeps its fixed empty initial state. The λ layer stores in `X` one record with the current depth, the highest depth reached, the variables' levels, the context Γ, the suspended constraints E and the λ-disequalities N (Chapter 12). Since S is persistent and `X` is copied forward by every goal, the data is branch-local for free: each branch of an `or°` sees its own Γ, as A2(a) requires.

11.4.2 Extensions and the registry

An extension is a record built by

```
  (define (make-µkanren-extension name #!key (term? #f) (atom? #f) (unify #f)
   ↪  (occurs #f) (post #f) (diseq #f) (tag #f) (parts #f)
                                          (value #f) (repr #f) (terms #f) (forms #f)
                                             ↪  (environment #f))
```

and `µkanren-extension-register!` adds it to a global list. Registering again under the same name replaces the old extension, so reloading a module is harmless. Dispatch is by term: the extension that owns a pair of terms is the first whose `term?` holds of either.

```
+  (define (µkanren-extension/term u v)
+    (let loop ((E µkanren-extensions))
```

```
+      (cond
+        ((null? E) #f)
+        ((let1 (term? (μkanren-extension-term? (car E))) (and term? (or (term? u)
 ↪  (term? v)))) (car E))
+        (else (loop (cdr E))))))
```

The call sites are guarded by `(pair? μkanren-extensions)` (those of `forms` and `environment`, in reification and in the runner, just map over the empty list), so with no extension registered a hook costs one `pair?` test, and none sits on micro's hottest path (the variable clauses and the pair–pair clause of the unifier). The registry is global rather than per state because `°->list` starts from `μkanren-state-empty`: a per-state hook list would have to be carried by every initial state.

The protocol's whole exported surface is small: `μkanren-state-X` (and the `X:` keyword of `μkanren-state-with`), `make-μkanren-extension`, `μkanren-extension?`, `μkanren-extension-name`, `μkanren-extension-register!`, and the two primitives `μkanren-post-unify` and `μkanren-state-forget` of Section 11.4.4. The hook accessors and the dispatchers (`μkanren-extension/term` and its siblings) stay internal to micro.

The λ layer registers itself once, at load time:

```
(define μkanren-λ-extension
  (make-μkanren-extension 'λProlog
    term?: μkanren-λ-term? atom?: (λ (u) (or (μkanren-eigen? u) (μkanren-bvar? u)))
     ↪  unify: μkanren-λ-unify occurs: μkanren-λ-occurs post: μkanren-λ-post
     ↪  diseq: μkanren-λ-diseq
    tag: μkanren-λ-tag parts: μkanren-λ-parts value: μkanren-λ-value repr:
     ↪  μkanren-λ-repr
    terms: μkanren-λ-terms forms: μkanren-λ-forms environment:
     ↪  μkanren-λ-environment))

(μkanren-extension-register! μkanren-λ-extension)
```

11.4.3 The hooks

Table 11.5 lists the thirteen hooks. Compared with d2's seven, the final protocol adds `atom?`, `occurs`, `tag`, `parts`, `terms` and `environment`, each answering a defect or a graft of the reviews (`atom?` the first-order fast path, `environment` live ground answers, the others the defects of Table 11.3 and the occurs failure found while grafting). The last column says which closed point of Table 10.4 each hook opens. Then a paragraph per hook gives its contract and its call site.

Table 11.5: The hooks of an extension. u, v: walked terms; s: a state; as: the bindings made so far, as `(α . t)` pairs; → gives the result, | separates alternatives.

hook	arguments → result; contract	called by	opens
`term?`	`(u)`: u is one of the extension's records	dispatch	—
`atom?`	`(u)`: u holds no variable	micro's occurs check, which then does not look inside u	the binding site
`unify`	`(u v s as)` → `(s* as*)` | `#f`: a pair with one extension term, neither a variable	`µkanren-state-unify`: the structure–structure clause and the final `else`	the unification `cond`
`occurs`	`(α v s as)` → `(s* as*)` | `#f`: micro's occurs check failed on $\alpha := v$	`µkanren-state-update`, failure path only	the binding site
`post`	`(s* as)` → `(s** as**)` | `#f`: runs after the bindings of `=°`, before D/A/T are verified; may bind more	`µkanren-post-unify`, called by `=°`	the post-`=°` chain
`diseq`	`(u v s)` → `#f` | state | `'fail`: may claim a disequality	`≠°`	`≠°` (store D)
`tag`	`(u tag s)` → state | `#f`: a type on its term	`symbol°`, `number°`, user types	the A store
`parts`	`(u s [projecting])` → `'stuck` | list: where `absent°` looks into its term; when projecting, the parts whose constraints an answer keeps	`absent°`, the re-verification of T, `µkanren-project-T`	the T store; reification
`value`, `repr`	`(u walk s)`: the deep value; the expression that rebuilds it	`µkanren-state-find/value`, `/repr`	reification
`terms`	`(s)` → terms: the terms of its residual constraints	`µkanren-project`	reification
`forms`	`(s s*)` → forms: the forms its constraints add to an answer	`µkanren-project`	reification
`environment`	`()`: where `°->list/ground` evaluates answers	`°->list`	the runner

`term?` identifies the extension's records. Every dispatch by term goes through it (`unify`, `tag`, `parts`, `value`, `repr`); the other hooks are consulted on every registered extension, so an extension without `term?` could still use them. For the λ layer, the four records are abstractions, applications, eigenvariables and de Bruijn indices.

unify is consulted when micro's unifier meets a pair of walked terms, one of which is the extension's and neither of which is a variable. Two clauses of the unification `cond` hand such pairs over: the structure–structure clause (two records, or a record and a vector) and the final `else` (an extension term against a pair or an atom).

```
         ((and (µkanren-structure? u*) (µkanren-structure? v*))
-          (µkanren-state-unify (µkanren-structure->list u*)
 ↪  (µkanren-structure->list v*) s assocs))
+          (cond
+            ((and (pair? µkanren-extensions) (µkanren-extension/term u* v*)) => (λ
 ↪  (e) ((µkanren-extension-unify e) u* v* s assocs)))
+            (else (µkanren-state-unify (µkanren-structure->list u*)
 ↪  (µkanren-structure->list v*) s assocs))))
         ; the hottest path: a unification answers `(s* assocs*)` or #f, no need for
          ↪  a checked match.
         ((and (pair? u*) (pair? v*))  (let1 (r (µkanren-state-unify (car u*) (car
          ↪  v*) s assocs))
                                         (and r (µkanren-state-unify (cdr u*) (cdr v*)
                                           ↪  (car r) (cadr r)))))
+        ; an extension term against a pair or an atom.
+        ((and (pair? µkanren-extensions) (µkanren-extension/term u* v*)) => (λ (e)
 ↪  ((µkanren-extension-unify e) u* v* s assocs)))
         (else #f))))
```

The variable clauses, above this excerpt, and the pair–pair clause are untouched. The hook may call `µkanren-state-unify` back on parts: the λ layer's ξ rule opens both bodies with one new name and unifies them with micro's own unifier. So a list nested in a λ-term is still unified by micro, and a λ-term nested in a list is still handed to the hook. Before the edit, two records unified slot by slot, so an abstraction would have been compared by its hint and its raw body.

atom? tells micro's occurs check that a record holds no variable, so the check need not look into its slots. For the λ layer, eigenvariables and indices are atoms. Without it micro rebuilt every eigenvariable as a slot list at each binding; adding it cut type inference of the Church numeral 400 from 311 to 254 ms (an 18% gain on the λ workloads).

```
-               ((µkanren-structure? w) (O (µkanren-structure->list w)))
+               ((µkanren-structure? w) (and (not (µkanren-extension/atom? w)) (O
 ↪  (µkanren-structure->list w))))
```

occurs gets the last word when micro's occurs check fails on $\alpha := v$ and v may hold extension terms. Inside a λ-term an occurrence may be only apparent: $q = F\,q$ holds with $F := \lambda z.\, z$, and $F = \lambda x.\, G\,(F\,x)$ with $G := \lambda z.\, z$. The binding site, closed in Table 10.4, is opened on its failure path only:

```
+    (cond
+      ((or unsafe (not (occur? v)))
+        (µkanren-state-match ((vc S D A T tags) s)
+          (let* ((S* (µkanren-update/sbral α v S))
```

```
+                (s* (make-μkanren-state/X vc S* D A T tags (μkanren-state-X s)))
+                (assocs* (cons `(,α . ,v) assocs)))
+           `(,s* ,assocs*))))
+     ; an occurrence inside an extension's term may be only apparent: the extension
 ↪ decides.
+     ((pair? μkanren-extensions) (μkanren-extension/occurs α v s assocs))
+     (else #f)))
```

The λ layer answers by β-normalising v (if it is α itself the equation is trivial) and otherwise by variable elimination, which fails on a rigid occurrence and suspends one in the argument of a flexible term (Chapter 12).

`post` is the hook that carries the scope check. It runs after the bindings that one `=°` made and *before* D, A and T are verified again; it receives those bindings, may bind more (lowering, pruning, waking suspended equations), and returns the extended bindings, so that micro re-verifies the stores they touch. The post-`=°` chain, closed and wired for D/A/T before the edit, becomes an exported procedure:

```
+ (define (μkanren-post-unify s* assocs)
+   (let loop ((E μkanren-extensions) (s* s*) (assocs assocs))
+     (cond
+       ((null? E) (cond
+                    ((μkanren-verify-D/touched (μkanren-state-D s*) s* assocs)
+                      => (μ D* (μkanren-verify-D/post D* (μkanren-state-A s*)
 ↪ (μkanren-state-T s*) s*)))
+                    (else #f)))
+       ((μkanren-extension-post (car E)) => (λ (post)
+                                              (match/first (post s* assocs)
+                                                (#f #f)
+                                                ((,s** ,assocs*) (loop (cdr E) s**
 ↪ assocs*)))))
+       (else (loop (cdr E) s* assocs)))))
```

and `=°` calls it:

```
   (define ((=° u v) s)
+    (define post°
+      (λ1-match/first
+        ((,s* ,assocs)  (cond
+                          ((μkanren-state-equal? s s*) (✓° s*))
+                          ((μkanren-post-unify s* assocs) => ✓°)
+                          (else (✗° s*))))))
+    (cond
+      ((μkanren-state-unify u v s '()) => post°)
+      (else (✗° s))))
```

Read `post°` clause by clause. If raw unification returned the very state it was given (`μkanren-state-equal?` is the `eq?` test of Proposition 10.3), nothing was bound, so no hook runs and no store is re-verified; this is why invariant (I1) of Section 11.4.6 matters. Otherwise the bindings

`assocs` go through every extension's `post`, in registration order, each one receiving the state and bindings the previous one answered; then D is re-verified on the touched prefixes and A and T after it. A `#f` anywhere fails the `=°`.

This is how A1(c) is met without opening the binding site: *every* binding reaches the store through some `=°`, micro's included, and every `=°` ends in `µkanren-post-unify`. The binding is checked a moment after it is made, but before any other goal can see it. Running `post` before the D/A/T verification, not after, matters for two reasons: its own bindings must reach `µkanren-verify-D/touched`, and the λ layer moves a disequality prefix that has come to hold a λ-term out of D *before* micro re-unifies it.

Pitfall: A goal that unifies on its own must end in `µkanren-post-unify`

The guarantee "every binding is checked" holds because every goal of micro that commits a binding to the search state is an `=°`; micro's other calls of raw unification are trials whose states are thrown away (in `≠°`, in `absent°`, in subsumption) or reification. A goal of an extension that calls the raw `µkanren-state-unify` and returns its state bypasses `post` exactly as Trick 1's `bind/raw` bypasses D/A/T (Section 10.9.2): an eigenvariable could reach an older variable unchecked. Such a goal must pass the state and the exact new bindings to `µkanren-post-unify`, which is why it is exported; the λ layer does so at `∀°` exit (Section 11.4.4).

`diseq` lets an extension claim a disequality; micro's original body becomes `µkanren-≠`.

```
+  ; an extension may take the disequality over, when its terms are involved.
   (define ((≠° u v) s)
+    (cond
+      ((and (pair? µkanren-extensions) (µkanren-extension/diseq u v s)) => (λ (r)
 ↪  (if (eq? r 'fail) (✗° s) (✓° r))))
+      (else (µkanren-≠ u v s))))
+
+  (define (µkanren-≠ u v s)
```

`#f` leaves $u \neq v$ to the store D, a state keeps it in the extension's store, `'fail` says that u and v are already equal. A separate hook is needed because a D prefix is a list of bindings: the mgu of a pattern trial can contain variables the trial itself created, which are existential inside the negation (the `∃` of item A8), and a prefix cannot express that.

`tag` decides a type goal (`symbol°`, `number°`, a type made by `define-µkanren-tag`) on an extension term. Before the edit such a goal failed on every record that is not of the type, so `(symbol° (@ F 'a))` failed even though $F := \lambda z.\, z$ would make it a symbol; now the λ layer makes it wait.

```
+      ((,u* ⊣ (µkanren-extension/hook u* µkanren-extension-tag)) (cond
+
 ↪  (((µkanren-extension/hook u* µkanren-extension-tag) u* tag s) => ✓°)
+                                                                  (else (✗° s))))
```

`parts` says where `absent°` looks into an extension term: a list of parts, or `'stuck` when the term cannot be looked into yet, in which case `absent°` watches it whole, like a variable. Micro walks record slots, so without the hook `(absent° 0 q)` would reject λx.x because of the index 0 in its body, as it did in d1. With `projecting` true, the same hook tells the projection of T on an answer which parts' constraints the answer keeps, instead of splitting a λ-term into its raw slots.

```
+  (define (μkanren-absent-parts u s)
+    (cond
+      ((μkanren-extension/hook u μkanren-extension-parts) => (λ (parts) (parts u
 ↪  s)))
+      (else (μkanren-compound->list u))))
```

Its call sites are `absent°`, `μkanren-verify-T+` (both read `'stuck` as "watch") and `μkanren-project-T`, through the twin `μkanren-project-parts`.

`value` and `repr` give the deep value of an extension term and the Scheme expression that rebuilds it; `walk` is passed so that the hook can recurse on the parts. They are called by `μkanren-state-find/value` and `/repr`, so β-normalisation applies wherever micro deep-walks: answers, the stores in answers, `project°` and aggregation.

```
+            ((and (record-instance? w*) (not (μkanren-var? w*)))
+              (cond
+                ((and (pair? μkanren-extensions) (μkanren-extension/term w* w*)) =>
 ↪  (λ (e) ((μkanren-extension-value e) w* A s)))
+                (else (apply make-record-instance (map A (μkanren-structure->list
 ↪  w*))))))
```

`terms` and `forms` give the extension a say in the answer. `terms` returns the terms of its residual constraints whose variables the answer should name: micro reifies them after the answer, so a residual prints `α`, `β` instead of `_2`. `forms` returns the forms its constraints add to the answer's body, after micro's own.

```
+    (let* ((w* (μkanren-state-find/value w s))
+           (extra (μkanren-extension/terms s)))
+      ; the variables of the extensions' residual constraints are named too, after
 ↪  those of the answer.
+      (match1/first ((,s* ,c ,vars-reversed ,assocs) (μkanren-state-reify (if
 ↪  (null? extra) w* (cons w* extra)) s))
```

```
+               (X**   (μkanren-extension/forms s s*))
+               (body  (μkanren-form w** D** A** T** X** s*)))
```

`environment` names the environment in which `°->list/ground` evaluates answers. The λ layer gives its own module environment, where `ϰ`, `@` and `≐` are bound, so a ground answer evaluates back to a live λ-term that can be applied again (the pedagogy graft from d3).

```
-                         (let ((E (eval `(lambda ,(cadr expr) ,(last expr))
 ↪  (µkanren-answer-environment)))
+                         (let ((E (eval `(lambda ,(cadr expr) ,(last expr))
 ↪  (µkanren-extension/environment)))
```

11.4.4 Two primitives: forget and post-unify

The protocol exports two procedures besides the registry.

`(µkanren-state-forget α s)` unbinds α:

```
+  (define (µkanren-state-forget α s)
+    (let1 (S (µkanren-state-S s))
+      (µkanren-state-with s S: (update/sbral (µkanren-var-index/sbral S α)
 ↪  *µkanren-unbound* S))))
```

A `post` hook uses it to take back a binding that `=°` has just made, turning it into a constraint of its own, and leaves that binding out of the bindings it answers. Since `post` runs before D, A and T are verified, those stores never see the binding. The λ layer needs this when the scope check of $X := t$ cannot be decided yet, because t contains a non-pattern that only a later binding can resolve: the binding becomes the suspended equation $X \doteq t$, as Teyjus would delay it. For instance $\exists q\, \exists G\, \exists H\, \forall y.\ q = G\,(H\ y)$ leaves `q` unbound with a residual; a later $G := \lambda a.\,\mathrm{k}$ solves it with $q = \mathrm{k}$ (Chapter 12).

`µkanren-post-unify`, shown above, is exported for goals of an extension that unify on their own and must then run the same chain as `=°`. The λ layer calls it at ∀ exit, where the raising of the variables of a residual constraint makes new bindings that must wake the suspended equations and be checked against D, A and T.

11.4.5 One generic change to absent°

The third review found a defect of micro itself, independent of λ-terms. Micro decided an atom against an *absento* datum with `equal?` only, although a datum that holds a variable may still become that atom. With the unedited micro:

```
(pretty-print (°->list #f (fresh° (q D) (absent° D q) (=° q 'a) (=° D 'a))))
(pretty-print (°->list #f (fresh° (q D) (=° q 'a) (=° D 'a) (absent° D q))))
(pretty-print (°->list #f (fresh° (q D) (absent° D q) (=° q 'a))))
```

```
((λ () 'a))
((λ () 'a))
((λ () 'a))
```

The first two answers claim that `a` is absent from `a`. With the edited micro, which decides such an atom by `µkanren-tag-unify` (as it already did for a compound) and watches it while it is pending:

```
()
()
((λ () 'a))
```

The third answer stays, correctly: `D` is not in the answer and can be chosen different from `a`. Two smaller generic changes come with this one. An answer keeps a pending atom constraint when its datum names a variable of the answer, and prints that datum as a term, `(absent? (@ α 'b) 'a)`. And when a type turns an *absento* into a disequality prefix of D (micro's subsumption), a datum that holds an extension term stays in T instead (the predicate `μkanren-extension/holds-term?`), so D never holds one:

```
+      ((((,α* . ,tag) . ,T*) ⊣ (and (equal? α α*) (not
 ↪  (μkanren-extension/holds-term? (μkanren-tag-name tag)))))
+        (let1 (D* (μkanren-ext-D α tag D s)) (μkanren-update-D/T+ α T+ D* T* s)))
```

11.4.6 The two inherited invariants

A client of the protocol must respect two invariants that micro already relies on.

Definition 11.13 (Protocol invariants)**.**

(I1) **eq?-entailment.** A hook that changes nothing returns the very state object it was given.

(I2) **Extended bindings.** A hook that binds returns the list of bindings it received extended with every binding it made, pruning and lowering included.

(I1) is Proposition 10.3 carried over to the hooks. Micro reads "unification gave back *s*" as "the terms are already equal": `≠°` then fails, `=°` skips re-verification, and the re-verification of a D prefix declares it violated. A hook that rebuilt an unchanged state would make `(≠° (ƛ (a) a) (ƛ (b) b))` look satisfiable (the λ analogue of `(≠° 1 1)` in the pitfall after Proposition 10.3). For the λ layer this has a concrete consequence: the names that the ξ rule opens under binders live outside the state, so a ξ step that binds nothing returns *s* itself. So the λ disequality below is recognised as an equality that already holds, and fails, exactly as micro's `(≠° 1 1)` does:

```
(pretty-print (°->list #f (fresh° (q) (≠° (ƛ (a) a) (ƛ (b) b)))))
(pretty-print (°->list #f (fresh° (q) (≠° 1 1))))
```

```
()
()
```

(I2) is what keeps the stores honest. `μkanren-verify-D/touched` re-verifies only the prefixes whose variables appear among the new bindings; a binding made by a hook and not reported would leave a violated disequality in D. This is Trick 1's lesson (Section 10.9.2): a state produced by raw unification must go through the D/A/T chain with its exact bindings.

Pitfall: A hook that forgets a binding must also drop it from the bindings

(I2) has a converse. When `post` takes a binding back with `μkanren-state-forget`, it must remove it from the bindings it answers; otherwise the stores would be re-verified against a binding that S no longer holds. The λ layer's `μkanren-enforce` does both, through `μkanren-drop`.

11.4.7 From closed points to hooks

Table 11.6 reads the protocol against Table 10.4: for each closed point of (aux kanren micro), what the edit opens and what it leaves closed on purpose.

Table 11.6: Closed points of (aux kanren micro) and the hooks that open them.

closed point (Table 10.4)	opened by	still closed
the field list of the state, its positional constructions	the field `X`, copied at the two constructions	the six-ary `µkanren-state-match`
the unification `cond`	`unify` in the structure–structure and `else` clauses	the variable clauses, pair–pair
the binding site `µkanren-state-update`	`atom?` inside the occurs check; `occurs` on the failure path; the scope check moved to `post`	var–var orientation (older to newer)
the post-`=°` chain	`post`, threaded; `µkanren-post-unify` exported; `µkanren-state-forget`	—
reification (D/A/T only)	`value`, `repr`, `terms`, `forms`; `parts` in projection mode	purification and subsumption of micro's stores
`°->§`: fixed empty initial state	not needed: `X = '()` means empty; `environment` for ground answers	fixed query index 0
(listed as open: `≠°`, `absent°` and types work on any record, slot by slot)	`diseq`, `parts`, `tag`	—

The three needs listed at the end of Chapter 10 correspond to three rows: "a named place in the state" is `X`, "a call after every binding" is `post`, and "a say in reification" is `value`, `repr`, `terms` and `forms`. The rest of the protocol exists because micro's stores and occurs check, written for first-order data, look into records slot by slot.

Tutorial: One =° on a λ-term, through micro and the hooks

To watch the protocol at work we re-register the λ extension under its own name with some hooks wrapped by a printer. Registering under the same name replaces it (Section 11.4.2). Micro exports no accessor of the hooks, so the script reads the slots of the registered record directly, with `record-instance-slot` of `(chicken memory representation)` (imported with `srfi-1`, besides `(aux base)`, `(aux stream)`, (aux kanren micro) and `(aux kanren lambda)`); this is for tracing only.

```
(define fields '(name term? atom? unify occurs post diseq tag parts
                 value repr terms forms environment))
(define (orig h)
  (record-instance-slot µkanren-λ-extension (list-index (λ (f) (eq? f h))
   ↪  fields)))
(define (D t) (if (µkanren-λ-term? t) (µkanren-datum t) t))
(define (B as) (map (λ (p) (list (car p) ':= (D (cdr p)))) as))
(define (say . xs) (display "  ") (for-each display xs) (newline))
```

```
(define traced
  (make-μkanren-extension 'λProlog     ; same name: replaces the λ extension
    term?: (orig 'term?)
    atom?: (λ (u) (let1 (r ((orig 'atom?) u))
                    (say "atom?  " (D u) " => " r) r))
    unify: (λ (u v s as)
             (say "unify  " (D u) " =? " (D v))
             (let1 (r ((orig 'unify) u v s as))
               (say "unify  => " (and r (B (cadr r)))) r))
    occurs: (orig 'occurs)
    post: (λ (s as)
            (say "post   " (B as) " at depth " (μkanren-λ-depth s))
            (let1 (r ((orig 'post) s as))
              (say "post   => " (and r (B (cadr r)))) r))
    diseq: (orig 'diseq) tag: (orig 'tag) parts: (orig 'parts)
    value: (orig 'value) repr: (orig 'repr)
    terms: (λ (s) (let1 (r ((orig 'terms) s)) (say "terms  => " r) r))
    forms: (λ (s s*) (let1 (r ((orig 'forms) s s*)) (say "forms  => " r) r))
    environment: (orig 'environment)))
(μkanren-extension-register! traced)
(pretty-print (°->list/datum (fresh° (q) (∀° (x) (=° (@ q x) `(g ,x ,x))))))
(pretty-print (°->list #f (fresh° (q) (∀° (x) (=° q x)))))
```

```
  unify  (@ _0 (eigen x)) =? (g (eigen x) (eigen x))
  atom?  (ƛ (x) (g x x)) => #f
  atom?  (bvar 0) => #t
  atom?  (bvar 0) => #t
  unify  => ((_0 := (ƛ (x) (g x x))))
  post   ((_0 := (ƛ (x) (g x x)))) at depth 1
  post   => ((_0 := (ƛ (x) (g x x))))
  terms  => ()
  forms  => ()
((ƛ (x) (g x x)))
  atom?  (eigen x) => #t
  post   ((_0 := (eigen x))) at depth 1
  post   => #f
()
```

The first query, `pi x\ F x = g x x`, step by step.

1. `∀°` raises the depth to 1 and makes the eigenvariable `x` of level 1; `q` is `_0`, of level 0.
2. Micro's `=°` calls `μkanren-state-unify`. Both sides are walked: the left one is an application record, the right one a pair. No variable clause applies, the left side is a structure but the right one is not, and the pair–pair clause does not apply: the final `else` finds the owning extension and calls `unify`.
3. The hook head-normalises both sides. The left one is flexible, `_0` applied to the pattern `(x)`: one eigenvariable, whose level 1 exceeds the level 0 of `q`. So this

is flex–rigid. Variable elimination replaces `x` by the bound variable of the new abstraction, and the hook calls micro back to bind `_0 := (ƛ (x) (g x x))`.

4. That binding goes through micro's binding site and its occurs check. The check looks into the abstraction (`atom?` is false) and stops at each `(bvar 0)` (`atom?` is true). No occurrence of `_0`: the binding is made, and the hook returns it.

5. Back in `=°`, the state has changed, so `µkanren-post-unify` runs the `post` hook on the new binding. Levels exist, and `_0` is below the highest level reached, so the scope check runs: the bound term holds no eigenvariable of level > 0 (`x` has become an index), and no variable to lower. The binding passes; E is empty, so nothing is woken. Then micro verifies D, A and T, all empty.

6. The `∀°` body has answered; `∀°` gives the state back depth 0 and discharges the constraints mentioning `x` (there are none).

7. Reification asks the extension for the terms of its residual constraints (none) and for its forms (none); the `value` and `repr` hooks, not traced here, print the abstraction.

The second query, `sigma Y\ pi x\ Y = x`, never calls `unify`: `q` is a variable, so micro's variable clause binds it directly. The occurs check meets the eigenvariable, an atom, and does not look into it. Micro makes the binding `_0 := x`; but the `post` hook sees that `x` has level 1 and `_0` level 0, which violates condition (2) of A1, and answers `#f. =°` fails before any other goal could see the binding. This is A1(c) at work: the binding was made by micro's own clause, and the extension still had the last word.

11.4.8 Cost

Table 14.3 in Chapter 14 gives the cost of the protocol on first-order search, compiled with `csc -O2 -d1`: the unedited micro against the edited micro, without and with `(aux kanren lambda)` loaded but unused. The protocol costs at most 1% end to end on `append°` (5923 to 5959 ms in round 2), within the 2.5% round-to-round noise of the unedited binary. With the λ extension loaded, the disequality-heavy perm6 costs 8–12% (d2 cost 45–50%): the remainder is the `post` hook's look at the new bindings while D is not empty. One cost is not removed and is documented: once any `∀°` has run, levels exist, and every later binding below the top level gets the scope check, which costs first-order search 1.7 to 2.3 times (Chapter 14 plans one walk for the occurs and the scope check).

11.5 Review history

After the choice of d2 and the grafts, the proposal went through three adversarial review rounds. Each round attacked the proposal itself with hand-written probes and with randomly generated goals, run with their conjunctions in both orders. Every report was first reproduced, then fixed,

a code fix with a regression case (in round 3 each new case was checked to fail on the proposal before the fix).

Table 11.7: The three adversarial review rounds (design notes, §10.4–10.6).

round	reports	regression cases	typical defects
1	13	12 (44 assertions)	disequality prefixes holding an eigenvariable left in D (a free `(eigen x)` printed, a crash); pruning inside an argument that a non-pattern may drop; occurs check not modulo β; raw record slots in projected *absento*; a binder hint capturing `cons` in ground answers
2	8	6 (31 assertions)	raising at ∀ exit not closing every constraint; η-redexes created by later bindings; occurs failure on non-rigid arguments; the extensional reading absorbed by T; first-order cost after a ∀°
3	12	9 (53 assertions)	non-patterns with functional arguments; disequalities over unreachable arguments; *absento* of an eigenvariable under a type; user types on λ-terms; η-short occurs check; a datum reducing to an atom; two types on one flexible term; first-order cost with D or A non-empty; λ-terms in the show module's cells view

All 33 reports were real. In round 1 the twelve code defects were fixed, each pinned by a case, and one documentation error was corrected. In round 2 two reports were one defect; six were fixed with regression cases, and the seventh, the cost of first-order search after a `∀°`, was measured and documented rather than changed, since the cheap fix would give up the guarantee that host code cannot smuggle an eigenvariable into an older variable. In round 3 three pairs of reports were one defect each; all nine defects were fixed with a regression case each. Round 3 also produced the generic change to `absent°` of Section 11.4.5. The defects themselves belong to the λ layer and are discussed where it is described (Chapter 12).

11.6 Summary

- The requirements are nine acceptance items: the scope of ∀ under Nadathur, Jayaraman and Kwon's binding conditions, for *every* binding (A1); AUGMENT with the book's clause order and variable sharing (A2); type inference as the test of ∀ and ⊃ together (A3); λ-tree syntax with α, β and η (A4); L_λ unification with mgus and delayed non-patterns (A5); λProlog's answers where Ma and Friedman differ (A6); the book's programs (A7); micro's stores on λ-terms (A8); micro's suites unchanged (A9). Order independence runs through all of them. They are pinned by 83 cases and 320 assertions.
- Of the four meanings of `=°` on λ-terms, pattern unification with universe levels is retained: it is λProlog's, it is decidable and unitary (one state per `=°`), and levels give GENERIC for one integer per variable and one comparison per binding.
- Three prototypes were built: a layer with no edit (d1), a hooks protocol (d2), a native engine (d3). d1 cannot check bindings made by micro's `=°`, so eigenvariables leak through `cons°`; d3 had a crash and an η bug that made `≠°` unsound; d2 had seven non-soundness

defects. The reviews scored them 15.5, 23 and 21; d2 became the base, and d3's pedagogical strengths and d1's precision were grafted onto it.

- The protocol adds a seventh state field `X` that micro copies and never reads, a global registry of extensions dispatched by `term?`, and thirteen hooks: `atom?` and `occurs` in the binding site, `unify` in the unification `cond`, `post` in the chain after `=°`, `diseq`, `tag` and `parts` in the stores, `value`, `repr`, `terms` and `forms` in reification, `environment` in the runner.
- The scope check lives in `post`: every binding passes through some `=°`, and every `=°` ends in `µkanren-post-unify`, which runs the hooks before D, A and T are verified. An extension goal that unifies on its own must end in `µkanren-post-unify` too. `µkanren-state-forget` lets `post` turn a binding into a suspended equation.
- A client must return the very state when it changes nothing (eq?-entailment) and report every binding it makes (extended bindings).
- The edit is +235/−41 lines, λ-agnostic, and costs about 1% on first-order search; micro's 125 cases pass unchanged with and without the λ layer. Three adversarial rounds sent 13, 8 and 12 reports, all real; every defect but one documented cost was fixed with a regression case.

12 The module (aux kanren lambda)

What and why

Chapter 10 took (aux kanren micro) apart and ended, in Section 10.9, with the list of points an extension cannot reach from outside: the state has no free slot, and the unifier, the walkers and the reifier are closed. Chapter 11 opened exactly those points with a small, λ-agnostic protocol of hooks and explained why the proposal is a client module of that protocol rather than a native engine. This chapter is about the client itself: the module `(aux kanren lambda)`, which brings into (aux kanren micro) the three ingredients of Part I,

- λ-tree syntax (Chapter 6): object-level binders are meta-level abstractions;
- higher-order pattern unification, L_λ (Chapter 7), with universe levels in place of the mixed prefix and with non-patterns suspended ("dynamic L_λ");
- universal and hypothetical goals, GENERIC and AUGMENT, over hereditary Harrop formulas (Chapter 3).

The chapter is the formal heart of the proposal. It states what the terms are (Section 12.1), what the module keeps in the state (Section 12.2), the unification rules with their side conditions and the termination argument (Section 12.3), the rules for `∀°` and `⊃°` and why levels make them sound (Section 12.4), how micro's disequality, absence and type constraints read λ-terms (Section 12.5), how answers are printed (Section 12.6), and, as a reference, the exported API (Section 12.7). The curated book examples are the subject of Chapter 13; the known limits, the costs and the merge plan are in Chapter 14.

The module is 1255 lines, 885 of them neither blank nor comment. In the repository it lives, with its tests and examples, under `doc/hol-kanren/prototype/`, and after merging as `src/aux.kanren.lambda.scm`, with the suites `src/test/microkanren-lambda.scm` (83 cases, 320 assertions) and `src/test/microkanren-lambda-limits.scm` (6 cases). Every Scheme snippet below was run against the module and micro with the protocol applied; its output is pasted verbatim under it. Before the imports below work, the edited micro must be built: Section 14.1.4 shows how, without installing anything, and Section 14.4 how to merge it. The snippets import

```
(import scheme (chicken base) (chicken pretty-print)
        (chicken condition) (chicken memory representation)
        (aux base) (aux stream) (aux kanren micro) (aux kanren micro show)
        (aux kanren lambda))
```

Several of them use a *probe*: a goal that prints part of the state and succeeds. A goal of (aux kanren micro) is a procedure from a state to a stream of states (Definition 10.7), so a probe is three lines; each snippet shows the one it uses.

12.1 Terms

12.1.1 Four records

First-order structure stays Scheme data, exactly as in (aux kanren micro): pairs, vectors, foreign records and atoms (Definition 10.1). The module adds four record types, defined at the top of the file:

```
(define-record µkanren-lam hint body)        ; λ.body, locally nameless
(define-record µkanren-app head args)        ; head applied to a list of arguments
(define-record µkanren-eigen name level id)  ; a scoped constant of GENERIC
(define-record µkanren-bvar index)           ; a de Bruijn index, only inside bodies
```

Definition 12.1 (λ-terms of the module)**.** Extend the terms of (aux kanren micro) (Definition 10.1) with

$$t \ ::= \ \ldots \mid c^{\ell} \mid \#i \mid \lambda^{h}.t \mid t \mathbin{@} \langle t_1, \ldots, t_n \rangle$$

where c^{ℓ} is an eigenvariable of level $\ell \in \mathbb{N} \cup \{\infty\}$ (`µkanren-eigen`), $\#i$ a bound variable with de Bruijn index $i \geq 0$ (`µkanren-bvar`), $\lambda^{h}.t$ an abstraction whose binder carries the printing hint h, a symbol (`µkanren-lam`), and $t \mathbin{@} \bar{t}$ an application with $n \geq 1$ (`µkanren-app`). A term is *locally closed* when every $\#i$ in it lies below at least $i + 1$ abstractions. Only locally closed terms are ever bound to variables or stored in constraints.

The representation is *locally nameless* [6]: bound variables are de Bruijn indices [3], free ones are names (logic variables and eigenvariables). Three design choices follow from it.

- **The hint is only for printing**, and it is an *interned* symbol. The binder's identity is its position. Two terms built alike are therefore `equal?`, so λ-terms can be grouped by `groupby°` and compared by the host.
- **An eigenvariable's identity is its `id`**, an uninterned symbol, compared with `eq?`. Its `name` is a hint and its `level` is the universe it lives in (Section 12.2). Level ∞ (`+inf.0`) marks the names that the unifier opens when it descends under binders and the placeholders that `ϰ` uses while building a body.
- **The first 64 bound variables are preallocated** in a vector, so the common indices allocate nothing.

The de Bruijn operations (shift, instantiate, close) never walk the substitution: a term bound to a variable is locally closed, so a binding never contributes a loose index. Instantiation shifts the substituted term only when it has loose indices itself, which is decided once per instantiation.

12.1.2 The constructors `ϰ` and `@`

The user never builds these records by hand. Two constructors give the surface syntax of λ-tree terms:

```
(define-syntax ƛ
  (syntax-rules ()
    ((_ () body) body)
    ((_ (x y ...) body) (μkanren-abstract 'x (λ (x) (ƛ (y ...) body))))))

; λ.(f p), with the placeholder p closed into #0
(define (μkanren-abstract name f)
  (let1 (p (μkanren-eigen* name +inf.0))
    (μkanren-lam/η name (μkanren-close (f p) p))))

(define (@ f . args) (μkanren-beta f args))   ; β at once on an abstraction
```

`(ƛ (x) body)` binds the *Scheme* variable `x` to a fresh placeholder eigenvariable while `body` is evaluated, then closes that placeholder into the index #0. The object binder is thus a Scheme binder: shadowing, lexical scope and quasiquotation all behave as a Scheme programmer expects, which is the point of λ-tree syntax [MN p. 177]. Two invariants are enforced at construction.

- **η-short.** `μkanren-lam/η` builds $\lambda x.(h\,\bar{a}\,x)$ as $h\,\bar{a}$ when x is not free in $h\,\bar{a}$. The unifier builds its abstractions the same way. This matters because micro's own occurs check runs before any hook can see an equation: $\lambda x.(F\,x) = F$ holds by η, but in η-long form micro would see F inside the value it binds F to and fail; built η-short, the left side *is* F and the equation is trivial.
- **β at construction.** `@` applied to an abstraction β-reduces at once, and nested applications are flattened: `(@ (@ f a) b)` is `(@ f a b)`.

Example 12.2 (Construction and printing)**.** Every record has a printer that shows its term as data. The raw record of $\lambda x.(f\ x)$ shows the hint `x` and the index in the body:

```
(ƛ (x y) (cons x y))
(record->vector (ƛ (x) `(f ,x)))     ; from (chicken memory representation)
(equal? (ƛ (x) `(f ,x)) (ƛ (x) `(f ,x)))
(equal? (ƛ (x) x) (ƛ (y) y))
(μkanren-α=? (ƛ (x) x) (ƛ (y) y))
(@ (ƛ (x) `(g ,x ,x)) 'a)
(ƛ (x) (@ 'g x))
(ƛ (x) `(g ,x))
(@ 'g 'a)
(equal? (@ 'g 'a) '(g a))
(ƛ (a) (ƛ (a) a))
```

```
=> (ƛ (x y) (x . y))
=> #(aux.kanren.lambda#μkanren-lam x (f #<aux.kanren.lambda#μkanren-bvar>))
=> #t
=> #f
=> #t
=> (g a a)
=> g
=> (ƛ (x) (g x))
```

```
=> (@ g a)
=> #f
=> (ƛ (a a1) a1)
```

Line by line: nested binders print merged, `(ƛ (x y) ...)`; the body of a `µkanren-lam` holds an index, not the name; terms built alike are `equal?`; α-variants with *different* hints are not `equal?` but are α-equal; `@` reduced the redex; $\lambda x.(g\,x)$ written with `@` is the η-redex of g and was built as `g`; written with a quasiquoted list it is no η-redex (the list `(g x)` is data, not an application), so it stays an abstraction; the λ-tree application of a constant prints with `@` and is not the list `(g a)`; shadowing is lexical, and the printer renames the inner binder apart (Section 12.6).

12.1.3 Three kinds of application

An application is classified by its head *after walking it* in the current substitution:

Definition 12.3 (Flexible, β-redex, rigid; head normal form)**.** An application $h\,@\,\bar{t}$ is *flexible* if h walks to an unbound logic variable, a *β-redex* if h walks to an abstraction, and *rigid* otherwise (a constant, an eigenvariable, a bound variable). The *head normal form* hnf(t) (`µkanren-hnorm`) walks t, flattens nested applications, contracts head β-redexes, and contracts an abstraction whose body has *become* an η-redex through a binding (e.g. $\lambda x.F\,(G\,x)$ once $G := \lambda z.z$).

Under the L_λ restriction every contraction hnf performs is a β_0 renaming (Definition 6.13). On instantiated heads `@` gives full β, which is what the book calls dynamic L_λ [MN p. 225–226].

```
(µkanren-hnorm (@ (ƛ (x) (@ x 'b)) (ƛ (y) `(g ,y ,y))) µkanren-state-empty)
(°->list #f (fresh° (q F) (=° q (@ F 'a))))
(°->list/datum (fresh° (q F) (=° q (list F (@ F 'a))) (=° F (ƛ (z) `(h ,z)))))
```

```
(g b b)
((λ (α) (@ α 'a)))
(((ƛ (z) (h z)) (h a)))
```

The second answer is a flexible term, printed as `(@ α 'a)`; once F is bound (third line) the same term is a β-redex and its value is `(h a)`.

12.1.4 Scheme lists versus λ-tree application

The object language of the book's programs is written with Scheme lists, as (aux kanren micro) always did: `abs` M is `` `(abs ,M) ``, `app` M N is `` `(app ,M ,N) ``, with M a meta-level abstraction for a binder. The λ-tree application `(@ 'g 'a)` and the list `'(g a)` are distinct terms and never unify:

```
(°->list #f (fresh° (q) (=° (@ 'g 'a) '(g a))))
```

```
  => ()
```

This is deliberate. One of the three prototypes identified the two (a spine notation for lists) and lost commutativity of conjunction: with η, the same pair of goals gave different answers in the two orders. Keeping lists as data and `@` as the only application keeps (aux kanren micro)'s first-order unification untouched for lists, and confines the higher-order rules to terms that contain a λ-record.

Pitfall: What is and is not an η-redex

η-contraction applies to λ-tree applications only. `(ƛ (x) (@ 'g x))` *is* `g`, but `(ƛ (x) `(g ,x))` is an abstraction over a two-element list, and stays one. The module is untyped, so it cannot η-*expand* a term against its type: η-expansion happens only on demand, when one side of an equation is an abstraction (rule η below), as in Miller's untyped treatment [34, §9.3]. A rigid list such as `'(g a)` therefore equals an abstraction only when the abstraction's body equals the λ-tree application `(@ '(g a) x)` of the list to the bound variable (and `ƛ` already builds such an η-redex as the list itself); never when the body is a list such as `` `(g a ,x) ``.

12.2 The extension's state

12.2.1 The record in `X`

The protocol gives (aux kanren micro)'s state a seventh slot `X` that micro copies but never reads (Chapter 11). The module keeps one record in it:

```
(define-record μkanren-hh depth top L Ln Γ E N)
(define μkanren-hh0 (make-μkanren-hh 0 0 empty/sbral 0 '() '() '()))
(define (μkanren-hh s) (let1 (x (μkanren-state-X s)) (if (null? x) μkanren-hh0 x)))
```

`X` = `'()`, the value in `μkanren-state-empty`, reads as "everything empty", so the initial state of every runner works unchanged.

Definition 12.4 (The λ state)**.** The record $\langle d,\ top,\ L,\ L_n,\ \Gamma,\ E,\ N\rangle$ holds:

d **(`depth`)**
: the current universe level, raised by `∀°` for the extent of its body;

top
: the highest level reached so far in this branch;

L, L_n
: the levels of the logic variables: L is a skew-binary random-access list (aux's `sbral`) indexed like the substitution S, holding the levels of the variables with index $< L_n$. The level of variable i is

$$\ell(i) \;=\; \begin{cases} 0 & i < 0 \text{ (a reified variable)} \\ L[L_n - 1 - i] & 0 \le i < L_n \\ d & i \ge L_n \end{cases}$$

  so a variable created since L was last extended has the current depth. L is padded with d exactly when the depth changes and whenever the unifier creates a variable of another level; lookup is $O(\log n)$;

Γ
: the assumed clauses: an alist from a relation to its clauses, most recent first, one entry per distinct relation;

E the suspended constraints: equations `(= u v)` outside L_λ (or bindings whose scope check a non-pattern decides), types `(tag tag u)` on flexible terms, and absentos `(absent d u)` whose datum outlived its eigenvariable's scope;

N the disequalities that involve λ-terms or eigenvariables, as `(reading u . v)`. micro's D never holds one (Section 12.5).

Levels are the module's whole account of quantifier order. A logic variable created at depth d stands for an ∃ placed after the first d universals of its branch; an eigenvariable created by the $(d{+}1)$-th `∀°` has level $d+1$. The binding conditions of Nadathur, Jayaraman and Kwon [43] (Definitions 9.15 and A.27) are then stated on levels: a variable of level ℓ may be bound only to a term whose eigenvariables have level $\leq \ell$, and the variables of level $> \ell$ in that term are *lowered* to ℓ.

12.2.2 Copied, never mutated

The record is immutable. Every change builds a new one with `μkanren-hh-with`, a keyword-argument copy, and installs it with `(μkanren-state-with s X: x)`. Since a state is a value and `or°` gives each branch the state it was given, two branches can never see each other's depth, Γ or suspended constraints. What needs care is the *end* of a scope: `∀°` and `⊃°` change the record for their body only, and each answer of the body must give the change back. Both do so on every answer state (Section 12.4). The following probe shows depth and levels through two nested `∀°`:

```
(define ((probe label . vars) s)
  (display label) (display ": depth ") (display (μkanren-λ-depth s))
  (display ", levels ") (write (map (λ (v) (μkanren-λ-level v s)) vars))
  (display ", E ") (write (μkanren-λ-delayed s))
  (display ", N ") (write (μkanren-λ-disequalities s)) (newline)
  (✓° s))
(°->list #f
  (fresh° (q)
    (probe "start" q)
    (∀° (x)
      (fresh° (Y)
        (probe "inside ∀x" q Y)
        (∀° (z) (fresh° (W) (probe "inside ∀x∀z" q Y W)))
        (probe "back in ∀x" q Y)
        (=° (@ q x) (@ Y x x))
        (probe "after q x = Y x x" q Y)))
    (probe "after ∀x" q)))
(°->list #f (fresh° (q) (or° (∀° (x) (probe "branch 1" q) (=° q 1))
                             (and° (probe "branch 2" q) (=° q 2)))))
```

```
start: depth 0, levels (0), E (), N ()
inside ∀x: depth 1, levels (0 1), E (), N ()
inside ∀x∀z: depth 2, levels (0 1 2), E (), N ()
back in ∀x: depth 1, levels (0 1), E (), N ()
```

```
after q x = Y x x: depth 1, levels (0 1), E (), N ()
after ∀x: depth 0, levels (0), E (), N ()
((λ (α) (ϗ (x) (@ α x x x))))
branch 1: depth 1, levels (0), E (), N ()
branch 2: depth 0, levels (0), E (), N ()
((λ () 1) (λ () 2))
```

Tutorial: Reading the probe

q is created at depth 0, Y inside `∀° (x)` at depth 1, W at depth 2; after each `∀°` the depth is back where it was. Written as a mixed prefix, the goal is $\exists q\, \forall x\, \exists Y\, \forall z\, \exists W$, and each level is the number of universals to the left of the quantifier: $q:0$, $x:1$, $Y:1$, $z:2$, $W:2$. The equation $q\,x \doteq Y\,x\,x$ is solved inside the scope, and its answer shows the levels at work. q has level 0 and its argument x (level 1) is a pattern for it, so the unifier eliminates q: $q := \lambda x.\, t'$ where t' is $Y\,x\,x$ with x turned into the bound variable. But Y has level 1 and q level 0: q's value may not mention a variable that could see x unless x is passed explicitly. $Y\,x\,x$ is no pattern (x is repeated), but both its arguments are q's own argument x, so none of them has to be dropped. Y is therefore *lowered* and *raised* at once, $Y := \lambda a\, b.\, Y'\,a\,b\,x$ with $\ell(Y') = 0$, and $q := \lambda x.\, Y'\,x\,x\,x$. The answer `(ϗ (x) (@ α x x x))` is that binding; the rules responsible are FR and case (6c) of variable elimination, Definition 12.6. The last two lines show the branches: `∀°` raised the depth in branch 1 only.

The module's hooks keep the two protocol invariants of Definition 11.13; in particular, every function of the module that may leave the state unchanged is written to return it `eq?`.

12.3 Unification

12.3.1 Where the rules run

(aux kanren micro)'s unifier `µkanren-state-unify` still does all first-order work: variables, pairs, vectors, foreign records and atoms. Through the protocol's `unify` hook it hands any pair with a walked λ-record on one side, neither side a variable, to `µkanren-λ-unify`, which applies the rules of Fig. 12.1. The rules call back into `µkanren-state-unify` for subterms, so the two unifiers interleave freely. After `=°` has made its bindings, the `post` hook `µkanren-λ-post` runs before micro verifies D, A and T:

1. *settle*: the scope check **Check** of every new binding, then the wake-up of E, in rounds while a round binds (Section 12.3.7);
2. *migrate*: the prefixes of D that now hold a λ-term or an eigenvariable move to N, and the types of A on variables just bound to λ-terms move to the `tag` hook (Section 12.5);
3. *verify N*: every λ-disequality is tried again.

12.3.2 Notation

Write $\ell(X)$ for the level of a logic variable and $\ell(c)$ for that of an eigenvariable; the names the unifier opens under binders have $\ell(c) = \infty$. A *name* is an eigenvariable or, inside variable elimination, a bound variable of the term being walked.

Definition 12.5 (Pattern, read off levels)**.** A list of arguments $\bar{a} = a_1 \dots a_n$ is a *pattern for* a variable F if every hnf(a_i) is an eigenvariable, the a_i are pairwise distinct, and $\ell(a_i) > \ell(F)$ for every i (`µkanren-pattern?`). A flexible term $F\,\bar{a}$ is a pattern when $\bar{a}$ is a pattern for F.

For example, inside `(∀° (x y) ...)` with $\ell(F) = 0$, $\ell(x) = 1$, $\ell(y) = 2$, the lists $\langle x, y\rangle$ and $\langle y\rangle$ are patterns for F, while $\langle x, x\rangle$ (repeated), $\langle \mathtt{a}\rangle$ (a constant) and $\langle Y\rangle$ (a variable) are not. The same definition, with a bound variable of the term being walked counted as an eigenvariable of level ∞, is used inside variable elimination (case (6b) below). This is Miller's L_λ condition (Definition 7.15): the arguments are distinct variables "quantified inside the scope of F". With a prefix, "inside the scope" means "to the right of F"; with levels it means "of a level above F's", which is how Nadathur and Linnell read the condition without a prefix [44, §4]. An eigenvariable F could already see is not a pattern argument: in $\forall x\, \exists F.\ F\,x = \dots$ the term $F\,x$ is no pattern, since F may use x directly and the equation has several incomparable solutions.

Throughout, $\lambda.b$ is an abstraction with body b, $b[c/0]$ its body with #0 instantiated to c, $h\,\bar{u}$ an application, and F, G, H, X, Y logic variables. Both sides are put in hnf before a rule is chosen.

12.3.3 The rules

The rules of Fig. 12.1 are transformations, read top-down as in Figs. A.4 and A.6: the equation above the line is replaced by the equations, bindings or suspension below it. They are the rules of Fig. A.6 with the prefix erased into levels; Section 7.6 gives the same rules under other names, recalled in parentheses. ξ is ABS (Abs), η is ETA (Eta), Rigid is RR-DEC and RR-CLASH (Dec and Clash), FF-SAME is FF-SAME (the same-head case of FlexFlex), Delay is DELAY. FR covers ELIM (Elim) with its pruning, OCCURS and SCOPE, and also FF-DIFF (FlexFlex with distinct heads): instead of binding both heads to a new H, it eliminates F and prunes G only where needed, which gives the same mgu up to the name of H (Example 12.12). Var and Occurs are ELIM and OCCURS for a variable with no arguments, a case that micro's first-order unifier meets first, and they are where the eager scope check, **Check**, enters; FF-ONE orients a flexible–flexible pair as SWAP orients a flexible–rigid one.

The side conditions, in the order the code tests them (`µkanren-λ-unify` and the functions it calls):

Var — One side is an unbound variable. micro binds it, with its occurs check; the scope check **Check** runs afterwards, in the `post` hook. Variable–variable pairs are micro's as well.

ξ — Both sides are abstractions. c is a new eigenvariable of level ∞, so no variable can capture it except through its own pattern arguments. Because c is local to this very unification, the bindings made under it are checked at once (`µkanren-scoped`), not at the end of `=°`.

$$\frac{X \doteq t}{X := t \ \text{(micro, occurs check)} \quad \mathbf{Check}(X := t)}\ \textsc{Var}$$

$$\frac{\lambda.b \doteq \lambda.b'}{b[c/0] \doteq b'[c/0] \quad \mathbf{Check}\ \text{its bindings}}\ \xi \qquad \frac{\lambda.b \doteq t}{b[c/0] \doteq t\ @\ c \quad \mathbf{Check}\ \text{its bindings}}\ \eta$$

$$\frac{h\,\bar{u} \doteq h'\,\bar{v}}{h \doteq h' \quad \bar{u} \doteq \bar{v}}\ \textsc{Rigid} \qquad \frac{F\,\bar{a} \doteq F\,\bar{b}}{F := \lambda\bar{x}.\,H\,\langle x_i \mid a_i = b_i\rangle}\ \text{FF-}\textsc{same}$$

$$\frac{F\,\bar{a} \doteq t}{F := \lambda\bar{x}.\,t' \qquad t' = \mathrm{Elim}(\ell(F), F, \bar{a}, t)}\ \text{FR} \qquad \frac{F\,\bar{a} \doteq G\,\bar{b}}{G\,\bar{b} \doteq F\,\bar{a} \ \text{ by FR}}\ \text{FF-}\textsc{one}$$

$$\frac{u \doteq v}{E := E \cup \{\lambda L.\,u \doteq \lambda L.\,v\}}\ \textsc{Delay} \qquad \frac{X \doteq t \qquad X \text{ occurs in } t}{\top \text{ if } \mathrm{hnf}(t) = X;\ \text{else } X := \mathrm{Elim}(\ell(X), X, \langle\rangle, t)}\ \textsc{Occurs}$$

Figure 12.1: The unification rules of `µkanren-λ-unify`, applied after hnf of both sides; above the line the equation, below what is done with it. In ξ and η, c is a new eigenvariable of level ∞. In FF-SAME and FR, $\bar{x}$ are new binders, one per argument a_i, and Elim replaces each a_i in t by the bound variable of x_i. Variable elimination, Elim, is Definition 12.6; **Check** is Definition 12.7. Each rule may also answer ⊥ or suspend, as its side conditions say.

η One side is an abstraction and the other is not. The other side is applied to the new c. This is the only η-expansion the module performs.

RIGID Both sides are rigid applications with the same number of arguments: heads and argument lists are unified by micro. Two eigenvariables unify iff they are the same (`eq?` ids), two bound variables iff their indices agree. Any other pair with a λ-term on one side fails: different rigid heads, different arities (untyped), a λ-record against a pair or an atom.

FF-SAME Same head on both sides, both argument lists patterns. Equal lists: success with no binding. Different lengths: ⊥. Otherwise F keeps the positions where the arguments agree, with a new H of level $\ell(F)$ [MN p. 222]. If $\bar{b}$ is no pattern, DELAY.

FR One side is a pattern $F\,\bar{a}$; the other side t is rigid, or flexible with another head ("FF-diff": variable elimination prunes G as needed [MN p. 222]). If Elim fails the rule fails; if Elim suspends, DELAY. The binding $\lambda\bar{x}.\,t'$ is built η-short, its binders named after the eigenvariables $\bar{a}$.

FF-ONE Two flexible terms, only the right one a pattern: FR on G.

Delay No side is a usable pattern. The equation is pushed on E, closed over the names L that the unifier had opened so far ($\lambda L.u \doteq \lambda L.v$, so no ∞-level name escapes the unification that created it), *unless* u and v are already α-equal, in which case it is dropped.

Occurs Not a rule of `µkanren-λ-unify` but of the protocol's `occurs` hook: micro's occurs check failed on $X := t$ and t holds a λ-term. If hnf(t) is X itself the equation holds ($q = F\,q$ with $F := \lambda z.z$); otherwise X is eliminated with no arguments, which may fail, succeed, or suspend ($F = \lambda x.\, G\,(F\,x)$ holds with $G := \lambda z.z$). A first-order cycle, $X = f\,X$, holds no λ-term and fails in micro as before.

12.3.4 Variable elimination

FR is the book's variable elimination [MN p. 223] (Definition 7.21) with levels in place of the prefix. It walks the right-hand side once and builds the body of F's binding, pruning, lowering and raising the variables it meets on the way. Compared with Definition 7.21, three things are added: *lowering* with delayed raising (case (4)), where Definition 7.21 assumes a prefix already raised into $\forall\exists$ form (the eager raising of Example 7.31); the treatment of *non-patterns* inside the right-hand side (cases (6c) and (7)), which the book never meets since it stays in L_λ; and a finer *occurs* test (case (6a)), since an occurrence of F is fatal only when no instance can make it smaller. The contexts below exist to decide these three cases.

Definition 12.6 (Variable elimination)**.** Elim($\ell, F, \bar{a}, t$), with $\bar{a}$ a pattern for F and $\ell = \ell(F)$ (or F absent and $\bar{a}$ empty, for **Check**), walks hnf(t) at a local depth k (the binders crossed inside t) and in a *context*: *strict* (`top`: below binders only; `rigid`: below a first rigid constructor that is an application headed by a bound variable or by an a_i; `firm`: below a first rigid constructor that is a constant, an eigenvariable not in $\bar{a}$, a pair, a vector or a record, a root no instance of F can project onto) or *soft* (inside an argument of a flexible term that is no pattern, or `weak`: below a flexible head inside such an argument). It answers a term t' with new bindings, $\bot$, or *suspend*. By cases on the walked subterm:

(1) *Eigenvariable* c: if $c = a_i$, the bound variable of a_i; else if $\ell(c) \le \ell$, c itself; else $\bot$ (scope failure, Example 7.25). In a weak context the last case is trouble, not $\bot$.

(2) *Bound variable*: itself.

(3) *Abstraction* $\lambda.b$: λ.Elim(b) at depth $k+1$.

(4) *Unbound variable* Y: if $Y = F$, $\bot$ when $\bar{a}$ is empty (the first-order occurs check) and suspend otherwise; if $\ell(Y) \le \ell$, Y; in a soft context, trouble; otherwise *lower and raise*: $Y := Y'\,\bar{a}_Y$ with $\ell(Y') = \ell$ and $\bar{a}_Y = \langle a_i \mid \ell(a_i) \le \ell(Y)\rangle$, the arguments of F that Y could see; the result is Y' applied to their bound variables ("delayed raising", Example 7.31).

(5) *Rigid application, pair, vector, record*: homomorphic. A `top` context becomes `rigid` below an application whose head is a bound variable or in $\bar{a}$, and `firm` below any other head, a pair, a vector or a record; any other context is passed on unchanged.

(6) *Flexible $G\,\bar{b}$ in a strict context*:

   (a) if $G = F$: $\bot$ (occurs failure) when the context is not `top`, no b_j is *functional* (an abstraction, a variable or a flexible term, which may become an abstraction), and either the context is `firm` or $|\bar{b}| = |\bar{a}| + k_{\text{top}}$, with k_{top} the binders on top of t; otherwise suspend;

   (b) if $\bar{b}$ is a pattern for G (each b_j a local bound variable or an eigenvariable of level $> \ell(G)$, pairwise distinct): keep b_j iff it is local, or in $\bar{a}$, or $\ell(b_j) \leq \ell$; let *extra* $= \langle a_i \mid \ell(a_i) \leq \ell(G) \rangle$ if $\ell(G) > \ell$, else empty. If every b_j is kept, *extra* is empty and $\ell(G) \leq \ell$, $G\,\bar{b}$ is unchanged. Otherwise *prune*: $G := \lambda\bar{y}.\, G'\ \bar{y}|_{keep}\ extra$ with $\ell(G') = \min(\ell, \ell(G))$, and the result is G' applied to the images of the kept arguments and of *extra* (Example 7.23);

   (c) if $\bar{b}$ is no pattern: each b_j is walked on its own in a soft context, binding nothing; it is *kept*, as the walk rebuilt it (its a_i turned into bound variables), if the walk succeeds, *dropped* if it fails by scope on a rigid path (it holds a forbidden eigenvariable that every instance keeps), and any other trouble suspends the whole equation. Then, if nothing was dropped and G needs no lowering, $G\,\bar{b}$ stays as it is; otherwise, if some b_j is functional, suspend; else prune and lower as in (b).

(7) *Flexible $G\,\bar{b}$ in a soft context*: $G = F$ or $\ell(G) > \ell$ is trouble; otherwise the arguments are walked in a weak context and kept as they are. Inside an argument, the application of a variable bound inside that argument is treated as flexible too.

Inside an argument of a non-pattern, case (6c) reinterprets every failure: a scope failure of case (1) in a soft, non-weak context drops the argument, and any other $\bot$ or *trouble* suspends the equation.

Tutorial: Reading the contexts

The context answers one question about the subterm being walked: *will every instance of the right-hand side still contain it?* At `top` and below rigid constructors (`rigid`, `firm`) the answer is yes, because no substitution can remove a rigid constructor; so a forbidden eigenvariable there is a definite failure, and a variable there can be lowered or pruned without losing solutions. Inside an argument of a non-pattern $G\,\bar{b}$ the answer is "not necessarily", since G may become a function that ignores that argument; so nothing is bound there and doubts turn into a suspension. `firm` and `rigid` differ only in the occurs test (6a): below a constant, an eigenvariable outside $\bar{a}$ or a pair, F's instance cannot be a projection, so an inner $F\,\bar{b}$ with rigid arguments is fatal whatever their number; below a bound variable or an a_i at the head, the number matters. The next two tutorials give the reasons in detail.

Tutorial: Strict and non-strict occurrences

Pruning removes an argument from a variable; lowering makes a variable forget the eigenvariables it could see. Both are sound only on *strict* occurrences, those not inside

an argument of a flexible term that is no pattern (Dowek, Hardin, Kirchner and Pfenning [8]). The reason is that the flexible head G may later turn out to drop that argument. Take $\exists q\, \exists G\, \forall y\, \exists Y.\ q = G\,Y \wedge Y = y \wedge G = \lambda a.\, k$: it has the solution $q = k$. If elimination of q lowered Y inside the argument of $G\,Y$, the later $Y = y$ would fail by scope and the answer would be lost in that goal order, which breaks the commutativity of conjunction. So nothing is bound inside such an argument: a variable to lower (case (4)) or a flexible term to prune (case (7)) is trouble there, and trouble suspends the equation. The exception of case (6c) is the one pruning that *is* strict: an argument that holds, on a rigid path, an eigenvariable that F may not see is held by every instance of that argument, so G must drop it. This makes $\exists q\, \exists F\, \forall x.\ q = F\,x\,x$ solvable as $F := \lambda a\, b.\, F'$, $q := F'$. Even that pruning waits when an argument is, or may become, an abstraction: G could apply it to another argument and drop that one (Section 12.3.8 shows $\exists q\, \forall x\, \exists G.\ q = G\,(\lambda w.\, k) \wedge G = \lambda y.\, y\,x$, solved in both orders).

Tutorial: When an occurrence of F is fatal

Case (6a) is Nipkow's size argument [48]: $F\,\bar{b}$ inside the body of $F\,\bar{a}$ makes the equation unsolvable only if no instance of $F\,\bar{b}$ can be smaller than the instance of $F\,\bar{a}$. That holds when the b_j are rigid (no substitution makes them reduce) and are as many as $\bar{a}$ plus the binders on top ($F\,\bar{a} = \lambda\bar{y}.\, t'$ is $F\,\bar{a}\,\bar{y} = t'$ by η), as in $F = \lambda x.\, g\,(F\,x)$ and Example 7.24. The count does not matter when the root of the body is a constant, an eigenvariable not in $\bar{a}$ or a pair (context `firm`): no instance $\lambda\bar{x}.M$ of F can then be a projection, so M has that root and $F\,\bar{b}$ sits strictly inside it. So the η-short $F = \lambda x\, y.\, g\,(F\,y)$ fails as Miller and Nipkow fail it. Otherwise the instance may shrink and the equation is outside L_λ: it is suspended. For instance $\forall x.\, F\,x = x\,(F\,(\lambda z.z))$ holds with $F := \lambda a.\, a\,c$, and $\forall x\, \forall y.\, F\,x\,y = x\,(F\,y)$ holds with $F := \lambda w.w$.

12.3.5 The eager scope check

Definition 12.7 (Check)**.** For a binding $X := t$ that micro has just made, **Check**$(X := t)$ is $\mathrm{Elim}(\ell(X), -, \langle\rangle, t)$ with no head to eliminate (`μkanren-enforce`). If it answers $\bot$, `=°` fails. If it suspends, the binding is *taken back*: `μkanren-state-forget` unbinds X and the equation $X \doteq t$ is pushed on E. Otherwise its own bindings (lowerings, prunings) are kept.

Check is NJK's binding rule with its lowering (Definition A.27): "t contains no constant with tag $> i$; change to i the tags of the variables in t whose tag is $> i$" [43, p. 16], extended from variables to flexible terms by pruning. Taking a binding back is what Teyjus does with a binding whose check depends on a non-pattern. Its effect is an invariant:

Proposition 12.8 (No captured eigenvariable)**.** *After every* `=°`*, no binding $X := t$ in the substitution has an eigenvariable of level $> \ell(X)$ in the value of t.*

Argument. Every binding made by micro goes through **Check** unless the fast paths of Section 12.3.6 prove it unnecessary; **Check** either fails, or rewrites the variables of t so that it

holds, or unbinds X. The bindings that **Check** itself makes are lowerings and prunings, which by construction respect the levels. The remaining gap is host code (`project°` and `set!`), discussed in Chapter 14. □

12.3.6 Fast paths

Check on every binding would be costly on first-order search, so three cheap tests come first.

- **No `∀°` yet.** While no `∀°` has run, the depth is 0 and L is empty (`µkanren-levels?` is false): every level is 0, no check can fail, none runs.
- **Plain terms.** `µkanren-plain?` is an allocation-free walk, like micro's occurs check. It accepts t when t holds no eigenvariable or variable of a level above $\ell(X)$ and no application with a variable or abstraction head (which Elim might prune, lower or reduce). Only otherwise does Elim run.
- **Top-level variables.** A variable of level *top*, the highest level reached, can see every eigenvariable and every variable there is, so its bindings are skipped. The bindings made under ξ or η are checked anyway, by `µkanren-scoped`.

The protocol's `atom?` hook belongs here as well: it tells micro's occurs check that an eigenvariable or a bound variable holds no variable, so micro does not rebuild it as a slot list. On the type inference of the Church numeral 400 these steps took the time from 463 ms to 330 ms (`plain?`), 311 ms (the `top` skip) and 254 ms (`atom?`).

Pitfall: The first `∀°` switches the check on for good

The first fast path is about the whole branch, not the current scope: once a `∀°` has run, levels exist and every binding below `top` is checked, even after the `∀°` has exited. First-order search that follows a `∀°` pays for it, measured at +66% on `append°` backwards and +127% on open `append°`. Skipping the check at depth 0 would be unsound against host escapes; the fix planned is one walk for micro's occurs check and the scope check together (Chapter 14).

12.3.7 Wake-up and termination

When `=°` binds something and E is not empty, `µkanren-settle` retries every suspended item, oldest first (`µkanren-wake`). An item is retried by its kind:

- an equation $u \doteq v$: if one side is now an unbound variable X and the other no variable, X is bound *only once its check passes* (`µkanren-retry/bind`); otherwise the equation is unified again, which may solve it, fail, or suspend it again;
- a type on a flexible term: decided if the head is now known, else suspended again;
- an absento: decided, kept, or dropped when levels entail it (Section 12.5).

The new bindings of a round are checked like those of `=°`, and rounds repeat while a round binds. A constraint still undecided is suspended again *without* binding anything.

Proposition 12.9 (Wake-up terminates)**.** *Suppose every hnf computed during the wake-up terminates. Then the loop of* `µkanren-settle` *terminates.*

Argument (a sketch, not a mechanised proof). A round that binds nothing ends the loop, so it suffices that only finitely many rounds bind. A round that binds does so for good: the only binding ever taken back is the one that **Check** has just examined, and a retried $X \doteq t$ binds X only after its check passed, so the retry does not bind and unbind the same variable forever. The variables a round may create come from lowering (case (4)), whose new variable has a strictly smaller level than the one it replaces, and from pruning (cases (6b) and (6c)), whose new variable either has a strictly smaller level (when G is lowered, possibly with *extra* arguments) or the same level and strictly fewer arguments (FF-SAME likewise creates H at the level of F with fewer arguments). So every new variable *replaces* exactly one variable that is bound for good, and is below it in the lexicographic order on (level, number of arguments), which is well founded. A chain of replacements (a variable, the one that replaces it, the one that replaces that, ...) is therefore finite; with finitely many variables at the start, finitely many variables are ever created, each is bound at most once, and only finitely many rounds can bind. This is the decreasing measure the design relies on. One point a full proof would have to settle: the number of arguments is that of the occurrence being pruned, which an untyped term does not fix once and for all. □

The hypothesis on hnf is not a formality: `@` gives full β, and on untyped meta-terms full β may diverge (Chapter 14). On pattern problems no such reduction happens (every contraction is a β_0 renaming), and the rules are Miller's and Nipkow's algorithm with the prefix erased into levels: it terminates, fails only when there is no unifier, and otherwise computes a most general one [34, Thm 7.1, Prop. 7.3], [44, Thm 2]. The suite checks the book's mgus verbatim: $U = \lambda x\, y.\, g\,(V'\, y)$ and $V = \lambda y\, w.\, V'\, y$ for Example 7.23, $U = \lambda x.\, f\,(V'\, x)$ for Example 7.31, and $G, H := \lambda w.\, T$ for $\lambda x\, \lambda y.\, G\, x = \lambda x\, \lambda y.\, H\, y$ (Example 7.2).

Pitfall: Suspended is not solved

Non-patterns are suspended, never enumerated: there is no Huet search [MN p. 213]. An answer may therefore be conditional on residual equations that are jointly unsatisfiable. $F\, a = b$ and $F\, a = c$ (a a constant, so no pattern) are both suspended, and the answer is printed with the two residuals; only a binding of F reveals the conflict:

```
(°->list #f (fresh° (q F) (=° q F) (=° (@ F 'a) 'b) (=° (@ F 'a) 'c)))
(°->list/datum (fresh° (q F) (=° q F) (=° (@ F 'a) 'b) (=° (@ F 'a) 'c)
                       (=° F (ƛ (z) 'b))))
```

```
((λ (α) (assert (≐ (@ α 'a) 'b)) (assert (≐ (@ α 'a) 'c)) α))
()
```

This is floundering reported, not detected. The residuals are the honest output: they say exactly which equations the search did not decide.

12.3.8 Worked traces

Each trace below poses a problem, follows the rules, and ends with the module's actual output. The probe used here prints the depth, the levels of the given variables and E:

```
(define ((probe label . vars) s)
  (display label) (display ": depth ") (display (μkanren-λ-depth s))
  (display ", levels ") (write (map (λ (v) (μkanren-λ-level v s)) vars))
  (display ", E ") (write (μkanren-λ-delayed s)) (newline)
  (✓° s))
```

Example 12.10 (Warm-up: flexible–rigid, and ξ)**.** Two equations that touch no pruning or lowering: $\exists q\, \forall x\, \forall y.\ q\, x\, y \;=\; g\, y\, x$, and $\exists q.\ \lambda x.\,(g\ x) \;=\; \lambda y.\,(g\ (q\, y))$, where both $(g\ \ldots)$ are Scheme lists.

```
(°->list/datum (fresh° (q) (∀° (x y) (=° (@ q x y) `(g ,y ,x)))))
(°->list/datum (fresh° (q) (=° (ƛ (x) `(g ,x)) (ƛ (y) `(g ,(@ q y))))))
```

```
((ƛ (x y) (g y x)))
((ƛ (x) x))
```

Trace of the first. $\ell(q) = 0$, $\ell(x) = 1$, $\ell(y) = 2$. The left side is flexible and $\langle x, y\rangle$ is a pattern for q; the right side is the list `(g y x)`, rigid: FR. Elim walks the list in context `firm`: `g` is an atom and stays; y is a_2 and becomes the bound variable of the second binder; x is a_1 and becomes that of the first (case (1)). Nothing was bound on the way, and $q \;:=\; \lambda x\, y.\,(g\ y\ x)$, the binders named after the eigenvariables.

Trace of the second. Both sides are abstractions: ξ opens them with one new eigenvariable c of level ∞ (hint `x`), leaving `(g c)` $\doteq$ `(g (@ q c))`. These are lists, so micro decomposes them and hands $c \doteq q\, c$ back to the hook. $\langle c\rangle$ is a pattern for q ($\infty > 0$), and Elim turns c into the bound variable: $q \;:=\; \lambda x.\, x$, a projection. The binding was made under ξ, so `μkanren-scoped` checks it at once; it holds no eigenvariable and passes.

Example 12.11 (Pruning: problem 8.H [MN p. 223–224], Example 7.23)**.** The problem (Example 7.23) is

$$\exists U\, \exists V\, \forall w\, \forall x\, \forall y.\ f\,(U\, x\, y) = f\,(g\,(V\, y\, w)).$$

```
(°->list/datum (fresh° (q u v) (=° q (list u v))
  (∀° (w x y) (probe "before" u v)
              (=° `(f ,(@ u x y)) `(f (g ,(@ v y w))))
              (probe "after" u v))))
```

```
before: depth 3, levels (0 0), E ()
after: depth 3, levels (0 0), E ()
(((ƛ (x y) (g (@ α y))) (ƛ (y w) (@ α y))))
```

Trace. U, V have level 0 and w, x, y levels 1, 2, 3. Both sides are lists, so micro decomposes them: `f` against `f`, then $U\, x\, y \doteq g\,(V\, y\, w)$ with a λ-record on the left, handed to the hook. The left side is a pattern (x, y distinct, levels $2, 3 > 0$) and the right is rigid: FR, Elim$(0, U, \langle x, y\rangle, g\,(V\, y\, w))$. The root is the list `(g ...)`: case (5), context `firm`. Inside, $V\, y\, w$ is flexible with a pattern for V

(levels $3, 1 > 0$): case (6b). Keep y (it is in $\bar{a}$, it becomes the bound variable of y); drop w (not local, not in $\bar{a}$, and $\ell(w) = 1 > 0$); *extra* is empty since $\ell(V) = 0$ is not above $\ell = 0$. Something was dropped, so prune: $V := \lambda y\, w.\, V'\, y$, $\ell(V') = 0$, and the body of U is $g\,(V'\, y)$. Finally $U := \lambda x\, y.\, g\,(V'\, y)$. Both are the book's mgu, with α for V'. The binders of V are named after the eigenvariables y, w it was applied to.

Example 12.12 (Flexible–flexible)**.** Different heads, $\forall x\, \forall y.\, F\, x\, y = G\, y\, x$, and the same head, $\forall x\, \forall y.\, F\, x\, y = F\, y\, x$:

```
(°->list #f (fresh° (q f g) (=° q (list f g)) (∀° (x y) (=° (@ f x y) (@ g y x)))))
(°->list #f (fresh° (q) (∀° (x y) (=° (@ q x y) (@ q y x)))))
```

```
((λ (α) (cons (ƛ (x y) (@ α y x)) (cons α '()))))
((λ (α) (ƛ (x y) α)))
```

Trace. In the first, both sides are patterns with different heads: FR on F. In Elim, $G\, y\, x$ is case (6b) with both arguments in $\bar{a}$, kept; $\ell(G) = 0$ is not above $\ell(F) = 0$, so nothing is pruned or lowered and G stays unbound: $F := \lambda x\, y.\, G\, y\, x$. That is the book's mgu for different heads [MN p. 222] with the common variables listed as y, x: $F := \lambda x\, \lambda y.\, H\, y\, x$ and $G := \lambda y\, \lambda x.\, H\, y\, x$, which is H by η, up to the name of H. In the second, FF-SAME: the positions where $x\, y$ and $y\, x$ agree are none, so $F := \lambda x\, y.\, H$ with H new.

Example 12.13 (A non-pattern, suspended and woken)**.** $\exists F\, \forall x.\, F\,(g\, x) = h\,(g\, x)$ is not in L_λ: the argument $g\, x$ is not an eigenvariable. It has two solutions, $F := \lambda z.\, h\, z$ and $F := \lambda z.\, h\,(g\, x)$, but the second is illegal here because F is quantified outside x. The module neither guesses nor fails:

```
(°->list #f (fresh° (q) (∀° (x) (=° (@ q `(g ,x)) `(h (g ,x))) (probe "inside" q))
                        (probe "after ∀" q)))
(°->list/datum (fresh° (q) (∀° (x) (=° (@ q `(g ,x)) `(h (g ,x))))
                           (probe "before F :=" q)
                           (=° q (ƛ (z) `(h ,z)))
                           (probe "after F :=" q)))
(°->list/datum (fresh° (q) (∀° (x) (=° (@ q `(g ,x)) `(h (g ,x))))
                           (=° q (ƛ (z) 'k))))
(°->list/datum (fresh° (q) (=° q (ƛ (z) `(h ,z)))
                           (∀° (x) (=° (@ q `(g ,x)) `(h (g ,x))))))
```

```
inside: depth 1, levels (0), E ((= (@ _0 (g (eigen x))) (h (g (eigen x)))))
after ∀: depth 0, levels (0), E ((= (ƛ (x) (@ _0 (g x))) (ƛ (x) (h (g x)))))
((λ (α)
    (assert
      (≐ (ƛ (x) (@ α (cons 'g (cons x '()))))
         (ƛ (x) (cons 'h (cons (cons 'g (cons x '())) '()))))))
    α))
before F :=: depth 0, levels (0), E ((= (ƛ (x) (@ _0 (g x))) (ƛ (x) (h (g x)))))
after F :=: depth 0, levels (0), E ()
((ƛ (z) (h z)))
()
((ƛ (z) (h z)))
```

Trace. Inside the scope the left side is flexible but no pattern and the right is rigid: DELAY, and E holds the equation, with the eigenvariable x in it. At the exit of `∀° (x)` the equation mentions x, so it is *discharged* (Section 12.4.2): no variable in it has a level $\geq \ell(x) = 1$, so nothing is raised, and the equation is closed into $\lambda x.\, F\,(g\,x) \doteq \lambda x.\, h\,(g\,x)$, equivalent by ξ. It is retried at once, opens x again as an ∞-level name, is still no pattern, and is suspended again. The run ends with it, and the answer carries it as a residual `(assert (≐ ...))`. In the second run the binding $F := \lambda z.\, h\,z$ wakes it: ξ, then β turns the left side into $h\,(g\,c)$, which equals the right side, and E is empty. In the third run $F := \lambda z.\, k$ makes the retry compare k with $h\,(g\,c)$: a clash, so `=°` fails and there is no answer. In the fourth run the binding comes first and the equation is a plain β-redex, solved with no suspension. The answer is the same in both goal orders.

Example 12.14 (A binding taken back)**.** $\exists q\, \exists G\, \exists H\, \forall y.\ q = G\,(H\,y)$. micro binds $q := G\,(H\,y)$ at once, as VAR. **Check** at level $\ell(q) = 0$ then meets $G\,(H\,y)$, a flexible term whose argument $H\,y$ is no eigenvariable: case (6c), walking $H\,y$ in a soft context. There y (level 1, not allowed for a level-0 variable) sits below the flexible head H, a weak context: trouble, not ⊥, since H may drop y, and G may drop all of $H\,y$. So the check suspends, the binding is taken back, and $q \doteq G\,(H\,y)$ waits in E:

```
(°->list #f (fresh° (q G H) (∀° (y) (=° q (@ G (@ H y))) (probe "inside" q G H))))
(°->list/datum (fresh° (q G H) (∀° (y) (=° q (@ G (@ H y)))) (=° G (ƛ (a) 'k))))
```

```
inside: depth 1, levels (0 0 0), E ((= _0 (@ _1 (@ _2 (eigen y)))))
((λ (α β γ) (assert (≐ (ƛ (y) α) (ƛ (y) (@ β (@ γ y))))) α))
(k)
```

At the exit of the scope the equation is closed over y and reported, q unbound, which is all one can say. With $G := \lambda a.\, k$ the retry binds $q := k$, a plain term, and the residual is gone. The same happens in every order of the goals.

Example 12.15 (Occurs and scope failures, and a soft occurrence)**.**

```
(°->list #f (fresh° (q) (=° q (list q))))                          ; X = f X
(°->list #f (fresh° (q) (∀° (w x y) (=° (@ q x y) `(g ,(@ q y w))))))  ; 8.I
(°->list #f (fresh° (q) (∀° (w x y) (=° (@ q x y) `(g ,w)))))          ; 8.J
(°->list #f (fresh° (F) (=° F (ƛ (x) `(g ,(@ F x))))))         ; F = λx.g (F x)
(°->list #f (fresh° (q F G) (=° q (list F G))
              (∀° (x) (=° (@ F x) (@ G (@ F x))))))
```

```
()
()
()
()
((λ (α β) (assert (≐ α (ƛ (x) (@ β (@ α x))))) (cons α (cons β '())))
```

The first is micro's occurs check, unchanged. In 8.I, $U\,y\,w$ occurs below the list `(g ...)`, a `firm` context, and its arguments are rigid: case (6a), ⊥ (Example 7.24); it also has as many arguments as $U\,x\,y$, so the count alone would fail it. In 8.J, w has level 1, is not an argument of U, and U has level 0: case (1), ⊥ by scope (Example 7.25). The fourth fails micro's occurs check, the `occurs` hook passes it to Elim with no arguments, and $F\,x$ below `g` is case (6a) again. The last is the *soft* occurs check: $F\,x$ occurs inside the argument of G, which may be $\lambda z.z$, so the equation is suspended (and is closed over x at the exit of the scope) instead of failing.

Example 12.16 (An argument that may be an abstraction). $\exists q\,\forall x\,\exists G.\, q = G\,(\lambda w.\, k) \wedge G = \lambda y.\, y\, x$ in both orders:

```
(°->list/datum (fresh° (q) (∀° (x) (fresh° (G) (=° q (@ G (ϰ (w) 'k)))
                                                (=° G (ϰ (y) (@ y x)))))))
(°->list/datum (fresh° (q) (∀° (x) (fresh° (G) (=° G (ϰ (y) (@ y x)))
                                                (=° q (@ G (ϰ (w) 'k)))))))
```

```
(k)
(k)
```

In the first order, **Check** of $q := G\,(\lambda w.k)$ at level 0 finds G of level 1, which must be lowered; but its argument is an abstraction that G may apply to x and so drop G's own x. Lowering G would not be strict, so case (6c) suspends. Then $G := \lambda y.\, y\, x$ wakes the equation, β gives $(\lambda w.k)\, x = k$, and $q := k$. In the other order β happens at once. A version that lowered G here lost the answer in the first order (third adversarial review).

12.4 ∀° and ⊃°

The book's goal-directed search for hereditary Harrop formulas consists of the rules AND, OR, INSTAN, GENERIC, AUGMENT and BACKCHAIN [MN p. 77], [MN p. 82], [MN p. 124] (Fig. 3.1; the names are those of Fig. A.1). Uniform proofs are complete for intuitionistic provability of these formulas [37] (Theorem 3.11), so implementing the rules faithfully *is* the semantics. AND, OR and ∃ are micro's `and°`, `or°` (interleaving) and `fresh°`. This section gives the other three as the module implements them.

12.4.1 GENERIC

$$\frac{\Sigma \cup \{c\}; \mathscr{P} \longrightarrow G[c/x]}{\Sigma; \mathscr{P} \longrightarrow \forall x.\, G}\ \textsc{generic} \quad (c \notin \Sigma)$$

`(∀° (x) g ...)` expands into `(∀ª 'x (λ (x) (and° g ...)))`, and `(∀° (x y) g ...)` into one `∀ª` per name, nested, so `(∀° (w x y) ...)` gives w, x, y the levels $d+1, d+2, d+3$. The Scheme variable `x` is bound to the eigenvariable, as `ϰ` binds its placeholder. `∀ª` takes a state s whose λ record has depth d and does the following.

1. Record the level d of every variable created so far (pad L up to the variable count).
2. Run the body `(f c)` for a new eigenvariable c of level $d+1$, in the state with depth $d+1$ and $top := \max(top, d+1)$.
3. For each answer state s_1 of the body: record the level $d+1$ of the variables created inside (pad L again), set the depth back to d, and *discharge* the constraints on c (Section 12.4.2); if discharge fails, drop the answer.

```
(define ((∀ª name f) s)
  (let* ((x  (μkanren-hh s))
         (d  (μkanren-hh-depth x))
         (x* (μkanren-hh-with (μkanren-hh-pad x (μkanren-state-vars-count s))
                              depth: (add1 d) top: (max (add1 d) (μkanren-hh-top x))))
         (c  (μkanren-eigen* name (add1 d))))
    (append-map§
      (λ (s1)
        (let1 (s2 (μkanren-with s1
                    (μkanren-hh-with (μkanren-hh-pad (μkanren-hh s1)
                                                     (μkanren-state-vars-count s1))
                                     depth: d)))
          (match/first (μkanren-discharge s2 c d)
            (#f (✗° s2))
            (,s3 (✓° s3)))))
      (δ ((f c) (μkanren-with s x*))))))
```

The shape is that of micro's `and`ª (Chapter 10): the body's stream is delayed with `δ`, and `append-map§` post-processes it answer by answer, so the restoring step runs on every answer, however late the stream produces it. Steps 1 and 3 pad L because a variable's level is *implicit* (the current depth) until L is extended past it: padding fixes it before the depth changes.

The signature Σ is not carried. What carries the quantifier structure is the level of each variable: a variable created at depth d stands for an $\exists$ after the first d universals of its branch (Miller's mixed prefix [34], linearised by creation), and **Check** forbids exactly the substitutions that would violate that prefix. NJK prove this tagging sound and complete (Proposition A.28, [43, Prop. 4.1]).

Tutorial: Why the depth must be restored on every answer

A creation timestamp would tell a variable created before c from one created after it, and one might think that suffices: older variables may not see c. But a variable created *after* the `∀°` has exited may not see c either. With the depth restored on every answer, such a variable gets level $d < \ell(c)$, so it cannot take c. This is λProlog's universe register [43]. The two classic cases, [MN p. 85]:

```
(list (°->list #f (fresh° (q) (∀° (x) (=° q x))))         ; ∃Y∀x. Y = x
      (°->list #f (fresh° (q) (∀° (x) (fresh° (y) (=° y x)))
                              (=° q 'ok))))                ; ∀x∃Y. Y = x
```

```
(() ((λ () 'ok)))
```

In the first, q has level 0 and x level 1: **Check** fails by case (1). In the second, y is created inside, at level 1, and may take x.

Example 12.17 (Lowering, seen from the state)**.** A variable created inside the scope and placed inside the value of an outer one must forget the scope:

```
(define ((probe . vars) s)
  (for-each (λ (v) (let1 (w (μkanren-state-find v s))
```

```
                    (write v) (display " ↦ ") (write w)
                    (when (µkanren-var? w)
                      (display " level ") (display (µkanren-λ-level w s)))
                    (display "; ")))
            vars)
  (newline) (✓° s))
(°->list #f (fresh° (A) (∀° (x) (fresh° (B) (probe A B)
                                          (=° A `(arr ,B c))
                                          (probe A B)))))
```

```
_0 ↦ _0 level 0; _1 ↦ _1 level 1;
_0 ↦ (arr _1 c); _1 ↦ _2 level 0;
((λ (α) (cons 'arr (cons α (cons 'c '())))) _0)
```

A (`_0`) has level 0 and B (`_1`) level 1. Binding $A := (\texttt{arr}\ B\ c)$ triggers **Check** at level 0, which meets the unbound B of level $1 > 0$: case (4), lower. A new variable `_2` of level 0 is created and $B := \texttt{_2}$; **Check** eliminates no head and has no arguments $\bar{a}$ to raise over, so the raising part is empty. This is NJK's "change to i the tags of the variables in t whose tag is $> i$", done by a binding instead of a mutation, so that it is undone by backtracking for free.

12.4.2 Discharge at the exit of a scope

When `∀°` exits, some constraints may still mention c: suspended equations, generic disequalities of N, absentos of micro's T. A residual that mentions c outside its scope would be meaningless, and the printer must never show an eigenvariable free. `µkanren-discharge` closes them, back at depth d:

1. **Raise.** Every unbound variable Y of the equations and disequalities to be closed with $\ell(Y) \geq \ell(c)$ is raised: $Y := Y'\,c$ with $\ell(Y') = d$. Inside the scope Y could depend on c; Y' lives outside it, so the dependency becomes an explicit argument: $\forall c\,\exists Y$ is turned into $\exists Y'\,\forall c$ with $Y = Y'\,c$. This is Miller's raising [MN p. 107] (Proposition 4.14), a bijection on solutions.

2. **Close.** A suspended equation $u \doteq v$ that mentions c stands for $\forall c\,(u = v)$ and becomes $\lambda c.u \doteq \lambda c.v$, equivalent by ξ. A generic disequality is closed the same way, under GENERIC's reading of c as a new name. An absento of T whose *term* mentions c is closed too, since absence in $\lambda c.u$ is absence in u. An absento whose *datum* mentions c moves to E as `(absent d u)`, unless the levels entail it: if u does not hold c and all its variables have levels below $\ell(c)$, no instance of u can ever hold c, and the constraint is dropped.

3. **Retry.** The raise bindings go through `µkanren-post-unify`, which wakes E, so the closed constraints are retried. With no raise, the closed equations are retried anyway.

4. **Repeat.** A raise may bring c into more constraints (an absento of T on a raised Y now watches $Y'\,c$; a prefix of D on Y moves to N holding c). Discharge repeats until no raise is left.

The constraints that cannot be closed by an abstraction (a type on $F\,x$; an absento whose datum is c on a term that may still hold c) keep c and print under a ∀ (Section 12.6).

Example 12.18 (Discharge: raising, closing, retrying).

```
;; inspect a state: bindings, constraints, and the levels of unbound variables
(define (µkanren-state-inspect s)
  (let ((vars (µkanren-state-vars s)))
    (for-each (λ (c) (display "constraint: ") (write (µkanren-constraint-goal c)) (newline))
              (µkanren-state-constraints s))
    (for-each (λ (v) (display "bind: ") (write v) (display " ↦ ")
                     (write (µkanren-state-find/value v s)) (newline))
              vars)
    (for-each (λ (v) (display "unbound: ") (write v)
                     (display ", level ") (display (µkanren-λ-level v s)) (newline))
              vars)
    (for-each (λ (label)
                (display label) (display ": depth ") (display (µkanren-λ-depth s))
                (for-each (λ (v) (display ", ") (write v) (display " ↦ ")
                                 (write (µkanren-state-find/value v s))
                                 (let1 (w (µkanren-state-find v s))
                                   (when (µkanren-var? w)
                                     (display " level ") (display (µkanren-λ-level w s)))))
                          vars)
                (display ", E ") (write (µkanren-λ-delayed s)) (newline))
              (list "inside" "outside"))))
(°->list #f (fresh° (q) (∀° (a) (fresh° (f) (=° (@ f a) `(g ,a ,a))
                                            (probe "inside" f)))
                        (=° q 'ok)))
(°->list #f (fresh° (q) (∀° (x) (=° (@ q x) `(g ,q)) (probe "inside" q))))
```

```
inside: depth 1, _1 ↦ _1 level 1, E ((= (@ _1 (eigen a)) (g (eigen a) (eigen a))))
((λ (α)
    (assert (≐ (ϡ (a) (@ α a a)) (ϡ (a) (cons 'g (cons a (cons a '()))))))
    'ok))
inside: depth 1, _0 ↦ _0 level 0, E ((= (@ _0 (eigen x)) (g _0)))
()
```

In the first, f is created inside the scope of a (level 1), so $f\,a$ is no pattern (the book's 8.A, four unifiers and no mgu, [MN p. 213]) and is suspended. At the exit, f has level $1 \geq \ell(a)$: it is raised, $f := f'\,a$ with $\ell(f') = 0$, and the equation closes into $\lambda a.\, f'\,a\,a \doteq \lambda a.\, g\,a\,a$. That is still no pattern (the argument a is repeated), so it is reported. In the second, $F\,x \doteq g\,F$ is a pattern equation, but F itself occurs with no argument in its own body, case (4) with $\bar{a} \neq \langle\rangle$: suspend. At the exit nothing needs raising ($\ell(F) = 0$), the equation closes into $\lambda x.\, F\,x \doteq \lambda x.\, g\,F$, which by η is $F \doteq \lambda x.\, g\,F$, and the retry fails the occurs check: no answer, as it must be.

12.4.3 AUGMENT

$$\frac{\Sigma;\mathscr{P} \cup \{D\} \longrightarrow G}{\Sigma;\mathscr{P} \longrightarrow D \supset G}\ \textsc{augment}$$

`(⊃° (D ...) g ...)` expands into `⊃ª`, which pushes the clauses on Γ for the body and gives the old Γ back to every answer of the body:

```
(define ((⊃ª clauses g) s)
  (let* ((x (µkanren-hh s))
         (Γ (µkanren-hh-Γ x))
         (Γ* (fold-right
               (λ (c Γ*)
                 (let1 (old (cond ((assq (car c) Γ*) => cdr) (else '())))
                   (cons (cons (car c) (cons (cdr c) old))
                         (if (null? old) Γ* (alist-delete (car c) Γ* eq?)))))
```

```
              Γ clauses)))
    (append-map§
      (λ (s1) (✓° (μkanren-with s1 (μkanren-hh-with (μkanren-hh s1) Γ: Γ))))
      (δ (g (μkanren-with s (μkanren-hh-with x Γ: Γ*)))))))
```

A clause is a pair `(relation . procedure)`: the relation is the dynamic relation's own procedure, used as an `eq?` key, and the procedure maps the list of a call's arguments to the goal that the clause makes of them. Since Γ lives in the state, `or°` branches cannot interfere, and restoring it on every answer is the stack discipline of the book [MN p. 78], [MN p. 83]. Assuming a clause for a relation already in Γ replaces that relation's entry by one with the new clause in front of the old ones, so a lookup costs the number of distinct assumed relations, not the number of clauses. The `fold-right` puts the first clause of a list on top.

12.4.4 BACKCHAIN and `define-dynamic-relation`

In λProlog every predicate can be extended by an assumption. Here only relations made by `define-dynamic-relation` can:

```
(define-syntax define-dynamic-relation
  (syntax-rules ()
    ((_ (name arg ...))
     (define name
       (μkanren-dynamic
         (λ (arg ...) (λ (s) (δ ((μkanren-hyps° name (list arg ...) #f) s)))))))
    ((_ (name arg ...) g ...)
     (define name
       (μkanren-dynamic
         (λ (arg ...)
           (λ (s) (δ ((μkanren-hyps° name (list arg ...) (and° g ...)) s)))))))))

(define ((μkanren-hyps° key args static) s)
  (let* ((gs (map (λ (c) (c args))
                  (cond ((assq key (μkanren-hh-Γ (μkanren-hh s))) => cdr) (else '()))))
         (gs (if static (append gs (list static)) gs)))
    (if (null? gs) '() ((reduce-right orª #f gs) s))))
```

A call disjoins, in one flat `or°`, the assumed clauses for the relation, *most recent first*, and then the static body. If $C_1, \ldots, C_m$ are the clauses assumed for r in the current Γ, in the order they were assumed, and B_r is the static body of r (absent when it has none), then

$$r\,\bar{t} \;\rightsquigarrow\; C_m(\bar{t}) \;\vee\; \ldots \;\vee\; C_1(\bar{t}) \;\vee\; B_r(\bar{t}),$$

where $C_i(\bar{t})$ unifies $\bar{t}$ with the head of C_i, its own `∀` variables renamed and its free variables shared (see the clause syntax below), and runs its body. This is BACKCHAIN (Fig. 3.1) with the choice of clause made by `or°` instead of by a selection rule; a relation with neither hypotheses nor static body fails. When every clause answers at once, the answers come in λProlog's order; otherwise interleaving decides. The relation is tagged with CHICKEN's `extend-procedure`, so `μkanren-dynamic?` is an $O(1)$ test and no global registry is needed. Assuming a clause for a relation that is not dynamic is an error, not a silent no-op:

```
(define-dynamic-relation (n° x) (=° x 1))
(list (°->list/datum (fresh° (q) (⊃° ((n° 2)) (⊃° ((n° 3)) (n° q)))))
      (°->list/datum (fresh° (q) (⊃° ((n° 2) (n° 3)) (n° q)))))
(define (error-message thunk)     ; the message of the error `thunk` raises
  (handle-exceptions exn ((condition-property-accessor 'exn 'message #f) exn)
    (begin (thunk) #f)))
(define-relation (r° x) (=° x 1))
(error-message (λ () (°->list #f (fresh° (q) (⊃° ((r° 2)) (r° q))))))
```

```
((3 2 1) (2 3 1))
"not a dynamic relation, see define-dynamic-relation"
```

The two orders are the book's [MN p. 78]: `p 2 => p 3 => p X` gives 3, 2, 1, and `(p 2 & p 3) => p X` gives 2, 3, 1. In the first query the inner `⊃°` runs last, so Γ maps `n°` to the clauses for 3 and 2, in this order, and the call disjoins $q = 3$, $q = 2$ and the static $q = 1$. In the second, `fold-right` puts the first clause of the list on top, so the clause for 2 comes before the one for 3.

12.4.5 Clause syntax

clause D	meaning and expansion
`(rel t ...)`	a fact; `(λ (args) (fresh° (v ...) (=° args (list t ...))))`
`((rel t ...) ← g ...)`	a rule, infix
`(← (rel t ...) g ...)`	the same rule, prefix; rewritten to the infix form
`(∀ (w ...) D)`	D with its own variables `w ...`, accumulated into the `fresh°` of the expansion

A rule expands to `(μkanren-clause* rel (λ (args) (fresh° (v ...) (=° args (list t ...)) g ...)))`, and `μkanren-clause*` raises the error above when `rel` is not dynamic. Two kinds of variables appear in a clause, as in the book [MN p. 86–88]:

- its *free* logic variables are those of the closure: shared with the context, and among all uses of the clause;
- its *own* `∀` variables are made by the `fresh°` inside the procedure, so they are renamed at each use, at the current level. That is NJK's "new tagged instance" of a clause [43, p. 14].

```
(define-dynamic-relation (h° x))
(list (°->list/datum (fresh° (q y) (⊃° ((h° `(,y ,y))) (h° '(1 1)) (h° '(2 2)))))
      (°->list/datum (fresh° (q) (⊃° ((∀ (z) (h° `(,z ,z)))) (h° '(1 1)) (h° '(2 2)))
                                  (=° q 'ok))))
```

```
(() (ok))
```

With the shared y, the first use binds $y := 1$ and the second use cannot match `(2 2)`; with the clause's own z, each use gets a new z.

Example 12.19 (Levels reach into clauses [43, p. 16])**.** With the program $\forall z\,(q\,z \supset p\,(d\,z))$, the goal $\exists x\,\forall y\,(q\,y \supset p\,x)$ must fail and $\forall y\,\exists x\,(q\,y \supset p\,x)$ must succeed (Example A.29):

```
(define-dynamic-relation (q° z))
(define-dynamic-relation (p° x) (fresh° (z) (=° x `(d ,z)) (q° z)))
(list (°->list #f (fresh° (x) (∀° (y) (⊃° ((q° y)) (p° x)))))
      (°->list #f (fresh° (r) (∀° (y) (fresh° (x) (⊃° ((q° y)) (p° x)))) (=° r 'ok))))
```

```
(() ((λ () 'ok)))
```

In the first goal, x has level 0. Backchaining on `p°` creates the clause variable z inside the scope of y (level 1) and binds $x := (d\ z)$; **Check** lowers z to level 0. The hypothesis `(q° y)` then asks $z = y$, a level-0 variable against a level-1 eigenvariable: scope failure. Without lowering, z would keep level 1 and take y, and $x = (d\ y)$ would leak y out of its scope.

12.4.6 Worked example: type inference

The book's type inference for untyped λ-terms (Section 6.5.4, [MN p. 190–191]) uses all three rules at once:

```
typeof (app M N) A :- typeof M (arr B A), typeof N B.
typeof (abs M) (arr A B) :- pi x\ typeof x A => typeof (M x) B.
```

In the module's syntax (`test/programs.scm` of `doc/hol-kanren/prototype/`):

```
(define-dynamic-relation (typeof° m t)
  (cond°
    ((fresh° (M N B) (=° m `(app ,M ,N)) (typeof° M `(arr ,B ,t)) (typeof° N B)))
    ((fresh° (R A B) (=° m `(abs ,R)) (=° t `(arr ,A ,B))
       (∀° (x) (⊃° ((typeof° x A)) (typeof° (@ R x) B)))))))
```

The object term `abs x\ M` is `` `(abs ,(ƛ (x) M)) ``; `(M x)` is `(@ R x)`, a β-redex once R is bound. The answers for S, I and $\lambda x.\,x\,x$ (from `examples.out` in the same directory):

```
(((arr (arr α (arr β γ)) (arr (arr α β) (arr α γ)))) ((arr α α)) ())
```

To see the rules at work, here is the same program with a trace goal that prints the depth and the walked arguments, with the level of each unbound variable after `@`. The trace in the static body runs only when the static clauses are tried, after the hypotheses:

```
(define ((trace label . ts) s)
  (display label) (display " [depth ") (display (μkanren-λ-depth s)) (display "]")
  (for-each (λ (t) (display " ") (write (μkanren-state-find/value t s))
                   (let1 (w (μkanren-state-find t s))
                     (when (μkanren-var? w)
                       (display "@") (display (μkanren-λ-level w s)))))
            ts)
  (newline) (✓° s))
(define-dynamic-relation (typeof° m t)
  (trace "static typeof°" m t)
```

```
  (cond°
    ((fresh° (M N B) (=° m `(app ,M ,N)) (trace " app" M N B)
       (typeof° M `(arr ,B ,t)) (typeof° N B)))
    ((fresh° (R A B) (=° m `(abs ,R)) (=° t `(arr ,A ,B)) (trace " abs" R A B)
       (∀° (x) (⊃° ((typeof° x A)) (typeof° (@ R x) B)))))))
(°->list/datum (fresh° (q) (typeof° `(abs ,(ƛ (x) `(abs ,(ƛ (y) x)))) q)))
(°->list/datum (fresh° (q) (typeof° `(abs ,(ƛ (x) `(app ,x ,x))) q)))
```

```
static typeof° [depth 0] (abs (ƛ (x) (abs (ƛ (y) x)))) _0@0
 abs [depth 0] (ƛ (x) (abs (ƛ (y) x))) _2@0 _3@0
static typeof° [depth 1] (abs (ƛ (y) (eigen x))) _3@0
 abs [depth 1] (ƛ (y) (eigen x)) _7@0 _8@0
static typeof° [depth 2] (eigen x) _8@0
((arr α (arr β α)))
static typeof° [depth 0] (abs (ƛ (x) (app x x))) _0@0
 abs [depth 0] (ƛ (x) (app x x)) _2@0 _3@0
static typeof° [depth 1] (app (eigen x) (eigen x)) _3@0
 app [depth 1] (eigen x) (eigen x) _6@1
static typeof° [depth 1] (eigen x) (arr _6 _3)
static typeof° [depth 1] (eigen x) _7@0
()
```

Tutorial: $\lambda x.\lambda y.\, x$, step by step

1. *Depth 0, Γ empty.* The query T is `_0`. No hypothesis for `typeof°`, so the static body runs. The `abs` clause creates R, A, B (`_1`, `_2`, `_3`, level 0), binds $R := \lambda x.\, \texttt{abs}\ (\lambda y.\, x)$ and $T := (\texttt{arr}\ A\ B)$.

2. GENERIC. `∀° (x)` records levels, raises the depth to 1 and makes the eigenvariable x of level 1.

3. AUGMENT. `⊃°` pushes $\langle \texttt{typeof°}, \lambda args.\ args \doteq (x\ A) \rangle$ on Γ.

4. *β.* The goal is `(typeof° (@ R x) B)`; `@` reduces the redex to `abs` $(\lambda y.\, x)$, with x now the eigenvariable, printed `(eigen x)`.

5. BACKCHAIN, depth 1. The hypothesis is tried first: (`abs` …) $\doteq x$ clashes. Then the static body: the `abs` clause creates R', A', B' at level 1 and binds $B := (\texttt{arr}\ A'\ B')$. Here **Check** runs: B has level 0, A' and B' level 1, so both are *lowered*, to new variables `_7`, `_8` of level 0 (the trace shows `_7@0 _8@0`, the walked values). This is right: the type of the inner abstraction is part of the outer type, which lives outside the scope of x.

6. GENERIC, AUGMENT again. Depth 2, eigenvariable y of level 2, Γ now holds two clauses for `typeof°`, $(y\ A')$ on top of $(x\ A)$.

7. BACKCHAIN, depth 2. The goal is `(typeof° x B')`. The most recent hypothesis gives $x \doteq y$, a clash of two eigenvariables. The older one gives $B' \doteq A$: a level-0 variable

bound to another, fine. The static body is tried as well (the last trace line) and fails, since x is neither an application nor an abstraction.

8. *Exits.* Each `∀°` gives its answers back their depth and discharges nothing (no constraint mentions x or y); each `⊃°` gives Γ back. The answer is $T = \texttt{arr}\ A\ (\texttt{arr}\ A'\ A)$, printed `(arr α (arr β α))`.

Tutorial: $\lambda x.\, x\ x$ fails by the occurs check

At depth 1, with the hypothesis `typeof` x A, the goal is `typeof` (`app` x x) B. The `app` clause binds $M, N := x$ and creates its B'' (`_6`) at level 1. The first subgoal `typeof` x (`arr` B'' B) uses the hypothesis: $A := (\texttt{arr}\ B''\ B)$, and since $\ell(A) = 0$, **Check** lowers B'' to the level-0 `_7`. The second subgoal `typeof` x B'' (printed with B'' walked to `_7`) asks $\texttt{_7} \doteq A = (\texttt{arr}\ \texttt{_7}\ B)$: micro's occurs check fails, and since the term holds no λ-term, the `occurs` hook does not intervene. The static attempts fail too, and there is no answer, the book's "no" [MN p. 191].

12.5 micro's stores on λ-terms

λProlog has no disequality, no absento and no type constraints such as `symbol°` (its types are static). (aux kanren micro) has all three (Section 10.5), and every relation built on them must keep working when its arguments are λ-terms. The readings adopted here extend micro's semantics over λProlog's term model: equality is $\alpha\beta\eta$-equality, a flexible term is undecided until its head is known, and an eigenvariable is, by default, a new name.

12.5.1 Disequality: the `diseq` hook and the store N

Definition 12.20 (Trial)**.** The module *claims* a disequality $u \neq v$ when either side, walked, holds a λ-record or an eigenvariable, and also, under the extensional reading once a `∀°` has run, a first-order one (in D an absento of T could absorb it, and absence reads eigenvariables generically). Its *trial* unifies $u \doteq v$ in the current state, with **Check**, and classifies the result (`μkanren-trial`):

`unsat`
: no unifier: the disequality holds for good and is dropped;

`entailed`
: a unifier that binds no variable older than the trial (or binds older variables only to distinct variables the trial itself created) and suspends nothing: $u = v$ already holds, and `≠°` fails;

`kept`
: otherwise: the pair is stored in N as `(reading u . v)` and re-tried after every `=°`.

The variables a trial creates (by pruning, by FF-SAME, by lowering) are *existential inside the negation*: $\lambda x\, y.\, F\, x \neq \lambda x\, y.\, G\, y$ means $\neg\exists H.\ F = \lambda x.H \wedge G = \lambda y.H$. A prefix of micro's D would wrongly store them as free variables, which is one reason N is a separate store.

```
(°->list #f (fresh° (q) (=° q (ƛ (x) x)) (≠° q (ƛ (y) y))))     ; entailed mod α
(°->list #f (fresh° (q) (≠° (@ (ƛ (x) `(g ,x)) 'a) '(g a))))    ; entailed mod β
(°->list #f (fresh° (q) (≠° (ƛ (x) x) (ƛ (x) 'k)) (=° q 'ok)))  ; holds for good
(°->list/datum (fresh° (q F) (=° q F) (≠° (@ F 'a) 'a)
                         (=° F (ƛ (z) z))))                ; woken, violated
(°->list/datum (fresh° (q F) (=° q F) (≠° (@ F 'a) 'a)
                         (=° F (ƛ (z) 'b))))                ; woken, holds
(°->list #f (fresh° (q f g) (=° q (list f g))
              (≠° (ƛ (x y) (@ f x)) (ƛ (x y) (@ g y)))))
```

```
()
()
((λ () 'ok))
()
((ƛ (z) b))
((λ (α β)
    (begin (deny (∃ (_3) (and (equal? α (ƛ (x) _3)) (equal? β (ƛ (y) _3))))))
    (cons α (cons β '()))))
```

Line by line: the first trial succeeds under ξ binding nothing, so it is `entailed` and `≠°` fails; the second never reaches the module, since `@` has already β-reduced its left side to the list `(g a)` and micro's own `≠°` fails on two equal terms; the third trial clashes (x against k under ξ), so the disequality is `unsat` and dropped. In the fourth and fifth the trial $F\, a \doteq a$ is no pattern (a is a constant) and suspends, so the pair is kept in N; the binding of F retries it, and it is violated by $\lambda z.z$ and holds for good with $\lambda z.b$. In the last, the trial is the flexible–flexible problem of Example 12.12 under two binders: it binds the older F and G to $\lambda x.H$ and $\lambda y.H$ with a new H, so the pair is kept, and the answer denies exactly that mgu, with H (`_3`) existential inside the `deny`.

Migration from D to N. A prefix of D born first-order may come to hold a λ-term or an eigenvariable later, at any depth below its variables. The `post` hook moves it to N, as the tuple disequality it stands for, *before* micro re-verifies D. So D never holds a λ-term, and micro's plain trial never decides a λ-equation without the scope check and the reading. The probe prints N, D and E:

```
(define ((probe label) s)
  (display label) (display ": N ") (write (μkanren-λ-disequalities s))
  (display ", D ") (write (μkanren-state-D s))
  (display ", E ") (write (μkanren-λ-delayed s))
  (newline) (✓° s))
(°->list #f (fresh° (q Y) (≠° q (list Y)) (probe "before")
                          (=° Y (ƛ (z) z)) (probe "after")))
```

```
before: N (), D (((_0 _1))), E ()
after: N ((generic (_0) (_1))), D (), E ()
((λ (α) (begin (deny (equal? α (cons (ƛ (z) z) '())))) α))
```

To keep first-order search cheap, the hook looks at no more than 64 nodes of the new bindings to decide whether they may hold a λ-term; past that bound it walks D and A instead (Chapter 14 has the measurements).

Subsumption by A. A trial that binds a variable typed in A to a rigid λ-term or an eigenvariable of which the type's predicate is false has no solution; `symbol°` and `number°` hold of neither. So the disequality is dropped, as micro drops `(≠° q 5)` under `(symbol° q)`:

```
(°->list #f (fresh° (q) (symbol° q) (≠° q (ƛ (x) x))))
```

```
((λ (α) (assert (every (μ v (symbol? v)) (list α))) α))
```

Two readings of ∀. What does $\forall x.\, q \neq x$ mean? λProlog has no ≠, but it has `pi x\ not (q = x)`, which succeeds: x is a new name, distinct from everything. The parameter `μkanren-λ-reading` chooses, and each disequality keeps the reading in force when it was posted.

- `'generic` (the default) reads an eigenvariable as GENERIC's new name, ∇-like, as αKanren's noms [5]. At the exit of the scope the constraint is closed: $\forall x.\, F\, x \neq x$ becomes $F \neq \lambda x.x$, consistently whichever comes first, the binding or the ≠.
- `'extensional` reads an eigenvariable as any term of its universe, as in the miniKanren work on universal quantification [41, 26]. The trial replaces each eigenvariable by a new variable of its level, after raising the variables that may depend on it, and counts as entailed when it binds older variables only to distinct such variables. Then $\forall x.\, q \neq x$ fails, since x could be q.

```
(°->list #f (fresh° (q) (∀° (x) (≠° q x))))
(°->list #f (fresh° (q F) (=° q F) (∀° (x) (≠° (@ F x) x))))
(°->list #f (fresh° (q F) (=° q F) (∀° (x) (≠° (@ F x) x)) (=° F (ƛ (z) z))))
(°->list #f (fresh° (q F) (=° q F) (∀° (x) (≠° (@ F x) x)) (=° F (ƛ (z) 'c))))
(parameterize ((μkanren-λ-reading 'extensional))
  (list (°->list #f (fresh° (q) (∀° (x) (≠° q x))))
        (°->list #f (fresh° (q F) (=° q F) (∀° (x) (≠° (@ F x) x))))
        (°->list #f (fresh° (q F) (=° q F) (∀° (x) (≠° (@ F x) x))
                                  (=° F (ƛ (z) 'c))))))
```

```
((λ (α) α))
((λ (α) (begin (deny (equal? α (ƛ (x) x)))) α))
()
((λ () (ƛ (z) 'c)))
(() ((λ (α) (∀ (x) (begin (deny (≐ (@ α x) x)))) α)) ())
```

Under the generic reading, $\forall x.\, F\, x \neq x$ is the disequality $F \neq \lambda x.x$: violated by $F := \lambda z.z$, satisfied by $F := \lambda z.c$. Under the extensional reading it says that $F\, t \neq t$ for *every* t of the universe, so $F := \lambda z.c$ violates it ($t = c$); since it cannot be closed by an abstraction, it is printed under a `∀`.

Projection. micro drops a disequality over a variable that is out of the answer, since such a variable can always be chosen to keep the sides apart. That fails for a variable inside an argument of a flexible term: $\exists Y.\, G \neq F\,Y$ is $F \neq \lambda z.G$, since F may ignore its argument. Such an entry is kept, and its variable prints as a working variable, read existentially:

```
(°->pretty (fresh° (q) (fresh° (F G Y) (=° q (list F G)) (≠° G (@ F Y)))))
```

```
(((α β) : (≠ β (@ α _3))))
```

12.5.2 Absence: the `parts` hook

`absent°` looks for its datum in the *parts* of a λ-term (`µkanren-λ-parts`), never in binder names or indices. A flexible term has no parts yet: it is `'stuck` and watched whole, like a variable, and decided once its head is known. Below a binder, the parts that mention the binder stay closed ($\lambda x.(a\,.\,b)$ is looked into as $\lambda x.a$ and $\lambda x.b$), so a watched term never has a loose index.

```
(°->list #f (fresh° (q F) (=° q F) (absent° 'c (@ F 'c))))
(°->list/datum (fresh° (q F) (=° q F) (absent° 'c (@ F 'c)) (=° F (ƛ (z) 'b))))
(°->list #f (fresh° (q) (absent° 'x (ƛ (x) `(g ,x))) (=° q 'ok)))
(°->list #f (fresh° (q F) (=° q F) (absent° 'c (ƛ (x) `(,(@ F x) c)))))
(°->list #f (fresh° (q F) (=° q F) (∀° (x) (absent° x (@ F x)))))
(°->list #f (fresh° (q) (∀° (x) (absent° x q))))
```

```
((λ (α) (assert (absent? 'c (@ α 'c))) α))
((ƛ (z) b))
((λ () 'ok))
()
((λ (α) (∀ (x) (assert (absent? x (@ α x)))) α))
((λ (α) α))
```

In order: $c \notin F\,c$ waits; $F := \lambda z.b$ drops the argument, so it holds; the binder name `x` is not the symbol `'x`; c is found at once outside the flexible part; an absento of an eigenvariable on a term that may still hold it keeps x and prints under `∀`; and $\forall x.\ x \notin q$ is entailed by the levels (q has level $0 < 1$, so no instance of q can hold x) and dropped. The datum may itself be a λ-term, or reduce to an atom (`(@ G 'b)` is `a` once $G := \lambda z.a$): it is then unified, modulo βη, as micro unifies a compound datum.

12.5.3 Types: the `tag` hook

A type on a flexible term waits in E as `(tag tag u)`; the `post` hook does the same for a variable of A just bound to one. A rigid λ-term or an eigenvariable is decided by the type's predicate: abstractions and eigenvariables are neither symbols nor numbers, while a type of the user's made with `define-µkanren-tag` may hold of them. micro's types are disjoint, so two different types on the same flexible term fail at once:

```
(°->list #f (fresh° (q F) (=° q (@ F 'a)) (symbol° q)))
(°->list/datum (fresh° (q F) (=° q (@ F 'a)) (symbol° q) (=° F (ƛ (z) z))))
(°->list #f (fresh° (q F) (=° q (@ F 'a)) (symbol° q) (number° q)))
(°->list #f (fresh° (q) (∀° (x) (symbol° x))))
(°->list #f (fresh° (q F) (=° q F) (∀° (x) (symbol° (@ F x)))))
```

```
((λ (α) (assert (every (μ v (symbol? v)) (list (@ α 'a)))) (@ α 'a)))
(a)
()
()
((λ (α) (∀ (x) (assert (every (μ v (symbol? v)) (list (@ α x))))) α))
```

Pitfall: Readings are per constraint, and only for ≠

The extensional reading applies to `≠°` only; `absent°` and the types always read an eigen-variable as GENERIC does. A disequality keeps the reading in force when it was posted; a first-order disequality that migrates from D to N later takes the reading in force when it migrates, since D records none. The two can differ only if the program changes `μkanren-λ-reading` between the two goals.

12.6 Reification

micro reifies an answer as a λ-expression whose parameters name the answer's variables and whose body lists the constraints and ends with the value (Section 10.7). The module plugs into that through the hooks `value`, `repr`, `terms`, `forms` and `environment` (Chapter 11).

Values. Deep values are β-normal and η-short (`μkanren-λ-value`). They are used by answers, by D, A and T, by `project°` and by aggregation.

Binders. An abstraction prints as `(ƛ (x y) body)`, nested binders merged. A binder is named after its hint, renamed apart from the binders around it, from every symbol of its body, and from the operators of the rebuilding code (`cons`, `quote`, `vector`, `make-record-instance`, `ƛ`, `@`, `eigen`, `bvar`). So neither capture nor shadowing can mislead the reader (Ma and Friedman's "true = false", Section 9.2.3), and a ground answer evaluates back to the same term:

```
(°->list/datum (fresh° (q) (=° q (ƛ (a) (ƛ (a) a)))))
(μkanren-datum (μkanren-abstract 'cons (λ (c) `(a ,c))))
```

```
((ƛ (a a1) a1))
(ƛ (cons1) (a cons1))
```

Applications print as `(@ h a ...)`, so a flexible application `(@ α x)` cannot be mistaken for the list `(α x)`.

Residual forms. The constraints of the λ layer print as Scheme forms that can be evaluated:

constraint	form in the answer
suspended equation (E)	`(assert (≐ u v))`
type on a flexible term (E)	`(assert (every (μ v (symbol? v)) (list u)))`
absento with a pending term (E, T)	`(assert (absent? d u))`
disequality whose trial binds (N)	`(begin (deny (equal? α t)))`, with `(∃ (_3) ...)` inside `deny` when the trial creates variables
disequality whose trial only suspends (N)	`(begin (deny (≐ u v)))`
any of them still holding an eigenvariable	wrapped in `(∀ (x) form)`

`≐` is exported: it decides αβη-equality of closed terms by unifying them in the empty state, so the forms evaluate. Through the `terms` hook, the variables of residual equations are reified *after* those of the answer, so they are named α, β, … rather than `_2` (Example 12.14). micro's show module gives the αKanren-style view `°->pretty`:

```
(°->pretty (fresh° (q) (∀° (x) (=° (@ q `(g ,x)) `(h (g ,x))))))
(°->pretty (fresh° (q) (∀° (x) (=° (@ q x) x))))
```

```
((α : (= (ϡ (x) (@ α (g x))) (ϡ (x) (h (g x))))))
((α : (= (ϡ (x) (@ α x x)) (ϡ (x) x))))
```

No free eigenvariable. Every constraint that holds an eigenvariable past its scope is closed, by `ϡ` or by `∀`, so no eigenvariable ever appears free in an answer. The one value that could hold one is the answer itself, when the variable that `°->list` reifies is created inside a `∀°`. λProlog reports no binding for a variable quantified inside `pi x\ `, and this answer cannot be told without x, so reifying it is an error:

```
(error-message (λ () (°->list #f (∀° (x) (fresh° (q) (=° q x))))))
```

```
"the answer holds the eigenvariable of a ∀° it was created in: create the variable it reifies
↪  outside every ∀°"
```

Live ground answers. `ϡ`, `@` and `≐` are bound in the module's environment, which the `environment` hook hands to `°->list/ground`. A ground answer is therefore a λ-term one can apply:

```
(@ (car (°->list/ground (fresh° (q) (=° q (ϡ (x y) `(g ,x ,y)))))) 'b 'b)
```

```
(g b b)
```

`°->list/datum` answers plain data instead, comparable with `equal?` against quoted book answers; that is what the suite uses, and `μkanren-repr->datum` turns micro's rebuilding code into such data.

Pitfall: Hints are not part of equality

`equal?` sees hints, the unifier does not. `(ƛ (x) x)` and `(ƛ (y) y)` unify and are `µkanren-α=?`, but they are not `equal?`: `equal?` compares records slot by slot, hint included, and the hint is kept in the record so that answers print the user's binder names. When comparing λ-terms in host code, use `µkanren-α=?` or `≐`. A related trap is Scheme's own: `(ƛ (quote) 'a)` shadows `quote` while the body is built; terms built through `µkanren-abstract` with such a hint print and evaluate correctly.

12.7 API reference

name	meaning
Terms	
`(ƛ (x ...) body)`	λ-abstraction over Scheme binders, η-short
`(@ f a ...)`	application; β at once on an abstraction, nested applications flattened
`(≐ u v)`	αβη-equality of closed terms, for residual forms
`(µkanren-abstract name f)`	the abstraction of the term `(f p)`, `p` a placeholder; binder hinted `name`
`(µkanren-apply f args)`	`@` on a list of arguments
`(µkanren-hnorm t s)`	head normal form of `t` in state `s` (Definition 12.3)
`(µkanren-α=? u v)`	α-equivalence of deep values, hints ignored
`(µkanren-datum t)`	a term as data, printed without a state
`µkanren-lam?`	record predicates; the last is their union
`µkanren-app?`	
`µkanren-eigen?`	
`µkanren-bvar?`	
`µkanren-λ-term?`	
`µkanren-lam-hint`	accessors
`µkanren-eigen-name`	
`µkanren-eigen-level`	
Goals	
`(∀° (x ...) g ...)`	GENERIC; `(∀ª name f)` its procedural form, `f` taking the eigenvariable
`(⊃° (D ...) g ...)`	AUGMENT; `(⊃ª clauses g)` its procedural form
`(define-dynamic-relation (r a ...) g ...)`	a relation that hypotheses can extend; with no `g`, no static clauses
`(µkanren-dynamic f)`	tag a procedure as dynamic, test the tag
`(µkanren-dynamic? f)`	

name	meaning
`µkanren-hyps°`	what the macros expand to
`µkanren-clause`	
`µkanren-clause/vars`	
`µkanren-clause*`	
Clauses D	
`(r t ...)`	fact, rule (infix and prefix), own variables
`((r t ...) ← g ...)`	
`(← (r t ...) g ...)`	
`(∀ (v ...) D)`	
Answers	
`(°->list/datum g)`	the answers of `g` as plain data, constraints dropped
`(µkanren-repr->datum r)`	micro's rebuilding code as data
Engine	
`µkanren-λ-reading`	parameter, `'generic` (default) or `'extensional`
`(µkanren-λ-depth s)`	the current universe level
`(µkanren-λ-level α s)`	the level of variable `α`
`(µkanren-λ-delayed s)`	the suspended constraints E
`(µkanren-λ-disequalities s)`	the λ-disequalities N
`µkanren-λ-extension`	the registered extension record
In `(aux kanren micro show)`	
`(°->pretty g)`	αKanren-style answers, `(value : constraint ...)`
`µkanren-answer->pretty`	

12.8 Summary

- **Terms.** Four records, locally nameless: `µkanren-lam` (interned hint, body), `µkanren-app`, `µkanren-eigen` (name, level, uninterned id), `µkanren-bvar`. `ƛ` uses Scheme binders and builds η-short; `@` β-reduces at once. Terms built alike are `equal?`; α-equality is `µkanren-α=?` and `≐`. Lists remain data: `(@ 'g 'a)` is not `'(g a)`.

- **State.** One immutable record in micro's slot `X`: depth, top, the levels L/L_n (an sbral, lazily padded), Γ (one entry per relation), E (suspended constraints), N (λ-disequalities). `∀°` and `⊃°` restore depth and Γ on every answer; branches never share it.

- **Unification.** micro does the first-order part; the hook applies ξ, η, Rigid, FF-same, FR/FF-diff by variable elimination, and suspends non-patterns: the rules of Fig. A.6 with the prefix erased into levels. Patterns are read off levels. Elim prunes and lowers only strict occurrences, fails an occurrence of F only by Nipkow's size argument, and suspends whatever a later binding could decide.

- **Check.** Every binding is scope-checked at its variable's level (NJK's binding rule with pruning), or taken back into E when a non-pattern decides it. Fast paths skip it before

any `∀°`, on plain terms and for top-level variables. Wake-up retries E in rounds while a round binds, and terminates when hnf does.

- **`∀°` and `⊃°`.** GENERIC runs its body one universe deeper with a new eigenvariable and discharges the residuals on exit (raise, close by ξ, retry, repeat). AUGMENT pushes closures on Γ. Dynamic relations backchain on hypotheses first, most recent first, then on their static body; a clause's free variables are shared, its `∀` variables renamed per use.

- **Stores.** `≠°` on λ-terms goes to N with a trial whose new variables are existential; D never holds a λ-term; two readings of ∀, generic (default) and extensional. `absent°` looks into parts and watches flexible terms; types wait on flexible terms and are decided by their predicate.

- **Answers.** β-normal, η-short, binders renamed apart; residuals as `(assert (≐ u v))`, `(deny ...)`, `(absent? ...)`, closed with `∀` when they keep an eigenvariable; never a free eigenvariable; ground answers are live terms.

13 The book's examples in `(aux kanren lambda)`

Chapter 12 presented the module `(aux kanren lambda)` rule by rule: the terms `ƛ` and `@`, the levels that police the scope of eigenvariables, the unifier for L_λ with its delayed equations, and the goals `∀°` and `⊃°`. This chapter runs that machinery on the programs of Part I. Each example was introduced in an earlier chapter, where we explained what it means in λProlog. Here we ask a narrower question: what does the module *do* with it? Which rule fires, why does the answer have the shape it has, and does it come out in the order λProlog gives?

The examples follow the book's progression. We start with the terms themselves (α, β, shadowing), go on to the two new goals (`∀°`, then `⊃°`, then both together in type inference), then to programs over λ-tree syntax (substitution, de Bruijn indices, prenex forms), higher-order programming (`mappred`, functional difference lists, Fischer's CPS transformation), and finally to the unification problems of the book's Chapter 8 one by one. The last section takes the queries of Ma and Friedman's 2021 paper [29] and answers them as λProlog would, explaining each point where the answers differ.

13.1 How to read the examples

Every example has four parts.

1. The **λProlog original**, copied from the book, with its printed page [MN p. n].
2. The **Scheme version**, copied verbatim from the curated script `examples.scm`, or from the programs it includes, `test/programs.scm`. In the repository both live under `doc/hol-kanren/prototype/`. After the merge, the programs move with the test suite to `src/test/` (Section 14.4).
3. The **answers**, pasted verbatim from `examples.out`, the actual output of the script. A few examples that the script does not contain were run separately with the same imports; each of them says so.
4. A **walk-through**: the rules of Chapter 12 that fire, the state changes they make, and how the order of the answers compares with λProlog.

The "p.N" in the labels of `examples.scm` are PDF pages; the prose uses printed pages (see Section 1.5).

13.1.1 The dictionary

The header of `examples.scm` gives the translation that every example uses:

```
;;; The dictionary, λProlog → here:
;;;   x\ t                (ƛ (x) t)              object binder by a Scheme binder
 ↪  (λ-tree syntax)
;;;   (F t)               (@ F t)                application of a variable or an
 ↪  abstraction
;;;   pi x\ G             (∀° (x) G)             GENERIC
;;;   D => G              (⊃° (D) G)             AUGMENT, for a relation made by
 ↪  define-dynamic-relation
;;;   pi X\ H :- B        (∀ (X) (H ← B))        a hypothetical clause with its own
 ↪  variables
;;;   sigma X\ G          (fresh° (X) G)
;;;   G1, G2   G1 ; G2    (and° G1 G2)  (or° G1 G2)
;;; Object-level syntax is Scheme data: `abs M` is `(abs ,M), `app M N` is `(app ,M
 ↪  ,N).
```

Object-level syntax stays first-order Scheme data, as everywhere in (aux kanren micro). Only the binders are meta-level: `abs x\ app x x` becomes `` `(abs ,(ƛ (x) `(app ,x ,x))) ``, a list whose second element is a λ-term. Inside the body of `ƛ`, the Scheme variable `x` is bound to a placeholder, so Scheme scoping gives the object binders, shadowing included (Section 6.2).

13.1.2 Three ways to print an answer

The script prints each query with a small macro, its label first:

```
(define-syntax show
  (syntax-rules ()
    ((_ label e) (begin (display ";; ") (display label) (newline) (pretty-print e)
     ↪  (newline)))))
```

and uses three runners.

- `(°->list/datum g)` gives each answer as plain data, comparable with `equal?` against the book's answers. Abstractions print as `(ƛ (x y) body)`, with nested binders merged and renamed apart. Applications whose head is not an abstraction print as `(@ h a ...)`. Unbound logic variables print as α, β, γ, ..., in order of first occurrence.
- `(°->list #f g)` gives micro's own answer form, a closed expression `(λ (α ...) constraint ... term)` (Section 10.7). The empty list `()` means that the query *failed*. `((λ () 'ok))` means one answer, with the ground value `ok`. A residual equation prints as `(assert (≐ u v))`.
- `(°->pretty g)`, from `(aux kanren micro show)`, gives the αKanren-style view `(α : constraint ...)`, which is the most readable one for suspended equations.

Many queries end in `(=° q 'ok)` or `(=° q 'solved)`. A query whose interesting part is a success or a failure has no answer variable of its own, so these goals give it one to print.

13.1.3 The rules, in one table

The walk-throughs name the rules of Chapter 12: the unification rules of Fig. 12.1, variable elimination (Definition 12.6), the scope check **Check** (Definition 12.7), and the three search rules of Section 12.4. We recall them here in one line each. Two notions are needed first.

- The *level* $\ell(X)$ of a logic variable is the number of `∀°` goals around the point where it was created. The level $\ell(c)$ of an eigenvariable is the depth of the `∀°` that created it, counting that `∀°`. The names that the unifier opens below binders have level ∞. A variable of level ℓ may be bound only to a term whose eigenvariables have level $\leq \ell$: it stands for an `∃` placed after the first ℓ universals of its branch.
- A *pattern* for F is a list of arguments that reduce to distinct eigenvariables, each of level greater than $\ell(F)$. This is Miller's condition "quantified inside the scope of F", read off the levels (Section 7.5, Appendix A.10, and Definition 12.5).

For instance, in `(fresh° (q) (∀° (x) (fresh° (y) ...)))` we have $\ell(q) = 0$, $\ell(x) = 1$ and $\ell(y) = 1$: the list (x) is a pattern for q but not for y, and y may be bound to x while q may not.

rule	fires on	effect
VAR, **Check**	$X \doteq t$, X unbound	micro binds $X := t$ after its occurs check. Then the scope check $\mathrm{Elim}(\ell(X), -, \langle\rangle, t)$ fails on an eigenvariable of level $> \ell(X)$, *lowers* the younger variables of t to $\ell(X)$, and prunes flexible subterms. A non-pattern it cannot decide turns the binding back into a suspended equation.
ξ	$\lambda.b \doteq \lambda.b'$	open both bodies with one new name c (level ∞), unify them, and **Check** the bindings made under it
η	$\lambda.b \doteq t$, t no abstraction	unify $b[c/0]$ with $(t\ c)$, c new (level ∞); only when one side is an abstraction
RIGID	$h\,\bar{u} \doteq h'\,\bar{v}$	same head and arity: unify the arguments; otherwise fail
FF-SAME	$F\,\bar{a} \doteq F\,\bar{b}$, both patterns	$\bar{a} = \bar{b}$: success; arities differ: fail; else $F := \lambda\bar{x}.\, H\,\langle x_i \mid a_i = b_i\rangle$, $\ell(H) = \ell(F)$
FR	$F\,\bar{a} \doteq t$, $\bar{a}$ a pattern, t rigid or flexible with another head (the case "FF-diff")	$F := \lambda\bar{x}.\, \mathrm{Elim}(\ell(F), F, \bar{a}, t)$. In t: each a_i becomes the i-th bound variable (projection); constants, and eigenvariables of level $\leq \ell(F)$, are kept (imitation); an eigenvariable out of reach fails (8.J); a flexible pattern $G\,\bar{b}$ loses the arguments F may not see (*pruning*, 8.H); a younger variable is lowered to $\ell(F)$ and raised over the a_i it may see (8.K); an occurrence of F fails when no instance can make it smaller (8.I), and otherwise suspends
FF-ONE	$F\,\bar{a} \doteq G\,\bar{b}$, only $\bar{b}$ a pattern	FR on G
DELAY	no side a pattern	unless the sides are α-equal, suspend $\lambda L.u \doteq \lambda L.v$ in the store E (L: the names opened so far); every later binding retries it; at the end it prints as `(assert (≐ u v))`

rule	fires on	effect
OCCURS	$X \doteq t$ with X inside a λ-term of t	succeed if t reduces to X, else eliminate X with no arguments
GENERIC	`(∀° (x) g)`	a new eigenvariable at depth + 1; g runs one level deeper; every answer gets the depth back
DISCHARGE	leaving `∀°` (back to depth d) with residuals on c	raise $Y := Y'\,c$, $\ell(Y') = d$, for each variable Y of the residual with $\ell(Y) \geq \ell(c)$; close the residual into $\lambda c.u \doteq \lambda c.v$; retry it; repeat while a raise brings c into more constraints (Section 12.4.2)
AUGMENT	`(⊃° (D ...) g ...)`	push the clauses on Γ for g only; every answer gets Γ back
BACKCHAIN	a call of a dynamic relation	one flat `or°`: the hypotheses, most recent first, then the static body

13.2 λ-tree syntax: α, β and shadowing

The first examples involve no search at all. They check that the terms behave as the book's λ-terms do: equal up to the names of bound variables (Section 4.2), with β as part of equality, and with binders that obey lexical scope (Section 6.2.3).

```
(show "λx.λy.(x . y), as data and as micro's rebuilding code"
      (list (°->list/datum (fresh° (q) (=° q (ƛ (x y) (cons x y)))))
            (°->list #f (fresh° (q) (=° q (ƛ (x y) (cons x y)))))))

(show "α-equivalence: λa.a = λb.b, and the β-redex (λx.g x x) a"
      (list (°->list #f (fresh° (q) (=° (ƛ (a) a) (ƛ (b) b))))
            (°->list/datum (fresh° (q f) (=° f (ƛ (x) `(g ,x ,x))) (=° q (@ f 'a))))))

(show "shadowing, λa.λa.a, printed λa.λa1.a1 (Ma–Friedman's \"true = false\" hazard)"
      (°->list/datum (fresh° (q) (=° q (ƛ (a) (ƛ (a) a))))))
```

```
;; λx.λy.(x . y), as data and as micro's rebuilding code
(((ƛ (x y) (x . y))) ((λ () (ƛ (x y) (cons x y)))))

;; α-equivalence: λa.a = λb.b, and the β-redex (λx.g x x) a
(((λ (α) α)) ((g a a)))

;; shadowing, λa.λa.a, printed λa.λa1.a1 (Ma–Friedman's "true = false" hazard)
((ƛ (a a1) a1))
```

Tutorial: what happens

1. `(ƛ (x y) (cons x y))` is built at once, before any goal runs. The body is a pair whose two cells hold the de Bruijn indices 1 and 0, and the names `x`, `y` survive only as printing hints. `=°` binds `q` to it by VAR: no eigenvariable is involved, so **Check** has nothing to do. The rebuilding code `(λ () (ƛ (x y) (cons x y)))` is

an expression that rebuilds the same term when evaluated, since `ƛ` is bound in the module.

2. `(=° (ƛ (a) a) (ƛ (b) b))` is decided by ξ: both bodies are opened with the same new name c, and $c \doteq c$ holds. Nothing is bound, so `q` stays free and prints as `α`. In fact the two terms are already the same record up to the hint: locally nameless terms make α-equivalence a structural fact.

3. `(@ f 'a)` receives the logic variable `f`, not its value: `@` is a Scheme procedure and does not consult the state. So it builds the flexible application $f\ a$. Once `f` is bound to $\lambda x.\, g\ x\ x$, the reifier's deep walk contracts the β-redex, and the answer is `(g a a)`, as `(x\ g x x) a` is in λProlog.

4. λa.λa.a is the term λa.λb.b: the inner binder shadows the outer one, because the inner Scheme `a` shadows the outer `a`. The printer renames apart, so the answer reads `(ƛ (a a1) a1)`: Church's *false*, not *true*. Section 13.11 returns to this.

13.3 Universal goals and the scope of eigenvariables

Section 3.4.3 introduced the simplest example of quantifier scope: `sigma x\ pi y\ x = y` is not provable, while `pi y\ sigma x\ x = y` is. In the book's words:

```
?- sigma x\ pi y\ x = y.
```

"This query is not provable because (pi y\ t = y) is not provable for any term t in which y does not appear" [MN p. 85]. `examples.scm` writes the two queries with an answer variable:

```
;; ?- pi x\ (Y = x).        no                ∃Y∀x. Y = x
;; ?- pi x\ sigma Y\ Y = x.  yes                ∀x∃Y. Y = x
(show "∃Y∀x. Y = x  and  ∀x∃Y. Y = x"
      (list (°->list #f (fresh° (q) (∀° (x) (=° q x))))
            (°->list #f (fresh° (q) (∀° (x) (fresh° (y) (=° y x))) (=° q 'ok)))))

(show "escape through an alias (q = (y), y = x), and through micro's cons°: both fail"
      (list (°->list #f (fresh° (q) (∀° (x) (fresh° (y) (=° q (list y)) (=° y x)))))
            (°->list #f (fresh° (q) (∀° (x) (cons° x '() q))))))
```

```
;; ∃Y∀x. Y = x  and  ∀x∃Y. Y = x
(() ((λ () 'ok)))

;; escape through an alias (q = (y), y = x), and through micro's cons°: both fail
(() ())
```

Tutorial: the levels at work

Write ℓ for levels. In each query `q` is created by the outer `fresh°`, at depth 0, so $\ell(q) = 0$.

1. $\exists Y \forall x.\, Y = x$**.** GENERIC creates the eigenvariable x with $\ell(x) = 1$ and runs `(=° q x)` at depth 1. micro binds $q := x$, then the `post` hook runs **Check**: x is an eigenvariable with $\ell(x) = 1 > \ell(q) = 0$, so the binding fails. The query has no answer: `()`.

2. $\forall x \exists Y.\, Y = x$**.** Now `y` is created inside the `∀°`, so $\ell(y) = 1$ and $y := x$ passes **Check**. When the `∀°` exits, the depth is set back to 0 for the rest of the query, and `(=° q 'ok)` gives the answer. The binding of `y` is not part of the answer, which is what λProlog does for a variable quantified inside `pi x\`.

3. **The alias.** `y` has level 1 again. The first goal binds $q := (y)$. **Check** of this binding finds the younger variable y inside a term bound to the older q, and *lowers* it: $y := y'$ with $\ell(y') = 0$. This is condition (3) of Nadathur, Jayaraman and Kwon's binding rule, quoted in Section 3.4.3: the variables of t take the level of X. The second goal walks y to y' and binds $y' := x$, which **Check** now rejects, since $\ell(x) = 1 > 0$. Without lowering, x would escape into q through y. The suite checks both orders of the two goals (A1/alias).

4. **micro's `cons°`.** `cons°` is an ordinary relation of micro, defined with `=°`. Since the check runs in `=°`'s `post` hook, every relation built on `=°` inherits it. Of the three prototypes, the one with its own unifier failed exactly here (Section 11.3.1).

The same check applies to logic variables that reach a clause. Section 3.5 quoted Nadathur, Jayaraman and Kwon's example: with the clause $\forall z\,(q\,z \supset p\,(d\,z))$, the goal $\exists x \forall y\,(q\,y \supset p\,x)$ must fail, while $\forall y \exists x\,(q\,y \supset p\,x)$ succeeds [43, p. 16]. This is the suite's case A1/levels-reach-clauses, shown in Example 12.19 as well; we ran it separately:

```
(show "∃x∀y(q y ⊃ p x) with ∀z(q z ⊃ p (d z))"
      (let ()
        (define-dynamic-relation (q° z))
        (define-dynamic-relation (p° x) (fresh° (z) (=° x `(d ,z)) (q° z)))
        (list (°->list #f (fresh° (x) (∀° (y) (⊃° ((q° y)) (p° x)))))
              (°->list #f (fresh° (q) (∀° (y) (fresh° (x) (⊃° ((q° y)) (p° x)))) (=°
               ↪  q 'ok))))))
```

```
;; ∃x∀y(q y ⊃ p x) with ∀z(q z ⊃ p (d z))
(() ((λ () 'ok)))
```

In the first query $\ell(x) = 0$ and $\ell(y) = 1$. BACKCHAIN on `(p° x)` runs the static body, which creates the clause's own `z` at depth 1, so $\ell(z) = 1$. The binding $x := (d\,z)$ puts the younger z inside the older x, and **Check** lowers it: $z := z'$ with $\ell(z') = 0$. The call `(q° z')` then finds the hypothesis `(q° y)`, and $z' \doteq y$ fails, since $\ell(y) = 1 > 0$. In the second query x is created inside the `∀°`, so $\ell(x) = 1$; z is not lowered, and $z \doteq y$ succeeds. This is NJK's "new tagged instance"

of a clause: its own variables are created at the current level, at each use, and then obey the binding rule like any other variable.

13.4 Hypothetical goals and the program context

Chapter 3 introduced AUGMENT: to prove $D \supset G$, add D to the program and prove G. In the module the program context Γ is a field of the state. `⊃°` extends it for the extent of its body and gives every answer the old Γ back. Only relations made by `define-dynamic-relation` read Γ (Section 14.2.1); assuming a clause for any other relation raises an error.

13.4.1 Clause order under augmentation

"λProlog uses the rule that when an implicational goal augments the current program, it does so by adding the new clauses at the front of the program" [MN p. 78]. With the program `p 1`:

```
?- p 2 => p 3 => p X.
X = 3;
X = 2;
X = 1
?- (p 2 & p 3) => p X.
X = 2;
X = 3;
X = 1
```

```
;; p 1.
;; ?- p 2 => p 3 => p X.          X = 3; X = 2; X = 1
;; ?- (p 2 & p 3) => p X.         X = 2; X = 3; X = 1
(define-dynamic-relation (p° x) (=° x 1))
(show "clause order under augmentation (p.94)"
      (list (°->list/datum (fresh° (q) (⊃° ((p° 2)) (⊃° ((p° 3)) (p° q)))))
            (°->list/datum (fresh° (q) (⊃° ((p° 2) (p° 3)) (p° q))))))
```

```
;; clause order under augmentation (p.94)
((3 2 1) (2 3 1))
```

Tutorial: the state of Γ, step by step

1. `(define-dynamic-relation (p° x) (=° x 1))` makes `p°` a dynamic relation whose static body is `(=° x 1)`, the clause `p 1`.
2. First query. The outer `⊃°` sets $\Gamma = [\texttt{p°} \mapsto (p\,2)]$. The inner one puts its clause in front: $\Gamma = [\texttt{p°} \mapsto (p\,3,\ p\,2)]$.
3. The call `(p° q)` is BACKCHAIN: one flat disjunction $(q \doteq 3) \vee (q \doteq 2) \vee (q \doteq 1)$, the hypotheses most recent first, then the static body. Each disjunct succeeds at once, so micro's interleaving delivers them in order: `(3 2 1)`.
4. Second query. One `⊃°` assumes both clauses, which are kept in the order written:

$\Gamma = [\texttt{p°} \mapsto (p\,2,\ p\,3)]$, and the answers are `(2 3 1)`.

Both orders agree with the book. The agreement is not an accident of the search: the dynamic relation is built as *one* flat `or°`, so when every clause answers immediately, the answers come in clause order. When the clauses do more work before answering, interleaving decides the order instead (Section 13.7.2).

13.4.2 The course database (book Fig. 3.1)

Section 3.3.2 presented the database and its four hypothetical queries:

```
fact (finished kim 102) & fact (finished dana 101).
fact (finished kim 210) & fact (finished dana 250).
fact (cs_major X) :-
   (fact (finished X 101); fact (finished X 102)),
   fact (finished X 250), fact (finished X 301).
fact (graduates X) :-
   (fact (finished X 101); fact (finished X 102)),
   (fact (finished X 210); fact (finished X 250)),
   fact (finished X 301).
false :- fact (finished X 210), fact (finished X 250).

?- fact (finished dana X) => false.
X = 210;
no
?- fact (finished X Y) => (fact (graduates X), fact (cs_major X)).
X = dana
Y = 301;
no
?- fact (finished kim X) => fact (finished kim Y) =>
   (fact (graduates kim), fact (cs_major kim)).
X = 250
Y = 301
?- fact (finished kim 250) => fact (finished kim 301) => false.
solved
```

(Figure 3.1 without its type declarations, and the four sessions, [MN p. 80–81].) In `test/programs.scm` the program is one dynamic relation `fact°`, since every query assumes `fact` clauses:

```
(define-dynamic-relation (fact° e)
  (fresh° (x)
    (cond°
      ((=° e '(finished kim 102))) ((=° e '(finished dana 101)))
      ((=° e '(finished kim 210))) ((=° e '(finished dana 250)))
      ((=° e `(cs_major ,x))
       (or° (fact° `(finished ,x 101)) (fact° `(finished ,x 102)))
       (fact° `(finished ,x 250))
       (fact° `(finished ,x 301)))
```

```
      ((=° e `(graduates ,x))
       (or° (fact° `(finished ,x 101)) (fact° `(finished ,x 102)))
       (or° (fact° `(finished ,x 210)) (fact° `(finished ,x 250)))
       (fact° `(finished ,x 301))))))

(define-dynamic-relation (false°) (fresh° (x) (fact° `(finished ,x 210)) (fact°
↪  `(finished ,x 250))))
```

and the queries are

```
(show "courses: which course would make Dana's record inconsistent"
      (°->list/datum (fresh° (q) (⊃° ((fact° `(finished dana ,q))) (false°)))))
(show "courses: who graduates in CS after one more course, and which"
      (°->list/datum (fresh° (q x y) (=° q (list x y)) (⊃° ((fact° `(finished ,x
      ↪  ,y))) (fact° `(graduates ,x)) (fact° `(cs_major ,x))))))
(show "courses: Kim's two courses (each order is a proof, each found twice; the book
↪  shows the first)"
      (sorted (°->list/datum (fresh° (q x y) (=° q (list x y))
                             (⊃° ((fact° `(finished kim ,x))) (⊃° ((fact° `(finished
                               ↪  kim ,y))) (fact° '(graduates kim)) (fact°
                               ↪  '(cs_major kim))))))))
(show "courses: no consistent way for Kim"
      (°->list/datum (fresh° (q) (⊃° ((fact° '(finished kim 250))) (⊃° ((fact°
      ↪  '(finished kim 301))) (false°))) (=° q 'solved))))
```

```
;; courses: which course would make Dana's record inconsistent
(210)

;; courses: who graduates in CS after one more course, and which
((dana 301))

;; courses: Kim's two courses (each order is a proof, each found twice; the book shows the
↪  first)
((250 301) (250 301) (301 250) (301 250))

;; courses: no consistent way for Kim
(solved)
```

Tutorial: a logic variable inside a hypothesis

Take the first query. `q` is created before `⊃°` runs, so the assumed clause `(fact° `(finished dana ,q))` is *one* still unknown fact, not a schema: `q` is a free variable of the clause's closure, shared with the query ([MN p. 86–88]; BACKCHAIN in Section 12.4).

1. `(false°)` runs its body: `x` is new, and `(fact° `(finished ,x 210))` is a BACKCHAIN on $\Gamma$ first. The hypothesis unifies `(finished dana q)` with `(finished x 210)`: $x :=$ dana, $q := 210$.
2. `(fact° '(finished dana 250))` succeeds from the static body.
3. The static clauses also prove `(finished x 210)`, with $x :=$ kim. But then `(finished`

`kim 250)` must hold, and it is neither a fact nor an instance of the hypothesis, which is about Dana. So the only answer is `(210)`, as in the book.

In the third query there are four answers instead of the book's one: the book shows only the first answer. They are proofs of the same goal. The symmetric pair appears because either hypothesis can supply either course. Each answer appears twice because the disjunct `(or° (fact° `(finished ,x 210)) (fact° `(finished ,x 250)))` in `graduates` has two proofs once 250 is assumed: from Kim's static 210, and from the hypothesis. The script sorts them, since the order among them is the interleaving's. Section 3.3.2 reached the same count with the throwaway interpreter of Section 3.8.3.

13.4.3 Jars and bugs

The example of Section 3.4 combines GENERIC and AUGMENT:

```
sterile J :- pi x\ bug x => in x J => dead x.
dead B    :- heated J, in B J, bug B.
heated j.
```

This is the program of [MN p. 83]; its signature has no constant of type `bug`. In Scheme:

```
;; p.98-99:  sterile J :- pi x\ bug x => in x J => dead x.
;;           dead B :- heated J, in B J, bug B.        heated j.
(define-dynamic-relation (bug° b))
(define-dynamic-relation (in° b j))
(define-relation (heated° j) (=° j 'j))
(define-dynamic-relation (dead° b) (fresh° (j) (heated° j) (in° b j) (bug° b)))
(define-relation (sterile° j) (∀° (x) (⊃° ((bug° x)) (⊃° ((in° x j)) (dead° x)))))
(show "jars and bugs: ?- sterile Q." (°->list/datum (fresh° (q) (sterile° q))))
```

```
;; jars and bugs: ?- sterile Q.
(j)
```

Tutorial: trace of `sterile Q`

1. `(sterile° q)`: $\ell(q) = 0$. GENERIC creates the eigenvariable x, $\ell(x) = 1$, and the depth becomes 1.
2. Two AUGMENTS: $\Gamma = [\texttt{in°} \mapsto (\mathsf{in}\, x\, q), \texttt{bug°} \mapsto (\mathsf{bug}\, x)]$. The clause `(in° x j)` closes over the `j` of `sterile°`, which is `q`.
3. `(dead° x)` has no hypothesis, so BACKCHAIN runs its static body. It creates `j'` at depth 1, and `(heated° j')` binds $j' := \mathsf{j}$.
4. `(in° x j')` backchains on the hypothesis: it unifies $(x\ \mathsf{j})$ with $(x\ q)$, so $q := \mathsf{j}$. **Check** accepts it: the constant `j` contains no eigenvariable.

5. `(bug° x)` succeeds from the hypothesis. The `∀°` exits: depth 0 and the old Γ come back, and there is no residual to discharge. The answer is `(j)`.

The proof is about an arbitrary bug: `bug°` and `in°` have no static clauses at all, and the only facts about x are the two hypotheses. Had a step tried to bind `q` to something containing x, **Check** would have rejected it.

13.4.4 Reverse with assumed clauses (book Fig. 3.4)

Section 3.5 showed how an assumed clause can "tie" a variable of the goal:

```
reverse L K :-
   (rev nil K &
    (pi X\ pi L\ pi K\ rev (X::L) K :- rev L (X::K)))
       => rev L nil.
```

(Figure 3.4, [MN p. 87]). The first clause mentions the `K` of `reverse`, the second has its own universally quantified `X`, `L`, `K`. The Scheme version says the same with `∀`:

```
;; Figure 3.4 (p.104):
;; reverse L K :- (rev nil K & (pi X\ pi L\ pi K\ rev (X::L) K :- rev L (X::K))) =>
 ↪  rev L nil.
;; the free K of `rev nil K` is the caller's, the clause's own X, L, K are renamed at
 ↪  each use.
(define-dynamic-relation (rev° l k))
(define-relation (reverse° l k)
  (⊃° ((rev° '() k) (∀ (X L K) ((rev° `(,X . ,L) K) ← (rev° L `(,X . ,K))))) (rev° l
   ↪  '())))
(show "reverse with assumed clauses, forwards and backwards"
      (list (°->list/datum (fresh° (q) (reverse° '(1 2 3) q)))
            (°->list/datum (take° 1 (fresh° (q) (reverse° q '(1 2 3)))))))
```

```
;; reverse with assumed clauses, forwards and backwards
(((3 2 1)) ((3 2 1)))
```

Tutorial: two kinds of clause variables

`⊃°` expands each clause into a pair `(rev° . procedure)`, the procedure taking the list of arguments of a call. For `(rev° '() k)` the procedure is `(=° args (list '() k))`: `k` is a free variable of the closure, the caller's `q`. For the `∀` clause the procedure is `(fresh° (X L K) (=° args ...) (rev° L ...))`: its variables are created anew at each call, which is how λProlog renames a clause's own variables (Section 12.4).
Forwards, `(rev° '(1 2 3) '())` uses the second clause three times, building the accumulator `(3 2 1)`, and then `(rev° '() '(3 2 1))` uses the first one, which binds $k := (3\,2\,1)$.
Backwards, `k` is `(1 2 3)` and `l` is unknown. The second clause guesses longer and longer lists, and the first clause succeeds only when the accumulator equals `(1 2 3)`, after three

steps. That is the only answer, but nothing tells the search so. In a separate run, `(take° 2 (fresh° (q) (reverse° q '(1 2 3))))` had not returned after 20 seconds, so the script takes one answer.

13.5 Type inference: ∀ and ⊃ together

Type inference (Sections 6.5.4 and 8.2.1) is the book's standard example of a universal goal and a hypothesis working together. To type $\lambda x.\, M$, take a new constant x, assume that it has type A, and type the body:

```
typeof (app M N) A :- typeof M (arr B A), typeof N B.
typeof (abs M) (arr A B) :- pi x\ typeof x A => typeof (M x) B.

?- typeof (abs x\ abs y\ abs z\ app (app x z) (app y z)) Ty.
Ty = arr (arr T1 (arr T2 T3)) (arr (arr T1 T2) (arr T1 T3))
?- typeof (abs x\x) Ty.
Ty = arr T1 T1
?- typeof (abs x\ app x x) Ty.
no
```

These are the clauses and the session of [MN p. 190–191]. In `test/programs.scm`:

```
; §7.4.3, type inference:  typeof (abs M) (arr A B) :- pi x\ typeof x A => typeof (M
↪  x) B.
(define-dynamic-relation (typeof° m t)
  (cond°
    ((fresh° (M N B) (=° m `(app ,M ,N)) (typeof° M `(arr ,B ,t)) (typeof° N B)))
    ((fresh° (R A B) (=° m `(abs ,R)) (=° t `(arr ,A ,B))
       (∀° (x) (⊃° ((typeof° x A)) (typeof° (@ R x) B)))))))
```

and the queries:

```
(show "typeof S, I and λx.x x"
      (list (°->list/datum (fresh° (q) (typeof° `(abs ,(ƛ (x) `(abs ,(ƛ (y) `(abs ,(ƛ
       ↪  (z) `(app (app ,x ,z) (app ,y ,z)))))))) q)))
            (°->list/datum (fresh° (q) (typeof° `(abs ,(ƛ (x) x)) q)))
            (°->list/datum (fresh° (q) (typeof° `(abs ,(ƛ (x) `(app ,x ,x))) q)))))
(show "typeof backwards: the first terms of type i → i (interleaving finds them
↪  where DFS loops)"
      (°->list/datum (take° 3 (fresh° (q) (typeof° q '(arr i i))))))
```

```
;; typeof S, I and λx.x x
(((arr (arr α (arr β γ)) (arr (arr α β) (arr α γ)))) ((arr α α)) ())

;; typeof backwards: the first terms of type i → i (interleaving finds them where DFS loops)
((abs (ƛ (x) x))
 (abs (ƛ (x) (app (abs (ƛ (x1) x1)) x)))
 (abs (ƛ (x) (app (abs (ƛ (x1) x)) x))))
```

Tutorial: typing `abs x\x`, state by state

1. `(typeof° m q)` with m = `(abs ,(ƛ (x) x))`. BACKCHAIN: Γ has no hypothesis yet, so only the static `cond°` runs. The `app` branch fails on `(=° m `(app ,M ,N))`.
2. The `abs` branch binds $R := \lambda x.\, x$ and $q := (\text{arr } A\ B)$; R, A, B have level 0.
3. GENERIC: an eigenvariable `x`, call it c, with $\ell(c) = 1$. AUGMENT: $\Gamma = [\texttt{typeof°} \mapsto (\text{typeof } c\ A)]$.
4. The goal `(typeof° (@ R x) B)`. As in the previous example, `(@ R x)` receives the logic variable R, not its value, so it is the flexible term $R\,c$. Since R is bound, every walk reduces it to c.
5. BACKCHAIN on Γ first: the hypothesis unifies $(c\ B)$ with $(c\ A)$, so $A \doteq B$, a variable–variable binding with no eigenvariable in it (micro binds the older variable to the newer one). Then the static clauses fail: c is neither an `app` nor an `abs` list.
6. The `∀°` exits: depth 0, Γ empty. The answer is $q = (\text{arr } A\ A)$, printed `(arr α α)`.

For $\lambda x.\, x\,x$ the hypothesis is used twice: once for the head, giving $A := (\text{arr } B'\ B)$, once for the argument, giving $B' \doteq (\text{arr } B'\ B)$, which fails micro's ordinary occurs check. For S, three nested `∀°` goals put three hypotheses in Γ, all for `typeof°`, most recent first. The answer has the book's shape, with α, β, γ in place of T1, T2, T3. Section 12.4.6 prints the same steps with a probe, for $\lambda x.\, \lambda y.\, x$ and for $\lambda x.\, x\,x$.

Kanren angle: running type inference backwards

The book never asks `typeof Ty (arr i i)` with the term unknown, and for a good reason: depth-first search tries the `app` clause first, which asks again for a term of unknown type, forever. micro's interleaving search alternates between the branches, so it finds terms of a given type, and even (term, type) pairs. We ran the suite's case A3/backwards separately:

```
(show "typeof backwards: (term type) pairs"
      (°->list/datum (take° 4 (fresh° (q m t) (=° q (list m t)) (typeof° m
      ↪  t)))))
```

```
;; typeof backwards: (term type) pairs
(((abs (ƛ (x) x)) (arr α α))
 ((abs (ƛ (x) (abs (ƛ (x1) x1)))) (arr α (arr β β)))
 ((abs (ƛ (x) (abs (ƛ (x1) x)))) (arr α (arr β α)))
 ((app (abs (ƛ (x) x)) (abs (ƛ (x) x))) (arr α α)))
```

The interesting step is $R\,c \doteq c$ with R unknown: a pattern, solved by FR with the projection $R := \lambda x.\, x$. The binder of the answer is named `x` after the eigenvariable of the `∀°`, the name the clause gave it. This is one of the places where the module finds answers that λProlog misses (Section 14.2.4).

13.6 Object-level substitution

Section 6.3 explained the central idea of λ-tree syntax: object-level substitution is meta-level β. Section 6.7 gave the alternative that avoids β altogether, `copy` clauses assumed under `pi`. Both run here.

13.6.1 Call-by-name evaluation

```
cbn (abs R) (abs R).
cbn (app M N) V :- cbn M (abs R), cbn (R N) V.

?- cbn (app (abs x\ abs w\w) (app (abs x\ app x x) (abs x\ app x x))) V.
```

gives `V = abs w\w`, while call-by-value loops (Figure 7.3, [MN p. 182–183]).

```
; Figure 7.3, call-by-name evaluation: β of the object language is β of the
 ↪  meta-language.
(define-relation (cbn° m v)
  (cond°
    ((fresh° (R) (=° m `(abs ,R)) (=° v m)))
    ((fresh° (M N R) (=° m `(app ,M ,N)) (cbn° M `(abs ,R)) (cbn° (@ R N) v)))))
```

```
(show "call by name: (λx.λw.w) Ω"
      (let* ((ω `(abs ,(ƛ (x) `(app ,x ,x)))) (Ω `(app ,ω ,ω)))
        (°->list/datum (fresh° (q) (cbn° `(app (abs ,(ƛ (x) `(abs ,(ƛ (w) w)))) ,Ω)
         ↪  q)))))
```

```
;; call by name: (λx.λw.w) Ω
((abs (ƛ (w) w)))
```

Tutorial: β on demand

In the `app` branch, `(@ R N)` is built from the logic variable `R`, as in the type-inference trace: it is a flexible application. `(cbn° M `(abs ,R))` evaluates the operator `(abs ,(ƛ (x) ...))`, which is already a value: $R := \lambda x.\,\mathrm{abs}\,(\lambda w.\,w)$. When the second call `(cbn° (@ R N) v)` unifies its first argument with `` `(abs ,R') ``, the walk of $R\,N$ contracts the redex to `(abs ,(ƛ (w) w))`. The argument $N = \Omega$ is dropped by that β step and never evaluated, so the search terminates, as call-by-name should. No rule of the unifier is needed beyond β: every step is a pattern-free, first-order unification on a reduced term.

13.6.2 `subst R N (R N)`: dynamic L_λ

The one-line `subst R N (R N)` is the book's example of a clause outside L_λ that is "meant only to compute the effect of substitution through reduction" [MN p. 225–226], since `R` is expected to be known when the clause is used (Section 7.9.1). The module accepts the non-pattern $R\,N$ in either order of the goals. This query is the suite's case A4/beta, run separately, with R bound *after* the equation:

```
(show "subst R N (R N): R bound after"
      (°->list/datum (fresh° (q r) (=° q (@ r `(abs ,(ƛ (y) y)))) (=° r (ƛ (x) `(app
       ↪  ,x ,x))))))
```

```
;; subst R N (R N): R bound after
((app (abs (ƛ (y) y)) (abs (ƛ (y) y))))
```

The first goal binds q to the flexible term $r\,(\text{abs}\ldots)$. The argument is a list, not an eigenvariable, so this is no pattern, but no `∀°` has run, so **Check** has nothing to forbid and the binding stands. The second goal binds r, and the reifier's deep walk contracts the redex. Had the first binding involved an eigenvariable that only the value of r could keep or drop, **Check** would have turned it into a suspended equation (Delay), retried when r is bound (Section 13.10.5 and Example 12.14).

13.6.3 Copy and substitution without β (book Fig. 7.10)

```
copy (app M N) (app P Q) :- copy M P, copy N Q.
copy (abs M) (abs N) :- pi x\ copy x x => copy (M x) (N x).
subst M T S :- pi x\ copy x T => copy (M x) S.

?- copy (abs x\ abs y\ app y x) M.
```

"Solving for all the variables yields the unique answer substitution" `M = abs v\abs u\app u v` (Figure 7.10 and the trace, [MN p. 199–200]).

```
; Figure 7.10, copy and subst on untyped λ-terms.
(define-dynamic-relation (copy° m n)
  (cond°
    ((fresh° (M N P Q) (=° m `(app ,M ,N)) (=° n `(app ,P ,Q)) (copy° M P) (copy° N
     ↪  Q)))
    ((fresh° (M N) (=° m `(abs ,M)) (=° n `(abs ,N))
       (∀° (x) (⊃° ((copy° x x)) (copy° (@ M x) (@ N x)))))))
(define-relation (subst° m t s) (∀° (x) (⊃° ((copy° x t)) (copy° (@ m x) s))))
```

```
(show "copy (p.215), and subst (x\\ app x x) (abs y\\ y)"
      (list (°->list/datum (fresh° (q) (copy° `(abs ,(ƛ (x) `(abs ,(ƛ (y) `(app ,y
       ↪  ,x))))) q)))
            (°->list/datum (fresh° (q) (subst° (ƛ (x) `(app ,x ,x)) `(abs ,(ƛ (y) y))
             ↪  q)))))
```

```
;; copy (p.215), and subst (x\ app x x) (abs y\ y)
(((abs (ƛ (x) (abs (ƛ (x1) (app x1 x))))))
 ((app (abs (ƛ (y) y)) (abs (ƛ (y) y)))))
```

Tutorial: the book's trace, replayed

The book traces this query with the eigenvariables c and d [MN p. 199]. In the module:

1. The `abs` clause binds $M_0 := \lambda x.\,\text{abs}\,(\lambda y.\,\text{app}\ y\ x)$ and $q := (\text{abs}\ N_0)$.

GENERIC gives c at level 1, AUGMENT assumes `(copy° c c)`, and the goal is `copy°` (abs (λy. app y c)) (N_0 c).

2. The `abs` clause again. The equation $N_0\ c \doteq$ (abs N_1) is a pattern for N_0: FR binds $N_0 := \lambda x.$ abs $(N_1'\ x)$. N_1 was created at level 1, inside the first `∀°`, so it is lowered and raised over c: $N_1 := N_1'\ c$ (the 8.K step of Section 13.10). The book's M_1 is our N_0.

3. A second GENERIC gives d at level 2, and `(copy° d d)` is assumed. The goal is `copy°` (app d c) ($N_1'\ c\ d$). The `app` clause binds $N_1'\ c\ d \doteq$ (app P Q) by FR, and the hypotheses give $P = d$, $Q = c$, by projection.

4. Each `∀°` exits with no residual. The answer is abs $(\lambda x.$ abs $(\lambda x_1.$ app x_1 $x))$, the book's `abs v\abs u\app u v` up to the names.

The binders are named `x` and `x1`: the unifier names the abstractions it builds after the eigenvariables they abstract, and both eigenvariables come from the clause's `(∀° (x) ...)`. The printer renames the second one apart.

13.6.4 Existential generalisation (book Fig. 7.11)

Figure 7.11 specifies substitution into first-order formulas with `copyterm` and `copyform` clauses [MN p. 201]. Because `subst` is a relation, it can run backwards and abstract a term out of a formula: "given a term T and a formula S, it is possible to compute an abstraction M such that (subst M T S) is derivable" [MN p. 202]:

```
?- subst M (f a) ((p (f a)) && (q a (f a))).
M = x\ (p x) && (q a x)
M = x\ (p x) && (q a (f a))
M = x\ (p (f a)) && (q a x)
M = x\ (p (f a)) && (q a (f a))
```

These are the four existential generalisations of the formula on $f\ a$ (Section 6.7). `test/programs.scm` has the clauses for the constants used here:

```
; Figure 7.11, copy and subst on first-order formulas.
(define-dynamic-relation (copyterm° t s)
  (cond°
    ((=° t 'a) (=° s 'a))
    ((fresh° (x u) (=° t `(f ,x)) (=° s `(f ,u)) (copyterm° x u)))
    ((fresh° (x y u v) (=° t `(g ,x ,y)) (=° s `(g ,u ,v)) (copyterm° x u) (copyterm°
     ↪ y v)))))
(define-relation (copyform° b d)
  (cond°
    ((fresh° (B C D E op) (=° b `(,op ,B ,C)) (=° d `(,op ,D ,E)) (or° (=° op 'and)
     ↪ (=° op 'or) (=° op 'imp))
       (copyform° B D) (copyform° C E)))
```

```
    ((fresh° (B D op) (=° b `(,op ,B)) (=° d `(,op ,D)) (or° (=° op 'all) (=° op
    ↪  'some))
       (∀° (y) (⊃° ((copyterm° y y)) (copyform° (@ B y) (@ D y))))))
    ((fresh° (x u) (=° b `(p ,x)) (=° d `(p ,u)) (copyterm° x u)))
    ((fresh° (x y u v) (=° b `(q ,x ,y)) (=° d `(q ,u ,v)) (copyterm° x u) (copyterm°
    ↪  y v)))))
(define-relation (substform° m t n) (∀° (x) (⊃° ((copyterm° x t)) (copyform° (@ m x)
↪  n))))
```

```
(show "existential generalization: the book's four answers, in its order"
     (°->list/datum (fresh° (q) (substform° q '(f a) '(and (p (f a)) (q a (f a)))))))
```

```
;; existential generalization: the book's four answers, in its order
((ƛ (x) (and (p x) (q a x)))
 (ƛ (x) (and (p x) (q a (f a))))
 (ƛ (x) (and (p (f a)) (q a x)))
 (ƛ (x) (and (p (f a)) (q a (f a)))))
```

Tutorial: how an unknown abstraction is built, piece by piece

1. `substform°` creates the eigenvariable x (level 1), assumes `(copyterm° x '(f a))`, and calls `copyform°` on $q\,x$, a flexible pattern, against the formula.
2. The first `copyform°` clause matches `(op B C)` on the left: $q\,x \doteq (\mathrm{op}\ B\ C)$ is solved by FR. The variables op, B, C were created at level 1, so elimination lowers them and raises them over x: $q := \lambda x.\,(\mathrm{op}'\ x\ \ B'\ x\ \ C'\ x)$. The next goal unifies the formula `(and ...)` with `(op D E)`, so $\mathrm{op}'\ x \doteq$ and gives $\mathrm{op}' := \lambda x.$ and by imitation, and the `or°` then accepts `and`. The two halves are copied in turn.
3. For the first half, the `p` clause gives $B'\ x \doteq (\mathrm{p}\ X)$ and then `(copyterm° (X' x) '(f a))`. BACKCHAIN tries the hypothesis *first*: $X'\ x \doteq x$ gives $X' := \lambda x.\,x$ by projection, which is the answer $p\,x$. Then the static clauses: $X'\ x \doteq (\mathrm{f}\ X'')$ and `(copyterm° (X'' x) 'a)`, where only the constant `a` clause applies, and $X'' := \lambda x.\,a$ by imitation: the answer $p\,(f\ a)$.
4. The second half does the same for $q\,a\,(f\ a)$. The occurrence of `a` has only the static answer, since the hypothesis copies x to $f\ a$, not to a.

The order is the book's: the hypothesis comes first for each occurrence, and the first occurrence varies slowest.

13.7 Recursion over binders: de Bruijn terms and prenex forms

13.7.1 Translating to and from de Bruijn syntax

Section 6.5.5 translated λ-tree syntax into de Bruijn's nameless terms by assuming, for each binder crossed, the depth at which it was crossed:

```
trans D (abs M) (ab P) :- pi c\ depth D c => (E is D + 1, trans E (M c) P).
trans D (app M N) (ap P Q) :- trans D M P, trans D N Q.
trans D X (deb E) :- depth N X, E is (D - N).

?- trans 1 (abs x\ app x (abs y\ app x (abs w\ app w x))) D.
D = ab (ap (deb 1) (ab (ap (deb 2) (ab (ap (deb 1) (deb 3))))))
?- trans 1 P (ab (ap (deb 1) (ab (ap (deb 2) (ab (ap (deb 1) (deb 3)))))))).
P = abs W1\ app W1 (abs W2\ app W1 (abs W3\ app W3 W1))
```

These are the clauses and the session of [MN p. 193]. micro has no `is`, so the Scheme version computes with `project°`:

```
; §7.4.4, translation to and from de Bruijn syntax, with the `depth` assumptions.
(define-dynamic-relation (depth° n x))
(define-relation (trans° d t b)
  (cond°
    ((fresh° (M P) (=° t `(abs ,M)) (=° b `(ab ,P))
       (∀° (c) (⊃° ((depth° d c)) (fresh° (e) (project° ((d* d)) (=° e (+ d* 1)))
        ↪  (trans° e (@ M c) P))))))
    ((fresh° (M N P R) (=° t `(app ,M ,N)) (=° b `(ap ,P ,R)) (trans° d M P) (trans°
     ↪  d N R)))
    ((fresh° (n e) (=° b `(deb ,e)) (depth° n t) (project° ((d* d) (n* n)) (=° e (-
     ↪  d* n*)))))))
```

```
(show "de Bruijn, both directions"
      (list (°->list/datum (fresh° (q) (trans° 1 `(abs ,(ƛ (x) `(app ,x (abs ,(ƛ (y)
       ↪  `(app ,x (abs ,(ƛ (w) `(app ,w ,x)))))))) q)))
            (°->list/datum (fresh° (q) (trans° 1 q '(ab (ap (deb 1) (ab (ap (deb 2)
             ↪  (ab (ap (deb 1) (deb 3))))))))))))
```

```
;; de Bruijn, both directions
(((ab (ap (deb 1) (ab (ap (deb 2) (ab (ap (deb 1) (deb 3)))))))
 ((abs (ƛ (c) (app c (abs (ƛ (c1) (app c (abs (ƛ (c2) (app c2 c))))))))))
```

Both answers are the book's terms. The backward one is `abs W1\ app W1 (abs W2\ app W1 (abs W3\ app W3 W1))` with `c`, `c1`, `c2` for `W1`, `W2`, `W3`: the unifier named the abstractions after the eigenvariable `c` of the clause.

The book traces a smaller query step by step, `trans 1 (abs x\ abs y\ abs z\ y) P`, and ends with "the instantiation of P with (ab (ab (ab (deb 2))))" [MN p. 193–194]. We ran it separately:

```
(define t3 `(abs ,(ƛ (x) `(abs ,(ƛ (y) `(abs ,(ƛ (z) y)))))))
(show "trans 1 (abs x\\ abs y\\ abs z\\ y) P"
      (°->list/datum (fresh° (q) (trans° 1 t3 q))))
```

```
;; trans 1 (abs x\ abs y\ abs z\ y) P
((ab (ab (ab (deb 2)))))
```

Tutorial: the book's trace as state changes

Read `depth` for the level of the `∀°`, and Γ for the `depth°` hypotheses.

goal	depth	Γ for `depth°` (most recent first)
`trans 1 (abs x\ abs y\ abs z\ y) P`	0	—
`trans 2 (abs y\ abs z\ y) P1`	1	$(1\ c_1)$
`trans 3 (abs z\ c2) P2`	2	$(2\ c_2)$, $(1\ c_1)$
`trans 4 c2 P3`	3	$(3\ c_3)$, $(2\ c_2)$, $(1\ c_1)$

(We write c_1, c_2, c_3 for the three eigenvariables, all hinted `c`.) In the last goal only the third clause applies, since c_2 is neither an `abs` nor an `app` list. `(depth° n c2)` backchains on Γ, most recent first: $(3\ c_3)$ and $(1\ c_1)$ fail on $c_3 \neq c_2$ and $c_1 \neq c_2$, while $(2\ c_2)$ gives $n = 2$, and `project°` computes $e = 4 - 2 = 2$. Each `∀°` then exits, restoring the depth and Γ. The book's `depth c 1` is the same fact with its arguments in the order of the trace; the clause and our code write `depth D c`.

In the backward direction the first argument of `trans°` is $M\,c$ with M unknown, a pattern. The `app` clause solves $M\,c \doteq (\text{app}\ M_1\ N_1)$ by FR, lowering M_1, N_1 and raising them over c. The `deb` clause then meets `(depth° n (M1' c))`: the hypothesis `(depth° 1 c)` gives $M_1'\,c \doteq c$, a projection. Every equation in this program is a pattern.

13.7.2 Prenex normal forms (book Fig. 7.8)

Section 6.6.3 showed that `prenex` is a relation, not a function:

```
?- prenex ((all x\ q x x) && (all z\ all y\ q z y)) P.
```

"will generate the following five answer substitutions for P" [MN p. 197]:

```
all z\ all y\ (q z z) && (q z y)
all x\ all z\ all y\ (q x x) && (q z y)
all z\ all x\ (q x x) && (q z x)
all z\ all x\ all y\ (q x x) && (q z y)
all z\ all y\ all x\ (q x x) && (q z y)
```

The program of `test/programs.scm` transcribes Figure 7.8 in the book's order, one `cond°` branch per clause, except that clauses differing only in their connective or quantifier (such as `prenex (B && C) D` and `prenex (B !! C) D`) share one branch through an `or°`. We show the main relation and the first two of the seven `merge°` branches:

```
(define-relation (prenex° b d)
  (cond°
    ((atom° b) (=° d b))
    ((fresh° (a) (=° b `(neg ,a)) (atom° a) (=° d b)))
    ((fresh° (op B C U V) (=° b `(,op ,B ,C)) (or° (=° op 'and) (=° op 'or)) (prenex°
     ↪  B U) (prenex° C V) (merge° `(,op ,U ,V) d)))
```

```
    ((fresh° (q B D) (=° b `(,q ,B)) (=° d `(,q ,D)) (or° (=° q 'all) (=° q 'some))
     ↪  (∀° (x) (prenex° (@ B x) (@ D x)))))))
(define-relation (merge° b d)
  (cond°
    ((fresh° (B C D) (=° b `(and (all ,B) (all ,C))) (=° d `(all ,D))
       (∀° (x) (merge° `(and ,(@ B x) ,(@ C x)) (@ D x)))))
    ((fresh° (q B C D) (=° b `(and (,q ,B) ,C)) (=° d `(,q ,D)) (or° (=° q 'all) (=°
     ↪  q 'some))
       (∀° (x) (merge° `(and ,(@ B x) ,C) (@ D x)))))
```

```
(show "prenex is a relation: the book's five answers (here in the order 1 3 2 4 5)"
      (°->list/datum (fresh° (q) (prenex° `(and (all ,(ƛ (x) `(q ,x ,x))) (all ,(ƛ
       ↪  (z) `(all ,(ƛ (y) `(q ,z ,y)))))) q))))
```

```
;; prenex is a relation: the book's five answers (here in the order 1 3 2 4 5)
((all (ƛ (x) (all (ƛ (x1) (and (q x x) (q x x1))))))
 (all (ƛ (x) (all (ƛ (x1) (and (q x1 x1) (q x x1))))))
 (all (ƛ (x) (all (ƛ (x1) (all (ƛ (x2) (and (q x x) (q x1 x2)))))))
 (all (ƛ (x) (all (ƛ (x1) (all (ƛ (x2) (and (q x1 x1) (q x x2)))))))
 (all (ƛ (x) (all (ƛ (x1) (all (ƛ (x2) (and (q x2 x2) (q x x1)))))))))
```

Tutorial: matching the answers to the book's

Rename the binders: our second answer, $\forall x \forall x_1 . (q\, x_1\, x_1) \wedge (q\, x\, x_1)$, is the book's third, `all z\ all x\ (q x x) && (q z x)`; our third is the book's second. The other three are in the book's position. So all five forms are found, in the order 1 3 2 4 5.

Why the difference? `merge°` has three clauses that apply to a conjunction of two universals: the one that merges both quantifiers into one (`(all B) && (all C)`), and the two that pull out only the left or only the right one. λProlog tries them depth first, so all the answers of the first clause come before those of the second. micro interleaves the branches of `cond°`, so an answer that a later branch reaches in fewer steps can come first (Sections 2.12.1 and 10.6). The set of answers is the book's, and each answer is found once. The unique prenex form of the book's first example [MN p. 196] comes out unique too (we ran it separately):

```
(define f1 `(or (all ,(ƛ (x) `(and (and (p ,x) (all ,(ƛ (y) `(q ,x ,y)))) (p
 ↪  (f ,x))))) (p a)))
(show "prenex: the book's formula with a unique prenex form"
      (°->list/datum (fresh° (q) (prenex° f1 q))))
```

```
;; prenex: the book's formula with a unique prenex form
((all (ƛ (x) (all (ƛ (x1) (or (and (and (p x) (q x x1)) (p (f x))) (p a)))))))
```

(We read the book's `&&` as left-associative.) The answer is the book's `all x\ all y\ ((p x) && (q x y) && (p (f x))) !! (p a)`. Every equation in this program is a pattern: `(@ B x)` and `(@ D x)` apply a variable to the eigenvariable of the enclosing `∀°`.

13.8 Higher-order programming

13.8.1 `mappred`

Example 5.8 introduced `mappred` (Figures 5.3–5.4):

```
mappred P nil nil.
mappred P (X::L) (Y::K) :- P X Y, mappred P L K.
age bob 23 & age sue 24 & age ned 23.

?- mappred (x\y\ age x y) (ned::bob::sue::nil) L.
L = (23::23::24::nil)
?- mappred (x\y\ age y x) (23::24::nil) K.
K = (bob::sue::nil);
K = (ned::sue::nil)
```

These are the clauses of [MN p. 127] and the session of [MN p. 129]. A predicate argument is a Scheme procedure here: `age°` itself for `x\y\ age x y`, which is η-equal to `age`, and a Scheme closure for the swapped version.

```
; §5.3, Figures 5.3-5.4 (p.143): mappred with a relation argument, a Scheme closure
↪  here.
(define-relation (age° n a) (cond° ((=° n 'bob) (=° a 23)) ((=° n 'sue) (=° a 24))
↪  ((=° n 'ned) (=° a 23))))
(define-relation (mappred° p l k)
  (cond°
    ((null° l) (null° k))
    ((fresh° (x xs y ys) (=° l `(,x . ,xs)) (=° k `(,y . ,ys)) (p x y) (mappred° p xs
     ↪  ys)))))
```

```
(show "mappred (p.143, p.145)"
      (list (°->list/datum (fresh° (q) (mappred° age° '(ned bob sue) q)))
            (°->list/datum (fresh° (q) (mappred° (λ (x y) (age° y x)) '(23 24) q)))))
```

```
;; mappred (p.143, p.145)
(((23 23 24)) ((bob sue) (ned sue)))
```

Both answers, and their order, are the book's. Nothing of the λ layer is involved: this is the higher-order programming that any Scheme-hosted miniKanren has for free (Section 5.11). The price is the one stated in Section 14.2.1: a predicate is a closure, not a term, so a program cannot unify predicates, quantify over them (`pi rev\`), or assume clauses for a predicate variable.

13.8.2 Palindromes with functional difference lists (book Fig. 5.11)

Section 5.7.2 represented a list by an abstraction over its tail, so that both ends can be reached in one unification step:

```
palindrome (fdl x\x).
palindrome (fdl x\ Y::x).
palindrome (fdl x\ Y::(F (Y::x))) :- palindrome (fdl F).
```

```
?- palindrome (fdl x\ a::b::c::b::a::x).
solved
?- palindrome (fdl x\ a::b::a::(F x)).
F = x\ x;
F = x\ b::a::x;
F = x\ a::b::a::x;
F = x\ Y::a::b::a::x;
F = x\ Z::Z::a::b::a::x;
F = x\ Y::Z::Y::a::b::a::x;
F = x\ Y::Z::Z::Y::a::b::a::x;
F = x\ Y::Z::U::Z::Y::a::b::a::x;
F = x\ Y::Z::U::U::Z::Y::a::b::a::x
```

This is the program and the session of [MN p. 142–143]. The third clause is outside L_λ: F is applied to `Y::x`, which is not an eigenvariable.

```
; §5.8.2, Figure 5.11 (p.158): a functional difference list is an abstraction over
 ↪  its tail, and the
; palindrome clauses destructure one by unification, `x\ Y::(F (Y::x))` being no
 ↪  pattern: it waits.
(define-relation (palindrome° f)
  (cond°
    ((=° f `(fdl ,(ƛ (x) x))))
    ((fresh° (Y) (=° f `(fdl ,(ƛ (x) `(,Y . ,x))))))
    ((fresh° (Y F) (=° f `(fdl ,(ƛ (x) `(,Y . ,(@ F `(,Y . ,x))))))) (palindrome°
     ↪  `(fdl ,F))))))
```

```
(show "functional difference lists: palindromes (p.158), the book's first nine
 ↪  answers in its order"
      (list (°->list/datum (fresh° (q) (palindrome° `(fdl ,(ƛ (x) `(a b c b a .
       ↪  ,x)))) (=° q 'solved)))
            (°->list/datum (take° 9 (fresh° (q) (palindrome° `(fdl ,(ƛ (x) `(a b a .
             ↪  ,(@ q x))))))))))
```

```
;; functional difference lists: palindromes (p.158), the book's first nine answers in its order
((solved)
 ((ƛ (x) x)
  (ƛ (x) (b a . x))
  (ƛ (x) (a b a . x))
  (ƛ (x) (α a b a . x))
  (ƛ (x) (α α a b a . x))
  (ƛ (x) (α β α a b a . x))
  (ƛ (x) (α β β α a b a . x))
  (ƛ (x) (α β γ β α a b a . x))
  (ƛ (x) (α β γ γ β α a b a . x))))
```

Tutorial: a non-pattern that waits for the recursive call

Follow the third clause on the query `fdl x\ a::b::a::(F x)`, our `q`.

1. micro unifies the two lists `(fdl ...)` cell by cell, and hands the two abstractions to the hook. ξ opens them with one name c (level ∞): $a :: b :: a :: (q\,c) \doteq Y :: (F\,(Y :: c))$.

2. The pairs are unified cell by cell: $Y := a$, and $b :: a :: (q\,c) \doteq F\,(a :: c)$. The right side applies F to a list, so it is no pattern; the left side is rigid. No side is a pattern, so FR cannot fire, and DELAY suspends the equation, closed over the opened name c: $\lambda c.\, b :: a :: (q\,c) \doteq \lambda c.\, F\,(a :: c)$.

3. The recursive call `(palindrome° `(fdl ,F))` binds F in each of its branches. Each binding retries the suspended equation, which has become a pattern equation for q, or a failure. The second clause, for instance, binds $F := \lambda x.\, Y' :: x$. The equation becomes $b :: a :: (q\,c) \doteq Y' :: a :: c$, so $Y' := b$ and $q\,c \doteq c$, and FR gives $q := \lambda x.\, x$: the first answer, the list `a::b::a`.

The first clause, $F := \lambda x.\, x$, fails on $b \doteq a$; deeper recursions give the longer answers. The answers are the book's nine, in the book's order, with α, β, γ for the book's variables in order of first occurrence (so the fifth answer's α is the book's `Z`). A procedure that rejected non-patterns, as ELPI does by default, would lose this program (Section 11.2).

13.9 Fischer's CPS transformation (book Fig. 10.6)

Section 8.2.5 presented the book's relational CPS transformation. It has no clause for variables: going under a binder, `phi` assumes how the bound variable translates.

```
ftrans (abs V) (abs k\ k @ U) :- phi (abs V) U.
ftrans (M @ N) (abs k\ P @ (abs m\ Q @ (abs n\ m @ k @ n))) :-
   ftrans M P, ftrans N Q.
phi (abs M) (abs k\ abs x\ (P x) @ k) :-
   pi x\ pi y\ ftrans x (abs k\ k @ y) => ftrans (M x) (P y).

?- ftrans ((abs u\u) @ (abs u\u)) F.
F = (abs W1\ (abs (W2\ W2 @ abs W3\ abs W4\ abs (W5\ W5 @ W4) @ W3)) @
     (abs W2\ (abs (W3\ W3 @ abs W4\ abs W5\ abs (W6\ W6 @ W5) @ W4)) @
     (abs W3\ W2 @ W1 @ W3))).
```

This is Figure 10.6 and the session of [MN p. 257]. The object application `M @ N` is the list `(app M N)`:

```
; §10.3.2, Figure 10.6 (p.273): Fischer's CPS transformation; object application `M
↪  @ N` is `(app M N)`.
; There is no clause for variables: going under a binder assumes ftrans x (abs k\ k @
↪  y).
```

```
(define-dynamic-relation (ftrans° m f)
  (cond°
    ((fresh° (V U) (=° m `(abs ,V)) (=° f `(abs ,(ƛ (k) `(app ,k ,U)))) (phi° m U)))
    ((fresh° (M N P Q) (=° m `(app ,M ,N)) (ftrans° M P) (ftrans° N Q)
       (=° f `(abs ,(ƛ (k) `(app ,P (abs ,(ƛ (m) `(app ,Q (abs ,(ƛ (n) `(app (app ,m
        ↪  ,k) ,n)))))))))))))
(define-relation (phi° m u)
  (fresh° (M P) (=° m `(abs ,M)) (=° u `(abs ,(ƛ (k) `(abs ,(ƛ (x) `(app ,(@ P x)
  ↪  ,k))))))
    (∀° (x y) (⊃° ((ftrans° x `(abs ,(ƛ (k) `(app ,k ,y))))) (ftrans° (@ M x) (@ P
     ↪  y))))))
```

```
(show "ftrans ((abs u\\u) @ (abs u\\u)) F" (°->list/datum (fresh° (q) (ftrans° `(app
↪  (abs ,(ƛ (u) u)) (abs ,(ƛ (u) u))) q))))
```

```
;; ftrans ((abs u\u) @ (abs u\u)) F
((abs (ƛ (k)
        (app (abs (ƛ (k1)
                    (app k1
                         (abs (ƛ (k2)
                                 (abs (ƛ (x) (app (abs (ƛ (k3) (app k3 x))) k2))))))))
             (abs (ƛ (m)
                    (app (abs (ƛ (k1)
                                 (app k1
                                      (abs (ƛ (k2)
                                              (abs (ƛ (x) (app (abs (ƛ (k3) (app k3 x)))
                                               ↪  k2))))))))
                         (abs (ƛ (n) (app (app m k) n)))))))))))
```

Tutorial: reading the answer against the book's term

Map `k` to `W1` and `m` to the outer `W2`. The operator part `(abs (ƛ (k1) (app k1 (abs (ƛ (k2) (abs (ƛ (x) (app (abs (ƛ (k3) (app k3 x))) k2))))))))` is the book's `abs (W2\ W2 @ abs W3\ abs W4\ abs (W5\ W5 @ W4) @ W3)` with `k1`, `k2`, `x`, `k3` for `W2`, `W3`, `W4`, `W5`. The final continuation `(abs (ƛ (n) (app (app m k) n)))` is `abs W3\ W2 @ W1 @ W3`. So the answer is exactly the book's term up to α. The suite compares it with `equal?` against this datum (A7/cps).

The steps that involve the λ layer are in `phi°`, called on `abs u\u`:

1. $M := \lambda u.\, u$, and the second argument of `phi°`, the variable `u` of the clause, is bound to `` `(abs ,(ƛ (k) `(abs ,(ƛ (x) `(app ,(@ P x) ,k))))) ``. Here `(@ P x)` is built inside `ƛ`, with `x` the placeholder of an object binder: it is the flexible term P applied to a bound variable.

2. GENERIC twice: eigenvariables x, y. AUGMENT: the clause `(ftrans° x `(abs ,(ƛ (k) `(app ,k ,y))))`.

3. The goal `(ftrans° (@ M x) (@ P y))`, that is, `ftrans°` $x\ (P\ y)$, since $M\ x$ reduces to x. BACKCHAIN on Γ first: $P\ y \doteq \mathrm{abs}\,(\lambda k.\, \mathrm{app}\ k\ y)$ is a pattern, $\ell(y) = 2 > \ell(P)$. FR

projects y and imitates the rest: $P := \lambda y.\, \mathrm{abs}\,(\lambda k.\, \mathrm{app}\ k\ y)$. The static clauses fail on x, which is neither an `abs` nor an `app` list.

4. Once P is bound, the subterm $P\,x$ of `u`, under the object binder x, reduces to `(abs (ƛ (k3) (app k3 x)))`, the book's `abs (W5\ W5 @ W4)`.

13.10 Higher-order pattern unification

The book's Chapter 8 develops pattern unification on small problems with explicit quantifier prefixes (Chapter 7). In the module a prefix is written with goals: a variable created by `fresh°` outside a `∀°` is an outer ∃, an eigenvariable is a ∀. So $\exists F\,\forall x\,\forall y\,[\ldots]$ is `(fresh° (F) (∀° (x y) ...))`. We use `q` for the first variable, since it is the one the runners print. The labels 8.A–8.M are those of Section 7.7.

13.10.1 Flex-rigid and flex-flex

```
;; ?- pi x\ F x = g x x.                    F = x\ g x x
;; ?- pi x\ pi y\ F x y = G y x.            F = x\ y\ H y x,  G = H
;; ?- pi x\ pi y\ F x y = F y x.            F = x\ y\ H
(show "flex-rigid and flex-flex"
      (list (°->list/datum (fresh° (q) (∀° (x) (=° (@ q x) `(g ,x ,x)))))
            (°->list/datum (fresh° (q f g) (=° q (list f g)) (∀° (x y) (=° (@ f x y)
             ↪  (@ g y x)))))
            (°->list/datum (fresh° (q) (∀° (x y) (=° (@ q x y) (@ q y x)))))))
```

```
;; flex-rigid and flex-flex
(((ƛ (x) (g x x))) (((ƛ (x y) (@ α y x)) α)) ((ƛ (x y) α)))
```

Tutorial: three rules, three answers

1. $q\,x \doteq g\,x\,x$: the left side is a pattern ($\ell(x) = 1 > \ell(q) = 0$), the right side is rigid. FR replaces each x by the bound variable: $q := \lambda x.\, g\,x\,x$.

2. $f\,x\,y \doteq g\,y\,x$: two patterns with different heads, so FR on f (the case FF-diff). Elimination of f keeps g with both its arguments, since both are among f's, and nothing needs pruning: $f := \lambda x\,y.\, g\,y\,x$, and g stays free. The book's H is our g, printed `α`.

3. $q\,x\,y \doteq q\,y\,x$: FF-same. The arguments agree in no position, so $q := \lambda x\,y.\, H$: a constant function ([MN p. 222]; Section 7.5.2).

13.10.2 Example 8.H: pruning

```
∀f ∀g ∃U ∃V ∀w ∀x ∀y [(f (U x y)) = (f (g (V y w)))]
```

“variable elimination becomes applicable. Using it first generates the pruning substitution λy λw (V′ y) for V and subsequently the substitution λx λy (g (V′ y)) for U” [MN p. 223–224]. We solved it by hand in Example 7.23.

```
;; Example 8.H (p.239-240): ?- pi w\ pi x\ pi y\ f (U x y) = f (g (V y w)).
;; U = x\ y\ g (V' y),  V = y\ w\ V' y
(show "Example 8.H, pruning: (U V)" (°->list/datum (fresh° (q u v) (=° q (list u v))
 ↪  (∀° (w x y) (=° `(f ,(@ u x y)) `(f (g ,(@ v y w))))))))
```

```
;; Example 8.H, pruning: (U V)
(((ƛ (x y) (g (@ α y))) (ƛ (y w) (@ α y))))
```

Tutorial: 8.H with levels

The prefix gives $\ell(u) = \ell(v) = 0$ and $\ell(w) = 1$, $\ell(x) = 2$, $\ell(y) = 3$. Here `f` is the symbol `'f` at the head of a list, a constant of the signature. The book quantifies f and g universally at the front of the prefix, before U and V; an eigenvariable that every variable may see behaves like a constant, so nothing is lost.

1. micro unifies the two lists `(f ...)`: `f` against `f`, then the hook gets $u\,x\,y \doteq g\,(v\,y\,w)$. The left side is a pattern, so FR runs $\mathrm{Elim}(0, u, (x, y),\ g\,(v\,y\,w))$.
2. The walk keeps the constant `g`. It meets the pattern $v\,y\,w$: y is among u's arguments and is kept; w is not, and $\ell(w) = 1 > 0$, so it must go. *Pruning* binds $v := \lambda y\,w.\,v'\,y$, with $\ell(v') = 0$.
3. The result $g\,(v'\,y)$ is abstracted over u's arguments: $u := \lambda x\,y.\,g\,(v'\,y)$.

The answer is the book's mgu, with `α` for V'. The binders of v's value are named `y` and `w`, after the eigenvariables of the query.

13.10.3 Example 8.K: delayed raising

```
∀f ∃U ∀x ∀y ∀z ∃V [(U x) = (f V)]
```

“we would replace V with the term (V′ x), where V′ is a new variable quantified at the same place in the prefix as U and would produce the unifying substitution λx (f (V′ x)) for U” [MN p. 225]; see Example 7.31.

```
;; Example 8.K (p.240-241): ?- pi x\ pi y\ pi z\ sigma V\ U x = f V.      U = x\ f
 ↪  (V' x)
(show "Example 8.K, delayed raising" (°->list/datum (fresh° (q) (∀° (x y z) (fresh°
 ↪  (v) (=° (@ q x) `(f ,v)))))))
```

```
;; Example 8.K, delayed raising
((ƛ (x) (f (@ α x))))
```

Here $\ell(q) = 0$ and v is created after three $\forall^\circ$, so $\ell(v) = 3$. Elimination of q over the pattern (x) meets v, a variable younger than q. It is *lowered* to q's level and *raised* over those arguments of q that v may see: $\ell(x) = 1 \leq \ell(v)$, so $v := v'\ x$ with $\ell(v') = 0$, and $q := \lambda x.\ f\ (v'\ x)$. No raising over y and z ever happens: the levels record the information that the book proposes to keep "with each existentially quantified variable", and elimination reads it only when it is needed.

13.10.4 Failures: occurs check, scope (8.G, 8.I, 8.J)

```
;; Examples 8.G, 8.I, 8.J: X = f X;  U x y = g (U y w);  U x y = g w.      no, no, no
(show "Examples 8.G (occurs), 8.I (occurs under a flexible head), 8.J (scope)"
      (list (°->list #f (fresh° (q) (=° q `(f ,q))))
            (°->list #f (fresh° (q) (∀° (w x y) (=° (@ q x y) `(g ,(@ q y w))))))
            (°->list #f (fresh° (q) (∀° (w x y) (=° (@ q x y) `(g ,w)))))))

;; ∃F∀x. F = x fails; ∃F∀x. F x = x gives F = λz.z.
(show "scope: F = x versus F x = x"
      (list (°->list #f (fresh° (q) (∀° (x) (=° q x))))
            (°->list/datum (fresh° (q) (∀° (x) (=° (@ q x) x))))))
```

```
;; Examples 8.G (occurs), 8.I (occurs under a flexible head), 8.J (scope)
(() () ())

;; scope: F = x versus F x = x
(() ((ϡ (x) x)))
```

- 8.G, $X \doteq f\ X$, is first-order: micro's own occurs check rejects it.
- 8.I, $U\ x\ y \doteq g\ (U\ y\ w)$ (Example 7.24): elimination of U meets $U\ y\ w$ below the rigid constant g, with as many rigid arguments as $U\ x\ y$ has. No instance of U can make the right side as small as the left, so this is an occurs failure (Nipkow's size argument, case (6a) of Definition 12.6).
- 8.J, $U\ x\ y \doteq g\ w$ (Example 7.25): w is not among U's arguments and $\ell(w) = 1 > \ell(U) = 0$. It appears "in a position that is not the argument of a flexible term" [MN p. 224], so no pruning can remove it: a scope failure.
- $\exists F \forall x.\ F\ =\ x$ is the first query of Section 13.3 again, while $\exists F \forall x.\ F\ x\ =\ x$ is a pattern problem with the projection $F := \lambda x.\ x$ as its answer. The first variable cannot *be* x, but it can be a function that *returns* it.

13.10.5 Non-patterns: suspended, reported, woken

Two of the book's problems have no most general unifier. Example 8.M, $\forall x\ (F\ x\ x = x)$, has the two solutions $\lambda u\ v.\ u$ and $\lambda u\ v.\ v$ [MN p. 208]. Example 8.A, $\forall a \forall g \exists F\ [(F\ a) = (g\ a\ a)]$, has four [MN p. 213] (Example 7.5). Without Huet's procedure the module cannot enumerate them, as Teyjus 1 does. It does what Teyjus 2 does: it suspends the equation and reports it (dynamic L_λ, Section 7.9.1).

```
;; Example 8.M (p.224): pi x\ F x x = x has two incomparable unifiers: it waits, and
↪  is reported.
;; Example 8.A (p.229): F a = g a a, four unifiers: likewise (no Huet enumeration).
(show "non-patterns: 8.M and 8.A, suspended; 8.M woken by F := λa.λb.b"
      (list (°->pretty (fresh° (q) (∀° (x) (=° (@ q x x) x))))
            (°->pretty (fresh° (q) (=° (@ q 'a) '(g a a))))
           (°->list/datum (fresh° (q) (∀° (x) (=° (@ q x x) x)) (=° q (ƛ (a b) b))))))
```

```
;; non-patterns: 8.M and 8.A, suspended; 8.M woken by F := λa.λb.b
(((α : (= (ƛ (x) (@ α x x)) (ƛ (x) x))))
 ((α : (= (@ α a) (g a a))))
 ((ƛ (a b) b)))
```

Tutorial: from suspension to answer

1. $q\,x\,x \doteq x$ repeats x, so it is no pattern: DELAY stores it in E. When the `∀°` exits, DISCHARGE closes it over x, since a residual that mentions x stands for $\forall x\,(u = v)$. It is then retried, and stays suspended. The answer is q unconstrained except for this equation, printed as `(= (ƛ (x) (@ α x x)) (ƛ (x) x))`: no eigenvariable appears free in an answer.

2. In the second query `'a` is a constant, not an eigenvariable, so $q\,a$ is no pattern either. This is 8.A with `a` in the signature, and it is reported as it stands.

3. In the third query the suspended 8.M equation is woken by the binding $q := \lambda a\,b.\,b$: $(\lambda a\,b.\,b)\,x\,x$ reduces to x, and the equation holds. The answer is the value of q, with no residual left.

A program can thus solve a non-pattern problem by *generating* candidates for the unknown function with ordinary goals. The suspended equation then acts as a test. The book's form of 8.A, with a universal and F inside its scope, shows DISCHARGE at work. This is the suite's case A5/non-pattern, run separately:

```
(show "8.A under ∀ a, raised at exit"
      (°->list #f (fresh° (q) (∀° (a) (fresh° (f) (=° (@ f a) `(g ,a ,a)))) (=°
       ↪  q 'ok))))
```

```
;; 8.A under ∀ a, raised at exit
((λ (α)
    (assert (≐ (ƛ (a) (@ α a a)) (ƛ (a) (cons 'g (cons a (cons a '()))))))
    'ok))
```

Here f is created inside the scope of a, so $\ell(f) = 1$, and $f\,a$ is no pattern for it (a is not *younger* than f). At the exit of `∀°`, DISCHARGE raises $f := f'\,a$ with $\ell(f') = 0$, Miller's raising (Section 4.5), and closes the equation into $\lambda a.\,f'\,a\,a \doteq \lambda a.\,g\,a\,a$, over f', which prints as `α`.

13.10.6 Occurrences that must not be pruned

The subtle part of dynamic L_λ is what elimination may do *inside* the arguments of a non-pattern. The book's advice is to "defer further consideration of this equation until such time that substitutions resulting from solving other parts of the problem have altered its status" [MN p. 226]. Two examples of `examples.scm` show why deferring is necessary.

```
;; Dynamic L_λ (p.241-242): G may drop its argument, so nothing inside it is pruned
 ↪  or lowered. The
;; equation ∃q∃G∃H∀y. q = G (H y) waits, closed over y at ∀ exit, and is solved by G
 ↪  := λa.k. Likewise
;; pi y\ F a = F y holds with any constant F: it waits, then F := λz.c solves it, and
 ↪  F := λz.z does not.
(show "non-strict occurrences: q = G (H y), and F a = F y"
      (list (°->pretty (fresh° (q G H) (∀° (y) (=° q (@ G (@ H y))))))
            (°->list/datum (fresh° (q G H) (∀° (y) (=° q (@ G (@ H y)))) (=° G (ƛ (a)
             ↪  'k))))
            (°->pretty (fresh° (q) (∀° (y) (=° (@ q 'a) (@ q y)))))
            (°->list/datum (fresh° (q) (∀° (y) (=° (@ q 'a) (@ q y))) (=° q (ƛ (z)
             ↪  'c))))
            (°->list/datum (fresh° (q) (∀° (y) (=° (@ q 'a) (@ q y))) (=° q (ƛ (z)
             ↪  z))))))
```

```
;; non-strict occurrences: q = G (H y), and F a = F y
(((α : (= (ƛ (y) α) (ƛ (y) (@ β (@ γ y))))))
 (k)
 ((α : (= α (ƛ (y) (@ α a)))))
 ((ƛ (z) c))
 ())
```

Tutorial: why `q = G (H y)` must wait

All of q, G, H have level 0, and $\ell(y) = 1$.

1. micro binds $q := G(H\,y)$, and **Check** looks at the bound term. $G(H\,y)$ is no pattern: its argument is a flexible term, not an eigenvariable. Inside that argument is y, which q may not see. Pruning H's argument ($H := \lambda z.\,H'$) would be a guess: if G ignores its argument, y disappears with no pruning at all, so pruning would lose solutions. The occurrence of y is not *strict* (Definition 12.6 and the tutorial after it, after Dowek, Hardin, Kirchner and Pfenning [8]).

2. So **Check** takes the binding back (`μkanren-state-forget`) and suspends the equation $q \doteq G(H\,y)$. At the exit of `∀°` it is closed over y, and prints as $\lambda y.\,q \doteq \lambda y.\,G(H\,y)$.

3. In the second query $G := \lambda a.\,k$ wakes it: $\lambda y.\,q \doteq \lambda y.\,k$, so $q := k$, and the answer is `(k)`.

The equation $q\,a \doteq q\,y$ is similar. It holds for every constant function q, so failing it as an occurs error, or pruning, would be wrong. Only the right side $q\,y$ is a pattern, so

FF-ONE runs FR on it, with the sides swapped; elimination meets q itself below binders only, and suspends. At the exit the equation is closed over y into $\lambda y.\, q\, y \doteq \lambda y.\, q\, a$, whose left side η-contracts to q: that is the printed `(= α (ϰ (y) (@ α a)))`. $q := \lambda z.\, c$ satisfies it; $q := \lambda z.\, z$ would need $a = y$, and fails.

Pitfall: a residual is a condition, not a proof

An answer with a suspended equation means: *this* is a solution *if* the residual has one. The module does not decide that, since general higher-order unification is undecidable (Section 4.9). Several residuals may even be jointly unsatisfiable, and the answer would still be printed (Section 14.2.3, the case limits/no-huet). Treat `(assert (≐ ...))` in an answer as a question returned to the program. Generating candidates, as in the 8.M query above, is one way to answer it.

13.11 Ma and Friedman 2021, answered by λProlog's semantics

Section 9.2 described Ma and Friedman's proposal [29]. Their unifier treats a logic variable under binders as a *static closure* that may refer to the binders by position, so that `(== (λ (a) X) (λ (b) b))` succeeds with X = `d0`, the level of the binder. Section 9.3.1 gave three counter-examples that compare this reading with pattern unification and with nominal unification. The module takes the other side: a logic variable outside a binder cannot depend on it, as in λProlog. This section runs the paper's queries and states each difference. All the outputs below are from `examples.out`, except the ones marked as run separately.

The paper does not number its code boxes. We call them E1–E12 in their order of appearance, with the paper's page; "Example 1–3" are the paper's own numbered examples of its Section 4.

13.11.1 Where the answers agree

The paper's E2 (p. 2) and E12 (p. 7):

```
> (run 1 X
    (== (λ (a) a) (λ (b) b)))
'(_0)
>(run 1 F
   (all (A)
     (== A (F A))))
'((lambda (_0) _0))
```

(the paper's transcripts). In the module, run separately (the second query is also in `examples.out`):

```
(show "E2: λa.a = λb.b;  E8: λx.λy.x y = λa.λa.a a"
      (list (°->list #f (fresh° (q) (=° (ϰ (a) a) (ϰ (b) b))))
            (°->list #f (fresh° (q) (=° (ϰ (x y) (@ x y)) (ϰ (a a) (@ a a)))))))
(show "E12: (run 1 F (all (A) (== A (F A))))"
      (°->list/datum (fresh° (q) (∀° (a) (=° a (@ q a))))))
```

```
;; E2: λa.a = λb.b;  E8: λx.λy.x y = λa.λa.a a
(((λ (α) α)) ())

;; E12: (run 1 F (all (A) (== A (F A))))
((ϗ (a) a))
```

E2 is α-equivalence of closed terms, decided by ξ. E12 is the pattern $F\,a \doteq a$ under GENERIC, solved by projection, exactly as the paper says: "the == operator recognizes β-equivalence" (p. 7). E8, $\lambda x.\,\lambda y.\,x\,y \doteq \lambda a.\,\lambda a.\,a\,a$, is the paper's second "false positive" of binder-merging (p. 4). Here it fails, as it must: the right side is $\lambda a.\,\lambda b.\,b\,b$, and RIGID meets x against b, two different opened names.

13.11.2 Where the answers differ

```
(show "λa.X = λb.b fails (the paper: X = d0); λa.λa.X = λc.λd.c fails; ∀A. A = F A
↪  gives F = λA.A"
      (list (°->list #f (fresh° (q) (=° (ϗ (a) q) (ϗ (b) b))))
            (°->list #f (fresh° (q) (=° (ϗ (a) (ϗ (a) q)) (ϗ (c) (ϗ (d) c)))))
            (°->list/datum (fresh° (q) (∀° (a) (=° a (@ q a)))))))
```

```
;; λa.X = λb.b fails (the paper: X = d0); λa.λa.X = λc.λd.c fails; ∀A. A = F A gives F = λA.A
(() () ((ϗ (a) a)))
```

and, run separately, the paper's Example 3, the three counter-examples of Section 9.3.1 (the first is the paper's conjunction, the third is shown again with two ground variants), and the paper's `assume` query:

```
(show "their Example 3: λa.λb.X = λc.λd.Y"
     (°->list/datum (fresh° (q X Y) (=° q (list X Y)) (=° (ϗ (a b) X) (ϗ (c d) Y)))))
(show "counter-example 1: λa.X = λb.b ∧ λc.X = λd.d"
      (°->list #f (fresh° (q) (=° (ϗ (a) q) (ϗ (b) b)) (=° (ϗ (c) q) (ϗ (d) d)))))
(show "counter-example 2: Q = X ∧ λa.X = λb.b"
      (°->list #f (fresh° (q x) (=° q x) (=° (ϗ (a) x) (ϗ (b) b)))))
(show "counter-example 3 and its variants"
      (list (°->list #f (fresh° (q) (=° (ϗ (a) (ϗ (a) q)) (ϗ (c) (ϗ (d) c)))))
            (°->list #f (fresh° (q) (=° (ϗ (a) (ϗ (a) a)) (ϗ (c) (ϗ (d) c)))))
            (°->list #f (fresh° (q) (=° (ϗ (a) (ϗ (a) a)) (ϗ (c) (ϗ (d) d))) (=° q
             ↪  'ok)))))
(define-dynamic-relation (eq° a b) (=° a b))
(show "(assume (== apple orange) (== apple X)) with a dynamic eq°"
      (°->list/datum (fresh° (q) (⊃° ((eq° 'apple 'orange)) (eq° 'apple q)))))
```

```
;; their Example 3: λa.λb.X = λc.λd.Y
((α α))

;; counter-example 1: λa.X = λb.b ∧ λc.X = λd.d
()

;; counter-example 2: Q = X ∧ λa.X = λb.b
()
```

```
;; counter-example 3 and its variants
(() () ((λ () 'ok)))

;; (assume (== apple orange) (== apple X)) with a dynamic eq°
(orange apple)
```

For comparison, the paper's transcripts of E5, E6 (p. 3) and E11 (p. 7):

```
> (run 1 X
    (== (λ (a) X) (λ (b) b)))
'(d0)
> (run 1 X
    (== (λ (a) X) (λ (b) b))
    (== (λ (c) X) (λ (d) d)))
'(d0)
>(run 2 X
   (assume (== 'apple 'orange)
     (== 'apple X)))
'(apple orange)
```

Tutorial: each difference, and the reason for it

1. $\lambda a.\, X \doteq \lambda b.\, b$ (E3–E5, p. 2–3). The paper answers `(d0)`. Here ξ opens both sides with a new name c and leaves $X \doteq c$, which **Check** rejects: c was opened below a binder that X is outside of. Read with quantifiers, the equation is $\exists X\, \forall c.\, X = c$, the first query of Section 13.3 and Example 8.J with no arguments, which is false: no single X equals every c ([MN p. 85]; Example 7.25). The paper itself grants that this failure is what every existing algorithm answers: "Under any existing higher-order unification algorithm, X and a cannot be unified due to the scope restrictions" (p. 2). It then calls the failure "counter-intuitive" (p. 3).

2. **The conjunction** with $\lambda c.\, X \doteq \lambda d.\, d$ (E6, p. 3). The paper again answers `(d0)`, by the same grafting. Both conjuncts fail here.

3. $Q = X \wedge \lambda a.\, X \doteq \lambda b.\, b$ (counter-example 2). With static closures, Q would be bound to a level that is not available where Q lives. Here it fails like item 1. Under the quantifier reading the variables have no positions to refer to, so this problem cannot arise.

4. $\lambda a.\, \lambda a.\, X \doteq \lambda c.\, \lambda d.\, c$ (E7, p. 4). It fails, in the paper's algorithm too (their Example 2). The paper's warning is about binder-merging implementations that turn the *ground* term $\lambda a.\, \lambda a.\, a$, which is *false*, into *true*. Here the inner `a` shadows the outer one by Scheme's scoping, and the printer renames apart (Section 13.2). So *false* = *true* fails, and *false* = *false* succeeds: the second and third answers.

5. **Example 3,** $\lambda a.\, \lambda b.\, X \doteq \lambda c.\, \lambda d.\, Y$ (p. 6). The paper needs a *context*, a postponed equation between closures, because X and Y might refer to the binders by position.

Here neither may depend on the binders, so the equation is simply $X \doteq Y$, solved by VAR: `((α α))`, with no postponed part.

6. **`assume`** (E11, p. 7). The paper's query, shown above, gives `'(apple orange)`. The paper gives no operational account; the natural reading is ordinary unification first, then the hypothesis. λProlog cannot write this query, since `=` is not a predicate one can assume clauses for. With a dynamic `eq°` whose static body is `=°`, the hypothesis comes *first*, by the rule of [MN p. 78] (Section 13.4.1): `(orange apple)`.

Kanren angle: what the comparison shows

The two systems agree wherever the problem is about closed terms or about patterns under `∀`: α, the "true = false" hazard of ground terms, and E12. On unification they differ exactly where a logic variable occurs under an object binder that it is not applied to (the order of the `assume` answers is a separate matter, of clause order). There, Ma and Friedman give the variable access to the binders by position. The module keeps the quantifier reading: a variable outside a binder cannot see it, and a variable that should depend on a binder must be applied to it. We ran

```
(show "λa.F a = λb.b: a variable applied to the binder"
      (°->list/datum (fresh° (q) (=° (ƛ (a) (@ q a)) (ƛ (b) b)))))
```

```
;; λa.F a = λb.b: a variable applied to the binder
((ƛ (b) b))
```

Read with quantifiers this is $\exists q\, \forall a.\, q\, a = a$, a pattern problem whose answer is the projection. The design options are compared in Section 9.8, and the choice is argued in Chapter 11.

13.12 The examples at a glance

One row per example: the book's page, the place in Part I where the example was introduced, and how the module's answers compare with λProlog's. "Same" means the same answers in the same order.

example	book	Part I	order of the answers against λProlog
∃∀ / ∀∃, aliases	[MN p. 85]	Section 3.4.3	failure/success as in the book
clause order	[MN p. 78]	Section 3.2	same: `(3 2 1)`, `(2 3 1)`
courses	[MN p. 80–81]	Section 3.3.2	same answers; all proofs listed, the book shows the first

example	book	Part I	order of the answers against λProlog
jars	[MN p. 83]	Section 3.4	same
reverse	[MN p. 87]	Section 3.5	same; one answer backwards
typeof	[MN p. 190–191]	Section 6.5.4	same; backwards finds terms where DFS loops
cbn	[MN p. 183]	Section 6.3.2	same
copy, subst	[MN p. 199–200]	Section 6.7	same
existential generalisation	[MN p. 202]	Section 6.7	same four, same order
de Bruijn	[MN p. 193–194]	Section 6.5.5	same terms, both directions
prenex	[MN p. 196–197]	Section 6.6.3	same five, in the order 1 3 2 4 5
mappred	[MN p. 129]	Example 5.8	same
palindromes	[MN p. 142–143]	Section 5.7.2	same nine, same order
CPS	[MN p. 257]	Section 8.2.5	same term
8.H, 8.K	[MN p. 223–225]	Examples 7.23 and 7.31	the book's mgus
8.G, 8.I, 8.J	[MN p. 222,224]	Examples 7.24 and 7.25	failures
8.M, 8.A	[MN p. 208,213]	Example 7.5	suspended and reported (no enumeration)
Ma–Friedman	—	Section 9.3.1	λProlog's answers; differences above

13.13 Summary

- Every core example of the book runs in `(aux kanren lambda)` with the book's answers: the hypothetical queries of Chapter 3, type inference, substitution, the de Bruijn translation, prenex forms, palindromes and the CPS transformation, and the unification problems of Chapter 8 (the two without a most general unifier are reported, not solved).
- The answer *order* is the book's in every example except prenex normal forms (1 3 2 4 5) and the courses query, whose duplicate proofs come in the interleaving's order. Answers come in clause order when every clause answers at once, because a dynamic relation is one flat disjunction: hypotheses most recent first, then the static body.
- Scope is enforced by levels at every binding. `∃Y∀x. Y = x` fails, `∀x∃Y. Y = x` succeeds, and an eigenvariable cannot escape through an alias, through a relation built on `=°`, or through the variables of a clause: lowering gives each such variable the level of the variable it is bound into.

- Clauses assumed with `⊃°` share their free variables with the context, as `rev nil K` and `fact (finished dana X)` need. Their own `∀` variables are renamed at each use.
- β is applied on demand: `(@ R N)` built with `R` unknown reduces as soon as `R` is bound. Call-by-name evaluation works, and `subst R N (R N)` works with `R` bound before or after the equation.
- Pattern problems give the book's most general unifiers, read off the levels: projection, imitation, pruning (8.H), lowering with delayed raising (8.K), and the occurs and scope failures (8.I, 8.J).
- Non-patterns are suspended, closed over the eigenvariables that leave their scope, retried at every binding, and reported as `(assert (≐ ...))`. That is enough for the palindrome program and for generate-and-test over 8.M, but not for enumerating the unifiers of 8.M or 8.A.
- Against Ma and Friedman 2021 the module gives λProlog's answers. It agrees on α, on the "true = false" hazard and on E12. It disagrees where a variable occurs under a binder it is not applied to (`λa.X = λb.b` fails), and on the order of the answers of `assume`.

14 Limitations, costs and a roadmap

What and why

Chapters 11 to 13 presented the proposal from the inside. The first gave the extension protocol added to (aux kanren micro), the second the module `(aux kanren lambda)` built on it, and the third the book's programs running beside their λProlog originals. This chapter looks at the proposal from the outside, as a reviewer would before a merge. It asks five questions.

1. *How do we know it works?* What the test suites check, how a case is written, and what a green run does and does not establish (Section 14.1).
2. *Where does it differ from λProlog?* Nine documented limitations and deviations, each pinned by a test case, most with a small run (Section 14.2).
3. *What does it cost?* The measured overhead on first-order search, after a `∀°`, and on λ workloads, and where that time goes (Section 14.3).
4. *How does it enter the repository?* A staged merge, file by file, with a review order (Section 14.4), and six later work items (Section 14.5).
5. *What must the author decide?* The questions that only the maintainer of aux can answer (Section 14.6).

Every Scheme snippet below was run against the prototype: micro with the protocol applied, plus the module. Its output is pasted verbatim. The prototype will live in `doc/hol-kanren/prototype/`: the module, `micro.diff`, `show.diff`, the design notes `DESIGN.md`, the tests (with the two copies of micro's suites, `micro-tests/` and `micro-tests-with-lambda/`), the examples, the benchmarks, and the scripts `build.sh` and `run-tests.sh`. After the merge it moves to `src/aux.kanren.lambda.scm` and `src/test/microkanren-lambda*.scm`. The snippets import `(aux base)`, `(aux stream)`, (aux kanren micro), `(aux kanren micro show)` and `(aux kanren lambda)`.

Page numbers "p.N" in the prototype's comments are PDF pages (see Section 1.5).

14.1 How the proposal is tested

14.1.1 Three layers of tests

The tests come in three layers, each with its own job.

Acceptance (A1–A9). The acceptance criteria that the three prototypes shared, these cases say what the module must do: the scope of ∀, hypothetical clauses, type inference, object-level substitution, L_λ unification, the differences from Ma and Friedman [29], the larger programs of the book, and micro's stores on λ-terms. A9 says that micro alone has not changed.

Regression (R, R2, R3). The proposal went through three adversarial review rounds, which sent thirteen, eight and twelve reports. Every report was reproduced before it was fixed, and every code fix came with a regression case; each case of the third round fails on the code before its fix. Several reports turned out to describe the same defect, and two were documentation errors or a cost, not code defects. So the rounds left 12, 6 and 9 cases.

Limits. A separate suite, `microkanren-lambda-limits.scm`, pins what the module does *not* do, or does differently from λProlog. Its opening doc string reads "Known limits of (aux kanren lambda), each pinned so that a change of behaviour is noticed." When one of these cases fails, a limitation has changed, perhaps for the better, and the documentation must change with it.

Table 14.1: The two suites of the module, group by group.

group	subject	cases	assertions
A1	∀ scope: escapes, aliases, levels reaching clauses	9	22
A2	⊃ hypothetical clauses: scope, order, sharing; courses, jars, reverse	8	21
A3	∀ and ⊃ together: type inference, forwards and backwards	3	8
A4	object-level substitution: cbn, β, copy, subst, existential generalisation	4	9
A5	L_λ: flex–rigid, flex–flex, pruning, 8.G–8.M, non-patterns, η	10	40
A6	Ma & Friedman 2021, case by case	5	13
A7	de Bruijn, prenex, mappred, palindromes, Fischer's CPS	5	12
A8	micro's stores on λ-terms: `≠°`, `absent°`, types, printing, grouping	11	65
A9	a first-order case inside the suite	1	2
R	the first review round	12	44
R2	the second review round	6	31
R3	the third review round	9	53
	`microkanren-lambda.scm`	83	320
	`microkanren-lambda-limits.scm`	6	13

14.1.2 How a case is written

A case is a list of assertions (`⊨= expected actual`) from `(aux unittest)`. Every expected answer was written by hand, from the book or from the logic, and none was captured from a run. This matters: one prototype had a generated snapshot as its suite, which would have frozen its bugs as expected answers. Here is the A1 case that tests escapes through an alias:

```
((test/A1/alias _)
 ; the escape through a younger variable is caught whatever the order of the
 ↪  bindings and their kind.
```

```
(⊢= '() (°->list #f (fresh° (q) (∀° (x) (fresh° (y) (=° q (list y)) (=° y x))))))
(⊢= '() (°->list #f (fresh° (q) (∀° (x) (fresh° (y) (=° y x) (=° q (list y)))))))
(⊢= '() (°->list #f (fresh° (q) (∀° (x) (fresh° (y z) (=° q (list y)) (=° y z) (=° z
 ↪  x))))))
(⊢= '() (°->list #f (fresh° (q) (∀° (x) (fresh° (y) (=° y q) (=° y x))))))
(⊢= '() (°->list #f (fresh° (q) (∀° (x) (=° (vector q) (vector x)))))))
```

Three habits of the suite show here. First, the *same* property is asserted under several goal orders. Conjunction must be commutative, and many defects found by the reviews broke exactly that: an answer lost in one order of the goals and found in another. Second, the runner states what is compared. `(°->list #f g)` gives micro's answer as rebuilding code, constraints included, while `°->list/datum` gives plain data comparable with `equal?` against an answer quoted from the book. An error is compared through its message. Third, comments cite the book's page or the review report that the case pins.

Tutorial: What the levels do in the second assertion

Both assertions encode $\exists q\, \forall x\, \exists y.\ q = (y) \wedge y = x$, which has no proof: it would need $q = (x)$, and q is quantified outside x. Section 13.3 follows the first order, where **Check** lowers y when $q := (y)$ is made and then rejects $y' := x$. Here $\ell(q) = 0$ and $\ell(x) = \ell(y) = 1$ (Definition 11.1), and `(=° y x)` comes first. $\ell(y) = 1 = \mathit{top}$, the highest level reached (Definition 12.4), and a variable of level *top* sees every eigenvariable that exists, so $y := x$ is not checked (a fast path, Section 12.3.6). Then `(=° q (list y))` binds $q := (y)$; the check (Definition 12.7) walks (y) to (x) and finds $\ell(x) = 1 > \ell(q) = 0$. No answer. The two orders fail at different bindings, for the same reason. The other three assertions vary the route: a chain of two variables, a var–var binding in the other direction, a vector instead of a list.

14.1.3 micro's own suites, run twice

Acceptance item A9 lives mostly outside the module's suite, which holds only one first-order case under that name. A9 is micro's six existing suites, 125 cases in all, run twice against the edited micro: the core suite and those of aggregation, untagged terms, show, arithmetic and dates. The first copy is unchanged. The second differs only in its import line, which also loads the module, so that the extension is *registered* while first-order code runs. For the core suite:

```
5c5
<   (aux unittest) (aux base) (aux stream) (aux kanren micro) (aux fds sbral))
---
>   (aux unittest) (aux base) (aux stream) (aux kanren micro) (aux kanren lambda) (aux fds
 ↪  sbral))
```

The expectations are not touched, answer order included. A seventh suite, the one of union-find, tests `(aux kanren unionfind)`, a separate engine that does not import micro, so it was not run.

14.1.4 The run

The script `build.sh` compiles the edited micro, its sibling modules (show, arith, date) and the module with `csc -O2 -d1 -s -J` into a private repository, `repo/` beside the script, that shadows the installed micro. The script `run-tests.sh` then runs the two suites of the module, the six suites of micro twice (from `micro-tests/` and `micro-tests-with-lambda/`), and the curated examples, with the private repository first on the search path:

```
cd doc/hol-kanren/prototype
sh build.sh
sh run-tests.sh
```

Both scripts set `CHICKEN_REPOSITORY_PATH` to `repo/` followed by the installed repository, and `build.sh` passes `-I` for the installed headers; both paths are hard-coded to `~/.local` and must be adapted on another machine. Nothing is installed. The output of `run-tests.sh`, verbatim:

```
test/microkanren-lambda ((ran 83) (failed 0))
test/microkanren-lambda-limits ((ran 6) (failed 0))
micro-tests/microkanren ((ran 24) (failed 0))
micro-tests/microkanren-aggregation ((ran 6) (failed 0))
micro-tests/microkanren-untagged ((ran 7) (failed 0))
micro-tests/microkanren-show ((ran 15) (failed 0))
micro-tests/microkanren-arith ((ran 52) (failed 0))
micro-tests/microkanren-date ((ran 21) (failed 0))
micro-tests-with-lambda/microkanren ((ran 24) (failed 0))
micro-tests-with-lambda/microkanren-aggregation ((ran 6) (failed 0))
micro-tests-with-lambda/microkanren-untagged ((ran 7) (failed 0))
micro-tests-with-lambda/microkanren-show ((ran 15) (failed 0))
micro-tests-with-lambda/microkanren-arith ((ran 52) (failed 0))
micro-tests-with-lambda/microkanren-date ((ran 21) (failed 0))
examples.scm ok
```

That is $83 + 6 = 89$ cases of the module, plus micro's $24 + 6 + 7 + 15 + 52 + 21 = 125$ cases run twice, 339 cases in all, with 0 failures. The aux repository was never written while the prototype was built: its `git status` shows only the untracked files it had at the start, and both diffs pass `patch --dry-run` against the current `src/`.

Pitfall: What a green run does not say

The suites are example-based. They show that the module gives the book's answers on the book's examples, and that every defect found by review stays fixed. They do not prove soundness. The guarantees on pattern problems come from the theorems the algorithm follows: Miller's termination and most-general-unifier results [34], the soundness and completeness of universe tagging [43], and its levels formulation [44]. The tests check that the code follows those algorithms on the cases tried. And a passing limits suite is no good news in itself: it means the limitations are still there.

14.2 Limitations and deviations from λProlog

λProlog is a typed language with its own implementation of the hohh search. `(aux kanren lambda)` is an untyped library over a miniKanren with interleaving search and constraint stores that λProlog does not have. Some departures follow from that: they are the price of the host. Others are choices that could be revisited. Table 14.2 lists all nine, with the case that pins each. The subsections take them in order.

Table 14.2: The limitations, each pinned by a case.

	limitation	pinned by
1	only dynamic relations take hypotheses; no predicate quantification	`A2/static-relations`
2	untyped: η only at abstractions, `@` distinct from lists, β may diverge	`limits/untyped`
3	non-patterns suspended, never enumerated: floundering reported, not detected	`limits/no-huet`
4	answer order is micro's interleaving	`A7/prenex`, `A3/backwards`
5	`≠°`, `absent°` and types are extensions, with a chosen reading of `∀`	`A8/diseq-extensional`, `R2/extensional-absent-order`
6	host escapes: levels police logic variables, not Scheme values	`limits/host-escape`
7	binder names are hints; some projected constraints keep a working variable	`A6/alpha`, `limits/absent-projection`
8	costs: no indexing of E and N; the scope check after any `∀°`	`bench/` (Section 14.3)
9	the answer variable must be created outside every `∀°`	`R2/answer-inside-forall`

14.2.1 Only dynamic relations take hypotheses

In λProlog every predicate is dynamic: `D => G` may add clauses for any predicate, including one that has static clauses in the program. Here only a relation defined with `define-dynamic-relation` consults the context Γ, and assuming a clause for any other relation is an error. A `⊃°` that silently ignored such a clause would be worse, and all three prototypes did that.

```
(define-relation (r° x) (=° x 1))
(define-dynamic-relation (d° x) (=° x 1))
(error-message (λ () (°->list #f (fresh° (q) (⊃° ((r° 2)) (r° q))))))
(°->list/datum (fresh° (q) (⊃° ((d° 2)) (d° q))))
```

```
"not a dynamic relation, see define-dynamic-relation"
(2 1)
```

The dynamic relation answers 2 before 1, because hypotheses come first, most recent first, then the static body ([MN p. 78], Section 3.2).

Two consequences follow, both because clauses are Scheme closures and not formulas. A program cannot build a clause as a term and then assume it. And there is no quantification over predicates, so the book's `reverse`, which hides its auxiliary `rev` under `pi rev\ ...` [MN p. 125–126] (Definition 5.7), cannot be transcribed as it stands. The suite runs instead the *fohh* `reverse` of Figure 3.4 [MN p. 87], which assumes the clauses of its auxiliary predicate but does not hide its name: here `rev°` is a dynamic relation declared at top level, with no static clauses (`A2/reverse`). Roadmap item (e) addresses predicate quantification (Section 14.5.5).

14.2.2 Untyped terms

λProlog's terms are simply typed, and its unifier works on βη-long normal forms. The module has no types. It keeps terms η-short and η-expands one side only when the other is an abstraction, which is Miller's treatment of the untyped case [34]. Three facts follow.

```
(°->list #f (fresh° (q) (=° (@ 'g 'a) '(g a))))
(µkanren-datum (ƛ (x) (@ 'g x)))
```

```
()
g
```

- λ-tree application `(@ 'g 'a)` is not the list `'(g a)`: the first is a term of the meta-language, the second Scheme data of the object language. Keeping them apart is what keeps conjunction commutative. One prototype conflated them and lost that property.
- $\lambda x.\,(g\ x)$ is built η-short as g.
- Full β through `@` on instantiated heads is not guaranteed to terminate on untyped meta-terms. With types, β-reduction would terminate.

There is also no check that a solution exists in a possibly empty type (Section 4.6). Roadmap item (f) addresses types (Section 14.5.6).

14.2.3 Non-patterns are suspended, never enumerated

Outside L_λ the module does what Teyjus does: it suspends the equation in the store E, retries it after every binding in case it has become a pattern, and reports it in the answer if it is still suspended (Section 7.9). It does not run Huet's procedure (Section 7.4). Two runs show what this means:

```
(°->list #f (fresh° (q F) (=° q F) (=° (@ F 'a) 'b) (=° (@ F 'a) 'c)))
(°->list #f (fresh° (q F) (=° q F) (=° (@ F 'a) '(g a a))))
```

```
((λ (α) (assert (≐ (@ α 'a) 'b)) (assert (≐ (@ α 'a) 'c)) α))
((λ (α) (assert (≐ (@ α 'a) (cons 'g (cons 'a (cons 'a '()))))) α))
```

The second problem is the book's Example 8.A (Example 7.5), $F\,a = g\,a\,a$, written with list data for the right-hand side. It has four unifiers and no most general one [MN p. 213]. The module does not choose: it answers that F is any α satisfying the residual. That answer is correct but conditional.

The first problem shows the cost of not choosing. $F\,a = b$ and $F\,a = c$ have no common solution, since $b \neq c$. Huet's procedure would find that out, but the module only suspends both equations and returns an answer whose residuals are jointly unsatisfiable. This is *floundering reported but not detected*: the answer says what was left undecided, and it is up to the reader, or a later binding, to find out that it is empty. Roadmap item (d) would add an optional Huet search over E (Section 14.5.4).

14.2.4 Answer order is micro's interleaving

λProlog searches depth-first, left to right; micro interleaves the branches of a disjunction. Where the book lists answers, the two orders often agree: augmentation order [MN p. 78], the existential generalisations [MN p. 203], the palindromes [MN p. 142], reverse, the course database (whose extra, duplicate proofs come in the interleaving's order), and the CPS term [MN p. 257]. They differ on `prenex`, whose five answers [MN p. 196–197] come out as the book's first, third, second, fourth and fifth:

```
((all (x) (all (x1) (and (q x x) (q x x1))))))
 (all (x) (all (x1) (and (q x1 x1) (q x x1))))))
 (all (x) (all (x1) (all (x2) (and (q x x) (q x1 x2))))))))
 (all (x) (all (x1) (all (x2) (and (q x1 x1) (q x x2))))))))
 (all (x) (all (x1) (all (x2) (and (q x2 x2) (q x x1)))))))))
```

The difference also helps. Where depth-first search loops, interleaving finds answers that λProlog misses. Asked for terms of type $i \to i$, `typeof` run backwards loops under depth-first search. Here it enumerates terms:

```
((abs (x) x))
 (abs (x) (app (abs (x1) x1)) x)))
 (abs (x) (app (abs (x1) x)) x))))
```

The suite compares answer *sets*, through a sorted copy, wherever interleaving rather than logic decides the order. Where it compares a sequence against the book, the book's order is the claim being tested.

14.2.5 Disequality, absence and types under ∀

λProlog has no disequality, no *absento* and no type constraints. micro has all three, so the module had to decide what they mean when an eigenvariable is involved. For `≠°` there are two defensible readings of $\forall x.\ q \neq x$.

Definition 14.1 (two readings of an eigenvariable under ≠)**.** Under the *generic* reading, an eigenvariable is the new constant $c \notin \Sigma$ introduced by GENERIC (Fig. A.1), distinct from every term the older variables can denote. Under the *extensional* reading, an eigenvariable stands for an arbitrary term of its universe [41, 26].

The generic reading is the default: it matches λProlog, where `pi x\ not (q = x)` succeeds, and it behaves like αKanren's noms [5]. The extensional reading is chosen by the parameter `μkanren-λ-reading`:

```
(°->list #f (fresh° (q) (∀° (x) (≠° q x))))
(parameterize ((μkanren-λ-reading 'extensional)) (°->list #f (fresh° (q) (∀° (x) (≠°
↪  q x)))))
(parameterize ((μkanren-λ-reading 'extensional))
  (list (°->list #f (fresh° (q) (∀° (x) (fresh° (y) (≠° y x) (=° y 'a))) (=° q 'ok)))
        (°->list #f (fresh° (q) (∀° (x) (fresh° (y) (≠° y x) (=° y `(f ,x)))) (=° q
        ↪  'ok)))))
(°->list #f (fresh° (q) (∀° (x) (fresh° (y) (≠° y x) (=° y 'a))) (=° q 'ok)))
```

```
((λ (α) α))
()
(() ((λ () 'ok)))
((λ () 'ok))
```

Read them as statements about all x.

- Generic, $\forall x.\, q \neq x$: q is older than x and can never denote it, so the disequality holds for good and vanishes from the answer.
- Extensional, $\forall x.\, q \neq x$: x ranges over every term, q's value included, so the disequality cannot hold for all x and the goal fails.
- Extensional, $\forall x\, \exists y.\, y \neq x \wedge y = a$: the instance $x = a$ refutes it. With $y = (f\ x)$ it holds, since no term x equals the list $(f\ x)$ (occurs check).
- Generic, $\forall x\, \exists y.\, y \neq x \wedge y = a$: a is not the new name x, so it holds.

Both readings were implemented so that the order of a binding and a disequality does not change the answer. `A8/diseq-extensional` pins both orders of $\forall x.\, F\, x \neq x$ and $F := \lambda z.\, c$, and `R2/extensional-absent-order` pins four goal orders of $\forall x\, \exists Y.\, Y \neq a \wedge \mathit{absent}(a, Y) \wedge Y = x$, which fails in every order.

The reading has edges, and the proposal documents them. `absent°` and the types always use the generic reading. A disequality keeps the reading in force when it is posted, with one exception. A first-order disequality sits in micro's store D, which records no reading. If it later *migrates* to the module's store N, because one of its sides came to hold a λ-term or an eigenvariable, it takes the reading in force at the migration. The two readings then differ only if a program changes `μkanren-λ-reading` between the two goals. Under the extensional reading, once a `∀°` has run, a first-order disequality goes to N at once, with its reading.

14.2.6 Host escapes

Levels police logic variables. A Scheme value that host code carries out of a scope is not a logic variable, and the levels know nothing about the route it took. The limits suite pins both halves of the story:

```
(define stash #f)
(°->list #f (fresh° (q) (∀° (x) (project° ((v x)) (begin (set! stash v) ✓°)))
                        (λ (s) ((=° q stash) s))))
(°->list #f (fresh° (q) (∀° (x) (project° ((v x)) (begin (set! stash v) ✓°)))
                        (∀° (y) (fresh° (z) (=° z stash) (=° q 'leaked)))))
```

```
()
((λ () 'leaked))
```

Tutorial: Why the first escape is caught and the second is not

In both goals the first `∀°` raises the depth to 1 and creates x with $\ell(x) = 1$. `project°` stores the eigenvariable record in a Scheme variable, and the scope exits, so the depth is back to 0.

1. First goal: q was created at depth 0. Binding $q := x$ is checked, and $\ell(x) = 1 > \ell(q) = 0$ is a scope failure. The smuggled name cannot reach an *older* variable.
2. Second goal: a second `∀°` raises the depth to 1 again, and z is created inside it with $\ell(z) = 1$. Binding $z := x$ passes the check, since $\ell(x) = 1 \leq \ell(z)$. A level is a depth, not the identity of a scope, so two sibling scopes at the same depth look alike.

Inside the logic no such program can be written: a variable created in a later scope never sees the earlier eigenvariable, because it is out of its lexical scope. The escape needs `project°` and `set!`. The point of the first case is that even this route cannot violate the quantifier prefix of the older variables. That guarantee has a price, discussed in Section 14.3.3.

14.2.7 Binder names are hints; projected constraints

A λ-record stores an interned hint, the name the user wrote, for printing. Terms built alike are therefore `equal?`, and they group in `groupby°`. α-variants written with different names are not `equal?`, although they are α-equivalent, which `≛` and `μkanren-α=?` decide:

```
(list (equal? (ϰ (x) `(f ,x)) (ϰ (x) `(f ,x)))
      (equal? (ϰ (x) `(f ,x)) (ϰ (y) `(f ,y)))
      (≛ (ϰ (x) `(f ,x)) (ϰ (y) `(f ,y))))
```

```
(#t #f #t)
```

A hint that `equal?` ignored could not be stored in the record. The proposal chose to print answers with the user's binder names, and gave up `equal?` modulo α for that (Section 14.6).

The second half of this item is about answers. An *absento* on a flexible term whose head is in the answer is kept whole, and an argument out of the answer prints as a working variable:

```
(°->list #f (fresh° (q) (fresh° (Y) (absent° 'c (@ q Y)))))
(°->list #f (fresh° (q) (fresh° (F) (absent° 'c (@ F q)))))
```

```
((λ (α) (assert (absent? 'c (@ α _1))) α))
((λ (α) α))
```

The first answer says: there is some _1 such that c does not occur in α _1. It cannot be weakened to an absento on α alone: $\alpha := \lambda z.\, z\, c$ contains c, yet with _1 $:= \lambda v.\, b$ the application reduces to b. In the second answer the head F is out of the answer and may be a constant function, so the constraint is dropped. A disequality with such an argument, `(deny (equal? β (@ α _3)))`, and an absento whose datum may still reduce to an atom of the answer, `(absent? (@ α 'b) 'a)`, are likewise kept in the answer (`R3/diseq-unreachable-argument`, `R3/absento-datum-reduces`).

14.2.8 Costs

The engine pays for three things that λProlog implementations avoid. Every binding retries all the suspended equations of E and the disequalities of N when they are not empty, with no index by variable (the wake-up loop of Section 12.3.7). The scope check walks bound terms below the top level from the first `∀°` on, for the rest of the search, even after that `∀°` has exited. And deeply nested binders scale quadratically. Section 14.3 gives the figures.

14.2.9 The answer variable outside every ∀°

`°->list` reifies variable 0, the first one created. If it is created inside a `∀°` and its value holds that scope's eigenvariable, no answer can be written without the eigenvariable. λProlog reports no binding for a variable quantified inside `pi x\` . The module raises an error instead of printing an eigenvariable free:

```
(error-message (λ () (°->list #f (∀° (x) (fresh° (q) (=° q x))))))
(°->list #f (∀° (x) (fresh° (q) (=° q 'a))))
(°->list #f (fresh° (q) (∀° (x) (=° q 'a))))
```

```
"the answer holds the eigenvariable of a ∀° it was created in: create the variable it reifies
↪  outside every ∀°"
((λ () 'a))
((λ () 'a))
```

An answer that does not hold the eigenvariable is reported as usual, wherever its variable was created. The idiomatic form is the third one: create the query variable first.

14.3 Performance

All figures are in milliseconds, compiled with `csc -O2 -d1`, the best of three runs per round, with two rounds. Three binaries are compared. *before* is the installed micro, unedited. *after* is the edited micro with no extension loaded. *ext* is the edited micro with `(aux kanren lambda)` loaded, so its hooks are registered. The first-order benchmark is the one the engineering review used to compare the three prototypes.

14.3.1 The protocol on first-order search

Read Table 14.3 column by column. From *before* to *after*, the protocol alone costs at most 1% end to end on `append°` (5923 to 5959 ms in round 2), within the 2.5% round-to-round noise

Table 14.3: The cost of the protocol on first-order search.

benchmark (ms, round 1 / round 2)	before	after	ext
`append°` backwards, 2000 answers, full	6068 / 5923	6121 / 5959	6151 / 5929
the same, search only	672 / 662	690 / 669	691 / 668
open `append°`, 1000 answers, search	1311 / 1314	1356 / 1326	1327 / 1328
perm6 with 15 pairwise `≠°`, 720 answers	176 / 178	184 / 178	192 / 199

of the unedited binary. That is what one `pair?` test per call site costs when no extension is registered. All call sites are off micro's hottest path: the variable clauses and the pair–pair clause of unification are untouched.

From *after* to *ext*, `append°` does not move, but perm6 does: 192 / 199 ms, 8–12% more than the unedited micro's 176 / 178 ms (4–12% more than *after*). perm6 keeps 15 disequalities in D throughout, so after every `=°` the `post` hook must decide whether a new binding brought a λ-term or an eigenvariable into one of D's prefixes (Definition 10.6), which would then have to migrate to N (Section 14.2.5). That look costs a few microseconds per binding while D is non-empty. The d2 prototype, on which the proposal is based, paid 45–50% here.

14.3.2 With D or A non-empty

The third review round found that this look was too expensive in another case: one unrelated constraint before a long first-order search. Recall micro's stores (Section 10.5): D holds the disequalities, A the type constraints such as `symbol°`, and T the absentos. With D or A non-empty, the proposal walked every new binding through the substitution, and `append°` backwards binds long ground tails at every step. That walk is the factor of 1.65 to 1.76 in the middle column below. The last row is a control: its constraint goes to T, which this walk never concerned. The fix bounds the look: the hook inspects the new bindings for at most 64 nodes, and past

Table 14.4: One unrelated constraint before the search (`t` is not used by `append°`).

`append°` backwards, 2000 answers, search (ms)	after	ext, before the fix	ext, now
plain	673 / 669	665	657 / 664
+ `symbol° t`	674 / 675	1097	687 / 695
+ `≠° t 1`	655 / 657	1172	670 / 675
+ `absent° t`	661 / 664	—	656 / 666

that it walks D and A themselves. The order matters. Walking D and A at every `=°`, the first fix tried, cost perm6 60% (288 / 302 ms), because perm6's 15 prefixes are walked each time. With the bounded look first, perm6 stays at 193 / 194 ms, against 181 / 179 for *after*.

14.3.3 After a `∀°`

The second review round measured first-order search that runs after some `∀°` has run and exited. In Table 14.5, `pre` is `(∀° (z) ✓°)`, which binds nothing and exits at once, and `typeof°` is one

type inference of $\lambda x.\, x$ run before the search.

Table 14.5: First-order search after a `∀°` has run. A second run gave 680 / 1125 / 1154 and 1354 / 3055 ms; a third, after the third review's fixes, 663 / 1101 / 1090 and 1310 / 2918 ms.

benchmark (ms)	no `∀°` before	`pre` before	`typeof°` before
`append°` backwards, 2000 answers, search	663	1104 (+66%)	1092 (+65%)
open `append°`, 1000 answers, search	1296	2940 (+127%)	—

Tutorial: Where the time goes after a `∀°`

Take one step of `append°` backwards on the list L of 1999 numbers. It binds a variable to a pair whose tail is a ground suffix of L.

1. *No `∀°` has run.* The level table is empty, no scope check runs, and the only walk of the bound term is micro's occurs check.

2. *`pre` has run.* The depth went to 1 and back to 0, so the highest level reached is $top = 1$. The search's variables are at level 0, below top. Each binding of such a variable gets the scope check, whose allocation-free `µkanren-plain?` walks the bound term through the substitution, beside micro's occurs check. On `append°` the two walk the same ground tail of the list, so each binding walks that tail twice.

Why not skip the check when the depth is back to 0 and nothing of a higher level is reachable? For logic variables that would be safe: an exited scope reaches a depth-0 binding only through constraints, which are checked when they wake. But an eigenvariable smuggled out by host code would then reach an older variable unchecked, which is the first case of Section 14.2.6. The cheap fix gives up that guarantee, so it was not made. The proper fix is one walk that does both checks (Section 14.5.1).

14.3.4 λ workloads

Table 14.6 compares the final module with the three prototypes. The prototypes' columns are the engineering review's measurements on the same programs.

The final module is 1.7 times faster than d2, from which it grew, and 2.4 times slower than d3 on the Church numeral. Its history on `typeof` of Church 400 shows where the time went:

step	ms
d2 ported, with instantiation that shifts only open terms	463
allocation-free `µkanren-plain?` for the scope check	330
no check for bindings of variables at the top level	311
the `atom?` hook: micro's occurs check stops rebuilding eigenvariables	254

The last step is worth explaining. micro's occurs check rebuilds a foreign record as the list of its slots to look inside it. An eigenvariable holds no logic variable, so the hook lets micro skip

Table 14.6: λ workloads. d1 is the layered prototype, d2 the hooks prototype the proposal grew from, d3 the native one.

benchmark (ms)	final	d2	d3	d1
`typeof` of the Church numeral 400	254	441	105	385
`copy` of the Church numeral 400	256	442	116	314
`typeof` of 150 nested binders	1306	—	1409	—
`copy` of 150 nested binders	1745	—	1779	—
`typeof` backwards, 30 (term, type) pairs	10	11	—	—
one pattern equation $F\ x_1 \ldots x_{50} = g\ x_{50} \ldots x_1$ under 50 ∀	2	6	—	—

it, and that alone saved 18%.

A profile of the remaining time puts it in micro's walks, `µkanren-state-find` and its sbral lookup. These are the var–var chains that the type variables of `typeof` form, since micro binds the older variable to the newer, plus the occurs check of every binding of a subterm. Both are on micro's hot path, outside the extension's reach. d3's lead comes from its in-core binding, which does the occurs check and the level check in one walk. That is roadmap item (a). The remaining figures show no gap: on 150 nested binders the final module is 2–8% faster than d3. The module's cost on nested binders grows quadratically with the nesting (Section 14.2.8).

The fixes of the review rounds cost nothing measurable here. After the second round: typeof Church 400 260 ms, copy 259 ms, nested typeof 1292 ms, nested copy 1732 ms. After the third: 253, 256, 1284 and 1729 ms.

14.4 A staged merge into the aux repository

The proposal is three independent pieces of different risk. They should enter the repository in three stages, each reviewed and tested on its own.

Table 14.7: Which file goes where.

stage	from `doc/hol-kanren/prototype/`	to
1	`micro.diff` (+235/−41)	`src/aux.kanren.micro.scm`
2	`aux.kanren.lambda.scm` (1255 lines)	`src/aux.kanren.lambda.scm`
	`test/microkanren-lambda.scm`, `-limits.scm`	`src/test/microkanren-lambda*.scm`
	`test/programs.scm`	`src/test/programs.scm`
	one `extension` line	`src/aux.egg`
3	`show.diff` (+42/−12)	`src/aux.kanren.micro.show.scm`

14.4.1 Stage 1: the protocol

The diff is self-contained and mentions neither λ-terms, levels nor clauses. It could carry another store as well, nominal terms or finite domains, so it is worth having even if stage 2 is postponed. Both diffs were checked with `patch --dry-run` against the current `src/`. Applying

the first and, once the egg is rebuilt and installed, running micro's suites (without installing anything, the private repository of `build.sh` and `run-tests.sh`, Section 14.1.4, gives the same run):

```
cd src
patch aux.kanren.micro.scm < ../doc/hol-kanren/prototype/micro.diff
make test-microkanren test-microkanren-show test-microkanren-arith
 ↪  test-microkanren-date
```

These four targets run 112 cases. The aggregation and untagged suites have no target of their own: the `test` target runs them with the other four (`src/Makefile`, lines 46–51), or they run alone with `cd test && csi -s microkanren-aggregation.scm`, and likewise for `microkanren-untagged.scm`. The expected result is Section 14.1.4's: 125 cases, unchanged answers. The benchmark's *after* column (Table 14.3) is the performance criterion. The diff already documents each hook in micro's comments; the merge should add to them a short section on the protocol as a whole.

What to review, in this order:

1. *The state.* A seventh field `X`, the extensions' slot, which micro never reads. It is copied at micro's two positional constructions (`µkanren-state-update` and `freshᵍ`), `µkanren-state-with` accepts `X:`, and `make-µkanren-state` keeps its six-argument signature, so the positional states of `test/microkanren.scm` still work.

2. *The registry and the call sites.* `make-µkanren-extension` and `µkanren-extension-register!`; each call site costs one `pair?` test when the registry is empty. There are thirteen hooks: `term?`, `atom?`, `unify`, `occurs`, `post`, `diseq`, `tag`, `parts`, `value`, `repr`, `terms`, `forms` and `environment`; Chapter 11 gives their contracts.

3. *The two invariants* that every hook must keep (Definition 11.13): a hook that changes nothing answers the very state it was given, and a hook that binds returns the extended `assocs`.

4. *The two exported primitives*: `µkanren-state-forget`, which unbinds a variable so that a `post` hook can turn a binding just made back into a constraint before D, A and T see it, and `µkanren-post-unify`, the post-`=°` chain that an extension goal unifying on its own must call (Section 11.4.4).

5. *One change of micro's own behaviour.* micro decided an atom against an absento's datum with `equal?` only, so `(absent° D q) (=° q 'a) (=° D 'a)` succeeded in micro alone. The diff decides it by unification and watches it while pending. This is a bug fix of micro, independent of λ, and deserves its own look.

14.4.2 Stage 2: the module and its suites

Besides `srfi-1` and CHICKEN's own modules, the module imports `(aux base)`, `(aux stream)`, `(aux fds sbral)` and (aux kanren micro), and registers its extension when it is loaded. The egg gets one line after micro's, in the style of its neighbours:

```
(extension aux.kanren.micro)
(extension aux.kanren.lambda)
(extension aux.kanren.arith)
```

Other entries of `src/aux.egg` state their dependencies with `component-dependencies`. Writing `(component-dependencies aux.kanren.micro)` would make this one explicit too. The suite includes `programs.scm` from its own directory, so the file goes beside it in `src/test/`. The Makefile would follow its existing pattern: two lines in `test`, after micro's suites, and a target of its own.

```
cd test && ${CSI} -s microkanren-lambda.scm
cd test && ${CSI} -s microkanren-lambda-limits.scm
```

Then micro's six suites should be run once more with the module loaded, as in Section 14.1.3. The simplest way is to keep the second copy of the six suites under `doc/hol-kanren/prototype/`, since they differ from `src/test/` only in their import line.

What to review, in this order. First the limits suite, 51 lines that state what is not promised. Then the scope check and variable elimination, which carry soundness (Chapter 12). Then the discharge at `∀°` exit, then the store N. The printing code comes last: a mistake there shows up in answers, not in the search.

14.4.3 Stage 3: the show module

`show.diff` teaches `(aux kanren micro show)` to read `ϰ`, `@`, `≐`, `∃` and `∀` in answers, draws λ-terms in the mermaid cells view with a node per binder, and adds `°->pretty`. It reads answer forms syntactically and does not import the module, so it is independent of stage 2. It is useful only with it.

```
cd src
patch aux.kanren.micro.show.scm < ../doc/hol-kanren/prototype/show.diff
make test-microkanren-show
```

14.5 Later work

Six items were left out of the proposal on purpose. Each should be measured separately, against the figures of Section 14.3 and the suites of Section 14.1. For each, the paragraphs below say how it would fit the protocol and which concept of the book it serves.

14.5.1 (a) One walk for the occurs and the scope check

What. micro's binding site, `μkanren-state-update`, runs the occurs check. The module's `post` hook then walks the same term again for the scope check. A combined walk does both, as d3's in-core `occurs/fast` does.

How it fits. A new hook at the binding site, which returns a *verdict* instead of micro's boolean: no occurrence and nothing out of scope; an occurs failure; or a term that needs variable elimination. With no extension registered, micro keeps its own check.

What it buys. Most of the cost of first-order search after a `∀°` (+65% to +127%, Table 14.5) and the remaining factor of 2.4 against d3 on Church 400 (Table 14.6). It keeps the host-escape guarantee, because the check still runs at depth 0.

Book concept. The mixed prefix of a unification problem (Section 4.4), read as universe levels (Appendix A.10, Section 7.9). Nadathur, Jayaraman and Kwon's binding rule [43] is one condition on the bound term, so it is natural to check it in the same walk as the occurs check.

14.5.2 (b) Var–var orientation by level

What. When two variables are unified, bind the one of higher level to the one of lower level. micro binds the older to the newer, so a variable of a deep scope may become the value of an outer one, and the scope check must then lower it (Definition A.27).

How it fits. It needs a hook in micro's var–var clause, the hottest path after pair–pair. That is why the graft from d3 was declined. **Check** (Definition 12.7) already lowers the younger variable, which gives the same most general unifier up to renaming, so this item is about cost only: fewer lowering bindings, and perhaps shorter var–var chains, which the profile of Section 14.3.4 places among the remaining costs. It must be measured on first-order search too, where the hook would cost on every var–var binding.

Book concept. Raising and lowering in a mixed prefix (Section 4.5). Binding the deeper variable to the outer one is always within scope. Binding the outer variable to the deeper one is allowed only after the deeper one has been lowered, which is the extra work this item saves.

14.5.3 (c) Indexing E and N by variable

What. Today every `=°` retries all suspended equations of E and all disequalities of N, oldest first, in rounds, until no round binds (Section 14.2.8). An index from each variable to the constraints that mention it would retry only those the new bindings touch.

How it fits. Entirely inside the extension. E and N live in the slot `X`, which micro never reads, and the `post` hook already receives the new bindings as `assocs`. No change to micro. The care needed is in what "touches" means: a binding wakes a constraint on a variable it reaches through the substitution, not only on the variable it binds, and the variables created by lowering and pruning must be indexed too.

Book concept. Dynamic L_λ (Section 7.9): an equation that does not respect the L_λ condition is deferred "until such time that substitutions resulting from solving other parts of the problem have altered its status" [MN p. 226]. An index decides cheaply which deferred equations a substitution may have altered.

14.5.4 (d) Huet pre-unification as an optional search

What. An opt-in goal that takes the suspended equations of E and searches their solutions with Huet's imitation and projection steps [25], each choice a branch of the interleaving search. On Example 8.A it would enumerate the four unifiers of Example 7.5. On the first problem of Section 14.2.3 it would find no solution and fail, so floundering would be detected, not only reported.

How it fits. E is the natural agenda: it already holds the non-pattern equations, closed over the eigenvariables they mention, and woken equations are retried by the existing loop. Huet's branching is don't-know nondeterminism, which a miniKanren stream represents directly. It should be a goal the user calls, not the default, because the search may not terminate (Section 7.3).

The obstacle. Projection needs the target type and arities, and imitation the arity and argument types of the constant: "Huet's procedure cannot run without types" (Section 7.4). So this item presupposes (f).

Book concept. Huet's procedure and its matching trees [MN p. 214–220], and the limit of pattern unification that Section 5.8 discusses.

14.5.5 (e) Predicate quantification

What. Goals such as `pi rev\ (... rev ...) => rev L K nil`, which introduce a hidden predicate local to one call [MN p. 125–126].

How it fits. Through dynamic relations created at run time. Γ is an alist keyed by relation, so a `∀` over a predicate would allocate a new key, a new dynamic relation with no static clauses, and pass it to the body. Clauses assumed for it are then visible only inside that body, and no program text outside can name it. This is a sketch: the proposal does not implement it.

Book concept. $hohh^+$ (Definition 5.7): a clause head may be a variable bound by an essentially universal quantifier. By the time the clause is assumed, $\forall R$ has replaced the variable by a new constant, so the operational behaviour does not change. Only the visibility of names does.

14.5.6 (f) Simple types

What. Optional simple types on λ-terms and on logic variables.

How it fits. Through the interplay of the `atom?` and `tag` hooks, which the proposal names for this item without designing it. One reading: a type is a tag on a term, and a typed variable can be η-expanded by its type. Unification could then work on η-long forms, as λProlog does, instead of η-expanding only when one side is an abstraction (Section 14.2.2).

What it buys. Type-directed η, β that is guaranteed to terminate, the existence check on possibly empty types, and with them item (d).

Book concept. The simply typed terms of Chapter 4 and the βη-long normal forms on which the book's unification algorithms are stated (Section 4.3).

14.6 Open questions for the author

The proposal makes a choice wherever one was needed to build it and test it. Some of those choices are the maintainer's to make, and are listed here as questions. Each says what the proposal does now and what is at stake.

1. *The module's name.* The proposal keeps `(aux kanren lambda)`. The layer is λProlog's logic, and λKanren is the name the literature gives this line of work, Ma and Friedman's paper included [29]. A name such as `(aux kanren hohh)` would describe only the goals,

while the module also gives λ-tree syntax and L_λ unification. The names inside follow aux: goals end in `°`, their applicative forms in `ª`, term constructors have no suffix (`ƛ`, `@`), and internals carry the `µkanren-` prefix.

2. *The extensional reading.* `µkanren-λ-reading` exposes a second meaning of `≠°` under `∀°` (Definition 14.1). It costs code and one edge case, the reading of a migrated disequality (Section 14.2.5). Keep it public, keep it as an internal option, or remove it and keep only λProlog's generic reading?

3. *Should micro own the protocol?* The proposal puts a generic extension protocol in micro, a global registry, and one client. The alternatives were a layered module that leaves micro untouched (d1, which leaked eigenvariables through micro's own `=°`) and λ-terms native in micro (d3, a 931-line diff that special-cases every walker). A per-state hook list instead of a global registry would require every initial state to carry the hooks, since `°->list` starts from `µkanren-state-empty`. Is a protocol in micro, with its contracts and invariants, something the maintainer wants to keep stable?

4. *The cost after a `∀°`.* First-order search after any `∀°` takes 1.7 to 2.3 times as long (Table 14.5). The cheap fix gives up the host-escape guarantee (Section 14.2.6); the proper one is roadmap item (a). Should stage 2 be merged now and (a) done later, or should (a) be a condition of the merge? And is it worth this price to catch escapes made with `project°` and `set!`?

5. *Which relations are dynamic.* In λProlog every predicate takes hypotheses; here only those made by `define-dynamic-relation`, and `⊃°` on any other is an error (Section 14.2.1). Should `define-relation` itself become dynamic when the module is loaded? The proposal does not measure what that would cost.

6. *α-equivalence and `equal?`.* Hints are interned, so terms built alike are `equal?`, but α-variants with different names are not (Section 14.2.7). Truly nameless records would make `equal?` decide α, at the price of answers printed with generated names instead of the user's. Which matters more for the intended use?

7. *An answer inside `∀°`.* Reifying a variable created inside a `∀°` that holds its eigenvariable is an error, where λProlog reports no binding (Section 14.2.9). Keep the error, or answer with no binding as λProlog does?

8. *A defect of micro left as it is.* Under a type of the user's that holds of pairs, micro's subsumption turns `absent° '(a) q` into `q ≠ (a)`, so `(absent° '(a) q) (nonnum° q) (=° q '((a)))` succeeds: the subsumption assumes that typed values are atoms. It involves no λ-term, so the proposal did not touch it. Fix it in micro, with the protocol or separately?

9. *Two clause syntaxes.* Hypothetical clauses may be written infix, `(H ← B)`, or prefix, `(← H B)`; both are accepted. Keep both, or settle on one?

14.7 Summary

- The module has two suites: 83 cases with 320 hand-written assertions (acceptance A1–A9 and the regressions R, R2, R3 of three adversarial review rounds), and 6 cases that pin its limits. micro's six suites, 125 cases, pass with the edited micro both without and with the module loaded, answer order unchanged: 339 cases, 0 failures.
- The suites test behaviour on examples, under several goal orders; the guarantees on pattern problems come from the theorems of Miller, of Nadathur, Jayaraman and Kwon, and of Nadathur and Linnell, which the algorithm follows.
- Nine limitations are documented and pinned. Only dynamic relations take hypotheses, and there is no predicate quantification. Terms are untyped. Non-patterns are suspended, never enumerated, so floundering is reported, not detected. Answer order is micro's interleaving. `≠°`, `absent°` and types use the generic reading of `∀`, with an extensional option for `≠°`. Levels police logic variables, not Scheme values. Binder names are hints. Constraints are retried unindexed, and the scope check runs after any `∀°`. The answer variable must be created outside every `∀°`.
- The protocol costs about 1% on first-order search, and the loaded extension 8–12% on a search that keeps many disequalities. After a `∀°`, first-order search takes 1.7–2.3 times as long, because the scope check walks the bound term beside the occurs check. On λ workloads the module is 1.7 times faster than the d2 prototype it grew from and 2.4 times slower than the native d3 on Church 400, the rest of the time being in micro's walks.
- The merge has three stages. First the protocol, into micro's source. Then the module, its suites, `programs.scm` and one egg line. Last the show extension. Review the protocol's invariants and its one change to micro's semantics first.
- Six later items: one walk for the occurs and scope checks; var–var orientation by level; an index of E and N; an optional Huet search, which needs types; predicate quantification through dynamic relations made at run time; simple types.
- Nine questions are the author's to answer, among them the module's name, the extensional reading, whether micro should own a protocol, and whether the cost after a `∀°` must be paid down before the merge.

Appendices

A Reference card: rules and algorithms

This appendix lists the formal rules used in the document, each with the page where its source states it; Chapters 2 to 5 and 7 explain them. Page references [MN p. n] use the printed page numbers of [36]; papers are cited with their own page numbers.

A.1 λProlog concrete syntax and its logical reading

Table A.1 maps every λProlog construct used in the document to the logical formula it denotes.

Table A.1: λProlog syntax and logic.

λProlog	logic	notes
`true`	$\top$	type o [MN p. 35]
`G1, G2`	$G_1 \wedge G_2$	infix, left assoc.; joins goals [MN p. 35,47–48]
`D1 & D2`	$D_1 \wedge D_2$	infix, right assoc.; joins clauses [MN p. 35,47–48]
`G1 ; G2`	$G_1 \vee G_2$	infix, left assoc. [MN p. 35]
`D => G`	$D \supset G$	infix, right assoc.; implication in goals [MN p. 35,79]
`A :- G`	$G \supset A$	infix, non-assoc.; "is implied by", used in clauses [MN p. 35,47–48]
`pi x\ B`, `pi (x:T)\ B`	$\forall x\, B$, $\forall_\tau x\, B$	$\forall : (A \to o) \to o$ applied to $\lambda x.\, B$ [MN p. 36–37,99]
`sigma x\ B`	$\exists x\, B$	$\exists : (A \to o) \to o$ applied to $\lambda x.\, B$ [MN p. 36–37,99]
`x\ t`, `(x:T)\ t`	$\lambda x.\, t$, $\lambda (x : \tau)\, t$	body extends as far right as possible [MN p. 97]
`t1 t2`	$(t_1\ t_2)$	application, left assoc., binds tightest [MN p. 97]
`X`, `L1` (free, uppercase)	$\forall$ at clause top; $\exists$ in a query	implicit quantification [MN p. 43,46–47]
`_`	a fresh variable per occurrence	anonymous, $\forall$ over the clause [MN p. 46–47]
`X = Y`	$X = Y$	built-in equality, solved by unification [MN p. 106,287]
`false` (user constant)	$\bot$ of minimal logic	no ex falso for it; $\neg B$ is $B \supset \bot$ [MN p. 89–90]
`A -> B` (in types)	$\tau \to \sigma$	right assoc. [MN p. 12–13]
`o`	o	the type of propositions [MN p. 34–35]

λProlog	logic	notes
`kind c type -> type.`	a type constructor of arity 1	kind declaration [MN p. 11–12]
`type c T.`	$c : \tau \in \Sigma$	constant declaration [MN p. 47]

Binding strength, tightest first: application > `=>` > `&` > `,` > `;` > `:-` [MN p. 35]; the scope of `x\` extends as far right as possible [MN p. 37,97].

Pitfall: precedence

`D => G1, G2` parses as `(D => G1), G2`, so a hypothesis meant for a whole conjunction needs parentheses [MN p. 35].

A.2 Goal and clause grammars

The four languages share one top-level shape and differ in which connectives goals may contain and in what an atom may be (Table A.2).

Table A.2: Goal formulas G and program clauses D. A: atom; A_r: rigid atom.

language	grammar	source
fohc	$G ::= \top \mid A \mid G \wedge G \mid G \vee G \mid \exists_\tau x\, G$ $D ::= A \mid G \supset D \mid D \wedge D \mid \forall_\tau x\, D$	[MN p. 43]
fohh	$G ::= \top \mid A \mid G \wedge G \mid G \vee G \mid \exists_\tau x\, G \mid D \supset G \mid \forall_\tau x\, G$ $D ::= A \mid G \supset D \mid D \wedge D \mid \forall_\tau x\, D$	[MN p. 75]
hohc	as *fohc*, with $A \in \mathscr{H}_1^\Sigma$ in goals and heads $A_r \in \mathscr{H}_1^\Sigma$	[MN p. 122–123]
hohh	as *fohh*, with $A \in \mathscr{H}_2^\Sigma$ in goals and heads $A_r \in \mathscr{H}_2^\Sigma$	[MN p. 124]
hohh$^+$	*hohh*, plus atomic heads whose head is a variable bound by an essentially universal quantifier that has the clause in its scope	[MN p. 125]

Definition A.1 (Herbrand universes [MN p. 122]). $\mathscr{H}_1^\Sigma$ is the set of (possibly open) λ-normal terms over Σ that contain neither $\forall$ nor $\supset$; $\mathscr{H}_2^\Sigma$ is the set of those that do not contain $\supset$.

Definition A.2 (rigid and flexible atoms [MN p. 119]). An atom is *flexible* if the head h of its λ-normal form $(h\, t_1 \ldots t_n)$ is a variable, and *rigid* if h is a non-logical constant.

Definition A.3 (polarity; hereditary Harrop [MN p. 76]). B is positive in B; a positive (negative) occurrence in B stays positive (negative) in $B \wedge B'$, $B' \wedge B$, $B \vee B'$, $B' \vee B$, $B' \supset B$, $\forall x\, B$, $\exists x\, B$, and becomes negative (positive) in $B \supset B'$. A hereditary Harrop formula is one in which no positive subformula occurrence is a disjunction or an existential quantification.

Definition A.4 (essentially universal / existential [MN p. 205]). In a goal, a bound-variable occurrence is *essentially universal* if it is bound by a positive $\forall$, a negative $\exists$ or a term-level λ, and *essentially existential* otherwise; in a clause the classification is dual (negative $\forall$, positive $\exists$ or λ are essentially universal).

Definition A.5 (L_λ [MN p. 205–206,220])**.** A goal or clause B is in L_λ if all quantification in B is over non-predicate types and every subterm $(x\, t_1 \ldots t_n)$ of B with x essentially existential has $t_1, \ldots, t_n$ distinct essentially universal variables bound within the scope of x.

A.3 Uniform proofs

A sequent $\Sigma; \mathscr{P} \longrightarrow G$ says that G is provable from program $\mathscr{P}$ over signature Σ; $\Sigma; \mathscr{P} \xrightarrow{D} A$ says that A is proved by backchaining on the clause D [MN p. 39,45]. Read bottom-up, the right rules (goal reduction) and the left rules (backchaining) are the two phases of a *uniform*, i.e. goal-directed, proof; a logic in which such proofs are complete is an *abstract logic programming language* [MN p. 73].

$$\frac{}{\Sigma; \mathscr{P} \longrightarrow \top}\ \textsc{true} \qquad \frac{\Sigma; \mathscr{P} \longrightarrow B_1 \qquad \Sigma; \mathscr{P} \longrightarrow B_2}{\Sigma; \mathscr{P} \longrightarrow B_1 \wedge B_2}\ \textsc{and} \qquad \frac{\Sigma; \mathscr{P} \longrightarrow B_i}{\Sigma; \mathscr{P} \longrightarrow B_1 \vee B_2}\ \textsc{or}\ (i = 1, 2)$$

$$\frac{\Sigma; \mathscr{P} \longrightarrow B[t/x] \qquad \Sigma; \varnothing \Vdash t : \tau}{\Sigma; \mathscr{P} \longrightarrow \exists_\tau x\, B}\ \textsc{instan} \qquad \frac{\Sigma; \mathscr{P}, B_1 \longrightarrow B_2}{\Sigma; \mathscr{P} \longrightarrow B_1 \supset B_2}\ \textsc{augment}$$

$$\frac{c : \tau, \Sigma; \mathscr{P} \longrightarrow B[c/x]}{\Sigma; \mathscr{P} \longrightarrow \forall_\tau x\, B}\ \textsc{generic}\ (c \notin \Sigma)$$

Figure A.1: Right rules (search semantics of the connectives) [MN p. 39–40]. In the book they are $\top R$, $\wedge R$, $\vee R$, $\exists R$, $\supset R$, $\forall R$; $\mathscr{P}, B$ is $\mathscr{P} \cup \{B\}$.

$$\frac{\Sigma; \mathscr{P} \xrightarrow{D} A}{\Sigma; \mathscr{P} \longrightarrow A}\ \textsc{decide}\ (D \in \mathscr{P}) \qquad \frac{}{\Sigma; \mathscr{P} \xrightarrow{A} A}\ \textsc{initial} \qquad \frac{\Sigma; \mathscr{P} \xrightarrow{D} A \qquad \Sigma; \mathscr{P} \longrightarrow G}{\Sigma; \mathscr{P} \xrightarrow{G \supset D} A} \supset L$$

$$\frac{\Sigma; \mathscr{P} \xrightarrow{D_i} A}{\Sigma; \mathscr{P} \xrightarrow{D_1 \wedge D_2} A} \wedge L\ (i = 1, 2) \qquad \frac{\Sigma; \mathscr{P} \xrightarrow{D[t/x]} A \qquad \Sigma; \varnothing \Vdash t : \tau}{\Sigma; \mathscr{P} \xrightarrow{\forall_\tau x\, D} A} \forall L$$

$$\frac{\Sigma; \mathscr{P} \longrightarrow A_1\theta \qquad \cdots \qquad \Sigma; \mathscr{P} \longrightarrow A_n\theta}{\Sigma; \mathscr{P} \xrightarrow{\forall \bar{x}\, (A_1 \wedge \cdots \wedge A_n \supset A_0)} A}\ \textsc{backchain}$$

(θ maps each x_i to a Σ-term of type τ_i, and $A = A_0\theta$)

Figure A.2: Backchaining (left) rules [MN p. 45]; BACKCHAIN is the compiled form for Prolog-shaped clauses [MN p. 45].

Definition A.6 (O-proof [MN p. 45,77])**.** A derivation built from the typing rules of terms and the rules of Figs. A.1 and A.2.

Logic-variable versions [MN p. 61]. INSTAN and $\forall L$ replace the guessed t by a new logic variable X with $\Sigma; \emptyset \Vdash X : \tau$, and INITIAL becomes $\Sigma; \mathscr{P} \xrightarrow{A'} A$ provided A and A' unify, the unifier being applied to the whole derivation built so far.

Higher-order versions [MN p. 123–124]. (i) Formulas are taken in λ-normal form when matched against a rule; (ii) INITIAL requires A' and A to be α-convertible; (iii) the term t of INSTAN and $\forall L$ is a λ-term with $\Sigma; \emptyset \Vdash t : \tau$ and $t \in \mathscr{H}_1^\Sigma$ (*hohc*) or $t \in \mathscr{H}_2^\Sigma$ for the current Σ (*hohh*).

Backchaining by elaboration [34, p. 9]. With $\langle \emptyset, D \rangle \in |\Gamma|_\Sigma$ for $D \in \Gamma$, conjunctions split, $\langle \Delta, G \supset D \rangle \mapsto \langle \Delta \cup \{G\}, D \rangle$ and $\langle \Delta, \forall x\, D \rangle \mapsto \langle \Delta, \lambda\mathrm{norm}(D[t/x]) \rangle$:

$$\frac{\{\Sigma; \Gamma \longrightarrow G\}_{G \in \Delta}}{\Sigma; \Gamma \longrightarrow A}\ \textsc{bc}\ (A \text{ atomic}, \langle \Delta, A \rangle \in |\Gamma|_\Sigma)$$

Search conventions of λProlog [MN p. 60,78]. Conjunctive goals left to right; the left disjunct of a goal and the left conjunct of a clause first; clauses tried in program order (the program is a list); AUGMENT adds its clauses at the *front*; signature and program grow and shrink in a stack discipline.

Flexible goals [MN p. 134]. A goal $(P\, t_1 \ldots t_n)$ with P unbound can be (1) suspended until rigid, and if only flexible goals remain each head is bound to $\lambda x_1 \ldots \lambda x_n\, \top$ (complete for intuitionistic higher-order logic); (2) bound eagerly to $\lambda x_1 \ldots \lambda x_n\, \top$; or (3) reported as an error; (2) and (3) are incomplete, yet experience favours (3), since a flexible goal is usually a programming mistake such as a capitalised predicate name.

Table A.3: What O-proofs are complete for.

language	$\Sigma; \mathscr{P} \longrightarrow G$ has an O-proof iff it is provable in	source
fohc	classical logic, iff intuitionistic logic	[MN p. 45,62]
fohh	minimal logic, equivalently intuitionistic logic (classical logic is not sound for this operational semantics)	[MN p. 89,91]
hohc	classical, equivalently intuitionistic, Simple Theory of Types without extensionality, infinity and choice	[MN p. 124,148]
hohh	higher-order intuitionistic logic	[MN p. 124]

Proposition A.7 (flatness of *fohc* [MN p. 46])**.** *Every sequent in an O-proof of an fohc sequent* $\Sigma; \mathscr{P} \longrightarrow G$ *has the form* $\Sigma; \mathscr{P} \longrightarrow G'$ *or* $\Sigma; \mathscr{P} \xrightarrow{D} A$*: signature and program never change.*

Kanren angle: which rules a miniKanren already has

(aux kanren micro) has goals for TRUE (`✓°`), AND (`and°`), OR (`or°`) and logic-variable INSTAN (`fresh°`); a relation defined as a Scheme procedure plays the role of DECIDE/BACKCHAIN,

and its unifier is the first-order one of Fig. A.4, occurs check included. Two differences matter: `or°` interleaves its branches instead of following the depth-first order above, and AUGMENT and GENERIC have no counterpart, so (aux kanren micro) covers *fohc* but not *fohh*; see Chapter 11.

A.4 Typed λ-terms

The typing judgement $\Sigma;\Gamma \Vdash t : \tau$ assigns types to λ-terms; quantifiers need no rule of their own because they are constants applied to abstractions [MN p. 98–99].

$$\frac{c : \sigma \in \Sigma}{\Sigma;\Gamma \Vdash c : \tau}\ (\tau \lhd \sigma) \qquad \frac{x : \tau \in \Gamma}{\Sigma;\Gamma \Vdash x : \tau} \qquad \frac{\Sigma;\Gamma \Vdash g : \tau_1 \to \tau_2 \qquad \Sigma;\Gamma \Vdash t : \tau_1}{\Sigma;\Gamma \Vdash (g\,t) : \tau_2}$$

$$\frac{\Sigma;\Gamma, x : \tau \Vdash t : \sigma}{\Sigma;\Gamma \Vdash \lambda(x{:}\tau)\,t : \tau \to \sigma}\ (\dagger) \qquad \frac{\Sigma;\Gamma \Vdash B : \tau}{\Sigma;\Gamma \Vdash C : \tau}\ (\ddagger)$$

Figure A.3: Typing rules [MN p. 98]. $\tau \lhd \sigma$: τ is an instance of σ; (†) x not declared in Σ or Γ; (‡) B and C differ only in bound-variable names.

Definition A.8 (order of a type [MN p. 13])**.** $\mathrm{ord}(\tau) = 0$ if τ is non-functional; $\mathrm{ord}(\tau_1 \to \tau_2) = \max(\mathrm{ord}(\tau_1) + 1, \mathrm{ord}(\tau_2))$.

A.5 λ-conversion and normal forms

λ-conversion is the equality of the logic, and every typed term has a unique normal form, so equality is decidable [MN p. 101–102].

Definition A.9 (free for; substitution [MN p. 85,100])**.** t is *free for* x in s if no free occurrence of x in s is in the scope of an abstraction (or quantifier) binding a variable free in t. $s[t/x]$ replaces the free occurrences of x in s by t, after α-renaming s so that t is free for x (capture-avoiding substitution) [MN p. 101].

Table A.4: Rewrite rules on subterms [MN p. 100–101].

rule	redex ⇒ contractum	proviso
α-rewriting	$\lambda x.\,s \Rightarrow \lambda y.\,s[y/x]$	y free for x in s, y not free in s
β-contraction	$(\lambda x.\,s)\,t \Rightarrow s[t/x]$	none (capture-avoiding $s[t/x]$)
η-contraction	$\lambda x.\,(s\,x) \Rightarrow s$	x not free in s
β_0 [MN p. 206]	$(\lambda x.\,s)\,x \Rightarrow s$; up to α: $(\lambda y.\,s)\,x \Rightarrow s[x/y]$	(second form) x not free in $\lambda y.\,s$

Definition A.10 (closures [MN p. 100–101])**.** α-conversion: reflexive, symmetric, transitive closure of α-rewriting. β-reduction: reflexive, transitive closure of α-conversion ∪ β-contraction; β-conversion: its symmetric, transitive closure. η-reduction / η-conversion: likewise from η-contraction. λ-conversion ($=_{\beta\eta}$): transitive closure of α-, β- and η-conversion; it is a congruence and is the equality of higher-order logic.

Definition A.11 (normal forms [MN p. 101–102,111])**.** A term is *β-normal* if it has no β-redex, *λ-normal* (βη-normal) if it is β-normal with no η-redex. Every β-normal term has the shape $\lambda x_1 \dots \lambda x_n\,(h\,t_1 \dots t_p)$ $(n, p \geq 0)$, with *binder* $x_1 \dots x_n$, *head* h (a constant or variable), *arguments* t_i (β-normal) and *body* $(h\,t_1 \dots t_p)$. It is in *βη-long normal form* if moreover each t_i is βη-long and the body has non-functional (for simple types: primitive) type.

Theorem A.12 (normalisation [MN p. 102])**.** *Every typed λ-term has a λ-normal form, unique up to α-conversion; it is computed by β-contracting until β-normal and then η-contracting. Two terms of the same type are λ-convertible iff their λ-normal forms are α-equivalent. Every β-normal form can be η-expanded to an equal βη-long form* [MN p. 111].

Example A.13 (η-long expansion [MN p. 111])**.** With $f : (i \to i) \to i$: $f \;\rightsquigarrow\; \lambda w.\, f\,w$ $(w : i \to i) \;\rightsquigarrow\; \lambda w.\, f\,(\lambda u.\, w\,u)$ $(u : i)$, which is βη-long.

Definition A.14 (rigid and flexible terms [MN p. 111–112,214])**.** A β-normal term $\lambda \bar{x}\,(h\,\bar{t})$ in a unification problem is *rigid* if h is universally quantified in the prefix (a constant) or a member of its binder $\bar{x}$, and *flexible* if h is existentially quantified; equations are rigid-rigid, flexible-rigid, rigid-flexible or flexible-flexible [MN p. 113].

A.6 Unification problems, raising, Skolemization

Higher-order unification problems carry a mixed quantifier prefix, which raising normalises to ∀∃∀ [MN p. 105–108,212].

Definition A.15 (unification problem [MN p. 105,211])**.** A formula $Q_1 x_1 \dots Q_n x_n\,[t_1 = s_1 \wedge \dots \wedge t_m = s_m]$ with $Q_i \in \{\forall, \exists\}$ and t_j, s_j λ-terms of the same type; the quantifiers form the *prefix*, the conjunction the *matrix*.

Definition A.16 (unifier, solution [MN p. 109])**.** For $\forall u_1 \dots \forall u_p\, \exists x_1 \dots \exists x_n\, \forall w_1 \dots \forall w_q\,[t_1 = s_1 \wedge \dots \wedge t_m = s_m]$, a substitution θ for $x_1, \dots, x_n$ is *legitimate* if no w_j is free in its range; a *unifier* is a legitimate θ with $\theta(t_j) =_{\beta\eta} \theta(s_j)$ for all j; a *solution* is a unifier whose range has free variables only among $u_1, \dots, u_p$. Only a solution guarantees provability, since types are not assumed inhabited [MN p. 109–110].

Example A.17 (the prefix order matters [MN p. 110,213])**.** With $g : i \to i \to i$, $\forall a\, \exists F\,[(F\,a) = (g\,a\,a)]$ has the four unifiers $F := \lambda x.\, g\,a\,a$, $\lambda x.\, g\,x\,a$, $\lambda x.\, g\,a\,x$, $\lambda x.\, g\,x\,x$, none an instance of another, so there is no most general unifier; in $\exists F\, \forall a\,[(F\,a) = (g\,a\,a)]$ the solution F cannot mention a, and $\lambda a.\, g\,a\,a$ is the only one.

Proposition A.18 (raising [MN p. 107])**.** *With $x : \tau$, $y : \sigma$ and $h : \sigma \to \tau$: $\forall y\, \exists x\, G$ is provable iff $\exists h\, \forall y\, G[(h\,y)/x]$ is provable.*

As a rewrite on prefixes, for a block of universals:

$$\forall y_1 : \sigma_1 \ldots \forall y_k : \sigma_k \; \exists x : \tau \; G \;\rightsquigarrow\; \exists h : \sigma_1 \to \ldots \to \sigma_k \to \tau \; \forall y_1 \ldots \forall y_k \; G[(h\, y_1 \ldots y_k)/x].$$

Repeated raising turns any prefix into $\forall\exists\forall$ or even $\exists\forall$ form [MN p. 108], and it preserves L_λ [MN p. 208]. Miller's transition form [34, p. 14–16]: if $\exists v$ is to the left of $\exists u$ with universals $\bar{w}$ in between, replace u by $(u'\ \bar{w})$ and move $\exists u'$ next to $\exists v$.

Skolemization [MN p. 108]. An assumption $\forall x \, \exists y \, D$ becomes $\forall x \, D[(f\, x)/y]$ with $f : \sigma \to \tau$ a new constant; dually a goal $\exists x \, \forall y \, G$ becomes $\forall f \, \exists x \, G[(f\, x)/y]$. Skolemization moves an $\exists$ inwards and introduces a constant of raised type; raising moves a $\forall$ inwards and introduces an existential variable of raised type; relating unifiers of Skolemized problems to the original ones is problematic for higher-order variables, whereas raising is an exact equivalence [MN p. 107–108].

Normal form of problems [MN p. 212]. Before solving, (1) the prefix is raised to $\forall\bar{a} \, \exists\bar{X} \, \forall\bar{y}$, with $\forall\bar{a}$ the signature; (2) $\theta(t)$ is defined as $((\lambda x_1 \ldots \lambda x_n \, t)\, t_1 \ldots t_n)$ for $\theta = \{\langle x_i, t_i \rangle\}$, so terms may be taken β-normal; (3) η-expansion makes the two sides of each equation have binders of the same length; (4) by $(\lambda x.\, t = \lambda x.\, s) \equiv \forall x \, (t = s)$ the binders are moved into the inner $\forall$ block:

$$\frac{\langle \mathcal{Q};\ \{\lambda x.\, s = \lambda x.\, t\} \uplus E \rangle}{\langle \mathcal{Q}\ \forall x;\ \{s = t\} \uplus E \rangle}\ \textsc{abs} \qquad \frac{\langle \mathcal{Q};\ \{\lambda x.\, s = t\} \uplus E \rangle}{\langle \mathcal{Q};\ \{\lambda x.\, s = \lambda x.\, (t\, x)\} \uplus E \rangle}\ \textsc{eta}\ (t \text{ not an abstraction})$$

Here and below a rule is read downwards as one transformation step, $\uplus$ is multiset union, x is fresh, and ETA also applies with the two sides exchanged.

A.7 First-order unification

Term reduction, reorientation and variable elimination with occurs check; higher-order pattern unification has the same two critical parts, simplification and variable elimination [MN p. 224].

$$\frac{E \uplus \{f\, t_1 \ldots t_n = f\, s_1 \ldots s_n\}}{E \uplus \{t_1 = s_1, \ldots, t_n = s_n\}}\ \textsc{decompose} \qquad \frac{E \uplus \{f\, \bar{t} = g\, \bar{s}\}}{\bot}\ \textsc{clash}\ (f \neq g \text{ or different types})$$

$$\frac{E \uplus \{t = x\}}{E \uplus \{x = t\}}\ \textsc{orient}\ (t \text{ not a variable}) \qquad \frac{E \uplus \{x = x\}}{E}\ \textsc{delete} \qquad \frac{E \uplus \{x = t\}}{\bot}\ \textsc{check}\ (x \in \mathrm{fv}(t),\ t \neq x)$$

$$\frac{E \uplus \{x = t\}}{E[t/x] \uplus \{x = t\}}\ \textsc{eliminate}\ (x \notin \mathrm{fv}(t),\ x \in \mathrm{fv}(E))$$

Figure A.4: Term reduction, reorientation and variable elimination [MN p. 28]; the occurs check is mandatory [MN p. 32].

Theorem A.19 ([MN p. 28–29])**.** *Every sequence of steps (never eliminating twice on the same unchanged equation) terminates in* $\bot$ *or in a* solved form *(each left side a variable occurring nowhere else), which is a most general unifier, unique up to renaming.*

A.8 Huet's procedure

On a normalised problem (Appendix A.6) Huet's procedure simplifies rigid-rigid equations and guesses the head of a flexible term facing a rigid one [MN p. 214–216].

$$\frac{\langle \mathcal{Q};\ \{c\,s_1 \dots s_n = c\,t_1 \dots t_n\} \uplus E\rangle}{\langle \mathcal{Q};\ \{s_1 = t_1, \dots, s_n = t_n\} \uplus E\rangle}\ \textsc{simp}\ (c \text{ universal}) \qquad \frac{\langle \mathcal{Q};\ \{c\,\bar{s} = d\,\bar{t}\} \uplus E\rangle}{\bot}\ \textsc{clash}\ (c \neq d \text{ universal})$$

$$\frac{\langle \mathcal{Q};\ \{r = f\} \uplus E\rangle}{\langle \mathcal{Q};\ \{f = r\} \uplus E\rangle}\ \textsc{swap}\ (r \text{ rigid},\ f \text{ flexible})$$

$$\frac{\langle \mathcal{Q}_1\, \exists F\, \mathcal{Q}_2;\ \{F\,t_1 \dots t_n = c\,s_1 \dots s_m\} \uplus E\rangle}{\langle \mathcal{Q}_1\, \exists H_1 \dots \exists H_m\, \mathcal{Q}_2;\ \theta(\{F\,\bar{t} = c\,\bar{s}\} \uplus E)\downarrow_\beta\rangle}\ \textsc{imitate}$$

$$\theta = \{\langle F,\ \lambda x_1 \dots \lambda x_n\,(c\,(H_1\,x_1 \dots x_n) \cdots (H_m\,x_1 \dots x_n))\rangle\}, \quad \forall c \in \mathcal{Q}_1, \quad H_i : \tau_1 \to \dots \to \tau_n \to \sigma_i$$

$$\frac{\langle \mathcal{Q}_1\, \exists F\, \mathcal{Q}_2;\ \{F\,t_1 \dots t_n = c\,s_1 \dots s_m\} \uplus E\rangle}{\langle \mathcal{Q}_1\, \exists H_1 \dots \exists H_k\, \mathcal{Q}_2;\ \theta(\{F\,\bar{t} = c\,\bar{s}\} \uplus E)\downarrow_\beta\rangle}\ \textsc{project}_i$$

$$\theta = \{\langle F,\ \lambda x_1 \dots \lambda x_n\,(x_i\,(H_1\,x_1 \dots x_n) \cdots (H_k\,x_1 \dots x_n))\rangle\}, \quad \tau_i = \varphi_1 \to \dots \to \varphi_k \to \beta, \quad H_j : \tau_1 \to \dots \to \tau_n \to \varphi_j$$

Figure A.5: Huet's transformation rules [MN p. 214–216], with $F : \tau_1 \to \dots \to \tau_n \to \beta$, $c : \sigma_1 \to \dots \to \sigma_m \to \beta$, β primitive. IMITATE needs F in the scope of c; $\textsc{project}_i$ needs the target of τ_i to be β; so at most $n+1$ choices exist per equation [MN p. 216]. The rewriting format is ours, in the style of the transformation-based presentation of Huet's procedure [25] mentioned on [MN p. 227]; side conditions and substitution terms are the book's.

Definition A.20 (pre-unification; canonical substitution [MN p. 217–218])**.** Reducing a problem to one with only flexible-flexible equations is *pre-unification*, and the composed substitution is a *pre-unifier*. The *canonical substitution* maps each existential of type $\tau_1 \to \dots \to \tau_n \to \sigma$ (σ primitive) to $\lambda x_1 \dots \lambda x_n\, H^\sigma$, with one fixed new existential H^σ per primitive type; on a flexible-flexible problem it is a unifier.

Theorem A.21 (matching trees [MN p. 219–220])**.** *In the matching tree (nodes: simplified matrices; arcs: imitation/projection substitutions for one selected flexible-rigid equation; leaves: failure, or success when only flexible-flexible equations remain), if the root problem has a unifier then a success node exists at finite depth, and that unifier is the composed pre-unifier followed by a unifier of the success node, whichever equation is selected at each node. Since unifiability is undecidable,*

some trees have an infinite path and no success node [MN p. 220]. *As a test for unifiability the procedure is nonredundant* [MN p. 227]*; mgus may not exist, complete sets of unifiers may be infinite, and searching for them must be redundant* [MN p. 213,226].

Example A.22 (a matching tree [MN p. 216–219])**.** For $\forall a\,\forall g\,\exists F\,[(F\,a) = (g\,a\,a)]$, PROJECT$_1$ ($F := \lambda x.\,x$) gives $a = (g\,a\,a)$, a CLASH; IMITATE ($F := \lambda x.\,g\,(H_1\,x)\,(H_2\,x)$) and SIMP give $\{(H_1\,a) = a,\ (H_2\,a) = a\}$, and each equation again has an imitation ($\lambda x.\,a$) and a projection ($\lambda x.\,x$), both succeeding. The four leaves compose to the four unifiers of Example A.17. In $\forall g\,\exists F\,\forall x\,[(F\,(g\,x)) = (g\,(F\,x))]$, imitation reproduces the same problem for a new H, so the tree has an infinite path, alongside the unifiers $\lambda x.\,x$, $\lambda x.\,g\,x$, $\lambda x.\,g\,(g\,x)$, … [MN p. 218].

A.9 Higher-order pattern unification

In L_λ every flexible term is a pattern, at most one substitution is useful for each flexible-rigid equation, and the procedure becomes a deterministic algorithm with most general unifiers [MN p. 221–223].

Definition A.23 (pattern [MN p. 220])**.** A flexible term $(F\,c_1 \ldots c_n)$ $(n \geq 0)$ is a *pattern* if $c_1, \ldots, c_n$ are distinct universal variables quantified to the right of F (within its scope); $\bar{c} \cap \bar{d}$ denotes some listing of the variables common to $\bar{c}$ and $\bar{d}$.

Theorem A.24 ([MN p. 208,221–224])**.** *On L_λ problems the rules of Fig. A.6 without* DELAY *terminate (thanks to the occurs check,* [MN p. 222–223]*) and return $\bot$ or a most general unifier: L_λ unification is decidable and unitary [34]. Types are never consulted to build the unifier* [MN p. 224]*; the untyped variant fails rigid-rigid equations of different arities [34, § 9.3].*

Example A.25 (ELIM, OCCURS, SCOPE [MN p. 223–224])**.** In $\forall f\,\forall g\,\exists U\,\exists V\,\forall w\,\forall x\,\forall y\,[(f\,(U\,x\,y)) = (f\,(g\,(V\,y\,w)))]$, RR-DEC leaves $(U\,x\,y) = (g\,(V\,y\,w))$; ELIM prunes $V := \lambda y.\,\lambda w.\,V'\,y$ (since $w \notin \{x, y\}$) and binds $U := \lambda x.\,\lambda y.\,g\,(V'\,y)$, a most general unifier. With right side $(g\,(U\,y\,w))$ instead, OCCURS fails; with $(g\,w)$, SCOPE fails, because w is not among the arguments of U and is not inside a flexible subterm.

Example A.26 (ELIM with pruning [MN p. 224–225])**.** In $\forall f\,\exists U\,\forall x\,\forall y\,\forall z\,\exists V\,[(U\,x) = (f\,V)]$, raising V over x, y, z gives $(U\,x) = (f\,(V'\,x\,y\,z))$; ELIM prunes $V' := \lambda x \lambda y \lambda z\,(W\,x)$ and binds $U := \lambda x\,(f\,(W\,x))$.

Implementations of the same rules. Nipkow's functional version works on de Bruijn terms with a $\forall\exists\forall$ prefix [48]; Dowek, Hardin, Kirchner and Pfenning recast the rules (Invert, Pruning, Same-variable) in the $\lambda\sigma$-calculus of explicit substitutions, where raising is the rule Anti-Exp-λ [8, p. 8–9]; Nadathur and Linnell drop the prefix in favour of level tags [44] (Appendix A.10).

$$\frac{\langle \mathcal{Q};\ \{\lambda x.\, s = \lambda x.\, t\} \uplus E\rangle}{\langle \mathcal{Q}\ \forall x;\ \{s = t\} \uplus E\rangle}\ \textsc{abs} \qquad \frac{\langle \mathcal{Q};\ \{\lambda x.\, s = t\} \uplus E\rangle}{\langle \mathcal{Q};\ \{\lambda x.\, s = \lambda x.\, (t\ x)\} \uplus E\rangle}\ \textsc{eta}\ (t \text{ not an abstraction})$$

$$\frac{\langle \mathcal{Q};\ \{r = F\,\bar{c}\} \uplus E\rangle}{\langle \mathcal{Q};\ \{F\,\bar{c} = r\} \uplus E\rangle}\ \textsc{swap}$$

$$\frac{\langle \mathcal{Q};\ \{h\,s_1 \ldots s_n = h\,t_1 \ldots t_n\} \uplus E\rangle}{\langle \mathcal{Q};\ \{s_1 = t_1, \ldots, s_n = t_n\} \uplus E\rangle}\ \textsc{rr-dec}\ (h \text{ universal}) \qquad \frac{\langle \mathcal{Q};\ \{h\,\bar{s} = k\,\bar{t}\} \uplus E\rangle}{\bot}\ \textsc{rr-clash}\ (h \neq k)$$

$$\frac{\langle \mathcal{Q};\ \{F\,\bar{c} = G\,\bar{d}\} \uplus E\rangle}{\langle \mathcal{Q}\ \exists H;\ \theta(E)\rangle}\ \textsc{ff-diff}\ (F \neq G) \qquad \theta = \{\langle F, \lambda\bar{c}.\, H\,(\bar{c} \cap \bar{d})\rangle,\ \langle G, \lambda\bar{d}.\, H\,(\bar{c} \cap \bar{d})\rangle\}$$

$$\frac{\langle \mathcal{Q};\ \{F\,c_1 \ldots c_n = F\,d_1 \ldots d_n\} \uplus E\rangle}{\langle \mathcal{Q}\ \exists H;\ \theta(E)\rangle}\ \textsc{ff-same} \qquad \theta = \{\langle F, \lambda\bar{c}.\, H\,\bar{e}\rangle\},\ \bar{e} = \text{the } c_i \text{ with } c_i = d_i$$

$$\frac{\langle \mathcal{Q};\ \{F\,\bar{c} = t\} \uplus E\rangle}{\langle \mathcal{Q}\ \exists \bar{G}';\ \theta(\rho(E))\rangle}\ \textsc{elim} \qquad \begin{array}{l} t \text{ rigid};\ \rho = \{\langle G, \lambda\bar{d}.\, G'\,(\bar{d} \cap \bar{c})\rangle \mid (G\,\bar{d}) \text{ flexible in } t\} \text{ (pruning)};\\ t' = \rho(t);\ F \notin \mathrm{fv}(t');\ \text{every universal of } t' \text{ in the scope of } F \text{ is in } \bar{c};\\ \theta = \{\langle F, \lambda\bar{c}.\, t'\rangle\} \end{array}$$

$$\frac{\langle \mathcal{Q};\ \{F\,\bar{c} = t\} \uplus E\rangle}{\bot}\ \textsc{occurs} \qquad (t \text{ rigid},\ F \in \mathrm{fv}(\rho(t)))$$

$$\frac{\langle \mathcal{Q};\ \{F\,\bar{c} = t\} \uplus E\rangle}{\bot}\ \textsc{scope} \qquad (t \text{ rigid},\ \rho(t) \text{ has a universal in the scope of } F \text{ not in } \bar{c})$$

$$\frac{\langle \mathcal{Q};\ \{s = t\} \uplus E\rangle}{\langle \mathcal{Q};\ E\rangle\ \text{with } s = t \text{ deferred}}\ \textsc{delay}$$

(a flexible side of $s = t$ is not a pattern; the equation is resumed when substitutions change its status)

Figure A.6: Pattern unification on ∀∃∀ problems [MN p. 212,214,221–223]; DELAY is the dynamic-L_λ extension [MN p. 225–226]. $\mathcal{Q}\ \exists H$ means a new existential in the ∃ block; the bindings θ (and ρ) are composed into the answer substitution. Rule names are ours.

A.10 Universe levels

Tagging every eigenvariable and logic variable with a number replaces the explicit prefix: the tag records which constants a variable may see [43, p. 15–18].

$$\frac{\langle G_1 \wedge G_2, \mathscr{P}, I\rangle}{\langle G_1, \mathscr{P}, I\rangle,\ \langle G_2, \mathscr{P}, I\rangle}\ (1) \qquad \frac{\langle G_1 \vee G_2, \mathscr{P}, I\rangle}{\langle G_i, \mathscr{P}, I\rangle}\ (2)\ (i = 1, 2) \qquad \frac{\langle \exists x\, G, \mathscr{P}, I\rangle}{\langle G[w/x], \mathscr{P}, I\rangle}\ (3)\ (w \text{ new, tag } I)$$

$$\frac{\langle (D_1 \wedge \cdots \wedge D_n) \supset G, \mathscr{P}, I\rangle}{\langle G, \mathscr{P} \cup \{D_1, \ldots, D_n\}, I\rangle}\ (4) \qquad \frac{\langle \forall x\, G, \mathscr{P}, I\rangle}{\langle G[c/x], \mathscr{P}, I+1\rangle}\ (5)\ (c \text{ new, tag } I+1)$$

$$\frac{\langle A, \mathscr{P}, I\rangle}{\sigma(\langle G, \mathscr{P}, I\rangle)}\ (6)\ (\text{instance } G \supset A') \qquad \frac{\langle A, \mathscr{P}, I\rangle}{\text{solved}}\ (7)\ (\text{instance } A')$$

Figure A.7: The abstract interpreter on decorated goals $\langle G, \mathscr{P}, I\rangle$ [43, p. 17–18]. In (6) and (7) the clause instance is a new tagged instance relative to I of a clause of $\mathscr{P}$, and $\sigma = \mathrm{mgu}(A, A')$ is applied to all remaining goals. A new tagged instance relative to I gives tag 1 to untagged constants and a new variable of tag I to each $\forall x$ of the clause; the original goal starts at $I = 1$ with its constants and free variables tagged 1.

Definition A.27 (binding condition and lowering [43, p. 16])**.** A logic variable X of tag i may be bound to t only if (a) X does not occur in t (occurs check) and (b) no constant in t has a tag greater than i. Before the binding is made, every variable in t with a tag greater than i has its tag *lowered* to i, so that its own later instances respect the restriction on X.

Proposition A.28 ([43, Prop. 4.1])**.** *Derivations of the interpreter are sound and complete with respect to intuitionistic provability, and the choice of the next decorated goal to reduce is immaterial.*

Example A.29 (scope failure [MN p. 85,106])**.** `?- sigma x\ pi y\ x = y.` fails: the query gives X^1, the $\forall$ gives c^2, and $X^1 := c^2$ violates (b). With the program $\forall z\,(q(z) \supset p(d(z)))$, the goal $\exists x\,\forall y\,(q(y) \supset p(x))$ becomes $q(c^2) \supset p(X^1)$, at $I = 2$; backchaining creates Z^2, unifying $p(d(Z^2))$ with $p(X^1)$ binds $X^1 := d(Z^2)$ and *lowers* Z to tag 1, so the subgoal $q(Z^1)$ fails against the hypothesis $q(c^2)$; without lowering, $Z^2 := c^2$ would succeed and smuggle c into X [43, p. 16].

Levels for higher-order patterns [44, p. 8–9]. Let $l(y)$ count the times an $\exists$ is immediately followed by a $\forall$ in the prefix up to and including the quantifier of y; then a universal u is in the scope of an existential x iff $l(x) < l(u)$, which makes the pattern test of Definition A.23 local. Raising of g is restricted to $\bar{y}\Uparrow g = \{u \in \bar{y} \mid l(u) \leq l(g)\}$. New existentials created by imitation, projection, pruning or flexible-flexible steps are quantified "at the same place" as the variable they replace [MN p. 221–222,225], i.e. they take its level, not a new one. The book describes the

same device as lists of universals "over which it can be raised", initial segments of the prefix, represented by numerical indices [MN p. 225].

Remark A.30 (higher-order binding condition; reconstruction in our notes, not in the book). In ELIM for $X\,\bar{c} = t$ the tag test (b) applies to universals in rigid positions of t not among $\bar{c}$ (SCOPE in Fig. A.6); inside the arguments of a flexible $(Y\,\bar{d})$ such universals are pruned instead; and lowering a Y of higher level binds Y to $(Y'\,\bar{a})$ with Y' at the level of X, which is delayed raising [MN p. 225].

Kanren angle: what levels require

A miniKanren with GENERIC goals must record a level for every logic variable and eigenvariable and must enforce the binding condition, with lowering, inside its unifier; see Chapter 11.

A.11 Summary

- Syntax: Table A.1; grammars of *fohc*, *fohh*, *hohc*, *hohh*, $hohh^+$ and L_λ: Table A.2 and Definition A.5.
- Proof search: right rules Fig. A.1, backchaining Fig. A.2, completeness Table A.3.
- Terms: typing Fig. A.3, conversion Table A.4, normal forms Definition A.11 and Theorem A.12.
- Prefixes: unifier vs solution Definition A.16, prefix order Example A.17, raising Proposition A.18, problem normal form Appendix A.6.
- Unification: first-order Fig. A.4, Huet Fig. A.5 and Example A.22, patterns Fig. A.6 and Example A.25.
- Scope by numbers: Fig. A.7 and Definition A.27; lowering is what makes the second half of Example A.29 fail.

B Glossary

The preceding chapters use vocabulary from three communities. The first is the proof theory of Miller and Nadathur's book [36], where you find sequents, uniform proofs and eigenvariables. The second is the literature on unification of λ-terms (patterns, raising, pruning, universe levels). The third is miniKanren, with streams, interleaving, soft cut and reification, and in particular the engine (aux kanren micro) that we want to extend. The same word sometimes means different things in different communities ("suspension", "level", "scope extrusion", "fresh"), and the same idea sometimes has three names (GENERIC, `pi`, `all`). This appendix collects all of them in one alphabetical list. Each entry gives a short, precise definition and tells you where the concept is treated in this document and in the book.

B.1 How to read this glossary

Pointers. Most entries end with a pointer of the form "→ Chapter 3; [MN p. 82]". The first part is the chapter of this document where the concept is developed. The second part is a *printed* page of the book [36]; most of these come from the book's own index. Entries that come only from the miniKanren literature or from the aux sources have no book page; they cite the paper instead. Names of (aux kanren micro) procedures and macros are set in `typewriter` type. We describe only what exists today. The new operators are the subject of Chapters 11 and 12.

Notation. Table B.1 lists the symbols that occur most often in the entries.

Table B.1: Notation used throughout the glossary.

Symbol	Meaning
$\Sigma; \mathscr{P} \longrightarrow G$	sequent: signature Σ, program $\mathscr{P}$, goal G [MN p. 39]
$\Sigma; \mathscr{P} \xrightarrow{D} A$	focused sequent: the clause D is selected to prove the atom A [MN p. 44]
G, D	goal formula, program clause (definite formula)
$\top, \wedge, \vee, \supset, \forall, \exists$	logical constants; concrete syntax `true`, `&` or `,`, `;`, `=>` or `:-`, `pi`, `sigma`
$\lambda x.t$	abstraction; λProlog `x\ t`
$s[t/x]$	capture-avoiding substitution of t for x in s [MN p. 85]
$=_{\beta\eta}$, λ-conversion	equality of terms modulo α, β, η [MN p. 100]
$\tau \to \sigma$, o	function type; type of formulas
$Q_1 x_1 \ldots Q_n x_n[\bigwedge_j t_j = s_j]$	unification problem under a mixed prefix [MN p. 105]
$a \# t$	nominal freshness: the atom a does not occur free in t
$\langle \phi; t \rangle$	static closure of Ma and Friedman: term t under the scope ϕ
°, ª, §, ð	aux suffixes and glyphs: goal or relation, applicative form, stream, delay

Pitfall: one word, several meanings

Several terms below are overloaded across the sources. Keep the distinctions in mind when you read the papers.

- **Suspension.** In (aux kanren micro), a suspension is a `δ`-promise, the immature part of a stream. In nominal unification, $\pi \cdot X$ is a permutation suspended on a variable. In Teyjus, the suspension $[[t, ol, nl, e]]$ is an explicit-substitution term. In ELPI, a suspended goal is a syntactic constraint.
- **Level.** A de Bruijn level numbers a bound variable from the root inwards. A universe level, or universe index, is the `∀`-nesting depth at which a variable or constant was created. ELPI makes the two coincide, but they are different ideas.
- **Scope extrusion.** The book uses this name for a classical equivalence that moves an implication over a disjunction [MN p. 89]. In the π-calculus it means sending a restricted name out of its scope. In implementation discussions it means an eigenvariable escaping into an older variable.
- **Fresh.** In miniKanren, `fresh` is the existential quantifier and introduces logic variables. In αKanren, `fresh` introduces *noms*, which are names rather than variables.
- **Cut.** The cut rule of the sequent calculus has no connection to Prolog's pruning operator `!` [MN p. 63].
- **Substitution and grafting.** Instantiating a unification variable is capture-avoiding in standard higher-order unification. It is capture-permitting (grafting) in the Ma–Friedman and DHKP formulations.

B.2 Terms from A to Z

A

Abstract datatype A module that exports a type constructor and the predicates over it but hides the data constructors, so other code can use only the exported operations (book Ch. 6). → [MN p. 165].

Abstract logic programming language A logic in which searching for uniform proofs is a complete proof procedure. This is the book's criterion for a logic to count as a logic programming language. fohc, fohh, hohc and hohh all qualify [37]. → Chapter 2; [MN p. 73].

Abstraction The term $\lambda x.t$ (λProlog `x\ t`), the only binder of the term language. x is the bound variable and t the body. The quantifiers are the constants `pi` and `sigma` applied to abstractions. → Chapter 4; [MN p. 97].

Accumulation (module, signature) `accumulate` inlines the code of other modules after renaming their private names. `accum_sig` inserts the declarations of other signatures, either to re-export them or to declare a parameter that a later module supplies. → [MN p. 154], [MN p. 155].

Administrative redex A β-redex that a program transformation (here the CPS transforma-

tion) introduces itself, as opposed to one present in the source. The two-phase CPS of book Ch. 10 marks these redexes and reduces them afterwards. → Chapter 8; [MN p. 257].

α-conversion Renaming bound variables. Terms that differ only by α-conversion are identified, so the names of bound variables are "a fiction" and an implementation may not represent them at all. → Chapters 4 and 6; [MN p. 100], [MN p. 183], [MN p. 205].

αKanren Byrd and Friedman's nominal miniKanren [5, 4]. `fresh` introduces noms, `(tie a t)` binds the nom a in t, and `(hash a t)` asserts the freshness $a \# t$. Unification is nominal unification. The implementation does not support disequality constraints. → Chapter 9.

AND The goal-reduction rule for $G_1 \wedge G_2$: prove both conjuncts with the same signature and program. → Chapter 2; [MN p. 39].

Anonymous variable `_`: each occurrence is a distinct variable, and its binding is not reported. → Chapter 2; [MN p. 47].

Answer substitution The bindings that a successful search reports for the free variables of a query. These variables are implicitly existential. The miniKanren counterpart is the reified answer. → Chapter 2; [MN p. 43].

Applicative form An aux naming convention. The suffix `ª` marks the procedure behind a goal macro: `freshª` behind `fresh°`, and `andª`, `orª` behind `and°`, `or°`. → Chapter 10.

`assume`, `all` The λKanren goals for $D \supset G$ and $\forall x\, G$. See *AUGMENT*, *GENERIC* and *λKanren*. → Chapter 9.

Atom (nominal), nom, name In nominal logic and αKanren, a constant-like name that stands for an object-level variable. A nom unifies only with itself or with an unbound logic variable. → Chapter 9.

Atomic formula A term of type o whose head is a predicate symbol, that is, a constant with target type o. → Chapter 2; [MN p. 34].

AUGMENT The goal-reduction rule for $D \supset G$: add D to the program and prove G. This rule is the basis of hypothetical reasoning; the clause is visible only during the proof of G. λProlog writes `D => G`; λKanren calls it `assume`.

$$\frac{\Sigma; \mathscr{P} \cup \{D\} \longrightarrow G}{\Sigma; \mathscr{P} \longrightarrow D \supset G}\ \textsc{augment}$$

→ Chapter 3; [MN p. 39], [MN p. 77].

Available level (Ma–Friedman) A de Bruijn level of a scope ϕ whose binder name is not rebound further in. A shadowed binder's level is unavailable, so $\lambda a.\lambda a.X \stackrel{?}{=} \lambda c.\lambda d.c$ fails [29]. → Chapter 9.

B

Backchaining Proving an atomic goal A by selecting a program clause, possibly an assumed one, whose head matches A, and then proving the clause's body. It is the only phase of search that consults the program. The book formalises it with focused sequents and the rules DECIDE, INITIAL, $\supset$L, $\wedge$L and $\forall$L. → Chapter 2; [MN p. 3], [MN p. 44].

β-conversion, β-redex, β-reduction A β-redex is $(\lambda x.\, s)\, t$, and it reduces to $s[t/x]$. β-reduction is the computation of functional programming, and it is part of the equality of λProlog terms. → Chapter 4; [MN p. 100]–[MN p. 101].

β_0-conversion The special case $(\lambda x.\, s)\, x = s$ of β. With α, it is equivalent to $(\lambda y.\, s)\, x = s[x/y]$ when x is not free in $\lambda y.\, s$, so it only renames. It is the only β needed in L_λ. → Chapters 6 and 7; [MN p. 206], [MN p. 220].

β-normal form A term without β-redexes. It has the shape $\lambda x_1 \ldots \lambda x_n.\, (h\, t_1 \ldots t_m)$, with *binder* $x_1 \ldots x_n$, *head* h and *arguments* $t_1 \ldots t_m$. → Chapter 4; [MN p. 101], [MN p. 111].

$\beta\eta$-long normal form A β-normal form whose arguments are themselves in long form and whose body has primitive type. Every β-normal form can be η-expanded into an equal long one; for $f : (i \to i) \to i$, the long form of f is $\lambda w.\, f\, (\lambda u.\, w\, u)$. Higher-order unification works on these forms. → Chapter 4; [MN p. 111].

Big-step / small-step semantics A big-step (natural) semantics relates a term to its final value. A small-step (structural operational) semantics relates a term to the result of one atomic step. → Chapter 8; [MN p. 250]–[MN p. 251].

Binary clause A clause whose body is at most one, possibly complex, atom. Every clause can be put in this form by threading a continuation goal $K : \mathrm{o}$ (continuation-passing style). → Chapter 5; [MN p. 130].

Binder A syntactic construct that introduces a locally scoped name: a quantifier, λ, $\int$, Σ or a local declaration. In λ-tree syntax, object-level binders are encoded by meta-level λ. → Chapter 6.

Binder mobility The way a binder moves across levels during computation. A term-level λ becomes a formula-level $\forall$ (`pi`), which in turn becomes a proof-level eigenvariable. This movement lets λProlog programs descend under binders without managing names. → Chapter 6; [MN p. 184], [MN p. 205].

Binder-merging (Ma–Friedman) The implementation technique of renaming one side's binders to the other side's before comparing the bodies structurally. With name shadowing and a naive reading of levels back into names, it can make Church *false* $\lambda a.\lambda a.a$ unify with Church *true* $\lambda c.\lambda d.c$. → Chapter 9.

Bisimulation, simulation Coinductive behavioural equivalence and preorder on processes. Book Ch. 11 shows that a simulation written with `pi` is incorrect, because `pi` is intensional. → Chapter 8; [MN p. 271].

C

Canonical substitution For a problem whose equations are all flex-flex: fix one new existential variable H^σ for each primitive type σ, and map every existential variable of type $\tau_1 \to \ldots \to \tau_n \to \sigma$ to $\lambda x_1 \ldots \lambda x_n.\, H^\sigma$. This substitution always unifies such a problem, which is why a flex-flex remainder counts as success. → Chapter 7; [MN p. 217]–[MN p. 218].

Capture-avoiding substitution $s[t/x]$, carried out after renaming bound variables of s so that no free variable of t is captured. It is defined only up to α. Contrast grafting. → Chapters 3 and 4; [MN p. 85]–[MN p. 86], [MN p. 100].

Church numeral $\ulcorner n \urcorner = \lambda f.\lambda x.\, f^n x$. The book uses Church numerals to show β-reduction as computation and to encode Diophantine equations as unification problems. → Chapter 4; [MN p. 102], [MN p. 115].

Clausal order The nesting depth of implications to the left in a formula, analogous to the order of a type. → Chapter 3; [MN p. 76].

Closed-world / open-world assumption Whether the world, that is, the signature–program pair, can grow. Under the open-world assumption that λProlog adopts, generic and universal judgments coincide. → Chapter 3; [MN p. 94].

Complete set of unifiers (CSU) A set of unifiers such that every unifier is an instance of some member. Higher-order problems have CSUs, possibly infinite, but in general no mgu. One might ask instead for a *non-redundant* CSU, in which no two members have a common instance, but the book notes that this cannot be guaranteed in general either. → Chapter 7; [MN p. 213].

Completeness (of a search strategy) In the book, completeness is a property of a proof system: uniform proofs exist whenever proofs do. A search *strategy* is a different matter. Depth-first search is incomplete. miniKanren's interleaving search is complete for answers, meaning every answer with a finite derivation is eventually produced [52]. It does not guarantee termination or finite failure. → Chapters 2 and 10.

Computation-as-deduction The view that computation is the construction or transformation of proofs. It comes in two forms: *proof normalisation* (functional programming) and *proof search* (logic programming). *Computation-as-model* instead uses logic to reason about computations. → Chapter 1; [MN p. 2].

`cond°` (aux kanren micro)'s `conde`: an interleaving disjunction of conjunctions of goals. There is no `else` keyword. → Chapter 10.

Consistent / inconsistent program A program is inconsistent if every formula is provable from it. Every fohc and fohh program is consistent. Clause heads must be rigid: with the two clauses `P 5 :- q.` and `q.`, whose first head is flexible, any formula becomes provable. → Chapter 5; [MN p. 119].

Constant clash The failure of first-order unification on an equation whose two sides have distinct head constants. → Chapter 1; [MN p. 30].

Context (Ma–Friedman) The set of postponed equations between static closures of unification variables. A solution in their algorithm is a pair (grafting, context), much like Huet's postponed flex-flex pairs. → Chapter 9.

Constraint store ((aux kanren micro)) Besides the substitution `S`, a state carries three stores: `D` for disequalities (`≠°`), `A` for type tags (`symbol°`, `number°`) and `T` for absento constraints (`absent°`). After a unification, the constraints touched by its new bindings are checked again. → Chapter 10.

Constructive negation (Moiseenko) Negation for miniKanren that collects all the answers of g and negates their disjunction. The states then contain universally quantified disequalities, which need a dedicated solver. It requires stratification, diverges on goals with infinitely many answers, and defines $\forall$ as $\neg\exists\neg$ [41]. → Chapter 9.

Contraction The structural rule that duplicates a hypothesis into a premise. It is the source of non-termination in naive provers for propositional intuitionistic logic. Contraction-free calculi such as Dyckhoff's terminate. → Chapter 8; [MN p. 245].

Copy clauses Clauses `copy` that specify structural equality over an object signature, one clause per constructor. Adding the hypothesis `copy c T` for an eigenvariable `c` turns copying into object-level substitution, entirely inside L_λ. → Chapter 6; [MN p. 199].

CPS (continuation-passing style) In book Ch. 5, the threading of a continuation goal that yields binary clauses. In book Ch. 10, Fischer's CPS transformation of functional programs. → Chapters 5 and 8; [MN p. 131], [MN p. 256].

Cut rule, cut elimination The cut rule is the lemma rule of the sequent calculus. Gentzen's theorem shows it is admissible [16]. Proof search looks only for cut-free proofs. The cut rule is unrelated to Prolog's `!`. → Chapter 2; [MN p. 63], [MN p. 72].

Cut, Prolog's `!` The pruning operator. λProlog defines `if` and `not` with it, and it affects completeness. (aux kanren micro) has no `!`. Its closest tools are the soft cut `if°` and `(take° 1 g)`. → Chapters 5 and 10; [MN p. 137].

D

D-formula, G-formula Program clauses and goals. Each language defines them by a pair of grammars. For fohc: $G ::= \top \mid A \mid G \wedge G \mid G \vee G \mid \exists x\, G$ and $D ::= A \mid G \supset D \mid D \wedge D \mid \forall x\, D$. fohh adds $D \supset G$ and $\forall x\, G$ to the goals. → Chapters 2 and 3; [MN p. 43], [MN p. 76].

de Bruijn index, de Bruijn level Nameless representations of bound variables [3]. An *index* counts binders outwards from the occurrence (1-based in the book). A *level* counts binders inwards from the root. In $\lambda c.\lambda d.c$, c has level 0 and index 1 when counting from 0. Teyjus uses indices in terms and levels in the context. ELPI uses levels throughout. → Chapters 6 and 9; [MN p. 192].

Decide, initial The two rules that start and end backchaining. DECIDE picks a clause $D \in \mathscr{P}$ to focus on. INITIAL closes a focused atom whose head matches the goal. → Chapter 2;

[MN p. 45].

`define-relation` The (aux kanren micro) macro that defines a relation. It is inverse-η-delayed: the body is built only when a state arrives, and then it suspends. → Chapter 10.

Definite formula A program formula that does not contain "indefinite information of the kind let in by disjunctions". Disjunction (and, by the grammar, top-level ∃) is banned from programs. → Chapter 2; [MN p. 42].

Delay, `δ` In (aux kanren micro), `δ` is CHICKEN's memoising `delay`. A promise is an immature stream and is the unit of interleaving. → Chapter 10.

Delayed (postponed) equations Equations outside the decidable fragment, or flex-flex pairs, kept as constraints until further instantiation makes them tractable. Teyjus, DHKP [8] and the Ma–Friedman *context* postpone them. ELPI raises an error by default. The book's version is dynamic L_λ. → Chapters 7 and 9; [MN p. 225]–[MN p. 226].

Delayed raising Instead of raising eagerly, keep with each existential variable the list of universals it may be raised over, and raise only when a binding needs it. These lists are initial segments of the prefix, so numerical indices represent them [44]. → Chapter 7; [MN p. 225].

Determinate type A type whose type variables are all transparent, that is, occur in the target type. → Chapter 2; [MN p. 67].

Difference list, functional difference list `(dl L K)` denotes L minus its suffix K. `(fdl F)`, with `F : list A -> list A`, denotes `F nil`, and appending is function composition. → Chapter 5; [MN p. 140].

Disagreement set In nominal unification, the set of atoms on which two permutations differ. In Huet-style presentations, the multiset of pairs still to be unified. → Chapter 9.

Disequality constraint `≠°` in (aux kanren micro), stored in the store `D` as the list of bindings that would make the two terms equal. Moiseenko's constructive negation needs universally quantified disequalities instead [41]. → Chapters 9 and 10.

Disjunction / existence property In intuitionistic logic, $\vdash F_1 \vee F_2$ implies $\vdash F_i$ for some i, and $\vdash \exists x\, F$ yields a witness. Harrop formulas guarantee these properties, which is why answer substitutions make sense. → Chapter 3; [MN p. 93].

Dynamic L_λ The practical regime in which L_λ is not imposed statically on clauses. Instead, the unification problems met at run time are expected to fall inside it, as in `subst R N (R N)` with `R` and `N` already instantiated. → Chapter 7; [MN p. 225]–[MN p. 226].

E

E-formula $E ::= D \mid E \wedge E \mid \exists x\, E$: a program formula with existentially quantified, local names. It gives modules with hidden constants a logical reading. → [MN p. 160]–[MN p. 162].

Eigenvariable, scoped constant The new constant that GENERIC (∀R) introduces to prove

$\forall x\, G$. It is not in the current signature and is never instantiated. The *eigenvariable condition* says that no logic variable created before it may be bound to a term containing it. The book's example is the sterile jar: to prove `pi x\ bug x => in x j => dead x`, pick a new bug constant and reason about it alone. → Chapters 3 and 4; [MN p. 82]–[MN p. 83].

`eigen` (Hemann, Byrd, Friedman) The operator of miniKanren-with-symbolic-constraints that creates eigen objects and installs an "eigen-absento" constraint over the list of all variables created so far. It is supported only together with == [21]. → Chapter 9.

Elementary substitution Huet's partial bindings for a flex-rigid pair: imitation and projection. In Ma–Friedman's reading, each unification variable takes every visible bound variable as an argument. → Chapter 7.

ELPI An embeddable λProlog interpreter [9]. It uses de Bruijn levels and annotates unification variables with levels X^j. It reports an error outside the pattern fragment by default, and it offers syntactic constraints. → Chapter 9.

Empty type A type with no closed inhabitant. Higher-order intuitionistic logic does not exclude empty types, and they matter for ∃ answers and for unification *solutions*. → Chapter 4; [MN p. 108].

`eq?`-invariant ((aux kanren micro)) A unification that adds no binding returns the very same state object. The disequality and entailment checks rely on this to recognise that nothing new was learned. → Chapter 10.

Essentially universal / essentially existential A classification of bound-variable occurrences. In a goal, an occurrence is essentially universal if it is bound by a positive ∀, a negative ∃, or a term-level λ. Otherwise it is essentially existential. Clauses use the dual classification. During search, essentially existential variables become logic variables and essentially universal ones become eigenvariables. → Chapter 6; [MN p. 205].

η-conversion $\lambda x.\,(t\ x) = t$ when x is not free in t. → Chapter 4; [MN p. 101].

Evaluation context A term with a hole that marks where the next redex is. In book Ch. 10 it is a meta-level function `tm -> tm`. → Chapter 8; [MN p. 253].

Ex falso quodlibet From ⊥, infer anything. Intuitionistic logic is minimal logic plus this rule. → Chapter 3; [MN p. 90].

Excluded middle $B \vee \neg B$: valid classically, rejected intuitionistically. It is one of the counterexamples that motivate banning disjunction from programs. → Chapter 2; [MN p. 42].

Existential generalisation From $M(t)$ infer $\exists x\, M(x)$. It explains the multiple answers of `subst` when run backwards. → Chapter 6; [MN p. 203].

Explicit substitution Calculi that make substitution a term constructor, so it can be delayed and composed. Examples are $\lambda\sigma$ (DHKP) and Teyjus's suspension notation. → Chapter 9; [MN p. 209].

F

Flexible term, rigid term A $\beta\eta$-normal term is *flexible* if its head is an existential (unification) variable. It is *rigid* if its head is a constant or a universally bound or λ-bound variable. The names are suggestive: Ma and Friedman, following Huet, explain that a flexible term is one that "can be removed through a substitution (or a grafting)", while a substitution never changes the head of a rigid term [29]. → Chapters 4 and 7; [MN p. 111], [MN p. 214].

Flexible atom, rigid atom, flexible goal An atom is flexible if its head is a variable and rigid if its head is a non-logical constant. Clause heads must be rigid. A flexible goal whose head is still uninstantiated when it is called is underconstrained. → Chapter 5; [MN p. 119], [MN p. 132].

Flex-flex, flex-rigid, rigid-rigid The classification of equations by the heads of their two sides. Rigid-rigid pairs simplify by decomposition or fail by a clash. Flex-rigid pairs call for imitation or projection. Flex-flex pairs always have unifiers, though not necessarily solutions, and are postponed. → Chapter 7; [MN p. 113], [MN p. 217].

Focused proof Andreoli's generalisation of the two-phase structure of goal reduction followed by backchaining. Focusing groups small rules into large ones [1]. → Chapter 2; [MN p. 73].

fohc, fohh, hohc, hohh, hohh$^+$ The book's languages: first-order Horn clauses [MN p. 43], first-order hereditary Harrop formulas [MN p. 76], higher-order Horn clauses [MN p. 122], higher-order hereditary Harrop formulas [MN p. 124], and hohh$^+$, which additionally allows atomic clauses headed by an essentially universal variable [MN p. 125]. → Chapters 2, 3 and 5.

Free for t is free for x in B if substituting t for x in B captures no free variable of t. → Chapter 3; [MN p. 85].

Fresh, `fresh°` miniKanren's existential quantifier, which introduces logic variables. In (aux kanren micro), `fresh°` introduces them one at a time, and each allocation suspends once. For the αKanren meaning, see *Atom (nominal)*. → Chapter 10.

Freshness constraint $a \# t$: the atom a does not occur free in t, also after later instantiation of the variables in t. It is written `hash` in αKanren [58]. → Chapter 9.

G

Generic judgment, universal judgment A generic judgment is parametric in a new object and is proved once, with an eigenvariable. A universal judgment holds for all terms. The two coincide under the open-world assumption. They correspond to the intensional and the extensional readings of ∀. → Chapter 3; [MN p. 83], [MN p. 94].

GENERIC The rule for $\forall x\, G$: add a new constant c to the signature and prove $G[c/x]$. It is `pi`

`x\ G` in λProlog and `all` in λKanren.

$$\frac{\Sigma \cup \{c\}; \mathscr{P} \longrightarrow G[c/x]}{\Sigma; \mathscr{P} \longrightarrow \forall x\, G}\ \textsc{generic} \quad (c \notin \Sigma)$$

→ Chapter 3; [MN p. 39], [MN p. 82].

Goal (miniKanren) In (aux kanren micro), a procedure from a state to a stream of states. `✓°` succeeds once and `✗°` fails. → Chapter 10.

Goal-directed search, goal reduction A non-atomic goal is decomposed by its top-level connective, independently of the program. This fixes the "search semantics" of the connectives. Only atomic goals consult the program. → Chapters 1 and 2; [MN p. 3], [MN p. 39].

Grafting First-order, capture-permitting replacement of a metavariable by a term, written $\{X \mapsto t\}$: grafting $X \mapsto a$ into $\lambda a.X$ gives $\lambda a.a$. DHKP "pre-cook" terms so that grafting respects scopes. Ma and Friedman combine grafting with de Bruijn levels [29]. → Chapter 9.

Grounding In (aux kanren micro), `°->list/ground` evaluates a reified answer, applying its λ to its own variable names. Constraints are dropped. → Chapter 10.

H

Harrop formula $H ::= \top \mid A \mid H \wedge H \mid B \supset H \mid \forall x\, H$. These formulas guarantee the disjunction and existence properties at the root of a proof. → Chapter 3; [MN p. 93].

Herbrand universe H_1^Σ, H_2^Σ The terms allowed as quantifier instances. H_1^Σ (for hohc) contains the λ-normal terms over Σ without $\forall$ and $\supset$. H_2^Σ (for hohh) contains those without $\supset$. → Chapter 5; [MN p. 122].

Hereditary Harrop formula A D-formula of fohh or hohh. Equivalently, a formula with no positive occurrence of a disjunction or an existential. Unlike Horn clauses, these formulas let the signature and the program grow during search. → Chapter 3; [MN p. 4], [MN p. 76].

Higher-order abstract syntax (HOAS) The principle of encoding object-level binders by meta-level binders [49]. It is ambiguous between a functional reading, in which binders are host functions that cannot be analysed, and the logical reading, λ-tree syntax. → Chapter 6; [MN p. 204].

Higher-order logic The book distinguishes three senses: second-order model theory; proof theory with predicate quantification and comprehension; and Church's simple theory of types, with λ-terms and quantification at higher types. The book works in the third. → Chapter 1; [MN p. 5].

Higher-order pattern A term in which every occurrence of an existential variable F is applied to distinct variables, either essentially universal or λ-bound, whose binders lie within the scope of F. For example, $\lambda x.\, \lambda y.\, F\, y\, x$ is a pattern, while $\lambda x.\, F\, x\, x$ (repeated argument) and $F\, c$ with c a signature constant (not a bound variable) are not. → Chapters 6 and 7; [MN p. 205], [MN p. 220].

Higher-order pattern unification (L_λ-unification) Unification restricted to patterns. It is decidable, and unifiable problems have an mgu [34, 48]. Its steps include an occurs check and variable elimination with pruning. → Chapter 7; [MN p. 220]–[MN p. 223].

Higher-order programming Passing predicates as arguments, as in `mappred` and `foldl`. → Chapter 5; [MN p. 126].

Higher-order unification Unification of simply typed λ-terms modulo λ-conversion. It is undecidable (the book reduces Post's correspondence problem and Hilbert's tenth problem to it), has no mgus in general, and is semi-decided by Huet's procedure [25]. → Chapters 4 and 7; [MN p. 111], [MN p. 114]–[MN p. 115], [MN p. 214].

Horn clauses The class of programs in which the signature and the program stay fixed during search: they are "flat and global". → Chapter 2; [MN p. 43], [MN p. 46].

Huet's procedure, pre-unification Repeated simplification, imitation and projection, stopping when only flex-flex pairs remain [25]. The result is a pre-unifier, whose search space is explored as a matching tree. → Chapter 7; [MN p. 218]–[MN p. 219].

Hypothetical reasoning Proving a goal under temporarily assumed clauses, by AUGMENT. The book's example is the question "If person X took just one more course Y, will he or she graduate with a CS degree?", posed as the query `fact (finished X Y) => (fact (graduates X), fact (cs_major X))`. → Chapter 3; [MN p. 79]–[MN p. 80].

I

`if`, `not` λProlog's conditional and negation-as-failure, both defined with the cut. → Chapter 5; [MN p. 137].

`if°` (soft cut) `(if° g? gt gf)`: if `g?` has at least one answer, every one of its answers continues with `gt`; otherwise `gf` runs. It corresponds to miniKanren's `conda` and LogicT's `ifte`. λProlog's `if` becomes `(if° (take° 1 P) Q R)`. → Chapter 10.

Imitation, projection Huet's partial bindings for a flex-rigid pair. *Imitation* copies the rigid head: $F := \lambda\bar{x}.\, c\,(H_1\bar{x})\cdots(H_m\bar{x})$. *Projection* selects an argument: $F := \lambda\bar{x}.\, x_i\,(H_1\bar{x})\cdots(H_k\bar{x})$, which requires matching target types. → Chapter 7; [MN p. 215].

Intensional / extensional equality Intensional equality compares the structure of λ-terms. Extensional equality compares what they denote. λProlog's `=` is intensional (modulo λ-conversion). → Chapter 5; [MN p. 146].

Intensional / extensional `∀` `pi` is intensional: it requires one generic proof, so it cannot express "for all instances". This is why book Ch. 11's simulation is incorrect. Jin et al. implement an extensional `∀` in miniKanren, which is a different logic [26]. → Chapters 3 and 8; [MN p. 83], [MN p. 94].

Interleaving miniKanren's search discipline. In (aux kanren micro), `orª` uses `appendº/interleaved/2`, which swaps the two streams whenever the first one is a suspension, and conjunction (`append-mapº`) interleaves too. The right-nested `or°` gives the first of three

infinite branches about half of the answers. → Chapter 10.

Inverse-η delay The wrapping `(λ (s) (δ ((and° g …) s)))` that `define-relation` puts around a body, so that a recursive relation without `fresh°` does not loop while the goal tree is built. → Chapter 10.

Inversion of a pattern substitution The core step of pattern unification. To solve $X\,\bar{y} = t$, where $\bar{y}$ are distinct bound variables, abstract t over $\bar{y}$, that is, $X := \lambda\bar{y}.\,t$. This works only if every bound variable free in t is among $\bar{y}$ and X does not occur in t; otherwise pruning or the occurs check takes over. DHKP write the inverse as $b[\xi]^{-1}$, and Nipkow's code remaps de Bruijn indices. → Chapter 7.

Invertible rule A rule whose premises are provable if and only if its conclusion is. Such rules can be applied eagerly without losing completeness. → Chapter 8; [MN p. 243].

Iterative deepening Retrying a search with bound $N+1$ after failing with bound N. In book Ch. 9 it bounds the multiplicity of ∃-formulas. → Chapter 8; [MN p. 237], [MN p. 239].

K

Kind The "type" of a type constructor, of the form `type -> ... -> type`, introduced by a `kind` declaration. → Chapter 1; [MN p. 11]–[MN p. 12].

Kripke model The possible-worlds semantics of intuitionistic logic. Its worlds correspond to signature–program pairs. → Chapter 3; [MN p. 94].

L

L_λ The sublanguage of hohh in which quantification is never over predicate types, and every essentially existential variable is applied only to distinct essentially universal variables bound within its scope. Unification inside L_λ needs only β_0, is decidable and produces mgus. → Chapters 6 and 7; [MN p. 205]–[MN p. 206], [MN p. 220].

λ-conversion, λ-normal form λ-conversion is the closure of α, β and η, and it is the equality of the logic. The λ-normal form $\lambda norm(s)$ is the $\beta\eta$-normal form. → Chapter 4; [MN p. 100]–[MN p. 102].

λ-tree syntax HOAS with simply typed λ-terms, taken modulo α, a weak β (β_0) and possibly η, used as a data structure that unification can still analyse. → Chapter 6; [MN p. 204].

λKanren miniKanren extended with higher-order logic programming. The 2020 version has terms `tie` and `app`, goals `assume` ($D \supset G$) and `all` ($\forall x\, G$), and Miller's pattern unification over an explicit prefix [30]. The 2021 paper proposes static closures [29]. → Chapter 9.

Left- and right-introduction rules Right rules introduce a connective on the right of $\longrightarrow$ and implement goal reduction. Left rules decompose the clause in focus and implement backchaining. → Chapter 2; [MN p. 40], [MN p. 42], [MN p. 45].

Legitimate substitution For a problem $\forall\bar{u}\,\exists\bar{x}\,\forall\bar{w}\,[\ldots]$, a substitution for $\bar{x}$ in whose range

no w_j occurs free. Only legitimate substitutions can be unifiers. This is the eigenvariable condition, stated for unification. → Chapter 4; [MN p. 109].

Leibniz equality $x = y$ if and only if $\forall P\,(P\,x \supset P\,y)$. → Chapter 5; [MN p. 120].

Level of a unification variable The scope depth of a variable. ELPI's X^j may mention only context names below j. In Teyjus, a variable may be bound only to terms whose constants have a tag no larger than its own. Binding a variable must lower the levels of the variables in the bound term. → Chapters 7 and 9.

Locally nameless A representation with bound variables as de Bruijn indices and free variables as names [31, 6]. Eigen constants would play the role of the names. → Chapter 9.

Logic variable A placeholder that search instantiates by unification. It is the operational device behind the delayed choice of terms in the ∃R, ∀L and INITIAL rules. → Chapter 2; [MN p. 61].

M

Matching A unification problem in which only one side contains existential variables. Higher-order matching is decidable. → Chapter 7; [MN p. 227].

Matching tree The search tree of Huet's procedure. Its nodes are multisets of equations and its arcs are substitutions. A failure node is a problem that obviously has no solution (for example, a rigid-rigid clash); a success node contains at most flex-flex pairs, and reaching one is a pre-unification. Some paths may be infinite. → Chapter 7; [MN p. 219].

Matrix The conjunction of equations $t_1 = s_1 \wedge \cdots \wedge t_m = s_m$ under the quantifier prefix of a unification problem. → Chapter 7; [MN p. 211].

May judgment, must judgment A may judgment asserts that some computation path exists. A must judgment states a property of all paths, and needs negation-as-failure or a meta-level ∀. → Chapter 8; [MN p. 269].

Metalanguage, object language λProlog, the language in which we write, and the language encoded in it. → Chapter 1; [MN p. 22].

Minimal logic Logic in which $\bot$ is an ordinary constant. Negation is defined as $\neg B := B \supset$ false. → Chapter 3; [MN p. 89]–[MN p. 90].

miniKanren A family of small relational languages embedded in a host language [15, 4]. → Chapter 10.

Mixed prefix The quantifier prefix ∀∃∀ … of a unification problem. In a ∀∃∀ problem the outer universals act as the signature and the inner ones as eigenvariables, and no raising is needed. → Chapters 4 and 7; [MN p. 105], [MN p. 211].

Module, signature (Teyjus) A module is a named collection of declarations and clauses. Its signature restricts what is visible from outside, and hidden constants are read as existentially quantified (book Ch. 6). → Chapter 1; [MN p. 152]–[MN p. 153], [MN p. 277].

Most general unifier (mgu) A unifier θ such that every other unifier is θ followed by a further substitution. In the first-order case an mgu exists whenever a unifier does, and it is unique up to renaming. Pattern problems have mgus too, but general higher-order problems do not. → Chapters 1 and 7; [MN p. 28], [MN p. 213], [MN p. 222].

μKanren Hemann and Friedman's minimal miniKanren, in which goals are functions from a state to a stream of states [22]. (aux kanren micro) is a μKanren with constraint stores. → Chapter 10.

Multiplicity In the classical prover of book Ch. 9, the number of times an ∃-formula may be instantiated. Bounding it, and raising the bound after each failure, gives a complete semi-decision procedure. → Chapter 8; [MN p. 237].

N

∇ (nabla) The Miller–Tiu generic quantifier for fresh names, in a logic with definitions (closed world). It is not part of λProlog. → Chapters 3 and 8; [MN p. 94], [MN p. 275].

Name shadowing An inner binder that reuses an outer binder's name, so that the outer binder can no longer be named. → Chapter 9.

Natural deduction Introduction and elimination rules, with hypotheses discharged by ⊃I, ∨E and ∃E. Book Ch. 9 encodes it with `=>` and `pi`. → Chapter 8; [MN p. 231].

Negation-as-failure `not G` succeeds if and only if `G` finitely fails. It is non-logical and unsound on non-ground goals. In (aux kanren micro) it is written `(if° (take° 1 g) ✗° ✓°)`. Without the `take° 1`, it diverges whenever the search for further answers of `g` does not terminate, even if `g` has a single answer (Appendix B.3 runs an example). → Chapters 5 and 8; [MN p. 137], [MN p. 269].

Negation normal form, prenex normal form NNF has no implications and applies negation only to atoms. A prenex form has all its quantifiers in front. → Chapter 6; [MN p. 195].

Nominal logic, nominal unification A first-order treatment of binding with names, swappings and freshness. Nominal unification works modulo α and never generates new names [58]. → Chapter 9; [MN p. 210].

O

o The type of propositions, named after Church's omicron. The quantifier names `pi` and `sigma` likewise follow Church's Π and Σ. → Chapter 1; [MN p. 74].

Object-level substitution Substituting into the body of an encoded binder. In λ-tree syntax it is meta-level β-reduction, as in the clause `subst R N (R N).` with `R : tm -> tm`. That clause lies outside L_λ, but it is used only with `R` and `N` instantiated (dynamic L_λ). Copy clauses specify the same relation inside L_λ. → Chapter 6; [MN p. 180], [MN p. 199], [MN p. 225].

O-proof A derivation that uses only goal reduction (right rules) followed by backchaining (focused left rules). Every provable fohc sequent has an O-proof. → Chapter 2; [MN p. 45].

Occurs check $x = t$ fails when x occurs properly in t. In pattern unification, the corresponding failure is a flexible head occurring rigidly on the other side. (aux kanren micro) performs the check when it extends the substitution. → Chapters 1 and 7; [MN p. 30], [MN p. 223].

Order of a type 0 for non-functional types, and $\max(\operatorname{ord}\tau_1 + 1, \operatorname{ord}\tau_2)$ for $\tau_1 \to \tau_2$. → Chapters 1 and 4; [MN p. 13].

P

Pattern See *Higher-order pattern*. Ma and Friedman's phrasing, for an application form $F\,a$: "if F is a unification variable, then a must be a list of zero or more distinct bound variables".

Peirce's formula $((p \supset q) \supset p) \supset p$: valid classically but not intuitionistically. It has no O-proof. → Chapter 3; [MN p. 89].

Pervasive Built in and globally available: sorts such as `int`, constants such as `nil` and `::`, and predicates such as `is`. → Chapter 1; [MN p. 14], [MN p. 49].

`pi`, `sigma` λProlog's quantifiers, constants of type `(A -> o) -> o`. $\forall x\, G$ is written `pi x\ G`, the application of `pi` to the abstraction `x\ G`, so the term-level λ is the only binder. → Chapters 1 and 4; [MN p. 36], [MN p. 99].

π-calculus Milner's calculus of mobile processes, in which names are the only data. Book Ch. 11 encodes it in λ-tree syntax, using `in` and `nu` as binders. → Chapter 8; [MN p. 261].

Polarity, positive and negative occurrence An occurrence is positive if it lies to the left of an even number of implications and negative if odd. Cut-free search preserves polarity, unless implications appear inside atoms. → Chapters 3 and 5; [MN p. 76], [MN p. 121]–[MN p. 122].

Pre-cooking (DHKP) Translating a metavariable that occurs under n binders into $X[\uparrow^n]$, so that first-order grafting agrees with capture-avoiding substitution [8]. → Chapter 9.

Pre-unifier A substitution that reduces a problem to flex-flex pairs only. Such a remainder always has a unifier, the canonical substitution. → Chapter 7; [MN p. 217]–[MN p. 218].

`project°` (aux kanren micro)'s escape to Scheme: it binds Scheme variables to deep-walked values, in which unbound variables remain variable records, and then runs goals. It is non-relational. → Chapter 10.

Proof term A λ-term of type `proof` that records a derivation, as in the judgment `P # A` ("P is a proof of A"). → Chapter 8; [MN p. 233].

Pruning Removing from a variable its dependency on arguments that cannot occur in any solution: $G := \lambda\bar{d}.\,(G'\,\bar{e})$, where $\bar{e}$ keeps only the allowed arguments. → Chapter 7; [MN p. 223].

Purification The step of (aux kanren micro)'s reification that drops constraints mentioning variables unreachable from the answer. → Chapter 10.

R

Raising $\forall y\, \exists x\, G \rightsquigarrow \exists h\, \forall y\, G[(h\, y)/x]$: an existential in the scope of a universal becomes the application of a new, higher-type existential quantified further out. Raising preserves L_λ. The book calls it a dual of Skolemization: Skolemization introduces a new *constant* of a raised type, raising a new *existential variable* of a raised type. → Chapters 4 and 7; [MN p. 108], [MN p. 208], [MN p. 212].

Reduction rules AND, OR, INSTAN, AUGMENT, GENERIC and TRUE: the search meaning of each logical symbol. → Chapter 2; [MN p. 39].

Reduction-free fragment L_λ^β (ELPI) The patterns in which a unification variable X^j is applied exactly to the bound variables $x_j \ldots x_{j+k-1}$ that follow its level. Binding such a variable needs no β-reduction and takes constant time, but the fragment is not stable under resolution [9]. → Chapter 9.

Reification Turning the final state into a presentable answer. (aux kanren micro) produces a closed Scheme expression `(λ (α β …) constraint-form … term)`, naming free variables in order of first occurrence. αKanren prints noms as `a.0`, `a.1`, … and adds freshness constraints. → Chapter 10.

Relation ((aux kanren micro)) A procedure from terms to a goal, conventionally named `name°` and defined with `define-relation`. → Chapter 10.

Relative complement (Jin et al.) A goal that describes the values of a variable not covered by a state. It is used to implement an extensional `∀` [26]. → Chapter 9.

Representation independence Replacing a module by an equivalent one does not change the meaning of the larger program. → [MN p. 151].

Rigid See *Flexible term, rigid term* and *Flexible atom.*

S

Scope check, eigenvariable condition No variable older than an eigenvariable may reach it. The implementations differ: an explicit quantifier prefix (Miller; λKanren 2020), universe indices or levels (Teyjus, ELPI), an absento-like constraint over the variables in scope (`eigen`), or an end-of-scope check, which is fragile. → Chapters 9 and 10.

Scope extrusion The book's name for the classical equivalence $B_1 \vee (B_2 \supset B_3) \equiv B_2 \supset (B_1 \vee B_3)$, which moves the scope of an implication over a disjunction. It fails intuitionistically: $p \vee (p \supset q)$ has no O-proof, although its classical equivalent $p \supset (p \vee q)$ does. The term is borrowed from the π-calculus, where it means sending a restricted name out and re-closing its scope at the receiver. → Chapters 3 and 8; [MN p. 89], [MN p. 95].

Scope restriction Ma and Friedman's name for the eigenvariable condition in unification: a `∀`-bound variable may occur in the instantiation of an `∃`-variable only if it precedes that variable in the prefix. → Chapter 9.

Shallow embedding Implementing a logic language as host-language procedures and macros, as miniKanren does in Scheme, instead of writing an interpreter or compiler for it. λKanren keeps this style [30]. → Chapters 9 and 10.

Sequent $\Sigma; \mathscr{P} \longrightarrow G$: signature, program and goal. It is read both as the state of an interpreter and as a logical judgment. → Chapter 2; [MN p. 39].

Signature A set of typed non-logical constants. In Teyjus, a signature is also a module's interface (`sig`). → Chapter 1; [MN p. 17], [MN p. 153].

Signature–program pair $\langle \Sigma, \mathscr{P} \rangle$, the context in which queries are posed. Under the open-world assumption these pairs are the worlds of a Kripke model. → Chapters 2 and 3; [MN p. 39], [MN p. 94].

Simplification Repeated decomposition, or clash, of rigid-rigid pairs in higher-order unification. → Chapter 7; [MN p. 214].

Skolemization Replacing an existential in an assumption by a function of the enclosing universals, as in $\forall x\, D[f\ x/y]$. It is the dual of raising. → Chapter 4; [MN p. 108].

Soft cut See `if°`. A soft cut is not complete in general.

Solution A unifier whose range has free variables only among the outer universals. A solution amounts to a proof of the quantified equation. → Chapters 4 and 7; [MN p. 109].

Solved form A set of equations $x = t$ in which each left-hand variable occurs nowhere else. It is read as an idempotent mgu. → Chapter 1; [MN p. 29].

Static closure $\langle \phi; t \rangle$ (Ma–Friedman): a term paired with the scope, the list of binder names around it, manipulated by ext and db. The authors call the paper "a preview" and leave soundness and completeness to future work [29]. → Chapter 9.

Stream (§) In aux, a stream is `'()`, a mature pair `(state . §)`, or an immature promise. → Chapter 10.

Subject reduction Evaluation preserves types. → Chapter 6; [MN p. 191].

Substitution ((aux kanren micro)) The store `S`: a triangular substitution kept in a skew-binary random-access list indexed by variable number. → Chapter 10.

Subsumption ((aux kanren micro)) Dropping a constraint that another constraint implies, or that a type tag or an absento constraint makes vacuous. → Chapter 10.

Suspension Four different things, depending on the source; see the pitfall at the start of this appendix.

Swapping $(a\ b)$ exchanges the atoms a and b throughout a term. Permutations are lists of swappings. → Chapter 9.

Syntactic constraint (ELPI) A goal suspended on some variables with `declare_constraint` and resumed when one of them is assigned. It keeps its hypothetical context. → Chapter 9.

T

`take°` `(take° n g)` keeps the first `n` answers of `g`; a negative `n` keeps them all. It is a goal, so it nests, and `(take° 1 g)` is miniKanren's `once`. `µkanren-run` is `take°` around a `fresh°` query. → Chapter 10.

Tactic, tactical A tactic is a relation `goal -> goal -> o` from a primitive goal to the goal of its premises. Tacticals such as `then`, `orelse`, `repeat`, `try` and `maptac` combine tactics. Book Ch. 9 recasts the LCF architecture relationally. → Chapter 8; [MN p. 241], [MN p. 243].

Target type τ_0 in $\tau_1 \to \ldots \to \tau_n \to \tau_0$. → Chapter 1; [MN p. 13].

Term reduction, variable elimination Two of the three transformations of first-order unification; the third is reorientation. Term reduction decomposes equal heads. Variable elimination replaces a variable by its value everywhere. → Chapter 1; [MN p. 28].

Teyjus The λProlog implementation used by the book, with compiler `tjcc`, linker `tjlink` and simulator `tjsim`. Teyjus 2 uses universe-level tags. → Chapter 1; [MN p. 277].

Trace, complete trace, trace equivalence A trace is a sequence of actions of a process, and a complete trace ends in a terminal process. Two processes are trace equivalent if no trace separates them. → Chapter 8; [MN p. 267], [MN p. 269], [MN p. 271].

Transparent type variable A type variable that occurs in the target type. → Chapter 2; [MN p. 67].

Type lowering A polarity-indexed translation from a relation at a higher type to formulas over base-type relations. It generates copy clauses. → Chapter 6; [MN p. 201]–[MN p. 202].

U

Unification problem, unifier In the first-order case, a problem is a finite multiset of equations, and a unifier is a type-preserving substitution that makes both sides of each equation equal. In general, a problem is a quantified formula $Q_1 x_1 \ldots Q_n x_n[\bigwedge_j t_j = s_j]$, and a unifier is a legitimate substitution (no inner universals in its range) that makes each pair λ-convertible. → Chapters 1 and 4; [MN p. 26], [MN p. 105], [MN p. 109].

Unification ((aux kanren micro)), `=°` First-order syntactic unification with occurs check. Pairs, vectors and records are decomposed slot by slot, so a vector and a record with the same slots unify. When two variables meet, the one with the lower index is bound to the other. → Chapter 10.

Uniform proof A cut-free sequent proof in which every sequent with a non-atomic goal is the conclusion of the right-introduction rule for that goal's top-level connective. It formalises goal-directed search [37]. → Chapter 2; [MN p. 73].

Universe index (Nadathur, Jayaraman, Kwon) A numeric tag on constants and logic variables that records the ∀-nesting at their creation. Entering a universal goal increments the index. A variable may be bound only to terms whose constants have a tag no larger than its own,

and the binding lowers the tags of the variables in the term [43]. The book mentions the idea as numerical indices on quantified variables [MN p. 225] and as universe level tags [MN p. 227]. → Chapters 7 and 9.

Universe level See *Universe index* and *Level of a unification variable*.

V

Vacuous quantifier A quantifier whose variable does not occur in its body. → Chapter 6; [MN p. 195].

Value constructor A constant of functional type used to build data, such as `::`. → Chapter 1; [MN p. 14].

Variable elimination (patterns) The step of pattern unification that combines imitation, projection, the occurs check and pruning into one binding $F := \lambda\bar{c}.\,t'$. → Chapter 7; [MN p. 223].

W

Walk Resolving a variable through the substitution. `μkanren-state-find` is the shallow walk: it follows the chain of variable bindings until it reaches a non-variable or an unbound variable. `μkanren-state-find/value` is the deep walk: it also walks inside pairs, vectors and records. → Chapter 10.

WAM The Warren Abstract Machine, the classical implementation model of Prolog. → [MN p. 74].

Working variable, reified variable In (aux kanren micro), a working variable has index ≥ 0, lives during search and prints as `_i`. A reified variable has a negative index, appears only in answers and prints as α, β, … → Chapter 10.

B.3 A vocabulary bridge

Table B.2 lines up the names that the three communities use for the same concept. Read it across a row: a cell left empty means the concept has no counterpart in that column today.

Kanren angle: what the glossary says about the missing pieces

The empty cells of Table B.2 are the λProlog features that (aux kanren micro) lacks: a dynamic clause database for AUGMENT, eigenvariables with a scope check for GENERIC, and terms with binders together with a unifier that respects them. The entries *Universe index*, *Level of a unification variable*, *Scope check*, *Higher-order pattern unification* and *Delayed equations* list the known ways to provide them. Chapter 11 chooses among them and Chapter 12 implements the choice.

Table B.2: The same concepts under three names, and in the proposed module. The empty cells of the third column are what `(aux kanren lambda)` supplies (Chapter 12).

Book (λProlog)	miniKanren literature	(aux kanren micro) today	`(aux kanren lambda)`
logic variable, ∃ in goals (`sigma`)	`fresh`	`fresh°`	`fresh°`, with a level
backchaining on a clause	calling a relation	a `define-relation` call	also a `define-dynamic-relation` call
∨ in goals (`;`)	`conde`	`or°`, `cond°`	unchanged
first-order unification with occurs check	`==`	`=°`	`=°`, through the `unify` and `post` hooks
`not`, `if` (via `!`)	`conda`, `condu`	`if°`, `take°`	unchanged
AUGMENT, `D => G`	`assume` (λKanren)	—	`⊃°`, `define-dynamic-relation`
GENERIC, `pi x\ G`, eigenvariable	`all` (λKanren), `eigen`	—	`∀°`, with universe levels
λ-terms, `x\ t`	`tie/app` (λKanren), `tie` on noms (αKanren)	first-order encodings only (e.g. records with de Bruijn indices)	`ẋ`, `@`
pattern unification, raising, pruning	λKanren 2020 unifier; static closures (2021)	—	the `unify` and `post` hooks with levels; non-patterns delayed in E
answer substitution	reified answer	`μkanren-project`, `°->list`	`°->list/datum`, `°->pretty`

Kanren angle: three entries checked on today's engine

The following file exercises the entries *Interleaving*, *Negation-as-failure* and *Reification*. The relation `num°` enumerates $0, 1, 2, \ldots$.

```
(import (aux kanren micro) (aux base) (aux stream))

; num° enumerates 0, 1, 2, ... (project° escapes to Scheme to add 1)
(define-relation (num° n)
  (or° (=° n 0) (fresh° (m) (num° m) (project° ((m* m)) (=° n (+ 1 m*))))))

; Interleaving: three infinite branches; the right-nested or° favours the first
(print (°->list/ground
         (take° 12 (fresh° (q n)
                     (or° (and° (num° n) (=° q (list 'a n)))
                          (and° (num° n) (=° q (list 'b n)))
                          (and° (num° n) (=° q (list 'c n))))))))

; Negation as failure: (take° 1 g) makes the test finite
(print (°->list/ground (fresh° (q) (=° q 1) (if° (take° 1 (num° q)) ✗° ✓°))))
```

```
(print (°->list/ground (fresh° (q) (=° q 1) (if° (=° q 2) ✗° ✓°))))
(print (°->list/ground (fresh° (q) (if° (=° q 2) ✗° ✓°))))

; Reification: free variables become α, β, ...; constraints first
(print (°->list #f (fresh° (q x) (=° q (list x 'a)) (≠° x 'b))))
```

```
((a 0) (b 0) (a 1) (c 0) (a 2) (b 1) (c 1) (a 3) (a 4) (b 2) (c 2) (a 5))
()
(1)
()
((λ (α) (begin (deny (equal? α (quote b)))) (cons α (cons (quote a) (quote ())))))
```

Read the lines in order. (1) Of the first twelve answers, six come from branch `a` and three each from `b` and `c`: the shares are $\frac{1}{2}, \frac{1}{4}, \frac{1}{4}$, as the entry *Interleaving* says. (2) $q = 1$ is a number, so `not (num° 1)` fails. If you drop the `take° 1`, the run never returns: `num° 1` has only one answer, but the search for a second one never ends. `if°` continues every answer of the test with `✗°`, and over a stream that never ends this continuation never finishes either. (3) and (4) show negation-as-failure on a ground goal and on a non-ground one. With q unbound, the test `(=° q 2)` succeeds by binding q, so the negation fails, although $q = 1$ would satisfy it. This is the classic unsoundness. (5) The reified answer is a closed λ-expression whose body lists the constraint before the term.

B.4 Summary

- Proof-theoretic terms (sequent, uniform proof, AUGMENT, GENERIC, eigenvariable) describe *what* search must do. Unification terms (pattern, raising, pruning, level) describe *how* binders and quantifiers are respected. miniKanren terms (goal, stream, interleaving, reification) describe the engine the ideas must fit.
- Watch out for overloaded words: suspension, level, scope extrusion, fresh, cut, substitution.
- Every entry points to the chapter that develops it and to the book page where it is defined. Use the pointers to return to the full treatment.

Bibliography

[1] Jean-Marc Andreoli. Logic programming with focusing proofs in linear logic. *Journal of Logic and Computation* 2(3):297–347, 1992.

[2] Pascal Brisset and Olivier Ridoux. Naïve reverse can be linear. In *Eighth International Logic Programming Conference*, Paris, France, June 1991. MIT Press, Cambridge, MA, 1992.

[3] N. G. de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church–Rosser theorem. *Indagationes Mathematicae* 34(5):381–392, 1972.

[4] William E. Byrd. *Relational Programming in miniKanren: Techniques, Applications, and Implementations*. PhD thesis, Indiana University, 2009.

[5] William E. Byrd and Daniel P. Friedman. αKanren: A fresh name in nominal logic programming. In *Proceedings of the 2007 Workshop on Scheme and Functional Programming*, Université Laval Technical Report DIUL-RT-0701, pp. 79–90, 2007.

[6] Arthur Charguéraud. The locally nameless representation. *Journal of Automated Reasoning* 49(3):363–408, 2012.

[7] James Cheney. Relating nominal and higher-order pattern unification. In *Proceedings of UNIF 2005*, pp. 104–119, 2005.

[8] Gilles Dowek, Thérèse Hardin, Claude Kirchner, and Frank Pfenning. Unification via explicit substitutions: the case of higher-order patterns. In M. Maher, editor, *Joint International Conference and Symposium on Logic Programming (JICSLP)*, pp. 259–273. MIT Press, 1996.

[9] Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λProlog interpreter. In *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR-20)*, LNCS 9450, pp. 460–468. Springer, 2015.

[10] Roy Dyckhoff. Contraction-free sequent calculi for intuitionistic logic. *Journal of Symbolic Logic* 57(3):795–807, September 1992.

[11] Matthias Felleisen and Robert Hieb. The revised report on the syntactic theories of sequential control and state. *Theoretical Computer Science* 103:235–271, 1992.

[12] Amy Felty. Implementing tactics and tacticals in a higher-order logic programming language. *Journal of Automated Reasoning* 11(1):43–81, August 1993.

[13] Amy Felty and Dale Miller. Specifying theorem provers in a higher-order logic programming language. In *Ninth International Conference on Automated Deduction*, LNCS 310, pp. 61–80. Springer-Verlag, New York, May 1988.

[14] Michael J. Fischer. Lambda calculus schemata. *ACM SIGPLAN Notices* 7(1):104–109, January 1972.

[15] Daniel P. Friedman, William E. Byrd, Oleg Kiselyov, and Jason Hemann. *The Reasoned Schemer*, second edition. MIT Press, 2018.

[16] Gerhard Gentzen. Investigations into logical deductions. In M. E. Szabo, editor, *The Collected Papers of Gerhard Gentzen*, pp. 68–131. North-Holland, Amsterdam, 1969. Translation of articles that appeared in 1934–35.

[17] Warren Goldfarb. The undecidability of the second-order unification problem. *Theoretical Computer Science* 13:225–230, 1981.

[18] Michael J. Gordon, Arthur J. Milner, and Christopher P. Wadsworth. *Edinburgh LCF: A Mechanised Logic of Computation*, LNCS 78. Springer-Verlag, New York, 1979.

[19] John Hannan and Dale Miller. From operational semantics to abstract machines. *Mathematical Structures in Computer Science* 2(4):415–459, 1992.

[20] R. Harrop. Concerning formulas of the types $A \to B \vee C$, $A \to (Ex)B(x)$ in intuitionistic formal systems. *Journal of Symbolic Logic* 25:27–32, 1960.

[21] Jason Hemann, William E. Byrd, and Daniel P. Friedman. The `eigen` operator of *miniKanren-with-symbolic-constraints*. Software, 2013. `https://github.com/webyrd/miniKanren-with-symbolic-constraints`.

[22] Jason Hemann and Daniel P. Friedman. μKanren: A minimal functional core for relational programming. In *Proceedings of the 2013 Workshop on Scheme and Functional Programming (Scheme '13)*, 2013.

[23] Jörg Hudelmaier. Bounds on cut-elimination in intuitionistic propositional logic. *Archive for Mathematical Logic* 31:331–353, 1992.

[24] Gérard Huet. The undecidability of unification in third order logic. *Information and Control* 22:257–267, 1973.

[25] Gérard Huet. A unification algorithm for typed λ-calculus. *Theoretical Computer Science* 1:27–57, 1975.

[26] Ende Jin, Gregory Rosenblatt, Matthew Might, and Lisa Zhang. Universal quantification and implication in miniKanren. In *miniKanren and Relational Programming Workshop*, 2021.

[27] Gilles Kahn. Natural semantics. In *Proceedings of the Symposium on Theoretical Aspects of Computer Science*, LNCS 247, pp. 22–39. Springer, New York, March 1987.

[28] Jordi Levy and Mateu Villaret. Nominal unification from a higher-order perspective. In *Rewriting Techniques and Applications (RTA 2008)*, LNCS 5117, pp. 246–260. Springer, 2008.

[29] Weixi Ma and Daniel P. Friedman. A new higher-order unification algorithm for λKanren. In *miniKanren and Relational Programming Workshop*, 2021.

[30] Weixi Ma, Kuang-Chen Lu, and Daniel P. Friedman. Higher-order logic programming with λKanren. In *miniKanren and Relational Programming Workshop*, 2020.

[31] Conor McBride and James McKinna. Functional pearl: I am not a number — I am a free variable. In *Proceedings of the ACM SIGPLAN Workshop on Haskell*, 2004.

[32] Raymond McDowell and Dale Miller. Cut-elimination for a logic with definitions and induction. *Theoretical Computer Science* 232:91–119, 2000.

[33] Spiro Michaylov and Frank Pfenning. An empirical study of the runtime behavior of higher-order logic programs. In *Proceedings of the Workshop on the λProlog Programming Language*, 1992.

[34] Dale Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. *Journal of Logic and Computation* 1(4):497–536, 1991.

[35] Dale Miller. Unification under a mixed prefix. *Journal of Symbolic Computation* 14(4):321–358, 1992.

[36] Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge University Press, Cambridge, 2012.

[37] Dale Miller, Gopalan Nadathur, Frank Pfenning, and Andre Scedrov. Uniform proofs as a foundation for logic programming. *Annals of Pure and Applied Logic* 51:125–157, 1991.

[38] Dale Miller and Alwen Tiu. A proof theory for generic judgments. *ACM Transactions on Computational Logic* 6(4):749–783, October 2005.

[39] Robin Milner. Functions as processes. In *Automata, Languages and Programming, 17th International Colloquium*, LNCS 443, pp. 167–180. Springer-Verlag, New York, July 1990.

[40] Robin Milner, Joachim Parrow, and David Walker. A calculus of mobile processes, part I. *Information and Computation* 100(1):1–40, September 1992.

[41] Evgenii Moiseenko. Constructive negation for miniKanren. In *miniKanren and Relational Programming Workshop*, 2019.

[42] Gopalan Nadathur. A treatment of higher-order features in logic programming. *Theory and Practice of Logic Programming* 5(3):305–354, 2005. arXiv:cs/0404020.

[43] Gopalan Nadathur, Bharat Jayaraman, and Keehang Kwon. Scoping constructs in logic programming: implementation problems and their solution. *Journal of Logic Programming* 25(2):119–161, November 1995.

[44] Gopalan Nadathur and Natalie Linnell. Practical higher-order pattern unification with on-the-fly raising. In *ICLP 2005: 21st International Logic Programming Conference*, LNCS 3668, pp. 371–386, Sitges, Spain, October 2005. Springer, 2006.

[45] Gopalan Nadathur and Dale Miller. Higher-order Horn clauses. *Journal of the ACM* 37(4):777–814, October 1990.

[46] Gopalan Nadathur and Dustin J. Mitchell. System description: Teyjus — a compiler and abstract machine based implementation of λProlog. In *Automated Deduction (CADE-16)*, LNAI 1632, pp. 287–291. Springer, 1999.

[47] Joseph P. Near, William E. Byrd, and Daniel P. Friedman. αleanTAP: A declarative theorem prover for first-order classical logic. In *Logic Programming (ICLP 2008)*, LNCS 5366, pp. 238–252. Springer, 2008.

[48] Tobias Nipkow. Functional unification of higher-order patterns. In M. Vardi, editor, *8th Symposium on Logic in Computer Science*, pp. 64–74. IEEE, New York, June 1993.

[49] Frank Pfenning and Conal Elliott. Higher-order abstract syntax. In *Proceedings of the ACM-SIGPLAN Conference on Programming Language Design and Implementation*, pp. 199–208. ACM Press, New York, June 1988.

[50] Zhenyu Qian. Unification of higher-order patterns in linear time and space. *Journal of Logic and Computation* 6(3):315–341, 1996.

[51] Gregory Rosenblatt, Lisa Zhang, William E. Byrd, and Matthew Might. First-order miniKanren representation: great for tooling and search. In *miniKanren and Relational Programming Workshop*, 2019.

[52] Dmitry Rozplokhas, Andrey Vyatkin, and Dmitry Boulytchev. Certified semantics for relational programming. In *Programming Languages and Systems (APLAS 2020)*, LNCS 12470, pp. 167–185. Springer, 2020.

[53] Davide Sangiorgi. π-calculus, internal mobility and agent-passing calculi. *Theoretical Computer Science* 167(2):235–274, 1996.

[54] Raymond M. Smullyan. *First-Order Logic*. Springer-Verlag, New York, 1968.

[55] Antonis Stampoulis and Adam Chlipala. Prototyping a functional language using higher-order logic programming: a functional pearl on learning the ways of λProlog/-Makam. *Proceedings of the ACM on Programming Languages* 2(ICFP), Article 93, 2018. doi:10.1145/3236788.

[56] Richard Statman. The typed λ-calculus is not elementary recursive. *Theoretical Computer Science* 9:73–81, 1979.

[57] Christian Urban. Nominal unification revisited. In *Proceedings of UNIF 2010*, EPTCS 42, pp. 1–11, 2010. doi:10.4204/EPTCS.42.1.

[58] Christian Urban, Andrew M. Pitts, and Murdoch J. Gabbay. Nominal unification. *Theoretical Computer Science* 323(1–3):473–497, 2004.